

3. Code fragments with comments

Ada (body)

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 6) - [internal]

```
Power_Of_Two

function Power_Of_Two (Exponent : Block_Size) return Phys_Addr is
begin
    return Phys_Addr (Shift_Left (1, Integer (Exponent)));
end Power_Of_Two;

function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean is
    Mask : constant Phys_Addr := Power_Of_Two (Size_Exp) - 1;
begin
    -- Udowodnienie, że (Addr mod 2^N) == 0 jest równoważne (Addr and (2^N - 1)) == 0
    return (Addr and Mask) = 0;
end Is_Aligned;

// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 11) - [internal]

Is_Aligned

```
function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean is
    Mask : constant Phys_Addr := Power_Of_Two (Size_Exp) - 1;
begin
    -- Udowodnienie, że (Addr mod 2^N) == 0 jest równoważne (Addr and (2^N - 1)) == 0
    return (Addr and Mask) = 0;
end Is_Aligned;

function Get_Buddy_Address (
    Addr      : Phys_Addr;
    Size_Exp  : Block_Size
) return Phys_Addr is
    -- Kluczowy bit maski dla alokatora Buddy
    // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 18) - [internal]

Get_Buddy_Address

```
function Get_Buddy_Address (
    Addr      : Phys_Addr;
    Size_Exp  : Block_Size
) return Phys_Addr is
    -- Kluczowy bit maski dla alokatora Buddy
    Bit_Mask : constant Phys_Addr := Power_Of_Two (Size_Exp);
begin
    -- XOR z maską odwraca dokładnie ten jeden bit, który definiuje pozycję buddy.
    -- SPARK udowodni, że ponieważ Addr jest wyrównane (Pre condition z .ads),
    -- to wynik również będzie wyrównany, a XOR nie spowoduje przepełnienia.
    return Addr xor Bit_Mask;
end Get_Buddy_Address;
// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 31) - [internal]

Get_Left_Child_Address

```
function Get_Left_Child_Address (
    Parent_Addr : Phys_Addr;
    Parent_Exp  : Block_Size
) return Phys_Addr is
begin
    -- Lewe dziecko zawsze zaczyna się tam, gdzie rodzic
    return Parent_Addr;
end Get_Left_Child_Address;

function Get_Right_Child_Address (
    Parent_Addr : Phys_Addr;
    Parent_Exp  : Block_Size
) return Phys_Addr is
    // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 40) - [internal]

Get_Right_Child_Address

```
function Get_Right_Child_Address (
  Parent_Addr : Phys_Addr;
  Parent_Exp  : Block_Size
) return Phys_Addr is
  -- Prawe dziecko to rodzic + połowa rozmiaru rodzica
  Half_Size : constant Phys_Addr := Power_Of_Two (Parent_Exp - 1);
begin
  return Parent_Addr + Half_Size;
end Get_Right_Child_Address;

function Safe_Calculate_DMA_End (
  Base_Addr : Phys_Addr;
  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 50) - [internal]

Safe_Calculate_DMA_End

```
function Safe_Calculate_DMA_End (
  Base_Addr : Phys_Addr;
  Buffer_Len : Phys_Addr
) return Phys_Addr is
begin
  -- Ponieważ Pre condition w .ads gwarantuje, że Base + Len <= Last,
  -- kompilator SPARK wie, że to dodawanie nigdy nie "owinie" (wrap-around) pamięci.
  return Base_Addr + Buffer_Len;
end Safe_Calculate_DMA_End;

end DS_Buddy_Math;
```

kernel_Workspace/kernel/src/policy/ada/policy.adb (line 5) - [internal] Evaluate

```
function Evaluate (Ring : Ring_Id;
  Class : Call_Class;
  Op    : Unsigned_8;
  Arg   : U64)
  return Decision
is
  pragma Unreferenced (Arg);
begin
  if Ring = Ring_Kernel then
    return Allow;
  end if;

  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy_c.adb (line 5) - [internal] To_Ring

```
function To_Ring (V : Unsigned_8) return Ring_Id is
  (case V is
    when 0 => Ring_Kernel,
    when 1 => Ring_Driver,
    when others => Ring_User);

function To_Class (V : Unsigned_8) return Call_Class is
  (case V is
    when 0 => Cls_Sys,
    when 1 => Cls_Video,
    when 2 => Cls_Audio,
    when 3 => Cls_Input,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/policy/ada/policy_c.adb (line 11) - [internal] To_Class

```
function To_Class (V : Unsigned_8) return Call_Class is
  (case V is
    when 0 => Cls_Sys,
    when 1 => Cls_Video,
    when 2 => Cls_Audio,
    when 3 => Cls_Input,
    when 4 => Cls_Block,
    when others => Cls_Net);

function policy_evaluate (ring : Unsigned_8;
                          cls : Unsigned_8;
                          op : Unsigned_8;
                          // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/policy/ada/policy_c.adb (line 20) - [internal] policy_evaluate

```
function policy_evaluate (ring : Unsigned_8;
                          cls : Unsigned_8;
                          op : Unsigned_8;
                          arg : Unsigned_64)
  return Unsigned_8
is
  D : constant Decision :=
    Evaluate (To_Ring (ring), To_Class (cls), op, U64 (arg));
begin
  return (case D is
    when Allow => 0,
    when Deny => 1);
  // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.adb (line 16) - [internal] Valid

```
function Valid (A : Buddy_Allocator) return Boolean is
begin
  return
    (for all I in Page_Index =>
      (if A.State (I) = Used or else A.State (I) = Free then
        Aligned (I, A.Order (I)) and then In_Bounds (I, A.Order (I)))
      and then Used_Disjoint (A)
      and then Pages_Conserved (A));
end Valid;

function Used_Pages (A : Buddy_Allocator) return Natural is
  Total : Natural := 0;
  // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.adb (line 26) - [internal] Used_Pages

```
function Used_Pages (A : Buddy_Allocator) return Natural is
  Total : Natural := 0;
begin
  for I in Page_Index loop
    if A.State (I) = Used then
      Total := Total + Order_Size (A.Order (I));
    end if;
  end loop;
  return Total;
end Used_Pages;

function Free_Pages (A : Buddy_Allocator) return Natural is
  // ... (truncated, see source file for more)
```

```

lib-ada/tg_alloc_buddy.adb (line 37) - [internal] Free_Pages
function Free_Pages (A : Buddy_Allocator) return Natural is
  Total : Natural := 0;
begin
  for I in Page_Index loop
    if A.State (I) = Free then
      Total := Total + Order_Size (A.Order (I));
    end if;
  end loop;
  return Total;
end Free_Pages;

procedure Allocate
// ... (truncated, see source file for more)

lib-ada/tg_alloc_buddy.adb (line 48) - [internal] Allocate
procedure Allocate
(A : in out Buddy_Allocator;
Size : Natural;
I : out Page_Index;
O : out Order_Type;
Ok : out Boolean)
is
  Need : Order_Type := 0;
  Cur : Order_Type;
  Idx : Page_Index := 0;
begin
  I := 0;
  // ... (truncated, see source file for more)

lib-ada/tg_alloc_buddy.adb (line 107) - [internal] Free
procedure Free (A : in out Buddy_Allocator; I : Page_Index) is
  Cur : Order_Type := A.Order (I);
  Idx : Page_Index := I;
begin
  A.State (Idx) := Free;

  -- Merge with the buddy while both halves are free at the same order.
  while Cur < Max_Order loop
    declare
      Buddy : constant Page_Index :=
        Page_Index (Page_Word (Idx) xor Page_Word (Order_Size (Cur)));
    begin
      // ... (truncated, see source file for more)

lib-ada/tg_alloc_buddy.adb (line 140) - [internal] Lemma_Alloc_Disjoint
procedure Lemma_Alloc_Disjoint
(A : Buddy_Allocator; I : Page_Index; O : Order_Type) is
begin
  -- Valid(A) implies Used_Disjoint(A); since State(I) = Used, block
  -- (I, O) is disjoint from every other used block. GNATprove discharges
  -- this by unfolding `Valid` and `Used_Disjoint`.
  null;
end Lemma_Alloc_Disjoint;

procedure Lemma_No_Double_Free
(A : Buddy_Allocator; I : Page_Index) is
begin
  // ... (truncated, see source file for more)

lib-ada/tg_alloc_buddy.adb (line 149) - [internal] Lemma_No_Double_Free
procedure Lemma_No_Double_Free
(A : Buddy_Allocator; I : Page_Index) is
begin
  -- The precondition State(I) = Used guarantees that a second Free on the
  -- same block is rejected (the contract of Free requires Used; after one
  -- Free the state is Free, so the precondition no longer holds).
  null;
end Lemma_No_Double_Free;

end Tg_Alloc_Buddy;

```

```

lib-ada/tg_comm_security.adb (line 12) - [internal] Route_Allowed
function Route_Allowed (From, To : Layer) return Boolean is
begin
  case From is
    when Kernel => return To /= Kernel;           -- kernel -> any
    when Driverspace => return To = Kernel or To = Manager;
    when Userspace => return To = Kernel or To = Manager;
    when Manager => return To /= Manager;         -- manager -> any
    when User_Driver_Space => return To = Kernel or To = Manager;
  end case;
end Route_Allowed;

function Required_Right (O : Opcode) return Rights_Mask is
// ... (truncated, see source file for more)

lib-ada/tg_comm_security.adb (line 23) - [internal] Required_Right
function Required_Right (O : Opcode) return Rights_Mask is
begin
  case O.Class is
    when Sys => return Right_Call;
    when Mem =>
      if O.Op = 1 then return Right_Manage;
      elsif O.Op = 3 then return Right_Map or Right_Write;
      elsif O.Op = 5 then return Right_Grant or Right_Map;
      else return Right_Call;
      end if;
    when Cap => return Right_Manage;
    when Ipc =>
// ... (truncated, see source file for more)

lib-ada/tg_comm_security.adb (line 48) - [internal] Required_Type
function Required_Type (O : Opcode) return Type_Req is
begin
  case O.Class is
    when Sys | Cap => return Any_Type;
    when Mem => return Memory;
    when Ipc => return Endpoint;
    when Shmem => return Shmem_Region;
    when Video | Audio | Input | Block | Net | Pci | Vgpu | Fs =>
      return Device;
  end case;
end Required_Type;

// ... (truncated, see source file for more)

lib-ada/tg_comm_security.adb (line 60) - [internal] Rights_Of
function Rights_Of (T : Cap_Table; C : Natural) return Rights_Mask is
begin
  return T (Cap_Id (C)).Rights;
end Rights_Of;

function Type_Of (T : Cap_Table; C : Natural) return Object_Type is
begin
  return T (Cap_Id (C)).Obj_Type;
end Type_Of;

function Authorize (T : Cap_Table; M : Msg) return Verdict is
begin
// ... (truncated, see source file for more)

```

```

lib-ada/tg_comm_security.adb (line 65) - [internal] Type_Of
function Type_Of (T : Cap_Table; C : Natural) return Object_Type is
begin
  return T (Cap_Id (C)).Obj_Type;
end Type_Of;

function Authorize (T : Cap_Table; M : Msg) return Verdict is
begin
  -- Mirrors the Rust `authorize` in ../lib/src/filter.rs step by step.
  if not M.Well_Formed then
    return Denied;
  elsif not Route_Allowed (M.Src_Layer, M.Dst_Layer) then
    return Denied;
  // ... (truncated, see source file for more)

lib-ada/tg_comm_security.adb (line 70) - [internal] Authorize
function Authorize (T : Cap_Table; M : Msg) return Verdict is
begin
  -- Mirrors the Rust `authorize` in ../lib/src/filter.rs step by step.
  if not M.Well_Formed then
    return Denied;
  elsif not Route_Allowed (M.Src_Layer, M.Dst_Layer) then
    return Denied;
  elsif not Has_Cap (T, M.Cap) then
    return Denied;
  elsif not Contains (Rights_Of (T, M.Cap), Required_Right (M.Opcode_Value)) then
    return Denied;
  elsif not Matches (Required_Type (M.Opcode_Value), Type_Of (T, M.Cap)) then
    return Denied;
  // ... (truncated, see source file for more)

lib-ada/tg_comm_security.adb (line 88) - [internal] Lemma_Empty_Table_Denies_All
procedure Lemma_Empty_Table_Denies_All (M : Msg) is
begin
  -- Unfold Authorized(Empty_Table, M): it requires Has_Cap(Empty_Table,
  -- M.Cap). But Empty_Table(I).Valid = False for every I, so Has_Cap is
  -- False, so Authorized is False. GNATprove discharges this directly.
  null;
end Lemma_Empty_Table_Denies_All;

procedure Lemma_Revocation_Monotonic
(T, U : Cap_Table; C : Cap_Id; M : Msg) is
begin
  -- Revokes(T,U,C) means U is T with capability C invalidated and all
  // ... (truncated, see source file for more)

lib-ada/tg_comm_security.adb (line 96) - [internal] Lemma_Revocation_Monotonic
procedure Lemma_Revocation_Monotonic
(T, U : Cap_Table; C : Cap_Id; M : Msg) is
begin
  -- Revokes(T,U,C) means U is T with capability C invalidated and all
  -- other entries unchanged. Hence, for every handle X:
  --   Has_Cap(U,X) => Has_Cap(T,X)
  -- (removing a capability never creates one). Therefore
  --   Authorized(U,M) => Authorized(T,M)
  -- because Authorized is monotone in the table. The contrapositive is
  -- exactly the post-condition:
  --   not Authorized(T,M) => not Authorized(U,M).
  -- GNATprove discharges this by unfolding Has_Cap and Authorized and
  // ... (truncated, see source file for more)

```

Ada (spec)

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 4) - [public]
DS_Buddy_Math

```
package DS_Buddy_Math with
  SPARK_Mode,
  Pure
is
  -- Typy bazowe dla fizycznej pamieci (bare-metal)
  type Phys_Addr is new Unsigned_64;
  type Block_Size is new Unsigned_64;

  -- Minimalny i maksymalny rozmiar bloku (np. 4KB do 1GB)
  MIN_BLOCK_SIZE : constant Block_Size := 12; -- 2^12 = 4096 bytes (4KB)
  MAX_BLOCK_SIZE : constant Block_Size := 30; -- 2^30 = 1GB

  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 17) - [public]
Power_Of_Two

-- Funkcja pomocnicza: Obliczanie 2^N (Shift left)

```
function Power_Of_Two (Exponent : Block_Size) return Phys_Addr with
  Pre => Exponent <= 63,
  -- Use a bit-shift to produce a value of type Unsigned_64 / Phys_Addr
  Post => Power_Of_Two'Result = Shift_Left (Phys_Addr (1), Integer (Exponent));

-- Sprawdzenie, czy adres jest poprawnie wyrównany do rozmiaru bloku
function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean with
  Post => Is_Aligned'Result = ((Addr mod Power_Of_Two (Size_Exp)) = 0);

-- =====
-- ALGORYTM 1: OBLICZANIE ADRESU BUDDY (PAROWANIE)
-- =====
// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 23) - [public]
Is_Aligned

-- Sprawdzenie, czy adres jest poprawnie wyrównany do rozmiaru bloku

```
function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean with
  Post => Is_Aligned'Result = ((Addr mod Power_Of_Two (Size_Exp)) = 0);

-- =====
-- ALGORYTM 1: OBLICZANIE ADRESU BUDDY (PAROWANIE)
-- =====
-- W alokatorze Buddy, "buddy" bloku to blok o tym samym rozmiarze,
-- który różni się tylko jednym bitem (na pozycji odpowiadającej rozmiarowi).

function Get_Buddy_Address (
  Addr      : Phys_Addr;
  Size_Exp  : Block_Size
  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 32) - [public]

Get_Buddy_Address

```
-- =====  
-- ALGORYTM 1: OBLICZANIE ADRESU BUDDY (PAROWANIE)  
-- =====
```

```
-- W alokatorze Buddy, "buddy" bloku to blok o tym samym rozmiarze,  
-- który różni się tylko jednym bitem (na pozycji odpowiadającej rozmiarowi).
```

```
function Get_Buddy_Address (  
  Addr      : Phys_Addr;  
  Size_Exp  : Block_Size  
) return Phys_Addr with  
  Pre => Size_Exp >= MIN_BLOCK_SIZE and then Size_Exp <= MAX_BLOCK_SIZE,  
  Post => Is_Aligned (Get_Buddy_Address'Result, Size_Exp),  
  Post => Get_Buddy_Address'Result /= Addr, -- Buddy NIGDY nie jest tym samym adresem  
         Post => Shift_Right (Get_Buddy_Address'Result, Integer (Size_Exp)) =  
         (Shift_Right (Addr, Integer (Size_Exp)) xor Phys_Addr (1)); -- Udowodniona różnica dok  
  
-- =====  
-- ALGORYTM 2: DZIELENIE BLOKU (SPLITTING)  
// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 47) - [public]

Get_Left_Child_Address

```
-- =====  
-- ALGORYTM 2: DZIELENIE BLOKU (SPLITTING)  
-- =====
```

```
-- Dzielenie bloku o rozmiarze  $2^N$  na dwa bloki o rozmiarze  $2^{(N-1)}$ .
```

```
function Get_Left_Child_Address (  
  Parent_Addr : Phys_Addr;  
  Parent_Exp  : Block_Size  
) return Phys_Addr with  
  Pre => Parent_Exp > MIN_BLOCK_SIZE and then Is_Aligned (Parent_Addr, Parent_Exp),  
  Post => Get_Left_Child_Address'Result = Parent_Addr,  
  Post => Is_Aligned (Get_Left_Child_Address'Result, Parent_Exp - 1);  
  
function Get_Right_Child_Address (  
  Parent_Addr : Phys_Addr;  
  Parent_Exp  : Block_Size  
) return Phys_Addr with  
  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 55) - [public]

Get_Right_Child_Address

```
function Get_Right_Child_Address (  
  Parent_Addr : Phys_Addr;  
  Parent_Exp  : Block_Size  
) return Phys_Addr with  
  Pre => Parent_Exp > MIN_BLOCK_SIZE and then Is_Aligned (Parent_Addr, Parent_Exp),  
  Post => Get_Right_Child_Address'Result = Parent_Addr + Power_Of_Two (Parent_Exp - 1),  
  Post => Is_Aligned (Get_Right_Child_Address'Result, Parent_Exp - 1),  
  Post => Get_Right_Child_Address'Result > Parent_Addr; -- Prawe dziecko jest zawsze wyżej w pamięci  
  
-- =====  
-- ALGORYTM 3: BEZPIECZNE OBLICZANIE OFFSETU DMA  
-- =====  
// ... (truncated, see source file for more)
```


driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 69) - [public]
Safe_Calculate_DMA_End

```
-- =====  
-- ALGORYTM 3: BEZPIECZNE OBLICZANIE OFFSETU DMA  
-- =====  
-- Krytyczne dla sterowników: obliczanie adresu końcowego bufora bez przepełnienia (Overflow).  
  
function Safe_Calculate_DMA_End (  
  Base_Addr : Phys_Addr;  
  Buffer_Len : Phys_Addr  
) return Phys_Addr with  
  Pre => Base_Addr <= Phys_Addr'Last - Buffer_Len, -- Gwarancja braku przepełnienia UInt64  
  Post => Safe_Calculate_DMA_End'Result = Base_Addr + Buffer_Len;  
  
-- Eksport do C/Rust (FFI)  
pragma Export (C, Get_Buddy_Address, "ds_spark_buddy_get_buddy");  
pragma Export (C, Get_Left_Child_Address, "ds_spark_buddy_left_child");  
pragma Export (C, Get_Right_Child_Address, "ds_spark_buddy_right_child");  
pragma Export (C, Safe_Calculate_DMA_End, "ds_spark_safe_dma_end");  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy.ads (line 3) - [public] Policy

```
package Policy  
  with SPARK_Mode  
is  
  
  type Ring_Id is (Ring_Kernel, Ring_Driver, Ring_User)  
    with Size => 8;  
  
  type Call_Class is (Cls_Sys, Cls_Video, Cls_Audio,  
                     Cls_Input, Cls_Block, Cls_Net)  
    with Size => 8;  
  
  type Decision is (Allow, Deny)  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy.ads (line 21) - [public] Evaluate

```
function Evaluate (Ring : Ring_Id;  
                  Class : Call_Class;  
                  Op : Unsigned_8;  
                  Arg : U64)  
  return Decision  
  
with  
  Global => null,  
  Post =>  
    (if Ring = Ring_Kernel then Evaluate'Result = Allow)  
    and then  
    (if Ring = Ring_User  
     and then Class = Cls_Block  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy_c.ads (line 4) - [public] Policy_C

```
package Policy_C is  
  
  function policy_evaluate (ring : Unsigned_8;  
                           cls : Unsigned_8;  
                           op : Unsigned_8;  
                           arg : Unsigned_64)  
    return Unsigned_8  
  
  with Convention => C,  
  Export,  
  External_Name => "policy_evaluate";  
  
end Policy_C;
```

```

kernel_workspace/kernel/src/policy/ada/policy_c.ads (line 6) - [public] policy_evaluate
    function policy_evaluate (ring : Unsigned_8;
                              cls  : Unsigned_8;
                              op   : Unsigned_8;
                              arg  : Unsigned_64)
    return Unsigned_8

    with Convention => C,
         Export,
         External_Name => "policy_evaluate";

end Policy_C;

lib-ada/tg_alloc_buddy.ads (line 21) - [public] Tg_Alloc_Buddy
package Tg_Alloc_Buddy with SPARK_Mode => On is

    Max_Order : constant := 8;
    Max_Pages : constant := 2**Max_Order;    -- 256 pages

    subtype Order_Type is Natural range 0 .. Max_Order;
    subtype Page_Index is Natural range 0 .. Max_Pages - 1;

    type Block_State is (Free, Used, Tail);

    type State_Array is array (Page_Index) of Block_State;
    type Order_Array is array (Page_Index) of Order_Type;
    // ... (truncated, see source file for more)

lib-ada/tg_alloc_buddy.ads (line 48) - [public] Order_Size
-- Size in pages of a block of the given order -----
    function Order_Size (O : Order_Type) return Natural is (2**O);

    -- A block of order O starting at I is aligned on its own size -----
    function Aligned (I : Page_Index; O : Order_Type) return Boolean is
        (I mod Order_Size (O) = 0);

    -- A block of order O starting at I fits inside the arena -----
    function In_Bounds (I : Page_Index; O : Order_Type) return Boolean is
        (I + Order_Size (O) <= Max_Pages);

    -- Whether page P lies inside the block (I, O) -----
    function In_Block (I : Page_Index; O : Order_Type; P : Page_Index)
    // ... (truncated, see source file for more)

lib-ada/tg_alloc_buddy.ads (line 51) - [public] Aligned
-- A block of order O starting at I is aligned on its own size -----
    function Aligned (I : Page_Index; O : Order_Type) return Boolean is
        (I mod Order_Size (O) = 0);

    -- A block of order O starting at I fits inside the arena -----
    function In_Bounds (I : Page_Index; O : Order_Type) return Boolean is
        (I + Order_Size (O) <= Max_Pages);

    -- Whether page P lies inside the block (I, O) -----
    function In_Block (I : Page_Index; O : Order_Type; P : Page_Index)
        return Boolean is
        (P >= I and then P < I + Order_Size (O));

    // ... (truncated, see source file for more)

```

lib-ada/tg_alloc_buddy.ads (line 55) - [public] In_Bounds

-- A block of order O starting at I fits inside the arena -----

```
function In_Bounds (I : Page_Index; O : Order_Type) return Boolean is
  (I + Order_Size (O) <= Max_Pages);
```

-- Whether page P lies inside the block (I, O) -----

```
function In_Block (I : Page_Index; O : Order_Type; P : Page_Index)
  return Boolean is
  (P >= I and then P < I + Order_Size (O));
```

-- Whether two blocks overlap -----

```
function Overlap (I1 : Page_Index; O1 : Order_Type;
  I2 : Page_Index; O2 : Order_Type) return Boolean is
  ((I1 <= I2 and then I2 < I1 + Order_Size (O1))
  // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.ads (line 59) - [public] In_Block

-- Whether page P lies inside the block (I, O) -----

```
function In_Block (I : Page_Index; O : Order_Type; P : Page_Index)
  return Boolean is
  (P >= I and then P < I + Order_Size (O));
```

-- Whether two blocks overlap -----

```
function Overlap (I1 : Page_Index; O1 : Order_Type;
  I2 : Page_Index; O2 : Order_Type) return Boolean is
  ((I1 <= I2 and then I2 < I1 + Order_Size (O1))
  or else (I2 <= I1 and then I1 < I2 + Order_Size (O2)));
```

-- The allocator invariant -----

-- (a) every used/free head block is aligned and in bounds,
// ... (truncated, see source file for more)

lib-ada/tg_alloc_buddy.ads (line 64) - [public] Overlap

-- Whether two blocks overlap -----

```
function Overlap (I1 : Page_Index; O1 : Order_Type;
  I2 : Page_Index; O2 : Order_Type) return Boolean is
  ((I1 <= I2 and then I2 < I1 + Order_Size (O1))
  or else (I2 <= I1 and then I1 < I2 + Order_Size (O2)));
```

-- The allocator invariant -----

-- (a) every used/free head block is aligned and in bounds,
-- (b) used blocks are pairwise disjoint,
-- (c) pages are conserved.

```
function Valid (A : Buddy_Allocator) return Boolean;
```

-- All used blocks are pairwise disjoint (the write-correctness property) --

// ... (truncated, see source file for more)

lib-ada/tg_alloc_buddy.ads (line 73) - [public] Valid

-- The allocator invariant -----

-- (a) every used/free head block is aligned and in bounds,

-- (b) used blocks are pairwise disjoint,

-- (c) pages are conserved.

```
function Valid (A : Buddy_Allocator) return Boolean;
```

-- All used blocks are pairwise disjoint (the write-correctness property) --

```
function Used_Disjoint (A : Buddy_Allocator) return Boolean is
```

```
((for all I in Page_Index =>
  (for all J in Page_Index =>
    (if A.State (I) = Used and then A.State (J) = Used
      and then I /= J
      then not Overlap (I, A.Order (I), J, A.Order (J))))));
```

-- Accounting -----

```
function Used_Pages (A : Buddy_Allocator) return Natural;
```

// ... (truncated, see source file for more)

lib-ada/tg_alloc_buddy.ads (line 76) - [public] Used_Disjoint

-- All used blocks are pairwise disjoint (the write-correctness property) --

```
function Used_Disjoint (A : Buddy_Allocator) return Boolean is
  ((for all I in Page_Index =>
    (for all J in Page_Index =>
      (if A.State (I) = Used and then A.State (J) = Used
        and then I /= J
        then not Overlap (I, A.Order (I), J, A.Order (J))))));
```

```
-- Accounting -----
function Used_Pages (A : Buddy_Allocator) return Natural;
function Free_Pages (A : Buddy_Allocator) return Natural;

function Pages_Converted (A : Buddy_Allocator) return Boolean is
  // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.ads (line 84) - [public] Used_Pages

-- Accounting -----

```
function Used_Pages (A : Buddy_Allocator) return Natural;
function Free_Pages (A : Buddy_Allocator) return Natural;

function Pages_Converted (A : Buddy_Allocator) return Boolean is
  (Used_Pages (A) + Free_Pages (A) = Max_Pages);
```

```
-- Allocate -----
procedure Allocate
  (A : in out Buddy_Allocator;
   Size : Natural;
   I : out Page_Index;
   O : out Order_Type;
   // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.ads (line 85) - [public] Free_Pages

```
function Free_Pages (A : Buddy_Allocator) return Natural;

function Pages_Converted (A : Buddy_Allocator) return Boolean is
  (Used_Pages (A) + Free_Pages (A) = Max_Pages);
```

```
-- Allocate -----
procedure Allocate
  (A : in out Buddy_Allocator;
   Size : Natural;
   I : out Page_Index;
   O : out Order_Type;
   Ok : out Boolean) with
  // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.ads (line 87) - [public] Pages_Converted

```
function Pages_Converted (A : Buddy_Allocator) return Boolean is
  (Used_Pages (A) + Free_Pages (A) = Max_Pages);
```

```
-- Allocate -----
procedure Allocate
  (A : in out Buddy_Allocator;
   Size : Natural;
   I : out Page_Index;
   O : out Order_Type;
   Ok : out Boolean) with
  Pre => Valid (A),
  Post => Valid (A)
  // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.ads (line 91) - [public] Allocate

```
-- Allocate -----  
procedure Allocate  
  (A : in out Buddy_Allocator;  
   Size : Natural;  
   I : out Page_Index;  
   O : out Order_Type;  
   Ok : out Boolean) with  
  Pre => Valid (A),  
  Post => Valid (A)  
    and then Pages_Conserved (A)  
    and then (if Ok then  
      A.State (I) = Used  
      and then Order_Size (O) >= Size  
    // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.ads (line 110) - [public] Free

```
-- Free -----  
procedure Free (A : in out Buddy_Allocator; I : Page_Index) with  
  Pre => Valid (A) and then A.State (I) = Used,  
  Post => Valid (A)  
    and then A.State (I) = Free  
    and then Pages_Conserved (A);  
  
-- Lemma 1: two freshly-allocated blocks never overlap -----  
procedure Lemma_Alloc_Disjoint  
  (A : Buddy_Allocator; I : Page_Index; O : Order_Type) with  
  Ghost,  
  Pre => Valid (A) and then A.State (I) = Used,  
  Post => (for all J in Page_Index =>  
    // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.ads (line 117) - [public] Lemma_Alloc_Disjoint

```
-- Lemma 1: two freshly-allocated blocks never overlap -----  
procedure Lemma_Alloc_Disjoint  
  (A : Buddy_Allocator; I : Page_Index; O : Order_Type) with  
  Ghost,  
  Pre => Valid (A) and then A.State (I) = Used,  
  Post => (for all J in Page_Index =>  
    (if A.State (J) = Used and then J /= I then  
      not Overlap (I, O, J, A.Order (J))));  
  
-- Lemma 2: a freed block can be reused without overlap -----  
procedure Lemma_No_Double_Free  
  (A : Buddy_Allocator; I : Page_Index) with  
  Ghost,  
  // ... (truncated, see source file for more)
```

lib-ada/tg_alloc_buddy.ads (line 126) - [public] Lemma_No_Double_Free

```
-- Lemma 2: a freed block can be reused without overlap -----  
procedure Lemma_No_Double_Free  
  (A : Buddy_Allocator; I : Page_Index) with  
  Ghost,  
  Pre => Valid (A) and then A.State (I) = Used,  
  Post => True;  
  
end Tg_Alloc_Buddy;
```

```

lib-ada/tg_comm.ads (line 12) - [public] Tg_Comm
    package Tg_Comm is
        pragma Preelaborate;

        use type Interfaces.Unsigned_32;
        use type Interfaces.Unsigned_16;

        -- Protocol constants (mirror lib/src/consts.rs) -----
        Magic    : constant Interfaces.Unsigned_32 := 16#5447_434D#; -- "TGCM"
        Version   : constant Interfaces.Unsigned_8  := 1;
        Msg_Size   : constant := 64;
        Ring_Ctrl  : constant := 16;
        Cap_Table_Size_Entries : constant := 512;
        // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 57) - [public] Contains
    function Contains (Set, Subset : Rights) return Boolean is
        ((Set and Subset) = Subset);

        -- Opcode classes -----
        type Op_Class is (Sys, Mem, Cap, Ipc, Shmem, Video, Audio, Input,
            Block, Net, Pci, Vgpu, Fs);
        for Op_Class use (Sys => 0, Mem => 1, Cap => 2, Ipc => 3, Shmem => 4,
            Video => 5, Audio => 6, Input => 7, Block => 8,
            Net => 9, Pci => 10, Vgpu => 11, Fs => 12);
        for Op_Class'Size use 8;

        function Opcode (Cls : Op_Class; Op : Interfaces.Unsigned_16)
            // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 68) - [public] Opcode
    function Opcode (Cls : Op_Class; Op : Interfaces.Unsigned_16)
        return Interfaces.Unsigned_32 is
            (Interfaces.Unsigned_32 (Op_Class'Pos (Cls)) * 2**8
            or Interfaces.Unsigned_32 (Op and 16#FF#));

        -- Authorize result -----
        type Result is (Forwarded, Malformed, Unknown_Op, Route_Denied,
            No_Capability, Rights_Insufficient, Type_Mismatch);
        for Result use (Forwarded => 0, Malformed => 1, Unknown_Op => 2,
            Route_Denied => 3, No_Capability => 4,
            Rights_Insufficient => 5, Type_Mismatch => 6);
        for Result'Size use 32;
        // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 119) - [public] Tgcomm_Version
    -- Stable C ABI (provided by libtg_comm.a) -----
    function Tgcomm_Version return Interfaces.Unsigned_32;
    pragma Import (C, Tgcomm_Version, "tgcomm_version");

    function Tgcomm_Msg_Size return Interfaces.Unsigned_32;
    pragma Import (C, Tgcomm_Msg_Size, "tgcomm_msg_size");

    function Tgcomm_Ring_Size (Slots : Interfaces.Unsigned_32)
        return Interfaces.Unsigned_64;
    pragma Import (C, Tgcomm_Ring_Size, "tgcomm_ring_size");

    procedure Tgcomm_Msg_Init (Msg : in out Comm_Msg);
    pragma Import (C, Tgcomm_Msg_Init, "tgcomm_msg_init");
    // ... (truncated, see source file for more)

```

```

lib-ada/tg_comm.ads (line 122) - [public] Tgcomm_Msg_Size
    function Tgcomm_Msg_Size return Interfaces.Unsigned_32;
    pragma Import (C, Tgcomm_Msg_Size, "tgcomm_msg_size");

    function Tgcomm_Ring_Size (Slots : Interfaces.Unsigned_32)
        return Interfaces.Unsigned_64;
    pragma Import (C, Tgcomm_Ring_Size, "tgcomm_ring_size");

    procedure Tgcomm_Msg_Init (Msg : in out Comm_Msg);
    pragma Import (C, Tgcomm_Msg_Init, "tgcomm_msg_init");

    function Tgcomm_Authorize (Table : System.Address; Msg : System.Address)
        return Interfaces.Integer_32;
    // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 125) - [public] Tgcomm_Ring_Size
    function Tgcomm_Ring_Size (Slots : Interfaces.Unsigned_32)
        return Interfaces.Unsigned_64;
    pragma Import (C, Tgcomm_Ring_Size, "tgcomm_ring_size");

    procedure Tgcomm_Msg_Init (Msg : in out Comm_Msg);
    pragma Import (C, Tgcomm_Msg_Init, "tgcomm_msg_init");

    function Tgcomm_Authorize (Table : System.Address; Msg : System.Address)
        return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Authorize, "tgcomm_authorize");

    procedure Tgcomm_Ring_Init (Ring : System.Address;
        // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 129) - [public] Tgcomm_Msg_Init
    procedure Tgcomm_Msg_Init (Msg : in out Comm_Msg);
    pragma Import (C, Tgcomm_Msg_Init, "tgcomm_msg_init");

    function Tgcomm_Authorize (Table : System.Address; Msg : System.Address)
        return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Authorize, "tgcomm_authorize");

    procedure Tgcomm_Ring_Init (Ring : System.Address;
        Slots : Interfaces.Unsigned_32);
    pragma Import (C, Tgcomm_Ring_Init, "tgcomm_ring_init");

    function Tgcomm_Ring_Push (Ring : System.Address; Msg : System.Address)
        // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 132) - [public] Tgcomm_Authorize
    function Tgcomm_Authorize (Table : System.Address; Msg : System.Address)
        return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Authorize, "tgcomm_authorize");

    procedure Tgcomm_Ring_Init (Ring : System.Address;
        Slots : Interfaces.Unsigned_32);
    pragma Import (C, Tgcomm_Ring_Init, "tgcomm_ring_init");

    function Tgcomm_Ring_Push (Ring : System.Address; Msg : System.Address)
        return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Ring_Push, "tgcomm_ring_push");

    // ... (truncated, see source file for more)

```

```

lib-ada/tg_comm.ads (line 136) - [public] Tgcomm_Ring_Init
    procedure Tgcomm_Ring_Init (Ring : System.Address;
                                Slots : Interfaces.Unsigned_32);
    pragma Import (C, Tgcomm_Ring_Init, "tgcomm_ring_init");

    function Tgcomm_Ring_Push (Ring : System.Address; Msg : System.Address)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Ring_Push, "tgcomm_ring_push");

    function Tgcomm_Ring_Pop (Ring : System.Address; Out_Msg : System.Address)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Ring_Pop, "tgcomm_ring_pop");

    // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 140) - [public] Tgcomm_Ring_Push
    function Tgcomm_Ring_Push (Ring : System.Address; Msg : System.Address)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Ring_Push, "tgcomm_ring_push");

    function Tgcomm_Ring_Pop (Ring : System.Address; Out_Msg : System.Address)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Ring_Pop, "tgcomm_ring_pop");

    function Tgcomm_Ring_Available (Ring : System.Address)
                                    return Interfaces.Unsigned_32;
    pragma Import (C, Tgcomm_Ring_Available, "tgcomm_ring_available");

    // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 144) - [public] Tgcomm_Ring_Pop
    function Tgcomm_Ring_Pop (Ring : System.Address; Out_Msg : System.Address)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Ring_Pop, "tgcomm_ring_pop");

    function Tgcomm_Ring_Available (Ring : System.Address)
                                    return Interfaces.Unsigned_32;
    pragma Import (C, Tgcomm_Ring_Available, "tgcomm_ring_available");

    procedure Tgcomm_Cap_Table_Init (Table : System.Address);
    pragma Import (C, Tgcomm_Cap_Table_Init, "tgcomm_cap_table_init");

    function Tgcomm_Cap_Insert (Table : System.Address;
                                // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 148) - [public] Tgcomm_Ring_Available
    function Tgcomm_Ring_Available (Ring : System.Address)
                                    return Interfaces.Unsigned_32;
    pragma Import (C, Tgcomm_Ring_Available, "tgcomm_ring_available");

    procedure Tgcomm_Cap_Table_Init (Table : System.Address);
    pragma Import (C, Tgcomm_Cap_Table_Init, "tgcomm_cap_table_init");

    function Tgcomm_Cap_Insert (Table : System.Address;
                                Cap : Interfaces.Unsigned_32;
                                Obj_Id : Interfaces.Unsigned_64;
                                Obj_Type : Interfaces.Unsigned_8;
                                Rights_Mask : Interfaces.Unsigned_32)
                                // ... (truncated, see source file for more)

```



```

lib-ada/tg_comm.ads (line 152) - [public] Tgcomm_Cap_Table_Init
    procedure Tgcomm_Cap_Table_Init (Table : System.Address);
    pragma Import (C, Tgcomm_Cap_Table_Init, "tgcomm_cap_table_init");

    function Tgcomm_Cap_Insert (Table : System.Address;
                                Cap : Interfaces.Unsigned_32;
                                Obj_Id : Interfaces.Unsigned_64;
                                Obj_Type : Interfaces.Unsigned_8;
                                Rights_Mask : Interfaces.Unsigned_32)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Cap_Insert, "tgcomm_cap_insert");

    function Tgcomm_Cap_Check (Table : System.Address;
                                // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 155) - [public] Tgcomm_Cap_Insert
    function Tgcomm_Cap_Insert (Table : System.Address;
                                Cap : Interfaces.Unsigned_32;
                                Obj_Id : Interfaces.Unsigned_64;
                                Obj_Type : Interfaces.Unsigned_8;
                                Rights_Mask : Interfaces.Unsigned_32)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Cap_Insert, "tgcomm_cap_insert");

    function Tgcomm_Cap_Check (Table : System.Address;
                                Cap : Interfaces.Unsigned_32;
                                Required_Rights : Interfaces.Unsigned_32)
                                return Interfaces.Integer_32;
    // ... (truncated, see source file for more)

lib-ada/tg_comm.ads (line 163) - [public] Tgcomm_Cap_Check
    function Tgcomm_Cap_Check (Table : System.Address;
                                Cap : Interfaces.Unsigned_32;
                                Required_Rights : Interfaces.Unsigned_32)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Cap_Check, "tgcomm_cap_check");

    function Tgcomm_Cap_Remove (Table : System.Address;
                                Cap : Interfaces.Unsigned_32)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Cap_Remove, "tgcomm_cap_remove");

end Tg_Comm;

lib-ada/tg_comm.ads (line 169) - [public] Tgcomm_Cap_Remove
    function Tgcomm_Cap_Remove (Table : System.Address;
                                Cap : Interfaces.Unsigned_32)
                                return Interfaces.Integer_32;
    pragma Import (C, Tgcomm_Cap_Remove, "tgcomm_cap_remove");

end Tg_Comm;

lib-ada/tg_comm_security.ads (line 23) - [public] Tg_Comm_Security
package Tg_Comm_Security with SPARK_Mode => On is

    -- Rights -----
    type Rights_Mask is mod 2**32;
    Right_Read      : constant Rights_Mask := 2**0;
    Right_Write     : constant Rights_Mask := 2**1;
    Right_Exec      : constant Rights_Mask := 2**2;
    Right_Map       : constant Rights_Mask := 2**3;
    Right_Grant     : constant Rights_Mask := 2**4;
    Right_Transfer  : constant Rights_Mask := 2**5;
    Right_Send      : constant Rights_Mask := 2**6;
    Right_Recv      : constant Rights_Mask := 2**7;
    // ... (truncated, see source file for more)

```

```

lib-ada/tg_comm_security.ads (line 38) - [public] Contains
    function Contains (Set, Subset : Rights_Mask) return Boolean is
        ((Set and Subset) = Subset);

    -- Layers and routing -----
    type Layer is (Kernel, Driverspace, Userspace, Manager, User_Driver_Space);

    function Route_Allowed (From, To : Layer) return Boolean;

    -- Object types and the required-type relation -----
    type Object_Type is (Memory, Endpoint, Notification, Channel,
        Shmem_Region, Device, Irq);

    // ... (truncated, see source file for more)

lib-ada/tg_comm_security.ads (line 44) - [public] Route_Allowed
    function Route_Allowed (From, To : Layer) return Boolean;

    -- Object types and the required-type relation -----
    type Object_Type is (Memory, Endpoint, Notification, Channel,
        Shmem_Region, Device, Irq);

    type Type_Req is (Any_Type, Memory, Endpoint, Notification, Channel,
        Shmem_Region, Device, Irq);

    function Matches (Req : Type_Req; Actual : Object_Type) return Boolean is
        (case Req is
            when Any_Type      => True,
            // ... (truncated, see source file for more)

lib-ada/tg_comm_security.ads (line 53) - [public] Matches
    function Matches (Req : Type_Req; Actual : Object_Type) return Boolean is
        (case Req is
            when Any_Type      => True,
            when Memory        => Actual = Memory,
            when Endpoint      => Actual = Endpoint,
            when Notification  => Actual = Notification,
            when Channel       => Actual = Channel,
            when Shmem_Region  => Actual = Shmem_Region,
            when Device        => Actual = Device,
            when Irq           => Actual = Irq);

    -- Opcodes -----
    // ... (truncated, see source file for more)

lib-ada/tg_comm_security.ads (line 73) - [public] Required_Right
    function Required_Right (O : Opcode) return Rights_Mask;
    function Required_Type (O : Opcode) return Type_Req;

    -- Message (model of the 64-byte wire message) -----
    type Msg is record
        Src_Layer    : Layer;
        Dst_Layer    : Layer;
        Opcode_Value : Opcode;
        Cap          : Natural;
        Well_Formed  : Boolean;    -- magic + version + decodable layer bytes
    end record;

    // ... (truncated, see source file for more)

```

```

lib-ada/tg_comm_security.ads (line 74) - [public] Required_Type
    function Required_Type (O : Opcode) return Type_Req;

    -- Message (model of the 64-byte wire message) -----
    type Msg is record
        Src_Layer    : Layer;
        Dst_Layer    : Layer;
        Opcode_Value : Opcode;
        Cap          : Natural;
        Well_Formed  : Boolean;    -- magic + version + decodable layer bytes
    end record;

    -- Capability table (bounded, concrete) -----
    // ... (truncated, see source file for more)

lib-ada/tg_comm_security.ads (line 97) - [public] Empty_Table
    function Empty_Table return Cap_Table is
        ([others => (Valid => False, Obj_Type => Memory, Rights => 0)]);

    function Has_Cap (T : Cap_Table; C : Natural) return Boolean is
        (if C in Cap_Id then T (Cap_Id (C)).Valid else False);

    function Rights_Of (T : Cap_Table; C : Natural) return Rights_Mask with
        Pre => Has_Cap (T, C);

    function Type_Of (T : Cap_Table; C : Natural) return Object_Type with
        Pre => Has_Cap (T, C);

    // ... (truncated, see source file for more)

lib-ada/tg_comm_security.ads (line 100) - [public] Has_Cap
    function Has_Cap (T : Cap_Table; C : Natural) return Boolean is
        (if C in Cap_Id then T (Cap_Id (C)).Valid else False);

    function Rights_Of (T : Cap_Table; C : Natural) return Rights_Mask with
        Pre => Has_Cap (T, C);

    function Type_Of (T : Cap_Table; C : Natural) return Object_Type with
        Pre => Has_Cap (T, C);

    -- The security property (Ghost: exists only for proof) -----
    function Authorized (T : Cap_Table; M : Msg) return Boolean is
        (M.Well_Formed
        // ... (truncated, see source file for more)

lib-ada/tg_comm_security.ads (line 103) - [public] Rights_Of
    function Rights_Of (T : Cap_Table; C : Natural) return Rights_Mask with
        Pre => Has_Cap (T, C);

    function Type_Of (T : Cap_Table; C : Natural) return Object_Type with
        Pre => Has_Cap (T, C);

    -- The security property (Ghost: exists only for proof) -----
    function Authorized (T : Cap_Table; M : Msg) return Boolean is
        (M.Well_Formed
         and then Route_Allowed (M.Src_Layer, M.Dst_Layer)
         and then Has_Cap (T, M.Cap)
         and then Contains (Rights_Of (T, M.Cap), Required_Right (M.Opcode_Value))
        // ... (truncated, see source file for more)

```

```

lib-ada/tg_comm_security.ads (line 106) - [public] Type_Of
function Type_Of (T : Cap_Table; C : Natural) return Object_Type with
  Pre => Has_Cap (T, C);

-- The security property (Ghost: exists only for proof) -----
function Authorized (T : Cap_Table; M : Msg) return Boolean is
  (M.Well_Formed
   and then Route_Allowed (M.Src_Layer, M.Dst_Layer)
   and then Has_Cap (T, M.Cap)
   and then Contains (Rights_Of (T, M.Cap), Required_Right (M.Opcode_Value))
   and then Matches (Required_Type (M.Opcode_Value), Type_Of (T, M.Cap)))
  with Ghost;

  // ... (truncated, see source file for more)

lib-ada/tg_comm_security.ads (line 110) - [public] Authorized
-- The security property (Ghost: exists only for proof) -----
function Authorized (T : Cap_Table; M : Msg) return Boolean is
  (M.Well_Formed
   and then Route_Allowed (M.Src_Layer, M.Dst_Layer)
   and then Has_Cap (T, M.Cap)
   and then Contains (Rights_Of (T, M.Cap), Required_Right (M.Opcode_Value))
   and then Matches (Required_Type (M.Opcode_Value), Type_Of (T, M.Cap)))
  with Ghost;

-- The gate -----
type Verdict is (Forwarded, Denied);

function Authorize (T : Cap_Table; M : Msg) return Verdict with
  // ... (truncated, see source file for more)

lib-ada/tg_comm_security.ads (line 121) - [public] Authorize
function Authorize (T : Cap_Table; M : Msg) return Verdict with
  Post => (Authorize'Result = Forwarded) = Authorized (T, M);

-- Revocation relation: U is T with capability C removed -----
function Revokes (T, U : Cap_Table; C : Cap_Id) return Boolean is
  ((for all I in Cap_Id =>
    (if I = C then not U (I).Valid else U (I) = T (I))));

-- Lemma 1: the empty table denies every request -----
procedure Lemma_Empty_Table_Denies_All (M : Msg) with
  Ghost,
  Post => not Authorized (Empty_Table, M);
  // ... (truncated, see source file for more)

lib-ada/tg_comm_security.ads (line 125) - [public] Revokes
-- Revocation relation: U is T with capability C removed -----
function Revokes (T, U : Cap_Table; C : Cap_Id) return Boolean is
  ((for all I in Cap_Id =>
    (if I = C then not U (I).Valid else U (I) = T (I))));

-- Lemma 1: the empty table denies every request -----
procedure Lemma_Empty_Table_Denies_All (M : Msg) with
  Ghost,
  Post => not Authorized (Empty_Table, M);

-- Lemma 2: revocation never authorizes a previously-denied request -----
procedure Lemma_Revocation_Monotonic
  (T, U : Cap_Table; C : Cap_Id; M : Msg) with
  // ... (truncated, see source file for more)

```

```

lib-ada/tg_comm_security.ads (line 130) - [public] Lemma_Empty_Table_Denies_All
-- Lemma 1: the empty table denies every request -----
    procedure Lemma_Empty_Table_Denies_All (M : Msg) with
        Ghost,
        Post => not Authorized (Empty_Table, M);

    -- Lemma 2: revocation never authorizes a previously-denied request -----
    procedure Lemma_Revocation_Monotonic
        (T, U : Cap_Table; C : Cap_Id; M : Msg) with
        Ghost,
        Pre  => Revokes (T, U, C) and then not Authorized (T, M),
        Post => not Authorized (U, M);

end Tg_Comm_Security;

lib-ada/tg_comm_security.ads (line 135) - [public] Lemma_Revocation_Monotonic
-- Lemma 2: revocation never authorizes a previously-denied request -----
    procedure Lemma_Revocation_Monotonic
        (T, U : Cap_Table; C : Cap_Id; M : Msg) with
        Ghost,
        Pre  => Revokes (T, U, C) and then not Authorized (T, M),
        Post => not Authorized (U, M);

end Tg_Comm_Security;

```

Assembly

```

comgrub/src/boot.asm (line 10) - [internal] mboot_header_start
    mboot_header_start:
        dd MBOOT2_MAGIC
        dd MBOOT2_ARCH
        dd MBOOT2_LENGTH
        dd MBOOT2_CHECKSUM

        dw 0
        dw 0
        dd 8
    mboot_header_end:

    section .bss
        // ... (truncated, see source file for more)

comgrub/src/boot.asm (line 19) - [internal] mboot_header_end
    mboot_header_end:

    section .bss
        align 16
    stack_bottom:
        resb 65536
    stack_top:

    section .text
        bits 32

    global _start
        // ... (truncated, see source file for more)

```

comgrub/src/boot.asm (line 23) - [internal] stack_bottom

```
stack_bottom:
    resb 65536
stack_top:

section .text
bits 32

global _start
extern _rust_entry

_start:
    mov esp, stack_top
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 25) - [internal] stack_top

```
stack_top:

section .text
bits 32

global _start
extern _rust_entry

_start:
    mov esp, stack_top

    push ebx                ; Multiboot2: EBX = adres struktury informacyjnej
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 30) - [public] _start

```
global _start
extern _rust_entry

_start:
    mov esp, stack_top

    push ebx                ; Multiboot2: EBX = adres struktury informacyjnej
    push eax                ; Multiboot2: EAX = magic

    call _rust_entry

.hang:
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 33) - [internal] _start

```
_start:
    mov esp, stack_top

    push ebx                ; Multiboot2: EBX = adres struktury informacyjnej
    push eax                ; Multiboot2: EAX = magic

    call _rust_entry

.hang:
    cli
    hlt
    jmp .hang
```

dirtycode/mesa/src/freedreno/computerator/examples/branch.asm (line 10) - [internal] endif

```
endif:
(jp)(rpt5)nop
stib.b.untyped.1d.u32.1.imm r1.x, r0.x, 0
end
nop
```

dirtycode/mesa/src/freedreno/computerator/examples/early_preamble.asm (line 19) - [internal]
l_preamble_end

```
l_preamble_end:
(jp)nop

(rpt3)mov.u32u32 r1.x, (r)c0.x
(rpt5)nop
stib.b.untyped.1d.u32.4.imm r1.x, r0.x, 0
end
```

dirtycode/mesa/src/freedreno/computerator/examples/localmem.asm (line 14) - [internal] l_all

```
l_all:
(jp)ldlw.u32 r1.z, l[r1.x], 1
(ss)nop
stib.b.untyped.1d.u32.4.imm r1.z, r0.x, 0
end
```

dirtycode/mesa/src/freedreno/computerator/examples/ubo.asm (line 20) - [internal] main

```
main:

; load UBO 1 to r1
(jp)ldc.offset0.4.imm r1.x, 0, 1

(sy)(rpt3)add.u r1.x, (r)r1.x, (r)c0.x
(rpt2)nop

stib.b.untyped.1d.u32.4.imm r1.x, r0.x, 0
end
```

dirtycode/mesa/src/freedreno/tests/reference/qrisctest.asm (line 41) - [internal] CP_ME_INIT

```
CP_ME_INIT:
mov $02, 0x2
waitin
mov $01, $data

CP_MEM_WRITE:
mov $addr, 0xa0 << 24■; |NRT_ADDR
mov $02, 0x4
(xmov1)add $data, $02, $data
mov $addr, 0xa204 << 16■; |NRT_DATA
(rep)(xmov3)mov $data, $data
waitin
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisctest.asm (line 46) - [internal]
CP_MEM_WRITE

```
CP_MEM_WRITE:
mov $addr, 0xa0 << 24■; |NRT_ADDR
mov $02, 0x4
(xmov1)add $data, $02, $data
mov $addr, 0xa204 << 16■; |NRT_DATA
(rep)(xmov3)mov $data, $data
waitin
mov $01, $data

CP_SCRATCH_WRITE:
mov $02, 0xff
(rep)cwrite $data, [$02 + 0x1]!
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 55) - [internal]

CP_SCRATCH_WRITE

```
CP_SCRATCH_WRITE:
mov $02, 0xff
(rep)cwrite $data, [$02 + 0x1]!
waitin
mov $01, $data

CP_SET_DRAW_STATE:
(rep)(sds2)cwrite $data, [$00 + @DRAW_STATE_SET_HDR]
waitin
mov $01, $data

CP_SET_BIN_DATA5:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 61) - [internal]

CP_SET_DRAW_STATE

```
CP_SET_DRAW_STATE:
(rep)(sds2)cwrite $data, [$00 + @DRAW_STATE_SET_HDR]
waitin
mov $01, $data

CP_SET_BIN_DATA5:
sread $02, [$00 + %SP]
swrite $02, [$00 + %SP]
mov $02, 0x7
(rep)swrite $data, [$02 + 0x1]!
waitin
mov $01, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 66) - [internal]

CP_SET_BIN_DATA5

```
CP_SET_BIN_DATA5:
sread $02, [$00 + %SP]
swrite $02, [$00 + %SP]
mov $02, 0x7
(rep)swrite $data, [$02 + 0x1]!
waitin
mov $01, $data

CP_SET_SECURE_MODE:
mov $02, $data
setsecure $02, #l63

// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 74) - [internal]

CP_SET_SECURE_MODE

```
CP_SET_SECURE_MODE:
mov $02, $data
setsecure $02, #l63

fxn61:
l61:
jump #l61
nop
l63:
waitin
mov $01, $data

// ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 78) - [internal] fxn61

```
fxn61:
l61:
jump #l61
nop
l63:
waitin
mov $01, $data

fxn65:
l65:
cmp $04, $02, $03
breq $04, b0, #l72
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 79) - [internal] l61

```
l61:
jump #l61
nop
l63:
waitin
mov $01, $data

fxn65:
l65:
cmp $04, $02, $03
breq $04, b0, #l72
brne $04, b1, #l70
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 82) - [internal] l63

```
l63:
waitin
mov $01, $data

fxn65:
l65:
cmp $04, $02, $03
breq $04, b0, #l72
brne $04, b1, #l70
breq $04, b2, #l65
sub $03, $03, $02
l70:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 86) - [internal] fxn65

```
fxn65:
l65:
cmp $04, $02, $03
breq $04, b0, #l72
brne $04, b1, #l70
breq $04, b2, #l65
sub $03, $03, $02
l70:
jump #l65
sub $02, $02, $03
l72:
ret
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 87) - [internal] l65

```
l65:
cmp $04, $02, $03
breq $04, b0, #l72
brne $04, b1, #l70
breq $04, b2, #l65
sub $03, $03, $02
l70:
jump #l65
sub $02, $02, $03
l72:
ret
nop
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 93) - [internal] l70

```
l70:
jump #l65
sub $02, $02, $03
l72:
ret
nop

CP_REG_RMW:
cwrite $data, [$00 + @REG_READ_ADDR]
add $02, $regdata, 0x42
addhi $03, $00, $regdata
sub $02, $02, $regdata
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 96) - [internal] l72

```
l72:
ret
nop

CP_REG_RMW:
cwrite $data, [$00 + @REG_READ_ADDR]
add $02, $regdata, 0x42
addhi $03, $00, $regdata
sub $02, $02, $regdata
call #fxn65
subhi $03, $03, $regdata
and $02, $02, $regdata
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 100) - [internal] CP_REG_RMW

```
CP_REG_RMW:
cwrite $data, [$00 + @REG_READ_ADDR]
add $02, $regdata, 0x42
addhi $03, $00, $regdata
sub $02, $02, $regdata
call #fxn65
subhi $03, $03, $regdata
and $02, $02, $regdata
or $02, $02, 0x1
xor $02, $02, 0x1
not $02, $02
shl $02, $02, $regdata
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 124) - [internal] CP_MEMCPY

```
CP_MEMCPY:
mov $02, $data
mov $03, $data
mov $04, $data
mov $05, $data
mov $06, $data
l101:
breq $06, 0x0, #l107
cwrite $03, [$00 + @LOAD_STORE_HI]
load $07, [$02 + 0x4]!
cwrite $05, [$00 + @LOAD_STORE_HI]
jump #l101
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 130) - [internal] l101

```
l101:
breq $06, 0x0, #l107
cwrite $03, [$00 + @LOAD_STORE_HI]
load $07, [$02 + 0x4]!
cwrite $05, [$00 + @LOAD_STORE_HI]
jump #l101
store $07, [$04 + 0x4]!
l107:
waitin
mov $01, $data

CP_MEM_TO_MEM:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 137) - [internal] l107

```
l107:
waitin
mov $01, $data

CP_MEM_TO_MEM:
cwrite $data, [$00 + @MEM_READ_ADDR]
cwrite $data, [$00 + @MEM_READ_ADDR+0x1]
mov $02, $data
cwrite $data, [$00 + @LOAD_STORE_HI]
mov $rem, $data
cwrite $rem, [$00 + @MEM_READ_DWORDS]
(rep)store $memdata, [$02 + 0x4]!
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 141) - [internal]

CP_MEM_TO_MEM

```
CP_MEM_TO_MEM:
cwrite $data, [$00 + @MEM_READ_ADDR]
cwrite $data, [$00 + @MEM_READ_ADDR+0x1]
mov $02, $data
cwrite $data, [$00 + @LOAD_STORE_HI]
mov $rem, $data
cwrite $rem, [$00 + @MEM_READ_DWORDS]
(rep)store $memdata, [$02 + 0x4]!
waitin
mov $01, $data

IN_PREEMPT:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 152) - [internal] IN_PREEMPT

```
IN_PREEMPT:
preempt:
cread $02, [$00 + 0x101]
brne $02, 0x1, #l127
nop
bl #fxn61
nop
nop
nop
waitin
mov $01, $data
l127:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 153) - [internal] preempt

```
preempt:
cread $02, [$00 + 0x101]
brne $02, 0x1, #l127
nop
bl #fxn61
nop
nop
nop
waitin
mov $01, $data
l127:
iret
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 163) - [internal] l127

```
l127:
iret
nop

CP_BLIT:
CP_BOOTSTRAP_UCODE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH:
CP_CONTEXT_SWITCH_YIELD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 167) - [internal] CP_BLIT

```
CP_BLIT:
CP_BOOTSTRAP_UCODE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 168) - [internal]

CP_BOOTSTRAP_UCODE

```
CP_BOOTSTRAP_UCODE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 169) - [internal]

CP_COND_EXEC

```
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 170) - [internal]

CP_COND_REG_EXEC

```
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 171) - [internal]

CP_COND_WRITE5

```
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 172) - [internal]

CP_CONTEXT_REG_BUNCH

```
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 173) - [internal]

CP_CONTEXT_SWITCH

```
CP_CONTEXT_SWITCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 174) - [internal]

CP_CONTEXT_SWITCH_YIELD

```
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 175) - [internal]

CP_CONTEXT_UPDATE

```
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 176) - [internal]

CP_DRAW_AUTO

```
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 177) - [internal]

CP_DRAW_INDIRECT

```
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 178) - [internal]

CP_DRAW_INDIRECT_MULTI

```
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 179) - [internal]

CP_DRAW_INDX

```
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 180) - [internal]

CP_DRAW_INDX_INDIRECT

```
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 181) - [internal]

CP_DRAW_INDX_OFFSET

```
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 182) - [internal]

CP_DRAW_PRED_ENABLE_GLOBAL

```
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 183) - [internal]

CP_DRAW_PRED_ENABLE_LOCAL

```
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 184) - [internal]

CP_DRAW_PRED_SET

```
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 185) - [internal] CP_END_BIN

```
CP_END_BIN:
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 186) - [internal]

CP_EVENT_WRITE

```
CP_EVENT_WRITE:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 187) - [internal] CP_EXEC_CS

```
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 188) - [internal]

CP_EXEC_CS_INDIRECT

```
CP_EXEC_CS_INDIRECT:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 189) - [internal]

CP_INDIRECT_BUFFER

```
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 190) - [internal]

CP_INDIRECT_BUFFER_CHAIN

```
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 191) - [internal]

CP_INDIRECT_BUFFER_PFD

```
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 192) - [internal]

CP_INTERRUPT

```
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 193) - [internal]

CP_INVALIDATE_STATE

```
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 194) - [internal]

CP_LOAD_STATE6

```
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 195) - [internal]

CP_LOAD_STATE6_FRAG

```
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 196) - [internal]

CP_LOAD_STATE6_GEOM

```
CP_LOAD_STATE6_GEOM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 197) - [internal]

CP_MEM_TO_REG

```
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 198) - [internal]

CP_MEM_WRITE_CNTR

```
CP_MEM_WRITE_CNTR:
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 199) - [internal] CP_NOP

```
CP_NOP:
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 200) - [internal]

CP_PREEMPT_DISABLE

```
CP_PREEMPT_DISABLE:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 201) - [internal]

CP_RECORD_PFP_TIMESTAMP

```
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 202) - [internal]

CP_REG_TEST

```
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 203) - [internal]

CP_REG_TO_MEM

```
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 204) - [internal]

CP_REG_TO_MEM_OFFSET_MEM

```
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 205) - [internal]

CP_REG_TO_MEM_OFFSET_REG

```
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 206) - [internal]

CP_REG_TO_SCRATCH

```
CP_REG_TO_SCRATCH:
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 207) - [internal]

CP_REG_WRITE

```
CP_REG_WRITE:
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 208) - [internal]

CP_REG_WR_NO_CTXT

```
CP_REG_WR_NO_CTXT:
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 209) - [internal]

CP_RUN_OPENCL

```
CP_RUN_OPENCL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 210) - [internal]

CP_SCRATCH_TO_REG

```
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 211) - [internal]

CP_SET_AMBLE

```
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 212) - [internal]

CP_SET_BIN_DATA5_OFFSET

```
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 213) - [internal]

CP_SET_DRAW_INIT_FLAGS

```
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 214) - [internal]

CP_SET_MARKER

```
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 215) - [internal]

CP_SET_MODE

```
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 216) - [internal]

CP_SET_PROTECTED_MODE

```
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 217) - [internal]

CP_SET_PSEUDO_REG

```
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 218) - [internal]

CP_SET_STATE

```
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 219) - [internal]

CP_SET_SUBDRAW_SIZE

```
CP_SET_SUBDRAW_SIZE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 220) - [internal]

CP_SET_VISIBILITY_OVERRIDE

```
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 221) - [internal]

CP_SKIP_IB2_ENABLE_GLOBAL

```
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 222) - [internal]

CP_SKIP_IB2_ENABLE_LOCAL

```
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 223) - [internal]

CP_SMMU_TABLE_UPDATE

```
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 224) - [internal]

CP_START_BIN

```
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 225) - [internal]

CP_TEST_TWO_MEMS

```
CP_TEST_TWO_MEMS:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 226) - [internal]

CP_WAIT_FOR_IDLE

```
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 227) - [internal]

CP_WAIT_FOR_ME

```
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 228) - [internal]

CP_WAIT_MEM_GTE

```
CP_WAIT_MEM_GTE:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 229) - [internal]

CP_WAIT_MEM_WRITES

```
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 230) - [internal]

CP_WAIT_REG_EQ

```
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 231) - [internal]

CP_WAIT_REG_MEM

```
CP_WAIT_REG_MEM:
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 232) - [internal]

CP_WAIT_TWO_REGS

```
CP_WAIT_TWO_REGS:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN110:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 233) - [internal]

CP_WHERE_AM_I

```
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN110:
UNKN118:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 234) - [internal]

IN_GMU_INTERRUPT

```
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN110:
UNKN118:
UNKN119:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 235) - [internal] IN_IB_END

```
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN110:
UNKN118:
UNKN119:
UNKN12:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 236) - [internal] PKT4

```
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN110:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 237) - [internal] UNKN0

```
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN110:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 238) - [internal] UNKN1

```
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN110:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 239) - [internal] UNKN103

```
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN110:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 240) - [internal] UNKN104

```
UNKN104:
UNKN105:
UNKN106:
UNKN110:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 241) - [internal] UNKN105

```
UNKN105:
UNKN106:
UNKN110:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 242) - [internal] UNKN106

```
UNKN106:
UNKN110:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 243) - [internal] UNKN110

```
UNKN110:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
UNKN13:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 244) - [internal] UNKN118

```
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
UNKN13:
UNKN14:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 245) - [internal] UNKN119

```
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
UNKN13:
UNKN14:
UNKN2:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 246) - [internal] UNKN12

```
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
UNKN13:
UNKN14:
UNKN2:
UNKN21:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 247) - [internal] UNKN121

```
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
UNKN13:
UNKN14:
UNKN2:
UNKN21:
UNKN22:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 248) - [internal] UNKN122

```
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
UNKN13:
UNKN14:
UNKN2:
UNKN21:
UNKN22:
UNKN23:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 249) - [internal] UNKN123

```
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
UNKN13:
UNKN14:
UNKN2:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 250) - [internal] UNKN124

```
UNKN124:
UNKN125:
UNKN126:
UNKN127:
UNKN13:
UNKN14:
UNKN2:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
UNKN27:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 251) - [internal] UNKN125

```
UNKN125:
UNKN126:
UNKN127:
UNKN13:
UNKN14:
UNKN2:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
UNKN27:
UNKN28:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 252) - [internal] UNKN126

```
UNKN126:  
UNKN127:  
UNKN13:  
UNKN14:  
UNKN2:  
UNKN21:  
UNKN22:  
UNKN23:  
UNKN24:  
UNKN27:  
UNKN28:  
UNKN3:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 253) - [internal] UNKN127

```
UNKN127:  
UNKN13:  
UNKN14:  
UNKN2:  
UNKN21:  
UNKN22:  
UNKN23:  
UNKN24:  
UNKN27:  
UNKN28:  
UNKN3:  
UNKN30:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 254) - [internal] UNKN13

```
UNKN13:  
UNKN14:  
UNKN2:  
UNKN21:  
UNKN22:  
UNKN23:  
UNKN24:  
UNKN27:  
UNKN28:  
UNKN3:  
UNKN30:  
UNKN31:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 255) - [internal] UNKN14

```
UNKN14:  
UNKN2:  
UNKN21:  
UNKN22:  
UNKN23:  
UNKN24:  
UNKN27:  
UNKN28:  
UNKN3:  
UNKN30:  
UNKN31:  
UNKN32:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 256) - [internal] UNKN2

```
UNKN2:  
UNKN21:  
UNKN22:  
UNKN23:  
UNKN24:  
UNKN27:  
UNKN28:  
UNKN3:  
UNKN30:  
UNKN31:  
UNKN32:  
UNKN39:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 257) - [internal] UNKN21

```
UNKN21:  
UNKN22:  
UNKN23:  
UNKN24:  
UNKN27:  
UNKN28:  
UNKN3:  
UNKN30:  
UNKN31:  
UNKN32:  
UNKN39:  
UNKN43:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 258) - [internal] UNKN22

```
UNKN22:  
UNKN23:  
UNKN24:  
UNKN27:  
UNKN28:  
UNKN3:  
UNKN30:  
UNKN31:  
UNKN32:  
UNKN39:  
UNKN43:  
UNKN45:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 259) - [internal] UNKN23

```
UNKN23:  
UNKN24:  
UNKN27:  
UNKN28:  
UNKN3:  
UNKN30:  
UNKN31:  
UNKN32:  
UNKN39:  
UNKN43:  
UNKN45:  
UNKN48:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 260) - [internal] UNKN24

```
UNKN24:
UNKN27:
UNKN28:
UNKN3:
UNKN30:
UNKN31:
UNKN32:
UNKN39:
UNKN43:
UNKN45:
UNKN48:
UNKN5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 261) - [internal] UNKN27

```
UNKN27:
UNKN28:
UNKN3:
UNKN30:
UNKN31:
UNKN32:
UNKN39:
UNKN43:
UNKN45:
UNKN48:
UNKN5:
UNKN58:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 262) - [internal] UNKN28

```
UNKN28:
UNKN3:
UNKN30:
UNKN31:
UNKN32:
UNKN39:
UNKN43:
UNKN45:
UNKN48:
UNKN5:
UNKN58:
UNKN6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 263) - [internal] UNKN3

```
UNKN3:
UNKN30:
UNKN31:
UNKN32:
UNKN39:
UNKN43:
UNKN45:
UNKN48:
UNKN5:
UNKN58:
UNKN6:
UNKN7:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 264) - [internal] UNKN30

```
UNKN30:
UNKN31:
UNKN32:
UNKN39:
UNKN43:
UNKN45:
UNKN48:
UNKN5:
UNKN58:
UNKN6:
UNKN7:
UNKN73:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 265) - [internal] UNKN31

```
UNKN31:
UNKN32:
UNKN39:
UNKN43:
UNKN45:
UNKN48:
UNKN5:
UNKN58:
UNKN6:
UNKN7:
UNKN73:
UNKN8:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 266) - [internal] UNKN32

```
UNKN32:
UNKN39:
UNKN43:
UNKN45:
UNKN48:
UNKN5:
UNKN58:
UNKN6:
UNKN7:
UNKN73:
UNKN8:
UNKN88:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 267) - [internal] UNKN39

```
UNKN39:
UNKN43:
UNKN45:
UNKN48:
UNKN5:
UNKN58:
UNKN6:
UNKN7:
UNKN73:
UNKN8:
UNKN88:
UNKN89:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 268) - [internal] UNKN43

```
UNKN43:
UNKN45:
UNKN48:
UNKN5:
UNKN58:
UNKN6:
UNKN7:
UNKN73:
UNKN8:
UNKN88:
UNKN89:
UNKN9:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 269) - [internal] UNKN45

```
UNKN45:
UNKN48:
UNKN5:
UNKN58:
UNKN6:
UNKN7:
UNKN73:
UNKN8:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 270) - [internal] UNKN48

```
UNKN48:
UNKN5:
UNKN58:
UNKN6:
UNKN7:
UNKN73:
UNKN8:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 271) - [internal] UNKN5

```
UNKN5:
UNKN58:
UNKN6:
UNKN7:
UNKN73:
UNKN8:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN93:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 272) - [internal] UNKN58

```
UNKN58:
UNKN6:
UNKN7:
UNKN73:
UNKN8:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN93:
UNKN96:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 273) - [internal] UNKN6

```
UNKN6:
UNKN7:
UNKN73:
UNKN8:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN93:
UNKN96:
UNKN97:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 274) - [internal] UNKN7

```
UNKN7:
UNKN73:
UNKN8:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN93:
UNKN96:
UNKN97:
waitin
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 275) - [internal] UNKN73

```
UNKN73:
UNKN8:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN93:
UNKN96:
UNKN97:
waitin
mov $01, $data
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 276) - [internal] UNKN8

```
UNKN8:  
UNKN88:  
UNKN89:  
UNKN9:  
UNKN90:  
UNKN91:  
UNKN93:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
jumptbl:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 277) - [internal] UNKN88

```
UNKN88:  
UNKN89:  
UNKN9:  
UNKN90:  
UNKN91:  
UNKN93:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
jumptbl:  
    .jumptbl
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 278) - [internal] UNKN89

```
UNKN89:  
UNKN9:  
UNKN90:  
UNKN91:  
UNKN93:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
jumptbl:  
    .jumptbl
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 279) - [internal] UNKN9

```
UNKN9:  
UNKN90:  
UNKN91:  
UNKN93:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
jumptbl:  
    .jumptbl
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 280) - [internal] UNKN90

```
UNKN90:  
UNKN91:  
UNKN93:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
jumptbl:  
    .jumptbl
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 281) - [internal] UNKN91

```
UNKN91:
UNKN93:
UNKN96:
UNKN97:
waitin
mov $01, $data
jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 282) - [internal] UNKN93

```
UNKN93:
UNKN96:
UNKN97:
waitin
mov $01, $data
jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 283) - [internal] UNKN96

```
UNKN96:
UNKN97:
waitin
mov $01, $data
jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 284) - [internal] UNKN97

```
UNKN97:
waitin
mov $01, $data
jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test.asm (line 287) - [internal] jumptbl

```
jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 43) - [internal]

CP_ME_INIT

```
CP_ME_INIT:
mov $02, 0x2
waitin
mov $01, $data

CP_MEM_WRITE:
mov $addr, 0xa0 << 24■; |NRT_ADDR
mov $02, 0x4
(xmovl)add $data, $02, $data
mov $addr, 0xa204 << 16■; |NRT_DATA
(rep)(xmov3)mov $data, $data
waitin
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 48) - [internal]

CP_MEM_WRITE

```
CP_MEM_WRITE:
mov $addr, 0xa0 << 24■; |NRT_ADDR
mov $02, 0x4
(xmovl)add $data, $02, $data
mov $addr, 0xa204 << 16■; |NRT_DATA
(rep)(xmov3)mov $data, $data
waitin
mov $01, $data

CP_SCRATCH_WRITE:
mov $02, 0xff
(rep)cwrite $data, [$02 + 0x1]!
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 57) - [internal]

CP_SCRATCH_WRITE

```
CP_SCRATCH_WRITE:
mov $02, 0xff
(rep)cwrite $data, [$02 + 0x1]!
waitin
mov $01, $data

CP_SET_DRAW_STATE:
(rep)(sds2)cwrite $data, [$00 + @DRAW_STATE_SET_HDR]
waitin
mov $01, $data

CP_SET_BIN_DATA5:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 63) - [internal]

CP_SET_DRAW_STATE

```
CP_SET_DRAW_STATE:
(rep)(sds2)cwrite $data, [$00 + @DRAW_STATE_SET_HDR]
waitin
mov $01, $data

CP_SET_BIN_DATA5:
sread $02, [$00 + %SP]
swrite $02, [$00 + %SP]
mov $02, 0x7
(rep)swrite $data, [$02 + 0x1]!
waitin
mov $01, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 68) - [internal]

CP_SET_BIN_DATA5

```
CP_SET_BIN_DATA5:
sread $02, [$00 + %SP]
swrite $02, [$00 + %SP]
mov $02, 0x7
(rep)swrite $data, [$02 + 0x1]!
waitin
mov $01, $data

CP_SET_SECURE_MODE:
mov $02, $data
setsecure $02, #l65

// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 76) - [internal]

CP_SET_SECURE_MODE

```
CP_SET_SECURE_MODE:
mov $02, $data
setsecure $02, #l65

fxn63:
l63:
jump #l63
nop
l65:
waitin
mov $01, $data

// ... (truncated, see source file for more)
```

```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 80) - [internal] fxn63
    fxn63:
    l63:
    jump #l63
    nop
    l65:
    waitin
    mov $01, $data

    fxn67:
    l67:
    cmp $04, $02, $03
    breq $04, b0, #l74
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 81) - [internal] l63
    l63:
    jump #l63
    nop
    l65:
    waitin
    mov $01, $data

    fxn67:
    l67:
    cmp $04, $02, $03
    breq $04, b0, #l74
    brne $04, b1, #l72
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 84) - [internal] l65
    l65:
    waitin
    mov $01, $data

    fxn67:
    l67:
    cmp $04, $02, $03
    breq $04, b0, #l74
    brne $04, b1, #l72
    breq $04, b2, #l67
    sub $03, $03, $02
    l72:
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 88) - [internal] fxn67
    fxn67:
    l67:
    cmp $04, $02, $03
    breq $04, b0, #l74
    brne $04, b1, #l72
    breq $04, b2, #l67
    sub $03, $03, $02
    l72:
    jump #l67
    sub $02, $02, $03
    l74:
    ret
    // ... (truncated, see source file for more)

```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 89) - [internal] l67

```
l67:
cmp $04, $02, $03
breq $04, b0, #l74
brne $04, b1, #l72
breq $04, b2, #l67
sub $03, $03, $02
l72:
jump #l67
sub $02, $02, $03
l74:
ret
nop
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 95) - [internal] l72

```
l72:
jump #l67
sub $02, $02, $03
l74:
ret
nop

CP_REG_RMW:
cwrite $data, [$00 + @REG_READ_ADDR]
add $02, $regdata, 0x42
addhi $03, $00, $regdata
sub $02, $02, $regdata
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 98) - [internal] l74

```
l74:
ret
nop

CP_REG_RMW:
cwrite $data, [$00 + @REG_READ_ADDR]
add $02, $regdata, 0x42
addhi $03, $00, $regdata
sub $02, $02, $regdata
call #fxn67
subhi $03, $03, $regdata
and $02, $02, $regdata
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 102) - [internal]

CP_REG_RMW

```
CP_REG_RMW:
cwrite $data, [$00 + @REG_READ_ADDR]
add $02, $regdata, 0x42
addhi $03, $00, $regdata
sub $02, $02, $regdata
call #fxn67
subhi $03, $03, $regdata
and $02, $02, $regdata
or $02, $02, 0x1
xor $02, $02, 0x1
not $02, $02
shl $02, $02, $regdata
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 133) - [internal]

CP_MEMCPY

```
CP_MEMCPY:
mov $02, $data
mov $03, $data
mov $04, $data
mov $05, $data
mov $06, $data
l110:
breq $06, 0x0, #l116
cwrite $03, [$00 + @LOAD_STORE_HI]
load $07, [$02 + 0x4]!
cwrite $05, [$00 + @LOAD_STORE_HI]
jump #l110
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 139) - [internal] l110

```
l110:
breq $06, 0x0, #l116
cwrite $03, [$00 + @LOAD_STORE_HI]
load $07, [$02 + 0x4]!
cwrite $05, [$00 + @LOAD_STORE_HI]
jump #l110
store $07, [$04 + 0x4]!
l116:
waitin
mov $01, $data
```

```
CP_MEM_TO_MEM:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 146) - [internal] l116

```
l116:
waitin
mov $01, $data

CP_MEM_TO_MEM:
cwrite $data, [$00 + @MEM_READ_ADDR]
cwrite $data, [$00 + @MEM_READ_ADDR+0x1]
mov $02, $data
cwrite $data, [$00 + @LOAD_STORE_HI]
mov $rem, $data
cwrite $rem, [$00 + @MEM_READ_DWORDS]
(rep)store $memdata, [$02 + 0x4]!
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 150) - [internal]

CP_MEM_TO_MEM

```
CP_MEM_TO_MEM:
cwrite $data, [$00 + @MEM_READ_ADDR]
cwrite $data, [$00 + @MEM_READ_ADDR+0x1]
mov $02, $data
cwrite $data, [$00 + @LOAD_STORE_HI]
mov $rem, $data
cwrite $rem, [$00 + @MEM_READ_DWORDS]
(rep)store $memdata, [$02 + 0x4]!
waitin
mov $01, $data

IN_PREEMPT:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 161) - [internal]

IN_PREEMPT

```
IN_PREEMPT:
preempt:
cread $02, [$00 + 0x101]
brne $02, 0x1, #l136
nop
bl #fxn63
nop
nop
nop
waitin
mov $01, $data
l136:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 162) - [internal]

preempt

```
preempt:
cread $02, [$00 + 0x101]
brne $02, 0x1, #l136
nop
bl #fxn63
nop
nop
nop
waitin
mov $01, $data
l136:
iret
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 172) - [internal] l136

```
l136:
iret
nop

CP_BLIT:
CP_BOOTSTRAP_UCODE:
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 176) - [internal]

CP_BLIT

```
CP_BLIT:
CP_BOOTSTRAP_UCODE:
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 177) - [internal]

CP_BOOTSTRAP_UCODE

```
CP_BOOTSTRAP_UCODE:
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 178) - [internal]

CP_BV_BR_COUNT_OPS

```
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 179) - [internal]

CP_CCHE_INVALIDATE

```
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 180) - [internal]

CP_COND_EXEC

```
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 181) - [internal]

CP_COND_REG_EXEC

```
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 182) - [internal]

CP_COND_WRITE5

```
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 183) - [internal]

CP_CONTEXT_REG_BUNCH

```
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 184) - [internal]

CP_CONTEXT_SWITCH_YIELD

```
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 185) - [internal]

CP_CONTEXT_UPDATE

```
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 186) - [internal]

CP_DRAW_AUTO

```
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 187) - [internal]

CP_DRAW_INDIRECT

```
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 188) - [internal]

CP_DRAW_INDIRECT_MULTI

```
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 189) - [internal]

CP_DRAW_INDX

```
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 190) - [internal]

CP_DRAW_INDX_INDIRECT

```
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 191) - [internal]

CP_DRAW_INDX_OFFSET

```
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 192) - [internal]

CP_DRAW_PRED_ENABLE_GLOBAL

```
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 193) - [internal]

CP_DRAW_PRED_ENABLE_LOCAL

```
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 194) - [internal]

CP_DRAW_PRED_SET

```
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 195) - [internal]

CP_END_BIN

```
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 196) - [internal]

CP_EVENT_WRITE7

```
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 197) - [internal]

CP_EXEC_CS

```
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 198) - [internal]

CP_EXEC_CS_INDIRECT

```
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 199) - [internal]

CP_FIXED_STRIDE_DRAW_TABLE

```
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 200) - [internal]

CP_GLOBAL_TIMESTAMP

```
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 201) - [internal]

CP_INDIRECT_BUFFER

```
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 202) - [internal]

CP_INDIRECT_BUFFER_CHAIN

```
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 203) - [internal]

CP_INDIRECT_BUFFER_PFD

```
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 204) - [internal]

CP_INTERRUPT

```
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 205) - [internal]

CP_INVALIDATE_STATE

```
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 206) - [internal]

CP_LOAD_STATE6

```
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 207) - [internal]

CP_LOAD_STATE6_FRAG

```
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 208) - [internal]

CP_LOAD_STATE6_GEOM

```
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 209) - [internal]

CP_LOCAL_TIMESTAMP

```
CP_LOCAL_TIMESTAMP:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 210) - [internal]

CP_MEM_TO_OC_MEM

```
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 211) - [internal]

CP_MEM_TO_REG

```
CP_MEM_TO_REG:
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 212) - [internal]

CP_MEM_WRITE_CNTR

```
CP_MEM_WRITE_CNTR:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 213) - [internal]

CP_MODIFY_TIMESTAMP

```
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 214) - [internal]

CP_NON_CONTEXT_REG_BUNCH

```
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 215) - [internal]

CP_NOP

```
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 216) - [internal]

CP_RECORD_PFP_TIMESTAMP

```
CP_RECORD_PFP_TIMESTAMP:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 217) - [internal]

CP_REG_TEST

```
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 218) - [internal]

CP_REG_TO_MEM

```
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 219) - [internal]

CP_REG_TO_MEM_OFFSET_MEM

```
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 220) - [internal]

CP_REG_TO_MEM_OFFSET_REG

```
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 221) - [internal]

CP_REG_TO_SCRATCH

```
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 222) - [internal]

CP_REG_WR_NO_CTXT

```
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 223) - [internal]

CP_RESET_CONTEXT_STATE

```
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 224) - [internal]

CP_RESOURCE_LIST

```
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 225) - [internal]

CP_RUN_OPENCL

```
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 226) - [internal]

CP_SCOPE_CNTL

```
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 227) - [internal]

CP_SCRATCH_TO_REG

```
CP_SCRATCH_TO_REG:
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 228) - [internal]

CP_SET_AMBLE

```
CP_SET_AMBLE:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 229) - [internal]

CP_SET_BIN_DATA5_OFFSET

```
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 230) - [internal]

CP_SET_DRAW_INIT_FLAGS

```
CP_SET_DRAW_INIT_FLAGS:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 231) - [internal]

CP_SET_MARKER

```
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 232) - [internal]

CP_SET_MODE

```
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 233) - [internal]

CP_SET_PROTECTED_MODE

```
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 234) - [internal]

CP_SET_PSEUDO_REG

```
CP_SET_PSEUDO_REG:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 235) - [internal]

CP_SET_STATE

```
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 236) - [internal]

CP_SET_SUBDRAW_SIZE

```
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 237) - [internal]

CP_SET_UNK_BIN_DATA

```
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 238) - [internal]

CP_SET_VISIBILITY_OVERRIDE

```
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 239) - [internal]

CP_SKIP_IB2_ENABLE_GLOBAL

```
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 240) - [internal]

CP_SKIP_IB2_ENABLE_LOCAL

```
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 241) - [internal]

CP_SKIP_IB_MODE

```
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 242) - [internal]

CP_SMMU_TABLE_UPDATE

```
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 243) - [internal]

CP_START_BIN

```
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 244) - [internal]

CP_TEST_TWO_MEMS

```
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 245) - [internal]

CP_THREAD_CONTROL

```
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 246) - [internal]

CP_WAIT_FOR_IDLE

```
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 247) - [internal]

CP_WAIT_FOR_ME

```
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 248) - [internal]

CP_WAIT_MEM_WRITES

```
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 249) - [internal]

CP_WAIT_REG_EQ

```
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 250) - [internal]

CP_WAIT_REG_MEM

```
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 251) - [internal]

CP_WAIT_TIMESTAMP

```
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 252) - [internal]

CP_WHERE_AM_I

```
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 253) - [internal]

IN_GMU_INTERRUPT

```
IN_GMU_INTERRUPT:
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 254) - [internal]

IN_IB_END

```
IN_IB_END:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 255) - [internal] PKT4

```
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 256) - [internal] UNKN0

```
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 257) - [internal] UNKN1

```
UNKN1:  
UNKN103:  
UNKN104:  
UNKN105:  
UNKN106:  
UNKN109:  
UNKN110:  
UNKN112:  
UNKN118:  
UNKN119:  
UNKN12:  
UNKN121:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 258) - [internal]
UNKN103

```
UNKN103:  
UNKN104:  
UNKN105:  
UNKN106:  
UNKN109:  
UNKN110:  
UNKN112:  
UNKN118:  
UNKN119:  
UNKN12:  
UNKN121:  
UNKN122:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 259) - [internal]
UNKN104

```
UNKN104:  
UNKN105:  
UNKN106:  
UNKN109:  
UNKN110:  
UNKN112:  
UNKN118:  
UNKN119:  
UNKN12:  
UNKN121:  
UNKN122:  
UNKN123:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 260) - [internal]
UNKN105

```
UNKN105:  
UNKN106:  
UNKN109:  
UNKN110:  
UNKN112:  
UNKN118:  
UNKN119:  
UNKN12:  
UNKN121:  
UNKN122:  
UNKN123:  
UNKN124:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 261) - [internal]

UNKN106

```
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 262) - [internal]

UNKN109

```
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 263) - [internal]

UNKN110

```
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 264) - [internal]

UNKN112

```
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 265) - [internal]

UNKN118

```
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 266) - [internal]

UNKN119

```
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 267) - [internal]

UNKN12

```
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 268) - [internal]

UNKN121

```
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 269) - [internal]

UNKN122

```
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 270) - [internal]

UNKN123

```
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 271) - [internal]

UNKN124

```
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 272) - [internal]

UNKN125

```
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 273) - [internal]
UNKN126

```
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 274) - [internal]
UNKN13

```
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 275) - [internal]
UNKN14

```
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 276) - [internal] UNKN2

```
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 277) - [internal] UNKN3

```
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 278) - [internal]
UNKN30

```
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 279) - [internal]
UNKN32

```
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 280) - [internal]
UNKN43

```
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 281) - [internal]

UNKN48

```
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 282) - [internal] UNKN5

```
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 283) - [internal] UNKN6

```
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
waitin
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 284) - [internal] UNKN7

```
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
waitin
mov $01, $data
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 285) - [internal] UNKN8

```
UNKN8:  
UNKN84:  
UNKN88:  
UNKN89:  
UNKN9:  
UNKN90:  
UNKN91:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
nop  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 286) - [internal]

UNKN84

```
UNKN84:  
UNKN88:  
UNKN89:  
UNKN9:  
UNKN90:  
UNKN91:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
nop  
nop  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 287) - [internal]

UNKN88

```
UNKN88:  
UNKN89:  
UNKN9:  
UNKN90:  
UNKN91:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
nop  
nop  
nop  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 288) - [internal]

UNKN89

```
UNKN89:  
UNKN9:  
UNKN90:  
UNKN91:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
nop  
nop  
nop  
nop  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 289) - [internal] UNKN9

```
UNKN9:  
UNKN90:  
UNKN91:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
nop  
nop  
nop  
nop  
nop  
.align 32  
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 290) - [internal]

UNKN90

```
UNKN90:  
UNKN91:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
nop  
nop  
nop  
nop  
nop  
.align 32  
jumptbl:  
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 291) - [internal]

UNKN91

```
UNKN91:  
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
nop  
nop  
nop  
nop  
nop  
.align 32  
jumptbl:  
.jumptbl  
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 292) - [internal]

UNKN96

```
UNKN96:  
UNKN97:  
waitin  
mov $01, $data  
nop  
nop  
nop  
nop  
nop  
.align 32  
jumptbl:  
.jumptbl  
  
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 293) - [internal]
UNKN97

```
UNKN97:
waitin
mov $01, $data
nop
nop
nop
nop
.align 32
jumptbl:
.jumptbl

.section BV
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 301) - [internal]
jumptbl

```
jumptbl:
.jumptbl

.section BV
;
; BV microcode:
;
[01000001]
[#jumptbl]
cread $01, [$00 + @BV_INSTR_BASE]
cread $02, [$00 + @BV_INSTR_BASE+0x1]
add $01, $01, 0x4
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 339) - [internal]
CP_BLIT

```
CP_BLIT:
CP_BOOTSTRAP_UCODE:
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 340) - [internal]
CP_BOOTSTRAP_UCODE

```
CP_BOOTSTRAP_UCODE:
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 341) - [internal]

CP_BV_BR_COUNT_OPS

```
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 342) - [internal]

CP_CCHE_INVALIDATE

```
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 343) - [internal]

CP_COND_EXEC

```
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 344) - [internal]

CP_COND_REG_EXEC

```
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 345) - [internal]

CP_COND_WRITES5

```
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 346) - [internal]

CP_CONTEXT_REG_BUNCH

```
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 347) - [internal]

CP_CONTEXT_SWITCH_YIELD

```
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 348) - [internal]

CP_CONTEXT_UPDATE

```
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 349) - [internal]

CP_DRAW_AUTO

```
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 350) - [internal]

CP_DRAW_INDIRECT

```
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 351) - [internal]

CP_DRAW_INDIRECT_MULTI

```
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 352) - [internal]

CP_DRAW_INDX

```
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 353) - [internal]

CP_DRAW_INDX_INDIRECT

```
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 354) - [internal]

CP_DRAW_INDX_OFFSET

```
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 355) - [internal]

CP_DRAW_PRED_ENABLE_GLOBAL

```
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 356) - [internal]

CP_DRAW_PRED_ENABLE_LOCAL

```
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 357) - [internal]

CP_DRAW_PRED_SET

```
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 358) - [internal]

CP_END_BIN

```
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 359) - [internal]

CP_EVENT_WRITE7

```
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 360) - [internal]

CP_EXEC_CS

```
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 361) - [internal]

CP_EXEC_CS_INDIRECT

```
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 362) - [internal]

CP_FIXED_STRIDE_DRAW_TABLE

```
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 363) - [internal]

CP_GLOBAL_TIMESTAMP

```
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 364) - [internal]

CP_INDIRECT_BUFFER

```
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 365) - [internal]

CP_INDIRECT_BUFFER_CHAIN

```
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 366) - [internal]

CP_INDIRECT_BUFFER_PFD

```
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 367) - [internal]

CP_INTERRUPT

```
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 368) - [internal]

CP_INVALIDATE_STATE

```
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 369) - [internal]

CP_LOAD_STATE6

```
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 370) - [internal]

CP_LOAD_STATE6_FRAG

```
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 371) - [internal]

CP_LOAD_STATE6_GEOM

```
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 372) - [internal]

CP_LOCAL_TIMESTAMP

```
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 373) - [internal]

CP_MEMCPY

```
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 374) - [internal]

CP_MEM_TO_MEM

```
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 375) - [internal]

CP_MEM_TO_OC_MEM

```
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 376) - [internal]

CP_MEM_TO_REG

```
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 377) - [internal]

CP_MEM_WRITE

```
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 378) - [internal]

CP_MEM_WRITE_CNTR

```
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 379) - [internal]

CP_ME_INIT

```
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 380) - [internal]

CP_MODIFY_TIMESTAMP

```
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 381) - [internal]

CP_NON_CONTEXT_REG_BUNCH

```
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 382) - [internal]

CP_NOP

```
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 383) - [internal]

CP_RECORD_PFP_TIMESTAMP

```
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 384) - [internal]

CP_REG_RMW

```
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 385) - [internal]

CP_REG_TEST

```
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 386) - [internal]

CP_REG_TO_MEM

```
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 387) - [internal]

CP_REG_TO_MEM_OFFSET_MEM

```
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 388) - [internal]

CP_REG_TO_MEM_OFFSET_REG

```
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 389) - [internal]

CP_REG_TO_SCRATCH

```
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 390) - [internal]

CP_REG_WR_NO_CTXT

```
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 391) - [internal]

CP_RESET_CONTEXT_STATE

```
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 392) - [internal]

CP_RESOURCE_LIST

```
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 393) - [internal]

CP_RUN_OPENCL

```
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 394) - [internal]

CP_SCOPE_CNTL

```
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 395) - [internal]

CP_SCRATCH_TO_REG

```
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 396) - [internal]

CP_SCRATCH_WRITE

```
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 397) - [internal]

CP_SET_AMBLE

```
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 398) - [internal]

CP_SET_BIN_DATA5

```
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 399) - [internal]

CP_SET_BIN_DATA5_OFFSET

```
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 400) - [internal]

CP_SET_DRAW_INIT_FLAGS

```
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 401) - [internal]

CP_SET_DRAW_STATE

```
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 402) - [internal]

CP_SET_MARKER

```
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 403) - [internal]

CP_SET_MODE

```
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 404) - [internal]

CP_SET_PROTECTED_MODE

```
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 405) - [internal]

CP_SET_PSEUDO_REG

```
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 406) - [internal]

CP_SET_SECURE_MODE

```
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 407) - [internal]

CP_SET_STATE

```
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 408) - [internal]

CP_SET_SUBDRAW_SIZE

```
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 409) - [internal]

CP_SET_UNK_BIN_DATA

```
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 410) - [internal]

CP_SET_VISIBILITY_OVERRIDE

```
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 411) - [internal]

CP_SKIP_IB2_ENABLE_GLOBAL

```
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 412) - [internal]

CP_SKIP_IB2_ENABLE_LOCAL

```
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 413) - [internal]

CP_SKIP_IB_MODE

```
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 414) - [internal]

CP_SMMU_TABLE_UPDATE

```
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 415) - [internal]

CP_START_BIN

```
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 416) - [internal]

CP_TEST_TWO_MEMS

```
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 417) - [internal]

CP_THREAD_CONTROL

```
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 418) - [internal]

CP_WAIT_FOR_IDLE

```
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 419) - [internal]

CP_WAIT_FOR_ME

```
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 420) - [internal]

CP_WAIT_MEM_WRITES

```
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 421) - [internal]

CP_WAIT_REG_EQ

```
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 422) - [internal]

CP_WAIT_REG_MEM

```
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 423) - [internal]

CP_WAIT_TIMESTAMP

```
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 424) - [internal]

CP_WHERE_AM_I

```
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 425) - [internal]

IN_GMU_INTERRUPT

```
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 426) - [internal]

IN_IB_END

```
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 427) - [internal]

IN_PREEMPT

```
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 428) - [internal] PKT4

```
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 429) - [internal] UNKN0

```
UNKN0:  
UNKN1:  
UNKN103:  
UNKN104:  
UNKN105:  
UNKN106:  
UNKN109:  
UNKN110:  
UNKN112:  
UNKN118:  
UNKN119:  
UNKN12:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 430) - [internal] UNKN1

```
UNKN1:  
UNKN103:  
UNKN104:  
UNKN105:  
UNKN106:  
UNKN109:  
UNKN110:  
UNKN112:  
UNKN118:  
UNKN119:  
UNKN12:  
UNKN121:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 431) - [internal]
UNKN103

```
UNKN103:  
UNKN104:  
UNKN105:  
UNKN106:  
UNKN109:  
UNKN110:  
UNKN112:  
UNKN118:  
UNKN119:  
UNKN12:  
UNKN121:  
UNKN122:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 432) - [internal]
UNKN104

```
UNKN104:  
UNKN105:  
UNKN106:  
UNKN109:  
UNKN110:  
UNKN112:  
UNKN118:  
UNKN119:  
UNKN12:  
UNKN121:  
UNKN122:  
UNKN123:  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 433) - [internal]

UNKN105

```
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 434) - [internal]

UNKN106

```
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 435) - [internal]

UNKN109

```
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 436) - [internal]

UNKN110

```
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 437) - [internal]

UNKN112

```
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 438) - [internal]

UNKN118

```
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 439) - [internal]

UNKN119

```
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 440) - [internal]

UNKN12

```
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 441) - [internal]

UNKN121

```
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 442) - [internal]

UNKN122

```
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 443) - [internal]

UNKN123

```
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 444) - [internal]

UNKN124

```
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 445) - [internal]

UNKN125

```
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 446) - [internal]

UNKN126

```
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 447) - [internal]

UNKN13

```
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 448) - [internal]

UNKN14

```
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 449) - [internal] UNKN2

```
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 450) - [internal] UNKN3

```
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 451) - [internal]
UNKN30

```
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 452) - [internal]
UNKN32

```
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 453) - [internal]

UNKN43

```
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 454) - [internal]

UNKN48

```
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 455) - [internal] UNKN5

```
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 456) - [internal] UNKN6

```
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 457) - [internal] UNKN7

```
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 458) - [internal] UNKN8

```
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 459) - [internal]
UNKN84

```
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
.align 32
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 460) - [internal]
UNKN88

```
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
.align 32
jumptbl:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 461) - [internal]
UNKN89

```
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
.align 32
jumptbl:
.jumptbl
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 462) - [internal] UNKN9

```
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
.align 32
jumptbl:
.jumptbl

    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 463) - [internal]
UNKN90

```
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
.align 32
jumptbl:
.jumptbl

.section LPAC
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 464) - [internal]
UNKN91

```
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
.align 32
jumptbl:
.jumptbl

.section LPAC
;
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 465) - [internal]

UNKN96

```
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
.align 32
jumptbl:
.jumptbl

.section LPAC
;
; LPAC microcode:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 466) - [internal]

UNKN97

```
UNKN97:
preempt:
waitin
mov $01, $data
.align 32
jumptbl:
.jumptbl

.section LPAC
;
; LPAC microcode:
;
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 467) - [internal]

preempt

```
preempt:
waitin
mov $01, $data
.align 32
jumptbl:
.jumptbl

.section LPAC
;
; LPAC microcode:
;
[01000001]
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 471) - [internal]

jumptbl

```
jumptbl:
.jumptbl

.section LPAC
;
; LPAC microcode:
;
[01000001]
[#jumptbl]
cread $01, [$00 + @LPAC_INSTR_BASE]
cread $02, [$00 + @LPAC_INSTR_BASE+0x1]
add $01, $01, 0x4
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 503) - [internal]

CP_BLIT

```
CP_BLIT:
CP_BOOTSTRAP_UCODE:
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 504) - [internal]

CP_BOOTSTRAP_UCODE

```
CP_BOOTSTRAP_UCODE:
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 505) - [internal]

CP_BV_BR_COUNT_OPS

```
CP_BV_BR_COUNT_OPS:
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 506) - [internal]

CP_CCHE_INVALIDATE

```
CP_CCHE_INVALIDATE:
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 507) - [internal]

CP_COND_EXEC

```
CP_COND_EXEC:
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 508) - [internal]

CP_COND_REG_EXEC

```
CP_COND_REG_EXEC:
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 509) - [internal]

CP_COND_WRITE5

```
CP_COND_WRITE5:
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 510) - [internal]

CP_CONTEXT_REG_BUNCH

```
CP_CONTEXT_REG_BUNCH:
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 511) - [internal]

CP_CONTEXT_SWITCH_YIELD

```
CP_CONTEXT_SWITCH_YIELD:
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 512) - [internal]

CP_CONTEXT_UPDATE

```
CP_CONTEXT_UPDATE:
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 513) - [internal]

CP_DRAW_AUTO

```
CP_DRAW_AUTO:
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 514) - [internal]

CP_DRAW_INDIRECT

```
CP_DRAW_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 515) - [internal]

CP_DRAW_INDIRECT_MULTI

```
CP_DRAW_INDIRECT_MULTI:
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 516) - [internal]

CP_DRAW_INDX

```
CP_DRAW_INDX:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 517) - [internal]

CP_DRAW_INDX_INDIRECT

```
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 518) - [internal]

CP_DRAW_INDX_OFFSET

```
CP_DRAW_INDX_OFFSET:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 519) - [internal]

CP_DRAW_PRED_ENABLE_GLOBAL

```
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 520) - [internal]

CP_DRAW_PRED_ENABLE_LOCAL

```
CP_DRAW_PRED_ENABLE_LOCAL:
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 521) - [internal]

CP_DRAW_PRED_SET

```
CP_DRAW_PRED_SET:
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 522) - [internal]

CP_END_BIN

```
CP_END_BIN:
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 523) - [internal]

CP_EVENT_WRITE7

```
CP_EVENT_WRITE7:
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 524) - [internal]

CP_EXEC_CS

```
CP_EXEC_CS:
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 525) - [internal]

CP_EXEC_CS_INDIRECT

```
CP_EXEC_CS_INDIRECT:
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 526) - [internal]

CP_FIXED_STRIDE_DRAW_TABLE

```
CP_FIXED_STRIDE_DRAW_TABLE:
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 527) - [internal]

CP_GLOBAL_TIMESTAMP

```
CP_GLOBAL_TIMESTAMP:
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 528) - [internal]

CP_INDIRECT_BUFFER

```
CP_INDIRECT_BUFFER:
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 529) - [internal]

CP_INDIRECT_BUFFER_CHAIN

```
CP_INDIRECT_BUFFER_CHAIN:
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 530) - [internal]

CP_INDIRECT_BUFFER_PFD

```
CP_INDIRECT_BUFFER_PFD:
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 531) - [internal]

CP_INTERRUPT

```
CP_INTERRUPT:
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 532) - [internal]

CP_INVALIDATE_STATE

```
CP_INVALIDATE_STATE:
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 533) - [internal]

CP_LOAD_STATE6

```
CP_LOAD_STATE6:
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 534) - [internal]

CP_LOAD_STATE6_FRAG

```
CP_LOAD_STATE6_FRAG:
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 535) - [internal]

CP_LOAD_STATE6_GEOM

```
CP_LOAD_STATE6_GEOM:
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 536) - [internal]

CP_LOCAL_TIMESTAMP

```
CP_LOCAL_TIMESTAMP:
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 537) - [internal]

CP_MEMCPY

```
CP_MEMCPY:
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 538) - [internal]

CP_MEM_TO_MEM

```
CP_MEM_TO_MEM:
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 539) - [internal]

CP_MEM_TO_OC_MEM

```
CP_MEM_TO_OC_MEM:
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 540) - [internal]

CP_MEM_TO_REG

```
CP_MEM_TO_REG:
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 541) - [internal]

CP_MEM_WRITE

```
CP_MEM_WRITE:
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 542) - [internal]

CP_MEM_WRITE_CNTR

```
CP_MEM_WRITE_CNTR:
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 543) - [internal]

CP_ME_INIT

```
CP_ME_INIT:
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 544) - [internal]

CP_MODIFY_TIMESTAMP

```
CP_MODIFY_TIMESTAMP:
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 545) - [internal]

CP_NON_CONTEXT_REG_BUNCH

```
CP_NON_CONTEXT_REG_BUNCH:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 546) - [internal]

CP_NOP

```
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 547) - [internal]

CP_RECORD_PFP_TIMESTAMP

```
CP_RECORD_PFP_TIMESTAMP:
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 548) - [internal]

CP_REG_RMW

```
CP_REG_RMW:
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 549) - [internal]

CP_REG_TEST

```
CP_REG_TEST:
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 550) - [internal]

CP_REG_TO_MEM

```
CP_REG_TO_MEM:
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 551) - [internal]

CP_REG_TO_MEM_OFFSET_MEM

```
CP_REG_TO_MEM_OFFSET_MEM:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 552) - [internal]

CP_REG_TO_MEM_OFFSET_REG

```
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 553) - [internal]

CP_REG_TO_SCRATCH

```
CP_REG_TO_SCRATCH:
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 554) - [internal]

CP_REG_WR_NO_CTXT

```
CP_REG_WR_NO_CTXT:
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 555) - [internal]

CP_RESET_CONTEXT_STATE

```
CP_RESET_CONTEXT_STATE:
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 556) - [internal]

CP_RESOURCE_LIST

```
CP_RESOURCE_LIST:
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 557) - [internal]

CP_RUN_OPENCL

```
CP_RUN_OPENCL:
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 558) - [internal]

CP_SCOPE_CNTL

```
CP_SCOPE_CNTL:
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 559) - [internal]

CP_SCRATCH_TO_REG

```
CP_SCRATCH_TO_REG:
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 560) - [internal]

CP_SCRATCH_WRITE

```
CP_SCRATCH_WRITE:
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 561) - [internal]

CP_SET_AMBLE

```
CP_SET_AMBLE:
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 562) - [internal]

CP_SET_BIN_DATA5

```
CP_SET_BIN_DATA5:
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 563) - [internal]

CP_SET_BIN_DATA5_OFFSET

```
CP_SET_BIN_DATA5_OFFSET:
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 564) - [internal]

CP_SET_DRAW_INIT_FLAGS

```
CP_SET_DRAW_INIT_FLAGS:
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 565) - [internal]

CP_SET_DRAW_STATE

```
CP_SET_DRAW_STATE:
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 566) - [internal]

CP_SET_MARKER

```
CP_SET_MARKER:
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 567) - [internal]

CP_SET_MODE

```
CP_SET_MODE:
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 568) - [internal]

CP_SET_PROTECTED_MODE

```
CP_SET_PROTECTED_MODE:
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 569) - [internal]

CP_SET_PSEUDO_REG

```
CP_SET_PSEUDO_REG:
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 570) - [internal]

CP_SET_SECURE_MODE

```
CP_SET_SECURE_MODE:
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 571) - [internal]

CP_SET_STATE

```
CP_SET_STATE:
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 572) - [internal]

CP_SET_SUBDRAW_SIZE

```
CP_SET_SUBDRAW_SIZE:
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 573) - [internal]

CP_SET_UNK_BIN_DATA

```
CP_SET_UNK_BIN_DATA:
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 574) - [internal]

CP_SET_VISIBILITY_OVERRIDE

```
CP_SET_VISIBILITY_OVERRIDE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 575) - [internal]

CP_SKIP_IB2_ENABLE_GLOBAL

```
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 576) - [internal]

CP_SKIP_IB2_ENABLE_LOCAL

```
CP_SKIP_IB2_ENABLE_LOCAL:
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 577) - [internal]

CP_SKIP_IB_MODE

```
CP_SKIP_IB_MODE:
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 578) - [internal]

CP_SMMU_TABLE_UPDATE

```
CP_SMMU_TABLE_UPDATE:
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 579) - [internal]

CP_START_BIN

```
CP_START_BIN:
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 580) - [internal]

CP_TEST_TWO_MEMS

```
CP_TEST_TWO_MEMS:
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 581) - [internal]

CP_THREAD_CONTROL

```
CP_THREAD_CONTROL:
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 582) - [internal]

CP_WAIT_FOR_IDLE

```
CP_WAIT_FOR_IDLE:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 583) - [internal]

CP_WAIT_FOR_ME

```
CP_WAIT_FOR_ME:
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 584) - [internal]

CP_WAIT_MEM_WRITES

```
CP_WAIT_MEM_WRITES:
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 585) - [internal]

CP_WAIT_REG_EQ

```
CP_WAIT_REG_EQ:
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 586) - [internal]

CP_WAIT_REG_MEM

```
CP_WAIT_REG_MEM:
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 587) - [internal]

CP_WAIT_TIMESTAMP

```
CP_WAIT_TIMESTAMP:
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 588) - [internal]

CP_WHERE_AM_I

```
CP_WHERE_AM_I:
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 589) - [internal]

IN_GMU_INTERRUPT

```
IN_GMU_INTERRUPT:
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 590) - [internal]

IN_IB_END

```
IN_IB_END:
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 591) - [internal]
IN_PREEMPT

```
IN_PREEMPT:
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 592) - [internal] PKT4

```
PKT4:
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 593) - [internal] UNKN0

```
UNKN0:
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 594) - [internal] UNKN1

```
UNKN1:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 595) - [internal]

UNKN103

```
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 596) - [internal]

UNKN104

```
UNKN104:
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 597) - [internal]

UNKN105

```
UNKN105:
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 598) - [internal]

UNKN106

```
UNKN106:
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 599) - [internal]

UNKN109

```
UNKN109:
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 600) - [internal]

UNKN110

```
UNKN110:
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 601) - [internal]

UNKN112

```
UNKN112:
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 602) - [internal]

UNKN118

```
UNKN118:
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 603) - [internal]

UNKN119

```
UNKN119:
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 604) - [internal]

UNKN12

```
UNKN12:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 605) - [internal]

UNKN121

```
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 606) - [internal]

UNKN122

```
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 607) - [internal]

UNKN123

```
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 608) - [internal]

UNKN124

```
UNKN124:
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 609) - [internal]

UNKN125

```
UNKN125:
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 610) - [internal]

UNKN126

```
UNKN126:
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 611) - [internal]

UNKN13

```
UNKN13:
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 612) - [internal]

UNKN14

```
UNKN14:
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 613) - [internal] UNKN2

```
UNKN2:
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 614) - [internal] UNKN3

```
UNKN3:
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 615) - [internal]

UNKN30

```
UNKN30:
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 616) - [internal]

UNKN32

```
UNKN32:
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 617) - [internal]

UNKN43

```
UNKN43:
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 618) - [internal]

UNKN48

```
UNKN48:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
    // ... (truncated, see source file for more)
```

```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 619) - [internal] UNKN5
    UNKN5:
    UNKN6:
    UNKN7:
    UNKN8:
    UNKN84:
    UNKN88:
    UNKN89:
    UNKN9:
    UNKN90:
    UNKN91:
    UNKN96:
    UNKN97:
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 620) - [internal] UNKN6
    UNKN6:
    UNKN7:
    UNKN8:
    UNKN84:
    UNKN88:
    UNKN89:
    UNKN9:
    UNKN90:
    UNKN91:
    UNKN96:
    UNKN97:
    preempt:
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 621) - [internal] UNKN7
    UNKN7:
    UNKN8:
    UNKN84:
    UNKN88:
    UNKN89:
    UNKN9:
    UNKN90:
    UNKN91:
    UNKN96:
    UNKN97:
    preempt:
    waitin
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 622) - [internal] UNKN8
    UNKN8:
    UNKN84:
    UNKN88:
    UNKN89:
    UNKN9:
    UNKN90:
    UNKN91:
    UNKN96:
    UNKN97:
    preempt:
    waitin
    mov $01, $data
    // ... (truncated, see source file for more)

```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 623) - [internal]

UNKN84

```
UNKN84:
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
nop
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 624) - [internal]

UNKN88

```
UNKN88:
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
nop
nop
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 625) - [internal]

UNKN89

```
UNKN89:
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
nop
nop
nop
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 626) - [internal] UNKN9

```
UNKN9:
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
nop
nop
nop
nop
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 627) - [internal]

UNKN90

```
UNKN90:
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
nop
nop
nop
nop
nop
nop
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 628) - [internal]

UNKN91

```
UNKN91:
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
nop
nop
nop
nop
nop
nop
nop
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 629) - [internal]

UNKN96

```
UNKN96:
UNKN97:
preempt:
waitin
mov $01, $data
nop
nop
nop
nop
nop
nop
nop
.align 32
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/reference/qrisc_test_a7xx.asm (line 630) - [internal]

UNKN97

```
UNKN97:
preempt:
waitin
mov $01, $data
nop
nop
nop
nop
nop
nop
nop
.align 32
jumptbl:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 92) - [internal]

CP_SCRATCH_WRITE

```
CP_SCRATCH_WRITE:
; test (rep) + flags + non-zero offset with cwrite
; TODO: 0x4 flag is actually pre-increment addressing, handle it as such
mov $02, 0xff
(rep)cwrite $data, [$02 + 0x001]!
waitin
mov $01, $data

CP_SET_DRAW_STATE:
; test (sds)
(rep)(sds2) cwrite $data, [$00 + @DRAW_STATE_SET_HDR]
waitin
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 100) - [internal]

CP_SET_DRAW_STATE

```
CP_SET_DRAW_STATE:
; test (sds)
(rep)(sds2) cwrite $data, [$00 + @DRAW_STATE_SET_HDR]
waitin
mov $01, $data

CP_SET_BIN_DATA5:
; test SQE registers
sread $02, [$00 + %SP]
swrite $02, [$00 + %SP]
mov $02, 7
(rep)swrite $data, [$02 + 1]!
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 106) - [internal]

CP_SET_BIN_DATA5

```
CP_SET_BIN_DATA5:
; test SQE registers
sread $02, [$00 + %SP]
swrite $02, [$00 + %SP]
mov $02, 7
(rep)swrite $data, [$02 + 1]!
waitin
mov $01, $data

CP_SET_SECURE_MODE:
; test setsecure
mov $02, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 115) - [internal]

CP_SET_SECURE_MODE

```
CP_SET_SECURE_MODE:
; test setsecure
mov $02, $data
setsecure $02, #setsecure_success
err:
jump #err
nop
setsecure_success:
waitin
mov $01, $data

euclid:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 119) - [internal] err

```
err:
    jump #err
    nop
setsecure_success:
    waitin
    mov $01, $data

euclid:
    ; Euclid's algorithm in QRisc: https://en.wikipedia.org/wiki/Euclidean\_algorithm
    ; Since QRisc doesn't do modulo, we implement the subtraction-based version.
    ;
    ; Demonstrates/tests comparisons and conditional branches. This also
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 122) - [internal] setsecure_success

```
setsecure_success:
    waitin
    mov $01, $data

euclid:
    ; Euclid's algorithm in QRisc: https://en.wikipedia.org/wiki/Euclidean\_algorithm
    ; Since QRisc doesn't do modulo, we implement the subtraction-based version.
    ;
    ; Demonstrates/tests comparisons and conditional branches. This also
    ; demonstrates the common trick of branching in a delay slot. Note that if a
    ; branch is taken and its delay slot includes another branch, the second
    ; branch cannot also be taken, which is why the last branch in the sequence
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 126) - [internal] euclid

```
euclid:
    ; Euclid's algorithm in QRisc: https://en.wikipedia.org/wiki/Euclidean\_algorithm
    ; Since QRisc doesn't do modulo, we implement the subtraction-based version.
    ;
    ; Demonstrates/tests comparisons and conditional branches. This also
    ; demonstrates the common trick of branching in a delay slot. Note that if a
    ; branch is taken and its delay slot includes another branch, the second
    ; branch cannot also be taken, which is why the last branch in the sequence
    ; cannot be unconditional.
    ;
    ; Inputs are in $02 and $03, and output is in $02.
    cmp $04, $02, $03
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 142) - [internal] euclid_gt

```
euclid_gt:
    jump #euclid
    sub $02, $02, $03
euclid_exit:
    ret
    nop

CP_REG_RMW:
    ; Test various ALU instructions, and read/write $regdata
    cwrite $data, [$00 + @REG_READ_ADDR]
    add $02, $regdata, 0x42
    addhi $03, $00, $regdata
    // ... (truncated, see source file for more)
```

```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 145) - [internal] euclid_exit
    euclid_exit:
        ret
        nop

    CP_REG_RMW:
        ; Test various ALU instructions, and read/write $regdata
        cwrite $data, [$00 + @REG_READ_ADDR]
        add $02, $regdata, 0x42
        addhi $03, $00, $regdata
        sub $02, $02, $regdata
        call #euclid
        subhi $03, $03, $regdata
        // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 149) - [internal] CP_REG_RMW
    CP_REG_RMW:
        ; Test various ALU instructions, and read/write $regdata
        cwrite $data, [$00 + @REG_READ_ADDR]
        add $02, $regdata, 0x42
        addhi $03, $00, $regdata
        sub $02, $02, $regdata
        call #euclid
        subhi $03, $03, $regdata
        and $02, $02, $regdata
        or $02, $02, 0x1
        xor $02, $02, 0x1
        not $02, $02
        // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 174) - [internal] CP_MEMCPY
    CP_MEMCPY:
        ; implement CP_MEMCPY using load/store instructions
        mov $02, $data
        mov $03, $data
        mov $04, $data
        mov $05, $data
        mov $06, $data
    cpy_header:
        breq $06, 0, #cpy_exit
        cwrite $03, [$00 + @LOAD_STORE_HI]
        load $07, [$02 + 0x004]!
        cwrite $05, [$00 + @LOAD_STORE_HI]
        // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 181) - [internal] cpy_header
    cpy_header:
        breq $06, 0, #cpy_exit
        cwrite $03, [$00 + @LOAD_STORE_HI]
        load $07, [$02 + 0x004]!
        cwrite $05, [$00 + @LOAD_STORE_HI]
        jump #cpy_header
        store $07, [$04 + 0x004]!
    cpy_exit:
        waitin
        mov $01, $data

    CP_MEM_TO_MEM:
        // ... (truncated, see source file for more)

```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 188) - [internal] cpy_exit

```
cpy_exit:
waitin
mov $01, $data

CP_MEM_TO_MEM:
; implement CP_MEMCPY using mem read control regs
; tests @F00+0x1 for 64-bit control regs, and reading/writing $rem
cwrite $data, [$00 + @MEM_READ_ADDR]
cwrite $data, [$00 + @MEM_READ_ADDR+1]
mov $02, $data
cwrite $data, [$00 + @LOAD_STORE_HI]
mov $rem, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 192) - [internal] CP_MEM_TO_MEM

```
CP_MEM_TO_MEM:
; implement CP_MEMCPY using mem read control regs
; tests @F00+0x1 for 64-bit control regs, and reading/writing $rem
cwrite $data, [$00 + @MEM_READ_ADDR]
cwrite $data, [$00 + @MEM_READ_ADDR+1]
mov $02, $data
cwrite $data, [$00 + @LOAD_STORE_HI]
mov $rem, $data
cwrite $rem, [$00 + @MEM_READ_DWORDS]
(rep)store $memdata, [$02 + 0x004]!
waitin
mov $01, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 205) - [internal] IN_PREEMPT

```
IN_PREEMPT:
preempt:
; test bl + ired + conditional branch w/ immed
cread $02, [$00 + 0x101]
brne $02, 0x0001, #exit_ired
nop
bl #err
nop
nop
nop
waitin
mov $01, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 206) - [internal] preempt

```
preempt:
; test bl + ired + conditional branch w/ immed
cread $02, [$00 + 0x101]
brne $02, 0x0001, #exit_ired
nop
bl #err
nop
nop
nop
waitin
mov $01, $data
exit_ired:
// ... (truncated, see source file for more)
```

```
dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 217) - [internal] exit_iret
    exit_iret:
    iret
    nop

    UNKN0:
    UNKN1:
    UNKN2:
    UNKN3:
    PKT4:
    UNKN5:
    UNKN6:
    UNKN7:
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 221) - [internal] UNKN0
    UNKN0:
    UNKN1:
    UNKN2:
    UNKN3:
    PKT4:
    UNKN5:
    UNKN6:
    UNKN7:
    UNKN8:
    UNKN9:
    UNKN10:
    UNKN11:
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 222) - [internal] UNKN1
    UNKN1:
    UNKN2:
    UNKN3:
    PKT4:
    UNKN5:
    UNKN6:
    UNKN7:
    UNKN8:
    UNKN9:
    UNKN10:
    UNKN11:
    UNKN12:
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 223) - [internal] UNKN2
    UNKN2:
    UNKN3:
    PKT4:
    UNKN5:
    UNKN6:
    UNKN7:
    UNKN8:
    UNKN9:
    UNKN10:
    UNKN11:
    UNKN12:
    UNKN13:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 224) - [internal] UNKN3

```
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 225) - [internal] PKT4

```
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 226) - [internal] UNKN5

```
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 227) - [internal] UNKN6

```
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 228) - [internal] UNKN7

```
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 229) - [internal] UNKN8

```
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 230) - [internal] UNKN9

```
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 231) - [internal] UNKN10

```
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 232) - [internal] UNKN11

```
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 233) - [internal] UNKN12

```
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 234) - [internal] UNKN13

```
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 235) - [internal] UNKN14

```
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 236) - [internal] CP_NOP

```
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 237) - [internal]

CP_RECORD_PFP_TIMESTAMP

```
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 238) - [internal]

CP_WAIT_MEM_WRITES

```
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 239) - [internal]

CP_WAIT_FOR_ME

```
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 240) - [internal]

CP_WAIT_MEM_GTE

```
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 241) - [internal] UNKN21

```
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 242) - [internal] UNKN22

```
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 243) - [internal] UNKN23

```
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 244) - [internal] UNKN24

```
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 245) - [internal]

CP_DRAW_PRED_ENABLE_GLOBAL

```
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 246) - [internal]

CP_DRAW_PRED_ENABLE_LOCAL

```
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 247) - [internal] UNKN27

```
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 248) - [internal]

CP_PREEMPT_ENABLE

```
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 249) - [internal]

CP_SKIP_IB2_ENABLE_GLOBAL

```
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 250) - [internal]

CP_PREEMPT_TOKEN

```
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 251) - [internal] UNKN31

```
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 252) - [internal] UNKN32

```
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 253) - [internal] CP_DRAW_INDX

```
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 254) - [internal]

CP_SKIP_IB2_ENABLE_LOCAL

```
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 255) - [internal] CP_DRAW_AUTO

```
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 256) - [internal] CP_SET_STATE

```
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 257) - [internal]

CP_WAIT_FOR_IDLE

```
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 258) - [internal] CP_IM_LOAD

```
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 259) - [internal]

CP_DRAW_INDIRECT

```
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 260) - [internal]

CP_DRAW_INDX_INDIRECT

```
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 261) - [internal]

CP_DRAW_INDIRECT_MULTI

```
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 262) - [internal]

CP_IM_LOAD_IMMEDIATE

```
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 263) - [internal] CP_BLIT

```
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 264) - [internal]

CP_SET_CONSTANT

```
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_IDX_OFFSET:
CP_REG_TEST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 265) - [internal]

CP_SET_BIN_DATA5_OFFSET

```
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_IDX_OFFSET:
CP_REG_TEST:
UNKN58:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 266) - [internal] UNKN48

```
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_IDX_OFFSET:
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 267) - [internal] CP_RUN_OPENCL

```
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_IDX_OFFSET:
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 268) - [internal]

CP_LOAD_STATE6_GEOM

```
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 269) - [internal] CP_EXEC_CS

```
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 270) - [internal]

CP_LOAD_STATE6_FRAG

```
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 271) - [internal]

CP_SET_SUBDRAW_SIZE

```
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 272) - [internal]

CP_LOAD_STATE6

```
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 273) - [internal]

CP_INDIRECT_BUFFER_PFD

```
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 274) - [internal]

CP_DRAW_INDX_OFFSET

```
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 275) - [internal] CP_REG_TEST

```
CP_REG_TEST:
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 276) - [internal] UNKN58

```
UNKN58:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 277) - [internal]

CP_INVALIDATE_STATE

```
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 278) - [internal]

CP_WAIT_REG_MEM

```
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 279) - [internal] CP_REG_TO_MEM

```
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 280) - [internal]

CP_INDIRECT_BUFFER

```
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 281) - [internal] CP_INTERRUPT

```
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 282) - [internal]

CP_EXEC_CS_INDIRECT

```
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 283) - [internal] CP_MEM_TO_REG

```
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 284) - [internal] CP_COND_EXEC

```
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 285) - [internal]

CP_COND_WRITE5

```
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 286) - [internal]

CP_EVENT_WRITE

```
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 287) - [internal]

CP_COND_REG_EXEC

```
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 288) - [internal] UNKN73

```
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 289) - [internal]

CP_REG_TO_SCRATCH

```
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 290) - [internal]

CP_SET_DRAW_INIT_FLAGS

```
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 291) - [internal]

CP_SCRATCH_TO_REG

```
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 292) - [internal]

CP_DRAW_PRED_SET

```
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 293) - [internal]

CP_MEM_WRITE_CNTR

```
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 294) - [internal] CP_START_BIN

```
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 295) - [internal] CP_END_BIN

```
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 296) - [internal]

CP_WAIT_REG_EQ

```
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 297) - [internal]

CP_SMMU_TABLE_UPDATE

```
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 298) - [internal] UNKN84

```
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 299) - [internal] CP_SET_AMBLE

```
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 300) - [internal]

CP_SET_PSEUDO_REG

```
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 301) - [internal]

CP_INDIRECT_BUFFER_CHAIN

```
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 302) - [internal]

CP_EVENT_WRITE_SHD

```
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 303) - [internal]

CP_EVENT_WRITE_CFL

```
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 304) - [internal] UNKN90

```
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 305) - [internal]

CP_EVENT_WRITE_ZPD

```
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 306) - [internal]

CP_CONTEXT_REG_BUNCH

```
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 307) - [internal]

CP_WAIT_IB_PFD_COMPLETE

```
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 308) - [internal]

CP_CONTEXT_UPDATE

```
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 309) - [internal]

CP_SET_PROTECTED_MODE

```
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 310) - [internal] UNKN96

```
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 311) - [internal] UNKN97

```
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 312) - [internal] UNKN98

```
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 313) - [internal] CP_SET_MODE

```
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 314) - [internal]

CP_SET_VISIBILITY_OVERRIDE

```
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 315) - [internal] CP_SET_MARKER

```
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 316) - [internal] UNKN103

```
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 317) - [internal] UNKN104

```
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 318) - [internal] UNKN105

```
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 319) - [internal] UNKN106

```
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 320) - [internal] UNKN107

```
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 321) - [internal] UNKN108

```
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 322) - [internal] CP_REG_WRITE

```
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 323) - [internal] UNKN110

```
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 324) - [internal]

CP_BOOTSTRAP_UCODE

```
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 325) - [internal]

CP_WAIT_TWO_REGS

```
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 326) - [internal]

CP_TEST_TWO_MEMS

```
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 327) - [internal]

CP_REG_TO_MEM_OFFSET_REG

```
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 328) - [internal]

CP_REG_TO_MEM_OFFSET_MEM

```
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 329) - [internal] UNKN118

```
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 330) - [internal] UNKN119

```
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data

    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 331) - [internal]

CP_REG_WR_NO_CTXT

```
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data

jumptbl:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 332) - [internal] UNKN121

```
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data

jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 333) - [internal] UNKN122

```
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data

jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 334) - [internal] UNKN123

```
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data

jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 335) - [internal] UNKN124

```
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data

jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 336) - [internal] UNKN125

```
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data

jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 337) - [internal] UNKN126

```
UNKN126:
UNKN127:
    waitin
    mov $01, $data

jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 338) - [internal] UNKN127

```
UNKN127:
    waitin
    mov $01, $data
```

```
jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test.asm (line 342) - [internal] jumptbl

```
jumptbl:
.jumptbl
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 28) - [internal] loc02

```
loc02:
; packet table loading:
mov $01, 0x0830 ; CP_SQE_INSTR_BASE
mov $02, 0x0002
cwrite $01, [$00 + @REG_READ_ADDR]
cwrite $02, [$00 + @REG_READ_DWORDS]
; move hi/lo of SQE fw addrs to registers:
mov $01, $regdata
mov $02, $regdata
; Use byte unit labels to get the packet table address:
add $03, $01, #jumptbl * 4
addhi $04, $02, 0x0000
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 83) - [internal]

CP_ME_INIT

```
CP_ME_INIT:
; test label-as-immediate feature
mov $02, #loc02 ; should be 0x0002
waitin
mov $01, $data
```

```
CP_MEM_WRITE:
; test $addr + (rep) + (xmovN) with ALU
mov $addr, 0xa0 << 24
mov $02, 4
(xmov1)add $data, $02, $data
mov $addr, 0xa204 << 16
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 89) - [internal]

CP_MEM_WRITE

```
CP_MEM_WRITE:
; test $addr + (rep) + (xmovN) with ALU
mov $addr, 0xa0 << 24
mov $02, 4
(xmov1)add $data, $02, $data
mov $addr, 0xa204 << 16
(rep)(xmov3)mov $data, $data
waitin
mov $01, $data
```

```
CP_SCRATCH_WRITE:
; test (rep) + preincrement + non-zero offset with cwrite
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 99) - [internal]

CP_SCRATCH_WRITE

```
CP_SCRATCH_WRITE:
; test (rep) + preincrement + non-zero offset with cwrite
mov $02, 0xff
(rep)cwrite $data, [$02 + 0x001]!
waitin
mov $01, $data

CP_SET_DRAW_STATE:
; test (sds)
(rep)(sds2) cwrite $data, [$00 + @DRAW_STATE_SET_HDR]
waitin
mov $01, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 106) - [internal]

CP_SET_DRAW_STATE

```
CP_SET_DRAW_STATE:
; test (sds)
(rep)(sds2) cwrite $data, [$00 + @DRAW_STATE_SET_HDR]
waitin
mov $01, $data

CP_SET_BIN_DATA5:
; test SQE registers
sread $02, [$00 + %SP]
swrite $02, [$00 + %SP]
mov $02, 7
(rep)swrite $data, [$02 + 1]!
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 112) - [internal]

CP_SET_BIN_DATA5

```
CP_SET_BIN_DATA5:
; test SQE registers
sread $02, [$00 + %SP]
swrite $02, [$00 + %SP]
mov $02, 7
(rep)swrite $data, [$02 + 1]!
waitin
mov $01, $data

CP_SET_SECURE_MODE:
; test setsecure
mov $02, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 121) - [internal]

CP_SET_SECURE_MODE

```
CP_SET_SECURE_MODE:
; test setsecure
mov $02, $data
setsecure $02, #setsecure_success
err:
jump #err
nop
setsecure_success:
waitin
mov $01, $data

euclid:
// ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 125) - [internal] err

```
err:
    jump #err
    nop
    setsecure_success:
    waitin
    mov $01, $data

    euclid:
    ; Euclid's algorithm in QRisc: https://en.wikipedia.org/wiki/Euclidean\_algorithm
    ; Since QRisc doesn't do modulo, we implement the subtraction-based version.
    ;
    ; Demonstrates/tests comparisons and conditional branches. This also
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 128) - [internal] setsecure_success

```
    setsecure_success:
    waitin
    mov $01, $data

    euclid:
    ; Euclid's algorithm in QRisc: https://en.wikipedia.org/wiki/Euclidean\_algorithm
    ; Since QRisc doesn't do modulo, we implement the subtraction-based version.
    ;
    ; Demonstrates/tests comparisons and conditional branches. This also
    ; demonstrates the common trick of branching in a delay slot. Note that if a
    ; branch is taken and its delay slot includes another branch, the second
    ; branch cannot also be taken, which is why the last branch in the sequence
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 132) - [internal] euclid

```
    euclid:
    ; Euclid's algorithm in QRisc: https://en.wikipedia.org/wiki/Euclidean\_algorithm
    ; Since QRisc doesn't do modulo, we implement the subtraction-based version.
    ;
    ; Demonstrates/tests comparisons and conditional branches. This also
    ; demonstrates the common trick of branching in a delay slot. Note that if a
    ; branch is taken and its delay slot includes another branch, the second
    ; branch cannot also be taken, which is why the last branch in the sequence
    ; cannot be unconditional.
    ;
    ; Inputs are in $02 and $03, and output is in $02.
    cmp $04, $02, $03
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 148) - [internal]

euclid_gt

```
    euclid_gt:
    jump #euclid
    sub $02, $02, $03
    euclid_exit:
    ret
    nop

    CP_REG_RMW:
    ; Test various ALU instructions, and read/write $regdata
    cwrite $data, [$00 + @REG_READ_ADDR]
    add $02, $regdata, 0x42
    addhi $03, $00, $regdata
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 151) - [internal]
euclid_exit

```
euclid_exit:
ret
nop

CP_REG_RMW:
; Test various ALU instructions, and read/write $regdata
cwrite $data, [$00 + @REG_READ_ADDR]
add $02, $regdata, 0x42
addhi $03, $00, $regdata
sub $02, $02, $regdata
call #euclid
subhi $03, $03, $regdata
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 155) - [internal]
CP_REG_RMW

```
CP_REG_RMW:
; Test various ALU instructions, and read/write $regdata
cwrite $data, [$00 + @REG_READ_ADDR]
add $02, $regdata, 0x42
addhi $03, $00, $regdata
sub $02, $02, $regdata
call #euclid
subhi $03, $03, $regdata
and $02, $02, $regdata
or $02, $02, 0x1
xor $02, $02, 0x1
not $02, $02
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 187) - [internal]
CP_MEMCPY

```
CP_MEMCPY:
; implement CP_MEMCPY using load/store instructions
mov $02, $data
mov $03, $data
mov $04, $data
mov $05, $data
mov $06, $data
cpy_header:
breq $06, 0, #cpy_exit
cwrite $03, [$00 + @LOAD_STORE_HI]
load $07, [$02 + 0x004]!
cwrite $05, [$00 + @LOAD_STORE_HI]
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 194) - [internal]
cpy_header

```
cpy_header:
breq $06, 0, #cpy_exit
cwrite $03, [$00 + @LOAD_STORE_HI]
load $07, [$02 + 0x004]!
cwrite $05, [$00 + @LOAD_STORE_HI]
jump #cpy_header
store $07, [$04 + 0x004]!
cpy_exit:
waitin
mov $01, $data

CP_MEM_TO_MEM:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 201) - [internal] cpy_exit

```
cpy_exit:
waitin
mov $01, $data

CP_MEM_TO_MEM:
; implement CP_MEMCPY using mem read control regs
; tests @F00+0x1 for 64-bit control regs, and reading/writing $rem
cwrite $data, [$00 + @MEM_READ_ADDR]
cwrite $data, [$00 + @MEM_READ_ADDR+1]
mov $02, $data
cwrite $data, [$00 + @LOAD_STORE_HI]
mov $rem, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 205) - [internal]

CP_MEM_TO_MEM

```
CP_MEM_TO_MEM:
; implement CP_MEMCPY using mem read control regs
; tests @F00+0x1 for 64-bit control regs, and reading/writing $rem
cwrite $data, [$00 + @MEM_READ_ADDR]
cwrite $data, [$00 + @MEM_READ_ADDR+1]
mov $02, $data
cwrite $data, [$00 + @LOAD_STORE_HI]
mov $rem, $data
cwrite $rem, [$00 + @MEM_READ_DWORDS]
(rep)store $memdata, [$02 + 0x004]!
waitin
mov $01, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 218) - [internal]

IN_PREEMPT

```
IN_PREEMPT:
preempt:
; test bl + ired + conditional branch w/ immed
cread $02, [$00 + 0x101]
brne $02, 0x0001, #exit_ired
nop
bl #err
nop
nop
nop
waitin
mov $01, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 219) - [internal] preempt

```
preempt:
; test bl + ired + conditional branch w/ immed
cread $02, [$00 + 0x101]
brne $02, 0x0001, #exit_ired
nop
bl #err
nop
nop
nop
waitin
mov $01, $data
exit_ired:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 230) - [internal]

exit_iret

```
exit_iret:
    iret
    nop
```

```
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
```

// ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 234) - [internal] UNKN0

```
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
```

// ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 235) - [internal] UNKN1

```
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
```

// ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 236) - [internal] UNKN2

```
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
```

// ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 237) - [internal] UNKN3

```
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 238) - [internal] PKT4

```
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 239) - [internal] UNKN5

```
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 240) - [internal] UNKN6

```
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 241) - [internal] UNKN7

```
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 242) - [internal] UNKN8

```
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 243) - [internal] UNKN9

```
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 244) - [internal] UNKN10

```
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 245) - [internal] UNKN11

```
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 246) - [internal] UNKN12

```
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 247) - [internal] UNKN13

```
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 248) - [internal] UNKN14

```
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 249) - [internal] CP_NOP

```
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 250) - [internal]

CP_RECORD_PFP_TIMESTAMP

```
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 251) - [internal]

CP_WAIT_MEM_WRITES

```
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 252) - [internal]

CP_WAIT_FOR_ME

```
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 253) - [internal]

CP_WAIT_MEM_GTE

```
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 254) - [internal] UNKN21

```
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 255) - [internal] UNKN22

```
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 256) - [internal] UNKN23

```
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 257) - [internal] UNKN24

```
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 258) - [internal]

CP_DRAW_PRED_ENABLE_GLOBAL

```
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 259) - [internal]

CP_DRAW_PRED_ENABLE_LOCAL

```
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 260) - [internal] UNKN27

```
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 261) - [internal]

CP_PREEMPT_ENABLE

```
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_IDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 262) - [internal]

CP_SKIP_IB2_ENABLE_GLOBAL

```
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_IDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_IDX_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 263) - [internal]

CP_PREEMPT_TOKEN

```
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_IDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_IDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 264) - [internal] UNKN31

```
UNKN31:
UNKN32:
CP_DRAW_IDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_IDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 265) - [internal] UNKN32

```
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 266) - [internal]

```
CP_DRAW_INDX
    CP_DRAW_INDX:
    CP_SKIP_IB2_ENABLE_LOCAL:
    CP_DRAW_AUTO:
    CP_SET_STATE:
    CP_WAIT_FOR_IDLE:
    CP_IM_LOAD:
    CP_DRAW_INDIRECT:
    CP_DRAW_INDX_INDIRECT:
    CP_DRAW_INDIRECT_MULTI:
    CP_IM_LOAD_IMMEDIATE:
    CP_BLIT:
    CP_SET_CONSTANT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 267) - [internal]

```
CP_SKIP_IB2_ENABLE_LOCAL
    CP_SKIP_IB2_ENABLE_LOCAL:
    CP_DRAW_AUTO:
    CP_SET_STATE:
    CP_WAIT_FOR_IDLE:
    CP_IM_LOAD:
    CP_DRAW_INDIRECT:
    CP_DRAW_INDX_INDIRECT:
    CP_DRAW_INDIRECT_MULTI:
    CP_IM_LOAD_IMMEDIATE:
    CP_BLIT:
    CP_SET_CONSTANT:
    CP_SET_BIN_DATA5_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 268) - [internal]

```
CP_DRAW_AUTO
    CP_DRAW_AUTO:
    CP_SET_STATE:
    CP_WAIT_FOR_IDLE:
    CP_IM_LOAD:
    CP_DRAW_INDIRECT:
    CP_DRAW_INDX_INDIRECT:
    CP_DRAW_INDIRECT_MULTI:
    CP_IM_LOAD_IMMEDIATE:
    CP_BLIT:
    CP_SET_CONSTANT:
    CP_SET_BIN_DATA5_OFFSET:
    UNKN48:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 269) - [internal]

CP_SET_STATE

```
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 270) - [internal]

CP_WAIT_FOR_IDLE

```
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 271) - [internal]

CP_IM_LOAD

```
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 272) - [internal]

CP_DRAW_INDIRECT

```
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 273) - [internal]

CP_DRAW_INDX_INDIRECT

```
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 274) - [internal]

CP_DRAW_INDIRECT_MULTI

```
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 275) - [internal]

CP_IM_LOAD_IMMEDIATE

```
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 276) - [internal] CP_BLIT

```
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 277) - [internal]

CP_SET_CONSTANT

```
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_IDX_OFFSET:
CP_REG_TEST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 278) - [internal]

CP_SET_BIN_DATA5_OFFSET

```
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_IDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 279) - [internal] UNKN48

```
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_IDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 280) - [internal]

CP_RUN_OPENCL

```
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_IDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 281) - [internal]

CP_LOAD_STATE6_GEOM

```
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 282) - [internal]

CP_EXEC_CS

```
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 283) - [internal]

CP_LOAD_STATE6_FRAG

```
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 284) - [internal]

CP_SET_SUBDRAW_SIZE

```
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 285) - [internal]

CP_LOAD_STATE6

```
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 286) - [internal]

CP_INDIRECT_BUFFER_PFD

```
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 287) - [internal]

CP_DRAW_INDX_OFFSET

```
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 288) - [internal]

CP_REG_TEST

```
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 289) - [internal]

CP_CCHE_INVALIDATE

```
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 290) - [internal]

CP_INVALIDATE_STATE

```
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 291) - [internal]

CP_WAIT_REG_MEM

```
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 292) - [internal]

CP_REG_TO_MEM

```
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 293) - [internal]

CP_INDIRECT_BUFFER

```
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 294) - [internal]

CP_INTERRUPT

```
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 295) - [internal]

CP_EXEC_CS_INDIRECT

```
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 296) - [internal]

CP_MEM_TO_REG

```
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 297) - [internal]

CP_COND_EXEC

```
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 298) - [internal]

CP_COND_WRITE5

```
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 299) - [internal]

CP_EVENT_WRITE

```
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 300) - [internal]

CP_COND_REG_EXEC

```
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 301) - [internal] UNKN73

```
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 302) - [internal]

CP_REG_TO_SCRATCH

```
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 303) - [internal]

CP_SET_DRAW_INIT_FLAGS

```
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 304) - [internal]

CP_SCRATCH_TO_REG

```
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 305) - [internal]

CP_DRAW_PRED_SET

```
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 306) - [internal]

CP_MEM_WRITE_CNTR

```
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 307) - [internal]

CP_START_BIN

```
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 308) - [internal]

CP_END_BIN

```
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 309) - [internal]

CP_WAIT_REG_EQ

```
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 310) - [internal]

CP_SMMU_TABLE_UPDATE

```
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 311) - [internal] UNKN84

```
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 312) - [internal]

CP_SET_AMBLE

```
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 313) - [internal]

CP_SET_PSEUDO_REG

```
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 314) - [internal]

CP_INDIRECT_BUFFER_CHAIN

```
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 315) - [internal]

CP_EVENT_WRITE_SHD

```
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 316) - [internal]

CP_EVENT_WRITE_CFL

```
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 317) - [internal] UNKN90

```
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 318) - [internal]

CP_EVENT_WRITE_ZPD

```
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 319) - [internal]

CP_CONTEXT_REG_BUNCH

```
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 320) - [internal]

CP_WAIT_IB_PFD_COMPLETE

```
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 321) - [internal]

CP_CONTEXT_UPDATE

```
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 322) - [internal]

CP_SET_PROTECTED_MODE

```
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 323) - [internal] UNKN96

```
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 324) - [internal] UNKN97

```
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 325) - [internal] UNKN98

```
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 326) - [internal]

CP_SET_MODE

```
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 327) - [internal]

CP_SET_VISIBILITY_OVERRIDE

```
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 328) - [internal]

CP_SET_MARKER

```
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 329) - [internal] UNKN103

```
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 330) - [internal] UNKN104

```
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 331) - [internal] UNKN105

```
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 332) - [internal] UNKN106

```
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 333) - [internal] UNKN107

```
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 334) - [internal] UNKN108

```
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 335) - [internal]

CP_REG_WRITE

```
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 336) - [internal] UNKN110

```
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 337) - [internal]

CP_BOOTSTRAP_UCODE

```
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 338) - [internal]

CP_WAIT_TWO_REGS

```
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 339) - [internal]

CP_TEST_TWO_MEMS

```
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 340) - [internal]

CP_REG_TO_MEM_OFFSET_REG

```
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 341) - [internal]

CP_REG_TO_MEM_OFFSET_MEM

```
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 342) - [internal] UNKN118

```
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 343) - [internal] UNKN119

```
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data

    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 344) - [internal]

CP_REG_WR_NO_CTXT

```
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data

.align 32
    // ... (truncated, see source file for more)
```

```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 345) - [internal] UNKN121
    UNKN121:
    UNKN122:
    UNKN123:
    UNKN124:
    UNKN125:
    UNKN126:
    UNKN127:
        waitin
        mov $01, $data

    .align 32
    jumptbl:
        // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 346) - [internal] UNKN122
    UNKN122:
    UNKN123:
    UNKN124:
    UNKN125:
    UNKN126:
    UNKN127:
        waitin
        mov $01, $data

    .align 32
    jumptbl:
    .jumptbl
        // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 347) - [internal] UNKN123
    UNKN123:
    UNKN124:
    UNKN125:
    UNKN126:
    UNKN127:
        waitin
        mov $01, $data

    .align 32
    jumptbl:
    .jumptbl

        // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 348) - [internal] UNKN124
    UNKN124:
    UNKN125:
    UNKN126:
    UNKN127:
        waitin
        mov $01, $data

    .align 32
    jumptbl:
    .jumptbl

    .section BV
        // ... (truncated, see source file for more)

```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 349) - [internal] UNKN125

```
UNKN125:
UNKN126:
UNKN127:
    waitin
    mov $01, $data
```

```
.align 32
jumptbl:
.jumptbl
```

```
.section BV
; BV microcode
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 350) - [internal] UNKN126

```
UNKN126:
UNKN127:
    waitin
    mov $01, $data
```

```
.align 32
jumptbl:
.jumptbl
```

```
.section BV
; BV microcode
start:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 351) - [internal] UNKN127

```
UNKN127:
    waitin
    mov $01, $data
```

```
.align 32
jumptbl:
.jumptbl
```

```
.section BV
; BV microcode
start:
```

```
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 356) - [internal] jumptbl

```
jumptbl:
.jumptbl
```

```
.section BV
; BV microcode
start:
```

```
[01000001]
[#jumptbl]
```

```
; read BV fw addr
cread $01, [$00 + @BV_INSTR_BASE]
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 361) - [internal] start

; BV microcode

```
start:

[01000001]
[#jumptbl]

; read BV fw addr
cread $01, [$00 + @BV_INSTR_BASE]
cread $02, [$00 + @BV_INSTR_BASE+1]

; skip first dword
add $01, $01, 0x0004
addhi $02, $02, 0x0000
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 406) - [internal]

CP_ME_INIT

```
CP_ME_INIT:
CP_MEM_WRITE:
CP_SCRATCH_WRITE:
CP_SET_DRAW_STATE:
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 407) - [internal]

CP_MEM_WRITE

```
CP_MEM_WRITE:
CP_SCRATCH_WRITE:
CP_SET_DRAW_STATE:
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 408) - [internal]

CP_SCRATCH_WRITE

```
CP_SCRATCH_WRITE:
CP_SET_DRAW_STATE:
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 409) - [internal]

CP_SET_DRAW_STATE

```
CP_SET_DRAW_STATE:
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 410) - [internal]

CP_SET_BIN_DATA5

```
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 411) - [internal]

CP_SET_SECURE_MODE

```
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 412) - [internal]

CP_REG_RMW

```
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 413) - [internal]

CP_MEMCPY

```
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 414) - [internal]

CP_MEM_TO_MEM

```
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 415) - [internal]

IN_PREEMPT

```
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 416) - [internal] UNKN0

```
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 417) - [internal] UNKN1

```
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 418) - [internal] UNKN2

```
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 419) - [internal] UNKN3

```
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 420) - [internal] PKT4

```
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 421) - [internal] UNKN5

```
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 422) - [internal] UNKN6

```
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 423) - [internal] UNKN7

```
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 424) - [internal] UNKN8

```
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 425) - [internal] UNKN9

```
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 426) - [internal] UNKN10

```
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 427) - [internal] UNKN11

```
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 428) - [internal] UNKN12

```
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 429) - [internal] UNKN13

```
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 430) - [internal] UNKN14

```
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 431) - [internal] CP_NOP

```
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 432) - [internal]

CP_RECORD_PFP_TIMESTAMP

```
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 433) - [internal]

CP_WAIT_MEM_WRITES

```
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 434) - [internal]

CP_WAIT_FOR_ME

```
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 435) - [internal]

CP_WAIT_MEM_GTE

```
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 436) - [internal] UNKN21

```
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 437) - [internal] UNKN22

```
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 438) - [internal] UNKN23

```
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 439) - [internal] UNKN24

```
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 440) - [internal]

CP_DRAW_PRED_ENABLE_GLOBAL

```
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 441) - [internal]

CP_DRAW_PRED_ENABLE_LOCAL

```
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 442) - [internal] UNKN27

```
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 443) - [internal]

CP_PREEMPT_ENABLE

```
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 444) - [internal]

CP_SKIP_IB2_ENABLE_GLOBAL

```
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 445) - [internal]

CP_PREEMPT_TOKEN

```
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 446) - [internal] UNKN31

```
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 447) - [internal] UNKN32

```
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 448) - [internal]

CP_DRAW_INDX

```
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 449) - [internal]

CP_SKIP_IB2_ENABLE_LOCAL

```
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 450) - [internal]

CP_DRAW_AUTO

```
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 451) - [internal]

CP_SET_STATE

```
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 452) - [internal]

CP_WAIT_FOR_IDLE

```
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 453) - [internal]

CP_IM_LOAD

```
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 454) - [internal]

CP_DRAW_INDIRECT

```
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 455) - [internal]

CP_DRAW_INDX_INDIRECT

```
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 456) - [internal]

CP_DRAW_INDIRECT_MULTI

```
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 457) - [internal]

CP_IM_LOAD_IMMEDIATE

```
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 458) - [internal] CP_BLIT

```
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 459) - [internal]

CP_SET_CONSTANT

```
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 460) - [internal]

CP_SET_BIN_DATA5_OFFSET

```
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 461) - [internal] UNKN48

```
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 462) - [internal]

CP_RUN_OPENCL

```
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 463) - [internal]

CP_LOAD_STATE6_GEOM

```
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 464) - [internal]

CP_EXEC_CS

```
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 465) - [internal]

CP_LOAD_STATE6_FRAG

```
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 466) - [internal]

CP_SET_SUBDRAW_SIZE

```
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 467) - [internal]

CP_LOAD_STATE6

```
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 468) - [internal]

CP_INDIRECT_BUFFER_PFD

```
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 469) - [internal]

CP_DRAW_INDX_OFFSET

```
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 470) - [internal]

CP_REG_TEST

```
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 471) - [internal]

CP_CCHE_INVALIDATE

```
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 472) - [internal]

CP_INVALIDATE_STATE

```
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 473) - [internal]

CP_WAIT_REG_MEM

```
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 474) - [internal]

CP_REG_TO_MEM

```
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 475) - [internal]

CP_INDIRECT_BUFFER

```
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 476) - [internal]

CP_INTERRUPT

```
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 477) - [internal]

CP_EXEC_CS_INDIRECT

```
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 478) - [internal]

CP_MEM_TO_REG

```
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 479) - [internal]

CP_COND_EXEC

```
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 480) - [internal]

CP_COND_WRITE5

```
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 481) - [internal]

CP_EVENT_WRITE

```
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 482) - [internal]

CP_COND_REG_EXEC

```
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 483) - [internal] UNKN73

```
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 484) - [internal]

CP_REG_TO_SCRATCH

```
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 485) - [internal]

CP_SET_DRAW_INIT_FLAGS

```
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 486) - [internal]

CP_SCRATCH_TO_REG

```
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 487) - [internal]

CP_DRAW_PRED_SET

```
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 488) - [internal]

CP_MEM_WRITE_CNTR

```
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 489) - [internal]

CP_START_BIN

```
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 490) - [internal]

CP_END_BIN

```
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 491) - [internal]

CP_WAIT_REG_EQ

```
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 492) - [internal]

CP_SMMU_TABLE_UPDATE

```
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 493) - [internal] UNKN84

```
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 494) - [internal]

CP_SET_AMBLE

```
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 495) - [internal]

CP_SET_PSEUDO_REG

```
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 496) - [internal]

CP_INDIRECT_BUFFER_CHAIN

```
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 497) - [internal]

CP_EVENT_WRITE_SHD

```
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 498) - [internal]

CP_EVENT_WRITE_CFL

```
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 499) - [internal] UNKN90

```
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 500) - [internal]

CP_EVENT_WRITE_ZPD

```
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 501) - [internal]

CP_CONTEXT_REG_BUNCH

```
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 502) - [internal]

CP_WAIT_IB_PFD_COMPLETE

```
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 503) - [internal]

CP_CONTEXT_UPDATE

```
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 504) - [internal]

CP_SET_PROTECTED_MODE

```
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 505) - [internal] UNKN96

```
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 506) - [internal] UNKN97

```
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 507) - [internal] UNKN98

```
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 508) - [internal]

CP_SET_MODE

```
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 509) - [internal]

CP_SET_VISIBILITY_OVERRIDE

```
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 510) - [internal]

CP_SET_MARKER

```
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 511) - [internal] UNKN103

```
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 512) - [internal] UNKN104

```
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 513) - [internal] UNKN105

```
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 514) - [internal] UNKN106

```
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 515) - [internal] UNKN107

```
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 516) - [internal] UNKN108

```
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 517) - [internal]

CP_REG_WRITE

```
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 518) - [internal] UNKN110

```
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 519) - [internal]

CP_BOOTSTRAP_UCODE

```
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 520) - [internal]

CP_WAIT_TWO_REGS

```
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 521) - [internal]

CP_TEST_TWO_MEMS

```
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 522) - [internal]

CP_REG_TO_MEM_OFFSET_REG

```
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 523) - [internal]

CP_REG_TO_MEM_OFFSET_MEM

```
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 524) - [internal] UNKN118

```
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
waitin
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 525) - [internal] UNKN119

```
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
waitin
mov $01, $data
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 526) - [internal]

CP_REG_WR_NO_CTXT

```
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
waitin
mov $01, $data

// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 527) - [internal] UNKN121

```
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
waitin
mov $01, $data

.align 32
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 528) - [internal] UNKN122

```
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
waitin
mov $01, $data

.align 32
jumptbl:
// ... (truncated, see source file for more)
```



```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 529) - [internal] UNKN123
    UNKN123:
    UNKN124:
    UNKN125:
    UNKN126:
    UNKN127:
    preempt:
    waitin
    mov $01, $data

    .align 32
    jumptbl:
    .jumptbl
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 530) - [internal] UNKN124
    UNKN124:
    UNKN125:
    UNKN126:
    UNKN127:
    preempt:
    waitin
    mov $01, $data

    .align 32
    jumptbl:
    .jumptbl

    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 531) - [internal] UNKN125
    UNKN125:
    UNKN126:
    UNKN127:
    preempt:
    waitin
    mov $01, $data

    .align 32
    jumptbl:
    .jumptbl

    .section LPAC
    // ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 532) - [internal] UNKN126
    UNKN126:
    UNKN127:
    preempt:
    waitin
    mov $01, $data

    .align 32
    jumptbl:
    .jumptbl

    .section LPAC
    ; LPAC microcode
    // ... (truncated, see source file for more)

```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 533) - [internal] UNKN127

```
UNKN127:
preempt:
waitin
mov $01, $data

.align 32
jumptbl:
.jumptbl

.section LPAC
; LPAC microcode
start:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 534) - [internal] preempt

```
preempt:
waitin
mov $01, $data

.align 32
jumptbl:
.jumptbl

.section LPAC
; LPAC microcode
start:
[01000001]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 539) - [internal] jumptbl

```
jumptbl:
.jumptbl

.section LPAC
; LPAC microcode
start:
[01000001]
[#jumptbl]

; read LPAC fw addr
cread $01, [$00 + @LPAC_INSTR_BASE]
cread $02, [$00 + @LPAC_INSTR_BASE+1]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 544) - [internal] start

```
; LPAC microcode

start:
[01000001]
[#jumptbl]

; read LPAC fw addr
cread $01, [$00 + @LPAC_INSTR_BASE]
cread $02, [$00 + @LPAC_INSTR_BASE+1]

; skip first dword
add $01, $01, 0x0004
addhi $02, $02, 0x0000
mov $03, 0x0001
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 580) - [internal]

CP_ME_INIT

```
CP_ME_INIT:
CP_MEM_WRITE:
CP_SCRATCH_WRITE:
CP_SET_DRAW_STATE:
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 581) - [internal]

CP_MEM_WRITE

```
CP_MEM_WRITE:
CP_SCRATCH_WRITE:
CP_SET_DRAW_STATE:
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 582) - [internal]

CP_SCRATCH_WRITE

```
CP_SCRATCH_WRITE:
CP_SET_DRAW_STATE:
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 583) - [internal]

CP_SET_DRAW_STATE

```
CP_SET_DRAW_STATE:
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 584) - [internal]

CP_SET_BIN_DATA5

```
CP_SET_BIN_DATA5:
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 585) - [internal]

CP_SET_SECURE_MODE

```
CP_SET_SECURE_MODE:
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 586) - [internal]

CP_REG_RMW

```
CP_REG_RMW:
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 587) - [internal]

CP_MEMCPY

```
CP_MEMCPY:
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 588) - [internal]

CP_MEM_TO_MEM

```
CP_MEM_TO_MEM:
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 589) - [internal]

IN_PREEMPT

```
IN_PREEMPT:
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 590) - [internal] UNKN0

```
UNKN0:
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 591) - [internal] UNKN1

```
UNKN1:
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 592) - [internal] UNKN2

```
UNKN2:
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 593) - [internal] UNKN3

```
UNKN3:
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 594) - [internal] PKT4

```
PKT4:
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 595) - [internal] UNKN5

```
UNKN5:
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 596) - [internal] UNKN6

```
UNKN6:
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 597) - [internal] UNKN7

```
UNKN7:
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 598) - [internal] UNKN8

```
UNKN8:
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 599) - [internal] UNKN9

```
UNKN9:
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 600) - [internal] UNKN10

```
UNKN10:
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 601) - [internal] UNKN11

```
UNKN11:
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 602) - [internal] UNKN12

```
UNKN12:
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 603) - [internal] UNKN13

```
UNKN13:
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 604) - [internal] UNKN14

```
UNKN14:
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 605) - [internal] CP_NOP

```
CP_NOP:
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 606) - [internal]

CP_RECORD_PFP_TIMESTAMP

```
CP_RECORD_PFP_TIMESTAMP:
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 607) - [internal]

CP_WAIT_MEM_WRITES

```
CP_WAIT_MEM_WRITES:
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 608) - [internal]

CP_WAIT_FOR_ME

```
CP_WAIT_FOR_ME:
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 609) - [internal]

CP_WAIT_MEM_GTE

```
CP_WAIT_MEM_GTE:
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 610) - [internal] UNKN21

```
UNKN21:
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 611) - [internal] UNKN22

```
UNKN22:
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 612) - [internal] UNKN23

```
UNKN23:
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 613) - [internal] UNKN24

```
UNKN24:
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 614) - [internal]

CP_DRAW_PRED_ENABLE_GLOBAL

```
CP_DRAW_PRED_ENABLE_GLOBAL:
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 615) - [internal]

CP_DRAW_PRED_ENABLE_LOCAL

```
CP_DRAW_PRED_ENABLE_LOCAL:
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 616) - [internal] UNKN27

```
UNKN27:
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 617) - [internal]

CP_PREEMPT_ENABLE

```
CP_PREEMPT_ENABLE:
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 618) - [internal]

CP_SKIP_IB2_ENABLE_GLOBAL

```
CP_SKIP_IB2_ENABLE_GLOBAL:
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 619) - [internal]

CP_PREEMPT_TOKEN

```
CP_PREEMPT_TOKEN:
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 620) - [internal] UNKN31

```
UNKN31:
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 621) - [internal] UNKN32

```
UNKN32:
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 622) - [internal]

CP_DRAW_INDX

```
CP_DRAW_INDX:
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 623) - [internal]

CP_SKIP_IB2_ENABLE_LOCAL

```
CP_SKIP_IB2_ENABLE_LOCAL:
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 624) - [internal]

CP_DRAW_AUTO

```
CP_DRAW_AUTO:
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 625) - [internal]

CP_SET_STATE

```
CP_SET_STATE:
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 626) - [internal]

CP_WAIT_FOR_IDLE

```
CP_WAIT_FOR_IDLE:
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 627) - [internal]

CP_IM_LOAD

```
CP_IM_LOAD:
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 628) - [internal]

CP_DRAW_INDIRECT

```
CP_DRAW_INDIRECT:
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 629) - [internal]

CP_DRAW_INDX_INDIRECT

```
CP_DRAW_INDX_INDIRECT:
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 630) - [internal]

CP_DRAW_INDIRECT_MULTI

```
CP_DRAW_INDIRECT_MULTI:
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 631) - [internal]

CP_IM_LOAD_IMMEDIATE

```
CP_IM_LOAD_IMMEDIATE:
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 632) - [internal] CP_BLIT

```
CP_BLIT:
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 633) - [internal]

CP_SET_CONSTANT

```
CP_SET_CONSTANT:
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 634) - [internal]

CP_SET_BIN_DATA5_OFFSET

```
CP_SET_BIN_DATA5_OFFSET:
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 635) - [internal] UNKN48

```
UNKN48:
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 636) - [internal]

CP_RUN_OPENCL

```
CP_RUN_OPENCL:
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 637) - [internal]

CP_LOAD_STATE6_GEOM

```
CP_LOAD_STATE6_GEOM:
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 638) - [internal]

CP_EXEC_CS

```
CP_EXEC_CS:
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 639) - [internal]

CP_LOAD_STATE6_FRAG

```
CP_LOAD_STATE6_FRAG:
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 640) - [internal]

CP_SET_SUBDRAW_SIZE

```
CP_SET_SUBDRAW_SIZE:
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 641) - [internal]

CP_LOAD_STATE6

```
CP_LOAD_STATE6:
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 642) - [internal]

CP_INDIRECT_BUFFER_PFD

```
CP_INDIRECT_BUFFER_PFD:
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 643) - [internal]

CP_DRAW_INDX_OFFSET

```
CP_DRAW_INDX_OFFSET:
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 644) - [internal]

CP_REG_TEST

```
CP_REG_TEST:
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 645) - [internal]

CP_CCHE_INVALIDATE

```
CP_CCHE_INVALIDATE:
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 646) - [internal]

CP_INVALIDATE_STATE

```
CP_INVALIDATE_STATE:
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 647) - [internal]

CP_WAIT_REG_MEM

```
CP_WAIT_REG_MEM:
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 648) - [internal]

CP_REG_TO_MEM

```
CP_REG_TO_MEM:
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 649) - [internal]

CP_INDIRECT_BUFFER

```
CP_INDIRECT_BUFFER:
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 650) - [internal]

CP_INTERRUPT

```
CP_INTERRUPT:
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 651) - [internal]

CP_EXEC_CS_INDIRECT

```
CP_EXEC_CS_INDIRECT:
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 652) - [internal]

CP_MEM_TO_REG

```
CP_MEM_TO_REG:
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 653) - [internal]

CP_COND_EXEC

```
CP_COND_EXEC:
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 654) - [internal]

CP_COND_WRITE5

```
CP_COND_WRITE5:
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 655) - [internal]

CP_EVENT_WRITE

```
CP_EVENT_WRITE:
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 656) - [internal]

CP_COND_REG_EXEC

```
CP_COND_REG_EXEC:
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 657) - [internal] UNKN73

```
UNKN73:
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 658) - [internal]

CP_REG_TO_SCRATCH

```
CP_REG_TO_SCRATCH:
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 659) - [internal]

CP_SET_DRAW_INIT_FLAGS

```
CP_SET_DRAW_INIT_FLAGS:
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 660) - [internal]

CP_SCRATCH_TO_REG

```
CP_SCRATCH_TO_REG:
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 661) - [internal]

CP_DRAW_PRED_SET

```
CP_DRAW_PRED_SET:
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 662) - [internal]

CP_MEM_WRITE_CNTR

```
CP_MEM_WRITE_CNTR:
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 663) - [internal]

CP_START_BIN

```
CP_START_BIN:
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 664) - [internal]

CP_END_BIN

```
CP_END_BIN:
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 665) - [internal]

CP_WAIT_REG_EQ

```
CP_WAIT_REG_EQ:
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 666) - [internal]

CP_SMMU_TABLE_UPDATE

```
CP_SMMU_TABLE_UPDATE:
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 667) - [internal] UNKN84

```
UNKN84:
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 668) - [internal]

CP_SET_AMBLE

```
CP_SET_AMBLE:
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 669) - [internal]

CP_SET_PSEUDO_REG

```
CP_SET_PSEUDO_REG:
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 670) - [internal]

CP_INDIRECT_BUFFER_CHAIN

```
CP_INDIRECT_BUFFER_CHAIN:
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 671) - [internal]

CP_EVENT_WRITE_SHD

```
CP_EVENT_WRITE_SHD:
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 672) - [internal]

CP_EVENT_WRITE_CFL

```
CP_EVENT_WRITE_CFL:
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 673) - [internal] UNKN90

```
UNKN90:
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 674) - [internal]

CP_EVENT_WRITE_ZPD

```
CP_EVENT_WRITE_ZPD:
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 675) - [internal]

CP_CONTEXT_REG_BUNCH

```
CP_CONTEXT_REG_BUNCH:
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 676) - [internal]

CP_WAIT_IB_PFD_COMPLETE

```
CP_WAIT_IB_PFD_COMPLETE:
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 677) - [internal]

CP_CONTEXT_UPDATE

```
CP_CONTEXT_UPDATE:
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 678) - [internal]

CP_SET_PROTECTED_MODE

```
CP_SET_PROTECTED_MODE:
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 679) - [internal] UNKN96

```
UNKN96:
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 680) - [internal] UNKN97

```
UNKN97:
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 681) - [internal] UNKN98

```
UNKN98:
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 682) - [internal]

CP_SET_MODE

```
CP_SET_MODE:
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 683) - [internal]

CP_SET_VISIBILITY_OVERRIDE

```
CP_SET_VISIBILITY_OVERRIDE:
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 684) - [internal]

CP_SET_MARKER

```
CP_SET_MARKER:
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 685) - [internal] UNKN103

```
UNKN103:
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 686) - [internal] UNKN104

```
UNKN104:
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 687) - [internal] UNKN105

```
UNKN105:
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 688) - [internal] UNKN106

```
UNKN106:
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 689) - [internal] UNKN107

```
UNKN107:
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 690) - [internal] UNKN108

```
UNKN108:
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 691) - [internal]

CP_REG_WRITE

```
CP_REG_WRITE:
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 692) - [internal] UNKN110

```
UNKN110:
CP_BOOTSTRAP_UCODE:
CP_WAIT_TWO_REGS:
CP_TEST_TWO_MEMS:
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 693) - [internal]

```
CP_BOOTSTRAP_UCODE
    CP_BOOTSTRAP_UCODE:
    CP_WAIT_TWO_REGS:
    CP_TEST_TWO_MEMS:
    CP_REG_TO_MEM_OFFSET_REG:
    CP_REG_TO_MEM_OFFSET_MEM:
    UNKN118:
    UNKN119:
    CP_REG_WR_NO_CTXT:
    UNKN121:
    UNKN122:
    UNKN123:
    UNKN124:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 694) - [internal]

```
CP_WAIT_TWO_REGS
    CP_WAIT_TWO_REGS:
    CP_TEST_TWO_MEMS:
    CP_REG_TO_MEM_OFFSET_REG:
    CP_REG_TO_MEM_OFFSET_MEM:
    UNKN118:
    UNKN119:
    CP_REG_WR_NO_CTXT:
    UNKN121:
    UNKN122:
    UNKN123:
    UNKN124:
    UNKN125:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 695) - [internal]

```
CP_TEST_TWO_MEMS
    CP_TEST_TWO_MEMS:
    CP_REG_TO_MEM_OFFSET_REG:
    CP_REG_TO_MEM_OFFSET_MEM:
    UNKN118:
    UNKN119:
    CP_REG_WR_NO_CTXT:
    UNKN121:
    UNKN122:
    UNKN123:
    UNKN124:
    UNKN125:
    UNKN126:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 696) - [internal]

CP_REG_TO_MEM_OFFSET_REG

```
CP_REG_TO_MEM_OFFSET_REG:
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 697) - [internal]

CP_REG_TO_MEM_OFFSET_MEM

```
CP_REG_TO_MEM_OFFSET_MEM:
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 698) - [internal] UNKN118

```
UNKN118:
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
waitin
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 699) - [internal] UNKN119

```
UNKN119:
CP_REG_WR_NO_CTXT:
UNKN121:
UNKN122:
UNKN123:
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
waitin
mov $01, $data
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 700) - [internal]

CP_REG_WR_NO_CTXT

CP_REG_WR_NO_CTXT:

UNKN121:

UNKN122:

UNKN123:

UNKN124:

UNKN125:

UNKN126:

UNKN127:

preempt:

waitin

mov \$01, \$data

// ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 701) - [internal] UNKN121

UNKN121:

UNKN122:

UNKN123:

UNKN124:

UNKN125:

UNKN126:

UNKN127:

preempt:

waitin

mov \$01, \$data

.align 32

// ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 702) - [internal] UNKN122

UNKN122:

UNKN123:

UNKN124:

UNKN125:

UNKN126:

UNKN127:

preempt:

waitin

mov \$01, \$data

.align 32

jumptbl:

// ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 703) - [internal] UNKN123

UNKN123:

UNKN124:

UNKN125:

UNKN126:

UNKN127:

preempt:

waitin

mov \$01, \$data

.align 32

jumptbl:

.jumptbl

// ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 704) - [internal] UNKN124

```
UNKN124:
UNKN125:
UNKN126:
UNKN127:
preempt:
waitin
mov $01, $data
```

```
.align 32
jumptbl:
.jumptbl
```

// ... (truncated, see source file for more)

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 705) - [internal] UNKN125

```
UNKN125:
UNKN126:
UNKN127:
preempt:
waitin
mov $01, $data
```

```
.align 32
jumptbl:
.jumptbl
```

```
; test junk data after jump table
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 706) - [internal] UNKN126

```
UNKN126:
UNKN127:
preempt:
waitin
mov $01, $data
```

```
.align 32
jumptbl:
.jumptbl
```

```
; test junk data after jump table
[0100beef]
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 707) - [internal] UNKN127

```
UNKN127:
preempt:
waitin
mov $01, $data
```

```
.align 32
jumptbl:
.jumptbl
```

```
; test junk data after jump table
[0100beef]
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 708) - [internal] preempt

```
preempt:
waitin
mov $01, $data
```

```
.align 32
jumptbl:
.jumptbl
```

```
; test junk data after jump table
[0100beef]
```

dirtycode/mesa/src/freedreno/tests/traces/qrisc_test_a7xx.asm (line 713) - [internal] jumptbl

```
jumptbl:
.jumptbl

; test junk data after jump table
[0100beef]
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gf100.asm (line 13) - [internal]

```
gf100_div_u32
gf100_div_u32:
    bfind u32 $r2 $r1
    xor b32 $r2 $r2 0x1f
    mov b32 $r3 0x1
    shl b32 $r2 $r3 clamp $r2
    cvt u32 $r1 neg u32 $r1
    mul $r3 u32 $r1 u32 $r2
    add $r2 (mul high u32 $r2 u32 $r3) $r2
    mul $r3 u32 $r1 u32 $r2
    add $r2 (mul high u32 $r2 u32 $r3) $r2
    mul $r3 u32 $r1 u32 $r2
    add $r2 (mul high u32 $r2 u32 $r3) $r2
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gf100.asm (line 47) - [internal]

```
gf100_div_s32
gf100_div_s32:
    set $p2 0x1 lt s32 $r0 0x0
    set $p3 0x1 lt s32 $r1 0x0 xor $p2
    cvt s32 $r0 abs s32 $r0
    cvt s32 $r1 abs s32 $r1
    bfind u32 $r2 $r1
    xor b32 $r2 $r2 0x1f
    mov b32 $r3 0x1
    shl b32 $r2 $r3 clamp $r2
    cvt u32 $r1 neg u32 $r1
    mul $r3 u32 $r1 u32 $r2
    add $r2 (mul high u32 $r2 u32 $r3) $r2
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gf100.asm (line 88) - [internal]

```
gf100_rcp_f64
gf100_rcp_f64:
    nop
    ret

// RSQ F64: Newton Raphson rsqrt(x): r_{i+1} = r_i * (1.5 - 0.5 * x * r_i * r_i)
//
// INPUT:  $r0d (x)
// OUTPUT: $r0d (rsqrt(x))
// CLOBBER: $r2 - $r7
// SIZE:   14 * 8 bytes
//
gf100_rsqr_f64:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gf100.asm (line 99) - [internal]

```
gf100_rsqr_f64
gf100_rsqr_f64:
    nop
    ret

.section #gf100_builtin_offsets
.b64 #gf100_div_u32
.b64 #gf100_div_s32
.b64 #gf100_rcp_f64
.b64 #gf100_rsqr_f64
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 13) - [internal]
gk104_div_u32

```
gk104_div_u32:
    sched 0x28 0x4 0x28 0x4 0x28 0x28 0x28
    bfind u32 $r2 $r1
    long xor b32 $r2 $r2 0x1f
    long mov b32 $r3 0x1
    shl b32 $r2 $r3 clamp $r2
    long cvt u32 $r1 neg u32 $r1
    long mul $r3 u32 $r1 u32 $r2
    add $r2 (mul high u32 $r2 u32 $r3) $r2
    sched 0x28 0x28 0x28 0x28 0x28 0x28 0x28
    mul $r3 u32 $r1 u32 $r2
    add $r2 (mul high u32 $r2 u32 $r3) $r2
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 51) - [internal]
gk104_div_s32

```
gk104_div_s32:
    set $p2 0x1 lt s32 $r0 0x0
    set $p3 0x1 lt s32 $r1 0x0 xor $p2
    sched 0x20 0x28 0x28 0x4 0x28 0x04 0x28
    long cvt s32 $r0 abs s32 $r0
    long cvt s32 $r1 abs s32 $r1
    bfind u32 $r2 $r1
    long xor b32 $r2 $r2 0x1f
    long mov b32 $r3 0x1
    shl b32 $r2 $r3 clamp $r2
    cvt u32 $r1 neg u32 $r1
    sched 0x28 0x28 0x28 0x28 0x28 0x28 0x28
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 558) - [internal]
gk104_rcp_f64

```
gk104_rcp_f64:
    // Step 1: classify input according to exponent and value, and calculate
    // result for 0/inf/nan. $r2 holds the exponent value, which starts at
    // bit 52 (bit 20 of the upper half) and is 11 bits in length
    ext u32 $r2 $r1 0xb14
    add b32 $r3 $r2 0xffffffff
    joinat #rcp_rejoin
    // We want to check whether the exponent is 0 or 0x7ff (i.e. NaN, inf,
    // denorm, or 0). Do this by subtracting 1 from the exponent, which will
    // mean that it's > 0x7fd in those cases when doing unsigned comparison
    set $p0 0x1 gt u32 $r3 0x7fd
    // $r3: 0 for norms, 0x36 for denorms, -1 for others
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 581) - [internal]
rcp_inf_or_denorm_or_zero

```
rcp_inf_or_denorm_or_zero:
    and b32 $r4 $r1 0x7ff00000
    // Other values with nonzero in exponent field should be inf
    set $p0 0x1 eq s32 $r4 0x0
    sched 0x2b 0x04 0x2f 0x2d 0x2b 0x2f 0x20
    $p0 bra #rcp_denorm_or_zero
    // +/-Inf -> +/-0
    xor b32 $r1 $r1 0x7ff00000
    join mov b32 $r0 0x0
rcp_denorm_or_zero:
    set $p0 0x1 gtu f64 abs $r0d 0x0
    $p0 bra #rcp_denorm
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 590) - [internal]

rcp_denorm_or_zero

```
rcp_denorm_or_zero:
    set $p0 0x1 gtu f64 abs $r0d 0x0
    $p0 bra #rcp_denorm
    // +/-0 -> +/-Inf
    join or b32 $r1 $r1 0x7ff00000
rcp_denorm:
    // non-0 denorms: multiply with 2^54 (the 0x36 in $r3), join with norms
    mul rn f64 $r0d $r0d 0x4350000000000000
    sched 0x2f 0x28 0x2b 0x28 0x28 0x04 0x28
    join mov b32 $r3 0x36
rcp_rejoin:
    // All numbers with -1 in $r3 have their result ready in $r0d, return them
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 595) - [internal]

rcp_denorm

```
rcp_denorm:
    // non-0 denorms: multiply with 2^54 (the 0x36 in $r3), join with norms
    mul rn f64 $r0d $r0d 0x4350000000000000
    sched 0x2f 0x28 0x2b 0x28 0x28 0x04 0x28
    join mov b32 $r3 0x36
rcp_rejoin:
    // All numbers with -1 in $r3 have their result ready in $r0d, return them
    // others need further calculation
    set $p0 0x1 lt s32 $r3 0x0
    $p0 bra #rcp_end
    // Step 2: Before the real calculation goes on, renormalize the values to
    // range [1, 2) by setting exponent field to 0x3ff (the exponent of 1)
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 600) - [internal]

rcp_rejoin

```
rcp_rejoin:
    // All numbers with -1 in $r3 have their result ready in $r0d, return them
    // others need further calculation
    set $p0 0x1 lt s32 $r3 0x0
    $p0 bra #rcp_end
    // Step 2: Before the real calculation goes on, renormalize the values to
    // range [1, 2) by setting exponent field to 0x3ff (the exponent of 1)
    // result in $r6d. The exponent will be recovered later.
    ext u32 $r2 $r1 0xb14
    and b32 $r7 $r1 0x800fffff
    add b32 $r7 $r7 0x3ff00000
    long mov b32 $r6 $r0
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 669) - [internal]

rcp_result_inf_or_denorm

```
rcp_result_inf_or_denorm:
    // New exponent >= 0x7ff means that result is inf
    set $p0 0x1 ge s32 $r3 0x7ff
    (not $p0) bra #rcp_result_denorm
    sched 0x20 0x25 0x28 0x2b 0x23 0x25 0x2f
    // Infinity
    and b32 $r1 $r1 0x80000000
    long mov b32 $r0 0x0
    add b32 $r1 $r1 0x7ff00000
    bra #rcp_end
rcp_result_denorm:
    // Denorm result comes from huge input. The greatest possible fp64, i.e.
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 679) - [internal]
rcp_result_denorm

```
rcp_result_denorm:
    // Denorm result comes from huge input. The greatest possible fp64, i.e.
    // 0x7fefffffffffffffff's rcp is 0x0004000000000000, 1/4 of the smallest
    // normal value. Other rcp result should be greater than that. If we
    // set the exponent field to 1, we can recover the result by multiplying
    // it with 1/2 or 1/4. 1/2 is used if the "exponent" $r3 is 0, otherwise
    // 1/4 ($r3 should be -1 then). This is quite tricky but greatly simplifies
    // the logic here.
    set $p0 0x1 ne u32 $r3 0x0
    and b32 $r1 $r1 0x800fffff
    // 0x3e800000: 1/4
    $p0 cvt f64 $r6d f32 0x3e800000
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 696) - [internal]
rcp_end

```
rcp_end:
    long ret

// RSQ F64: Newton Raphson rsqrt(x): r_{i+1} = r_i * (1.5 - 0.5 * x * r_i * r_i)
//
// INPUT:  $r0d (x)
// OUTPUT: $r0d (rsqrt(x))
// CLOBBER: $r2 - $r7
// SIZE:   14 * 8 bytes
//
gk104_rsqr_f64:
    // Before getting initial result rsqrt64h, two special cases should be
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 706) - [internal]
gk104_rsqr_f64

```
gk104_rsqr_f64:
    // Before getting initial result rsqrt64h, two special cases should be
    // handled first.
    // 1. NaN: set the highest bit in mantissa so it'll be surely recognized
    //    as NaN in rsqrt64h
    set $p0 0x1 gt u f64 abs $r0d 0x7ff0000000000000
    $p0 or b32 $r1 $r1 0x00080000
    and b32 $r2 $r1 0x7fffffff
    sched 0x27 0x20 0x28 0x2c 0x25 0x28 0x28
    // 2. denorms and small normal values: using their original value will
    //    lose precision either at rsqrt64h or the first step in newton-raphson
    //    steps below. Take 2 as a threshold in exponent field, and multiply
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 738) - [internal]
rsqr_norm

```
rsqr_norm:
    // For others, do 4 Newton-Raphson steps with the formula:
    //    RSQ_{n + 1} = RSQ_{n} * (1.5 - 0.5 * x * RSQ_{n} * RSQ_{n})
    // In the code below, each step is written as:
    //    tmp1 = 0.5 * x * RSQ_{n}
    //    tmp2 = -RSQ_{n} * tmp1 + 0.5
    //    RSQ_{n + 1} = RSQ_{n} * tmp2 + RSQ_{n}
    long mov b32 $r4 0x0
    sched 0x2f 0x29 0x29 0x29 0x29 0x29 0x29
    // 0x3f000000: 1/2
    cvt f64 $r8d f32 0x3f000000
    mul rn f64 $r2d $r0d $r8d
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 799) - [internal]
spill_cfstack

```
spill_cfstack:
preret #end_exit
sub b32 $r3 $c $r3 0x1
lg $c bra #spill_cfstack
// retrieve pointer to trap info
mov b32 $r0 c0[0x1900]
mov b32 $r1 c0[0x1904]
// we only let a single faulting thread store its state
mov b32 $r3 0x1
exch b32 $r3 g[$r0d] $r3
joinat #end_exit
set $p0 0x1 eq u32 $r3 0x1
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 854) - [internal]
shared_loop

```
shared_loop:
long ld b32 $r2 s[$r3]
long st b32 wb g[$r0d+0x140] $r2
sub b32 $r0 $c $r0 0x4
sub b32 $r1 $r1 0x0 $c
sub b32 $r3 $c $r3 0x4
lg $c bra #shared_loop
shared_done:
// search the stack for trap entry to retrieve PC
mov b32 $r0 c0[0x1908]
mov b32 $r1 c0[0x190c]
membar sys
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 861) - [internal]
shared_done

```
shared_done:
// search the stack for trap entry to retrieve PC
mov b32 $r0 c0[0x1908]
mov b32 $r1 c0[0x190c]
membar sys
// invalidate caches so we can read stack entries via g[]
cctl ivall 0 l[0]
cctl ivall 0 g[$r0d]
// get offsets
mov b32 $r2 $physid
ext u32 $r3 $r2 0x0814 // MP id
ext u32 $r2 $r2 0x0608 // warp id
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 878) - [internal]
search_cstack

```
search_cstack:
mov b32 $r3 c0[0x1918] // cstack size
ld u8 $r2 cv g[$r0d+0x8]
set $p0 0x1 eq u32 $r2 0xa
$p0 bra #entry_found
add b32 $r0 $c $r0 0x10
add b32 $r1 $r1 0x0 $c
sub b32 $r3 $c $r3 0x10
lg $c bra #search_cstack
bra #end_exit
entry_found:
// load PC (may be unaligned and spread out)
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 888) - [internal]
entry_found

```
entry_found:
// load PC (may be unaligned and spread out)
ld b32 $r2 cv g[$r0d]
mov b32 $r0 c0[0x1900]
mov b32 $r1 c0[0x1904]
st b32 wb g[$r0d+0x4] $r2
join nop
// invalidate caches and exit
end_exit:
cctl ivall 0 g[0]
bpt pause 0x0
rtt terminate
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 896) - [internal]
end_exit

```
end_exit:
cctl ivall 0 g[0]
bpt pause 0x0
rtt terminate
end_cont:
bpt pause 0x0
mov $flags $r2 mask 0xffff
ld b128 $r0q cs l[0x00]
rtt

.section #gk104_built_in_offsets
.b64 #gk104_div_u32
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk104.asm (line 900) - [internal]
end_cont

```
end_cont:
bpt pause 0x0
mov $flags $r2 mask 0xffff
ld b128 $r0q cs l[0x00]
rtt

.section #gk104_built_in_offsets
.b64 #gk104_div_u32
.b64 #gk104_div_s32
.b64 #gk104_rcp_f64
.b64 #gk104_rsqr_f64
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 13) - [internal]
gk110_div_u32

```
gk110_div_u32:
sched 0x28 0x04 0x28 0x04 0x28 0x28 0x28
bfind u32 $r2 $r1
xor b32 $r2 $r2 0x1f
mov b32 $r3 0x1
shl b32 $r2 $r3 clamp $r2
cvt u32 $r1 neg u32 $r1
mul $r3 u32 $r1 u32 $r2
add $r2 (mul high u32 $r2 u32 $r3) $r2
sched 0x28 0x28 0x28 0x28 0x28 0x28 0x28
mul $r3 u32 $r1 u32 $r2
add $r2 (mul high u32 $r2 u32 $r3) $r2
// ... (truncated, see source file for more)
```


dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 51) - [internal]
gk110_div_s32

```
gk110_div_s32:
    set $p2 0x1 lt s32 $r0 0x0
    set $p3 0x1 lt s32 $r1 0x0 xor $p2
    sched 0x20 0x28 0x28 0x04 0x28 0x04 0x28
    cvt s32 $r0 abs s32 $r0
    cvt s32 $r1 abs s32 $r1
    bfind u32 $r2 $r1
    xor b32 $r2 $r2 0x1f
    mov b32 $r3 0x1
    shl b32 $r2 $r3 clamp $r2
    cvt u32 $r1 neg u32 $r1
    sched 0x28 0x28 0x28 0x28 0x28 0x28 0x28
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 100) - [internal]
gk110_rcp_f64

```
gk110_rcp_f64:
    // Step 1: classify input according to exponent and value, and calculate
    // result for 0/inf/nan. $r2 holds the exponent value, which starts at
    // bit 52 (bit 20 of the upper half) and is 11 bits in length
    ext u32 $r2 $r1 0xb14
    add b32 $r3 $r2 0xffffffff
    joinat #rcp_rejoin
    // We want to check whether the exponent is 0 or 0x7ff (i.e. NaN, inf,
    // denorm, or 0). Do this by subtracting 1 from the exponent, which will
    // mean that it's > 0x7fd in those cases when doing unsigned comparison
    set b32 $p0 0x1 gt u32 $r3 0x7fd
    // $r3: 0 for norms, 0x36 for denorms, -1 for others
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 123) - [internal]
rcp_inf_or_denorm_or_zero

```
rcp_inf_or_denorm_or_zero:
    and b32 $r4 $r1 0x7ff00000
    // Other values with nonzero in exponent field should be inf
    set b32 $p0 0x1 eq s32 $r4 0x0
    sched 0x2b 0x04 0x2f 0x2d 0x2b 0x2f 0x20
    $p0 bra #rcp_denorm_or_zero
    // +/-Inf -> +/-0
    xor b32 $r1 $r1 0x7ff00000
    join mov b32 $r0 0x0
rcp_denorm_or_zero:
    set $p0 0x1 gtu f64 abs $r0d 0x0
    $p0 bra #rcp_denorm
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 132) - [internal]
rcp_denorm_or_zero

```
rcp_denorm_or_zero:
    set $p0 0x1 gtu f64 abs $r0d 0x0
    $p0 bra #rcp_denorm
    // +/-0 -> +/-Inf
    join or b32 $r1 $r1 0x7ff00000
rcp_denorm:
    // non-0 denorms: multiply with 2^54 (the 0x36 in $r3), join with norms
    mul rn f64 $r0d $r0d 0x4350000000000000
    sched 0x2f 0x28 0x2b 0x28 0x28 0x04 0x28
    join mov b32 $r3 0x36
rcp_rejoin:
    // All numbers with -1 in $r3 have their result ready in $r0d, return them
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 137) - [internal]

rcp_denorm

```
rcp_denorm:
    // non-0 denorms: multiply with 2^54 (the 0x36 in $r3), join with norms
    mul rn f64 $r0d $r0d 0x4350000000000000
    sched 0x2f 0x28 0x2b 0x28 0x28 0x04 0x28
    join mov b32 $r3 0x36
rcp_rejoin:
    // All numbers with -1 in $r3 have their result ready in $r0d, return them
    // others need further calculation
    set b32 $p0 0x1 lt s32 $r3 0x0
    $p0 bra #rcp_end
    // Step 2: Before the real calculation goes on, renormalize the values to
    // range [1, 2) by setting exponent field to 0x3ff (the exponent of 1)
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 142) - [internal]

rcp_rejoin

```
rcp_rejoin:
    // All numbers with -1 in $r3 have their result ready in $r0d, return them
    // others need further calculation
    set b32 $p0 0x1 lt s32 $r3 0x0
    $p0 bra #rcp_end
    // Step 2: Before the real calculation goes on, renormalize the values to
    // range [1, 2) by setting exponent field to 0x3ff (the exponent of 1)
    // result in $r6d. The exponent will be recovered later.
    ext u32 $r2 $r1 0xb14
    and b32 $r7 $r1 0x800fffff
    add b32 $r7 $r7 0x3ff00000
    mov b32 $r6 $r0
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 211) - [internal]

rcp_result_inf_or_denorm

```
rcp_result_inf_or_denorm:
    // New exponent >= 0x7ff means that result is inf
    set b32 $p0 0x1 ge s32 $r3 0x7ff
    (not $p0) bra #rcp_result_denorm
    sched 0x20 0x25 0x28 0x2b 0x23 0x25 0x2f
    // Infinity
    and b32 $r1 $r1 0x80000000
    mov b32 $r0 0x0
    add b32 $r1 $r1 0x7ff00000
    bra #rcp_end
rcp_result_denorm:
    // Denorm result comes from huge input. The greatest possible fp64, i.e.
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 221) - [internal]

rcp_result_denorm

```
rcp_result_denorm:
    // Denorm result comes from huge input. The greatest possible fp64, i.e.
    // 0x7fefffffffffffffff's rcp is 0x0004000000000000, 1/4 of the smallest
    // normal value. Other rcp result should be greater than that. If we
    // set the exponent field to 1, we can recover the result by multiplying
    // it with 1/2 or 1/4. 1/2 is used if the "exponent" $r3 is 0, otherwise
    // 1/4 ($r3 should be -1 then). This is quite tricky but greatly simplifies
    // the logic here.
    set b32 $p0 0x1 ne u32 $r3 0x0
    and b32 $r1 $r1 0x800fffff
    // 0x3e800000: 1/4
    $p0 cvt f64 $r6d f32 0x3e800000
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 238) - [internal]
rcp_end

```
rcp_end:
    ret

// RSQ F64
//
// INPUT:  $r0d
// OUTPUT: $r0d
// CLOBBER: $r2 - $r9, $p0 - $p1
//
gk110_rsqr_f64:
    // Before getting initial result rsqrt64h, two special cases should be
    // handled first.
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 247) - [internal]
gk110_rsqr_f64

```
gk110_rsqr_f64:
    // Before getting initial result rsqrt64h, two special cases should be
    // handled first.
    // 1. NaN: set the highest bit in mantissa so it'll be surely recognized
    //    as NaN in rsqrt64h
    set $p0 0x1 gtu f64 abs $r0d 0x7ff0000000000000
    $p0 or b32 $r1 $r1 0x00080000
    and b32 $r2 $r1 0x7fffffff
    sched 0x27 0x20 0x28 0x2c 0x25 0x28 0x28
    // 2. denorms and small normal values: using their original value will
    //    lose precision either at rsqrt64h or the first step in newton-raphson
    //    steps below. Take 2 as a threshold in exponent field, and multiply
    //    ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gk110.asm (line 279) - [internal]
rsqr_norm

```
rsqr_norm:
    // For others, do 4 Newton-Raphson steps with the formula:
    //    RSQ_{n + 1} = RSQ_{n} * (1.5 - 0.5 * x * RSQ_{n} * RSQ_{n})
    // In the code below, each step is written as:
    //    tmp1 = 0.5 * x * RSQ_{n}
    //    tmp2 = -RSQ_{n} * tmp1 + 0.5
    //    RSQ_{n + 1} = RSQ_{n} * tmp2 + RSQ_{n}
    mov b32 $r4 0x0
    sched 0x2f 0x29 0x29 0x29 0x29 0x29 0x29
    // 0x3f000000: 1/2
    cvt f64 $r8d f32 0x3f000000
    mul rn f64 $r2d $r0d $r8d
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 13) - [internal]
gm107_div_u32

```
gm107_div_u32:
    sched (st 0xd wr 0x0 wt 0x3f) (st 0x1 wt 0x1) (st 0x6)
    flo u32 $r2 $r1
    lop xor 1 $r2 $r2 0x1f
    mov $r3 0x1 0xf
    sched (st 0x1) (st 0xf wr 0x0) (st 0x6 wr 0x0 wt 0x1)
    shl $r2 $r3 $r2
    i2i u32 u32 $r1 neg $r1
    imul u32 u32 $r3 $r1 $r2
    sched (st 0x6 wr 0x0 wt 0x1) (st 0x6 wr 0x0 wt 0x1) (st 0x6 wr 0x0 wt 0x1)
    imad u32 u32 hi $r2 $r2 $r3 $r2
    imul u32 u32 $r3 $r1 $r2
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 57) - [internal]
gm107_div_s32

```
gm107_div_s32:
    sched (st 0xd wr 0x3f) (st 0x1) (st 0x1 wr 0x0)
    isetp lt and $p2 0x1 $r0 0 1
    isetp lt xor $p3 1 $r1 0 $p2
    i2i s32 s32 $r0 abs $r0
    sched (st 0xf wr 0x1) (st 0xd wr 0x1 wt 0x2) (st 0x1 wt 0x2)
    i2i s32 s32 $r1 abs $r1
    flo u32 $r2 $r1
    lop xor 1 $r2 $r2 0x1f
    sched (st 0x6) (st 0x1) (st 0xf wr 0x1)
    mov $r3 0x1 0xf
    shl $r2 $r3 $r2
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 113) - [internal]
gm107_rcp_f64

```
gm107_rcp_f64:
    // Step 1: classify input according to exponent and value, and calculate
    // result for 0/inf/nan. $r2 holds the exponent value, which starts at
    // bit 52 (bit 20 of the upper half) and is 11 bits in length
    sched (st 0x0) (st 0x0) (st 0x0)
    bfe u32 $r2 $r1 0xb14
    iadd32i $r3 $r2 -1
    ssy #rcp_rejoin
    // We want to check whether the exponent is 0 or 0x7ff (i.e. NaN, inf,
    // denorm, or 0). Do this by subtracting 1 from the exponent, which will
    // mean that it's > 0x7fd in those cases when doing unsigned comparison
    sched (st 0x0) (st 0x0) (st 0x0)
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 140) - [internal]
rcp_inf_or_denorm_or_zero

```
rcp_inf_or_denorm_or_zero:
    lop32i and $r4 $r1 0x7ff00000
    sched (st 0x0) (st 0x0) (st 0x0)
    // Other values with nonzero in exponent field should be inf
    isetp eq and $p0 1 $r4 0x0 1
    $p0 bra #rcp_denorm_or_zero
    // +/-Inf -> +/-0
    lop32i xor $r1 $r1 0x7ff00000
    sched (st 0x0) (st 0x0) (st 0x0)
    mov $r0 0x0 0xf
    sync
rcp_denorm_or_zero:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 151) - [internal]
rcp_denorm_or_zero

```
rcp_denorm_or_zero:
    dsetp gtu and $p0 1 abs $r0 0x0 1
    sched (st 0x0) (st 0x0) (st 0x0)
    $p0 bra #rcp_denorm
    // +/-0 -> +/-Inf
    lop32i or $r1 $r1 0x7ff00000
    sync
rcp_denorm:
    // non-0 denorms: multiply with 2^54 (the 0x36 in $r3), join with norms
    sched (st 0x0) (st 0x0) (st 0x0)
    dmul $r0 $r0 0x4350000000000000
    mov $r3 0x36 0xf
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 158) - [internal]

rcp_denorm

```
rcp_denorm:
    // non-0 denorms: multiply with 2^54 (the 0x36 in $r3), join with norms
    sched (st 0x0) (st 0x0) (st 0x0)
    dmul $r0 $r0 0x4350000000000000
    mov $r3 0x36 0xf
    sync
rcp_rejoin:
    // All numbers with -1 in $r3 have their result ready in $r0d, return them
    // others need further calculation
    sched (st 0x0) (st 0x0) (st 0x0)
    isetp lt and $p0 1 $r3 0x0 1
    $p0 bra #rcp_end
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 164) - [internal]

rcp_rejoin

```
rcp_rejoin:
    // All numbers with -1 in $r3 have their result ready in $r0d, return them
    // others need further calculation
    sched (st 0x0) (st 0x0) (st 0x0)
    isetp lt and $p0 1 $r3 0x0 1
    $p0 bra #rcp_end
    // Step 2: Before the real calculation goes on, renormalize the values to
    // range [1, 2) by setting exponent field to 0x3ff (the exponent of 1)
    // result in $r6d. The exponent will be recovered later.
    bfe u32 $r2 $r1 0xb14
    sched (st 0x0) (st 0x0) (st 0x0)
    lop32i and $r7 $r1 0x800fffff
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 240) - [internal]

rcp_result_inf_or_denorm

```
rcp_result_inf_or_denorm:
    // New exponent >= 0x7ff means that result is inf
    isetp ge and $p0 1 $r3 0x7ff 1
    sched (st 0x0) (st 0x0) (st 0x0)
    not $p0 bra #rcp_result_denorm
    // Infinity
    lop32i and $r1 $r1 0x80000000
    mov $r0 0x0 0xf
    sched (st 0x0) (st 0x0) (st 0x0)
    iadd32i $r1 $r1 0x7ff00000
    bra #rcp_end
rcp_result_denorm:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 251) - [internal]

rcp_result_denorm

```
rcp_result_denorm:
    // Denorm result comes from huge input. The greatest possible fp64, i.e.
    // 0x7fefffffffffffffff's rcp is 0x0004000000000000, 1/4 of the smallest
    // normal value. Other rcp result should be greater than that. If we
    // set the exponent field to 1, we can recover the result by multiplying
    // it with 1/2 or 1/4. 1/2 is used if the "exponent" $r3 is 0, otherwise
    // 1/4 ($r3 should be -1 then). This is quite tricky but greatly simplifies
    // the logic here.
    isetp ne u32 and $p0 1 $r3 0x0 1
    sched (st 0x0) (st 0x0) (st 0x0)
    lop32i and $r1 $r1 0x800fffff
    // 0x3e800000: 1/4
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 269) - [internal]
rcp_end

```
rcp_end:
    ret

// RSQ F64
//
// INPUT:  $r0d
// OUTPUT: $r0d
// CLOBBER: $r2 - $r9, $p0 - $p1
//
gm107_rsq_f64:
    // Before getting initial result rsqrt64h, two special cases should be
    // handled first.
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 278) - [internal]
gm107_rsq_f64

```
gm107_rsq_f64:
    // Before getting initial result rsqrt64h, two special cases should be
    // handled first.
    // 1. NaN: set the highest bit in mantissa so it'll be surely recognized
    //    as NaN in rsqrt64h
    sched (st 0xd wr 0x0 wt 0x3f) (st 0xd wr 0x1) (st 0xd)
    dsetp gtu and $p0 1 abs $r0 0x7ff0000000000000 1
    $p0 lop32i or $r1 $r1 0x00080000
    lop32i and $r2 $r1 0x7fffffff
    // 2. denorms and small normal values: using their original value will
    //    lose precision either at rsqrt64h or the first step in newton-raphson
    //    steps below. Take 2 as a threshold in exponent field, and multiply
    //    ... (truncated, see source file for more)
```

dirtycode/mesa/src/gallium/drivers/nouveau/codegen/lib/gm107.asm (line 316) - [internal]
rsq_norm

```
rsq_norm:
    // For others, do 4 Newton-Raphson steps with the formula:
    //    RSQ_{n+1} = RSQ_{n} * (1.5 - 0.5 * x * RSQ_{n} * RSQ_{n})
    // In the code below, each step is written as:
    //    tmp1 = 0.5 * x * RSQ_{n}
    //    tmp2 = -RSQ_{n} * tmp1 + 0.5
    //    RSQ_{n+1} = RSQ_{n} * tmp2 + RSQ_{n}
    sched (st 0xd) (st 0xd wr 0x1) (st 0xd wr 0x1 rd 0x0 wt 0x3)
    mov $r4 0x0 0xf
    // 0x3f000000: 1/2
    f2f f32 f64 $r8 0x3f000000
    dmul $r2 $r0 $r8
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/break.asm (line 2) - [internal] LABEL1

```
LABEL1:
break(8)      JIP: LABEL2    UIP: LABEL2          { align1 1Q };
break(16)     JIP: LABEL2    UIP: LABEL2          { align1 1H };
LABEL2:
(+f0.0) break(8) JIP: LABEL3    UIP: LABEL3      { align1 1Q };
(+f0.0) break(16) JIP: LABEL3    UIP: LABEL3      { align1 1H };
(+f0.0.x) break(8) JIP: LABEL3    UIP: LABEL3      { align16 1Q };
LABEL3:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/break.asm (line 5) - [internal] LABEL2

```
LABEL2:
(+f0.0) break(8) JIP: LABEL3    UIP: LABEL3      { align1 1Q };
(+f0.0) break(16) JIP: LABEL3    UIP: LABEL3      { align1 1H };
(+f0.0.x) break(8) JIP: LABEL3    UIP: LABEL3      { align16 1Q };
LABEL3:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/break.asm (line 9) - [internal] LABEL3

```
LABEL3:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/cont.asm (line 2) - [internal] LABEL0

```
LABEL0:
cont(16)      JIP: LABEL1      UIP: LABEL2      { align1 1H };
LABEL1:
cont(8)       JIP: LABEL2      UIP: LABEL2      { align16 1Q };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/cont.asm (line 4) - [internal] LABEL1

```
LABEL1:
cont(8)       JIP: LABEL2      UIP: LABEL2      { align16 1Q };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/cont.asm (line 6) - [internal] LABEL2

```
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/else.asm (line 4) - [internal] LABEL0

```
LABEL0:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/endif.asm (line 2) - [internal] LABEL1

```
LABEL1:
endif(8)      JIP: LABEL2      { align1 1Q };
LABEL2:
endif(16)     JIP: LABEL3      { align1 1H };
LABEL3:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/endif.asm (line 4) - [internal] LABEL2

```
LABEL2:
endif(16)     JIP: LABEL3      { align1 1H };
LABEL3:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/endif.asm (line 6) - [internal] LABEL3

```
LABEL3:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/halt.asm (line 3) - [internal] LABEL1

```
LABEL1:
(-f0.1.any4h) halt(16) JIP: LABEL0      UIP: LABEL0      { align1 1H };
halt(16)      JIP: LABEL0      UIP: LABEL0      { align1 1H };
LABEL0:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/halt.asm (line 6) - [internal] LABEL0

```
LABEL0:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/if.asm (line 2) - [internal] LABEL0

```
LABEL0:
(+f0.0) if(8)  JIP: LABEL1      { align1 1Q };
(+f0.0) if(16) JIP: LABEL1      { align1 1H };
(+f0.0.x) if(8) JIP: LABEL1      { align16 1Q };
(-f0.0) if(8)  JIP: LABEL1      { align1 1Q };
(-f0.0) if(16) JIP: LABEL1      { align1 1H };
LABEL1:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/if.asm (line 8) - [internal] LABEL1

```
LABEL1:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen6/while.asm (line 1) - [internal] LABEL0

```
LABEL0:
while(8)      JIP: LABEL0      { align16 1Q };
while(8)      JIP: LABEL0      { align1 1Q };
while(16)     JIP: LABEL0      { align1 1H };
(-f0.0) while(8) JIP: LABEL0      { align1 1Q };
(-f0.0) while(16) JIP: LABEL0      { align1 1H };
(-f0.0.x) while(8) JIP: LABEL0      { align16 1Q };
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/break.asm (line 4) - [internal] LABEL0

```
LABEL0:
(+f0.0) break(8) JIP: LABEL1      UIP: LABEL1      { align1 1Q };
(+f0.0) break(16) JIP: LABEL1     UIP: LABEL1     { align1 1H };
(+f0.0.x) break(8) JIP: LABEL1    UIP: LABEL1     { align16 1Q };
LABEL1:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/break.asm (line 8) - [internal] LABEL1

LABEL1:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/cont.asm (line 2) - [internal] LABEL0

```
LABEL0:
cont(16)      JIP: LABEL1      UIP: LABEL2      { align1 1H };
LABEL1:
cont(8)       JIP: LABEL2      UIP: LABEL2      { align16 1Q };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/cont.asm (line 4) - [internal] LABEL1

```
LABEL1:
cont(8)       JIP: LABEL2      UIP: LABEL2      { align16 1Q };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/cont.asm (line 6) - [internal] LABEL2

LABEL2:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/else.asm (line 4) - [internal] LABEL0

LABEL0:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/endif.asm (line 2) - [internal] LABEL0

```
LABEL0:
endif(16)     JIP: LABEL1      { align1 1H };
endif(8)      JIP: LABEL1      { align16 1Q };
LABEL1:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/endif.asm (line 5) - [internal] LABEL1

LABEL1:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/halt.asm (line 3) - [internal] LABEL1

```
LABEL1:
(-f0.1.any4h) halt(16) JIP: LABEL0    UIP: LABEL0    { align1 1H };
halt(16)      JIP: LABEL0    UIP: LABEL0    { align1 1H };
LABEL0:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/halt.asm (line 6) - [internal] LABEL0

LABEL0:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/if.asm (line 2) - [internal] LABEL0

```
LABEL0:
(-f0.0) if(16) JIP: LABEL2    UIP: LABEL1    { align1 1H };
(+f0.0.x) if(8) JIP: LABEL2    UIP: LABEL1    { align16 1Q };
LABEL1:
(+f0.0) if(8)  JIP: LABEL2    UIP: LABEL2    { align16 1Q };
(+f0.0) if(8)  JIP: LABEL2    UIP: LABEL2    { align1 1Q };
(+f0.0) if(16) JIP: LABEL2    UIP: LABEL2    { align1 1H };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/if.asm (line 5) - [internal] LABEL1

```
LABEL1:
(+f0.0) if(8)  JIP: LABEL2    UIP: LABEL2    { align16 1Q };
(+f0.0) if(8)  JIP: LABEL2    UIP: LABEL2    { align1 1Q };
(+f0.0) if(16) JIP: LABEL2    UIP: LABEL2    { align1 1H };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/if.asm (line 9) - [internal] LABEL2

LABEL2:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7.5/while.asm (line 1) - [internal] LABEL0

```
LABEL0:
while(8)      JIP: LABEL0      { align1 1Q };
while(16)     JIP: LABEL0      { align1 1H };
while(8)      JIP: LABEL0      { align16 1Q };
(-f0.0) while(8) JIP: LABEL0   { align1 1Q };
(-f0.0) while(16) JIP: LABEL0  { align1 1H };
(-f0.0.x) while(8) JIP: LABEL0 { align16 1Q };
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/break.asm (line 2) - [internal] LABEL0

```
LABEL0:
break(16)      JIP: LABEL1      UIP: LABEL2      { align1 1H };
(+f0.0.x) break(8) JIP: LABEL1  UIP: LABEL2      { align16 1Q };
LABEL1:
(+f0.0) break(8) JIP: LABEL2      UIP: LABEL2      { align1 1Q };
(+f0.0) break(16) JIP: LABEL2     UIP: LABEL2      { align1 1H };
break(8)       JIP: LABEL2      UIP: LABEL2      { align16 1Q };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/break.asm (line 5) - [internal] LABEL1

```
LABEL1:
(+f0.0) break(8) JIP: LABEL2      UIP: LABEL2      { align1 1Q };
(+f0.0) break(16) JIP: LABEL2     UIP: LABEL2      { align1 1H };
break(8)       JIP: LABEL2      UIP: LABEL2      { align16 1Q };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/break.asm (line 9) - [internal] LABEL2

LABEL2:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/else.asm (line 4) - [internal] LABEL0

LABEL0:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/endif.asm (line 2) - [internal] LABEL1

```
LABEL1:
endif(8)      JIP: LABEL2      { align1 1Q };
LABEL2:
endif(16)     JIP: LABEL3      { align1 1H };
LABEL3:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/endif.asm (line 4) - [internal] LABEL2

```
LABEL2:
endif(16)     JIP: LABEL3      { align1 1H };
LABEL3:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/endif.asm (line 6) - [internal] LABEL3

LABEL3:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/halt.asm (line 3) - [internal] LABEL1

```
LABEL1:
(-f0.1.any4h) halt(16) JIP: LABEL0      UIP: LABEL0      { align1 1H };
halt(16)       JIP: LABEL0      UIP: LABEL0      { align1 1H };
LABEL0:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/halt.asm (line 6) - [internal] LABEL0

LABEL0:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/if.asm (line 2) - [internal] LABEL0

```
LABEL0:
(+f0.0) if(8)   JIP: LABEL2      UIP: LABEL1      { align16 1Q };
(+f0.0) if(8)   JIP: LABEL2      UIP: LABEL1      { align1 1Q };
LABEL1:
(+f0.0) if(16)  JIP: LABEL2      UIP: LABEL2      { align1 1H };
(-f0.0) if(8)   JIP: LABEL2      UIP: LABEL2      { align1 1Q };
(-f0.0) if(16)  JIP: LABEL2      UIP: LABEL2      { align1 1H };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/if.asm (line 5) - [internal] LABEL1

```
LABEL1:
(+f0.0) if(16) JIP: LABEL2      UIP: LABEL2      { align1 1H };
(-f0.0) if(8)  JIP: LABEL2      UIP: LABEL2      { align1 1Q };
(-f0.0) if(16) JIP: LABEL2      UIP: LABEL2      { align1 1H };
LABEL2:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/if.asm (line 9) - [internal] LABEL2

LABEL2:

dirtycode/mesa/src/intel/compiler/elk/tests/gen7/while.asm (line 1) - [internal] LABEL0

```
LABEL0:
while(8)      JIP: LABEL0      { align1 1Q };
while(16)     JIP: LABEL0      { align1 1H };
while(8)      JIP: LABEL0      { align16 1Q };
(-f0.0) while(8) JIP: LABEL0    { align1 1Q };
(-f0.0) while(16) JIP: LABEL0   { align1 1H };
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/break.asm (line 3) - [internal] LABEL0

```
LABEL0:
(+f0.0) break(8) JIP: LABEL1     UIP: LABEL1     { align1 1Q };
(+f0.0) break(16) JIP: LABEL1    UIP: LABEL1    { align1 1H };
LABEL1:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/break.asm (line 6) - [internal] LABEL1

LABEL1:

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/cont.asm (line 2) - [internal] LABEL0

```
LABEL0:
cont(16)      JIP: LABEL1      UIP: LABEL1      { align1 1H };
LABEL1:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/cont.asm (line 4) - [internal] LABEL1

LABEL1:

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/else.asm (line 4) - [internal] LABEL0

LABEL0:

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/endif.asm (line 4) - [internal] LABEL0

LABEL0:

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/halt.asm (line 3) - [internal] LABEL1

```
LABEL1:
(-f0.1.any4h) halt(16) JIP: LABEL0    UIP: LABEL0    { align1 1H };
halt(16)      JIP: LABEL0    UIP: LABEL0    { align1 1H };
LABEL0:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/halt.asm (line 6) - [internal] LABEL0

LABEL0:

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/if.asm (line 3) - [internal] LABEL0

```
LABEL0:
(-f0.0) if(16) JIP: LABEL1      UIP: LABEL1      { align1 1H };
(+f0.0) if(16) JIP: LABEL1      UIP: LABEL1      { align1 1H };
(+f0.0) if(32) JIP: LABEL1      UIP: LABEL1      { align1 };
LABEL1:
```

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/if.asm (line 7) - [internal] LABEL1

LABEL1:

dirtycode/mesa/src/intel/compiler/elk/tests/gen8/while.asm (line 1) - [internal] LABEL0

```
LABEL0:
while(8)      JIP: LABEL0      { align1 1Q };
while(16)     JIP: LABEL0      { align1 1H };
(-f0.0) while(8) JIP: LABEL0    { align1 1Q };
(-f0.0) while(16) JIP: LABEL0   { align1 1H };
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 21) - [public]

```
_blake3_hash_many_avx2
    .global _blake3_hash_many_avx2
    .global blake3_hash_many_avx2
#ifdef __APPLE__
    .text
#else
    .section .text
#endif
    .p2align 6
_blake3_hash_many_avx2:
blake3_hash_many_avx2:
    _CET_ENDBR
    push    r15
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 22) - [public]

```
blake3_hash_many_avx2
    .global blake3_hash_many_avx2
#ifdef __APPLE__
    .text
#else
    .section .text
#endif
    .p2align 6
_blake3_hash_many_avx2:
blake3_hash_many_avx2:
    _CET_ENDBR
    push    r15
    push    r14
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 29) - [internal]

```
_blake3_hash_many_avx2
_blake3_hash_many_avx2:
blake3_hash_many_avx2:
    _CET_ENDBR
    push    r15
    push    r14
    push    r13
    push    r12
    push    rbx
    push    rbp
    mov     rbp, rsp
    sub     rsp, 680
    and     rsp, 0xFFFFFFFFFFFFFFC0
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 30) - [internal]

```
blake3_hash_many_avx2
    blake3_hash_many_avx2:
        _CET_ENDBR
        push    r15
        push    r14
        push    r13
        push    r12
        push    rbx
        push    rbp
        mov     rbp, rsp
        sub     rsp, 680
        and     rsp, 0xFFFFFFFFFFFFFFC0
        neg     r9d
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1792) - [internal] ADD0

```
ADD0:
    .long 0, 1, 2, 3, 4, 5, 6, 7
ADD1:
    .long 8, 8, 8, 8, 8, 8, 8, 8
BLAKE3_IV_0:
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1794) - [internal] ADD1

```
ADD1:
    .long 8, 8, 8, 8, 8, 8, 8, 8
BLAKE3_IV_0:
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1796) - [internal] BLAKE3_IV_0

```
BLAKE3_IV_0:
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1799) - [internal] BLAKE3_IV_1

```
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1802) - [internal] BLAKE3_IV_2

```
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1805) - [internal] BLAKE3_IV_3

```
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1808) - [internal]

BLAKE3_BLOCK_LEN

```
BLAKE3_BLOCK_LEN:
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1811) - [internal] ROT16

```
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1813) - [internal] ROT8

```
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A
```

```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1815) - [internal] CMP_MSB_MASK
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 1818) - [internal] BLAKE3_IV
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 2) - [public]
_blake3_hash_many_avx2
    .global _blake3_hash_many_avx2
    .global blake3_hash_many_avx2
    .section .text
    .p2align 6
_blake3_hash_many_avx2:
blake3_hash_many_avx2:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    // ... (truncated, see source file for more)

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 3) - [public]
blake3_hash_many_avx2
    .global blake3_hash_many_avx2
    .section .text
    .p2align 6
_blake3_hash_many_avx2:
blake3_hash_many_avx2:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    push    rbx
    // ... (truncated, see source file for more)

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 6) - [internal]
_blake3_hash_many_avx2
    _blake3_hash_many_avx2:
blake3_hash_many_avx2:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    push    rbx
    push    rbp
    mov     rbp, rsp
    sub     rsp, 880
    // ... (truncated, see source file for more)

```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 7) - [internal]
blake3_hash_many_avx2

```
blake3_hash_many_avx2:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    push    rbx
    push    rbp
    mov     rbp, rsp
    sub     rsp, 880
    and     rsp, 0xFFFFFFFFFFFFC0
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1789) - [internal] ADD0

```
ADD0:
    .long   0, 1, 2, 3, 4, 5, 6, 7
ADD1:
    .long   8, 8, 8, 8, 8, 8, 8, 8
BLAKE3_IV_0:
    .long   0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
    .long   0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long   0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    .long   0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long   0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1791) - [internal] ADD1

```
ADD1:
    .long   8, 8, 8, 8, 8, 8, 8, 8
BLAKE3_IV_0:
    .long   0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
    .long   0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long   0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    .long   0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long   0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    .long   0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1793) - [internal]

BLAKE3_IV_0

```
BLAKE3_IV_0:
    .long   0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
    .long   0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long   0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    .long   0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long   0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    .long   0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long   0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    .long   0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1796) - [internal]

BLAKE3_IV_1

```
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1799) - [internal]

BLAKE3_IV_2

```
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1802) - [internal]

BLAKE3_IV_3

```
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1805) - [internal]

BLAKE3_BLOCK_LEN

```
BLAKE3_BLOCK_LEN:
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
    .long 0x00000040, 0x00000040, 0x00000040, 0x00000040
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1808) - [internal] ROT16

```
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1810) - [internal] ROT8

```
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1812) - [internal]

```
CMP_MSB_MASK
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 1815) - [internal]

```
BLAKE3_IV
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 26) - [public]

```
_blake3_hash_many_avx512
.global _blake3_hash_many_avx512
.global blake3_hash_many_avx512
.global blake3_compress_in_place_avx512
.global _blake3_compress_in_place_avx512
.global blake3_compress_xof_avx512
.global _blake3_compress_xof_avx512
.global blake3_xof_many_avx512
.global _blake3_xof_many_avx512

#ifdef __APPLE__
.text
#else
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 27) - [public]

```
blake3_hash_many_avx512
.global blake3_hash_many_avx512
.global blake3_compress_in_place_avx512
.global _blake3_compress_in_place_avx512
.global blake3_compress_xof_avx512
.global _blake3_compress_xof_avx512
.global blake3_xof_many_avx512
.global _blake3_xof_many_avx512

#ifdef __APPLE__
.text
#else
.section .text
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 28) - [public]

```
blake3_compress_in_place_avx512
    .global blake3_compress_in_place_avx512
    .global _blake3_compress_in_place_avx512
    .global blake3_compress_xof_avx512
    .global _blake3_compress_xof_avx512
    .global blake3_xof_many_avx512
    .global _blake3_xof_many_avx512

    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 29) - [public]

```
_blake3_compress_in_place_avx512
    .global _blake3_compress_in_place_avx512
    .global blake3_compress_xof_avx512
    .global _blake3_compress_xof_avx512
    .global blake3_xof_many_avx512
    .global _blake3_xof_many_avx512

    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
    .p2align 6
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 30) - [public]

```
blake3_compress_xof_avx512
    .global blake3_compress_xof_avx512
    .global _blake3_compress_xof_avx512
    .global blake3_xof_many_avx512
    .global _blake3_xof_many_avx512

    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
    .p2align 6
    _blake3_hash_many_avx512:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 31) - [public]

```
_blake3_compress_xof_avx512
    .global _blake3_compress_xof_avx512
    .global blake3_xof_many_avx512
    .global _blake3_xof_many_avx512

    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
    .p2align 6
    _blake3_hash_many_avx512:
    blake3_hash_many_avx512:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 32) - [public]

```
blake3_xof_many_avx512
    .global blake3_xof_many_avx512
    .global _blake3_xof_many_avx512

    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
    .p2align 6
    _blake3_hash_many_avx512:
    blake3_hash_many_avx512:
        _CET_ENDBR
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 33) - [public]

```
_blake3_xof_many_avx512
    .global _blake3_xof_many_avx512

    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
    .p2align 6
    _blake3_hash_many_avx512:
    blake3_hash_many_avx512:
        _CET_ENDBR
        push    r15
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 41) - [internal]

```
_blake3_hash_many_avx512
    _blake3_hash_many_avx512:
    blake3_hash_many_avx512:
        _CET_ENDBR
        push    r15
        push    r14
        push    r13
        push    r12
        push    rbx
        push    rbp
        mov     rbp, rsp
        sub     rsp, 144
        and     rsp, 0xFFFFFFFFFFFFFFC0
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 42) - [internal]

```
blake3_hash_many_avx512
    blake3_hash_many_avx512:
        _CET_ENDBR
        push    r15
        push    r14
        push    r13
        push    r12
        push    rbx
        push    rbp
        mov     rbp, rsp
        sub     rsp, 144
        and     rsp, 0xFFFFFFFFFFFFFFC0
        neg     r9
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 2399) - [internal]
_blake3_compress_in_place_avx512

```
_blake3_compress_in_place_avx512:
blake3_compress_in_place_avx512:
    _CET_ENDBR
    vmovdqu xmm0, xmmword ptr [rdi]
    vmovdqu xmm1, xmmword ptr [rdi+0x10]
    movzx   eax, r8b
    movzx   edx, dl
    shl     rax, 32
    add     rdx, rax
    vmovq   xmm3, rcx
    vmovq   xmm4, rdx
    vpunpcklqdq xmm3, xmm3, xmm4
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 2400) - [internal]
blake3_compress_in_place_avx512

```
blake3_compress_in_place_avx512:
    _CET_ENDBR
    vmovdqu xmm0, xmmword ptr [rdi]
    vmovdqu xmm1, xmmword ptr [rdi+0x10]
    movzx   eax, r8b
    movzx   edx, dl
    shl     rax, 32
    add     rdx, rax
    vmovq   xmm3, rcx
    vmovq   xmm4, rdx
    vpunpcklqdq xmm3, xmm3, xmm4
    vmovaps xmm2, xmmword ptr [BLAKE3_IV+rip]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 2482) - [internal]
_blake3_compress_xof_avx512

```
_blake3_compress_xof_avx512:
blake3_compress_xof_avx512:
    _CET_ENDBR
    vmovdqu xmm0, xmmword ptr [rdi]
    vmovdqu xmm1, xmmword ptr [rdi+0x10]
    movzx   eax, r8b
    movzx   edx, dl
    shl     rax, 32
    add     rdx, rax
    vmovq   xmm3, rcx
    vmovq   xmm4, rdx
    vpunpcklqdq xmm3, xmm3, xmm4
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 2483) - [internal]
blake3_compress_xof_avx512

```
blake3_compress_xof_avx512:
    _CET_ENDBR
    vmovdqu xmm0, xmmword ptr [rdi]
    vmovdqu xmm1, xmmword ptr [rdi+0x10]
    movzx   eax, r8b
    movzx   edx, dl
    shl     rax, 32
    add     rdx, rax
    vmovq   xmm3, rcx
    vmovq   xmm4, rdx
    vpunpcklqdq xmm3, xmm3, xmm4
    vmovaps xmm2, xmmword ptr [BLAKE3_IV+rip]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 2569) - [internal]

blake3_xof_many_avx512

```
blake3_xof_many_avx512:
_blake3_xof_many_avx512:
    _CET_ENDBR
    mov     r10,QWORD PTR [rsp+0x8]
    cmp     r10,0x1
    ja      2f
    vmovdqu xmm0,XMMWORD PTR [rdi]
    vmovdqu xmm1,XMMWORD PTR [rdi+0x10]
    movzx   eax,r8b
    movzx   edx,dl
    shl     rax,0x20
    add     rdx,rax
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 2570) - [internal]

_blake3_xof_many_avx512

```
_blake3_xof_many_avx512:
    _CET_ENDBR
    mov     r10,QWORD PTR [rsp+0x8]
    cmp     r10,0x1
    ja      2f
    vmovdqu xmm0,XMMWORD PTR [rdi]
    vmovdqu xmm1,XMMWORD PTR [rdi+0x10]
    movzx   eax,r8b
    movzx   edx,dl
    shl     rax,0x20
    add     rdx,rax
    vmovq   xmm3,rcx
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4811) - [internal] INDEX0

```
INDEX0:
    .long    0,  1,  2,  3, 16, 17, 18, 19
    .long    8,  9, 10, 11, 24, 25, 26, 27
INDEX1:
    .long    4,  5,  6,  7, 20, 21, 22, 23
    .long   12, 13, 14, 15, 28, 29, 30, 31
ADD0:
    .long    0,  1,  2,  3,  4,  5,  6,  7
    .long    8,  9, 10, 11, 12, 13, 14, 15
ADD1:  .long    1
ADD16: .long   16
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4814) - [internal] INDEX1

```
INDEX1:
    .long    4,  5,  6,  7, 20, 21, 22, 23
    .long   12, 13, 14, 15, 28, 29, 30, 31
ADD0:
    .long    0,  1,  2,  3,  4,  5,  6,  7
    .long    8,  9, 10, 11, 12, 13, 14, 15
ADD1:  .long    1
ADD16: .long   16
BLAKE3_BLOCK_LEN:
    .long   64
.p2align 6
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4817) - [internal] ADD0

```
ADD0:
    .long    0, 1, 2, 3, 4, 5, 6, 7
    .long    8, 9, 10, 11, 12, 13, 14, 15
ADD1:  .long    1

ADD16: .long    16
BLAKE3_BLOCK_LEN:
    .long    64
.p2align 6
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4820) - [internal] ADD1

```
ADD1:  .long    1

ADD16: .long    16
BLAKE3_BLOCK_LEN:
    .long    64
.p2align 6
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4822) - [internal] ADD16

```
ADD16: .long    16
BLAKE3_BLOCK_LEN:
    .long    64
.p2align 6
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4823) - [internal]

BLAKE3_BLOCK_LEN

```
BLAKE3_BLOCK_LEN:
    .long    64
.p2align 6
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4826) - [internal] BLAKE3_IV

```
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4827) - [internal]

BLAKE3_IV_0

```
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4829) - [internal]

BLAKE3_IV_1

```
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4831) - [internal]

BLAKE3_IV_2

```
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 4833) - [internal]

BLAKE3_IV_3

```
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 3) - [public]

_blake3_hash_many_avx512

```
.global _blake3_hash_many_avx512
.global blake3_hash_many_avx512
.global blake3_compress_in_place_avx512
.global _blake3_compress_in_place_avx512
.global blake3_compress_xof_avx512
.global _blake3_compress_xof_avx512

.section .text
.p2align 6
_blake3_hash_many_avx512:
blake3_hash_many_avx512:
    push    r15
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 4) - [public]

blake3_hash_many_avx512

```
.global blake3_hash_many_avx512
.global blake3_compress_in_place_avx512
.global _blake3_compress_in_place_avx512
.global blake3_compress_xof_avx512
.global _blake3_compress_xof_avx512

.section .text
.p2align 6
_blake3_hash_many_avx512:
blake3_hash_many_avx512:
    push    r15
    push    r14
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 5) - [public]

```
blake3_compress_in_place_avx512
    .global blake3_compress_in_place_avx512
    .global _blake3_compress_in_place_avx512
    .global blake3_compress_xof_avx512
    .global _blake3_compress_xof_avx512

    .section .text
    .p2align 6
    _blake3_hash_many_avx512:
blake3_hash_many_avx512:
    push    r15
    push    r14
    push    r13
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 6) - [public]

```
_blake3_compress_in_place_avx512
    .global _blake3_compress_in_place_avx512
    .global blake3_compress_xof_avx512
    .global _blake3_compress_xof_avx512

    .section .text
    .p2align 6
    _blake3_hash_many_avx512:
blake3_hash_many_avx512:
    push    r15
    push    r14
    push    r13
    push    r12
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 7) - [public]

```
blake3_compress_xof_avx512
    .global blake3_compress_xof_avx512
    .global _blake3_compress_xof_avx512

    .section .text
    .p2align 6
    _blake3_hash_many_avx512:
blake3_hash_many_avx512:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rdi
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 8) - [public]

```
_blake3_compress_xof_avx512
    .global _blake3_compress_xof_avx512

    .section .text
    .p2align 6
    _blake3_hash_many_avx512:
blake3_hash_many_avx512:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rdi
    push    rsi
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 12) - [internal]
_blake3_hash_many_avx512

```
_blake3_hash_many_avx512:  
blake3_hash_many_avx512:  
    push    r15  
    push    r14  
    push    r13  
    push    r12  
    push    rdi  
    push    rsi  
    push    rbx  
    push    rbp  
    mov     rbp, rsp  
    sub     rsp, 304  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 13) - [internal]
blake3_hash_many_avx512

```
blake3_hash_many_avx512:  
    push    r15  
    push    r14  
    push    r13  
    push    r12  
    push    rdi  
    push    rsi  
    push    rbx  
    push    rbp  
    mov     rbp, rsp  
    sub     rsp, 304  
    and     rsp, 0xFFFFFFFFFFFFFFC0  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2401) - [internal]
_blake3_compress_in_place_avx512

```
_blake3_compress_in_place_avx512:  
blake3_compress_in_place_avx512:  
    sub     rsp, 72  
    vmovdqa xmmword ptr [rsp], xmm6  
    vmovdqa xmmword ptr [rsp+0x10], xmm7  
    vmovdqa xmmword ptr [rsp+0x20], xmm8  
    vmovdqa xmmword ptr [rsp+0x30], xmm9  
    vmovdqu xmm0, xmmword ptr [rcx]  
    vmovdqu xmm1, xmmword ptr [rcx+0x10]  
    movzx   eax, byte ptr [rsp+0x70]  
    movzx   r8d, r8b  
    shl     rax, 32  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2402) - [internal]
blake3_compress_in_place_avx512

```
blake3_compress_in_place_avx512:  
    sub     rsp, 72  
    vmovdqa xmmword ptr [rsp], xmm6  
    vmovdqa xmmword ptr [rsp+0x10], xmm7  
    vmovdqa xmmword ptr [rsp+0x20], xmm8  
    vmovdqa xmmword ptr [rsp+0x30], xmm9  
    vmovdqu xmm0, xmmword ptr [rcx]  
    vmovdqu xmm1, xmmword ptr [rcx+0x10]  
    movzx   eax, byte ptr [rsp+0x70]  
    movzx   r8d, r8b  
    shl     rax, 32  
    add     r8, rax  
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2494) - [internal]
_blake3_compress_xof_avx512

```
_blake3_compress_xof_avx512:
blake3_compress_xof_avx512:
    sub     rsp, 72
    vmovdqa xmmword ptr [rsp], xmm6
    vmovdqa xmmword ptr [rsp+0x10], xmm7
    vmovdqa xmmword ptr [rsp+0x20], xmm8
    vmovdqa xmmword ptr [rsp+0x30], xmm9
    vmovdqu xmm0, xmmword ptr [rcx]
    vmovdqu xmm1, xmmword ptr [rcx+0x10]
    movzx   eax, byte ptr [rsp+0x70]
    movzx   r8d, r8b
    mov     r10, qword ptr [rsp+0x78]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2495) - [internal]
blake3_compress_xof_avx512

```
blake3_compress_xof_avx512:
    sub     rsp, 72
    vmovdqa xmmword ptr [rsp], xmm6
    vmovdqa xmmword ptr [rsp+0x10], xmm7
    vmovdqa xmmword ptr [rsp+0x20], xmm8
    vmovdqa xmmword ptr [rsp+0x30], xmm9
    vmovdqu xmm0, xmmword ptr [rcx]
    vmovdqu xmm1, xmmword ptr [rcx+0x10]
    movzx   eax, byte ptr [rsp+0x70]
    movzx   r8d, r8b
    mov     r10, qword ptr [rsp+0x78]
    shl     rax, 32
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2592) - [internal]
INDEX0

```
INDEX0:
    .long   0, 1, 2, 3, 16, 17, 18, 19
    .long   8, 9, 10, 11, 24, 25, 26, 27
INDEX1:
    .long   4, 5, 6, 7, 20, 21, 22, 23
    .long   12, 13, 14, 15, 28, 29, 30, 31
ADD0:
    .long   0, 1, 2, 3, 4, 5, 6, 7
    .long   8, 9, 10, 11, 12, 13, 14, 15
ADD1:     .long   1
ADD16:    .long   16
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2595) - [internal]
INDEX1

```
INDEX1:
    .long   4, 5, 6, 7, 20, 21, 22, 23
    .long   12, 13, 14, 15, 28, 29, 30, 31
ADD0:
    .long   0, 1, 2, 3, 4, 5, 6, 7
    .long   8, 9, 10, 11, 12, 13, 14, 15
ADD1:     .long   1
ADD16:    .long   16
BLAKE3_BLOCK_LEN:
    .long   64
.p2align 6
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2598) - [internal]

ADD0

```
ADD0:
    .long    0, 1, 2, 3, 4, 5, 6, 7
    .long    8, 9, 10, 11, 12, 13, 14, 15
ADD1:    .long    1

ADD16:   .long    16
BLAKE3_BLOCK_LEN:
    .long    64
.p2align 6
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2601) - [internal]

ADD1

```
ADD1:    .long    1

ADD16:   .long    16
BLAKE3_BLOCK_LEN:
    .long    64
.p2align 6
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2603) - [internal]

ADD16

```
ADD16:   .long    16
BLAKE3_BLOCK_LEN:
    .long    64
.p2align 6
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2604) - [internal]

BLAKE3_BLOCK_LEN

```
BLAKE3_BLOCK_LEN:
    .long    64
.p2align 6
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2607) - [internal]

BLAKE3_IV

```
BLAKE3_IV:
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2608) - [internal]

BLAKE3_IV_0

```
BLAKE3_IV_0:
    .long    0x6A09E667
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2610) - [internal]

BLAKE3_IV_1

```
BLAKE3_IV_1:
    .long    0xBB67AE85
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2612) - [internal]

BLAKE3_IV_2

```
BLAKE3_IV_2:
    .long    0x3C6EF372
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 2614) - [internal]

BLAKE3_IV_3

```
BLAKE3_IV_3:
    .long    0xA54FF53A
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 24) - [public]

blake3_hash_many_sse2

```
.global blake3_hash_many_sse2
.global _blake3_hash_many_sse2
.global blake3_compress_in_place_sse2
.global _blake3_compress_in_place_sse2
.global blake3_compress_xof_sse2
.global _blake3_compress_xof_sse2
#ifdef __APPLE__
.text
#else
.section .text
#endif
    .p2align 6
    // ... (truncated, see source file for more)
```

```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 25) - [public]
_blake3_hash_many_sse2
    .global _blake3_hash_many_sse2
    .global blake3_compress_in_place_sse2
    .global _blake3_compress_in_place_sse2
    .global blake3_compress_xof_sse2
    .global _blake3_compress_xof_sse2
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
        .p2align 6
    _blake3_hash_many_sse2:
        // ... (truncated, see source file for more)

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 26) - [public]
blake3_compress_in_place_sse2
    .global blake3_compress_in_place_sse2
    .global _blake3_compress_in_place_sse2
    .global blake3_compress_xof_sse2
    .global _blake3_compress_xof_sse2
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
        .p2align 6
    _blake3_hash_many_sse2:
    blake3_hash_many_sse2:
        // ... (truncated, see source file for more)

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 27) - [public]
_blake3_compress_in_place_sse2
    .global _blake3_compress_in_place_sse2
    .global blake3_compress_xof_sse2
    .global _blake3_compress_xof_sse2
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
        .p2align 6
    _blake3_hash_many_sse2:
    blake3_hash_many_sse2:
        _CET_ENDBR
        // ... (truncated, see source file for more)

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 28) - [public]
blake3_compress_xof_sse2
    .global blake3_compress_xof_sse2
    .global _blake3_compress_xof_sse2
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
        .p2align 6
    _blake3_hash_many_sse2:
    blake3_hash_many_sse2:
        _CET_ENDBR
        push    r15
        // ... (truncated, see source file for more)

```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 29) - [public]

```
_blake3_compress_xof_sse2
.global _blake3_compress_xof_sse2
#ifdef __APPLE__
.text
#else
.section .text
#endif
.p2align 6
_blake3_hash_many_sse2:
blake3_hash_many_sse2:
    _CET_ENDBR
    push    r15
    push    r14
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 36) - [internal]

```
_blake3_hash_many_sse2
_blake3_hash_many_sse2:
blake3_hash_many_sse2:
    _CET_ENDBR
    push    r15
    push    r14
    push    r13
    push    r12
    push    rbx
    push    rbp
    mov     rbp, rsp
    sub     rsp, 360
    and     rsp, 0xFFFFFFFFFFFFC0
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 37) - [internal]

```
blake3_hash_many_sse2
blake3_hash_many_sse2:
    _CET_ENDBR
    push    r15
    push    r14
    push    r13
    push    r12
    push    rbx
    push    rbp
    mov     rbp, rsp
    sub     rsp, 360
    and     rsp, 0xFFFFFFFFFFFFC0
    neg     r9d
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2035) - [internal]

```
blake3_compress_in_place_sse2
blake3_compress_in_place_sse2:
    _CET_ENDBR
    movups  xmm0, xmmword ptr [rdi]
    movups  xmm1, xmmword ptr [rdi+0x10]
    movaps  xmm2, xmmword ptr [BLAKE3_IV+rip]
    shl     r8, 32
    add     rdx, r8
    movq    xmm3, rcx
    movq    xmm4, rdx
    punpcklqdq xmm3, xmm4
    movups  xmm4, xmmword ptr [rsi]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2036) - [internal]
_blake3_compress_in_place_sse2

```
_blake3_compress_in_place_sse2:
    _CET_ENDBR
    movups  xmm0, xmmword ptr [rdi]
    movups  xmm1, xmmword ptr [rdi+0x10]
    movaps  xmm2, xmmword ptr [BLAKE3_IV+rip]
    shl     r8, 32
    add     rdx, r8
    movq    xmm3, rcx
    movq    xmm4, rdx
    punpckldq xmm3, xmm4
    movups  xmm4, xmmword ptr [rsi]
    movups  xmm5, xmmword ptr [rsi+0x10]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2147) - [internal]
blake3_compress_xof_sse2

```
blake3_compress_xof_sse2:
    _blake3_compress_xof_sse2:
    _CET_ENDBR
    movups  xmm0, xmmword ptr [rdi]
    movups  xmm1, xmmword ptr [rdi+0x10]
    movaps  xmm2, xmmword ptr [BLAKE3_IV+rip]
    movzx   eax, r8b
    movzx   edx, dl
    shl     rax, 32
    add     rdx, rax
    movq    xmm3, rcx
    movq    xmm4, rdx
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2148) - [internal]
_blake3_compress_xof_sse2

```
_blake3_compress_xof_sse2:
    _CET_ENDBR
    movups  xmm0, xmmword ptr [rdi]
    movups  xmm1, xmmword ptr [rdi+0x10]
    movaps  xmm2, xmmword ptr [BLAKE3_IV+rip]
    movzx   eax, r8b
    movzx   edx, dl
    shl     rax, 32
    add     rdx, rax
    movq    xmm3, rcx
    movq    xmm4, rdx
    punpckldq xmm3, xmm4
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2273) - [internal] BLAKE3_IV

```
BLAKE3_IV:
    .long  0x6A09E667, 0xBB67AE85
    .long  0x3C6EF372, 0xA54FF53A
ADD0:
    .long  0, 1, 2, 3
ADD1:
    .long  4, 4, 4, 4
BLAKE3_IV_0:
    .long  0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long  0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2276) - [internal] ADD0

```
ADD0:■
    .long 0, 1, 2, 3
ADD1:
■.long 4, 4, 4, 4
BLAKE3_IV_0:
■.long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
■.long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
■.long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2278) - [internal] ADD1

```
ADD1:
■.long 4, 4, 4, 4
BLAKE3_IV_0:
■.long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
■.long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
■.long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2280) - [internal] BLAKE3_IV_0

```
BLAKE3_IV_0:
■.long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
■.long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
■.long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2282) - [internal] BLAKE3_IV_1

```
BLAKE3_IV_1:
■.long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
■.long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
■.long 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2284) - [internal] BLAKE3_IV_2

```
BLAKE3_IV_2:
■.long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
■.long 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
■.long 0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2286) - [internal] BLAKE3_IV_3

```
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
■.long 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
■.long 0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
■.long 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2288) - [internal]

BLAKE3_BLOCK_LEN

```
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
■.long 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
■.long 0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
■.long 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
■.long 0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2290) - [internal] CMP_MSB_MASK

```
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
■.long 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
■.long 0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
■.long 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
■.long 0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2292) - [internal]

PBLENDW_0x33_MASK

```
PBLENDW_0x33_MASK:
■.long 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
■.long 0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
■.long 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
■.long 0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2294) - [internal]
PBLENDW_0xCC_MASK

```
PBLENDW_0xCC_MASK:
■.long 0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
■.long 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
■.long 0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2296) - [internal]
PBLENDW_0x3F_MASK

```
PBLENDW_0x3F_MASK:
■.long 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
■.long 0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 2298) - [internal]
PBLENDW_0xC0_MASK

```
PBLENDW_0xC0_MASK:
■.long 0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2) - [public]
blake3_hash_many_sse2

```
.global blake3_hash_many_sse2
.global _blake3_hash_many_sse2
.global blake3_compress_in_place_sse2
.global _blake3_compress_in_place_sse2
.global blake3_compress_xof_sse2
.global _blake3_compress_xof_sse2
.section .text
.p2align 6
_blake3_hash_many_sse2:
blake3_hash_many_sse2:
    push    r15
    push    r14
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 3) - [public]
_blake3_hash_many_sse2

```
.global _blake3_hash_many_sse2
.global blake3_compress_in_place_sse2
.global _blake3_compress_in_place_sse2
.global blake3_compress_xof_sse2
.global _blake3_compress_xof_sse2
.section .text
.p2align 6
_blake3_hash_many_sse2:
blake3_hash_many_sse2:
    push    r15
    push    r14
    push    r13
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 4) - [public]
blake3_compress_in_place_sse2

```
.global blake3_compress_in_place_sse2
.global _blake3_compress_in_place_sse2
.global blake3_compress_xof_sse2
.global _blake3_compress_xof_sse2
.section .text
.p2align 6
_blake3_hash_many_sse2:
blake3_hash_many_sse2:
    push    r15
    push    r14
    push    r13
    push    r12
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 5) - [public]
_blake3_compress_in_place_sse2

```
.global _blake3_compress_in_place_sse2
.global blake3_compress_xof_sse2
.global _blake3_compress_xof_sse2
.section .text
.p2align 6
_blake3_hash_many_sse2:
blake3_hash_many_sse2:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 6) - [public]
blake3_compress_xof_sse2

```
.global blake3_compress_xof_sse2
.global _blake3_compress_xof_sse2
.section .text
.p2align 6
_blake3_hash_many_sse2:
blake3_hash_many_sse2:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 7) - [public]
_blake3_compress_xof_sse2

```
.global _blake3_compress_xof_sse2
.section .text
.p2align 6
_blake3_hash_many_sse2:
blake3_hash_many_sse2:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    push    rbx
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 10) - [internal]
_blake3_hash_many_sse2

```
_blake3_hash_many_sse2:
blake3_hash_many_sse2:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    push    rbx
    push    rbp
    mov     rbp, rsp
    sub     rsp, 528
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 11) - [internal]

blake3_hash_many_sse2

```
blake3_hash_many_sse2:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    push    rbx
    push    rbp
    mov     rbp, rsp
    sub     rsp, 528
    and     rsp, 0xFFFFFFFFFFFFFFC0
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2038) - [internal]

blake3_compress_in_place_sse2

```
blake3_compress_in_place_sse2:
_blake3_compress_in_place_sse2:
    sub     rsp, 120
    movdqa  xmmword ptr [rsp], xmm6
    movdqa  xmmword ptr [rsp+0x10], xmm7
    movdqa  xmmword ptr [rsp+0x20], xmm8
    movdqa  xmmword ptr [rsp+0x30], xmm9
    movdqa  xmmword ptr [rsp+0x40], xmm11
    movdqa  xmmword ptr [rsp+0x50], xmm14
    movdqa  xmmword ptr [rsp+0x60], xmm15
    movups  xmm0, xmmword ptr [rcx]
    movups  xmm1, xmmword ptr [rcx+0x10]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2039) - [internal]

_blake3_compress_in_place_sse2

```
_blake3_compress_in_place_sse2:
    sub     rsp, 120
    movdqa  xmmword ptr [rsp], xmm6
    movdqa  xmmword ptr [rsp+0x10], xmm7
    movdqa  xmmword ptr [rsp+0x20], xmm8
    movdqa  xmmword ptr [rsp+0x30], xmm9
    movdqa  xmmword ptr [rsp+0x40], xmm11
    movdqa  xmmword ptr [rsp+0x50], xmm14
    movdqa  xmmword ptr [rsp+0x60], xmm15
    movups  xmm0, xmmword ptr [rcx]
    movups  xmm1, xmmword ptr [rcx+0x10]
    movaps  xmm2, xmmword ptr [BLAKE3_IV+rip]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2168) - [internal]

_blake3_compress_xof_sse2

```
_blake3_compress_xof_sse2:
blake3_compress_xof_sse2:
    sub     rsp, 120
    movdqa  xmmword ptr [rsp], xmm6
    movdqa  xmmword ptr [rsp+0x10], xmm7
    movdqa  xmmword ptr [rsp+0x20], xmm8
    movdqa  xmmword ptr [rsp+0x30], xmm9
    movdqa  xmmword ptr [rsp+0x40], xmm11
    movdqa  xmmword ptr [rsp+0x50], xmm14
    movdqa  xmmword ptr [rsp+0x60], xmm15
    movups  xmm0, xmmword ptr [rcx]
    movups  xmm1, xmmword ptr [rcx+0x10]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2169) - [internal]
blake3_compress_xof_sse2

```
blake3_compress_xof_sse2:
    sub     rsp, 120
    movdqa  xmmword ptr [rsp], xmm6
    movdqa  xmmword ptr [rsp+0x10], xmm7
    movdqa  xmmword ptr [rsp+0x20], xmm8
    movdqa  xmmword ptr [rsp+0x30], xmm9
    movdqa  xmmword ptr [rsp+0x40], xmm11
    movdqa  xmmword ptr [rsp+0x50], xmm14
    movdqa  xmmword ptr [rsp+0x60], xmm15
    movups  xmm0, xmmword ptr [rcx]
    movups  xmm1, xmmword ptr [rcx+0x10]
    movaps  xmm2, xmmword ptr [BLAKE3_IV+rip]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2306) - [internal]
BLAKE3_IV

```
BLAKE3_IV:
    .long   0x6A09E667, 0xBB67AE85
    .long   0x3C6EF372, 0xA54FF53A
ADD0:
    .long   0, 1, 2, 3
ADD1:
    .long   4, 4, 4, 4
BLAKE3_IV_0:
    .long   0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long   0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2309) - [internal] ADD0

```
ADD0:
    .long   0, 1, 2, 3
ADD1:
    .long   4, 4, 4, 4
BLAKE3_IV_0:
    .long   0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long   0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long   0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long   0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2311) - [internal] ADD1

```
ADD1:
    .long   4, 4, 4, 4
BLAKE3_IV_0:
    .long   0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long   0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long   0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long   0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long   64, 64, 64, 64
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2313) - [internal]

BLAKE3_IV_0

```
BLAKE3_IV_0:
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2315) - [internal]

BLAKE3_IV_1

```
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
    .long 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2317) - [internal]

BLAKE3_IV_2

```
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
    .long 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
    .long 0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2319) - [internal]

BLAKE3_IV_3

```
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
    .long 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
    .long 0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
    .long 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2321) - [internal]

BLAKE3_BLOCK_LEN

```
BLAKE3_BLOCK_LEN:
    .long    64, 64, 64, 64
CMP_MSB_MASK:
    .long    0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
    .long    0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
    .long    0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
    .long    0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
    .long    0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2323) - [internal]

CMP_MSB_MASK

```
CMP_MSB_MASK:
    .long    0x80000000, 0x80000000, 0x80000000, 0x80000000
PBLENDW_0x33_MASK:
    .long    0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
    .long    0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
    .long    0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
    .long    0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2325) - [internal]

PBLENDW_0x33_MASK

```
PBLENDW_0x33_MASK:
    .long    0xFFFFFFFF, 0x00000000, 0xFFFFFFFF, 0x00000000
PBLENDW_0xCC_MASK:
    .long    0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
    .long    0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
    .long    0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2327) - [internal]

PBLENDW_0xCC_MASK

```
PBLENDW_0xCC_MASK:
    .long    0x00000000, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF
PBLENDW_0x3F_MASK:
    .long    0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
    .long    0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2329) - [internal]

PBLENDW_0x3F_MASK

```
PBLENDW_0x3F_MASK:
    .long    0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000
PBLENDW_0xC0_MASK:
    .long    0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2331) - [internal]

PBLENDW_0xC0_MASK

```
PBLENDW_0xC0_MASK:
    .long    0x00000000, 0x00000000, 0x00000000, 0xFFFFFFFF
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 24) - [public]

```
blake3_hash_many_sse41
    .global blake3_hash_many_sse41
    .global _blake3_hash_many_sse41
    .global blake3_compress_in_place_sse41
    .global _blake3_compress_in_place_sse41
    .global blake3_compress_xof_sse41
    .global _blake3_compress_xof_sse41
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
        .p2align 6
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 25) - [public]

```
_blake3_hash_many_sse41
    .global _blake3_hash_many_sse41
    .global blake3_compress_in_place_sse41
    .global _blake3_compress_in_place_sse41
    .global blake3_compress_xof_sse41
    .global _blake3_compress_xof_sse41
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
        .p2align 6
        _blake3_hash_many_sse41:
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 26) - [public]

```
blake3_compress_in_place_sse41
    .global blake3_compress_in_place_sse41
    .global _blake3_compress_in_place_sse41
    .global blake3_compress_xof_sse41
    .global _blake3_compress_xof_sse41
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
        .p2align 6
        _blake3_hash_many_sse41:
        blake3_hash_many_sse41:
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 27) - [public]

```
_blake3_compress_in_place_sse41
    .global _blake3_compress_in_place_sse41
    .global blake3_compress_xof_sse41
    .global _blake3_compress_xof_sse41
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
        .p2align 6
        _blake3_hash_many_sse41:
        blake3_hash_many_sse41:
            _CET_ENDBR
        // ... (truncated, see source file for more)
```


dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 28) - [public]

```
blake3_compress_xof_sse41
    .global blake3_compress_xof_sse41
    .global _blake3_compress_xof_sse41
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
    .p2align 6
    _blake3_hash_many_sse41:
    blake3_hash_many_sse41:
        _CET_ENDBR
        push    r15
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 29) - [public]

```
_blake3_compress_xof_sse41
    .global _blake3_compress_xof_sse41
    #ifdef __APPLE__
    .text
    #else
    .section .text
    #endif
    .p2align 6
    _blake3_hash_many_sse41:
    blake3_hash_many_sse41:
        _CET_ENDBR
        push    r15
        push    r14
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 36) - [internal]

```
_blake3_hash_many_sse41
    _blake3_hash_many_sse41:
    blake3_hash_many_sse41:
        _CET_ENDBR
        push    r15
        push    r14
        push    r13
        push    r12
        push    rbx
        push    rbp
        mov     rbp, rsp
        sub     rsp, 360
        and     rsp, 0xFFFFFFFFFFFFC0
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 37) - [internal]

```
blake3_hash_many_sse41
    blake3_hash_many_sse41:
        _CET_ENDBR
        push    r15
        push    r14
        push    r13
        push    r12
        push    rbx
        push    rbp
        mov     rbp, rsp
        sub     rsp, 360
        and     rsp, 0xFFFFFFFFFFFFC0
        neg     r9d
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 1798) - [internal]

blake3_compress_in_place_sse41

```
blake3_compress_in_place_sse41:
_blake3_compress_in_place_sse41:
    _CET_ENDBR
    movups xmm0, xmmword ptr [rdi]
    movups xmm1, xmmword ptr [rdi+0x10]
    movaps xmm2, xmmword ptr [BLAKE3_IV+rip]
    shl    r8, 32
    add    rdx, r8
    movq   xmm3, rcx
    movq   xmm4, rdx
    punpcklqdq xmm3, xmm4
    movups xmm4, xmmword ptr [rsi]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 1799) - [internal]

_blake3_compress_in_place_sse41

```
_blake3_compress_in_place_sse41:
    _CET_ENDBR
    movups xmm0, xmmword ptr [rdi]
    movups xmm1, xmmword ptr [rdi+0x10]
    movaps xmm2, xmmword ptr [BLAKE3_IV+rip]
    shl    r8, 32
    add    rdx, r8
    movq   xmm3, rcx
    movq   xmm4, rdx
    punpcklqdq xmm3, xmm4
    movups xmm4, xmmword ptr [rsi]
    movups xmm5, xmmword ptr [rsi+0x10]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 1899) - [internal]

blake3_compress_xof_sse41

```
blake3_compress_xof_sse41:
_blake3_compress_xof_sse41:
    _CET_ENDBR
    movups xmm0, xmmword ptr [rdi]
    movups xmm1, xmmword ptr [rdi+0x10]
    movaps xmm2, xmmword ptr [BLAKE3_IV+rip]
    movzx  eax, r8b
    movzx  edx, dl
    shl    rax, 32
    add    rdx, rax
    movq   xmm3, rcx
    movq   xmm4, rdx
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 1900) - [internal]

_blake3_compress_xof_sse41

```
_blake3_compress_xof_sse41:
    _CET_ENDBR
    movups xmm0, xmmword ptr [rdi]
    movups xmm1, xmmword ptr [rdi+0x10]
    movaps xmm2, xmmword ptr [BLAKE3_IV+rip]
    movzx  eax, r8b
    movzx  edx, dl
    shl    rax, 32
    add    rdx, rax
    movq   xmm3, rcx
    movq   xmm4, rdx
    punpcklqdq xmm3, xmm4
    // ... (truncated, see source file for more)
```

```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2014) - [internal] BLAKE3_IV
    BLAKE3_IV:
        .long 0x6A09E667, 0xBB67AE85
        .long 0x3C6EF372, 0xA54FF53A
    ROT16:
        .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
    ROT8:
        .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
    ADD0:
        .long 0, 1, 2, 3
    ADD1:
        .long 4, 4, 4, 4
    BLAKE3_IV_0:
        // ... (truncated, see source file for more)

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2017) - [internal] ROT16
    ROT16:
        .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
    ROT8:
        .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
    ADD0:
        .long 0, 1, 2, 3
    ADD1:
        .long 4, 4, 4, 4
    BLAKE3_IV_0:
        .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
    BLAKE3_IV_1:
        .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
        // ... (truncated, see source file for more)

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2019) - [internal] ROT8
    ROT8:
        .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
    ADD0:
        .long 0, 1, 2, 3
    ADD1:
        .long 4, 4, 4, 4
    BLAKE3_IV_0:
        .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
    BLAKE3_IV_1:
        .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    BLAKE3_IV_2:
        .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
        // ... (truncated, see source file for more)

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2021) - [internal] ADD0
    ADD0:
        .long 0, 1, 2, 3
    ADD1:
        .long 4, 4, 4, 4
    BLAKE3_IV_0:
        .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
    BLAKE3_IV_1:
        .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    BLAKE3_IV_2:
        .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    BLAKE3_IV_3:
        .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
        // ... (truncated, see source file for more)

```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2023) - [internal] ADD1

```
ADD1:
■.long 4, 4, 4, 4
BLAKE3_IV_0:
■.long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
■.long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
■.long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
// ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2025) - [internal] BLAKE3_IV_0

```
BLAKE3_IV_0:
■.long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
■.long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
■.long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2027) - [internal] BLAKE3_IV_1

```
BLAKE3_IV_1:
■.long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
■.long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2029) - [internal] BLAKE3_IV_2

```
BLAKE3_IV_2:
■.long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2031) - [internal] BLAKE3_IV_3

```
BLAKE3_IV_3:
■.long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2033) - [internal]

BLAKE3_BLOCK_LEN

```
BLAKE3_BLOCK_LEN:
■.long 64, 64, 64, 64
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 2035) - [internal]

CMP_MSB_MASK

```
CMP_MSB_MASK:
■.long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2) - [public]

```
blake3_hash_many_sse41
.global blake3_hash_many_sse41
.global _blake3_hash_many_sse41
.global blake3_compress_in_place_sse41
.global _blake3_compress_in_place_sse41
.global blake3_compress_xof_sse41
.global _blake3_compress_xof_sse41
.section .text
    .p2align 6
_blake3_hash_many_sse41:
blake3_hash_many_sse41:
    push    r15
    push    r14
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 3) - [public]

```
_blake3_hash_many_sse41
.global _blake3_hash_many_sse41
.global blake3_compress_in_place_sse41
.global _blake3_compress_in_place_sse41
.global blake3_compress_xof_sse41
.global _blake3_compress_xof_sse41
.section .text
    .p2align 6
_blake3_hash_many_sse41:
blake3_hash_many_sse41:
    push    r15
    push    r14
    push    r13
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 4) - [public]

```
blake3_compress_in_place_sse41
.global blake3_compress_in_place_sse41
.global _blake3_compress_in_place_sse41
.global blake3_compress_xof_sse41
.global _blake3_compress_xof_sse41
.section .text
    .p2align 6
_blake3_hash_many_sse41:
blake3_hash_many_sse41:
    push    r15
    push    r14
    push    r13
    push    r12
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 5) - [public]

```
_blake3_compress_in_place_sse41
.global _blake3_compress_in_place_sse41
.global blake3_compress_xof_sse41
.global _blake3_compress_xof_sse41
.section .text
    .p2align 6
_blake3_hash_many_sse41:
blake3_hash_many_sse41:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 6) - [public]

blake3_compress_xof_sse41

```
.global blake3_compress_xof_sse41
.global _blake3_compress_xof_sse41
.section .text
.p2align 6
_blake3_hash_many_sse41:
blake3_hash_many_sse41:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 7) - [public]

_blake3_compress_xof_sse41

```
.global _blake3_compress_xof_sse41
.section .text
.p2align 6
_blake3_hash_many_sse41:
blake3_hash_many_sse41:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    push    rbx
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 10) - [internal]

_blake3_hash_many_sse41

```
_blake3_hash_many_sse41:
blake3_hash_many_sse41:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    push    rbx
    push    rbp
    mov     rbp, rsp
    sub     rsp, 528
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 11) - [internal]

blake3_hash_many_sse41

```
blake3_hash_many_sse41:
    push    r15
    push    r14
    push    r13
    push    r12
    push    rsi
    push    rdi
    push    rbx
    push    rbp
    mov     rbp, rsp
    sub     rsp, 528
    and     rsp, 0xFFFFFFFFFFFFFFC0
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 1801) - [internal]

blake3_compress_in_place_sse41

```
blake3_compress_in_place_sse41:
    _blake3_compress_in_place_sse41:
        sub     rsp, 120
        movdqa  xmmword ptr [rsp], xmm6
        movdqa  xmmword ptr [rsp+0x10], xmm7
        movdqa  xmmword ptr [rsp+0x20], xmm8
        movdqa  xmmword ptr [rsp+0x30], xmm9
        movdqa  xmmword ptr [rsp+0x40], xmm11
        movdqa  xmmword ptr [rsp+0x50], xmm14
        movdqa  xmmword ptr [rsp+0x60], xmm15
        movups  xmm0, xmmword ptr [rcx]
        movups  xmm1, xmmword ptr [rcx+0x10]
        // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 1802) - [internal]

_blake3_compress_in_place_sse41

```
_blake3_compress_in_place_sse41:
    sub     rsp, 120
    movdqa  xmmword ptr [rsp], xmm6
    movdqa  xmmword ptr [rsp+0x10], xmm7
    movdqa  xmmword ptr [rsp+0x20], xmm8
    movdqa  xmmword ptr [rsp+0x30], xmm9
    movdqa  xmmword ptr [rsp+0x40], xmm11
    movdqa  xmmword ptr [rsp+0x50], xmm14
    movdqa  xmmword ptr [rsp+0x60], xmm15
    movups  xmm0, xmmword ptr [rcx]
    movups  xmm1, xmmword ptr [rcx+0x10]
    movaps  xmm2, xmmword ptr [BLAKE3_IV+rip]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 1920) - [internal]

_blake3_compress_xof_sse41

```
_blake3_compress_xof_sse41:
blake3_compress_xof_sse41:
    sub     rsp, 120
    movdqa  xmmword ptr [rsp], xmm6
    movdqa  xmmword ptr [rsp+0x10], xmm7
    movdqa  xmmword ptr [rsp+0x20], xmm8
    movdqa  xmmword ptr [rsp+0x30], xmm9
    movdqa  xmmword ptr [rsp+0x40], xmm11
    movdqa  xmmword ptr [rsp+0x50], xmm14
    movdqa  xmmword ptr [rsp+0x60], xmm15
    movups  xmm0, xmmword ptr [rcx]
    movups  xmm1, xmmword ptr [rcx+0x10]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 1921) - [internal]

blake3_compress_xof_sse41

```
blake3_compress_xof_sse41:
    sub     rsp, 120
    movdqa  xmmword ptr [rsp], xmm6
    movdqa  xmmword ptr [rsp+0x10], xmm7
    movdqa  xmmword ptr [rsp+0x20], xmm8
    movdqa  xmmword ptr [rsp+0x30], xmm9
    movdqa  xmmword ptr [rsp+0x40], xmm11
    movdqa  xmmword ptr [rsp+0x50], xmm14
    movdqa  xmmword ptr [rsp+0x60], xmm15
    movups  xmm0, xmmword ptr [rcx]
    movups  xmm1, xmmword ptr [rcx+0x10]
    movaps  xmm2, xmmword ptr [BLAKE3_IV+rip]
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2047) - [internal]

BLAKE3_IV

```
BLAKE3_IV:
    .long 0x6A09E667, 0xBB67AE85
    .long 0x3C6EF372, 0xA54FF53A
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
ADD0:
    .long 0, 1, 2, 3
ADD1:
    .long 4, 4, 4, 4
BLAKE3_IV_0:
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2050) - [internal]

ROT16

```
ROT16:
    .byte 2, 3, 0, 1, 6, 7, 4, 5, 10, 11, 8, 9, 14, 15, 12, 13
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
ADD0:
    .long 0, 1, 2, 3
ADD1:
    .long 4, 4, 4, 4
BLAKE3_IV_0:
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2052) - [internal] ROT8

```
ROT8:
    .byte 1, 2, 3, 0, 5, 6, 7, 4, 9, 10, 11, 8, 13, 14, 15, 12
ADD0:
    .long 0, 1, 2, 3
ADD1:
    .long 4, 4, 4, 4
BLAKE3_IV_0:
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2054) - [internal] ADD0

```
ADD0:
    .long 0, 1, 2, 3
ADD1:
    .long 4, 4, 4, 4
BLAKE3_IV_0:
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
    // ... (truncated, see source file for more)
```


dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2056) - [internal] ADD1

```
ADD1:
    .long 4, 4, 4, 4
BLAKE3_IV_0:
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
    // ... (truncated, see source file for more)
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2058) - [internal]

```
BLAKE3_IV_0
BLAKE3_IV_0:
    .long 0x6A09E667, 0x6A09E667, 0x6A09E667, 0x6A09E667
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2060) - [internal]

```
BLAKE3_IV_1
BLAKE3_IV_1:
    .long 0xBB67AE85, 0xBB67AE85, 0xBB67AE85, 0xBB67AE85
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2062) - [internal]

```
BLAKE3_IV_2
BLAKE3_IV_2:
    .long 0x3C6EF372, 0x3C6EF372, 0x3C6EF372, 0x3C6EF372
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2064) - [internal]

```
BLAKE3_IV_3
BLAKE3_IV_3:
    .long 0xA54FF53A, 0xA54FF53A, 0xA54FF53A, 0xA54FF53A
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2066) - [internal]

```
BLAKE3_BLOCK_LEN
BLAKE3_BLOCK_LEN:
    .long 64, 64, 64, 64
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2068) - [internal]

CMP_MSB_MASK

```
CMP_MSB_MASK:
    .long 0x80000000, 0x80000000, 0x80000000, 0x80000000
```

kernel_Workspace/kernel-bin/src/boot.asm (line 2) - [public] kernel_bin_entry

; kernel_Workspace/kernel-bin/src/boot.asm

```
global kernel_bin_entry
extern kernel_bin_main
```

```
section .text
bits 64
```

```
kernel_bin_entry:
    ; Wywołaj główną funkcję Rust w kernel-bin
    call kernel_bin_main

    ; Jeśli kernel_bin_main somehow wróci
.hang:
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel-bin/src/boot.asm (line 8) - [internal] kernel_bin_entry

```
kernel_bin_entry:
    ; Wywołaj główną funkcję Rust w kernel-bin
    call kernel_bin_main

    ; Jeśli kernel_bin_main somehow wróci
.hang:
    cli
    hlt
    jmp .hang
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 5) - [public] trampoline_start

```
.global trampoline_start
.global trampoline_end
.global trampoline_cr3
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 6) - [public] trampoline_end

```
.global trampoline_end
.global trampoline_cr3
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 7) - [public] trampoline_cr3

```
.global trampoline_cr3
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 8) - [public] trampoline_entry

```
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 9) - [public] trampoline_stack

```
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 10) - [public] trampoline_arg

```
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/trampoline.s (line 13) - [internal] trampoline_start

```
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss

    lgdt %cs:(trampoline_gdtr - trampoline_start)

    mov %cr0, %eax
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/trampoline.s (line 31) - [internal] trampoline_32

```
trampoline_32:

    mov $0x10, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss

    mov %cr4, %eax
    or $(1 << 5) | (1 << 9) | (1 << 10), %eax
    mov %eax, %cr4

    movl $(TRAMPOLINE_BASE + (trampoline_cr3 - trampoline_start)), %eax
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/trampoline.s (line 59) - [internal] trampoline_64

```
trampoline_64:

    mov $0x20, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss
    xor %eax, %eax
    mov %ax, %fs
    mov %ax, %gs

    movabs $(TRAMPOLINE_BASE + (trampoline_arg - trampoline_start)), %rcx
    mov (%rcx), %rdi
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/trampoline.s (line 81) - [internal] trampoline_gdt

```
trampoline_gdt:
    .quad 0x0000000000000000
    .quad 0x00CF9A000000FFFF
    .quad 0x00CF92000000FFFF
    .quad 0x00AF9A000000FFFF
    .quad 0x00AF92000000FFFF
trampoline_gdt_end:

.align 4
trampoline_gdtr:
    .word (trampoline_gdt_end - trampoline_gdt - 1)
    .long (TRAMPOLINE_BASE + (trampoline_gdt - trampoline_start))
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 87) - [internal] trampoline_gdt_end

```
trampoline_gdt_end:

.align 4
trampoline_gdtr:
    .word (trampoline_gdt_end - trampoline_gdt - 1)
    .long (TRAMPOLINE_BASE + (trampoline_gdt - trampoline_start))

.align 8
trampoline_cr3:    .quad 0
trampoline_entry:  .quad 0
trampoline_stack:  .quad 0
trampoline_arg:    .quad 0
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 90) - [internal] trampoline_gdtr

```
trampoline_gdtr:
    .word (trampoline_gdt_end - trampoline_gdt - 1)
    .long (TRAMPOLINE_BASE + (trampoline_gdt - trampoline_start))

.align 8
trampoline_cr3:    .quad 0
trampoline_entry:  .quad 0
trampoline_stack:  .quad 0
trampoline_arg:    .quad 0

.global trampoline_end
trampoline_end:
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 95) - [internal] trampoline_cr3

```
trampoline_cr3:    .quad 0
trampoline_entry:  .quad 0
trampoline_stack:  .quad 0
trampoline_arg:    .quad 0

.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 96) - [internal] trampoline_entry

```
trampoline_entry:  .quad 0
trampoline_stack:  .quad 0
trampoline_arg:    .quad 0

.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 97) - [internal] trampoline_stack

```
trampoline_stack:  .quad 0
trampoline_arg:    .quad 0

.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 98) - [internal] trampoline_arg

```
trampoline_arg:    .quad 0

.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 100) - [public] trampoline_end

```
.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 101) - [internal] trampoline_end

```
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 3) - [public] tlb_asm_invlpg

```
.global tlb_asm_invlpg
.type tlb_asm_invlpg, @function
tlb_asm_invlpg:
    invlpg (%rdi)
    ret

.global tlb_asm_read_cr3
.type tlb_asm_read_cr3, @function
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 5) - [internal] tlb_asm_invlpg

```
tlb_asm_invlpg:
    invlpg (%rdi)
    ret

.global tlb_asm_read_cr3
.type tlb_asm_read_cr3, @function
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 9) - [public] tlb_asm_read_cr3

```
.global tlb_asm_read_cr3
.type tlb_asm_read_cr3, @function
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 11) - [internal] tlb_asm_read_cr3

```
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 15) - [public] tlb_asm_write_cr3
.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 17) - [internal] tlb_asm_write_cr3
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 21) - [public] tlb_asm_read_cr4
.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 23) - [internal] tlb_asm_read_cr4
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret

.global tlb_asm_invpclid
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 27) - [public] tlb_asm_write_cr4

```
.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret
```

```
.global tlb_asm_invpcid
.type tlb_asm_invpcid, @function
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 29) - [internal] tlb_asm_write_cr4

```
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret
```

```
.global tlb_asm_invpcid
.type tlb_asm_invpcid, @function
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
```

```
.global tlb_asm_wbinvd
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 34) - [public] tlb_asm_invpcid

```
.global tlb_asm_invpcid
.type tlb_asm_invpcid, @function
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
```

```
.global tlb_asm_wbinvd
.type tlb_asm_wbinvd, @function
tlb_asm_wbinvd:
    wbinvd
    ret
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 36) - [internal] tlb_asm_invpcid

```
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
```

```
.global tlb_asm_wbinvd
.type tlb_asm_wbinvd, @function
tlb_asm_wbinvd:
    wbinvd
    ret
```

```
.global tlb_asm_clflush
.type tlb_asm_clflush, @function
    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 40) - [public] tlb_asm_wbinvd

```
.global tlb_asm_wbinvd
.type tlb_asm_wbinvd, @function
tlb_asm_wbinvd:
    wbinvd
    ret

.global tlb_asm_clflush
.type tlb_asm_clflush, @function
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 42) - [internal] tlb_asm_wbinvd

```
tlb_asm_wbinvd:
    wbinvd
    ret

.global tlb_asm_clflush
.type tlb_asm_clflush, @function
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 46) - [public] tlb_asm_clflush

```
.global tlb_asm_clflush
.type tlb_asm_clflush, @function
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 48) - [internal] tlb_asm_clflush

```
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 4) -
[internal] vector_table

```
vector_table:
    .align 7
    b vector_default
    .align 7
    b vector_default
    .align 7
    b vector_default
    .align 7
    b vector_default

    .align 7
    b vector_default
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 41) -
[internal] vector_default

```
vector_default:
    eret

vector_hyper:
    sub sp, sp, #272

    stp x0, x1, [sp, #0]
    stp x2, x3, [sp, #16]
    stp x4, x5, [sp, #32]
    stp x6, x7, [sp, #48]
    stp x8, x9, [sp, #64]
    stp x10, x11, [sp, #80]
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 44) -
[internal] vector_hyper

```
vector_hyper:
    sub sp, sp, #272

    stp x0, x1, [sp, #0]
    stp x2, x3, [sp, #16]
    stp x4, x5, [sp, #32]
    stp x6, x7, [sp, #48]
    stp x8, x9, [sp, #64]
    stp x10, x11, [sp, #80]
    stp x12, x13, [sp, #96]
    stp x14, x15, [sp, #112]
    stp x16, x17, [sp, #128]
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 102) -
[public] tr_init

```
.global tr_init
.type tr_init, @function
tr_init:
    adr x1, vector_table
    msr vbar_el1, x1
    isb
    ret

.global tr_restore_ctx
.type tr_restore_ctx, @function
tr_restore_ctx:
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 104) -
[internal] tr_init

```
tr_init:
    adr x1, vector_table
    msr vbar_el1, x1
    isb
    ret

.global tr_restore_ctx
.type tr_restore_ctx, @function
tr_restore_ctx:
    ldr x9, [x0, #248]
    msr sp_el0, x9
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 111) -
[public] tr_restore_ctx

```
.global tr_restore_ctx
.type tr_restore_ctx, @function
tr_restore_ctx:
    ldr x9, [x0, #248]
    msr sp_el0, x9
    ldr x9, [x0, #256]
    msr elr_el1, x9
    ldr x9, [x0, #264]
    msr spsr_el1, x9

    ldr x30, [x0, #240]
    ldp x28, x29, [x0, #224]
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 113) -
[internal] tr_restore_ctx

```
tr_restore_ctx:
    ldr x9, [x0, #248]
    msr sp_el0, x9
    ldr x9, [x0, #256]
    msr elr_el1, x9
    ldr x9, [x0, #264]
    msr spsr_el1, x9

    ldr x30, [x0, #240]
    ldp x28, x29, [x0, #224]
    ldp x26, x27, [x0, #208]
    ldp x24, x25, [x0, #192]
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 3) -
[internal] kstack_top

```
kstack_top:
    .dword 0

.section .text
.align 2

.global trap_entry
trap_entry:
    csrrw sp, sscratch, sp        # sp = kernel, sscratch = world sp
    addi sp, sp, -288

    sd x1, 8(sp)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 9) -
[public] trap_entry

```
.global trap_entry
trap_entry:
    csrrw sp, sscratch, sp        # sp = kernel, sscratch = world sp
    addi sp, sp, -288

    sd x1, 8(sp)
    sd x3, 24(sp)
    sd x4, 32(sp)
    sd x5, 40(sp)
    sd x6, 48(sp)
    sd x7, 56(sp)
    sd x8, 64(sp)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 10) -
[internal] trap_entry

```
trap_entry:
    csrrw sp, sscratch, sp        # sp = kernel, sscratch = world sp
    addi sp, sp, -288

    sd x1, 8(sp)
    sd x3, 24(sp)
    sd x4, 32(sp)
    sd x5, 40(sp)
    sd x6, 48(sp)
    sd x7, 56(sp)
    sd x8, 64(sp)
    sd x9, 72(sp)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 98) -

```
[public] tr_init
    .global tr_init
    .type tr_init, @function
tr_init:
    la t0, kstack_top
    sd a0, 0(t0)

    csrw sscratch, a0

    la t0, trap_entry
    csrw stvec, t0

    li t0, (1 << 18)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 100) -

```
[internal] tr_init
    tr_init:
        la t0, kstack_top
        sd a0, 0(t0)

        csrw sscratch, a0

        la t0, trap_entry
        csrw stvec, t0

        li t0, (1 << 18)
        csrs sstatus, t0

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 114) -

```
[public] tr_restore_ctx
    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    ld t0, 264(a0)
    csrw sepc, t0
    ld t0, 272(a0)
    csrw sstatus, t0

    la t0, kstack_top
    ld t0, 0(t0)
    csrw sscratch, t0

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 116) -

```
[internal] tr_restore_ctx
    tr_restore_ctx:
        ld t0, 264(a0)
        csrw sepc, t0
        ld t0, 272(a0)
        csrw sstatus, t0

        la t0, kstack_top
        ld t0, 0(t0)
        csrw sscratch, t0

        ld sp, 256(a0)

        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 3) -
[public] tr_gdt
    .global tr_gdt
    tr_gdt:
        .quad 0x0000000000000000
        .quad 0x00209A0000000000
        .quad 0x0000920000000000
        .quad 0x0020BA0000000000
        .quad 0x0000B20000000000
        .quad 0x0020FA0000000000
        .quad 0x0000F20000000000
    .global tr_tss_desc
    tr_tss_desc:
        .space 16
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 4) -
[internal] tr_gdt
    tr_gdt:
        .quad 0x0000000000000000
        .quad 0x00209A0000000000
        .quad 0x0000920000000000
        .quad 0x0020BA0000000000
        .quad 0x0000B20000000000
        .quad 0x0020FA0000000000
        .quad 0x0000F20000000000
    .global tr_tss_desc
    tr_tss_desc:
        .space 16

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 12) -
[public] tr_tss_desc
    .global tr_tss_desc
    tr_tss_desc:
        .space 16

    gdt_ptr:
        .short (gdt_ptr - tr_gdt) - 1
        .quad tr_gdt

    .align 16
    .global tr_tss
    tr_tss:
        .space 104
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 13) -
[internal] tr_tss_desc
    tr_tss_desc:
        .space 16

    gdt_ptr:
        .short (gdt_ptr - tr_gdt) - 1
        .quad tr_gdt

    .align 16
    .global tr_tss
    tr_tss:
        .space 104

        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 16) -
[internal] gdt_ptr
    gdt_ptr:
        .short (gdt_ptr - tr_gdt) - 1
        .quad tr_gdt

    .align 16
    .global tr_tss
tr_tss:
    .space 104

    .section .text

    .global tr_init
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 21) -
[public] tr_tss
    .global tr_tss
tr_tss:
    .space 104

    .section .text

    .global tr_init
    .type tr_init, @function
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 22) -
[internal] tr_tss
    tr_tss:
        .space 104

    .section .text

    .global tr_init
    .type tr_init, @function
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    mov %eax, (tr_tss + 8)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 27) -
[public] tr_init
    .global tr_init
    .type tr_init, @function
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    mov %eax, (tr_tss + 8)

    lgdt gdt_ptr

    mov $0x10, %ax
    mov %ax, %ss
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 29) -
[internal] tr_init
    tr_init:
        mov %rdi, %rax
        mov %eax, (tr_tss + 4)
        shr $32, %rax
        mov %eax, (tr_tss + 8)

        lgdt gdt_ptr

        mov $0x10, %ax
        mov %ax, %ss
        mov %ax, %ds
        mov %ax, %es
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 54) -
[public] isr_hypercall
    .global isr_hypercall
    .type isr_hypercall, @function
isr_hypercall:
    push %rax
    push %rbx
    push %rcx
    push %rdx
    push %rsi
    push %rdi
    push %rbp
    push %r8
    push %r9
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 56) -
[internal] isr_hypercall
    isr_hypercall:
        push %rax
        push %rbx
        push %rcx
        push %rdx
        push %rsi
        push %rdi
        push %rbp
        push %r8
        push %r9
        push %r10
        push %r11
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 94) -
[public] isr_default
    .global isr_default
    .type isr_default, @function
isr_default:
    iretq

    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    // ... (truncated, see source file for more)

```

```

kernel_workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 96) -
[internal] isr_default
    isr_default:
        iretq

    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    pop %r12
    pop %r11
    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 99) -
[public] tr_restore_ctx
    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    pop %r12
    pop %r11
    pop %r10
    pop %r9
    pop %r8
    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 101) -
[internal] tr_restore_ctx
    tr_restore_ctx:
        mov %rdi, %rsp
        pop %r15
        pop %r14
        pop %r13
        pop %r12
        pop %r11
        pop %r10
        pop %r9
        pop %r8
        pop %rbp
        pop %rdi
        // ... (truncated, see source file for more)

triang-lang/out/x86_64/prog.asm (line 4) - [internal] buffer
    buffer: resb 16

    section .text
    global main
    main:
    fn_main:
        mov rax, 16
        mov rbx, 32
        mov rcx, rax
        add rcx, rbx
        lea r14, [rel buffer]
        mov r15, 16
        // ... (truncated, see source file for more)

```


triang-lang/out/x86_64/prog.asm (line 7) - [public] main

```
global main
main:
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    mov r12, 0
fill_1:
    test r15, r15
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 8) - [internal] main

```
main:
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    mov r12, 0
fill_1:
    test r15, r15
    jz fill_end_2
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 9) - [internal] fn_main

```
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    mov r12, 0
fill_1:
    test r15, r15
    jz fill_end_2
    mov byte [r14], r12b
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 17) - [internal] fill_1

```
fill_1:
    test r15, r15
    jz fill_end_2
    mov byte [r14], r12b
    inc r14
    dec r15
    jmp fill_1
fill_end_2:
    lea r14, [rel buffer]
    mov byte [r14 + 0], 65
    lea r14, [rel buffer]
    mov byte [r14 + 1], 66
    // ... (truncated, see source file for more)
```

```

triang-lang/out/x86_64/prog.asm (line 24) - [internal] fill_end_2
    fill_end_2:
        lea r14, [rel buffer]
        mov byte [r14 + 0], 65
        lea r14, [rel buffer]
        mov byte [r14 + 1], 66
        lea r14, [rel buffer]
        mov byte [r14 + 2], 67
        lea r14, [rel buffer]
        movzx rax, byte [r14 + 0]
        lea r14, [rel buffer]
        movzx rbx, byte [r14 + 1]
        mov rcx, rax
        // ... (truncated, see source file for more)

triang-lang/out/x86_64/prog.asm (line 41) - [internal] else_1
    else_1:
        mov rax, 0
    end_2:
    loop_3:
        cmp rax, 0
        je loop_end_4
        mov rbx, rax
        sub rbx, 1
        mov rax, rbx
        jmp loop_3
    loop_end_4:
        ret

triang-lang/out/x86_64/prog.asm (line 43) - [internal] end_2
    end_2:
    loop_3:
        cmp rax, 0
        je loop_end_4
        mov rbx, rax
        sub rbx, 1
        mov rax, rbx
        jmp loop_3
    loop_end_4:
        ret

triang-lang/out/x86_64/prog.asm (line 44) - [internal] loop_3
    loop_3:
        cmp rax, 0
        je loop_end_4
        mov rbx, rax
        sub rbx, 1
        mov rax, rbx
        jmp loop_3
    loop_end_4:
        ret

triang-lang/out/x86_64/prog.asm (line 51) - [internal] loop_end_4
    loop_end_4:
        ret

```

C

```
comlimine/src/main.c (line 6) - [public] kernel_main
extern void kernel_main(uint64_t magic, void *info);

LIMINE_BASE_REVISION(1);

static volatile struct limine_framebuffer_request framebuffer = {
    .id = LIMINE_FRAMEBUFFER_REQUEST,
    .revision = 0,
    .response = NULL,
};

static volatile struct limine_memmap_request memmap = {
    .id = LIMINE_MEMMAP_REQUEST,
    // ... (truncated, see source file for more)
```

```
comlimine/src/main.c (line 28) - [public] _start
void _start(void) {
    if (framebuffer.response == NULL ||
        memmap.response == NULL ||
        hhdm.response == NULL) {
        for (;;) { asm volatile ("hlt"); }
    }

    kernel_main(0, (void *)framebuffer.response);

    for (;;) { asm volatile ("hlt"); }
}
```

```
comlimine/src/main.c (line 35) - [public] kernel_main
kernel_main(0, (void *)framebuffer.response);

for (;;) { asm volatile ("hlt"); }
}
```

```
dirtycode/alsa-lib/aserver/aserver.c (line 101) - [public] SYSERROR
    ■■SYSERROR("socket failed");
    ■■return result;
    ■}

    ■unlink(filename);

    ■addr->sun_family = AF_LOCAL;
    ■memcpy(addr->sun_path, filename, l);

    ■if (bind(sock, (struct sockaddr *) addr, size) < 0) {
    ■■int result = -errno;
    ■■SYSERROR("bind failed");
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/aserver/aserver.c (line 112) - [public] SYSERROR
    ■■SYSERROR("bind failed");
    ■■close(sock);
    ■■return result;
    ■}

    ■return sock;
    }

static int make_inet_socket(int port)
{
    ■struct sockaddr_in addr;
    ■int sock;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 113) - [public] close

```
    close(sock);
    return result;
}

return sock;
}

static int make_inet_socket(int port)
{
    struct sockaddr_in addr;
    int sock;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 128) - [public] SYSERROR

```
    SYSERROR("socket failed");
    return result;
}

memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = INADDR_ANY;

if (bind(sock, (struct sockaddr *) &addr, sizeof(addr)) < 0) {
    int result = -errno;
    SYSERROR("bind failed");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 139) - [public] SYSERROR

```
    SYSERROR("bind failed");
    close(sock);
    return result;
}

return sock;
}

struct pollfd *pollfds;
unsigned int pollfds_count = 0;
typedef struct waiter waiter_t;
typedef int (*waiter_handler_t)(waiter_t *waiter, unsigned short events);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 140) - [public] close

```
    close(sock);
    return result;
}

return sock;
}

struct pollfd *pollfds;
unsigned int pollfds_count = 0;
typedef struct waiter waiter_t;
typedef int (*waiter_handler_t)(waiter_t *waiter, unsigned short events);
struct waiter {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 150) - [public] int

```
typedef int (*waiter_handler_t)(waiter_t *waiter, unsigned short events);
struct waiter {
    int fd;
    void *private_data;
    waiter_handler_t handler;
};
waiter_t *waiters;

static void add_waiter(int fd, unsigned short events, waiter_handler_t handler,
    void *data)
{
    waiter_t *w = &waiters[fd];
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 262) - [public] if

```
if (n != 1) {
    SYSError("write failed");
    return -errno;
}
} else if (events & POLLOUT) {
    n = read(client->poll_fd, buf, 1);
    if (n != 1) {
        SYSError("read failed");
        return -errno;
    }
}
del_waiter(waiter->fd);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 263) - [public] SYSError

```
SYSError("write failed");
return -errno;
}
} else if (events & POLLOUT) {
    n = read(client->poll_fd, buf, 1);
    if (n != 1) {
        SYSError("read failed");
        return -errno;
    }
}
del_waiter(waiter->fd);
client->polling = 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 268) - [public] if

```
if (n != 1) {
    SYSError("read failed");
    return -errno;
}
}
del_waiter(waiter->fd);
client->polling = 0;
return 0;
}
#endif
```

```
static void pcm_shm_hw_ptr_changed(snd_pcm_t *pcm, snd_pcm_t *src ATTRIBUTE_UNUSED)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 269) - [public] SYSERRORR

```
    ■■■SYSERRORR("read failed");
    ■■■return -errno;
    ■■}
    ■}
    ■del_waiter(waiter->fd);
    ■client->polling = 0;
    ■return 0;
    ■}
    ■endif

    static void pcm_shm_hw_ptr_changed(snd_pcm_t *pcm, snd_pcm_t *src ATTRIBUTE_UNUSED)
    {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 336) - [public] SYSERRORR

```
    ■■SYSERRORR("shmget failed");
    ■■goto _err;
    ■}
    ■result = shm_set_gid(shmid);
    ■if (result < 0) {
    ■■SYSERRORR("shmctl IPC_SET failed");
    ■■shmctl(shmid, IPC_RMID, 0);
    ■■goto _err;
    ■}
    ■client->transport.shm.ctrl_id = shmid;
    ■client->transport.shm.ctrl = shmat(shmid, 0, 0);
    ■if (client->transport.shm.ctrl == (void*) -1) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 341) - [public] SYSERRORR

```
    ■■SYSERRORR("shmctl IPC_SET failed");
    ■■shmctl(shmid, IPC_RMID, 0);
    ■■goto _err;
    ■}
    ■client->transport.shm.ctrl_id = shmid;
    ■client->transport.shm.ctrl = shmat(shmid, 0, 0);
    ■if (client->transport.shm.ctrl == (void*) -1) {
    ■■result = -errno;
    ■■shmctl(shmid, IPC_RMID, 0);
    ■■SYSERRORR("shmat failed");
    ■■goto _err;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 342) - [public] shmctl

```
    ■■shmctl(shmid, IPC_RMID, 0);
    ■■goto _err;
    ■}
    ■client->transport.shm.ctrl_id = shmid;
    ■client->transport.shm.ctrl = shmat(shmid, 0, 0);
    ■if (client->transport.shm.ctrl == (void*) -1) {
    ■■result = -errno;
    ■■shmctl(shmid, IPC_RMID, 0);
    ■■SYSERRORR("shmat failed");
    ■■goto _err;
    ■}
    ■*cookie = shmid;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 349) - [public] shmctl

```
■shmctl(shmid, IPC_RMID, 0);
■SYSEERROR("shmat failed");
■goto _err;
■}
■*cookie = shmid;
■return 0;
```

```
_err:
■snd_pcm_close(pcm);
■return result;
```

```
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 350) - [public] SYSEERROR

```
■SYSEERROR("shmat failed");
■goto _err;
■}
■*cookie = shmid;
■return 0;
```

```
_err:
■snd_pcm_close(pcm);
■return result;
```

```
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 368) - [public] del_waiter

```
■del_waiter(client->device.pcm.fd);
■client->polling = 0;
■}
■err = snd_pcm_close(client->device.pcm.handle);
■if (err < 0)
■ERROR("snd_pcm_close");
■ctrl = client->transport.shm.ctrl;
■if (ctrl) {
■ctrl->result = err;
■err = shmdt((void *)client->transport.shm.ctrl);
■if (err < 0)
■■SYSEERROR("shmdt failed");
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 373) - [public] ERROR

```
■ERROR("snd_pcm_close");
■ctrl = client->transport.shm.ctrl;
■if (ctrl) {
■ctrl->result = err;
■err = shmdt((void *)client->transport.shm.ctrl);
■if (err < 0)
■■SYSEERROR("shmdt failed");
■err = shmctl(client->transport.shm.ctrl_id, IPC_RMID, 0);
■if (err < 0)
■■SYSEERROR("shmctl IPC_RMID failed");
■client->transport.shm.ctrl = 0;
■}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 379) - [public] SYSERRORR

```
■■■SYSERRORR("shmctl failed");
■■err = shmctl(client->transport.shm.ctrl_id, IPC_RMID, 0);
■■if (err < 0)
■■■SYSERRORR("shmctl IPC_RMID failed");
■■client->transport.shm.ctrl = 0;
■■}
■■client->open = 0;
■■return 0;
}
```

```
static int shm_ack(client_t *client)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 382) - [public] SYSERRORR

```
■■■SYSERRORR("shmctl IPC_RMID failed");
■■client->transport.shm.ctrl = 0;
■■}
■■client->open = 0;
■■return 0;
}
```

```
static int shm_ack(client_t *client)
{
    struct pollfd pfd;
    int err;
    char buf[1] = {0};
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 423) - [public] shm_ack_fd

```
■■return shm_ack_fd(client, rbptr->fd);
}
```

```
static void async_handler(snd_async_handler_t *handler)
{
    client_t *client = snd_async_handler_get_callback_private(handler);
    /* FIXME: use sigqueue */
    kill(client->async_pid, client->async_sig);
}
```

```
static int pcm_shm_cmd(client_t *client)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 451) - [public] if

```
■■if (ctrl->u.async.sig >= 0) {
■■■assert(client->async_sig < 0);
■■■ctrl->result = snd_async_add_pcm_handler(&client->async_handler, pcm, async_handler, client);
■■■if (ctrl->result < 0)
■■■■break;
■■} else {
■■■assert(client->async_sig >= 0);
■■■snd_async_del_handler(client->async_handler);
■■}
■■client->async_sig = ctrl->u.async.sig;
■■client->async_pid = ctrl->u.async.pid;
■■break;
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/aserver/aserver.c (line 452) - [public] assert

```
    assert(client->async_sig < 0);
    ctrl->result = snd_async_add_pcm_handler(&client->async_handler, pcm, async_handler, client);
    if (ctrl->result < 0)
        break;
    } else {
        assert(client->async_sig >= 0);
        snd_async_del_handler(client->async_handler);
    }
    client->async_sig = ctrl->u.async.sig;
    client->async_pid = ctrl->u.async.pid;
    break;
case SNDRV_PCM_IOCTL_INFO:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 457) - [public] assert

```
    assert(client->async_sig >= 0);
    snd_async_del_handler(client->async_handler);
}
client->async_sig = ctrl->u.async.sig;
client->async_pid = ctrl->u.async.pid;
break;
case SNDRV_PCM_IOCTL_INFO:
    ctrl->result = snd_pcm_info(pcm, (snd_pcm_info_t *) &ctrl->u.info);
    break;
case SNDRV_PCM_IOCTL_HW_REFINE:
    ctrl->result = snd_pcm_hw_refine(pcm, (snd_pcm_hw_params_t *) &ctrl->u.hw_refine);
    break;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 458) - [public] snd_async_del_handler

```
    snd_async_del_handler(client->async_handler);
}
client->async_sig = ctrl->u.async.sig;
client->async_pid = ctrl->u.async.pid;
break;
case SNDRV_PCM_IOCTL_INFO:
    ctrl->result = snd_pcm_info(pcm, (snd_pcm_info_t *) &ctrl->u.info);
    break;
case SNDRV_PCM_IOCTL_HW_REFINE:
    ctrl->result = snd_pcm_hw_refine(pcm, (snd_pcm_hw_params_t *) &ctrl->u.hw_refine);
    break;
case SNDRV_PCM_IOCTL_HW_PARAMS:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 515) - [public] shm_ack_fd

```
    return shm_ack_fd(client, ctrl->u.channel_info.u.mmap.fd);
    break;
case SNDRV_PCM_IOCTL_REWIND:
    ctrl->result = snd_pcm_rewind(pcm, ctrl->u.rewind.frames);
    break;
case SND_PCM_IOCTL_FORWARD:
    ctrl->result = snd_pcm_forward(pcm, ctrl->u.forward.frames);
    break;
case SNDRV_PCM_IOCTL_LINK:
{
    /* FIXME */
    ctrl->result = -ENOSYS;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 552) - [public] shm_ack_fd

```
    return shm_ack_fd(client, _snd_pcm_poll_descriptor(pcm));
case SND_PCM_IOCTL_CLOSE:
    client->ops->close(client);
    break;
case SND_PCM_IOCTL_HW_PTR_FD:
    return shm_rbp_ptr_fd(client, &pcm->hw);
case SND_PCM_IOCTL_APPL_PTR_FD:
    return shm_rbp_ptr_fd(client, &pcm->appl);
default:
    ERROR("Bogus cmd: %x", ctrl->cmd);
    ctrl->result = -ENOSYS;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 557) - [public] shm_rbp_ptr_fd

```
    return shm_rbp_ptr_fd(client, &pcm->hw);
case SND_PCM_IOCTL_APPL_PTR_FD:
    return shm_rbp_ptr_fd(client, &pcm->appl);
default:
    ERROR("Bogus cmd: %x", ctrl->cmd);
    ctrl->result = -ENOSYS;
}
return shm_ack(client);
}

transport_ops_t pcm_shm_ops = {
    .open = pcm_shm_open,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 559) - [public] shm_rbp_ptr_fd

```
    return shm_rbp_ptr_fd(client, &pcm->appl);
default:
    ERROR("Bogus cmd: %x", ctrl->cmd);
    ctrl->result = -ENOSYS;
}
return shm_ack(client);
}

transport_ops_t pcm_shm_ops = {
    .open = pcm_shm_open,
    .cmd = pcm_shm_cmd,
    .close = pcm_shm_close,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 561) - [public] ERROR

```
    ERROR("Bogus cmd: %x", ctrl->cmd);
    ctrl->result = -ENOSYS;
}
return shm_ack(client);
}

transport_ops_t pcm_shm_ops = {
    .open = pcm_shm_open,
    .cmd = pcm_shm_cmd,
    .close = pcm_shm_close,
};

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 564) - [public] shm_ack

```
    return shm_ack(client);
}

transport_ops_t pcm_shm_ops = {
    .open = pcm_shm_open,
    .cmd = pcm_shm_cmd,
    .close = pcm_shm_close,
};

static int ctl_handler(waiter_t *waiter, unsigned short events)
{
    client_t *client = waiter->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 580) - [public] if

```
    if (n != 1) {
        SYSError("write failed");
        return -errno;
    }
}

del_waiter(waiter->fd);
client->polling = 0;
return 0;
}

static int ctl_shm_open(client_t *client, int *cookie)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 581) - [public] SYSError

```
    SYSError("write failed");
    return -errno;
}
}

del_waiter(waiter->fd);
client->polling = 0;
return 0;
}

static int ctl_shm_open(client_t *client, int *cookie)
{
    int shmid;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 605) - [public] SYSError

```
    SYSError("shmget failed");
    goto _err;
}
result = shm_set_gid(shmid);
if (result < 0) {
    SYSError("shmctl IPC_SET failed");
    shmctl(shmid, IPC_RMID, 0);
    goto _err;
}
client->transport.shm.ctrl_id = shmid;
client->transport.shm.ctrl = shmat(shmid, 0, 0);
if (!client->transport.shm.ctrl) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 610) - [public] SYSERROR

```
■SYSERROR("shmctl IPC_SET failed");
■shmctl(shmid, IPC_RMID, 0);
■goto _err;
■}
■client->transport.shm.ctrl_id = shmid;
■client->transport.shm.ctrl = shmat(shmid, 0, 0);
■if (!client->transport.shm.ctrl) {
■result = -errno;
■shmctl(shmid, IPC_RMID, 0);
■SYSERROR("shmat failed");
■goto _err;
■}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 611) - [public] shmctl

```
■shmctl(shmid, IPC_RMID, 0);
■goto _err;
■}
■client->transport.shm.ctrl_id = shmid;
■client->transport.shm.ctrl = shmat(shmid, 0, 0);
■if (!client->transport.shm.ctrl) {
■result = -errno;
■shmctl(shmid, IPC_RMID, 0);
■SYSERROR("shmat failed");
■goto _err;
■}
■*cookie = shmid;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 618) - [public] shmctl

```
■shmctl(shmid, IPC_RMID, 0);
■SYSERROR("shmat failed");
■goto _err;
■}
■*cookie = shmid;
■add_waiter(client->device.ctl.fd, POLLIN, ctl_handler, client);
■client->polling = 1;
■return 0;

_err:
■snd_ctl_close(ctl);
■return result;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 619) - [public] SYSERROR

```
■SYSERROR("shmat failed");
■goto _err;
■}
■*cookie = shmid;
■add_waiter(client->device.ctl.fd, POLLIN, ctl_handler, client);
■client->polling = 1;
■return 0;

_err:
■snd_ctl_close(ctl);
■return result;

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 638) - [public] del_waiter

```
    ■■del_waiter(client->device.ctl.fd);
    ■■client->polling = 0;
    ■}
    ■err = snd_ctl_close(client->device.ctl.handle);
    ■if (err < 0)
    ■■ERROR("snd_ctl_close");
    ■ctrl = client->transport.shm.ctrl;
    ■if (ctrl) {
    ■■ctrl->result = err;
    ■■err = shmdt((void *)client->transport.shm.ctrl);
    ■■if (err < 0)
    ■■■SYSEERROR("shmdt failed");
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 643) - [public] ERROR

```
    ■■ERROR("snd_ctl_close");
    ■ctrl = client->transport.shm.ctrl;
    ■if (ctrl) {
    ■■ctrl->result = err;
    ■■err = shmdt((void *)client->transport.shm.ctrl);
    ■■if (err < 0)
    ■■■SYSEERROR("shmdt failed");
    ■■err = shmctl(client->transport.shm.ctrl_id, IPC_RMID, 0);
    ■■if (err < 0)
    ■■■SYSEERROR("shmctl failed");
    ■■client->transport.shm.ctrl = 0;
    ■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 649) - [public] SYSEERROR

```
    ■■■SYSEERROR("shmdt failed");
    ■■err = shmctl(client->transport.shm.ctrl_id, IPC_RMID, 0);
    ■■if (err < 0)
    ■■■SYSEERROR("shmctl failed");
    ■■client->transport.shm.ctrl = 0;
    ■}
    ■client->open = 0;
    ■return 0;
}
```

```
static int ctl_shm_cmd(client_t *client)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 652) - [public] SYSEERROR

```
    ■■■SYSEERROR("shmctl failed");
    ■■client->transport.shm.ctrl = 0;
    ■}
    ■client->open = 0;
    ■return 0;
}

static int ctl_shm_cmd(client_t *client)
{
    ■snd_ctl_shm_ctrl_t *ctrl = client->transport.shm.ctrl;
    ■char buf[1];
    ■int err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 677) - [public] if

```
    if (ctrl->u.async.sig >= 0) {
        assert(client->async_sig < 0);
        ctrl->result = snd_async_add_ctl_handler(&client->async_handler, ctl, async_handler, client);
        if (ctrl->result < 0)
            break;
    } else {
        assert(client->async_sig >= 0);
        snd_async_del_handler(client->async_handler);
    }
    client->async_sig = ctrl->u.async.sig;
    client->async_pid = ctrl->u.async.pid;
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 678) - [public] assert

```
    assert(client->async_sig < 0);
    ctrl->result = snd_async_add_ctl_handler(&client->async_handler, ctl, async_handler, client);
    if (ctrl->result < 0)
        break;
    } else {
        assert(client->async_sig >= 0);
        snd_async_del_handler(client->async_handler);
    }
    client->async_sig = ctrl->u.async.sig;
    client->async_pid = ctrl->u.async.pid;
    break;
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 683) - [public] assert

```
    assert(client->async_sig >= 0);
    snd_async_del_handler(client->async_handler);
    }
    client->async_sig = ctrl->u.async.sig;
    client->async_pid = ctrl->u.async.pid;
    break;
    break;
    case SNDRV_CTL_IOCTL_SUBSCRIBE_EVENTS:
        ctrl->result = snd_ctl_subscribe_events(ctl, ctrl->u.subscribe_events);
        break;
    case SNDRV_CTL_IOCTL_CARD_INFO:
        ctrl->result = snd_ctl_card_info(ctl, &ctrl->u.card_info);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 684) - [public] snd_async_del_handler

```
    snd_async_del_handler(client->async_handler);
    }
    client->async_sig = ctrl->u.async.sig;
    client->async_pid = ctrl->u.async.pid;
    break;
    break;
    case SNDRV_CTL_IOCTL_SUBSCRIBE_EVENTS:
        ctrl->result = snd_ctl_subscribe_events(ctl, ctrl->u.subscribe_events);
        break;
    case SNDRV_CTL_IOCTL_CARD_INFO:
        ctrl->result = snd_ctl_card_info(ctl, &ctrl->u.card_info);
        break;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 699) - [public] if

```
    if (ctrl->u.element_list.space * sizeof(*ctrl->u.element_list.pids) > maxsize) {
        ctrl->result = -EFAULT;
        break;
    }
    ctrl->u.element_list.pids = (snd_ctl_elem_id_t*) ctrl->data;
    ctrl->result = snd_ctl_elem_list(ctl, &ctrl->u.element_list);
    break;
}
case SNDRV_CTL_IOCTL_ELEM_INFO:
    ctrl->result = snd_ctl_elem_info(ctl, &ctrl->u.element_info);
    break;
case SNDRV_CTL_IOCTL_ELEM_READ:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 760) - [public] shm_ack_fd

```
    return shm_ack_fd(client, _snd_ctl_poll_descriptor(ctl));
default:
    ERROR("Bogus cmd: %x", ctrl->cmd);
    ctrl->result = -ENOSYS;
}
return shm_ack(client);
}

transport_ops_t ctl_shm_ops = {
    .open = ctl_shm_open,
    .cmd = ctl_shm_cmd,
    .close = ctl_shm_close,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 762) - [public] ERROR

```
    ERROR("Bogus cmd: %x", ctrl->cmd);
    ctrl->result = -ENOSYS;
}
return shm_ack(client);
}

transport_ops_t ctl_shm_ops = {
    .open = ctl_shm_open,
    .cmd = ctl_shm_cmd,
    .close = ctl_shm_close,
};

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 765) - [public] shm_ack

```
    return shm_ack(client);
}

transport_ops_t ctl_shm_ops = {
    .open = ctl_shm_open,
    .cmd = ctl_shm_cmd,
    .close = ctl_shm_close,
};

static int snd_client_open(client_t *client)
{
    int err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 783) - [public] SYSERROR

```
    ■■SYSERROR("read failed");
    ■■exit(1);
    ■}
    ■if (err != sizeof(req)) {
    ■■ans.result = -EINVAL;
    ■■goto _answer;
    ■}
    ■if (sizeof(client->name) < (size_t)(req.namelen + 1)) {
    ■■ans.result = -EINVAL;
    ■■goto _answer;
    ■}
    ■name = alloca(req.namelen + 1);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 784) - [public] exit

```
    ■■exit(1);
    ■}
    ■if (err != sizeof(req)) {
    ■■ans.result = -EINVAL;
    ■■goto _answer;
    ■}
    ■if (sizeof(client->name) < (size_t)(req.namelen + 1)) {
    ■■ans.result = -EINVAL;
    ■■goto _answer;
    ■}
    ■name = alloca(req.namelen + 1);
    ■err = read(client->ctrl_fd, name, req.namelen);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 797) - [public] SYSERROR

```
    ■■SYSERROR("read failed");
    ■■exit(1);
    ■}
    ■if (err != req.namelen) {
    ■■ans.result = -EINVAL;
    ■■goto _answer;
    ■}

    ■switch (req.transport_type) {
    ■case SND_TRANSPORT_TYPE_SHM:
    ■■if (!client->local) {
    ■■■ans.result = -EINVAL;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 798) - [public] exit

```
    ■■exit(1);
    ■}
    ■if (err != req.namelen) {
    ■■ans.result = -EINVAL;
    ■■goto _answer;
    ■}

    ■switch (req.transport_type) {
    ■case SND_TRANSPORT_TYPE_SHM:
    ■■if (!client->local) {
    ■■■ans.result = -EINVAL;
    ■■■goto _answer;
    ■// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/aserver/aserver.c (line 807) - [public] if

```
    if (!client->local) {
        ans.result = -EINVAL;
        goto _answer;
    }
    switch (req.dev_type) {
        case SND_DEV_TYPE_PCM:
            client->ops = &pcm_shm_ops;
            break;
        case SND_DEV_TYPE_CONTROL:
            client->ops = &ctl_shm_ops;
            break;
        default:
            // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/aserver/aserver.c (line 811) - [public] switch

```
    switch (req.dev_type) {
        case SND_DEV_TYPE_PCM:
            client->ops = &pcm_shm_ops;
            break;
        case SND_DEV_TYPE_CONTROL:
            client->ops = &ctl_shm_ops;
            break;
        default:
            ans.result = -EINVAL;
            goto _answer;
    }
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 850) - [public] SYSERROR

```
    SYSERROR("write failed");
    exit(1);
}
return 0;
}
```

```
static int client_poll_handler(waiter_t *waiter, unsigned short events ATTRIBUTE_UNUSED)
{
    client_t *client = waiter->private_data;
    if (client->open)
        client->ops->close(client);
    close(client->poll_fd);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/aserver/aserver.c (line 851) - [public] exit

```
    exit(1);
}
return 0;
}
```

```
static int client_poll_handler(waiter_t *waiter, unsigned short events ATTRIBUTE_UNUSED)
{
    client_t *client = waiter->private_data;
    if (client->open)
        client->ops->close(client);
    close(client->poll_fd);
    close(client->ctrl_fd);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/aserver/aserver.c (line 876) - [public] close

```
    close(client->ctrl_fd);
    del_waiter(client->ctrl_fd);
    list_del(&client->list);
    free(client);
    return 0;
}
if (client->open)
    return client->ops->cmd(client);
else
    return snd_client_open(client);
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/aserver/aserver.c (line 877) - [public] del_waiter

```
del_waiter(client->ctrl_fd);
list_del(&client->list);
free(client);
return 0;
}
if (client->open)
    return client->ops->cmd(client);
else
    return snd_client_open(client);
}
```

```
static int inet_pending_handler(waiter_t *waiter, unsigned short events)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 878) - [public] list_del

```
list_del(&client->list);
free(client);
return 0;
}
if (client->open)
    return client->ops->cmd(client);
else
    return snd_client_open(client);
}
```

```
static int inet_pending_handler(waiter_t *waiter, unsigned short events)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 879) - [public] free

```
free(client);
return 0;
}
if (client->open)
    return client->ops->cmd(client);
else
    return snd_client_open(client);
}
```

```
static int inet_pending_handler(waiter_t *waiter, unsigned short events)
{
    inet_pending_t *pending = waiter->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 885) - [public] snd_client_open

```
    return snd_client_open(client);
}

static int inet_pending_handler(waiter_t *waiter, unsigned short events)
{
    inet_pending_t *pending = waiter->private_data;
    inet_pending_t *pdata;
    client_t *client;
    uint32_t cookie;
    struct list_head *item;
    int remove = 0;
    if (events & POLLHUP)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 910) - [public] close

```
    close(waiter->fd);
    list_del(&pending->list);
    free(pending);
    return 0;
}

list_for_each(item, &inet_pendings) {
    pdata = list_entry(item, inet_pending_t, list);
    if (pdata->cookie == cookie)
        goto found;
}
pending->cookie = cookie;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 911) - [public] list_del

```
    list_del(&pending->list);
    free(pending);
    return 0;
}

list_for_each(item, &inet_pendings) {
    pdata = list_entry(item, inet_pending_t, list);
    if (pdata->cookie == cookie)
        goto found;
}
pending->cookie = cookie;
return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 912) - [public] free

```
    free(pending);
    return 0;
}

list_for_each(item, &inet_pendings) {
    pdata = list_entry(item, inet_pending_t, list);
    if (pdata->cookie == cookie)
        goto found;
}
pending->cookie = cookie;
return 0;

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 946) - [public] SYSERRORR

```
    ■■SYSERRORR("accept failed");
    ■■return result;
    ■} else {
    ■■client_t *client = calloc(1, sizeof(*client));
    ■■client->ctrl_fd = sock;
    ■■client->local = 1;
    ■■client->open = 0;
    ■■add_waiter(sock, POLLIN | POLLHUP, client_ctrl_handler, client);
    ■■list_add_tail(&client->list, &clients);
    ■}
    ■return 0;
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 953) - [public] add_waiter

```
    ■■add_waiter(sock, POLLIN | POLLHUP, client_ctrl_handler, client);
    ■■list_add_tail(&client->list, &clients);
    ■}
    ■return 0;
    }
```

```
static int inet_handler(waiter_t *waiter, unsigned short events ATTRIBUTE_UNUSED)
{
    ■int sock;
    ■sock = accept(waiter->fd, 0, 0);
    ■if (sock < 0) {
    ■■int result = -errno;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 954) - [public] list_add_tail

```
    ■■list_add_tail(&client->list, &clients);
    ■}
    ■return 0;
    }
```

```
static int inet_handler(waiter_t *waiter, unsigned short events ATTRIBUTE_UNUSED)
{
    ■int sock;
    ■sock = accept(waiter->fd, 0, 0);
    ■if (sock < 0) {
    ■■int result = -errno;
    ■■SYSERRORR("accept failed");
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 965) - [public] SYSERRORR

```
    ■■SYSERRORR("accept failed");
    ■■return result;
    ■} else {
    ■■inet_pending_t *pending = calloc(1, sizeof(*pending));
    ■■pending->fd = sock;
    ■■pending->cookie = 0;
    ■■add_waiter(sock, POLLIN, inet_pending_handler, pending);
    ■■list_add_tail(&pending->list, &inet_pendings);
    ■}
    ■return 0;
    }
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/aserver/aserver.c (line 971) - [public] add_waiter

```
■add_waiter(sock, POLLIN, inet_pending_handler, pending);
■list_add_tail(&pending->list, &inet_pendings);
■}
■return 0;
■}
```

```
static int server(const char *sockname, int port)
{
■int err, result, sockn = -1, socki = -1;
■unsigned int k;
■long open_max;
```

```
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 972) - [public] list_add_tail

```
■list_add_tail(&pending->list, &inet_pendings);
■}
■return 0;
■}
```

```
static int server(const char *sockname, int port)
{
■int err, result, sockn = -1, socki = -1;
■unsigned int k;
■long open_max;
```

```
■if (!sockname && port < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 988) - [public] SYSERROR

```
■SYSERROR("sysconf failed");
■return result;
■}
■pollfds = calloc((size_t) open_max, sizeof(*pollfds));
■waiters = calloc((size_t) open_max, sizeof(*waiters));
```

```
■if (sockname) {
■sockn = make_local_socket(sockname);
■if (sockn < 0)
■return sockn;
■if (fcntl(sockn, F_SETFL, O_NONBLOCK) < 0) {
■result = -errno;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 998) - [public] if

```
■if (fcntl(sockn, F_SETFL, O_NONBLOCK) < 0) {
■result = -errno;
■SYSERROR("fcntl O_NONBLOCK failed");
■goto _end;
■}
■if (listen(sockn, 4) < 0) {
■result = -errno;
■SYSERROR("listen failed");
■goto _end;
■}
■add_waiter(sockn, POLLIN, local_handler, NULL);
■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1000) - [public] SYSERROR

```
    SYSERROR("fcntl O_NONBLOCK failed");
    goto _end;
}
if (listen(sockn, 4) < 0) {
    result = -errno;
    SYSERROR("listen failed");
    goto _end;
}
add_waiter(sockn, POLLIN, local_handler, NULL);
}
if (port >= 0) {
    socki = make_inet_socket(port);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1003) - [public] if

```
if (listen(sockn, 4) < 0) {
    result = -errno;
    SYSERROR("listen failed");
    goto _end;
}
add_waiter(sockn, POLLIN, local_handler, NULL);
}
if (port >= 0) {
    socki = make_inet_socket(port);
    if (socki < 0)
        return socki;
    if (fcntl(socki, F_SETFL, O_NONBLOCK) < 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1005) - [public] SYSERROR

```
    SYSERROR("listen failed");
    goto _end;
}
add_waiter(sockn, POLLIN, local_handler, NULL);
}
if (port >= 0) {
    socki = make_inet_socket(port);
    if (socki < 0)
        return socki;
    if (fcntl(socki, F_SETFL, O_NONBLOCK) < 0) {
        result = -errno;
        SYSERROR("fcntl failed");
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1008) - [public] add_waiter

```
add_waiter(sockn, POLLIN, local_handler, NULL);
}
if (port >= 0) {
    socki = make_inet_socket(port);
    if (socki < 0)
        return socki;
    if (fcntl(socki, F_SETFL, O_NONBLOCK) < 0) {
        result = -errno;
        SYSERROR("fcntl failed");
        goto _end;
    }
    if (listen(socki, 4) < 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1014) - [public] if

```
    if (fcntl(socki, F_SETFL, O_NONBLOCK) < 0) {
        result = -errno;
        SYSERROR("fcntl failed");
        goto _end;
    }
    if (listen(socki, 4) < 0) {
        result = -errno;
        SYSERROR("listen failed");
        goto _end;
    }
    add_waiter(socki, POLLIN, inet_handler, NULL);
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1016) - [public] SYSERROR

```
    SYSERROR("fcntl failed");
    goto _end;
}
if (listen(socki, 4) < 0) {
    result = -errno;
    SYSERROR("listen failed");
    goto _end;
}
add_waiter(socki, POLLIN, inet_handler, NULL);
}

while (1) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1019) - [public] if

```
    if (listen(socki, 4) < 0) {
        result = -errno;
        SYSERROR("listen failed");
        goto _end;
    }
    add_waiter(socki, POLLIN, inet_handler, NULL);
}

while (1) {
    struct pollfd pfd[open_max];
    size_t pfd_count;
    do {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1021) - [public] SYSERROR

```
    SYSERROR("listen failed");
    goto _end;
}
add_waiter(socki, POLLIN, inet_handler, NULL);
}

while (1) {
    struct pollfd pfd[open_max];
    size_t pfd_count;
    do {
        err = poll(pfd, pfd_count, -1);
    } while (err == 0);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1024) - [public] add_waiter

```
    add_waiter(socki, POLLIN, inet_handler, NULL);
}

while (1) {
    struct pollfd pfd[open_max];
    size_t pfd_count;
    do {
        err = poll(pfd, pfd_count, -1);
    } while (err == 0);
    if (err < 0) {
        SYSERROR("poll failed");
        continue;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/aserver/aserver.c (line 1033) - [public] if

```
    if (err < 0) {
        SYSERROR("poll failed");
        continue;
    }

    pfd_count = pfd_count;
    memcpy(pfd, pfd, sizeof(*pfd) * pfd_count);
    for (k = 0; k < pfd_count; k++) {
        struct pollfd *pfd = &pfd[k];
        if (pfd->revents) {
            waiter_t *w = &waiters[pfd->fd];
            if (!w->handler)
                // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/aserver/aserver.c (line 1034) - [public] SYSERROR

```
    SYSERROR("poll failed");
    continue;
}

pfd_count = pfd_count;
memcpy(pfd, pfd, sizeof(*pfd) * pfd_count);
for (k = 0; k < pfd_count; k++) {
    struct pollfd *pfd = &pfd[k];
    if (pfd->revents) {
        waiter_t *w = &waiters[pfd->fd];
        if (!w->handler)
            continue;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/aserver/aserver.c (line 1039) - [public] memcpy

```
    memcpy(pfd, pfd, sizeof(*pfd) * pfd_count);
    for (k = 0; k < pfd_count; k++) {
        struct pollfd *pfd = &pfd[k];
        if (pfd->revents) {
            waiter_t *w = &waiters[pfd->fd];
            if (!w->handler)
                continue;
            err = w->handler(w, pfd->revents);
            if (err < 0)
                ERROR("waiter handler failed");
        }
    }
    // ... (truncated, see source file for more)
}
```


dirtycode/alsa-lib/aserver/aserver.c (line 1042) - [public] if

```
    if (pfd->revents) {
        waiter_t *w = &waiters[pfd->fd];
        if (!w->handler)
            continue;
        err = w->handler(w, pfd->revents);
        if (err < 0)
            ERROR("waiter handler failed");
    }
}
_end:
if (sockn >= 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1048) - [public] ERROR

```
    ERROR("waiter handler failed");
}
}
_end:
if (sockn >= 0)
    close(sockn);
if (socki >= 0)
    close(socki);
free(pollfds);
free(waiters);
return result;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1054) - [public] close

```
close(sockn);
if (socki >= 0)
    close(socki);
free(pollfds);
free(waiters);
return result;
}
```

```
static void usage(void)
{
    fprintf(stderr,
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1056) - [public] close

```
close(socki);
free(pollfds);
free(waiters);
return result;
}
```

```
static void usage(void)
{
    fprintf(stderr,
        "Usage: %s [OPTIONS] server\n"
        "--help/-h help\n",
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1087) - [public] switch

```
    switch (c) {
        case 'h':
            usage();
            return 0;
        default:
            fprintf(stderr, "Try `%s --help' for more information\n", command);
            return 1;
    }
}
if (argc - optind != 1) {
    ERROR("you need to specify server name");
    return 1;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1089) - [public] usage

```
    usage();
    return 0;
    default:
        fprintf(stderr, "Try `%s --help' for more information\n", command);
        return 1;
    }
}
if (argc - optind != 1) {
    ERROR("you need to specify server name");
    return 1;
}
err = snd_config_update();
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1092) - [public] fprintf

```
    fprintf(stderr, "Try `%s --help' for more information\n", command);
    return 1;
}
}
if (argc - optind != 1) {
    ERROR("you need to specify server name");
    return 1;
}
err = snd_config_update();
if (err < 0) {
    ERROR("cannot read configuration file");
    return 1;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1097) - [public] ERROR

```
    ERROR("you need to specify server name");
    return 1;
}
err = snd_config_update();
if (err < 0) {
    ERROR("cannot read configuration file");
    return 1;
}
srvname = argv[optind];
err = snd_config_search_definition(snd_config, "server", srvname, &conf);
if (err < 0) {
    ERROR("Missing definition for server %s", srvname);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1102) - [public] ERROR

```
    ■■ERROR("cannot read configuration file");
    ■■return 1;
    ■}
    ■srvname = argv[optind];
    ■err = snd_config_search_definition(snd_config, "server", srvname, &conf);
    ■if (err < 0) {
    ■■ERROR("Missing definition for server %s", srvname);
    ■■return 1;
    ■}
    ■if (snd_config_get_type(conf) != SND_CONFIG_TYPE_COMPOUND) {
    ■■snd_error(ASERVER, "Invalid type for server %s definition", srvname);
    ■■return -EINVAL;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1108) - [public] ERROR

```
    ■■ERROR("Missing definition for server %s", srvname);
    ■■return 1;
    ■}
    ■if (snd_config_get_type(conf) != SND_CONFIG_TYPE_COMPOUND) {
    ■■snd_error(ASERVER, "Invalid type for server %s definition", srvname);
    ■■return -EINVAL;
    ■}
    ■snd_config_for_each(i, next, conf) {
    ■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■const char *id;
    ■■if (snd_config_get_id(n, &id) < 0)
    ■■■continue;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1112) - [public] snd_error

```
    ■■snd_error(ASERVER, "Invalid type for server %s definition", srvname);
    ■■return -EINVAL;
    ■}
    ■snd_config_for_each(i, next, conf) {
    ■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■const char *id;
    ■■if (snd_config_get_id(n, &id) < 0)
    ■■■continue;
    ■■if (strcmp(id, "comment") == 0)
    ■■■continue;
    ■■if (strcmp(id, "host") == 0)
    ■■■continue;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1124) - [public] if

```
    ■■if (strcmp(id, "socket") == 0) {
    ■■■err = snd_config_get_string(n, &sockname);
    ■■■if (err < 0) {
    ■■■■ERROR("Invalid type for %s", id);
    ■■■■return 1;
    ■■■}
    ■■■continue;
    ■■}
    ■■if (strcmp(id, "port") == 0) {
    ■■■err = snd_config_get_integer(n, &port);
    ■■■if (err < 0) {
    ■■■■ERROR("Invalid type for %s", id);
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1126) - [public] if

```
    if (err < 0) {
        ERROR("Invalid type for %s", id);
        return 1;
    }
    continue;
}
if (strcmp(id, "port") == 0) {
    err = snd_config_get_integer(n, &port);
    if (err < 0) {
        ERROR("Invalid type for %s", id);
        return 1;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1127) - [public] ERROR

```
    ERROR("Invalid type for %s", id);
    return 1;
}
continue;
}
if (strcmp(id, "port") == 0) {
    err = snd_config_get_integer(n, &port);
    if (err < 0) {
        ERROR("Invalid type for %s", id);
        return 1;
    }
    continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1132) - [public] if

```
    if (strcmp(id, "port") == 0) {
        err = snd_config_get_integer(n, &port);
        if (err < 0) {
            ERROR("Invalid type for %s", id);
            return 1;
        }
        continue;
    }
    if (strcmp(id, "gid") == 0) {
        char *group;
        err = snd_config_get_ascii(n, &group);
        if (err < 0) {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1134) - [public] if

```
    if (err < 0) {
        ERROR("Invalid type for %s", id);
        return 1;
    }
    continue;
}
if (strcmp(id, "gid") == 0) {
    char *group;
    err = snd_config_get_ascii(n, &group);
    if (err < 0) {
        ERROR("Invalid type for %s", id);
        return 1;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1135) - [public] ERROR

```
#####ERROR("Invalid type for %s", id);
#####return 1;
####}
####continue;
###}
###if (strcmp(id, "gid") == 0) {
###char *group;
###err = snd_config_get_ascii(n, &group);
###if (err < 0) {
#####ERROR("Invalid type for %s", id);
#####return 1;
#####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1140) - [public] if

```
###if (strcmp(id, "gid") == 0) {
###char *group;
###err = snd_config_get_ascii(n, &group);
###if (err < 0) {
#####ERROR("Invalid type for %s", id);
#####return 1;
#####}
###if (*group && parse_gid(group, &shm_gid) < 0) {
#####ERROR("unknown group: %s", group);
#####free(group);
#####return 1;
#####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1143) - [public] if

```
#####if (err < 0) {
#####ERROR("Invalid type for %s", id);
#####return 1;
#####}
###if (*group && parse_gid(group, &shm_gid) < 0) {
#####ERROR("unknown group: %s", group);
#####free(group);
#####return 1;
#####}
###free(group);
###continue;
###}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1144) - [public] ERROR

```
#####ERROR("Invalid type for %s", id);
#####return 1;
#####}
###if (*group && parse_gid(group, &shm_gid) < 0) {
#####ERROR("unknown group: %s", group);
#####free(group);
#####return 1;
#####}
###free(group);
###continue;
###}
###ERROR("Unknown field %s", id);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1147) - [public] if

```
    if (*group && parse_gid(group, &shm_gid) < 0) {
        ERROR("unknown group: %s", group);
        free(group);
        return 1;
    }
    free(group);
    continue;
}
ERROR("Unknown field %s", id);
return 1;
}
if (!sockname && port < 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1148) - [public] ERROR

```
    ERROR("unknown group: %s", group);
    free(group);
    return 1;
}
free(group);
continue;
}
ERROR("Unknown field %s", id);
return 1;
}
if (!sockname && port < 0) {
    ERROR("either socket or port need to be defined");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1149) - [public] free

```
    free(group);
    return 1;
}
free(group);
continue;
}
ERROR("Unknown field %s", id);
return 1;
}
if (!sockname && port < 0) {
    ERROR("either socket or port need to be defined");
    return 1;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1152) - [public] free

```
    free(group);
    continue;
}
ERROR("Unknown field %s", id);
return 1;
}
if (!sockname && port < 0) {
    ERROR("either socket or port need to be defined");
    return 1;
}
if (shm_gid == (gid_t)-1) {
    struct group *grp = getgrnam("audio");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1155) - [public] ERROR

```
    ■■■ERROR("Unknown field %s", id);
    ■■return 1;
    ■}
    ■if (!sockname && port < 0) {
    ■■ERROR("either socket or port need to be defined");
    ■■return 1;
    ■}
    ■if (shm_gid == (gid_t)-1) {
    ■■struct group *grp = getgrnam("audio");
    ■■if (!grp) {
    ■■■ERROR("'audio' group not found; use the 'gid' config item to specify a group");
    ■■■return 1;
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1159) - [public] ERROR

```
    ■■ERROR("either socket or port need to be defined");
    ■■return 1;
    ■}
    ■if (shm_gid == (gid_t)-1) {
    ■■struct group *grp = getgrnam("audio");
    ■■if (!grp) {
    ■■■ERROR("'audio' group not found; use the 'gid' config item to specify a group");
    ■■■return 1;
    ■■■}
    ■■shm_gid = grp->gr_gid;
    ■}
    ■server(sockname, port);
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/aserver/aserver.c (line 1164) - [public] if

```
    ■■if (!grp) {
    ■■■ERROR("'audio' group not found; use the 'gid' config item to specify a group");
    ■■■return 1;
    ■■■}
    ■■shm_gid = grp->gr_gid;
    ■}
    ■server(sockname, port);
    ■return 0;
    ■}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 86) - [public]

PyString_InternFromString

```
    ■return PyString_InternFromString(name);
    ■#else
    ■return PyUnicode_InternFromString(name);
    ■#endif
    ■}
```

```
static void *get_C_ptr(PyObject *obj, const char *attr)
{
    ■PyObject *o;
    ■long val;

    ■o = PyObject_GetAttr(obj, InternFromString(attr));
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 88) - [public]

PyUnicode_InternFromString

```
    return PyUnicode_InternFromString(name);
#endif
}

static void *get_C_ptr(PyObject *obj, const char *attr)
{
    PyObject *o;
    long val;

    o = PyObject_GetAttr(obj, InternFromString(attr));
    if (!o) {
        PyErr_Format(PyExc_TypeError, "missing '%s' attribute", attr);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 99) - [public] PyErr_Format

```
PyErr_Format(PyExc_TypeError, "missing '%s' attribute", attr);
return NULL;
}
if (get_long(o, &val)) {
    PyErr_Format(PyExc_TypeError, "'%s' attribute is not Int or Long", attr);
    return NULL;
}
return (void *)val;
}
```

```
static struct pymelem *melem_to_pymelem(snd_mixer_elem_t *elem)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 103) - [public] PyErr_Format

```
PyErr_Format(PyExc_TypeError, "'%s' attribute is not Int or Long", attr);
return NULL;
}
return (void *)val;
}
```

```
static struct pymelem *melem_to_pymelem(snd_mixer_elem_t *elem)
{
    return (struct pymelem *)((char *)snd_mixer_elem_get_private(elem) - offsetof(struct pymelem, selem));
}
```

```
static int pcall(struct pymelem *pymelem, const char *attr, PyObject *args, PyObject **_res)
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 123) - [public] PyErr_Format

```
PyErr_Format(PyExc_TypeError, "missing '%s' attribute", attr);
PyErr_Print();
PyErr_Clear();
Py_DECREF(args);
return -EIO;
}
res = PyObject_CallObject(obj, args);
Py_XDECREF(args);
if (res == NULL) {
    PyErr_Print();
    PyErr_Clear();
    return -EIO;
    // ... (truncated, see source file for more)
}
```


dirtycode/alsa-lib/modules/mixer/simple/python.c (line 124) - [public] PyErr_Print

```
    PyObject_Print(obj, stderr, 1);
    PyErr_Clear();
    Py_DECREF(args);
    return -EIO;
}

res = PyObject_CallObject(obj, args);
Py_XDECREF(args);
if (res == NULL) {
    PyErr_Print();
    PyErr_Clear();
    return -EIO;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 125) - [public] PyErr_Clear

```
    PyErr_Clear();
    Py_DECREF(args);
    return -EIO;
}

res = PyObject_CallObject(obj, args);
Py_XDECREF(args);
if (res == NULL) {
    PyErr_Print();
    PyErr_Clear();
    return -EIO;
}

if (_res && PyTuple_Check(res)) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 126) - [public] Py_DECREF

```
    Py_DECREF(args);
    return -EIO;
}

res = PyObject_CallObject(obj, args);
Py_XDECREF(args);
if (res == NULL) {
    PyErr_Print();
    PyErr_Clear();
    return -EIO;
}

if (_res && PyTuple_Check(res)) {
    *_res = res;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 132) - [public] PyErr_Print

```
    PyErr_Print();
    PyErr_Clear();
    return -EIO;
}

if (_res && PyTuple_Check(res)) {
    *_res = res;
    res = PyTuple_GetItem(res, 0);
}

if (PyLong_Check(res)) {
    xres = PyLong_AsLong(res);
#if PY_MAJOR_VERSION < 3
} else if (PyInt_Check(res)) {
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/modules/mixer/simple/python.c (line 154) - [public] Py_DECREF

```
    Py_DECREF(res);
    if (_res)
        *_res = NULL;
    return -EIO;
}
if (_res && *_res)
    return xres;
Py_DECREF(res);
return xres;
}
```

```
static int is_ops(snd_mixer_elem_t *elem, int dir, int cmd, int val)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 185) - [public] PyTuple_SET_ITEM

```
PyTuple_SET_ITEM(obj1, 0, PyInt_FromLong(dir));
if (xval)
    PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong(val));
}
res = pcall(pymelem, fcn, obj1, NULL);
return res < 0 ? 0 : res;
}
```

```
static int get_x_range_ops(snd_mixer_elem_t *elem, int dir,
    long *min, long *max, const char *attr)
{
    PyObject *obj1, *t1, *t2, *res;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 187) - [public] PyTuple_SET_ITEM

```
PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong(val));
}
res = pcall(pymelem, fcn, obj1, NULL);
return res < 0 ? 0 : res;
}
```

```
static int get_x_range_ops(snd_mixer_elem_t *elem, int dir,
    long *min, long *max, const char *attr)
{
    PyObject *obj1, *t1, *t2, *res;
    struct pymelem *pymelem = melem_to_pymelem(elem);
    int err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 206) - [public] if

```
if (PyLong_Check(t1) && PyLong_Check(t2)) {
    *min = PyLong_AsLong(PyTuple_GetItem(res, 1));
    *max = PyLong_AsLong(PyTuple_GetItem(res, 2));
    err = 0;
}
#if PY_MAJOR_VERSION < 3
} else if (PyInt_Check(t1) && PyInt_Check(t2)) {
    *min = PyInt_AsLong(PyTuple_GetItem(res, 1));
    *max = PyInt_AsLong(PyTuple_GetItem(res, 2));
    err = 0;
}
#endif
} else {
    PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 217) - [public] PyErr_Format

```
    PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
    PyErr_Print();
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
return err;
}
```

```
static int get_range_ops(snd_mixer_elem_t *elem, int dir,
    long *min, long *max)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 218) - [public] PyErr_Print

```
    PyErr_Print();
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
return err;
}
```

```
static int get_range_ops(snd_mixer_elem_t *elem, int dir,
    long *min, long *max)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 219) - [public] PyErr_Clear

```
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
return err;
}
```

```
static int get_range_ops(snd_mixer_elem_t *elem, int dir,
    long *min, long *max)
{
    return get_x_range_ops(elem, dir, min, max, "opsGetRange");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 230) - [public] get_x_range_ops

```
    return get_x_range_ops(elem, dir, min, max, "opsGetRange");
}
```

```
static int set_range_ops(snd_mixer_elem_t *elem, int dir,
    long min, long max)
{
    PyObject *obj1;
    struct pymelem *pymelem = melem_to_pymelem(elem);

    obj1 = PyTuple_New(3);
    PyTuple_SET_ITEM(obj1, 0, PyInt_FromLong(dir));
    PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong(min));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 243) - [public] pcall

```
    return pcall(pymelem, "opsGetRange", obj1, NULL);
}

static int get_x_ops(snd_mixer_elem_t *elem, int dir,
    long channel, long *value,
    const char *attr)
{
    PyObject *obj1, *t1, *res;
    struct pymelem *pymelem = melem_to_pymelem(elem);
    int err;

    obj1 = PyTuple_New(2);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 260) - [public] if

```
    if (PyLong_Check(t1)) {
        *value = PyLong_AsLong(t1);
        err = 0;
    #if PY_MAJOR_VERSION < 3
    } else if (PyInt_Check(t1)) {
        *value = PyInt_AsLong(t1);
        err = 0;
    #endif
    } else {
        PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
        PyErr_Print();
        PyErr_Clear();
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 269) - [public] PyErr_Format

```
        PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
        PyErr_Print();
        PyErr_Clear();
        err = -EIO;
    }
}
Py_XDECREF(res);
return err;
}

static int get_volume_ops(snd_mixer_elem_t *elem, int dir,
    snd_mixer_selem_channel_id_t channel, long *value)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 270) - [public] PyErr_Print

```
        PyErr_Print();
        PyErr_Clear();
        err = -EIO;
    }
}
Py_XDECREF(res);
return err;
}

static int get_volume_ops(snd_mixer_elem_t *elem, int dir,
    snd_mixer_selem_channel_id_t channel, long *value)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 271) - [public] PyErr_Clear

```
    ■■■PyErr_Clear();
    ■■■err = -EIO;
    ■■}
    ■}
    ■Py_XDECREF(res);
    ■return err;
    }

    static int get_volume_ops(snd_mixer_elem_t *elem, int dir,
    ■■■ snd_mixer_selem_channel_id_t channel, long *value)
    {
    ■return get_x_ops(elem, dir, channel, value, "opsGetVolume");
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 282) - [public] get_x_ops

```
    ■return get_x_ops(elem, dir, channel, value, "opsGetVolume");
    }

    static int get_switch_ops(snd_mixer_elem_t *elem, int dir,
    ■■■ snd_mixer_selem_channel_id_t channel, int *value)
    {
    ■long value1;
    ■int res;
    ■res = get_x_ops(elem, dir, channel, &value1, "opsGetSwitch");
    ■*value = value1;
    ■return res;
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 300) - [public] get_x_ops

```
    ■return get_x_ops(elem, dir, value, dbValue, "opsGetVolDB");
    }

    static int ask_dB_vol_ops(snd_mixer_elem_t *elem,
    ■■■ int dir,
    ■■■ long value,
    ■■■ long *dbValue,
    ■■■ int xdir)
    {
    ■PyObject *obj1, *t1, *res;
    ■struct pymelem *pymelem = melem_to_pymelem(elem);
    ■int err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 320) - [public] if

```
    ■■if (PyLong_Check(t1)) {
    ■■■*dbValue = PyLong_AsLong(t1);
    ■■■err = 0;
    ■#if PY_MAJOR_VERSION < 3
    ■■} else if (PyInt_Check(t1)) {
    ■■■*dbValue = PyInt_AsLong(t1);
    ■■■err = 0;
    ■#endif
    ■■} else {
    ■■■PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
    ■■■PyErr_Print();
    ■■■PyErr_Clear();
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 329) - [public] PyErr_Format

```
    PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
    PyErr_Print();
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
return err;
}
```

```
static int get_dB_ops(snd_mixer_elem_t *elem,
    int dir,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 330) - [public] PyErr_Print

```
    PyErr_Print();
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
return err;
}
```

```
static int get_dB_ops(snd_mixer_elem_t *elem,
    int dir,
    snd_mixer_selem_channel_id_t channel,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 331) - [public] PyErr_Clear

```
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
return err;
}
```

```
static int get_dB_ops(snd_mixer_elem_t *elem,
    int dir,
    snd_mixer_selem_channel_id_t channel,
    long *value)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 344) - [public] get_x_ops

```
return get_x_ops(elem, dir, channel, value, "opsGetDB");
}
```

```
static int get_dB_range_ops(snd_mixer_elem_t *elem, int dir,
    long *min, long *max)
{
return get_x_range_ops(elem, dir, min, max, "opsGetDBRange");
}
```

```
static int set_volume_ops(snd_mixer_elem_t *elem, int dir,
    snd_mixer_selem_channel_id_t channel, long value)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 350) - [public] get_x_range_ops

```
    return get_x_range_ops(elem, dir, min, max, "opsGetDBRange");
}

static int set_volume_ops(snd_mixer_elem_t *elem, int dir,
    snd_mixer_selem_channel_id_t channel, long value)
{
    PyObject *obj1;
    struct pymelem *pymelem = melem_to_pymelem(elem);

    obj1 = PyTuple_New(3);
    PyTuple_SET_ITEM(obj1, 0, PyInt_FromLong(dir));
    PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong(channel));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 363) - [public] pcall

```
    return pcall(pymelem, "opsSetVolume", obj1, NULL);
}

static int set_switch_ops(snd_mixer_elem_t *elem, int dir,
    snd_mixer_selem_channel_id_t channel, int value)
{
    PyObject *obj1;
    struct pymelem *pymelem = melem_to_pymelem(elem);

    obj1 = PyTuple_New(3);
    PyTuple_SET_ITEM(obj1, 0, PyInt_FromLong(dir));
    PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong(channel));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 376) - [public] pcall

```
    return pcall(pymelem, "opsSetSwitch", obj1, NULL);
}

static int set_db_ops(snd_mixer_elem_t *elem, int dir,
    snd_mixer_selem_channel_id_t channel,
    long db_gain, int xdir)
{
    PyObject *obj1;
    struct pymelem *pymelem = melem_to_pymelem(elem);

    obj1 = PyTuple_New(4);
    PyTuple_SET_ITEM(obj1, 0, PyInt_FromLong(dir));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 391) - [public] pcall

```
    return pcall(pymelem, "opsSetDB", obj1, NULL);
}

static int enum_item_name_ops(snd_mixer_elem_t *elem,
    unsigned int item,
    size_t maxlen, char *buf)
{
    PyObject *obj1, *obj2, *t1, *res;
    struct pymelem *pymelem = melem_to_pymelem(elem);
    int err;
    unsigned int len;
    char *s;
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/modules/mixer/simple/python.c (line 409) - [public] if

```
    if (PyUnicode_Check(t1)) {
        obj2 = PyUnicode_AsEncodedString(t1, "utf-8", "strict");
        if (obj2) {
            s = PyBytes_AsString(obj2);
            len = strlen(s);
            if (maxlen - 1 > len)
                len = maxlen - 1;
            memcpy(buf, s, len);
            buf[len] = '\0';
            Py_DECREF(obj2);
        } else {
            goto errlbl;
        }
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 411) - [public] if

```
    if (obj2) {
        s = PyBytes_AsString(obj2);
        len = strlen(s);
        if (maxlen - 1 > len)
            len = maxlen - 1;
        memcpy(buf, s, len);
        buf[len] = '\0';
        Py_DECREF(obj2);
    } else {
        goto errlbl;
    }
    #if PY_MAJOR_VERSION < 3
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 416) - [public] memcpy

```
    memcpy(buf, s, len);
    buf[len] = '\0';
    Py_DECREF(obj2);
} else {
    goto errlbl;
}
    #if PY_MAJOR_VERSION < 3
    } else if (PyString_Check(t1)) {
        s = PyString_AsString(t1);
        len = strlen(s);
        if (maxlen - 1 > len)
            len = maxlen - 1;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 418) - [public] Py_DECREF

```
    Py_DECREF(obj2);
} else {
    goto errlbl;
}
    #if PY_MAJOR_VERSION < 3
    } else if (PyString_Check(t1)) {
        s = PyString_AsString(t1);
        len = strlen(s);
        if (maxlen - 1 > len)
            len = maxlen - 1;
        memcpy(buf, s, len);
        buf[len] = '\0';
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 428) - [public] memcpy

```
    memcpy(buf, s, len);
    buf[len] = '\0';
#endif
} else {
errlbl:
    PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
    PyErr_Print();
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 433) - [public] PyErr_Format

```
    PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
    PyErr_Print();
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
return err;
}
```

```
static int get_enum_item_ops(snd_mixer_elem_t *elem,
    snd_mixer_selem_channel_id_t channel,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 434) - [public] PyErr_Print

```
    PyErr_Print();
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
return err;
}
```

```
static int get_enum_item_ops(snd_mixer_elem_t *elem,
    snd_mixer_selem_channel_id_t channel,
    unsigned int *itemp)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 435) - [public] PyErr_Clear

```
    PyErr_Clear();
    err = -EIO;
}
}
Py_XDECREF(res);
return err;
}
```

```
static int get_enum_item_ops(snd_mixer_elem_t *elem,
    snd_mixer_selem_channel_id_t channel,
    unsigned int *itemp)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 456) - [public] if

```
    if (PyLong_Check(t1)) {
        *itemp = PyLong_AsLong(t1);
        err = 0;
    #if PY_MAJOR_VERSION < 3
    } else if (PyInt_Check(t1)) {
        *itemp = PyInt_AsLong(t1);
        err = 0;
    #endif
    } else {
        PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
        PyErr_Print();
        PyErr_Clear();
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 465) - [public] PyErr_Format

```
    PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
    PyErr_Print();
    PyErr_Clear();
    err = -EIO;
}

Py_XDECREF(res);
return err;
}
```

```
static int set_enum_item_ops(snd_mixer_elem_t *elem,
    snd_mixer_selem_channel_id_t channel,
    // ... (truncated, see source file for more)
)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 466) - [public] PyErr_Print

```
    PyErr_Print();
    PyErr_Clear();
    err = -EIO;
}

Py_XDECREF(res);
return err;
}
```

```
static int set_enum_item_ops(snd_mixer_elem_t *elem,
    snd_mixer_selem_channel_id_t channel,
    unsigned int item)
    // ... (truncated, see source file for more)
)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 467) - [public] PyErr_Clear

```
    PyErr_Clear();
    err = -EIO;
}

Py_XDECREF(res);
return err;
}
```

```
static int set_enum_item_ops(snd_mixer_elem_t *elem,
    snd_mixer_selem_channel_id_t channel,
    unsigned int item)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 485) - [public] pcall

```
    return pcall(pymelem, "opsSetEnumItem", obj1, NULL);
}
```

```
static struct sm_elem_ops simple_python_ops = {
    .is = is_ops,
    .get_range = get_range_ops,
    .get_dB_range = get_dB_range_ops,
    .set_range = set_range_ops,
    .ask_vol_dB = ask_vol_dB_ops,
    .ask_dB_vol = ask_dB_vol_ops,
    .get_volume = get_volume_ops,
    .get_dB = get_dB_ops,
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 511) - [public] snd_mixer_selem_id_free

```
    snd_mixer_selem_id_free(simple->id);
    simple->id = NULL;
}
```

```
static PyObject *
pymelem_cap(struct pymelem *pymelem ATTRIBUTE_UNUSED, void *priv)
{
    return PyInt_FromLong((long)priv);
}
```

```
static PyObject *
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 519) - [public] PyInt_FromLong

```
    return PyInt_FromLong((long)priv);
}
```

```
static PyObject *
pymelem_get_caps(struct pymelem *pymelem, void *priv ATTRIBUTE_UNUSED)
{
    return PyInt_FromLong(pymelem->selem.caps);
}
```

```
static PyObject *
pymelem_get_name(struct pymelem *pymelem, void *priv ATTRIBUTE_UNUSED)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 525) - [public] PyInt_FromLong

```
    return PyInt_FromLong(pymelem->selem.caps);
}
```

```
static PyObject *
pymelem_get_name(struct pymelem *pymelem, void *priv ATTRIBUTE_UNUSED)
{
    return PyUnicode_FromString(snd_mixer_selem_id_get_name(pymelem->selem.id));
}
```

```
static PyObject *
pymelem_get_index(struct pymelem *pymelem, void *priv ATTRIBUTE_UNUSED)
{
    // ... (truncated, see source file for more)
}
```

```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 531) - [public] PyUnicode_FromString
    return PyUnicode_FromString(snd_mixer_selem_id_get_name(pymelem->selem.id));
}

static PyObject *
pymelem_get_index(struct pymelem *pymelem, void *priv ATTRIBUTE_UNUSED)
{
    return PyInt_FromLong(snd_mixer_selem_id_get_index(pymelem->selem.id));
}

static int
pymelem_set_caps(struct pymelem *pymelem, PyObject *val, void *priv ATTRIBUTE_UNUSED)
{
    // ... (truncated, see source file for more)
}

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 537) - [public] PyInt_FromLong
    return PyInt_FromLong(snd_mixer_selem_id_get_index(pymelem->selem.id));
}

static int
pymelem_set_caps(struct pymelem *pymelem, PyObject *val, void *priv ATTRIBUTE_UNUSED)
{
    if (PyLong_Check(val)) {
        pymelem->selem.caps = PyLong_AsLong(val);
        return 0;
    }
    #if PY_MAJOR_VERSION < 3
    if (PyInt_Check(val)) {
        // ... (truncated, see source file for more)
    }
}

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 572) - [public] PyInt_FromLong
    return PyInt_FromLong(-EIO);
}

static PyObject *
pymelem_attach(struct pymelem *pymelem, PyObject *args)
{
    PyObject *obj;
    snd_hctl_elem_t *helem;
    int err;

    if (!PyArg_ParseTuple(args, "O", &obj))
        return NULL;
    // ... (truncated, see source file for more)
}

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 589) - [public] PyErr_Format
    PyErr_Format(PyExc_RuntimeError, "Cannot attach hcontrol element to mixer element: %s", snd_strerror(err));
    return NULL;
}
Py_RETURN_NONE;
}

static PyObject *
pymelem_detach(struct pymelem *pymelem, PyObject *args)
{
    PyObject *obj;
    snd_hctl_elem_t *helem;
    int err;
    // ... (truncated, see source file for more)
}

```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 609) - [public] PyErr_Format

```
    PyErr_Format(PyExc_RuntimeError, "Cannot detach hcontrol element to mixer element: %s", snd_strerror(e));
    return NULL;
}
Py_RETURN_NONE;
}
```

```
static PyObject *
pymelem_event_info(struct pymelem *pymelem, PyObject *args)
{
    if (!PyArg_ParseTuple(args, ""))
        return NULL;
    return PyInt_FromLong(snd_mixer_elem_info(pymelem->melem));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 620) - [public] PyInt_FromLong

```
return PyInt_FromLong(snd_mixer_elem_info(pymelem->melem));
}
```

```
static PyObject *
pymelem_event_value(struct pymelem *pymelem, PyObject *args)
{
    if (!PyArg_ParseTuple(args, ""))
        return NULL;
    return PyInt_FromLong(snd_mixer_elem_value(pymelem->melem));
}
```

```
static int
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 628) - [public] PyInt_FromLong

```
return PyInt_FromLong(snd_mixer_elem_value(pymelem->melem));
}
```

```
static int
pymelem_init(struct pymelem *pymelem, PyObject *args, PyObject *kwargs ATTRIBUTE_UNUSED)
{
    char *name;
    long index, weight;
    snd_mixer_selem_id_t *id;
    int err;

    if (!PyArg_ParseTuple(args, "Osii", &pymelem->py_mixer, &name, &index, &weight))
        // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 651) - [public] snd_mixer_selem_id_free

```
snd_mixer_selem_id_free(id);
return -1;
}
return 0;
}
```

```
static void
pymelem_dealloc(struct pymelem *self)
{
    selem_free(self->melem);
}
```

```
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 735) - [public] PyErr_Format

```
    ■■PyErr_Format(PyExc_RuntimeError, "Cannot attach hctl: %s", snd_strerror(err));
    ■■return NULL;
    ■}
    ■hctls = realloc(pymixer->hctl, sizeof(void *) * (pymixer->hctl_count+1) * 2);
    ■if (hctls == NULL) {
    ■■PyErr_SetString(PyExc_RuntimeError, "No enough memory");
    ■■return NULL;
    ■}
    ■pymixer->hctl = hctls;
    ■pymixer->hctl[pymixer->hctl_count*2] = (void *)hctl;
    ■pymixer->hctl[pymixer->hctl_count*2+1] = (void *)obj;
    ■pymixer->hctl_count++;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 740) - [public] PyErr_SetString

```
    ■■PyErr_SetString(PyExc_RuntimeError, "No enough memory");
    ■■return NULL;
    ■}
    ■pymixer->hctl = hctls;
    ■pymixer->hctl[pymixer->hctl_count*2] = (void *)hctl;
    ■pymixer->hctl[pymixer->hctl_count*2+1] = (void *)obj;
    ■pymixer->hctl_count++;
    ■Py_INCREF(obj);
    ■Py_RETURN_NONE;
    }
```

```
static PyObject *
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 760) - [public] PyErr_Format

```
    ■■PyErr_Format(PyExc_RuntimeError, "Cannot register mixer: %s", snd_strerror(err));
    ■■return NULL;
    ■}
    ■Py_RETURN_NONE;
    }
```

```
static PyObject *
pymixer_melement_new(struct pymixer *pymixer, PyObject *args)
{
    ■PyObject *obj, *obj1, *obj2;
    ■char *class, *name;
    ■long index, weight;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 778) - [public] PyTuple_SET_ITEM

```
    ■■PyTuple_SET_ITEM(obj1, 0, (PyObject *)pymixer);
    ■■Py_INCREF((PyObject *)pymixer);
    ■■PyTuple_SET_ITEM(obj1, 1, PyUnicode_FromString(name));
    ■■PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong(index));
    ■■PyTuple_SET_ITEM(obj1, 3, PyInt_FromLong(weight));
    ■■obj2 = PyObject_CallObject(obj, obj1);
    ■■Py_XDECREF(obj1);
    ■■if (obj2) {
    ■■■■struct pymelem *pymelem = (struct pymelem *)obj2;
    ■■■■void **melems = realloc(pymixer->melem, sizeof(void *) * (pymixer->melem_count + 1) * 2);
    ■■■■if (melems == NULL) {
    ■■■■Py_DECREF(obj2);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 779) - [public] Py_INCREF

```
    Py_INCREF((PyObject *)pymixer);
    PyTuple_SET_ITEM(obj1, 1, PyUnicode_FromString(name));
    PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong(index));
    PyTuple_SET_ITEM(obj1, 3, PyInt_FromLong(weight));
    obj2 = PyObject_CallObject(obj, obj1);
    Py_XDECREF(obj1);
    if (obj2) {
        struct pymelem *pymelem = (struct pymelem *)obj2;
        void **melems = realloc(pymixer->melem, sizeof(void *) * (pymixer->melem_count + 1) * 2);
        if (melems == NULL) {
            Py_DECREF(obj2);
            return NULL;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 780) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(obj1, 1, PyUnicode_FromString(name));
    PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong(index));
    PyTuple_SET_ITEM(obj1, 3, PyInt_FromLong(weight));
    obj2 = PyObject_CallObject(obj, obj1);
    Py_XDECREF(obj1);
    if (obj2) {
        struct pymelem *pymelem = (struct pymelem *)obj2;
        void **melems = realloc(pymixer->melem, sizeof(void *) * (pymixer->melem_count + 1) * 2);
        if (melems == NULL) {
            Py_DECREF(obj2);
            return NULL;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 781) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong(index));
    PyTuple_SET_ITEM(obj1, 3, PyInt_FromLong(weight));
    obj2 = PyObject_CallObject(obj, obj1);
    Py_XDECREF(obj1);
    if (obj2) {
        struct pymelem *pymelem = (struct pymelem *)obj2;
        void **melems = realloc(pymixer->melem, sizeof(void *) * (pymixer->melem_count + 1) * 2);
        if (melems == NULL) {
            Py_DECREF(obj2);
            return NULL;
        }
        melems[pymixer->melem_count*2] = pymelem->melem;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 782) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(obj1, 3, PyInt_FromLong(weight));
    obj2 = PyObject_CallObject(obj, obj1);
    Py_XDECREF(obj1);
    if (obj2) {
        struct pymelem *pymelem = (struct pymelem *)obj2;
        void **melems = realloc(pymixer->melem, sizeof(void *) * (pymixer->melem_count + 1) * 2);
        if (melems == NULL) {
            Py_DECREF(obj2);
            return NULL;
        }
        melems[pymixer->melem_count*2] = pymelem->melem;
        melems[pymixer->melem_count*2+1] = obj2;
        // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/modules/mixer/simple/python.c (line 784) - [public] Py_XDECREF

```
    Py_XDECREF(obj1);
    if (obj2) {
        struct pymelem *pymelem = (struct pymelem *)obj2;
        void **melems = realloc(pymixer->melem, sizeof(void *) * (pymixer->melem_count + 1) * 2);
        if (melems == NULL) {
            Py_DECREF(obj2);
            return NULL;
        }
        melems[pymixer->melem_count*2] = pymelem->melem;
        melems[pymixer->melem_count*2+1] = obj2;
        Py_INCREF(obj2);
        pymixer->melem = melems;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 785) - [public] if

```
    if (obj2) {
        struct pymelem *pymelem = (struct pymelem *)obj2;
        void **melems = realloc(pymixer->melem, sizeof(void *) * (pymixer->melem_count + 1) * 2);
        if (melems == NULL) {
            Py_DECREF(obj2);
            return NULL;
        }
        melems[pymixer->melem_count*2] = pymelem->melem;
        melems[pymixer->melem_count*2+1] = obj2;
        Py_INCREF(obj2);
        pymixer->melem = melems;
        pymixer->melem_count++;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 788) - [public] if

```
    if (melems == NULL) {
        Py_DECREF(obj2);
        return NULL;
    }
    melems[pymixer->melem_count*2] = pymelem->melem;
    melems[pymixer->melem_count*2+1] = obj2;
    Py_INCREF(obj2);
    pymixer->melem = melems;
    pymixer->melem_count++;
} else {
    PyErr_Format(PyExc_RuntimeError, "Cannot find class '%s'", class);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 789) - [public] Py_DECREF

```
    Py_DECREF(obj2);
    return NULL;
}
melems[pymixer->melem_count*2] = pymelem->melem;
melems[pymixer->melem_count*2+1] = obj2;
Py_INCREF(obj2);
pymixer->melem = melems;
pymixer->melem_count++;
} else {
    PyErr_Format(PyExc_RuntimeError, "Cannot find class '%s'", class);
    return NULL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 794) - [public] Py_INCREF

```
    Py_INCREF(obj2);
    pymixer->melem = melems;
    pymixer->melem_count++;
}
} else {
    PyErr_Format(PyExc_RuntimeError, "Cannot find class '%s'", class);
    return NULL;
}
return obj2;
}

static PyObject *
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 799) - [public] PyErr_Format

```
    PyErr_Format(PyExc_RuntimeError, "Cannot find class '%s'", class);
    return NULL;
}
return obj2;
}

static PyObject *
pymixer_melement_add(struct pymixer *pymixer, PyObject *args)
{
    PyObject *obj;
    struct pymelem *pymelem;
    int err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 817) - [public] PyErr_Format

```
    PyErr_Format(PyExc_RuntimeError, "Cannot add mixer element: %s", snd_strerror(err));
    return NULL;
}
Py_RETURN_NONE;
}

static int
pymixer_init(struct pymixer *pymixer, PyObject *args, PyObject *kwargs ATTRIBUTE_UNUSED)
{
    long class, mixer;

    if (!PyArg_ParseTuple(args, "ii0", &class, &mixer, &pymixer->mdict))
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 847) - [public] snd_mixer_detach_hctl

```
    snd_mixer_detach_hctl(self->mixer, self->hctl[idx*2]);
    Py_DECREF((PyObject *)self->hctl[idx*2+1]);
}
if (self->hctl)
    free(self->hctl);
self->hctl_count = 0;
self->hctl = NULL;
for (idx = 0; idx < self->helem_count; idx++)
    Py_DECREF((PyObject *)self->helem[idx*2+1]);
if (self->helem)
    free(self->helem);
self->helem_count = 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 848) - [public] Py_DECREF

```
    Py_DECREF((PyObject *)self->hctl[idx*2+1]);
}
if (self->hctl)
    free(self->hctl);
self->hctl_count = 0;
self->hctl = NULL;
for (idx = 0; idx < self->helem_count; idx++)
    Py_DECREF((PyObject *)self->helem[idx*2+1]);
if (self->helem)
    free(self->helem);
self->helem_count = 0;
self->helem = NULL;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 851) - [public] free

```
    free(self->hctl);
self->hctl_count = 0;
self->hctl = NULL;
for (idx = 0; idx < self->helem_count; idx++)
    Py_DECREF((PyObject *)self->helem[idx*2+1]);
if (self->helem)
    free(self->helem);
self->helem_count = 0;
self->helem = NULL;
for (idx = 0; idx < self->melem_count; idx++)
    Py_DECREF((PyObject *)self->melem[idx*2+1]);
if (self->melem)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 855) - [public] Py_DECREF

```
    Py_DECREF((PyObject *)self->helem[idx*2+1]);
if (self->helem)
    free(self->helem);
self->helem_count = 0;
self->helem = NULL;
for (idx = 0; idx < self->melem_count; idx++)
    Py_DECREF((PyObject *)self->melem[idx*2+1]);
if (self->melem)
    free(self->melem);
self->melem_count = 0;
self->melem = NULL;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 857) - [public] free

```
    free(self->helem);
self->helem_count = 0;
self->helem = NULL;
for (idx = 0; idx < self->melem_count; idx++)
    Py_DECREF((PyObject *)self->melem[idx*2+1]);
if (self->melem)
    free(self->melem);
self->melem_count = 0;
self->melem = NULL;
}
```

```
static void
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 861) - [public] Py_DECREF

```
    Py_DECREF((PyObject *)self->melem[idx*2+1]);
    if (self->melem)
        free(self->melem);
    self->melem_count = 0;
    self->melem = NULL;
}

static void
pymixer_dealloc(struct pymixer *self)
{
    pymixer_free(self);
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 863) - [public] free

```
    free(self->melem);
    self->melem_count = 0;
    self->melem = NULL;
}

static void
pymixer_dealloc(struct pymixer *self)
{
    pymixer_free(self);
}

static PyGetSetDef pymixer_getseters[] = {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 913) - [public] if

```
    if (pymixer->hctl[idx] == hctl) {
        py_hctl = pymixer->hctl[idx*2+1];
        break;
    }
    if (py_hctl == NULL)
        return NULL;
    obj = PyDict_GetItemString(priv->py_mdict, "HElement");
    if (obj) {
        obj1 = PyTuple_New(3);
        PyTuple_SET_ITEM(obj1, 0, py_hctl);
        Py_INCREF(py_hctl);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 923) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(obj1, 0, py_hctl);
    Py_INCREF(py_hctl);
    PyTuple_SET_ITEM(obj1, 1, PyFloat_FromDouble(1));
    PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong((long)helem));
    obj2 = PyObject_CallObject(obj, obj1);
    if (obj2 == NULL) {
        PyErr_Print();
        PyErr_Clear();
    }
    Py_XDECREF(obj1);
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 924) - [public] Py_INCREF

```
    Py_INCREF(py_hctl);
    PyTuple_SET_ITEM(obj1, 1, PyFloat_FromDouble(1));
    PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong((long)helem));
    obj2 = PyObject_CallObject(obj, obj1);
    if (obj2 == NULL) {
        PyErr_Print();
        PyErr_Clear();
    }
    Py_XDECREF(obj1);
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return NULL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 925) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(obj1, 1, PyFloat_FromDouble(1));
    PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong((long)helem));
    obj2 = PyObject_CallObject(obj, obj1);
    if (obj2 == NULL) {
        PyErr_Print();
        PyErr_Clear();
    }
    Py_XDECREF(obj1);
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return NULL;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 926) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong((long)helem));
    obj2 = PyObject_CallObject(obj, obj1);
    if (obj2 == NULL) {
        PyErr_Print();
        PyErr_Clear();
    }
    Py_XDECREF(obj1);
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return NULL;
}
if (obj2) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 928) - [public] if

```
    if (obj2 == NULL) {
        PyErr_Print();
        PyErr_Clear();
    }
    Py_XDECREF(obj1);
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return NULL;
}
if (obj2) {
    struct pymixer *pymixer = (struct pymixer *)priv->py_mixer;
    void **helems = realloc(pymixer->helem, sizeof(void *) * (pymixer->helem_count + 1) * 2);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 929) - [public] PyErr_Print

```
    Py_DECREF(obj1);
    PyErr_Clear();
}
Py_XDECREF(obj1);
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return NULL;
}
if (obj2) {
    struct pymixer *pymixer = (struct pymixer *)priv->py_mixer;
    void **helems = realloc(pymixer->helem, sizeof(void *) * (pymixer->helem_count + 1) * 2);
    if (helems == NULL) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 930) - [public] PyErr_Clear

```
    PyErr_Clear();
}
Py_XDECREF(obj1);
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return NULL;
}
if (obj2) {
    struct pymixer *pymixer = (struct pymixer *)priv->py_mixer;
    void **helems = realloc(pymixer->helem, sizeof(void *) * (pymixer->helem_count + 1) * 2);
    if (helems == NULL) {
        Py_DECREF(obj2);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 932) - [public] Py_XDECREF

```
    Py_XDECREF(obj1);
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return NULL;
}
if (obj2) {
    struct pymixer *pymixer = (struct pymixer *)priv->py_mixer;
    void **helems = realloc(pymixer->helem, sizeof(void *) * (pymixer->helem_count + 1) * 2);
    if (helems == NULL) {
        Py_DECREF(obj2);
        return NULL;
    }
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 934) - [public] snd_error

```
    snd_error(MIXER, "Unable to create InternalMixer object");
    return NULL;
}
if (obj2) {
    struct pymixer *pymixer = (struct pymixer *)priv->py_mixer;
    void **helems = realloc(pymixer->helem, sizeof(void *) * (pymixer->helem_count + 1) * 2);
    if (helems == NULL) {
        Py_DECREF(obj2);
        return NULL;
    }
    helems[pymixer->helem_count*2] = helem;
    helems[pymixer->helem_count*2+1] = obj2;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 940) - [public] if

```
    if (helems == NULL) {
        Py_DECREF(obj2);
        return NULL;
    }
    helems[pymixer->helem_count*2] = helem;
    helems[pymixer->helem_count*2+1] = obj2;
    Py_INCREF(obj2);
    pymixer->helem = helems;
    pymixer->helem_count++;
}
return obj2;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 941) - [public] Py_DECREF

```
    Py_DECREF(obj2);
    return NULL;
}
helems[pymixer->helem_count*2] = helem;
helems[pymixer->helem_count*2+1] = obj2;
Py_INCREF(obj2);
pymixer->helem = helems;
pymixer->helem_count++;
}
return obj2;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 946) - [public] Py_INCREF

```
    Py_INCREF(obj2);
    pymixer->helem = helems;
    pymixer->helem_count++;
}
return obj2;
}

static PyObject *find_helem(struct python_priv *priv, snd_htcl_elem_t *helem)
{
    struct pymixer *pymixer = (struct pymixer *)priv->py_mixer;
    int idx;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 990) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(t, 0, (PyObject *)PyInt_FromLong(mask));
    o = find_helem(priv, helem);
    if (mask & SND_CTL_EVENT_MASK_ADD) {
        if (o == NULL)
            o = new_helem(priv, helem);
    }
    if (o == NULL)
        return 0;
    PyTuple_SET_ITEM(t, 1, o);
    Py_INCREF(o);
    o = melem ? find_melem(priv, melem) : Py_None;
    PyTuple_SET_ITEM(t, 2, o);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 992) - [public] if

```
    if (mask & SND_CTL_EVENT_MASK_ADD) {
        if (o == NULL)
            o = new_helem(priv, helem);
    }
    if (o == NULL)
        return 0;
    PyTuple_SET_ITEM(t, 1, o);
    Py_INCREF(o);
    o = melem ? find_melem(priv, melem) : Py_None;
    PyTuple_SET_ITEM(t, 2, o);
    Py_INCREF(o);
    r = PyObject_CallObject(priv->py_event_func, t);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 998) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(t, 1, o);
    Py_INCREF(o);
    o = melem ? find_melem(priv, melem) : Py_None;
    PyTuple_SET_ITEM(t, 2, o);
    Py_INCREF(o);
    r = PyObject_CallObject(priv->py_event_func, t);
    Py_DECREF(t);
    if (r) {
        if (PyLong_Check(r)) {
            res = PyLong_AsLong(r);
        }
        #if PY_MAJOR_VERSION < 3
        } else if (PyInt_Check(r)) {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 999) - [public] Py_INCREF

```
    Py_INCREF(o);
    o = melem ? find_melem(priv, melem) : Py_None;
    PyTuple_SET_ITEM(t, 2, o);
    Py_INCREF(o);
    r = PyObject_CallObject(priv->py_event_func, t);
    Py_DECREF(t);
    if (r) {
        if (PyLong_Check(r)) {
            res = PyLong_AsLong(r);
        }
        #if PY_MAJOR_VERSION < 3
        } else if (PyInt_Check(r)) {
            res = PyInt_AsLong(r);
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1001) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(t, 2, o);
    Py_INCREF(o);
    r = PyObject_CallObject(priv->py_event_func, t);
    Py_DECREF(t);
    if (r) {
        if (PyLong_Check(r)) {
            res = PyLong_AsLong(r);
        }
        #if PY_MAJOR_VERSION < 3
        } else if (PyInt_Check(r)) {
            res = PyInt_AsLong(r);
        }
    }
    #endif
    } else if (r == Py_None) {
        // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1002) - [public] Py_INCREF

```
    Py_INCREF(o);
    r = PyObject_CallObject(priv->py_event_func, t);
    Py_DECREF(t);
    if (r) {
        if (PyLong_Check(r)) {
            res = PyLong_AsLong(r);
        }
        #if PY_MAJOR_VERSION < 3
        } else if (PyInt_Check(r)) {
            res = PyInt_AsLong(r);
        }
        #endif
    } else if (r == Py_None) {
        res = 0;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1004) - [public] Py_DECREF

```
    Py_DECREF(t);
    if (r) {
        if (PyLong_Check(r)) {
            res = PyLong_AsLong(r);
        }
        #if PY_MAJOR_VERSION < 3
        } else if (PyInt_Check(r)) {
            res = PyInt_AsLong(r);
        }
        #endif
    } else if (r == Py_None) {
        res = 0;
    }
    Py_DECREF(r);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1005) - [public] if

```
    if (r) {
        if (PyLong_Check(r)) {
            res = PyLong_AsLong(r);
        }
        #if PY_MAJOR_VERSION < 3
        } else if (PyInt_Check(r)) {
            res = PyInt_AsLong(r);
        }
        #endif
    } else if (r == Py_None) {
        res = 0;
    }
    Py_DECREF(r);
} else {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1006) - [public] if

```
    if (PyLong_Check(r)) {
        res = PyLong_AsLong(r);
    }
    #if PY_MAJOR_VERSION < 3
    } else if (PyInt_Check(r)) {
        res = PyInt_AsLong(r);
    }
    #endif
    } else if (r == Py_None) {
        res = 0;
    }
    Py_DECREF(r);
} else {
    PyErr_Print();
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1015) - [public] Py_DECREF

```
    Py_DECREF(r);
    } else {
        PyErr_Print();
        PyErr_Clear();
        res = -EIO;
    }

    return res;
}

static void alsa_mixer_simple_free(snd_mixer_class_t *class)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1017) - [public] PyErr_Print

```
    PyErr_Print();
    PyErr_Clear();
    res = -EIO;
}

return res;
}

static void alsa_mixer_simple_free(snd_mixer_class_t *class)
{
    struct python_priv *priv = snd_mixer_sbasic_get_private(class);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1018) - [public] PyErr_Clear

```
    PyErr_Clear();
    res = -EIO;
}

return res;
}

static void alsa_mixer_simple_free(snd_mixer_class_t *class)
{
    struct python_priv *priv = snd_mixer_sbasic_get_private(class);

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1031) - [public] pymixer_free

```
    pymixer_free((struct pymixer *)priv->py_mixer);
    Py_DECREF(priv->py_mixer);
}
if (priv->py_initialized) {
    Py_XDECREF(priv->py_event_func);
    Py_Finalize();
}
free(priv);
}

static int alsa_mixer_simple_pyinit(struct python_priv *priv,
    PyObject *py_mod,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1032) - [public] Py_DECREF

```
    Py_DECREF(priv->py_mixer);
}
if (priv->py_initialized) {
    Py_XDECREF(priv->py_event_func);
    Py_Finalize();
}
free(priv);
}

static int alsa_mixer_simple_pyinit(struct python_priv *priv,
    PyObject *py_mod,
    FILE *fp,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1035) - [public] Py_XDECREF

```
    Py_XDECREF(priv->py_event_func);
    Py_Finalize();
}
free(priv);
}

static int alsa_mixer_simple_pyinit(struct python_priv *priv,
    PyObject *py_mod,
    FILE *fp,
    const char *file,
    snd_mixer_class_t *class,
    snd_mixer_t *mixer,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1036) - [public] Py_Finalize

```
    Py_Finalize();
}
free(priv);
}

static int alsa_mixer_simple_pyinit(struct python_priv *priv,
    PyObject *py_mod,
    FILE *fp,
    const char *file,
    snd_mixer_class_t *class,
    snd_mixer_t *mixer,
    const char *device)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1054) - [public] PyDict_SetItemString

```
    PyDict_SetItemString(mdict, "__file__", obj);
    Py_XDECREF(obj);
    obj = PyUnicode_FromString(device);
    if (obj)
        PyDict_SetItemString(mdict, "device", obj);
    Py_XDECREF(obj);
    Py_INCREF(&pymelem_type);
    Py_INCREF(&pymixer_type);
    PyModule_AddObject(py_mod, "InternalMElement", (PyObject *)&pymelem_type);
    PyModule_AddObject(py_mod, "InternalMixer", (PyObject *)&pymixer_type);
    obj = PyDict_GetItemString(mdict, "InternalMixer");
    if (obj) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1058) - [public] PyDict_SetItemString

```
    PyDict_SetItemString(mdict, "device", obj);
    Py_XDECREF(obj);
    Py_INCREF(&pymelem_type);
    Py_INCREF(&pymixer_type);
    PyModule_AddObject(py_mod, "InternalMElement", (PyObject *)&pymelem_type);
    PyModule_AddObject(py_mod, "InternalMixer", (PyObject *)&pymixer_type);
    obj = PyDict_GetItemString(mdict, "InternalMixer");
    if (obj) {
        obj1 = PyTuple_New(3);
        PyTuple_SET_ITEM(obj1, 0, PyInt_FromLong((long)class));
        PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong((long)mixer));
        PyTuple_SET_ITEM(obj1, 2, mdict);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1067) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(obj1, 0, PyInt_FromLong((long)class));
    PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong((long)mixer));
    PyTuple_SET_ITEM(obj1, 2, mdict);
    Py_INCREF(mdict);
    obj2 = PyObject_CallObject(obj, obj1);
    Py_XDECREF(obj1);
    PyDict_SetItemString(mdict, "mixer", obj2);
    priv->py_mixer = obj2;
    } else {
        snd_error(MIXER, "Unable to create InternalMixer object");
        return -EIO;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1068) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong((long)mixer));
    PyTuple_SET_ITEM(obj1, 2, mdict);
    Py_INCREF(mdict);
    obj2 = PyObject_CallObject(obj, obj1);
    Py_XDECREF(obj1);
    PyDict_SetItemString(mdict, "mixer", obj2);
    priv->py_mixer = obj2;
    } else {
        snd_error(MIXER, "Unable to create InternalMixer object");
        return -EIO;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1069) - [public] PyTuple_SET_ITEM

```
    PyTuple_SET_ITEM(obj1, 2, mdict);
    Py_INCREF(mdict);
    obj2 = PyObject_CallObject(obj, obj1);
    Py_XDECREF(obj1);
    PyDict_SetItemString(mdict, "mixer", obj2);
    priv->py_mixer = obj2;
    } else {
        snd_error(MIXER, "Unable to create InternalMixer object");
        return -EIO;
    }

    obj = PyRun_FileEx(fp, file, Py_file_input, mdict, mdict, 1);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1070) - [public] Py_INCREF

```
    Py_INCREF(mdict);
    obj2 = PyObject_CallObject(obj, obj1);
    Py_DECREF(obj1);
    PyDict_SetItemString(mdict, "mixer", obj2);
    priv->py_mixer = obj2;
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return -EIO;
}

obj = PyRun_FileEx(fp, file, Py_file_input, mdict, mdict, 1);
if (obj == NULL)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1072) - [public] Py_DECREF

```
    Py_DECREF(obj1);
    PyDict_SetItemString(mdict, "mixer", obj2);
    priv->py_mixer = obj2;
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return -EIO;
}

obj = PyRun_FileEx(fp, file, Py_file_input, mdict, mdict, 1);
if (obj == NULL)
    PyErr_Print();
Py_DECREF(obj);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1073) - [public] PyDict_SetItemString

```
    PyDict_SetItemString(mdict, "mixer", obj2);
    priv->py_mixer = obj2;
} else {
    snd_error(MIXER, "Unable to create InternalMixer object");
    return -EIO;
}

obj = PyRun_FileEx(fp, file, Py_file_input, mdict, mdict, 1);
if (obj == NULL)
    PyErr_Print();
Py_DECREF(obj);
priv->py_event_func = PyDict_GetItemString(mdict, "event");
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1076) - [public] snd_error

```
    snd_error(MIXER, "Unable to create InternalMixer object");
    return -EIO;
}

obj = PyRun_FileEx(fp, file, Py_file_input, mdict, mdict, 1);
if (obj == NULL)
    PyErr_Print();
Py_DECREF(obj);
priv->py_event_func = PyDict_GetItemString(mdict, "event");
if (priv->py_event_func == NULL) {
    snd_error(MIXER, "Unable to find python function 'event'");
    return -EIO;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1082) - [public] PyErr_Print

```
    PyObject_Print(obj);
    Py_XDECREF(obj);
    priv->py_event_func = PyDict_GetItemString(mdct, "event");
    if (priv->py_event_func == NULL) {
        snd_error(MIXER, "Unable to find python function 'event'");
        return -EIO;
    }
    return 0;
}

#if PY_MAJOR_VERSION >= 3
static struct PyModuleDef smixer_python_module = {
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1086) - [public] snd_error

```
    snd_error(MIXER, "Unable to find python function 'event'");
    return -EIO;
}

return 0;
}

#if PY_MAJOR_VERSION >= 3
static struct PyModuleDef smixer_python_module = {
    PyModuleDef_HEAD_INIT,
    "smixer_python",
    NULL,
    0,
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1125) - [public] snd_dlp_path

```
    snd_dlp_path(path, sizeof(path), SCRIPT);
    file = path;
}

fp = fopen(file, "r");
if (fp == NULL) {
    snd_error(MIXER, "Unable to find python module '%s'", file);
    return -ENODEV;
}

Py_Initialize();
if (PyType_Ready(&pymelem_type) < 0 ||
    // ... (truncated, see source file for more)
    PyType_Ready(&pymixer_type) < 0) {
    fclose(fp);
    return -EIO;
}

#if PY_MAJOR_VERSION < 3
Py_InitModule("smixer_python", python_methods);
// ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1131) - [public] snd_error

```
    snd_error(MIXER, "Unable to find python module '%s'", file);
    return -ENODEV;
}

Py_Initialize();
if (PyType_Ready(&pymelem_type) < 0 ||
    PyType_Ready(&pymixer_type) < 0) {
    fclose(fp);
    return -EIO;
}

#if PY_MAJOR_VERSION < 3
Py_InitModule("smixer_python", python_methods);
// ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1137) - [public] PyType_Ready

```
■ PyType_Ready(&pymixer_type) < 0) {
■ fclose(fp);
■ return -EIO;
■ }
■ if PY_MAJOR_VERSION < 3
■ Py_InitModule("smixer_python", python_methods);
■ else
■ PyModule_Create(&smixer_python_module);
■ endif
■ priv->py_initialized = 1;
■ main_interpreter = PyThreadState_Get()->interp;
■ obj = PyImport_GetModuleDict();
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1138) - [public] fclose

```
■ fclose(fp);
■ return -EIO;
■ }
■ if PY_MAJOR_VERSION < 3
■ Py_InitModule("smixer_python", python_methods);
■ else
■ PyModule_Create(&smixer_python_module);
■ endif
■ priv->py_initialized = 1;
■ main_interpreter = PyThreadState_Get()->interp;
■ obj = PyImport_GetModuleDict();
■ py_mod = PyDict_GetItemString(obj, "__main__");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1151) - [public]

alsa_mixer_simple_pyinit

```
■ alsa_mixer_simple_pyinit(priv, py_mod, fp, file, class, mixer, device);
■ return 0;
}
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 37) - [public] selem_read

```
/*
 * Prototypes
 */

static int selem_read(snd_mixer_elem_t *elem);

/*
 * Helpers
 */

static unsigned int chanmap_to_channels(unsigned int chanmap)
{
    unsigned int i, res;

    for (i = 0, res = 0; i < MAX_CHANNEL; i++)
        if (chanmap & (1 << i))
            // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 85) - [public] list_for_each

```
■ list_for_each(pos, &s->helems) {
    helem = list_entry(pos, struct helem_base, list);
    printf("min = %li, max = %li\n", helem->min, helem->max);
    if (helem->caps & mask[dir]) {
        s->dir[dir].min = helem->min;
        s->dir[dir].max = helem->max;
        ok_flag = 1;
        break;
    }
}
if (ok_flag)
    continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 87) - [public] printf

```
    printf("min = %li, max = %li\n", helem->min, helem->max);
    if (helem->caps & mask[dir]) {
        s->dir[dir].min = helem->min;
        s->dir[dir].max = helem->max;
        ok_flag = 1;
        break;
    }
}
if (ok_flag)
    continue;
list_for_each(pos, &s->helems) {
    helem = list_entry(pos, struct helem_base, list);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 88) - [public] if

```
    if (helem->caps & mask[dir]) {
        s->dir[dir].min = helem->min;
        s->dir[dir].max = helem->max;
        ok_flag = 1;
        break;
    }
}
if (ok_flag)
    continue;
list_for_each(pos, &s->helems) {
    helem = list_entry(pos, struct helem_base, list);
    if (helem->caps & gmask[dir]) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 97) - [public] list_for_each

```
list_for_each(pos, &s->helems) {
    helem = list_entry(pos, struct helem_base, list);
    if (helem->caps & gmask[dir]) {
        s->dir[dir].min = helem->min;
        s->dir[dir].max = helem->max;
        break;
    }
}
}

/*
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 99) - [public] if

```
    if (helem->caps & gmask[dir]) {
        s->dir[dir].min = helem->min;
        s->dir[dir].max = helem->max;
        break;
    }
}
}

/*
 * Simple Mixer Operations
 */
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 121) - [public] list_for_each

```
    list_for_each(pos, &s->helems) {
        helem = list_entry(pos, struct helem_base, list);
        if (helem->inactive)
            return 0;
    }
    return 1;
}

case SM_OPS_IS_MONO:
    return chanmap_to_channels(s->dir[dir].chanmap) == 1;

case SM_OPS_IS_CHANNEL:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 308) - [public] snd_mixer_selem_id_free

```
    snd_mixer_selem_id_free(simple->selem.id);
    list_for_each_safe(pos, npos, &simple->helems) {
        hsimple = list_entry(pos, struct helem_base, list);
        free(hsimple);
    }
    free(simple);
}

static int simple_event_add1(snd_mixer_class_t *class,
    snd_hctl_elem_t *helem,
    struct helem_selector *sel)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 311) - [public] free

```
    free(hsimple);
}
free(simple);
}

static int simple_event_add1(snd_mixer_class_t *class,
    snd_hctl_elem_t *helem,
    struct helem_selector *sel)
{
    struct bclass_private *priv = snd_mixer_sbasic_get_private(class);
    snd_mixer_elem_t *melem;
    snd_mixer_selem_id_t *id;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 337) - [public] if

```
    if (bsid->sids[ui].sid == sel->sid) {
        sid = &bsid->sids[ui];
        goto __sid_ok;
    }
}
return 0;

__sid_ok:
    snd_ctl_elem_info_alloca(&info);
    err = snd_hctl_elem_info(helem, info);
    if (err < 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 370) - [public] snd_mixer_selem_id_free

```
    ■■snd_mixer_selem_id_free(id);
    ■■return -ENOMEM;
    ■}
    ■switch (sel->purpose) {
    ■case PURPOSE_SWITCH:
    ■■if (ctype != SND_CTL_ELEM_TYPE_BOOLEAN) {
    ■■    __invalid_type:
    ■■■snd_mixer_selem_id_free(id);
    ■■■free(hsimple);
    ■■■return -EINVAL;
    ■■}
    ■■break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 375) - [public] if

```
    ■■if (ctype != SND_CTL_ELEM_TYPE_BOOLEAN) {
    ■■    __invalid_type:
    ■■■snd_mixer_selem_id_free(id);
    ■■■free(hsimple);
    ■■■return -EINVAL;
    ■■}
    ■■break;
    ■case PURPOSE_VOLUME:
    ■■if (ctype != SND_CTL_ELEM_TYPE_INTEGER)
    ■■■goto __invalid_type;
    ■■break;
    ■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 377) - [public] snd_mixer_selem_id_free

```
    ■■■snd_mixer_selem_id_free(id);
    ■■■free(hsimple);
    ■■■return -EINVAL;
    ■■}
    ■■break;
    ■case PURPOSE_VOLUME:
    ■■if (ctype != SND_CTL_ELEM_TYPE_INTEGER)
    ■■■goto __invalid_type;
    ■■break;
    ■}
    ■hsimple->purpose = sel->purpose;
    ■hsimple->caps = sel->caps;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 378) - [public] free

```
    ■■■free(hsimple);
    ■■■return -EINVAL;
    ■■}
    ■■break;
    ■case PURPOSE_VOLUME:
    ■■if (ctype != SND_CTL_ELEM_TYPE_INTEGER)
    ■■■goto __invalid_type;
    ■■break;
    ■}
    ■hsimple->purpose = sel->purpose;
    ■hsimple->caps = sel->caps;
    ■hsimple->min = min;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 396) - [public] if

```
    if (!simple) {
        snd_mixer_selem_id_free(id);
        free(hsimple);
        return -ENOMEM;
    }
    simple->selem.id = id;
    simple->selem.ops = &simple_ac97_ops;
    INIT_LIST_HEAD(&simple->helems);
    simple->sid = sel->sid;
    err = snd_mixer_elem_new(&melem, SND_MIXER_ELEM_SIMPLE,
        sid->weight,
        simple, selem_free);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 397) - [public] snd_mixer_selem_id_free

```
    snd_mixer_selem_id_free(id);
    free(hsimple);
    return -ENOMEM;
}
simple->selem.id = id;
simple->selem.ops = &simple_ac97_ops;
INIT_LIST_HEAD(&simple->helems);
simple->sid = sel->sid;
err = snd_mixer_elem_new(&melem, SND_MIXER_ELEM_SIMPLE,
    sid->weight,
    simple, selem_free);
if (err < 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 398) - [public] free

```
    free(hsimple);
    return -ENOMEM;
}
simple->selem.id = id;
simple->selem.ops = &simple_ac97_ops;
INIT_LIST_HEAD(&simple->helems);
simple->sid = sel->sid;
err = snd_mixer_elem_new(&melem, SND_MIXER_ELEM_SIMPLE,
    sid->weight,
    simple, selem_free);
if (err < 0) {
    snd_mixer_selem_id_free(id);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 403) - [public] INIT_LIST_HEAD

```
    INIT_LIST_HEAD(&simple->helems);
    simple->sid = sel->sid;
    err = snd_mixer_elem_new(&melem, SND_MIXER_ELEM_SIMPLE,
        sid->weight,
        simple, selem_free);
    if (err < 0) {
        snd_mixer_selem_id_free(id);
        free(hsimple);
        free(simple);
        return err;
    }
    new = 1;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 408) - [public] if

```
    if (err < 0) {
        snd_mixer_selem_id_free(id);
        free(hsimple);
        free(simple);
        return err;
    }
    new = 1;
} else {
    simple = snd_mixer_elem_get_private(melem);
    snd_mixer_selem_id_free(id);
}
list_add_tail(&hsimple->list, &simple->helems);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 409) - [public] snd_mixer_selem_id_free

```
    snd_mixer_selem_id_free(id);
    free(hsimple);
    free(simple);
    return err;
}
new = 1;
} else {
    simple = snd_mixer_elem_get_private(melem);
    snd_mixer_selem_id_free(id);
}
list_add_tail(&hsimple->list, &simple->helems);
hsimple->inactive = snd_ctl_elem_info_is_inactive(info);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 410) - [public] free

```
    free(hsimple);
    free(simple);
    return err;
}
new = 1;
} else {
    simple = snd_mixer_elem_get_private(melem);
    snd_mixer_selem_id_free(id);
}
list_add_tail(&hsimple->list, &simple->helems);
hsimple->inactive = snd_ctl_elem_info_is_inactive(info);
err = snd_mixer_elem_attach(melem, helem);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 411) - [public] free

```
    free(simple);
    return err;
}
new = 1;
} else {
    simple = snd_mixer_elem_get_private(melem);
    snd_mixer_selem_id_free(id);
}
list_add_tail(&hsimple->list, &simple->helems);
hsimple->inactive = snd_ctl_elem_info_is_inactive(info);
err = snd_mixer_elem_attach(melem, helem);
if (err < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 417) - [public] snd_mixer_selem_id_free

```
    ■■snd_mixer_selem_id_free(id);
    ■}
    ■list_add_tail(&hsimple->list, &simple->helems);
    ■hsimple->inactive = snd_ctl_elem_info_is_inactive(info);
    ■err = snd_mixer_elem_attach(melem, helem);
    ■if (err < 0)
    ■■goto __error;
    ■simple->dir[0].chanmap |= sid->chanmap[0];
    ■simple->dir[1].chanmap |= sid->chanmap[1];
    ■simple->selem.caps |= hsimple->caps;
    ■update_ranges(simple);
    ■if 0
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 450) - [public] snd_mixer_elem_free

```
    ■■snd_mixer_elem_free(melem);
    ■return -EINVAL;
    ■}

    static int simple_event_add(snd_mixer_class_t *class, snd_htcl_elem_t *helem)
    {
        ■struct bclass_private *priv = snd_mixer_sbasic_get_private(class);
        ■struct bclass_selector *sel;
        ■struct helem_selector *hsel;
        ■struct list_head *pos;
        ■snd_ctl_elem_iface_t iface = snd_htcl_elem_get_interface(helem);
        ■const char *name = snd_htcl_elem_get_name(helem);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 470) - [public] if

```
    ■■■if (hsel->iface == iface && !strcmp(hsel->name, name) && hsel->index == index) {
    ■■■■err = simple_event_add1(class, helem, hsel);
    ■■■■if (err < 0)
    ■■■■return err;■/* early exit? */
    ■■■}
    ■■}
    ■}
    ■return 0;
    ■}

    int alsa_mixer_sbasic_event(snd_mixer_class_t *class, unsigned int mask,
    ■■■    snd_htcl_elem_t *helem, snd_mixer_elem_t *melem)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 485) - [public] simple_event_remove

```
    ■■return simple_event_remove(helem, melem);
    ■if (mask & SND_CTL_EVENT_MASK_ADD) {
    ■■err = simple_event_add(class, helem);
    ■■if (err < 0)
    ■■■return err;
    ■}
    ■if (mask & SND_CTL_EVENT_MASK_INFO) {
    ■■err = simple_event_remove(helem, melem);
    ■■if (err < 0)
    ■■■return err;
    ■■err = simple_event_add(class, helem);
    ■■if (err < 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 504) - [public] if

```
    if (err) {
        err = snd_mixer_elem_value(melem);
        if (err < 0)
            return err;
    }
    return 0;

static void sbasic_cpriv_free(snd_mixer_class_t *class)
{
    struct bclass_private *priv = snd_mixer_sbasic_get_private(class);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 522) - [public] free

```
    free(sel);
}
list_for_each_safe(pos, pos1, &priv->sids) {
    sid = list_entry(pos, struct bclass_sid, list);
    free(sid);
}
free(priv);
}

void alsa_mixer_sbasic_initpriv(snd_mixer_class_t *class,
    struct bclass_private *priv)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 526) - [public] free

```
    free(sid);
}
free(priv);
}

void alsa_mixer_sbasic_initpriv(snd_mixer_class_t *class,
    struct bclass_private *priv)
{
    INIT_LIST_HEAD(&priv->selectors);
    INIT_LIST_HEAD(&priv->sids);
    snd_mixer_sbasic_set_private(class, priv);
    snd_mixer_sbasic_set_private_free(class, sbasic_cpriv_free);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 551) - [public] if

```
    if (priv == NULL) {
        free(sel);
        return -ENOMEM;
    }
    sel->selectors = selectors;
    sel->count = count;
    list_add_tail(&sel->list, &priv->selectors);
    return 0;
}

int alsa_mixer_sbasic_sidreg(snd_mixer_class_t *class,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 552) - [public] free

```
    free(sel);
    return -ENOMEM;
}

sel->selectors = selectors;
sel->count = count;
list_add_tail(&sel->list, &priv->selectors);
return 0;
}

int alsa_mixer_sbasic_sidreg(snd_mixer_class_t *class,
    struct melem_sids *sids,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 573) - [public] if

```
if (priv == NULL) {
    free(sid);
    return -ENOMEM;
}
INIT_LIST_HEAD(&priv->selectors);
INIT_LIST_HEAD(&priv->sids);
snd_mixer_sbasic_set_private(class, priv);
snd_mixer_sbasic_set_private_free(class, sbasic_cpriv_free);
}
sid->sids = sids;
sid->count = count;
list_add(&sid->list, &priv->sids);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 574) - [public] free

```
free(sid);
return -ENOMEM;
}
INIT_LIST_HEAD(&priv->selectors);
INIT_LIST_HEAD(&priv->sids);
snd_mixer_sbasic_set_private(class, priv);
snd_mixer_sbasic_set_private_free(class, sbasic_cpriv_free);
}
sid->sids = sids;
sid->count = count;
list_add(&sid->list, &priv->sids);
return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 577) - [public] INIT_LIST_HEAD

```
INIT_LIST_HEAD(&priv->selectors);
INIT_LIST_HEAD(&priv->sids);
snd_mixer_sbasic_set_private(class, priv);
snd_mixer_sbasic_set_private_free(class, sbasic_cpriv_free);
}
sid->sids = sids;
sid->count = count;
list_add(&sid->list, &priv->sids);
return 0;
}
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 578) - [public] INIT_LIST_HEAD

```
INIT_LIST_HEAD(&priv->sids);
snd_mixer_sbasic_set_private(class, priv);
snd_mixer_sbasic_set_private_free(class, sbasic_cpriv_free);
}
sid->sids = sids;
sid->count = count;
list_add(&sid->list, &priv->sids);
return 0;
}
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 579) - [public]

snd_mixer_sbasic_set_private

```
■ ■ ■ snd_mixer_sbasic_set_private(class, priv);
■ ■ ■ snd_mixer_sbasic_set_private_free(class, sbasic_cpriv_free);
■ }
■ sid->sids = sids;
■ sid->count = count;
■ list_add(&sid->list, &priv->sids);
■ return 0;
■ }
```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 580) - [public]

snd_mixer_sbasic_set_private_free

```
■ ■ ■ snd_mixer_sbasic_set_private_free(class, sbasic_cpriv_free);
■ }
■ sid->sids = sids;
■ sid->count = count;
■ list_add(&sid->list, &priv->sids);
■ return 0;
■ }
```

dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 60) - [public] free

```
■ ■ ■ free(priv);
■ ■ return -ENOMEM;
■ }
■ strcpy(xlib, path);
■ strcat(xlib, "/");
■ strcat(xlib, lib);
■ h = snd_dlopen(xlib, RTLD_NOW, errbuf, sizeof(errbuf));
■ if (h == NULL) {
■ ■ snd_error(MIXER, "Unable to open library '%s': %s", xlib, errbuf);
■ ■ goto __error;
■ }
■ initpriv = dlsym(h, "alsa_mixer_sbasic_initpriv");
■ // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 68) - [public] snd_error

```
■ ■ snd_error(MIXER, "Unable to open library '%s': %s", xlib, errbuf);
■ ■ goto __error;
■ }
■ initpriv = dlsym(h, "alsa_mixer_sbasic_initpriv");
■ if (initpriv == NULL) {
■ ■ snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_initpriv' was not found in '%s'", xlib);
■ ■ goto __error;
■ }
■ priv->ops.event = dlsym(h, "alsa_mixer_sbasic_event");
■ if (priv->ops.event == NULL) {
■ ■ snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_event' was not found in '%s'", xlib);
■ ■ goto __error;
■ }
■ // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 73) - [public] snd_error

```
■ ■ snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_initpriv' was not found in '%s'", xlib);
■ ■ goto __error;
■ }
■ priv->ops.event = dlsym(h, "alsa_mixer_sbasic_event");
■ if (priv->ops.event == NULL) {
■ ■ snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_event' was not found in '%s'", xlib);
■ ■ goto __error;
■ }
■ priv->ops.selreg = dlsym(h, "alsa_mixer_sbasic_selreg");
■ if (priv->ops.selreg == NULL) {
■ ■ snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_selreg' was not found in '%s'", xlib);
■ ■ goto __error;
■ }
■ // ... (truncated, see source file for more)
```



```
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 78) - [public] snd_error
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_event' was not found in '%s'", xlib);
    ■■goto __error;
    ■}
    ■priv->ops.selreg = dlsym(h, "alsa_mixer_sbasic_selreg");
    ■if (priv->ops.selreg == NULL) {
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_selreg' was not found in '%s'", xlib);
    ■■goto __error;
    ■}
    ■priv->ops.sidreg = dlsym(h, "alsa_mixer_sbasic_sidreg");
    ■if (priv->ops.sidreg == NULL) {
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_sidreg' was not found in '%s'", xlib);
    ■■goto __error;
    ■}
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 83) - [public] snd_error
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_selreg' was not found in '%s'", xlib);
    ■■goto __error;
    ■}
    ■priv->ops.sidreg = dlsym(h, "alsa_mixer_sbasic_sidreg");
    ■if (priv->ops.sidreg == NULL) {
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_sidreg' was not found in '%s'", xlib);
    ■■goto __error;
    ■}
    ■free(xlib);
    ■if (initflag)
    ■■initpriv(class, priv);
    ■priv->dl_sbase = h;
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 88) - [public] snd_error
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_sidreg' was not found in '%s'", xlib);
    ■■goto __error;
    ■}
    ■free(xlib);
    ■if (initflag)
    ■■initpriv(class, priv);
    ■priv->dl_sbase = h;
    ■if (ops)
    ■■*ops = &priv->ops;
    ■return 1;

    __error:
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 93) - [public] initpriv
    ■■initpriv(class, priv);
    ■priv->dl_sbase = h;
    ■if (ops)
    ■■*ops = &priv->ops;
    ■return 1;

    __error:
    ■if (initflag)
    ■■free(priv);
    ■if (h)
    ■■snd_dlcclose(h);
    ■free(xlib);
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 101) - [public] free
    ■■free(priv);
    ■if (h)
    ■■snd_dlcclose(h);
    ■free(xlib);
    ■return -ENXIO;
}
```

dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 103) - [public] snd_dlclose

```
■■snd_dlclose(h);
■free(xlib);
■return -ENXIO;
}
```

dirtycode/alsa-lib/src/async.c (line 40) - [public] snd_async_init

```
void snd_async_init(void) __attribute__((constructor));

void snd_async_init(void)
{
    ■snd_async_signo = __libc_allocate_rtsig(0);
    ■if (snd_async_signo < 0) {
        ■■snd_error(CORE, "Unable to find a RT signal to use for snd_async");
        ■■exit(1);
        ■}
    }
    #else
    /** async signal number */
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 46) - [public] snd_error

```
■■snd_error(CORE, "Unable to find a RT signal to use for snd_async");
■■exit(1);
■}
}
#else
/** async signal number */
static const int snd_async_signo = SIGIO;
#endif
```

```
static LIST_HEAD(snd_async_handlers);
```

```
static void snd_async_handler(int signo ATTRIBUTE_UNUSED, siginfo_t *siginfo, void *context ATTRIBUTE_UNUSED)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 47) - [public] exit

```
■■exit(1);
■}
}
#else
/** async signal number */
static const int snd_async_signo = SIGIO;
#endif
```

```
static LIST_HEAD(snd_async_handlers);
```

```
static void snd_async_handler(int signo ATTRIBUTE_UNUSED, siginfo_t *siginfo, void *context ATTRIBUTE_UNUSED)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 55) - [public] LIST_HEAD

```
static LIST_HEAD(snd_async_handlers);
```

```
static void snd_async_handler(int signo ATTRIBUTE_UNUSED, siginfo_t *siginfo, void *context ATTRIBUTE_UNUSED)
{
    #if defined(__DragonFly__) || defined(__FreeBSD__) || defined(__OpenBSD__)
    ■/* siginfo_t does not have si_fd */
    ■struct list_head *i;
    ■list_for_each(i, &snd_async_handlers) {
        ■■snd_async_handler_t *h = list_entry(i, snd_async_handler_t, glist);
        ■■if (h->callback)
            ■■■h->callback(h);
        ■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 131) - [public] memset

```
    memset(&act, 0, sizeof(act));
    act.sa_flags = SA_RESTART | SA_SIGINFO;
    act.sa_sigaction = snd_async_handler;
    sigemptyset(&act.sa_mask);
    assert(!previous_action.sa_sigaction);
    err = sigaction(snd_async_signo, &act, &previous_action);
    if (err < 0) {
        snd_errornum(CORE, "sigaction");
        return -errno;
    }
    return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 134) - [public] sigemptyset

```
    sigemptyset(&act.sa_mask);
    assert(!previous_action.sa_sigaction);
    err = sigaction(snd_async_signo, &act, &previous_action);
    if (err < 0) {
        snd_errornum(CORE, "sigaction");
        return -errno;
    }
    return 0;
}

/**
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 135) - [public] assert

```
    assert(!previous_action.sa_sigaction);
    err = sigaction(snd_async_signo, &act, &previous_action);
    if (err < 0) {
        snd_errornum(CORE, "sigaction");
        return -errno;
    }
    return 0;
}

/**
    * \brief Deletes an async handler.
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 137) - [public] if

```
    if (err < 0) {
        snd_errornum(CORE, "sigaction");
        return -errno;
    }
    return 0;
}

/**
    * \brief Deletes an async handler.
    * \param handler Handle of the async handler to delete.
    * \result Zero if successful, otherwise a negative error code.
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 138) - [public] snd_erronum

```
    snd_erronum(CORE, "sigaction");
    return -errno;
}
}
return 0;
}

/**
 * \brief Deletes an async handler.
 * \param handler Handle of the async handler to delete.
 * \result Zero if successful, otherwise a negative error code.
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 156) - [public] switch

```
switch (handler->type) {
#ifdef BUILD_PCM
case SND_ASYNC_HANDLER_PCM:
    alist = &handler->u.pcm->async_handlers;
    break;
#endif
case SND_ASYNC_HANDLER_CTL:
    alist = &handler->u.ctl->async_handlers;
    break;
default:
    assert(0);
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 166) - [public] assert

```
assert(0);
}
if (!list_empty(alist))
    list_del(&handler->hlist);
if (!list_empty(alist))
    goto _glist;
switch (handler->type) {
#ifdef BUILD_PCM
case SND_ASYNC_HANDLER_PCM:
    err2 = snd_pcm_async(handler->u.pcm, -1, 1);
    break;
#endif
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 169) - [public] list_del

```
list_del(&handler->hlist);
if (!list_empty(alist))
    goto _glist;
switch (handler->type) {
#ifdef BUILD_PCM
case SND_ASYNC_HANDLER_PCM:
    err2 = snd_pcm_async(handler->u.pcm, -1, 1);
    break;
#endif
case SND_ASYNC_HANDLER_CTL:
    err2 = snd_ctl_async(handler->u.ctl, -1, 1);
    break;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 172) - [public] switch

```
    switch (handler->type) {
    #ifdef BUILD_PCM
    case SND_ASYNC_HANDLER_PCM:
        err2 = snd_pcm_async(handler->u.pcm, -1, 1);
        break;
    #endif
    case SND_ASYNC_HANDLER_CTL:
        err2 = snd_ctl_async(handler->u.ctl, -1, 1);
        break;
    default:
        assert(0);
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 182) - [public] assert

```
    assert(0);
}
_glist:
if (!list_empty(&snd_async_handlers)) {
    list_del(&handler->glist);
    if (list_empty(&snd_async_handlers)) {
        err = sigaction(snd_async_signo, &previous_action, NULL);
        if (err < 0) {
            snd_errornum(CORE, "sigaction");
            return -errno;
        }
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 187) - [public] list_del

```
    list_del(&handler->glist);
    if (list_empty(&snd_async_handlers)) {
        err = sigaction(snd_async_signo, &previous_action, NULL);
        if (err < 0) {
            snd_errornum(CORE, "sigaction");
            return -errno;
        }
        memset(&previous_action, 0, sizeof(previous_action));
    }
    free(handler);
    return err ? err : err2;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 188) - [public] if

```
    if (list_empty(&snd_async_handlers)) {
        err = sigaction(snd_async_signo, &previous_action, NULL);
        if (err < 0) {
            snd_errornum(CORE, "sigaction");
            return -errno;
        }
        memset(&previous_action, 0, sizeof(previous_action));
    }
    free(handler);
    return err ? err : err2;
}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 190) - [public] if

```
    if (err < 0) {
        snd_errormsg(CORE, "sigaction");
        return -errno;
    }
    memset(&previous_action, 0, sizeof(previous_action));
}
free(handler);
return err ? err : err2;
}

/**
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 191) - [public] snd_errormsg

```
    snd_errormsg(CORE, "sigaction");
    return -errno;
}
memset(&previous_action, 0, sizeof(previous_action));
}
free(handler);
return err ? err : err2;
}

/**
 * \brief Returns the signal number assigned to an async handler.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/async.c (line 194) - [public] memset

```
    memset(&previous_action, 0, sizeof(previous_action));
}
free(handler);
return err ? err : err2;
}

/**
 * \brief Returns the signal number assigned to an async handler.
 * \param handler Handle to an async handler.
 * \result The signal number if successful, otherwise a negative error code.
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 524) - [internal] snd_config_lock

```
static inline void snd_config_lock(void) { }
static inline void snd_config_unlock(void) { }

#endif

/*
 * Add a directory to the paths to search included files.
 * param fd - File object that owns these paths to search files included by it.
 * param dir - Path of the directory to add. Allocated externally and need to
 *             be freed manually later.
 * return - Zero if successful, otherwise a negative error code.
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 525) - [internal] snd_config_unlock

```
static inline void snd_config_unlock(void) { }
```

```
#endif
```

```
/*
```

```
 * Add a directory to the paths to search included files.
 * param fd - File object that owns these paths to search files included by it.
 * param dir - Path of the directory to add. Allocated externally and need to
 *             be freed manually later.
 * return - Zero if successful, otherwise a negative error code.
 *
 * The direcotry should be a subdirectory of top configuration directory
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 547) - [public] list_for_each

```
■■list_for_each(pos, &fd1->include_paths) {
■■■path = list_entry(pos, struct include_path, list);
■■■if (strcmp(path->dir, dir) == 0)
■■■■return 0;
■■}
■}
```

```
■path = calloc(1, sizeof(*path));
■if (!path)
■■return -ENOMEM;

■path->dir = strdup(dir);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 560) - [public] free

```
■■free(path);
■■return -ENOMEM;
■}

■list_add_tail(&path->list, &fd->include_paths);
■return 0;
}
```

```
/*
```

```
 * Free all include paths of a file descriptor.
 * param fd - File object that owns these paths to search files included by it.
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 580) - [public] list_del

```
■■list_del(&path->list);
■■if (path->dir)
■■■free(path->dir);
■■free(path);
■}
}
```

```
#endif /* DOC_HIDDEN */
```

```
/**
```

```
 * \brief Returns the default top-level config directory
 * \return The top-level config directory path string
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 582) - [public] free

```
    free(path->dir);
    free(path);
}

#endif /* DOC_HIDDEN */

/**
 * \brief Returns the default top-level config directory
 * \return The top-level config directory path string
 *
 * This function returns the string of the top-level config directory path.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 583) - [public] free

```
    free(path);
}

#endif /* DOC_HIDDEN */

/**
 * \brief Returns the default top-level config directory
 * \return The top-level config directory path string
 *
 * This function returns the string of the top-level config directory path.
 * If the path is specified via the environment variable \c ALSA_CONFIG_DIR
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 647) - [public] snd_input_stdio_open

```
    return snd_input_stdio_open(inputp, file, "r");

    /* search file in user specified include paths. These directories
     * are subdirectories of /usr/share/alsa.
     */
    err = -ENOENT;
    while (current) {
        list_for_each(pos, &current->include_paths) {
            path = list_entry(pos, struct include_path, list);
            if (!path->dir)
                continue;

            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 654) - [public] list_for_each

```
    list_for_each(pos, &current->include_paths) {
        path = list_entry(pos, struct include_path, list);
        if (!path->dir)
            continue;

        snprintf(full_path, PATH_MAX, "%s/%s", path->dir, file);
        err = snd_input_stdio_open(inputp, full_path, "r");
        if (err == 0)
            return 0;
    }
    current = current->next;
}

// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 659) - [public] snprintf

```
    ■■■snprintf(full_path, PATH_MAX, "%s/%s", path->dir, file);
    ■■■err = snd_input_stdio_open(inputp, full_path, "r");
    ■■■if (err == 0)
    ■■■■return 0;
    ■■}
    ■■current = current->next;
    ■■}

    ■■return err;
    ■■}
```

```
int _snd_safe_strtoll_base(const char *str, long long *val, int base)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 722) - [public] snprintf

```
    ■■snprintf(locstr, sizeof(locstr), "%s", saved_locale);
    ■■setlocale(LC_NUMERIC, "C");
    ■■}
    #endif
    ■■errno = 0;
    ■■v = strtod(str, &end);
    ■■err = -errno;
    #ifdef HAVE_USELOCALE
    ■■if (c_locale != (locale_t)0) {
    ■■■uselocale(saved_locale);
    ■■■freelocale(c_locale);
    ■■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 723) - [public] setlocale

```
    ■■setlocale(LC_NUMERIC, "C");
    ■■}
    #endif
    ■■errno = 0;
    ■■v = strtod(str, &end);
    ■■err = -errno;
    #ifdef HAVE_USELOCALE
    ■■if (c_locale != (locale_t)0) {
    ■■■uselocale(saved_locale);
    ■■■freelocale(c_locale);
    ■■}
    #else
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 731) - [public] uselocale

```
    ■■uselocale(saved_locale);
    ■■freelocale(c_locale);
    ■■}
    #else
    ■■if (saved_locale)
    ■■■setlocale(LC_NUMERIC, locstr);
    #endif
    ■■if (err)
    ■■■return err;
    ■■if (*end)
    ■■■return -EINVAL;
    ■■*val = v;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 732) - [public] freelocale

```
    ■■freelocale(c_locale);
    ■}
    #else
    ■if (saved_locale)
    ■■setlocale(LC_NUMERIC, locstr);
    #endif
    ■if (err)
    ■■return err;
    ■if (*end)
    ■■return -EINVAL;
    ■*val = v;
    ■return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 736) - [public] setlocale

```
    ■■setlocale(LC_NUMERIC, locstr);
    #endif
    ■if (err)
    ■■return err;
    ■if (*end)
    ■■return -EINVAL;
    ■*val = v;
    ■return 0;
    }
```

```
static int get_char(input_t *input)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 766) - [public] if

```
    ■■if (fd->next) {
    ■■■snd_input_close(fd->in);
    ■■■free(fd->name);
    ■■■input->current = fd->next;
    ■■■free(fd);
    ■■■goto again;
    ■■}
    ■■return LOCAL_UNEXPECTED_EOF;
    ■default:
    ■■fd->column++;
    ■■break;
    ■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 767) - [public] snd_input_close

```
    ■■■snd_input_close(fd->in);
    ■■■free(fd->name);
    ■■■input->current = fd->next;
    ■■■free(fd);
    ■■■goto again;
    ■■}
    ■■return LOCAL_UNEXPECTED_EOF;
    ■default:
    ■■fd->column++;
    ■■break;
    ■}
    ■return (unsigned char)c;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 768) - [public] free

```
    free(fd->name);
    input->current = fd->next;
    free(fd);
    goto again;
}
return LOCAL_UNEXPECTED_EOF;
default:
    fd->column++;
    break;
}
return (unsigned char)c;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 770) - [public] free

```
    free(fd);
    goto again;
}
return LOCAL_UNEXPECTED_EOF;
default:
    fd->column++;
    break;
}
return (unsigned char)c;
}
```

```
static void unget_char(int c, input_t *input)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 788) - [public] get_delimstring

```
static int get_delimstring(char **string, int delim, input_t *input);
```

```
static int get_char_skip_comments(input_t *input)
{
    int c;
    while (1) {
        c = get_char(input);
        if (c == '<') {
            char *str;
            snd_input_t *in;
            struct filedesc *fd;
            DIR *dirp;
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 795) - [public] if

```
    if (c == '<') {
        char *str;
        snd_input_t *in;
        struct filedesc *fd;
        DIR *dirp;
        int err = get_delimstring(&str, '>', input);
        if (err < 0)
            return err;

        if (!strcmp(str, "searchdir:", 10)) {
            /* directory to search included files */
            char *tmp = _snd_config_path(str + 10);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 804) - [public] if

```
    if (!strcmp(str, "searchdir:", 10)) {
        /* directory to search included files */
        char *tmp = _snd_config_path(str + 10);
        free(str);
        if (tmp == NULL)
            return -ENOMEM;
        str = tmp;

        dirp = opendir(str);
        if (!dirp) {
            snd_error(CONFIG, "Invalid search dir %s", str);
            free(str);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 807) - [public] free

```
    free(str);
    if (tmp == NULL)
        return -ENOMEM;
    str = tmp;

    dirp = opendir(str);
    if (!dirp) {
        snd_error(CONFIG, "Invalid search dir %s", str);
        free(str);
        return -EINVAL;
    }
    closedir(dirp);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 813) - [public] if

```
    if (!dirp) {
        snd_error(CONFIG, "Invalid search dir %s", str);
        free(str);
        return -EINVAL;
    }
    closedir(dirp);

    err = add_include_path(input->current, str);
    if (err < 0) {
        snd_error(CORE, "Cannot add search dir %s", str);
        free(str);
        return err;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 814) - [public] snd_error

```
    snd_error(CONFIG, "Invalid search dir %s", str);
    free(str);
    return -EINVAL;
}
closedir(dirp);

err = add_include_path(input->current, str);
if (err < 0) {
    snd_error(CORE, "Cannot add search dir %s", str);
    free(str);
    return err;
}
// ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 815) - [public] free

```
        free(str);
        return -EINVAL;
    }
    closedir(dirp);

    err = add_include_path(input->current, str);
    if (err < 0) {
        snd_error(CORE, "Cannot add search dir %s", str);
        free(str);
        return err;
    }
    free(str);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 818) - [public] closedir

```
    closedir(dirp);

    err = add_include_path(input->current, str);
    if (err < 0) {
        snd_error(CORE, "Cannot add search dir %s", str);
        free(str);
        return err;
    }
    free(str);
    continue;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 821) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "Cannot add search dir %s", str);
        free(str);
        return err;
    }
    free(str);
    continue;
}

    if (!strcmp(str, "confdir:", 8)) {
        /* file in the specified directory */
        char *tmp = _snd_config_path(str + 8);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 822) - [public] snd_error

```
        snd_error(CORE, "Cannot add search dir %s", str);
        free(str);
        return err;
    }
    free(str);
    continue;
}

    if (!strcmp(str, "confdir:", 8)) {
        /* file in the specified directory */
        char *tmp = _snd_config_path(str + 8);
        free(str);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 823) - [public] free

```
    free(str);
    return err;
}

free(str);
continue;
}

if (!strcmp(str, "confdir:", 8)) {
    /* file in the specified directory */
    char *tmp = _snd_config_path(str + 8);
    free(str);
    if (tmp == NULL)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 826) - [public] free

```
    free(str);
    continue;
}

if (!strcmp(str, "confdir:", 8)) {
    /* file in the specified directory */
    char *tmp = _snd_config_path(str + 8);
    free(str);
    if (tmp == NULL)
        return -ENOMEM;
    str = tmp;
    err = snd_input_stdio_open(&in, str, "r");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 830) - [public] if

```
    if (!strcmp(str, "confdir:", 8)) {
        /* file in the specified directory */
        char *tmp = _snd_config_path(str + 8);
        free(str);
        if (tmp == NULL)
            return -ENOMEM;
        str = tmp;
        err = snd_input_stdio_open(&in, str, "r");
    } else { /* absolute or relative file path */
        err = input_stdio_open(&in, str, input->current);
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 833) - [public] free

```
    free(str);
    if (tmp == NULL)
        return -ENOMEM;
    str = tmp;
    err = snd_input_stdio_open(&in, str, "r");
} else { /* absolute or relative file path */
    err = input_stdio_open(&in, str, input->current);
}

if (err < 0) {
    snd_error(CORE, "Cannot access file %s", str);
    free(str);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 842) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "Cannot access file %s", str);
        free(str);
        return err;
    }
    fd = malloc(sizeof(*fd));
    if (!fd) {
        free(str);
        return -ENOMEM;
    }
    fd->name = str;
    fd->in = in;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 843) - [public] snd_error

```
    snd_error(CORE, "Cannot access file %s", str);
    free(str);
    return err;
}
fd = malloc(sizeof(*fd));
if (!fd) {
    free(str);
    return -ENOMEM;
}
fd->name = str;
fd->in = in;
fd->next = input->current;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 844) - [public] free

```
    free(str);
    return err;
}
fd = malloc(sizeof(*fd));
if (!fd) {
    free(str);
    return -ENOMEM;
}
fd->name = str;
fd->in = in;
fd->next = input->current;
fd->line = 1;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 848) - [public] if

```
    if (!fd) {
        free(str);
        return -ENOMEM;
    }
    fd->name = str;
    fd->in = in;
    fd->next = input->current;
    fd->line = 1;
    fd->column = 0;
    INIT_LIST_HEAD(&fd->include_paths);
    input->current = fd;
    continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 849) - [public] free

```
    free(str);
    return -ENOMEM;
}
fd->name = str;
fd->in = in;
fd->next = input->current;
fd->line = 1;
fd->column = 0;
INIT_LIST_HEAD(&fd->include_paths);
input->current = fd;
continue;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 857) - [public] INIT_LIST_HEAD

```
INIT_LIST_HEAD(&fd->include_paths);
input->current = fd;
continue;
}
if (c != '#')
    break;
while (1) {
    c = get_char(input);
    if (c < 0)
        return c;
    if (c == '\n')
        break;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 863) - [public] while

```
while (1) {
    c = get_char(input);
    if (c < 0)
        return c;
    if (c == '\n')
        break;
}

return c;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 881) - [public] switch

```
switch (c) {
case ' ':
case '\f':
case '\t':
case '\n':
case '\r':
    break;
default:
    return c;
}
}

// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 927) - [public] get_hexachar

```
    return get_hexachar(input);
    case '0': case '1': case '2': case '3':
    case '4': case '5': case '6': case '7':
    {
        int num = c - '0';
        int i = 1;
        do {
            c = get_char(input);
            if (c < '0' || c > '7') {
                unget_char(c, input);
                break;
            }
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 935) - [public] if

```
        if (c < '0' || c > '7') {
            unget_char(c, input);
            break;
        }
        num = num * 8 + c - '0';
        i++;
    } while (i < 3);
    return num;
}
default:
    return c;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 936) - [public] unget_char

```
        unget_char(c, input);
        break;
    }
    num = num * 8 + c - '0';
    i++;
} while (i < 3);
return num;
}
default:
    return c;
}
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 967) - [public] free

```
    free(s->buf);
}

static int add_char_local_string(struct local_string *s, int c)
{
    if (s->idx >= s->alloc) {
        size_t nalloc = s->alloc * 2;
        if (s->buf == s->tmpbuf) {
            s->buf = malloc(nalloc);
            if (s->buf == NULL)
                return -ENOMEM;
            memcpy(s->buf, s->tmpbuf, s->alloc);
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 974) - [public] if

```
    if (s->buf == s->tmpbuf) {
        s->buf = malloc(nalloc);
        if (s->buf == NULL)
            return -ENOMEM;
        memcpy(s->buf, s->tmpbuf, s->alloc);
    } else {
        char *ptr = realloc(s->buf, nalloc);
        if (ptr == NULL)
            return -ENOMEM;
        s->buf = ptr;
    }
    s->alloc = nalloc;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 978) - [public] memcpy

```
    memcpy(s->buf, s->tmpbuf, s->alloc);
} else {
    char *ptr = realloc(s->buf, nalloc);
    if (ptr == NULL)
        return -ENOMEM;
    s->buf = ptr;
}
s->alloc = nalloc;
}
s->buf[s->idx++] = c;
return 0;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 995) - [public] memcpy

```
    memcpy(dst, s->buf, s->idx);
    dst[s->idx] = '\0';
}
return dst;
}
```

```
static int get_freestring(char **string, int id, input_t *input)
{
    struct local_string str;
    int c;

    init_local_string(&str);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1009) - [public] if

```
    if (c < 0) {
        if (c == LOCAL_UNEXPECTED_EOF) {
            *string = copy_local_string(&str);
            if (! *string)
                c = -ENOMEM;
            else
                c = 0;
        }
        break;
    }
    switch (c) {
    case '.':
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1010) - [public] if

```
    if (c == LOCAL_UNEXPECTED_EOF) {
        *string = copy_local_string(&str);
        if (! *string)
            c = -ENOMEM;
        else
            c = 0;
    }
    break;
}
switch (c) {
case '.':
    if (!id)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1019) - [public] switch

```
switch (c) {
case '.':
    if (!id)
        break;
    /* fall through */
case ' ':
case '\f':
case '\t':
case '\n':
case '\r':
case '=':
case ',':
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1044) - [public] unget_char

```
unget_char(c, input);
c = 0;
}
goto _out;
default:
    break;
}
if (add_char_local_string(&str, c) < 0) {
    c = -ENOMEM;
    break;
}
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1051) - [public] if

```
if (add_char_local_string(&str, c) < 0) {
    c = -ENOMEM;
    break;
}
}
_out:
free_local_string(&str);
return c;
}
```

```
static int get_delimstring(char **string, int delim, input_t *input)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1071) - [public] if

```
    if (c == '\\') {
        c = get_quotedchar(input);
        if (c < 0)
            break;
        if (c == '\n')
            continue;
    } else if (c == delim) {
        *string = copy_local_string(&str);
        if (! *string)
            c = -ENOMEM;
        else
            c = 0;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1085) - [public] if

```
    if (add_char_local_string(&str, c) < 0) {
        c = -ENOMEM;
        break;
    }
}

free_local_string(&str);
return c;
}

/* Return 0 for free string, 1 for delimited string */
static int get_string(char **string, int id, input_t *input)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1090) - [public] free_local_string

```
free_local_string(&str);
return c;
}

/* Return 0 for free string, 1 for delimited string */
static int get_string(char **string, int id, input_t *input)
{
    int c = get_nonwhite(input), err;
    if (c < 0)
        return c;
    switch (c) {
    case '=':
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1118) - [public] unget_char

```
unget_char(c, input);
err = get_freestring(string, id, input);
if (err < 0)
    return err;
return 0;
}

static int _snd_config_make(snd_config_t **config, char **id, snd_config_type_t type)
{
    snd_config_t *n;
    assert(config);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1132) - [public] if

```
    if (*id) {
        free(*id);
        *id = NULL;
    }
    return -ENOMEM;
}
if (id) {
    n->id = *id;
    *id = NULL;
}
n->type = type;
if (type == SND_CONFIG_TYPE_COMPOUND)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1133) - [public] free

```
    free(*id);
    *id = NULL;
}
return -ENOMEM;
}
if (id) {
    n->id = *id;
    *id = NULL;
}
n->type = type;
if (type == SND_CONFIG_TYPE_COMPOUND)
    INIT_LIST_HEAD(&n->u.compound.fields);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1144) - [public] INIT_LIST_HEAD

```
    INIT_LIST_HEAD(&n->u.compound.fields);
    *config = n;
    return 0;
}
```

```
static int _snd_config_make_add(snd_config_t **config, char **id,
    snd_config_type_t type, snd_config_t *parent)
{
    snd_config_t *n;
    int err;
    assert(parent->type == SND_CONFIG_TYPE_COMPOUND);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1171) - [public] if

```
    if (len < 0) {
        if (strcmp(n->id, id) != 0)
            continue;
    } else if (strlen(n->id) != (size_t) len ||
        memcmp(n->id, id, (size_t) len) != 0)
        continue;
    if (result)
        *result = n;
    return 0;
}
return -ENOENT;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1194) - [public] free

```
    free(s);
    return 0;
}
if (err == 0 && ((s[0] >= '0' && s[0] <= '9') || s[0] == '-')) {
    long long i;
    errno = 0;
    err = safe_strtoll(s, &i);
    if (err < 0) {
        double r;
        err = safe_strtod(s, &r);
        if (err >= 0) {
            free(s);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 1201) - [public] if

```
if (err < 0) {
    double r;
    err = safe_strtod(s, &r);
    if (err >= 0) {
        free(s);
        if (n) {
            if (n->type != SND_CONFIG_TYPE_REAL) {
                snd_error(CORE, "%s is not a real", *id);
                return -EINVAL;
            }
        } else {
            err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_REAL, parent);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 1204) - [public] if

```
if (err >= 0) {
    free(s);
    if (n) {
        if (n->type != SND_CONFIG_TYPE_REAL) {
            snd_error(CORE, "%s is not a real", *id);
            return -EINVAL;
        }
    } else {
        err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_REAL, parent);
        if (err < 0)
            return err;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 1205) - [public] free

```
free(s);
if (n) {
    if (n->type != SND_CONFIG_TYPE_REAL) {
        snd_error(CORE, "%s is not a real", *id);
        return -EINVAL;
    }
} else {
    err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_REAL, parent);
    if (err < 0)
        return err;
}
n->u.real = r;
// ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 1206) - [public] if

```

    if (n) {
        if (n->type != SND_CONFIG_TYPE_REAL) {
            snd_error(CORE, "%s is not a real", *id);
            return -EINVAL;
        }
    } else {
        err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_REAL, parent);
        if (err < 0)
            return err;
    }
    n->u.real = r;
    *_n = n;
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/conf.c (line 1207) - [public] if

```

    if (n->type != SND_CONFIG_TYPE_REAL) {
        snd_error(CORE, "%s is not a real", *id);
        return -EINVAL;
    }
} else {
    err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_REAL, parent);
    if (err < 0)
        return err;
}
n->u.real = r;
*_n = n;
return 0;
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/conf.c (line 1208) - [public] snd_error

```

    snd_error(CORE, "%s is not a real", *id);
    return -EINVAL;
}
} else {
    err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_REAL, parent);
    if (err < 0)
        return err;
}
n->u.real = r;
*_n = n;
return 0;
}
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/conf.c (line 1221) - [public] free

```

    free(s);
    if (n) {
        if (n->type != SND_CONFIG_TYPE_INTEGER && n->type != SND_CONFIG_TYPE_INTEGER64) {
            snd_error(CORE, "%s is not an integer", *id);
            return -EINVAL;
        }
    } else {
        if (i <= INT_MAX)
            err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_INTEGER, parent);
        else
            err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_INTEGER64, parent);
        if (err < 0)
            // ... (truncated, see source file for more)
    }
}

```

dirtycode/alsa-lib/src/conf.c (line 1222) - [public] if

```
    if (n) {
        if (n->type != SND_CONFIG_TYPE_INTEGER && n->type != SND_CONFIG_TYPE_INTEGER64) {
            snd_error(CORE, "%s is not an integer", *id);
            return -EINVAL;
        }
    } else {
        if (i <= INT_MAX)
            err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_INTEGER, parent);
        else
            err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_INTEGER64, parent);
        if (err < 0)
            return err;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 1223) - [public] if

```
    if (n->type != SND_CONFIG_TYPE_INTEGER && n->type != SND_CONFIG_TYPE_INTEGER64) {
        snd_error(CORE, "%s is not an integer", *id);
        return -EINVAL;
    }
} else {
    if (i <= INT_MAX)
        err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_INTEGER, parent);
    else
        err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_INTEGER64, parent);
    if (err < 0)
        return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1224) - [public] snd_error

```
    snd_error(CORE, "%s is not an integer", *id);
    return -EINVAL;
}
} else {
    if (i <= INT_MAX)
        err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_INTEGER, parent);
    else
        err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_INTEGER64, parent);
    if (err < 0)
        return err;
}
if (n->type == SND_CONFIG_TYPE_INTEGER)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1244) - [public] if

```
    if (n->type != SND_CONFIG_TYPE_STRING) {
        snd_error(CORE, "%s is not a string", *id);
        free(s);
        return -EINVAL;
    }
} else {
    err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_STRING, parent);
    if (err < 0)
        return err;
}
free(n->u.string);
n->u.string = s;
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 1245) - [public] snd_error

```
    ■■■snd_error(CORE, "%s is not a string", *id);
    ■■■free(s);
    ■■■return -EINVAL;
    ■■}
    ■} else {
    ■■err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_STRING, parent);
    ■■if (err < 0)
    ■■■return err;
    ■■}
    ■free(n->u.string);
    ■n->u.string = s;
    ■*_n = n;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1246) - [public] free

```
    ■■■free(s);
    ■■■return -EINVAL;
    ■■}
    ■} else {
    ■■err = _snd_config_make_add(&n, id, SND_CONFIG_TYPE_STRING, parent);
    ■■if (err < 0)
    ■■■return err;
    ■■}
    ■free(n->u.string);
    ■n->u.string = s;
    ■*_n = n;
    ■return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1260) - [public] parse_defs

```
static int parse_defs(snd_config_t *parent, input_t *input, int skip, int override);
static int parse_array_defs(snd_config_t *farther, input_t *input, int skip, int override);

static int parse_array_def(snd_config_t *parent, input_t *input, int *idx, int skip, int override)
{
    ■char *id = NULL;
    ■int c;
    ■int err;
    ■snd_config_t *n = NULL;

    ■if (!skip) {
    ■■char static_id[12];
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1261) - [public] parse_array_defs

```
static int parse_array_defs(snd_config_t *farther, input_t *input, int skip, int override);

static int parse_array_def(snd_config_t *parent, input_t *input, int *idx, int skip, int override)
{
    ■char *id = NULL;
    ■int c;
    ■int err;
    ■snd_config_t *n = NULL;

    ■if (!skip) {
    ■■char static_id[12];
    ■■while (1) {
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1272) - [public] while

```
while (1) {
    snprintf(static_id, sizeof(static_id), "%i", *idx);
    if (_snd_config_search(parent, static_id, -1, &n) == 0) {
        if (override) {
            snd_config_delete(n);
            /* fallthrough to break */
        } else {
            /* merge */
            (*idx)++;
            continue;
        }
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 1273) - [public] snprintf

```
snprintf(static_id, sizeof(static_id), "%i", *idx);
if (_snd_config_search(parent, static_id, -1, &n) == 0) {
    if (override) {
        snd_config_delete(n);
        /* fallthrough to break */
    } else {
        /* merge */
        (*idx)++;
        continue;
    }
}
break;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1274) - [public] if

```
if (_snd_config_search(parent, static_id, -1, &n) == 0) {
    if (override) {
        snd_config_delete(n);
        /* fallthrough to break */
    } else {
        /* merge */
        (*idx)++;
        continue;
    }
}
break;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1275) - [public] if

```
if (override) {
    snd_config_delete(n);
    /* fallthrough to break */
} else {
    /* merge */
    (*idx)++;
    continue;
}
break;
n = NULL;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1276) - [public] snd_config_delete

```

    snd_config_delete(n);
    /* fallthrough to break */
} else {
    /* merge */
    (*idx)++;
    continue;
}
}
break;
}
n = NULL;
id = strdup(static_id);
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/conf.c (line 1301) - [public] if

```

if (!skip) {
    #if 0 /* n is always NULL for this moment */
    if (n) {
        if (n->type != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "%s is not a compound", id);
            err = -EINVAL;
            goto __end;
        }
    } else
    #else
    assert(n == NULL);
    #endif
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/conf.c (line 1303) - [public] if

```
if (!skip) {
    char static_id[12];
    while (1) {
        snprintf(static_id, sizeof(static_id), "%i", *idx);
        if (_snd_config_search(parent, static_id, -1, &n) == 0) {
            if (override) {
                snd_config_delete(n);
                /* fallthrough to break */
            } else {
                /* merge */
                (*idx)++;
                continue;
            }
        }
        break;
    }
    n = NULL;
    id = strdup(static_id);
    if (id == NULL)
        return -ENOMEM;
}
c = get_nonwhite(input);
if (c < 0) {
    err = c;
    goto __end;
}
switch (c) {
case '!':
case '[':
{
    char endchr;
    if (!skip) {
        #if 0 /* n is always NULL for this moment */
        if (n) {
            if (n->type != SND_CONFIG_TYPE_COMPOUND) {
                snd_error(CORE, "%s is not a compound", id);
                err = -EINVAL;
                goto __end;
            }
        } else
        #else
        assert(n == NULL);
        #endif
        {
            err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 1304) - [public] if

```
if (n->type != SND_CONFIG_TYPE_COMPOUND) {
    snd_error(CORE, "%s is not a compound", id);
    err = -EINVAL;
    goto __end;
} else
#else
assert(n == NULL);
#endif
{
    err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
    if (err < 0)
        // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 1305) - [public] snd_error

```
#####snd_error(CORE, "%s is not a compound", id);
#####err = -EINVAL;
#####goto __end;
#####}
#####} else
#else
#####assert(n == NULL);
#endif
#####{
#####err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
#####if (err < 0)
#####goto __end;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1311) - [public] assert

```
#####assert(n == NULL);
#endif
#####{
#####err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
#####if (err < 0)
#####goto __end;
#####}
#####}
#####if (c == '{') {
#####err = parse_defs(n, input, skip, override);
#####endchr = '}';
#####} else {
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1329) - [public] if

```
#####if (c < 0) {
#####err = c;
#####goto __end;
#####}
#####if (c != endchr) {
#####if (n)
#####snd_config_delete(n);
#####err = LOCAL_UNEXPECTED_CHAR;
#####goto __end;
#####}
#####break;
#####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1333) - [public] if

```
#####if (c != endchr) {
#####if (n)
#####snd_config_delete(n);
#####err = LOCAL_UNEXPECTED_CHAR;
#####goto __end;
#####}
#####break;
#####}
#####default:
#####unget_char(c, input);
#####err = parse_value(&n, parent, input, &id, skip);
#####if (err < 0)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1335) - [public] snd_config_delete

```
    ■■■■snd_config_delete(n);
    ■■■■err = LOCAL_UNEXPECTED_CHAR;
    ■■■■goto __end;
    ■■}
    ■■break;
    ■■}
    ■■default:
    ■■unget_char(c, input);
    ■■err = parse_value(&n, parent, input, &id, skip);
    ■■if (err < 0)
    ■■■■goto __end;
    ■■break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1342) - [public] unget_char

```
    ■■unget_char(c, input);
    ■■err = parse_value(&n, parent, input, &id, skip);
    ■■if (err < 0)
    ■■■■goto __end;
    ■■break;
    ■■}
    ■■err = 0;
    __end:
    ■■free(id);
    ■■return err;
    }
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 1361) - [public] unget_char

```
    ■■unget_char(c, input);
    ■■if (c == ']')
    ■■■■return 0;
    ■■err = parse_array_def(parent, input, &idx, skip, override);
    ■■if (err < 0)
    ■■■■return err;
    ■■idx++;
    ■■}
    ■■return 0;
    }
```

```
static int parse_def(snd_config_t *parent, input_t *input, int skip, int override)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1383) - [public] switch

```
    ■■switch (c) {
    ■■case '+':
    ■■■■mode = MERGE_CREATE;
    ■■■■break;
    ■■case '-':
    ■■■■mode = MERGE;
    ■■■■break;
    ■■case '?':
    ■■■■mode = DONT_OVERRIDE;
    ■■■■break;
    ■■case '!':
    ■■■■mode = OVERRIDE;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1398) - [public] unget_char

```
    unget_char(c, input);
}
err = get_string(&id, 1, input);
if (err < 0)
    return err;
c = get_nonwhite(input);
if (c != '.')
    break;
if (skip) {
    free(id);
    continue;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1406) - [public] if

```
if (skip) {
    free(id);
    continue;
}
if (_snd_config_search(parent, id, -1, &n) == 0) {
    if (mode == DONT_OVERRIDE) {
        skip = 1;
        free(id);
        continue;
    }
    if (mode != OVERRIDE) {
        if (n->type != SND_CONFIG_TYPE_COMPOUND) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 1407) - [public] free

```
free(id);
continue;
}
if (_snd_config_search(parent, id, -1, &n) == 0) {
    if (mode == DONT_OVERRIDE) {
        skip = 1;
        free(id);
        continue;
    }
    if (mode != OVERRIDE) {
        if (n->type != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "%s is not a compound", id);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 1410) - [public] if

```
if (_snd_config_search(parent, id, -1, &n) == 0) {
    if (mode == DONT_OVERRIDE) {
        skip = 1;
        free(id);
        continue;
    }
    if (mode != OVERRIDE) {
        if (n->type != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "%s is not a compound", id);
            return -EINVAL;
        }
        n->u.compound.join = true;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 1411) - [public] if

```
    if (mode == DONT_OVERRIDE) {
        skip = 1;
        free(id);
        continue;
    }
    if (mode != OVERRIDE) {
        if (n->type != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "%s is not a compound", id);
            return -EINVAL;
        }
        n->u.compound.join = true;
        parent = n;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1413) - [public] free

```
        free(id);
        continue;
    }
    if (mode != OVERRIDE) {
        if (n->type != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "%s is not a compound", id);
            return -EINVAL;
        }
        n->u.compound.join = true;
        parent = n;
        free(id);
        continue;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1416) - [public] if

```
    if (mode != OVERRIDE) {
        if (n->type != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "%s is not a compound", id);
            return -EINVAL;
        }
        n->u.compound.join = true;
        parent = n;
        free(id);
        continue;
    }
    snd_config_delete(n);
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1417) - [public] if

```
    if (n->type != SND_CONFIG_TYPE_COMPOUND) {
        snd_error(CORE, "%s is not a compound", id);
        return -EINVAL;
    }
    n->u.compound.join = true;
    parent = n;
    free(id);
    continue;
}
snd_config_delete(n);
}
if (mode == MERGE) {
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 1418) - [public] snd_error

```
#####snd_error(CORE, "%s is not a compound", id);
#####return -EINVAL;
#####}
#####n->u.compound.join = true;
#####parent = n;
#####free(id);
#####continue;
#####}
#####snd_config_delete(n);
#####}
#####if (mode == MERGE) {
#####snd_error(CORE, "%s does not exists", id);
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1423) - [public] free

```
#####free(id);
#####continue;
#####}
#####snd_config_delete(n);
#####}
#####if (mode == MERGE) {
#####snd_error(CORE, "%s does not exists", id);
#####err = -ENOENT;
#####goto __end;
#####}
#####err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
#####if (err < 0)
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1426) - [public] snd_config_delete

```
#####snd_config_delete(n);
#####}
#####if (mode == MERGE) {
#####snd_error(CORE, "%s does not exists", id);
#####err = -ENOENT;
#####goto __end;
#####}
#####err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
#####if (err < 0)
#####goto __end;
#####n->u.compound.join = true;
#####parent = n;
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1428) - [public] if

```
#####if (mode == MERGE) {
#####snd_error(CORE, "%s does not exists", id);
#####err = -ENOENT;
#####goto __end;
#####}
#####err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
#####if (err < 0)
#####goto __end;
#####n->u.compound.join = true;
#####parent = n;
#####}
#####if (c == '=') {
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1429) - [public] snd_error

```
    snd_error(CORE, "%s does not exists", id);
    err = -ENOENT;
    goto __end;
}
err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
if (err < 0)
    goto __end;
n->u.compound.join = true;
parent = n;
}
if (c == '=') {
    c = get_nonwhite(input);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1445) - [public] if

```
if (_snd_config_search(parent, id, -1, &n) == 0) {
    if (mode == DONT_OVERRIDE) {
        skip = 1;
        n = NULL;
    } else if (mode == OVERRIDE) {
        snd_config_delete(n);
        n = NULL;
    }
} else {
    n = NULL;
    if (mode == MERGE) {
        snd_error(CORE, "%s does not exists", id);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1446) - [public] if

```
if (mode == DONT_OVERRIDE) {
    skip = 1;
    n = NULL;
} else if (mode == OVERRIDE) {
    snd_config_delete(n);
    n = NULL;
}
} else {
    n = NULL;
    if (mode == MERGE) {
        snd_error(CORE, "%s does not exists", id);
        err = -ENOENT;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1450) - [public] snd_config_delete

```
snd_config_delete(n);
n = NULL;
}
} else {
    n = NULL;
    if (mode == MERGE) {
        snd_error(CORE, "%s does not exists", id);
        err = -ENOENT;
        goto __end;
    }
}
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1455) - [public] if

```
    if (mode == MERGE) {
        snd_error(CORE, "%s does not exists", id);
        err = -ENOENT;
        goto __end;
    }
}
switch (c) {
case '{':
case '[':
{
    char endchr;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1456) - [public] snd_error

```
    snd_error(CORE, "%s does not exists", id);
    err = -ENOENT;
    goto __end;
}
}
switch (c) {
case '{':
case '[':
{
    char endchr;
    if (!skip) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1467) - [public] if

```
    if (!skip) {
        if (n) {
            if (n->type != SND_CONFIG_TYPE_COMPOUND) {
                snd_error(CORE, "%s is not a compound", id);
                err = -EINVAL;
                goto __end;
            }
        } else {
            err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
            if (err < 0)
                goto __end;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1468) - [public] if

```
    if (n) {
        if (n->type != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "%s is not a compound", id);
            err = -EINVAL;
            goto __end;
        }
    } else {
        err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
        if (err < 0)
            goto __end;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1469) - [public] if

```
#####if (n->type != SND_CONFIG_TYPE_COMPOUND) {
#####snd_error(CORE, "%s is not a compound", id);
#####err = -EINVAL;
#####goto __end;
#####}
#####} else {
#####err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
#####if (err < 0)
#####goto __end;
#####}
#####}
#####}
#####if (c == '{') {
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1470) - [public] snd_error

```
#####snd_error(CORE, "%s is not a compound", id);
#####err = -EINVAL;
#####goto __end;
#####}
#####} else {
#####err = _snd_config_make_add(&n, &id, SND_CONFIG_TYPE_COMPOUND, parent);
#####if (err < 0)
#####goto __end;
#####}
#####}
#####}
#####if (c == '{') {
#####err = parse_defs(n, input, skip, override);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1488) - [public] if

```
#####if (c < 0) {
#####err = c;
#####goto __end;
#####}
#####if (c != endchr) {
#####if (n)
#####snd_config_delete(n);
#####err = LOCAL_UNEXPECTED_CHAR;
#####goto __end;
#####}
#####break;
#####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1492) - [public] if

```
#####if (c != endchr) {
#####if (n)
#####snd_config_delete(n);
#####err = LOCAL_UNEXPECTED_CHAR;
#####goto __end;
#####}
#####break;
#####}
#####default:
#####unget_char(c, input);
#####err = parse_value(&n, parent, input, &id, skip);
#####if (err < 0)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1494) - [public] snd_config_delete

```
■■■■snd_config_delete(n);
■■■■err = LOCAL_UNEXPECTED_CHAR;
■■■■goto __end;
■■}
■■break;
■■}
■default:
■■unget_char(c, input);
■■err = parse_value(&n, parent, input, &id, skip);
■■if (err < 0)
■■■goto __end;
■■break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1501) - [public] unget_char

```
■■unget_char(c, input);
■■err = parse_value(&n, parent, input, &id, skip);
■■if (err < 0)
■■■goto __end;
■■break;
■■}
■c = get_nonwhite(input);
■switch (c) {
■case ';':
■case ',':
■■break;
■default:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1513) - [public] unget_char

```
■■unget_char(c, input);
■■}
    __end:
■free(id);
■return err;
}
```

```
static int parse_defs(snd_config_t *parent, input_t *input, int skip, int override)
{
    int c, err;
    while (1) {
        c = get_nonwhite(input);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1527) - [public] unget_char

```
■■unget_char(c, input);
■■if (c == '}')
■■■return 0;
■■err = parse_def(parent, input, skip, override);
■■if (err < 0)
■■■return err;
■■}
■return 0;
}
```

```
static void string_print(char *str, int id, snd_output_t *out)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1542) - [public] snd_output_puts

```
    ■■snd_output_puts(out, "");
    ■■return;
    ■}
    ■if (!id) {
    ■■switch (*p) {
    ■■case '0': case '1': case '2': case '3': case '4':
    ■■case '5': case '6': case '7': case '8': case '9':
    ■■case '-':
    ■■goto quoted;
    ■■}
    ■}
    loop:
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1546) - [public] switch

```
    ■■switch (*p) {
    ■■case '0': case '1': case '2': case '3': case '4':
    ■■case '5': case '6': case '7': case '8': case '9':
    ■■case '-':
    ■■goto quoted;
    ■■}
    ■}
    loop:
    ■switch (*p) {
    ■case 0:
    ■■goto nonquoted;
    ■case ' ':
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1587) - [public] switch

```
    ■■switch (c) {
    ■■case '\n':
    ■■snd_output_putc(out, '\\');
    ■■snd_output_putc(out, 'n');
    ■■break;
    ■■case '\t':
    ■■snd_output_putc(out, '\\');
    ■■snd_output_putc(out, 't');
    ■■break;
    ■■case '\v':
    ■■snd_output_putc(out, '\\');
    ■■snd_output_putc(out, 'v');
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1589) - [public] snd_output_putc

```
    ■■snd_output_putc(out, '\\');
    ■■snd_output_putc(out, 'n');
    ■■break;
    ■■case '\t':
    ■■snd_output_putc(out, '\\');
    ■■snd_output_putc(out, 't');
    ■■break;
    ■■case '\v':
    ■■snd_output_putc(out, '\\');
    ■■snd_output_putc(out, 'v');
    ■■break;
    ■■case '\b':
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1590) - [public] snd_output_putc

```
    snd_output_putc(out, 'n');
    break;
    case '\t':
        snd_output_putc(out, '\\');
        snd_output_putc(out, 't');
        break;
    case '\v':
        snd_output_putc(out, '\\');
        snd_output_putc(out, 'v');
        break;
    case '\b':
        snd_output_putc(out, '\\');
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1593) - [public] snd_output_putc

```
    snd_output_putc(out, '\\');
    snd_output_putc(out, 't');
    break;
    case '\v':
        snd_output_putc(out, '\\');
        snd_output_putc(out, 'v');
        break;
    case '\b':
        snd_output_putc(out, '\\');
        snd_output_putc(out, 'b');
        break;
    case '\r':
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1594) - [public] snd_output_putc

```
    snd_output_putc(out, 't');
    break;
    case '\v':
        snd_output_putc(out, '\\');
        snd_output_putc(out, 'v');
        break;
    case '\b':
        snd_output_putc(out, '\\');
        snd_output_putc(out, 'b');
        break;
    case '\r':
        snd_output_putc(out, '\\');
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1597) - [public] snd_output_putc

```
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'v');
    break;
    case '\b':
        snd_output_putc(out, '\\');
        snd_output_putc(out, 'b');
        break;
    case '\r':
        snd_output_putc(out, '\\');
        snd_output_putc(out, 'r');
        break;
    case '\f':
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1598) - [public] snd_output_putc

```
    snd_output_putc(out, 'v');
    break;
    case '\b':
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'b');
    break;
    case '\r':
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'r');
    break;
    case '\f':
    snd_output_putc(out, '\\');
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1601) - [public] snd_output_putc

```
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'b');
    break;
    case '\r':
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'r');
    break;
    case '\f':
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'f');
    break;
    default:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1602) - [public] snd_output_putc

```
    snd_output_putc(out, 'b');
    break;
    case '\r':
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'r');
    break;
    case '\f':
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'f');
    break;
    default:
    if (c == q) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1605) - [public] snd_output_putc

```
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'r');
    break;
    case '\f':
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'f');
    break;
    default:
    if (c == q) {
    snd_output_putc(out, '\\');
    snd_output_putc(out, c);
    } else {
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 1606) - [public] snd_output_putc

```
    snd_output_putc(out, 'r');
    break;
case '\f':
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'f');
    break;
default:
    if (c == q) {
        snd_output_putc(out, '\\');
        snd_output_putc(out, c);
    } else {
        if (c >= 32 && c <= 126)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1609) - [public] snd_output_putc

```
    snd_output_putc(out, '\\');
    snd_output_putc(out, 'f');
    break;
default:
    if (c == q) {
        snd_output_putc(out, '\\');
        snd_output_putc(out, c);
    } else {
        if (c >= 32 && c <= 126)
            snd_output_putc(out, c);
        else
            snd_output_printf(out, "\\%04o", c);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1610) - [public] snd_output_putc

```
    snd_output_putc(out, 'f');
    break;
default:
    if (c == q) {
        snd_output_putc(out, '\\');
        snd_output_putc(out, c);
    } else {
        if (c >= 32 && c <= 126)
            snd_output_putc(out, c);
        else
            snd_output_printf(out, "\\%04o", c);
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1613) - [public] if

```
    if (c == q) {
        snd_output_putc(out, '\\');
        snd_output_putc(out, c);
    } else {
        if (c >= 32 && c <= 126)
            snd_output_putc(out, c);
        else
            snd_output_printf(out, "\\%04o", c);
    }
    break;
}
p++;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1614) - [public] snd_output_putc

```
    ■■■■snd_output_putc(out, '\\');
    ■■■■snd_output_putc(out, c);
    ■■■} else {
    ■■■■if (c >= 32 && c <= 126)
    ■■■■snd_output_putc(out, c);
    ■■■■else
    ■■■■snd_output_printf(out, "\\%04o", c);
    ■■■}
    ■■■break;
    ■■■}
    ■■■p++;
    ■■■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1615) - [public] snd_output_putc

```
    ■■■■snd_output_putc(out, c);
    ■■■} else {
    ■■■■if (c >= 32 && c <= 126)
    ■■■■snd_output_putc(out, c);
    ■■■■else
    ■■■■snd_output_printf(out, "\\%04o", c);
    ■■■}
    ■■■break;
    ■■■}
    ■■■p++;
    ■■■}
    ■■■snd_output_putc(out, q);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1618) - [public] snd_output_putc

```
    ■■■■snd_output_putc(out, c);
    ■■■■else
    ■■■■snd_output_printf(out, "\\%04o", c);
    ■■■}
    ■■■break;
    ■■■}
    ■■■p++;
    ■■■}
    ■■■snd_output_putc(out, q);
    }

    static void level_print(snd_output_t *out, unsigned int level)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1620) - [public] snd_output_printf

```
    ■■■■snd_output_printf(out, "\\%04o", c);
    ■■■}
    ■■■break;
    ■■■}
    ■■■p++;
    ■■■}
    ■■■snd_output_putc(out, q);
    }

    static void level_print(snd_output_t *out, unsigned int level)
    {
    ■■■char a[level + 1];
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1647) - [public] snd_output_printf

```
    ■■snd_output_printf(out, "%ld", n->u.integer);
    ■■break;
    ■case SND_CONFIG_TYPE_INTEGER64:
    ■■snd_output_printf(out, "%lld", n->u.integer64);
    ■■break;
    ■case SND_CONFIG_TYPE_REAL:
    ■■snd_output_printf(out, "%-16g", n->u.real);
    ■■break;
    ■case SND_CONFIG_TYPE_STRING:
    ■■string_print(n->u.string, 0, out);
    ■■break;
    ■case SND_CONFIG_TYPE_POINTER:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1650) - [public] snd_output_printf

```
    ■■snd_output_printf(out, "%lld", n->u.integer64);
    ■■break;
    ■case SND_CONFIG_TYPE_REAL:
    ■■snd_output_printf(out, "%-16g", n->u.real);
    ■■break;
    ■case SND_CONFIG_TYPE_STRING:
    ■■string_print(n->u.string, 0, out);
    ■■break;
    ■case SND_CONFIG_TYPE_POINTER:
    ■■snd_error(CORE, "cannot save runtime pointer type");
    ■■return -EINVAL;
    ■case SND_CONFIG_TYPE_COMPOUND:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1653) - [public] snd_output_printf

```
    ■■snd_output_printf(out, "%-16g", n->u.real);
    ■■break;
    ■case SND_CONFIG_TYPE_STRING:
    ■■string_print(n->u.string, 0, out);
    ■■break;
    ■case SND_CONFIG_TYPE_POINTER:
    ■■snd_error(CORE, "cannot save runtime pointer type");
    ■■return -EINVAL;
    ■case SND_CONFIG_TYPE_COMPOUND:
    ■■array = snd_config_is_array(n);
    ■■snd_output_putc(out, array ? '[' : '{');
    ■■snd_output_putc(out, '\n');
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1656) - [public] string_print

```
    ■■string_print(n->u.string, 0, out);
    ■■break;
    ■case SND_CONFIG_TYPE_POINTER:
    ■■snd_error(CORE, "cannot save runtime pointer type");
    ■■return -EINVAL;
    ■case SND_CONFIG_TYPE_COMPOUND:
    ■■array = snd_config_is_array(n);
    ■■snd_output_putc(out, array ? '[' : '{');
    ■■snd_output_putc(out, '\n');
    ■■err = _snd_config_save_children(n, out, level + 1, 0, array);
    ■■if (err < 0)
    ■■■return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1659) - [public] snd_error

```
    ■■snd_error(CORE, "cannot save runtime pointer type");
    ■■return -EINVAL;
    ■■case SND_CONFIG_TYPE_COMPOUND:
    ■■array = snd_config_is_array(n);
    ■■snd_output_putc(out, array ? '[' : '{');
    ■■snd_output_putc(out, '\n');
    ■■err = _snd_config_save_children(n, out, level + 1, 0, array);
    ■■if (err < 0)
    ■■return err;
    ■■level_print(out, level);
    ■■snd_output_putc(out, array ? ']' : '}');
    ■■break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1664) - [public] snd_output_putc

```
    ■■snd_output_putc(out, '\n');
    ■■err = _snd_config_save_children(n, out, level + 1, 0, array);
    ■■if (err < 0)
    ■■return err;
    ■■level_print(out, level);
    ■■snd_output_putc(out, array ? ']' : '}');
    ■■break;
    ■}
    ■return 0;
}
```

```
static void id_print(snd_config_t *n, snd_output_t *out, unsigned int joins)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1668) - [public] level_print

```
    ■■level_print(out, level);
    ■■snd_output_putc(out, array ? ']' : '}');
    ■■break;
    ■}
    ■return 0;
}
```

```
static void id_print(snd_config_t *n, snd_output_t *out, unsigned int joins)
{
    ■if (joins > 0) {
    ■■assert(n->parent);
    ■■id_print(n->parent, out, joins - 1);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1669) - [public] snd_output_putc

```
    ■■snd_output_putc(out, array ? ']' : '}');
    ■■break;
    ■}
    ■return 0;
}
```

```
static void id_print(snd_config_t *n, snd_output_t *out, unsigned int joins)
{
    ■if (joins > 0) {
    ■■assert(n->parent);
    ■■id_print(n->parent, out, joins - 1);
    ■■snd_output_putc(out, '.');
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1678) - [public] assert

```
    assert(n->parent);
    id_print(n->parent, out, joins - 1);
    snd_output_putc(out, '.');
}
string_print(n->id, 1, out);
}

static int _snd_config_save_children(snd_config_t *config, snd_output_t *out,
    unsigned int level, unsigned int joins,
    int array)
{
    int err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1679) - [public] id_print

```
    id_print(n->parent, out, joins - 1);
    snd_output_putc(out, '.');
}
string_print(n->id, 1, out);
}

static int _snd_config_save_children(snd_config_t *config, snd_output_t *out,
    unsigned int level, unsigned int joins,
    int array)
{
    int err;
    snd_config_iterator_t i, next;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1680) - [public] snd_output_putc

```
    snd_output_putc(out, '.');
}
string_print(n->id, 1, out);
}

static int _snd_config_save_children(snd_config_t *config, snd_output_t *out,
    unsigned int level, unsigned int joins,
    int array)
{
    int err;
    snd_config_iterator_t i, next;
    assert(config && out);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1701) - [public] level_print

```
    level_print(out, level);
    if (!array) {
        id_print(n, out, joins);
        snd_output_putc(out, ' ');
    }
    #if 0
        snd_output_putc(out, '=');
    #endif
}
err = _snd_config_save_node_value(n, out, level);
if (err < 0)
    return err;
    #if 0
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1702) - [public] if

```
    if (!array) {
        id_print(n, out, joins);
        snd_output_putc(out, ' ');
        if 0
            snd_output_putc(out, '=');
        #endif
    }
    err = _snd_config_save_node_value(n, out, level);
    if (err < 0)
        return err;
    if 0
        snd_output_putc(out, ';');
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1703) - [public] id_print

```
    id_print(n, out, joins);
    snd_output_putc(out, ' ');
    if 0
        snd_output_putc(out, '=');
    #endif
}
err = _snd_config_save_node_value(n, out, level);
if (err < 0)
    return err;
if 0
    snd_output_putc(out, ';');
#endif
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1704) - [public] snd_output_putc

```
    snd_output_putc(out, ' ');
    if 0
        snd_output_putc(out, '=');
    #endif
}
err = _snd_config_save_node_value(n, out, level);
if (err < 0)
    return err;
if 0
    snd_output_putc(out, ';');
#endif
snd_output_putc(out, '\n');
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1706) - [public] snd_output_putc

```
    snd_output_putc(out, '=');
    #endif
}
err = _snd_config_save_node_value(n, out, level);
if (err < 0)
    return err;
if 0
    snd_output_putc(out, ';');
#endif
snd_output_putc(out, '\n');
}
return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1715) - [public] snd_output_putc

```
■snd_output_putc(out, '\n');
■}
■return 0;
}
#endif /* DOC_HIDDEN */

/**
 * \brief Substitutes one configuration node to another.
 * \param dst Handle to the destination node.
 * \param src Handle to the source node. Must not be the same as \a dst.
 * \return Zero if successful, otherwise a negative error code.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1753) - [public] snd_config_for_each

```
■snd_config_for_each(i, next, src) {
■    ■snd_config_t *n = snd_config_iterator_entry(i);
■    ■n->parent = dst;
■}
■src->u.compound.fields.next->prev = &dst->u.compound.fields;
■src->u.compound.fields.prev->next = &dst->u.compound.fields;
■}
■free(dst->id);
■if (dst->type == SND_CONFIG_TYPE_STRING)
■    ■free(dst->u.string);
■if (src->parent)■/* like snd_config_remove */
■    ■list_del(&src->list);
■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1762) - [public] free

```
■free(dst->u.string);
■if (src->parent)■/* like snd_config_remove */
■    ■list_del(&src->list);
■dst->id = src->id;
■dst->type = src->type;
■dst->u = src->u;
■free(src);
■return 0;
}

/**
 * \brief Converts an ASCII string to a configuration node type.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1764) - [public] list_del

```
/**
 * \brief Substitutes one configuration node to another.
 * \param dst Handle to the destination node.
 * \param src Handle to the source node. Must not be the same as \a dst.
 * \return Zero if successful, otherwise a negative error code.
 *
 * If both nodes are compounds, the source compound node members will
 * be moved to the destination compound node. The original destination
 * compound node members will be deleted (overwritten).
 *
 * If the destination node is a compound and the source node is
 * an ordinary type, the compound members are deleted (including
 * their contents).
 *
 * Otherwise, the source node's value replaces the destination node's
 * value.
 *
 * In any case, a successful call to this function frees the source
 * node.
 */
int snd_config_substitute(snd_config_t *dst, snd_config_t *src)
{
    assert(dst && src && src != dst);
    if (dst->type == SND_CONFIG_TYPE_COMPOUND) {
        int err = snd_config_delete_compound_members(dst);
        if (err < 0)
            return err;
    }
    if (dst->type == SND_CONFIG_TYPE_COMPOUND &&
        src->type == SND_CONFIG_TYPE_COMPOUND) { /* overwrite */
        snd_config_iterator_t i, next;
        snd_config_for_each(i, next, src) {
            snd_config_t *n = snd_config_iterator_entry(i);
            n->parent = dst;
        }
        src->u.compound.fields.next->prev = &dst->u.compound.fields;
        src->u.compound.fields.prev->next = &dst->u.compound.fields;
    }
    free(dst->id);
    if (dst->type == SND_CONFIG_TYPE_STRING)
        free(dst->u.string);
    if (src->parent) /* like snd_config_remove */
        list_del(&src->list);
    dst->id = src->id;
    dst->type = src->type;
    dst->u = src->u;
    free(src);
    return 0;
}

/**
 * \brief Converts an ASCII string to a configuration node type.
 * \param[in] ascii A string containing a configuration node type.
 * \param[out] type The node type corresponding to \a ascii.
 * // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 1881) - [public] list_empty

```
return list_empty(&config->u.compound.fields);
}

/**
 * \brief Returns the id of a configuration node.
 * \param[in] config Handle to the configuration node.
 * \param[out] id The function puts the pointer to the id string at the
 *               address specified by \a id.
 * \return Zero if successful, otherwise a negative error code.
 *
 * The returned string is owned by the configuration node; the application
 * must not modify or delete it, and the string becomes invalid when the
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1928) - [public] if

```
if (config->parent) {
    snd_config_for_each(i, next, config->parent) {
        snd_config_t *n = snd_config_iterator_entry(i);
        if (n != config && strcmp(id, n->id) == 0)
            return -EEXIST;
    }
}
new_id = strdup(id);
if (!new_id)
    return -ENOMEM;
} else {
    if (config->parent)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1929) - [public] snd_config_for_each

```
snd_config_for_each(i, next, config->parent) {
    snd_config_t *n = snd_config_iterator_entry(i);
    if (n != config && strcmp(id, n->id) == 0)
        return -EEXIST;
}
}
new_id = strdup(id);
if (!new_id)
    return -ENOMEM;
} else {
    if (config->parent)
        return -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 1967) - [public] _snd_config_make

```
return _snd_config_make(config, 0, SND_CONFIG_TYPE_COMPOUND);
}

#ifdef DOC_HIDDEN
int _snd_config_load_with_include(snd_config_t *config, snd_input_t *in,
    int override, const char * const *include_paths)
{
    int err;
    input_t input;
    struct filedesc *fd, *fd_next;

    assert(config && in);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2005) - [public] switch

```
    switch (err) {
        case LOCAL_UNTERMINATED_STRING:
            str = "Unterminated string";
            err = -EINVAL;
            break;
        case LOCAL_UNTERMINATED_QUOTE:
            str = "Unterminated quote";
            err = -EINVAL;
            break;
        case LOCAL_UNEXPECTED_CHAR:
            str = "Unexpected char";
            err = -EINVAL;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2026) - [public] snd_error

```
    snd_error(CORE, "%s:%d:%d:%s", fd->name ? fd->name : "_toplevel_", fd->line, fd->column, str);
    goto _end;
}
err = get_char(&input);
fd = input.current;
if (err != LOCAL_UNEXPECTED_EOF) {
    snd_error(CORE, "%s:%d:%d:Unexpected }", fd->name ? fd->name : "", fd->line, fd->column);
    err = -EINVAL;
    goto _end;
}
err = 0;
_end:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2032) - [public] snd_error

```
    snd_error(CORE, "%s:%d:%d:Unexpected }", fd->name ? fd->name : "", fd->line, fd->column);
    err = -EINVAL;
    goto _end;
}
err = 0;
_end:
    while (fd->next) {
        fd_next = fd->next;
        snd_input_close(fd->in);
        free(fd->name);
        free_include_paths(fd);
        free(fd);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2040) - [public] snd_input_close

```
    snd_input_close(fd->in);
    free(fd->name);
    free_include_paths(fd);
    free(fd);
    fd = fd_next;
}

free_include_paths(fd);
free(fd);
return err;
}
#endif
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2041) - [public] free

```
    free(fd->name);
    free_include_paths(fd);
    free(fd);
    fd = fd_next;
}

free_include_paths(fd);
free(fd);
return err;
}
#endif
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 2042) - [public] free_include_paths

```
free_include_paths(fd);
free(fd);
fd = fd_next;
}

free_include_paths(fd);
free(fd);
return err;
}
#endif
```

```
/**
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2043) - [public] free

```
free(fd);
fd = fd_next;
}

free_include_paths(fd);
free(fd);
return err;
}
#endif
```

```
/**
 * \brief Loads a configuration tree.
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2071) - [public] _snd_config_load_with_include

```
return _snd_config_load_with_include(config, in, 0, NULL);
}

/**
 * \brief Loads a configuration tree from a string.
 * \param[out] config The function puts the handle to the configuration
 * node loaded from the file(s) at the address specified
 * by \a config.
 * \param[in] s String with the ASCII configuration
 * \param[in] size String size, if zero, a C string is expected (with termination)
 * \return Zero if successful, otherwise a negative error code.
 *
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2104) - [public] snd_input_close

```
    ■■snd_input_close(input);
    ■■return err;
    ■}
    ■err = snd_config_load(dst, input);
    ■snd_input_close(input);
    ■if (err < 0) {
    ■■snd_config_delete(dst);
    ■■return err;
    ■}
    ■*config = dst;
    ■return 0;
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2110) - [public] snd_config_delete

```
    ■■snd_config_delete(dst);
    ■■return err;
    ■}
    ■*config = dst;
    ■return 0;
    }

    /**
     * \brief Loads a configuration tree and overrides existing configuration nodes.
     * \param config Handle to a top level configuration node.
     * \param in Input handle to read the configuration from.
     * \return Zero if successful, otherwise a negative error code.
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2129) - [public] _snd_config_load_with_include

```
    ■return _snd_config_load_with_include(config, in, 1, NULL);
    }

    /**
     * \brief Adds a child to a compound configuration node.
     * \param parent Handle to a compound configuration node.
     * \param child Handle to the configuration node to be added.
     * \return Zero if successful, otherwise a negative error code.
     *
     * This function makes the node \a child a child of the node \a parent.
     *
     * The parent node then owns the child node, i.e., the child node gets
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2265) - [public] snd_config_remove

```
    ■■snd_config_remove(sn);
    ■■snprintf(id, sizeof(id), "%d", index++);
    ■■err = snd_config_set_id(sn, id);
    ■■if (err < 0) {
    ■■■snd_config_delete(sn);
    ■■■return err;
    ■■}
    ■■sn->parent = dst;
    ■■list_add_tail(&sn->list, &dst->u.compound.fields);
    ■}
    ■snd_config_delete(src);
    ■return 0;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2266) - [public] snprintf

```
    ■■snprintf(id, sizeof(id), "%d", index++);
    ■■err = snd_config_set_id(sn, id);
    ■■if (err < 0) {
    ■■■snd_config_delete(sn);
    ■■■return err;
    ■■}
    ■■sn->parent = dst;
    ■■list_add_tail(&sn->list, &dst->u.compound.fields);
    ■■}
    ■■snd_config_delete(src);
    ■■return 0;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2268) - [public] if

```
    ■■if (err < 0) {
    ■■■snd_config_delete(sn);
    ■■■return err;
    ■■}
    ■■sn->parent = dst;
    ■■list_add_tail(&sn->list, &dst->u.compound.fields);
    ■■}
    ■■snd_config_delete(src);
    ■■return 0;
    ■}

    /**
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2269) - [public] snd_config_delete

```
    ■■■snd_config_delete(sn);
    ■■■return err;
    ■■}
    ■■sn->parent = dst;
    ■■list_add_tail(&sn->list, &dst->u.compound.fields);
    ■■}
    ■■snd_config_delete(src);
    ■■return 0;
    ■}

    /**
    * \brief In-place merge of two config handles
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2273) - [public] list_add_tail

```
    ■■list_add_tail(&sn->list, &dst->u.compound.fields);
    ■■}
    ■■snd_config_delete(src);
    ■■return 0;
    ■}

    /**
    * \brief In-place merge of two config handles
    * \param[out] dst Config handle for the merged contents
    * \param[in] src Config handle to merge into dst (may be NULL)
    * \param[in] override Override flag
    * \return Zero if successful, otherwise a negative error code.
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2314) - [public] snd_config_substitute

```
    return snd_config_substitute(dst, src);
    array = snd_config_is_array(dst);
    if (array && snd_config_is_array(src))
        return _snd_config_array_merge(dst, src, array);
    snd_config_for_each(si, snext, src) {
        snd_config_t *sn = snd_config_iterator_entry(si);
        found = false;
        snd_config_for_each(di, dnext, dst) {
            snd_config_t *dn = snd_config_iterator_entry(di);
            if (strcmp(sn->id, dn->id) == 0) {
                if (override ||
                    sn->type != SND_CONFIG_TYPE_COMPOUND ||
                    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2317) - [public] _snd_config_array_merge

```
    return _snd_config_array_merge(dst, src, array);
    snd_config_for_each(si, snext, src) {
        snd_config_t *sn = snd_config_iterator_entry(si);
        found = false;
        snd_config_for_each(di, dnext, dst) {
            snd_config_t *dn = snd_config_iterator_entry(di);
            if (strcmp(sn->id, dn->id) == 0) {
                if (override ||
                    sn->type != SND_CONFIG_TYPE_COMPOUND ||
                    dn->type != SND_CONFIG_TYPE_COMPOUND) {
                    err = snd_config_substitute(dn, sn);
                    if (err < 0)
                        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2321) - [public] snd_config_for_each

```
    snd_config_for_each(di, dnext, dst) {
        snd_config_t *dn = snd_config_iterator_entry(di);
        if (strcmp(sn->id, dn->id) == 0) {
            if (override ||
                sn->type != SND_CONFIG_TYPE_COMPOUND ||
                dn->type != SND_CONFIG_TYPE_COMPOUND) {
                err = snd_config_substitute(dn, sn);
                if (err < 0)
                    return err;
            } else {
                err = snd_config_merge(dn, sn, 0);
                if (err < 0)
                    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2323) - [public] if

```
    if (strcmp(sn->id, dn->id) == 0) {
        if (override ||
            sn->type != SND_CONFIG_TYPE_COMPOUND ||
            dn->type != SND_CONFIG_TYPE_COMPOUND) {
            err = snd_config_substitute(dn, sn);
            if (err < 0)
                return err;
        } else {
            err = snd_config_merge(dn, sn, 0);
            if (err < 0)
                return err;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2339) - [public] if

```
    if (!found) {
        /* move config from src to dst */
        snd_config_remove(sn);
        sn->parent = dst;
        list_add_tail(&sn->list, &dst->u.compound.fields);
    }
}
snd_config_delete(src);
return 0;
}

/**
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2341) - [public] snd_config_remove

```
assert(dst);
if (src == NULL)
    return 0;
if (dst->type != SND_CONFIG_TYPE_COMPOUND || src->type != SND_CONFIG_TYPE_COMPOUND)
    return snd_config_substitute(dst, src);
array = snd_config_is_array(dst);
if (array && snd_config_is_array(src))
    return _snd_config_array_merge(dst, src, array);
snd_config_for_each(si, snext, src) {
    snd_config_t *sn = snd_config_iterator_entry(si);
    found = false;
    snd_config_for_each(di, dnext, dst) {
        snd_config_t *dn = snd_config_iterator_entry(di);
        if (strcmp(sn->id, dn->id) == 0) {
            if (override ||
                sn->type != SND_CONFIG_TYPE_COMPOUND ||
                dn->type != SND_CONFIG_TYPE_COMPOUND) {
                err = snd_config_substitute(dn, sn);
                if (err < 0)
                    return err;
            } else {
                err = snd_config_merge(dn, sn, 0);
                if (err < 0)
                    return err;
            }
            found = true;
            break;
        }
    }
    if (!found) {
        /* move config from src to dst */

        snd_config_remove(sn);
        sn->parent = dst;
        list_add_tail(&sn->list, &dst->u.compound.fields);
    }
    snd_config_delete(src);
    return 0;
}

/**
    * \brief Removes a configuration node from its tree.
    * \param config Handle to the configuration node to be removed.
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2343) - [public] list_add_tail

```
    list_add_tail(&sn->list, &dst->u.compound.fields);
}
}
snd_config_delete(src);
return 0;
}

/**
 * \brief Removes a configuration node from its tree.
 * \param config Handle to the configuration node to be removed.
 * \return Zero if successful, otherwise a negative error code.
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2367) - [public] list_del

```
list_del(&config->list);
config->parent = NULL;
return 0;
}

/**
 * \brief Frees a configuration node.
 * \param config Handle to the configuration node to be deleted.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This function frees a configuration node and all its resources.
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2407) - [public] while

```
while (i != &config->u.compound.fields) {
    struct list_head *nexti = i->next;
    snd_config_t *child = snd_config_iterator_entry(i);
    err = snd_config_delete(child);
    if (err < 0)
        return err;
    i = nexti;
}
break;
}
case SND_CONFIG_TYPE_STRING:
    free(config->u.string);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2418) - [public] free

```
free(config->u.string);
break;
default:
    break;
}
if (config->parent)
    list_del(&config->list);
free(config->id);
free(config);
return 0;
}

// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 2424) - [public] list_del

```
    list_del(&config->list);
    free(config->id);
    free(config);
    return 0;
}

/**
 * \brief Deletes the children of a node.
 * \param config Handle to the compound configuration node.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This function removes and frees all children of a configuration node.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2498) - [public] _snd_config_make

```
    return _snd_config_make(config, &id1, type);
}

/**
 * \brief Creates an integer configuration node.
 * \param[out] config The function puts the handle to the new node at
 *                  the address specified by \a config.
 * \param[in] id The id of the new node.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This function creates a new node of type #SND_CONFIG_TYPE_INTEGER and
 * with value \c 0.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2523) - [public] snd_config_make

```
    return snd_config_make(config, id, SND_CONFIG_TYPE_INTEGER);
}

/**
 * \brief Creates a 64-bit-integer configuration node.
 * \param[out] config The function puts the handle to the new node at
 *                  the address specified by \a config.
 * \param[in] id The id of the new node.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This function creates a new node of type #SND_CONFIG_TYPE_INTEGER64
 * and with value \c 0.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2548) - [public] snd_config_make

```
    return snd_config_make(config, id, SND_CONFIG_TYPE_INTEGER64);
}

/**
 * \brief Creates a real number configuration node.
 * \param[out] config The function puts the handle to the new node at
 *                  the address specified by \a config.
 * \param[in] id The id of the new node.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This function creates a new node of type #SND_CONFIG_TYPE_REAL and
 * with value \c 0.0.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2570) - [public] snd_config_make

```
■return snd_config_make(config, id, SND_CONFIG_TYPE_REAL);
}

/**
 * \brief Creates a string configuration node.
 * \param[out] config The function puts the handle to the new node at
 *                  the address specified by \a config.
 * \param[in] id The id of the new node.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This function creates a new node of type #SND_CONFIG_TYPE_STRING and
 * with value \c NULL.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2595) - [public] snd_config_make

```
■return snd_config_make(config, id, SND_CONFIG_TYPE_STRING);
}

/**
 * \brief Creates a pointer configuration node.
 * \param[out] config The function puts the handle to the new node at
 *                  the address specified by \a config.
 * \param[in] id The id of the new node.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This function creates a new node of type #SND_CONFIG_TYPE_POINTER and
 * with value \c NULL.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2617) - [public] snd_config_make

```
■return snd_config_make(config, id, SND_CONFIG_TYPE_POINTER);
}

/**
 * \brief Creates an empty compound configuration node.
 * \param[out] config The function puts the handle to the new node at
 *                  the address specified by \a config.
 * \param[in] id The id of the new node.
 * \param[in] join Join flag.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This function creates a new empty node of type
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2719) - [public] if

```
■if (p) {
    ■■err = _snd_config_search(root, key, p - key, &n);
    ■■if (err < 0) {
        ■■■size_t l = p - key;
        ■■■char *s = malloc(l + 1);
        ■■■if (s == NULL)
            ■■■return -ENOMEM;
        ■■■strncpy(s, key, l);
        ■■■s[l] = '\0';
        ■■■err = snd_config_make_compound(&n, s, join);
        ■■■free(s);
        ■■■if (err < 0)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2721) - [public] if

```
    if (err < 0) {
        size_t l = p - key;
        char *s = malloc(l + 1);
        if (s == NULL)
            return -ENOMEM;
        strncpy(s, key, l);
        s[l] = '\0';
        err = snd_config_make_compound(&n, s, join);
        free(s);
        if (err < 0)
            return err;
        err = snd_config_add(root, n);
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 2726) - [public] strncpy

```
    strncpy(s, key, l);
    s[l] = '\0';
    err = snd_config_make_compound(&n, s, join);
    free(s);
    if (err < 0)
        return err;
    err = snd_config_add(root, n);
    if (err < 0)
        return err;
}
root = n;
key = p + 1;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2729) - [public] free

```
    free(s);
    if (err < 0)
        return err;
    err = snd_config_add(root, n);
    if (err < 0)
        return err;
}
root = n;
key = p + 1;
} else {
    err = _snd_config_search(root, key, -1, config);
    if (err == 0) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 2740) - [public] if

```
    if (err == 0) {
        if ((*config)->type != SND_CONFIG_TYPE_COMPOUND) {
            if (override) {
                err = snd_config_delete(*config);
                if (err < 0)
                    return err;
                goto __make;
            } else {
                return -EACCES;
            }
        }
        return 0;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 2741) - [public] if

```
    if ((*config)->type != SND_CONFIG_TYPE_COMPOUND) {
        if (override) {
            err = snd_config_delete(*config);
            if (err < 0)
                return err;
            goto __make;
        } else {
            return -EACCES;
        }
    }
    return 0;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2742) - [public] if

```
    if (override) {
        err = snd_config_delete(*config);
        if (err < 0)
            return err;
        goto __make;
    } else {
        return -EACCES;
    }
    return 0;
}
__make:
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2882) - [public] if

```
    if (!tmp->u.string) {
        snd_config_delete(tmp);
        return -ENOMEM;
    }
    else {
        tmp->u.string = NULL;
    }
    *config = tmp;
    return 0;
}

/**
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2883) - [public] snd_config_delete

```
    snd_config_delete(tmp);
    return -ENOMEM;
}
else {
    tmp->u.string = NULL;
}
*config = tmp;
return 0;
}

/**
 * \brief Creates a string configuration node with the given initial value.
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2924) - [public] if

```
    if (!tmp->u.string) {
        snd_config_delete(tmp);
        return -ENOMEM;
    }

    for (c = tmp->u.string; *c; c++) {
        if (*c == ' ' || *c == '-' || *c == '_' ||
            (*c >= '0' && *c <= '9') ||
            (*c >= 'a' && *c <= 'z') ||
            (*c >= 'A' && *c <= 'Z'))
            continue;
        *c = '_';
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 2925) - [public] snd_config_delete

```
    snd_config_delete(tmp);
    return -ENOMEM;
}

for (c = tmp->u.string; *c; c++) {
    if (*c == ' ' || *c == '-' || *c == '_' ||
        (*c >= '0' && *c <= '9') ||
        (*c >= 'a' && *c <= 'z') ||
        (*c >= 'A' && *c <= 'Z'))
        continue;
    *c = '_';
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3156) - [public] free

```
    free(config->u.string);
    config->u.string = ptr;
}
break;
default:
    return -EINVAL;
}
return 0;
}

/**
 * \brief Returns the value of an integer configuration node.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3285) - [public] if

```
*ptr = config->u.integer64;

else if (config->type == SND_CONFIG_TYPE_STRING) {
    str = config->u.string;
    if (str == NULL || safe_strtoll_base(str, ptr, base) < 0)
        return -EINVAL;
    } else
        return -EINVAL;
    return 0;
}

/**
 * \brief Returns the value of a string configuration node.
 * \param[in] config Handle to the configuration node.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3381) - [public] if

```
    if (err < 0 || err == sizeof(res)) {
        assert(0);
        return -ENOMEM;
    }
    *ascii = strdup(res);
}
break;
case SND_CONFIG_TYPE_INTEGER64:
{
    char res[32];
    int err;
    err = snprintf(res, sizeof(res), "%lli", config->u.integer64);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3382) - [public] assert

```
    assert(0);
    return -ENOMEM;
}
*ascii = strdup(res);
}
break;
case SND_CONFIG_TYPE_INTEGER64:
{
    char res[32];
    int err;
    err = snprintf(res, sizeof(res), "%lli", config->u.integer64);
    if (err < 0 || err == sizeof(res)) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3393) - [public] if

```
    if (err < 0 || err == sizeof(res)) {
        assert(0);
        return -ENOMEM;
    }
    *ascii = strdup(res);
}
break;
case SND_CONFIG_TYPE_REAL:
{
    char res[32];
    int err;
    err = snprintf(res, sizeof(res), "%-16g", config->u.real);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3394) - [public] assert

```
    assert(0);
    return -ENOMEM;
}
*ascii = strdup(res);
}
break;
case SND_CONFIG_TYPE_REAL:
{
    char res[32];
    int err;
    err = snprintf(res, sizeof(res), "%-16g", config->u.real);
    if (err < 0 || err == sizeof(res)) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3405) - [public] if

```
    if (err < 0 || err == sizeof(res)) {
        assert(0);
        return -ENOMEM;
    }
    if (res[0]) { /* trim the string */
        char *ptr;
        ptr = res + strlen(res) - 1;
        while (ptr != res && *ptr == ' ')
            ptr--;
        if (*ptr != ' ')
            ptr++;
        *ptr = '\\0';
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 3406) - [public] assert

```
    assert(0);
    return -ENOMEM;
}
if (res[0]) { /* trim the string */
    char *ptr;
    ptr = res + strlen(res) - 1;
    while (ptr != res && *ptr == ' ')
        ptr--;
    if (*ptr != ' ')
        ptr++;
    *ptr = '\\0';
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3409) - [public] if

```
    if (res[0]) { /* trim the string */
        char *ptr;
        ptr = res + strlen(res) - 1;
        while (ptr != res && *ptr == ' ')
            ptr--;
        if (*ptr != ' ')
            ptr++;
        *ptr = '\\0';
    }
    *ascii = strdup(res);
}
break;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3450) - [public] strcmp

```
    return strcmp(config->id, id);
else
    return -1;
}

/**
 * \brief Dumps the contents of a configuration node or tree.
 * \param config Handle to the (root) configuration node.
 * \param out Output handle.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This function writes a textual representation of \a config's value to
 * // ... (truncated, see source file for more)
 */
```

dirtycode/alsa-lib/src/conf.c (line 3478) - [public] _snd_config_save_children

```
    return _snd_config_save_children(config, out, 0, 0, array);
} else {
    return _snd_config_save_node_value(config, out, 0);
}
```

```
/*
 * *** search macros ***
 */
```

```
#ifndef DOC_HIDDEN
```

```
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3480) - [public] _snd_config_save_node_value

```
    return _snd_config_save_node_value(config, out, 0);
}
```

```
/*
 * *** search macros ***
 */
```

```
#ifndef DOC_HIDDEN
```

```
#define SND_CONFIG_SEARCH(config, key, result, extra_code) \
{ \
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3501) - [public] if

```
    if (p) { \
        err = _snd_config_search(config, key, p - key, &n); \
        if (err < 0) \
            return err; \
        config = n; \
        key = p + 1; \
    } else \
        return _snd_config_search(config, key, -1, result); \
}
```

```
#define SND_CONFIG_SEARCHA(root, config, key, result, fcn, extra_code) \
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3508) - [public] _snd_config_search

```
    return _snd_config_search(config, key, -1, result); \
}
```

```
#define SND_CONFIG_SEARCHA(root, config, key, result, fcn, extra_code) \
{ \
    snd_config_t *n; \
    int err; \
    const char *p; \
    assert(config && key); \
    while (1) { \
        if (config->type != SND_CONFIG_TYPE_COMPOUND) { \
            // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 3519) - [public] if

```
    if (config->type != SND_CONFIG_TYPE_COMPOUND) { \
        if (snd_config_get_string(config, &p) < 0) \
            return -ENOENT; \
        err = fcn(root, root, p, &config); \
        if (err < 0) \
            return err; \
    } \
    { extra_code ; } \
    p = strchr(key, '.'); \
    if (p) { \
        err = _snd_config_search(config, key, p - key, &n); \
        if (err < 0) \
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3528) - [public] if

```
    if (p) { \
        err = _snd_config_search(config, key, p - key, &n); \
        if (err < 0) \
            return err; \
        config = n; \
        key = p + 1; \
    } else \
        return _snd_config_search(config, key, -1, result); \
    } \
}
```

```
#define SND_CONFIG_SEARCHV(config, result, fcn) \
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3535) - [public] _snd_config_search

```
    return _snd_config_search(config, key, -1, result); \
} \
}
```

```
#define SND_CONFIG_SEARCHV(config, result, fcn) \
{ \
    snd_config_t *n = NULL; \
    va_list arg; \
    assert(config); \
    va_start(arg, result); \
    while (1) { \
        const char *k = va_arg(arg, const char *); \
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3551) - [public] if

```
    if (err < 0) { \
        va_end(arg); \
        return err; \
    } \
    config = n; \
} \
va_end(arg); \
if (result) \
    *result = n; \
return 0; \
}
```

```
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3552) - [public] va_end

```
    va_end(arg); \
    return err; \
} \
config = n; \
} \
va_end(arg); \
if (result) \
    *result = n; \
return 0; \
}
```

```
#define SND_CONFIG_SEARCHVA(root, config, result, fcn) \
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3575) - [public] if

```
    if (err < 0) { \
        va_end(arg); \
        return err; \
    } \
    config = n; \
} \
va_end(arg); \
if (result) \
    *result = n; \
return 0; \
}
```

```
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3576) - [public] va_end

```
    va_end(arg); \
    return err; \
} \
config = n; \
} \
va_end(arg); \
if (result) \
    *result = n; \
return 0; \
}
```

```
#define SND_CONFIG_SEARCH_ALIAS(config, base, key, result, fcn1, fcn2) \
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3595) - [public] if

```
    if (old_key == NULL) { \
        err = -ENOMEM; \
        res = NULL; \
        break; \
    } \
    err = first && base ? -EIO : fcn1(config, config, key, &res); \
    if (err < 0) { \
        if (!base) \
            break; \
        err = fcn2(config, config, &res, base, key, NULL); \
        if (err < 0) \
            break; \
    } \
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3601) - [public] if

```
    if (err < 0) { \
        if (!base) \
            break; \
        err = fcn2(config, config, &res, base, key, NULL); \
        if (err < 0) \
            break; \
    } \
    if (snd_config_get_string(res, &key) < 0) \
        break; \
    assert(key); \
    if (!first && (strcmp(key, old_key) == 0 || maxloop <= 0)) { \
        if (maxloop == 0) \
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3610) - [public] assert

```
    assert(key); \
    if (!first && (strcmp(key, old_key) == 0 || maxloop <= 0)) { \
        if (maxloop == 0) \
            snd_error(CORE, "maximum loop count reached (circular configuration?"); \
        else \
            snd_error(CORE, "key %s refers to itself", key); \
        err = -EINVAL; \
        res = NULL; \
        break; \
    } \
    free(old_key); \
    first = 0; \
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3611) - [public] if

```
    if (!first && (strcmp(key, old_key) == 0 || maxloop <= 0)) { \
        if (maxloop == 0) \
            snd_error(CORE, "maximum loop count reached (circular configuration?"); \
        else \
            snd_error(CORE, "key %s refers to itself", key); \
        err = -EINVAL; \
        res = NULL; \
        break; \
    } \
    free(old_key); \
    first = 0; \
    maxloop--; \
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3613) - [public] snd_error

```
        snd_error(CORE, "maximum loop count reached (circular configuration?"); \
    else \
        snd_error(CORE, "key %s refers to itself", key); \
    err = -EINVAL; \
    res = NULL; \
    break; \
} \
free(old_key); \
first = 0; \
maxloop--; \
} \
free(old_key); \
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3615) - [public] snd_error

```
    snd_error(CORE, "key %s refers to itself", key); \
    err = -EINVAL; \
    res = NULL; \
    break; \
} \
free(old_key); \
first = 0; \
maxloop--; \
} \
free(old_key); \
if (!res) \
return err; \
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3620) - [public] free

```
free(old_key); \
first = 0; \
maxloop--; \
} \
free(old_key); \
if (!res) \
return err; \
if (result) \
*result = res; \
return 0; \
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 3700) - [public] snd_config_searcha

```
/**
 * \brief Searches for a node in a configuration tree, expanding aliases.
 * \param[in] root Handle to the root configuration node containing
 * \alias definitions.
 * \param[in] config Handle to the root of the configuration (sub)tree to search.
 * \param[in] key Search key: one or more node keys, separated with dots.
 * \param[out] result When \a result != \c NULL, the function puts the
 * handle to the node found at the address specified
 * by \a result.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This functions searches for a child node of \a config like
 * #snd_config_search. However, any compound node can also be
 * identified by an alias, which is a string node whose value is taken
 * as the id of a compound node below \a root.
 *
 * \a root must be a compound node.
 * \a root and \a config may be the same node.
 *
 * For example, with the following configuration, the call
 * \code
 * snd_config_searcha(root, config, "a.b.c.d", &result);
 * \endcode
 * would return the node with id \c d:
 * \code
 * config {
 *     a {
 *         b bb
 *     }
 * }
 * root {
 *     bb {
 *         c cc
 *     }
 * }
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3748) - [public] snd_config_searchv

```
/**
 * \brief Searches for a node in a configuration tree.
 * \param[in] config Handle to the root of the configuration (sub)tree to search.
 * \param[out] result When \a result != \c NULL, the function puts the
 * handle to the node found at the address specified
 * by \a result.
 * \param[in] ... One or more concatenated dot-separated search keys,
 * terminated with \c NULL.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This functions searches for a child node of \a config like
 * #snd_config_search, but the search key is the concatenation of all
 * passed search key strings. For example, the call
 * \code
 *     snd_config_searchv(cfg, &res, "a", "b.c", "d.e", NULL);
 * \endcode
 * is equivalent to the call
 * \code
 *     snd_config_search(cfg, "a.b.c.d.e", &res);
 * \endcode
 *
 * \par Errors:
 * <dl>
 * <dt>-ENOENT<dd>An id in a search key does not exist.
 * <dt>-ENOENT<dd>\a config or one of its child nodes to be searched is
 * not a compound node.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 3752) - [public] snd_config_search

```
/**
 * \brief Searches for a node in a configuration tree.
 * \param[in] config Handle to the root of the configuration (sub)tree to search.
 * \param[out] result When \a result != \c NULL, the function puts the
 * handle to the node found at the address specified
 * by \a result.
 * \param[in] ... One or more concatenated dot-separated search keys,
 * terminated with \c NULL.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This functions searches for a child node of \a config like
 * #snd_config_search, but the search key is the concatenation of all
 * passed search key strings. For example, the call
 * \code
 *     snd_config_searchv(cfg, &res, "a", "b.c", "d.e", NULL);
 * \endcode
 * is equivalent to the call
 * \code
 *     snd_config_search(cfg, "a.b.c.d.e", &res);
 * \endcode
 *
 * \par Errors:
 * <dl>
 * <dt>-ENOENT<dd>An id in a search key does not exist.
 * <dt>-ENOENT<dd>\a config or one of its child nodes to be searched is
 * not a compound node.
 * </dl>
 *
 * \par Conforming to:
 * LSB 3.2
 * // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/conf.c (line 3833) - [public] snd_config_hooks
static int snd_config_hooks(snd_config_t *config, snd_config_t *private_data);

/**
 * \brief Searches for a node in a configuration tree and expands hooks.
 * \param[in,out] config Handle to the root of the configuration
 *                  (sub)tree to search.
 * \param[in] key Search key: one or more node keys, separated with dots.
 * \param[out] result The function puts the handle to the node found at
 *                  the address specified by \a result.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This functions searches for a child node of \a config like
 * // ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/conf.c (line 4019) - [public] snd_error
■■snd_error(CORE, "Field func is missing");
■■return err;
■}
■err = snd_config_get_string(c, &str);
■if (err < 0) {
■■snd_error(CORE, "Invalid type for field func");
■■return err;
■}
■assert(str);
■err = snd_config_search_definition(root, "hook_func", str, &func_conf);
■if (err >= 0) {
■■snd_config_iterator_t i, next;
// ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/conf.c (line 4024) - [public] snd_error
■■snd_error(CORE, "Invalid type for field func");
■■return err;
■}
■assert(str);
■err = snd_config_search_definition(root, "hook_func", str, &func_conf);
■if (err >= 0) {
■■snd_config_iterator_t i, next;
■■if (snd_config_get_type(func_conf) != SND_CONFIG_TYPE_COMPOUND) {
■■■snd_error(CORE, "Invalid type for func %s definition", str);
■■■err = -EINVAL;
■■■goto _err;
■■}
// ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/conf.c (line 4031) - [public] if
■■if (snd_config_get_type(func_conf) != SND_CONFIG_TYPE_COMPOUND) {
■■■snd_error(CORE, "Invalid type for func %s definition", str);
■■■err = -EINVAL;
■■■goto _err;
■■}
■■snd_config_for_each(i, next, func_conf) {
■■■snd_config_t *n = snd_config_iterator_entry(i);
■■■const char *id = n->id;
■■■if (strcmp(id, "comment") == 0)
■■■■continue;
■■■if (strcmp(id, "lib") == 0) {
■■■■err = snd_config_get_string(n, &lib);
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/conf.c (line 4032) - [public] snd_error

```
    snd_error(CORE, "Invalid type for func %s definition", str);
    err = -EINVAL;
    goto _err;
}
snd_config_for_each(i, next, func_conf) {
    snd_config_t *n = snd_config_iterator_entry(i);
    const char *id = n->id;
    if (strcmp(id, "comment") == 0)
        continue;
    if (strcmp(id, "lib") == 0) {
        err = snd_config_get_string(n, &lib);
        if (err < 0) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 4036) - [public] snd_config_for_each

```
snd_config_for_each(i, next, func_conf) {
    snd_config_t *n = snd_config_iterator_entry(i);
    const char *id = n->id;
    if (strcmp(id, "comment") == 0)
        continue;
    if (strcmp(id, "lib") == 0) {
        err = snd_config_get_string(n, &lib);
        if (err < 0) {
            snd_error(CORE, "Invalid type for %s", id);
            goto _err;
        }
    }
    continue;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 4041) - [public] if

```
if (strcmp(id, "lib") == 0) {
    err = snd_config_get_string(n, &lib);
    if (err < 0) {
        snd_error(CORE, "Invalid type for %s", id);
        goto _err;
    }
    continue;
}
if (strcmp(id, "func") == 0) {
    err = snd_config_get_string(n, &func_name);
    if (err < 0) {
        snd_error(CORE, "Invalid type for %s", id);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 4043) - [public] if

```
if (err < 0) {
    snd_error(CORE, "Invalid type for %s", id);
    goto _err;
}
continue;
}
if (strcmp(id, "func") == 0) {
    err = snd_config_get_string(n, &func_name);
    if (err < 0) {
        snd_error(CORE, "Invalid type for %s", id);
        goto _err;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 4044) - [public] snd_error

```
#####snd_error(CORE, "Invalid type for %s", id);
#####goto _err;
#####}
#####continue;
#####}
#####if (strcmp(id, "func") == 0) {
#####err = snd_config_get_string(n, &func_name);
#####if (err < 0) {
#####snd_error(CORE, "Invalid type for %s", id);
#####goto _err;
#####}
#####continue;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4049) - [public] if

```
#####if (strcmp(id, "func") == 0) {
#####err = snd_config_get_string(n, &func_name);
#####if (err < 0) {
#####snd_error(CORE, "Invalid type for %s", id);
#####goto _err;
#####}
#####continue;
#####}
#####snd_error(CORE, "Unknown field %s", id);
#####}
#####}
#####if (!func_name) {
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4051) - [public] if

```
#####if (err < 0) {
#####snd_error(CORE, "Invalid type for %s", id);
#####goto _err;
#####}
#####continue;
#####}
#####snd_error(CORE, "Unknown field %s", id);
#####}
#####}
#####if (!func_name) {
#####int len = 16 + strlen(str) + 1;
#####buf = malloc(len);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4052) - [public] snd_error

```
#####snd_error(CORE, "Invalid type for %s", id);
#####goto _err;
#####}
#####continue;
#####}
#####snd_error(CORE, "Unknown field %s", id);
#####}
#####}
#####if (!func_name) {
#####int len = 16 + strlen(str) + 1;
#####buf = malloc(len);
#####if (! buf) {
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 4057) - [public] snd_error

```
    ■■snd_error(CORE, "Unknown field %s", id);
    ■■}
    ■}
    ■if (!func_name) {
    ■■int len = 16 + strlen(str) + 1;
    ■■buf = malloc(len);
    ■■if (! buf) {
    ■■■err = -ENOMEM;
    ■■■goto _err;
    ■■}
    ■■snprintf(buf, len, "snd_config_hook_%s", str);
    ■■buf[len-1] = '\0';
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4063) - [public] if

```
    ■■if (! buf) {
    ■■■err = -ENOMEM;
    ■■■goto _err;
    ■■}
    ■■snprintf(buf, len, "snd_config_hook_%s", str);
    ■■buf[len-1] = '\0';
    ■■func_name = buf;
    ■}
    ■h = INTERNAL(snd_dlopen)(lib, RTLD_NOW, errbuf, sizeof(errbuf));
    ■func = h ? snd_dlsym(h, func_name, SND_DLSYM_VERSION(SND_CONFIG_DLSYM_VERSION_HOOK)) : NULL;
    ■err = 0;
    ■if (!h) {
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4067) - [public] snprintf

```
    ■■snprintf(buf, len, "snd_config_hook_%s", str);
    ■■buf[len-1] = '\0';
    ■■func_name = buf;
    ■}
    ■h = INTERNAL(snd_dlopen)(lib, RTLD_NOW, errbuf, sizeof(errbuf));
    ■func = h ? snd_dlsym(h, func_name, SND_DLSYM_VERSION(SND_CONFIG_DLSYM_VERSION_HOOK)) : NULL;
    ■err = 0;
    ■if (!h) {
    ■■snd_error(CORE, "Cannot open shared library %s (%s)", lib, errbuf);
    ■■err = -ENOENT;
    ■} else if (!func) {
    ■■snd_error(CORE, "symbol %s is not defined inside %s", func_name, lib);
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4075) - [public] snd_error

```
    ■■snd_error(CORE, "Cannot open shared library %s (%s)", lib, errbuf);
    ■■err = -ENOENT;
    ■} else if (!func) {
    ■■snd_error(CORE, "symbol %s is not defined inside %s", func_name, lib);
    ■■snd_dlclose(h);
    ■■err = -ENXIO;
    ■}
    ■_err:
    ■if (func_conf)
    ■■snd_config_delete(func_conf);
    ■if (err >= 0) {
    ■■snd_config_t *nroot;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4078) - [public] snd_error

```
    ■■snd_error(CORE, "symbol %s is not defined inside %s", func_name, lib);
    ■■snd_dlclose(h);
    ■■err = -ENXIO;
    ■}
    ■_err:
    ■if (func_conf)
    ■■snd_config_delete(func_conf);
    ■if (err >= 0) {
    ■■snd_config_t *nroot;
    ■■err = func(root, config, &nroot, private_data);
    ■■if (err < 0)
    ■■■snd_error(CORE, "function %s returned error: %s", func_name, snd_strerror(err));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4079) - [public] snd_dlclose

```
    ■■snd_dlclose(h);
    ■■err = -ENXIO;
    ■}
    ■_err:
    ■if (func_conf)
    ■■snd_config_delete(func_conf);
    ■if (err >= 0) {
    ■■snd_config_t *nroot;
    ■■err = func(root, config, &nroot, private_data);
    ■■if (err < 0)
    ■■■snd_error(CORE, "function %s returned error: %s", func_name, snd_strerror(err));
    ■■snd_dlclose(h);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4084) - [public] snd_config_delete

```
    ■■snd_config_delete(func_conf);
    ■if (err >= 0) {
    ■■snd_config_t *nroot;
    ■■err = func(root, config, &nroot, private_data);
    ■■if (err < 0)
    ■■■snd_error(CORE, "function %s returned error: %s", func_name, snd_strerror(err));
    ■■snd_dlclose(h);
    ■■if (err >= 0 && nroot)
    ■■■err = snd_config_substitute(root, nroot);
    ■}
    ■free(buf);
    ■if (err < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4089) - [public] snd_error

```
    ■■■snd_error(CORE, "function %s returned error: %s", func_name, snd_strerror(err));
    ■■snd_dlclose(h);
    ■■if (err >= 0 && nroot)
    ■■■err = snd_config_substitute(root, nroot);
    ■}
    ■free(buf);
    ■if (err < 0)
    ■■return err;
    ■return 0;
}
```

```
static int snd_config_hooks(snd_config_t *config, snd_config_t *private_data)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4090) - [public] snd_dlclose

```
    ■■snd_dlclose(h);
    ■■if (err >= 0 && nroot)
    ■■■err = snd_config_substitute(root, nroot);
    ■}
    ■free(buf);
    ■if (err < 0)
    ■■return err;
    ■return 0;
    }
```

```
static int snd_config_hooks(snd_config_t *config, snd_config_t *private_data)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4112) - [public] snd_config_for_each

```
    ■■snd_config_for_each(i, next, n) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id = n->id;
    ■■■long i;
    ■■■err = safe_strtol(id, &i);
    ■■■if (err < 0) {
    ■■■■snd_error(CORE, "id of field %s is not and integer", id);
    ■■■■err = -EINVAL;
    ■■■■goto _err;
    ■■■}
    ■■■if (i == idx) {
    ■■■■err = snd_config_hooks_call(config, n, private_data);
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4117) - [public] if

```
    ■■■if (err < 0) {
    ■■■■snd_error(CORE, "id of field %s is not and integer", id);
    ■■■■err = -EINVAL;
    ■■■■goto _err;
    ■■■}
    ■■■if (i == idx) {
    ■■■■err = snd_config_hooks_call(config, n, private_data);
    ■■■■if (err < 0)
    ■■■■goto _err;
    ■■■■idx++;
    ■■■■hit = 1;
    ■■■}
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4118) - [public] snd_error

```
    ■■■■snd_error(CORE, "id of field %s is not and integer", id);
    ■■■■err = -EINVAL;
    ■■■■goto _err;
    ■■■}
    ■■■if (i == idx) {
    ■■■■err = snd_config_hooks_call(config, n, private_data);
    ■■■■if (err < 0)
    ■■■■goto _err;
    ■■■■idx++;
    ■■■■hit = 1;
    ■■■}
    ■■■}
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4122) - [public] if

```
    if (i == idx) {
        err = snd_config_hooks_call(config, n, private_data);
        if (err < 0)
            goto _err;
        idx++;
        hit = 1;
    }
} while (hit);
err = 0;
_err:
snd_config_delete(n);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4165) - [public] snd_input_close

```
snd_input_close(in);
if (err < 0)
    snd_error(CORE, "%s may be old or corrupted: consider to remove or fix it", filename);
} else
    snd_error(CORE, "cannot access file %s", filename);

return err;
}

static int config_file_load(snd_config_t *root, const char *fn, int errors)
{
    struct stat64 st;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4167) - [public] snd_error

```
snd_error(CORE, "%s may be old or corrupted: consider to remove or fix it", filename);
} else
    snd_error(CORE, "cannot access file %s", filename);

return err;
}

static int config_file_load(snd_config_t *root, const char *fn, int errors)
{
    struct stat64 st;
    struct dirent64 **namelist;
    int err, n;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4169) - [public] snd_error

```
snd_error(CORE, "cannot access file %s", filename);

return err;
}

static int config_file_load(snd_config_t *root, const char *fn, int errors)
{
    struct stat64 st;
    struct dirent64 **namelist;
    int err, n;

    if (!errors && access(fn, R_OK) < 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4183) - [public] snd_error

```
    ■■■snd_error(CORE, "cannot stat file/directory %s", fn);
    ■■■return 1;
    ■}
    ■if (!S_ISDIR(st.st_mode))
    ■■■return config_file_open(root, fn);
    #ifndef DOC_HIDDEN
    #if defined(__GNU_SOURCE) && \
        !defined(__NetBSD__) && \
        !defined(__FreeBSD__) && \
        !defined(__OpenBSD__) && \
        !defined(__DragonFly__) && \
        !defined(__sun) && \
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4187) - [public] config_file_open

```
    ■■■return config_file_open(root, fn);
    #ifndef DOC_HIDDEN
    #if defined(__GNU_SOURCE) && \
        !defined(__NetBSD__) && \
        !defined(__FreeBSD__) && \
        !defined(__OpenBSD__) && \
        !defined(__DragonFly__) && \
        !defined(__sun) && \
        !defined(__ANDROID__) && \
        !defined(__OHOS__)
    #define SORTFUNC ■versionsort64
    #else
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4207) - [public] if

```
    ■■■if (err >= 0) {
    ■■■■int sl = strlen(fn) + strlen(namelist[j]->d_name) + 2;
    ■■■■char *filename = malloc(sl);
    ■■■■snprintf(filename, sl, "%s/%s", fn, namelist[j]->d_name);
    ■■■■filename[sl-1] = '\0';

    ■■■■err = config_file_open(root, filename);
    ■■■■free(filename);
    ■■■}
    ■■■free(namelist[j]);
    ■■■}
    ■■■free(namelist);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4210) - [public] snprintf

```
    ■■■■snprintf(filename, sl, "%s/%s", fn, namelist[j]->d_name);
    ■■■■filename[sl-1] = '\0';

    ■■■■err = config_file_open(root, filename);
    ■■■■free(filename);
    ■■■}
    ■■■free(namelist[j]);
    ■■■}
    ■■■free(namelist);
    ■■■if (err < 0)
    ■■■■return err;
    ■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4214) - [public] free

```
    free(filename);
}
    free(namelist[j]);
}
    free(namelist);
    if (err < 0)
        return err;
}
return 0;
}
```

```
static int config_file_load_user(snd_config_t *root, const char *fn, int errors)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4216) - [public] free

```
    free(namelist[j]);
}
    free(namelist);
    if (err < 0)
        return err;
}
return 0;
}
```

```
static int config_file_load_user(snd_config_t *root, const char *fn, int errors)
{
    char *fn2;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4218) - [public] free

```
    free(namelist);
    if (err < 0)
        return err;
}
return 0;
}
```

```
static int config_file_load_user(snd_config_t *root, const char *fn, int errors)
{
    char *fn2;
    int err;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4232) - [public] config_file_load

```
    return config_file_load(root, fn, errors);
    err = config_file_load(root, fn2, errors);
    free(fn2);
    return err;
}
```

```
static int config_file_load_user_all(snd_config_t *_root, snd_config_t *_file, int errors)
{
    snd_config_t *file = _file, *root = _root, *n;
    char *name, *name2, *remain, *rname = NULL;
    int err;

    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/conf.c (line 4245) - [public] if
    if ((err = snd_config_search(_file, "file", &file)) < 0) {
        snd_error(CORE, "Field file not found");
        return err;
    }
    if ((err = snd_config_search(_file, "root", &root)) >= 0) {
        err = snd_config_get_ascii(root, &rname);
        if (err < 0) {
            snd_error(CORE, "Field root is bad");
            return err;
        }
        err = snd_config_make_compound(&root, rname, 0);
        if (err < 0)
            // ... (truncated, see source file for more)
    }
```

```
dirtycode/alsa-lib/src/conf.c (line 4246) - [public] snd_error
    snd_error(CORE, "Field file not found");
    return err;
}
if ((err = snd_config_search(_file, "root", &root)) >= 0) {
    err = snd_config_get_ascii(root, &rname);
    if (err < 0) {
        snd_error(CORE, "Field root is bad");
        return err;
    }
    err = snd_config_make_compound(&root, rname, 0);
    if (err < 0)
        return err;
    // ... (truncated, see source file for more)
}
```

```
dirtycode/alsa-lib/src/conf.c (line 4249) - [public] if
    if ((err = snd_config_search(_file, "root", &root)) >= 0) {
        err = snd_config_get_ascii(root, &rname);
        if (err < 0) {
            snd_error(CORE, "Field root is bad");
            return err;
        }
        err = snd_config_make_compound(&root, rname, 0);
        if (err < 0)
            return err;
    }
}
if ((err = snd_config_get_ascii(file, &name)) < 0)
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/conf.c (line 4251) - [public] if
    if (err < 0) {
        snd_error(CORE, "Field root is bad");
        return err;
    }
    err = snd_config_make_compound(&root, rname, 0);
    if (err < 0)
        return err;
}
}
if ((err = snd_config_get_ascii(file, &name)) < 0)
    goto _err;
name2 = name;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4252) - [public] snd_error

```
#####snd_error(CORE, "Field root is bad");
#####return err;
####}
####err = snd_config_make_compound(&root, rname, 0);
####if (err < 0)
#####return err;
####}
####}
####if ((err = snd_config_get_ascii(file, &name)) < 0)
#####goto _err;
####name2 = name;
####remain = strstr(name, "||");
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4265) - [public] if

```
####if (remain) {
#####*remain = '\\0';
#####remain += 3;
####}
####err = config_file_load_user(root, name2, errors);
####if (err < 0)
#####goto _err;
####if (err == 0)/* first hit wins */
#####break;
####if (!remain)
#####break;
####name2 = remain;
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4281) - [public] if

```
####if (err == 0) {
#####if (snd_config_get_type(root) == SND_CONFIG_TYPE_COMPOUND) {
#####if (snd_config_is_empty(root))
#####goto _del;
#####}
#####err = snd_config_make_path(&n, _root, rname, 0, 1);
#####if (err < 0)
#####goto _del;
#####err = snd_config_substitute(n, root);
#####if (err == 0)
#####goto _fin;
#####}
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4282) - [public] if

```
#####if (snd_config_get_type(root) == SND_CONFIG_TYPE_COMPOUND) {
#####if (snd_config_is_empty(root))
#####goto _del;
#####}
#####err = snd_config_make_path(&n, _root, rname, 0, 1);
#####if (err < 0)
#####goto _del;
#####err = snd_config_substitute(n, root);
#####if (err == 0)
#####goto _fin;
#####}
_del:
#####// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 4294) - [public] snd_config_delete

```
■ ■■ snd_config_delete(root);
■ }
_ fin:
■ free(name);
■ free(rname);
■ return err;
}

/**
 * \brief Loads and parses the given configurations files.
 * \param[in] root Handle to the root configuration node.
 * \param[in] config Handle to the configuration node for this hook.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4323) - [public] if

```
■ ■■ if (errors < 0) {
■ ■■ snd_error(CORE, "Invalid bool value in field errors");
■ ■■ return errors;
■ ■}
■ }
■ if ((err = snd_config_search(config, "files", &n)) < 0) {
■ ■ snd_error(CORE, "Unable to find field files in the pre-load section");
■ ■ return -EINVAL;
■ }
■ if ((err = snd_config_expand(n, root, NULL, private_data, &n)) < 0) {
■ ■ snd_error(CORE, "Unable to expand filenames in the pre-load section");
■ ■ return err;
■ // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4324) - [public] snd_error

```
■ ■■ snd_error(CORE, "Invalid bool value in field errors");
■ ■■ return errors;
■ ■}
■ }
■ if ((err = snd_config_search(config, "files", &n)) < 0) {
■ ■ snd_error(CORE, "Unable to find field files in the pre-load section");
■ ■ return -EINVAL;
■ }
■ if ((err = snd_config_expand(n, root, NULL, private_data, &n)) < 0) {
■ ■ snd_error(CORE, "Unable to expand filenames in the pre-load section");
■ ■ return err;
■ }
■ // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4329) - [public] snd_error

```
■ ■■ snd_error(CORE, "Unable to find field files in the pre-load section");
■ ■■ return -EINVAL;
■ }
■ if ((err = snd_config_expand(n, root, NULL, private_data, &n)) < 0) {
■ ■ snd_error(CORE, "Unable to expand filenames in the pre-load section");
■ ■ return err;
■ }
■ if (snd_config_get_type(n) != SND_CONFIG_TYPE_COMPOUND) {
■ ■ snd_error(CORE, "Invalid type for field filenames");
■ ■ goto _err;
■ }
■ do {
■ // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4333) - [public] snd_error

```
    ■■snd_error(CORE, "Unable to expand filenames in the pre-load section");
    ■■return err;
    ■}
    ■if (snd_config_get_type(n) != SND_CONFIG_TYPE_COMPOUND) {
    ■■snd_error(CORE, "Invalid type for field filenames");
    ■■goto _err;
    ■}
    ■do {
    ■■hit = 0;
    ■■snd_config_for_each(i, next, n) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id = n->id;
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4337) - [public] snd_error

```
    ■■snd_error(CORE, "Invalid type for field filenames");
    ■■goto _err;
    ■}
    ■do {
    ■■hit = 0;
    ■■snd_config_for_each(i, next, n) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id = n->id;
    ■■■long i;
    ■■■err = safe_strtol(id, &i);
    ■■■if (err < 0) {
    ■■■■snd_error(CORE, "id of field %s is not and integer", id);
    ■■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4342) - [public] snd_config_for_each

```
    ■■snd_config_for_each(i, next, n) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id = n->id;
    ■■■long i;
    ■■■err = safe_strtol(id, &i);
    ■■■if (err < 0) {
    ■■■■snd_error(CORE, "id of field %s is not and integer", id);
    ■■■■err = -EINVAL;
    ■■■■goto _err;
    ■■■}
    ■■■if (i == idx) {
    ■■■■err = config_file_load_user_all(root, n, errors);
    ■■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4347) - [public] if

```
    ■■■if (err < 0) {
    ■■■■snd_error(CORE, "id of field %s is not and integer", id);
    ■■■■err = -EINVAL;
    ■■■■goto _err;
    ■■■}
    ■■■if (i == idx) {
    ■■■■err = config_file_load_user_all(root, n, errors);
    ■■■■if (err < 0)
    ■■■■goto _err;
    ■■■■idx++;
    ■■■■hit = 1;
    ■■■}
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4348) - [public] snd_error

```
    snd_error(CORE, "id of field %s is not and integer", id);
    err = -EINVAL;
    goto _err;
}
if (i == idx) {
    err = config_file_load_user_all(root, n, errors);
    if (err < 0)
        goto _err;
    idx++;
    hit = 1;
}
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4352) - [public] if

```
    if (i == idx) {
        err = config_file_load_user_all(root, n, errors);
        if (err < 0)
            goto _err;
        idx++;
        hit = 1;
    }
} while (hit);
*dst = NULL;
err = 0;
_err:
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4372) - [public] snd_determine_driver

```
int snd_determine_driver(int card, char **driver);
#endif

static snd_config_t *_snd_config_hook_private_data(int card, const char *driver)
{
    snd_config_t *private_data, *v;
    int err;

    err = snd_config_make_compound(&private_data, NULL, 0);
    if (err < 0)
        goto _err;
    err = snd_config_imake_integer(&v, "integer", card);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4388) - [public] snd_config_delete

```
    snd_config_delete(v);
    goto _err;
}
err = snd_config_imake_string(&v, "string", driver);
if (err < 0)
    goto _err;
err = snd_config_add(private_data, v);
if (err < 0) {
    snd_config_delete(v);
    goto _err;
}
return private_data;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4396) - [public] snd_config_delete

```
■snd_config_delete(v);
■goto __err;
■}
■return private_data;
```

```
__err:
■snd_config_delete(private_data);
■return NULL;
}
```

```
static int _snd_config_hook_table(snd_config_t *root, snd_config_t *config, snd_config_t *private_data)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4415) - [public] snd_error

```
■snd_error(CORE, "Unable to expand table compound");
■return err;
■}
■if (snd_config_search(n, "id", &tn) < 0 ||
■    snd_config_get_string(tn, &id) < 0) {
■    snd_error(CORE, "Unable to find field table.id");
■    snd_config_delete(n);
■    return -EINVAL;
■}
■if (snd_config_search(n, "value", &tn) < 0 ||
■    snd_config_get_type(tn) != SND_CONFIG_TYPE_STRING) {
■    snd_error(CORE, "Unable to find field table.value");
■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4419) - [public] snd_config_get_string

```
■    snd_config_get_string(tn, &id) < 0) {
■    snd_error(CORE, "Unable to find field table.id");
■    snd_config_delete(n);
■    return -EINVAL;
■}
■if (snd_config_search(n, "value", &tn) < 0 ||
■    snd_config_get_type(tn) != SND_CONFIG_TYPE_STRING) {
■    snd_error(CORE, "Unable to find field table.value");
■    snd_config_delete(n);
■    return -EINVAL;
■}
■snd_config_remove(tn);
■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4420) - [public] snd_error

```
■snd_error(CORE, "Unable to find field table.id");
■snd_config_delete(n);
■return -EINVAL;
■}
■if (snd_config_search(n, "value", &tn) < 0 ||
■    snd_config_get_type(tn) != SND_CONFIG_TYPE_STRING) {
■    snd_error(CORE, "Unable to find field table.value");
■    snd_config_delete(n);
■    return -EINVAL;
■}
■snd_config_remove(tn);
■if ((err = snd_config_set_id(tn, id)) < 0) {
■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4421) - [public] snd_config_delete

```
    ■■snd_config_delete(n);
    ■■return -EINVAL;
    ■}
    ■if (snd_config_search(n, "value", &tn) < 0 ||
    ■    snd_config_get_type(tn) != SND_CONFIG_TYPE_STRING) {
    ■■snd_error(CORE, "Unable to find field table.value");
    ■■snd_config_delete(n);
    ■■return -EINVAL;
    ■}
    ■snd_config_remove(tn);
    ■if ((err = snd_config_set_id(tn, id)) < 0) {
    ■■snd_config_delete(tn);
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4425) - [public] snd_config_get_type

```
    ■    snd_config_get_type(tn) != SND_CONFIG_TYPE_STRING) {
    ■■snd_error(CORE, "Unable to find field table.value");
    ■■snd_config_delete(n);
    ■■return -EINVAL;
    ■}
    ■snd_config_remove(tn);
    ■if ((err = snd_config_set_id(tn, id)) < 0) {
    ■■snd_config_delete(tn);
    ■■snd_config_delete(n);
    ■■return err;
    ■}
    ■snd_config_delete(n);
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4426) - [public] snd_error

```
    ■■snd_error(CORE, "Unable to find field table.value");
    ■■snd_config_delete(n);
    ■■return -EINVAL;
    ■}
    ■snd_config_remove(tn);
    ■if ((err = snd_config_set_id(tn, id)) < 0) {
    ■■snd_config_delete(tn);
    ■■snd_config_delete(n);
    ■■return err;
    ■}
    ■snd_config_delete(n);
    ■if ((err = snd_config_add(root, tn)) < 0) {
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4427) - [public] snd_config_delete

```
    ■■snd_config_delete(n);
    ■■return -EINVAL;
    ■}
    ■snd_config_remove(tn);
    ■if ((err = snd_config_set_id(tn, id)) < 0) {
    ■■snd_config_delete(tn);
    ■■snd_config_delete(n);
    ■■return err;
    ■}
    ■snd_config_delete(n);
    ■if ((err = snd_config_add(root, tn)) < 0) {
    ■■snd_config_delete(tn);
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4432) - [public] snd_config_delete

```
    ■■snd_config_delete(tn);
    ■■snd_config_delete(n);
    ■■return err;
    ■}
    ■snd_config_delete(n);
    ■if ((err = snd_config_add(root, tn)) < 0) {
    ■■snd_config_delete(tn);
    ■■return err;
    ■}
    ■return 0;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 4433) - [public] snd_config_delete

```
    ■■snd_config_delete(n);
    ■■return err;
    ■}
    ■snd_config_delete(n);
    ■if ((err = snd_config_add(root, tn)) < 0) {
    ■■snd_config_delete(tn);
    ■■return err;
    ■}
    ■return 0;
}
```

/**

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 4438) - [public] snd_config_delete

```
    ■■snd_config_delete(tn);
    ■■return err;
    ■}
    ■return 0;
}
```

/**

* \brief Loads and parses the given configurations files for each
* installed sound card.
* \param[in] root Handle to the root configuration node.
* \param[in] config Handle to the configuration node for this hook.
* \param[out] dst The function puts the handle to the configuration
// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 4467) - [public] if

```
    ■■if (card >= 0) {
    ■■■snd_config_t *n, *private_data = NULL;
    ■■■const char *driver;
    ■■■char *fdriver = NULL;
    ■■■err = snd_determine_driver(card, &fdriver);
    ■■■if (err < 0)
    ■■■■return err;
    ■■■if (snd_config_search(root, fdriver, &n) >= 0) {
    ■■■■if (snd_config_get_string(n, &driver) < 0) {
    ■■■■■if (snd_config_get_type(n) == SND_CONFIG_TYPE_COMPOUND) {
    ■■■■■if (snd_config_get_id(n, &driver) < 0)
    ■■■■■goto __err;
    ■■■}
    ■■}
    ■■return 0;
    ■}
    ■goto __err;
    ■}
    ■return 0;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 4474) - [public] if

```
    if (snd_config_search(root, fdriver, &n) >= 0) {
        if (snd_config_get_string(n, &driver) < 0) {
            if (snd_config_get_type(n) == SND_CONFIG_TYPE_COMPOUND) {
                if (snd_config_get_id(n, &driver) < 0)
                    goto __err;
                goto __std;
            }
            goto __err;
        }
        while (1) {
            const char *s = strchr(driver, '.');
            if (s == NULL)
                // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/conf.c (line 4475) - [public] if

```
    if (snd_config_get_string(n, &driver) < 0) {
        if (snd_config_get_type(n) == SND_CONFIG_TYPE_COMPOUND) {
            if (snd_config_get_id(n, &driver) < 0)
                goto __err;
            goto __std;
        }
        goto __err;
    }
    while (1) {
        const char *s = strchr(driver, '.');
        if (s == NULL)
            break;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 4476) - [public] if

```
    if (snd_config_get_type(n) == SND_CONFIG_TYPE_COMPOUND) {
        if (snd_config_get_id(n, &driver) < 0)
            goto __err;
        goto __std;
    }
    goto __err;
    while (1) {
        const char *s = strchr(driver, '.');
        if (s == NULL)
            break;
        driver = s + 1;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 4483) - [public] while

```
    while (1) {
        const char *s = strchr(driver, '.');
        if (s == NULL)
            break;
        driver = s + 1;
    }
    if (snd_config_search(root, driver, &n) >= 0)
        goto __err;
    } else {
        driver = fdriver;
    }
    __std:
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 4496) - [public] if

```
    if (!private_data) {
        err = -ENOMEM;
        goto __err;
    }
    err = _snd_config_hook_table(root, config, private_data);
    if (err < 0)
        goto __err;
    err = snd_config_hook_load(root, config, &n, private_data);
    __err:
    if (private_data)
        snd_config_delete(private_data);
    free(fdriver);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4506) - [public] snd_config_delete

```
    snd_config_delete(private_data);
    free(fdriver);
    if (err < 0)
        return err;
}
while (card >= 0);
*dst = NULL;
return 0;
}
#ifdef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(snd_config_hook_load_for_all_cards, SND_CONFIG_DLSYM_VERSION_HOOK);
#endif
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4507) - [public] free

```
    free(fdriver);
    if (err < 0)
        return err;
}
while (card >= 0);
*dst = NULL;
return 0;
}
#ifdef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(snd_config_hook_load_for_all_cards, SND_CONFIG_DLSYM_VERSION_HOOK);
#endif

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4562) - [public] if

```
    if (!configs || !*configs) {
        const char *topdir = snd_config_topdir();
        char *s = alloca(strlen(topdir) +
            strlen("alsa.conf") + 2);
        sprintf(s, "%s/alsa.conf", topdir);
        configs = s;
    }
    for (k = 0, c = configs; (l = strchr(c, ':')) > 0; ) {
        c += l;
        k++;
        if (!*c)
            // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 4565) - [public] strlen

```
    ■■■■■ strlen("alsa.conf") + 2);
    ■■■■ sprintf(s, "%s/alsa.conf", topdir);
    ■■■■ configs = s;
    ■■■}
    ■}
    ■for (k = 0, c = configs; (l = strchr(c, ':')) > 0; ) {
    ■■c += l;
    ■■k++;
    ■■if (!*c)
    ■■■break;
    ■■c++;
    ■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4566) - [public] sprintf

```
    ■■■■ sprintf(s, "%s/alsa.conf", topdir);
    ■■■■ configs = s;
    ■■■}
    ■}
    ■for (k = 0, c = configs; (l = strchr(c, ':')) > 0; ) {
    ■■c += l;
    ■■k++;
    ■■if (!*c)
    ■■■break;
    ■■c++;
    ■}
    ■if (k == 0) {
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4587) - [public] free

```
    ■■free(local);
    ■■return -ENOMEM;
    ■}
    ■for (k = 0, c = configs; (l = strchr(c, ':')) > 0; ) {
    ■■char name[l + 1];
    ■■memcpy(name, c, l);
    ■■name[l] = 0;
    ■■err = snd_user_file(name, &local->finfo[k].name);
    ■■if (err < 0)
    ■■■goto _end;
    ■■c += l;
    ■■k++;
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4592) - [public] memcpy

```
    ■■memcpy(name, c, l);
    ■■name[l] = 0;
    ■■err = snd_user_file(name, &local->finfo[k].name);
    ■■if (err < 0)
    ■■■goto _end;
    ■■c += l;
    ■■k++;
    ■■if (!*c)
    ■■■break;
    ■■c++;
    ■}
    ■for (k = 0; k < local->count; ++k) {
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4609) - [public] if

```
    if (stat64(lf->name, &st) >= 0) {
        lf->dev = st.st_dev;
        lf->ino = st.st_ino;
        lf->mtime = st.st_mtime;
    } else {
        snd_error(CORE, "Cannot access file %s", lf->name);
        free(lf->name);
        local->count--;
        if (k < local->count) {
            memmove(&local->finfo[k], &local->finfo[k+1], sizeof(struct finfo) * (local->count - k));
            goto _nextf;
        } else {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 4614) - [public] snd_error

```
    snd_error(CORE, "Cannot access file %s", lf->name);
    free(lf->name);
    local->count--;
    if (k < local->count) {
        memmove(&local->finfo[k], &local->finfo[k+1], sizeof(struct finfo) * (local->count - k));
        goto _nextf;
    } else {
        break;
    }
}
}
if (!update)
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 4615) - [public] free

```
    free(lf->name);
    local->count--;
    if (k < local->count) {
        memmove(&local->finfo[k], &local->finfo[k+1], sizeof(struct finfo) * (local->count - k));
        goto _nextf;
    } else {
        break;
    }
}
}
if (!update)
    goto _reread;
// ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 4617) - [public] if

```
    if (k < local->count) {
        memmove(&local->finfo[k], &local->finfo[k+1], sizeof(struct finfo) * (local->count - k));
        goto _nextf;
    } else {
        break;
    }
}
}
if (!update)
    goto _reread;
if (local->count != update->count)
    goto _reread;
// ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 4618) - [public] memmove

```
    memmove(&local->finfo[k], &local->finfo[k+1], sizeof(struct finfo) * (local->count - k));
    goto _nextf;
} else {
    break;
}
}
}
if (!update)
    goto _reread;
if (local->count != update->count)
    goto _reread;
for (k = 0; k < local->count; ++k) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4642) - [public] if

```
    if (top) {
        snd_config_delete(top);
        *_top = NULL;
    }
    if (update) {
        snd_config_update_free(update);
        *_update = NULL;
    }
    if (local)
        snd_config_update_free(local);
    return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4643) - [public] snd_config_delete

```
    snd_config_delete(top);
    *_top = NULL;
}
if (update) {
    snd_config_update_free(update);
    *_update = NULL;
}
if (local)
    snd_config_update_free(local);
return err;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4646) - [public] if

```
    if (update) {
        snd_config_update_free(update);
        *_update = NULL;
    }
    if (local)
        snd_config_update_free(local);
    return err;

    _reread:
    *_top = NULL;
    *_update = NULL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4647) - [public] snd_config_update_free

```
    ■■■snd_config_update_free(update);
    ■■■*_update = NULL;
    ■■}
    ■}
    ■if (local)
    ■■snd_config_update_free(local);
    ■return err;

    _reread:
    ■*_top = NULL;
    ■*_update = NULL;
    ■if (update) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4652) - [public] snd_config_update_free

```
    ■■snd_config_update_free(local);
    ■return err;

    _reread:
    ■*_top = NULL;
    ■*_update = NULL;
    ■if (update) {
    ■■snd_config_update_free(update);
    ■■update = NULL;
    ■}
    ■if (top) {
    ■■snd_config_delete(top);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4659) - [public] snd_config_update_free

```
    ■■snd_config_update_free(update);
    ■■update = NULL;
    ■}
    ■if (top) {
    ■■snd_config_delete(top);
    ■■top = NULL;
    ■}
    ■err = snd_config_top(&top);
    ■if (err < 0)
    ■■goto _end;
    ■if (!local)
    ■■goto _skip;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4663) - [public] snd_config_delete

```
    ■■snd_config_delete(top);
    ■■top = NULL;
    ■}
    ■err = snd_config_top(&top);
    ■if (err < 0)
    ■■goto _end;
    ■if (!local)
    ■■goto _skip;
    ■for (k = 0; k < local->count; ++k) {
    ■■snd_input_t *in;
    ■■err = snd_input_stdio_open(&in, local->finfo[k].name, "r");
    ■■if (err >= 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4674) - [public] if

```
    if (err >= 0) {
        err = snd_config_load(top, in);
        snd_input_close(in);
        if (err < 0) {
            snd_error(CORE, "%s may be old or corrupted: consider to remove or fix it", local->finfo[k].name);
            goto _end;
        }
    } else {
        snd_error(CORE, "cannot access file %s", local->finfo[k].name);
    }
    _skip:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4676) - [public] snd_input_close

```
    snd_input_close(in);
    if (err < 0) {
        snd_error(CORE, "%s may be old or corrupted: consider to remove or fix it", local->finfo[k].name);
        goto _end;
    }
    _skip:
    err = snd_config_hooks(top, NULL);
    if (err < 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4677) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "%s may be old or corrupted: consider to remove or fix it", local->finfo[k].name);
        goto _end;
    }
    _skip:
    err = snd_config_hooks(top, NULL);
    if (err < 0) {
        snd_error(CORE, "hooks failed, removing configuration");
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4678) - [public] snd_error

```
    snd_error(CORE, "%s may be old or corrupted: consider to remove or fix it", local->finfo[k].name);
    goto _end;
    _skip:
    err = snd_config_hooks(top, NULL);
    if (err < 0) {
        snd_error(CORE, "hooks failed, removing configuration");
        goto _end;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4682) - [public] snd_error

```
    ■■■snd_error(CORE, "cannot access file %s", local->finfo[k].name);
    ■■}
    ■}
    _skip:
    ■err = snd_config_hooks(top, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "hooks failed, removing configuration");
    ■■goto _end;
    ■}
    ■*_top = top;
    ■*_update = local;
    ■return 1;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4688) - [public] snd_error

```
    ■■snd_error(CORE, "hooks failed, removing configuration");
    ■■goto _end;
    ■}
    ■*_top = top;
    ■*_update = local;
    ■return 1;
    }

    /**
     * \brief Updates #snd_config by rereading the global configuration files (if needed).
     * \return 0 if #snd_config was up to date, 1 if #snd_config was
     *         updated, otherwise a negative error code.
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4744) - [public] if

```
    ■■if (snd_config) {
    ■■if (top) {
    ■■■■snd_config->refcount++;
    ■■■■*top = snd_config;
    ■■■}
    ■■} else {
    ■■err = -ENODEV;
    ■■}
    ■}
    ■snd_config_unlock();
    ■return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4745) - [public] if

```
    ■■if (top) {
    ■■■■snd_config->refcount++;
    ■■■■*top = snd_config;
    ■■■}
    ■■} else {
    ■■err = -ENODEV;
    ■■}
    ■}
    ■snd_config_unlock();
    ■return err;
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4788) - [public] snd_config_delete

```
■■snd_config_delete(cfg);
■■snd_config_unlock();
}

/**
 * \brief Frees a private update structure.
 * \param[in] update The private update structure to free.
 * \return Zero if successful, otherwise a negative error code.
 */
int snd_config_update_free(snd_config_update_t *update)
{
    ■■unsigned int k;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4803) - [public] free

```
■■free(update->finfo[k].name);
■■free(update->finfo);
■■free(update);
■■return 0;
}

/**
 * \brief Frees the global configuration tree in #snd_config.
 * \return Zero if successful, otherwise a negative error code.
 *
 * This functions releases all resources of the global configuration
 * tree, and sets #snd_config to \c NULL.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4823) - [public] snd_config_delete

```
■■snd_config_delete(snd_config);
■■snd_config = NULL;
■■if (snd_config_global_update)
    ■■snd_config_update_free(snd_config_global_update);
■■snd_config_global_update = NULL;
■■snd_config_unlock();
■■/* FIXME: better to place this in another place... */
■■snd_dlobj_cache_cleanup();

■■return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4826) - [public] snd_config_update_free

```
■■snd_config_update_free(snd_config_global_update);
■■snd_config_global_update = NULL;
■■snd_config_unlock();
■■/* FIXME: better to place this in another place... */
■■snd_dlobj_cache_cleanup();

■■return 0;
}

/**
 * \brief Returns an iterator pointing to a node's first child.
 * \param[in] config Handle to a configuration node.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4905) - [public] list_entry

```
return list_entry(iterator, snd_config_t, list);
}

#ifndef DOC_HIDDEN
typedef enum _snd_config_walk_pass {
    SND_CONFIG_WALK_PASS_PRE,
    SND_CONFIG_WALK_PASS_POST,
    SND_CONFIG_WALK_PASS_LEAF,
} snd_config_walk_pass_t;
#endif

/* Return 1 if node needs to be attached to parent */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4942) - [public] snd_config_for_each

```
snd_config_for_each(i, next, src) {
    snd_config_t *s = snd_config_iterator_entry(i);
    snd_config_t *d = NULL;

    err = snd_config_walk(s, root, (dst && *dst) ? &d : NULL,
        callback, fcn, private_data);
    if (err < 0)
        goto _error;
    if (err && d) {
        err = snd_config_add(*dst, d);
        if (err < 0)
            goto _error;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 4950) - [public] if

```
if (err && d) {
    err = snd_config_add(*dst, d);
    if (err < 0)
        goto _error;
}
err = callback(src, root, dst, SND_CONFIG_WALK_PASS_POST, fcn, private_data);
if (err <= 0) {
    _error:
    if (dst && *dst)
        snd_config_delete(*dst);
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4957) - [public] if

```
if (err <= 0) {
    _error:
    if (dst && *dst)
        snd_config_delete(*dst);
}
break;
default:
    err = callback(src, root, dst, SND_CONFIG_WALK_PASS_LEAF, fcn, private_data);
    break;
}
return err;
}
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 4960) - [public] snd_config_delete

```
    snd_config_delete(*dst);
}
break;
default:
    err = callback(src, root, dst, SND_CONFIG_WALK_PASS_LEAF, fcn, private_data);
    break;
}
return err;
}

static int _snd_config_copy(snd_config_t *src,
    snd_config_t *root ATTRIBUTE_UNUSED,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4990) - [public] switch

```
switch (type) {
case SND_CONFIG_TYPE_INTEGER:
{
    long v;
    err = snd_config_get_integer(src, &v);
    assert(err >= 0);
    snd_config_set_integer(*dst, v);
    break;
}
case SND_CONFIG_TYPE_INTEGER64:
{
    long long v;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4995) - [public] assert

```
    assert(err >= 0);
    snd_config_set_integer(*dst, v);
    break;
}
case SND_CONFIG_TYPE_INTEGER64:
{
    long long v;
    err = snd_config_get_integer64(src, &v);
    assert(err >= 0);
    snd_config_set_integer64(*dst, v);
    break;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 4996) - [public] snd_config_set_integer

```
    snd_config_set_integer(*dst, v);
    break;
}
case SND_CONFIG_TYPE_INTEGER64:
{
    long long v;
    err = snd_config_get_integer64(src, &v);
    assert(err >= 0);
    snd_config_set_integer64(*dst, v);
    break;
}
case SND_CONFIG_TYPE_REAL:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5003) - [public] assert

```
    assert(err >= 0);
    snd_config_set_integer64(*dst, v);
    break;
}
case SND_CONFIG_TYPE_REAL:
{
    double v;
    err = snd_config_get_real(src, &v);
    assert(err >= 0);
    snd_config_set_real(*dst, v);
    break;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5004) - [public] snd_config_set_integer64

```
    snd_config_set_integer64(*dst, v);
    break;
}
case SND_CONFIG_TYPE_REAL:
{
    double v;
    err = snd_config_get_real(src, &v);
    assert(err >= 0);
    snd_config_set_real(*dst, v);
    break;
}
case SND_CONFIG_TYPE_STRING:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5011) - [public] assert

```
    assert(err >= 0);
    snd_config_set_real(*dst, v);
    break;
}
case SND_CONFIG_TYPE_STRING:
{
    const char *s;
    err = snd_config_get_string(src, &s);
    assert(err >= 0);
    err = snd_config_set_string(*dst, s);
    if (err < 0)
        return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5012) - [public] snd_config_set_real

```
    snd_config_set_real(*dst, v);
    break;
}
case SND_CONFIG_TYPE_STRING:
{
    const char *s;
    err = snd_config_get_string(src, &s);
    assert(err >= 0);
    err = snd_config_set_string(*dst, s);
    if (err < 0)
        return err;
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5019) - [public] assert

```
    ■■■assert(err >= 0);
    ■■■err = snd_config_set_string(dst, s);
    ■■■if (err < 0)
    ■■■return err;
    ■■■break;
    ■■}
    ■■default:
    ■■■assert(0);
    ■■}
    ■■break;
    ■■default:
    ■■break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5026) - [public] assert

```
    ■■■assert(0);
    ■■}
    ■■break;
    ■■default:
    ■■break;
    ■■}
    ■■return 1;
    }

    /**
     * \brief Creates a copy of a configuration node.
     * \param[out] dst The function puts the handle to the new configuration
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5056) - [public] snd_config_walk

```
    ■■return snd_config_walk(src, NULL, dst, _snd_config_copy, NULL, NULL);
    }

    static int _snd_config_expand_vars(snd_config_t **dst, const char *s, void *private_data)
    {
        ■■snd_config_t *val, *vars = private_data;
        ■■if (snd_config_search(vars, s, &val) < 0) {
        ■■*dst = NULL;
        ■■return 0;
        ■■}
        ■■return snd_config_copy(dst, val);
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5066) - [public] snd_config_copy

```
    ■■return snd_config_copy(dst, val);
    }

    static int _snd_config_expand(snd_config_t *src,
    ■■■    snd_config_t *root ATTRIBUTE_UNUSED,
    ■■■    snd_config_t **dst,
    ■■■    snd_config_walk_pass_t pass,
    ■■■    snd_config_expand_fcn_t fcn,
    ■■■    void *private_data)
    {
        ■■int err;
        ■■const char *id = src->id;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5090) - [public] switch

```
    switch (type) {
    case SND_CONFIG_TYPE_INTEGER:
    {
        long v;
        err = snd_config_get_integer(src, &v);
        assert(err >= 0);
        err = snd_config_imake_integer(dst, id, v);
        if (err < 0)
            return err;
        break;
    }
    case SND_CONFIG_TYPE_INTEGER64:
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5095) - [public] assert

```
        assert(err >= 0);
        err = snd_config_imake_integer(dst, id, v);
        if (err < 0)
            return err;
        break;
    }
    case SND_CONFIG_TYPE_INTEGER64:
    {
        long long v;
        err = snd_config_get_integer64(src, &v);
        assert(err >= 0);
        err = snd_config_imake_integer64(dst, id, v);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5105) - [public] assert

```
        assert(err >= 0);
        err = snd_config_imake_integer64(dst, id, v);
        if (err < 0)
            return err;
        break;
    }
    case SND_CONFIG_TYPE_REAL:
    {
        double v;
        err = snd_config_get_real(src, &v);
        assert(err >= 0);
        err = snd_config_imake_real(dst, id, v);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5115) - [public] assert

```
        assert(err >= 0);
        err = snd_config_imake_real(dst, id, v);
        if (err < 0)
            return err;
        break;
    }
    case SND_CONFIG_TYPE_STRING:
    {
        const char *s;
        snd_config_t *vars = private_data;
        err = snd_config_get_string(src, &s);
        if (err < 0)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5128) - [public] if

```
    if (s && *s == '$') {
        err = snd_config_evaluate_string(dst, s, fcn, vars);
        if (err < 0)
            return err;
        if (*dst == NULL)
            return 0;
        err = snd_config_set_id(*dst, id);
        if (err < 0) {
            snd_config_delete(*dst);
            return err;
        }
    } else {
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 5135) - [public] if

```
    if (err < 0) {
        snd_config_delete(*dst);
        return err;
    }
    } else {
        err = snd_config_imake_string(dst, id, s);
        if (err < 0)
            return err;
    }
    break;
}
default:
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 5136) - [public] snd_config_delete

```
    snd_config_delete(*dst);
    return err;
}
    } else {
        err = snd_config_imake_string(dst, id, s);
        if (err < 0)
            return err;
    }
    break;
}
default:
    assert(0);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 5147) - [public] assert

```
    assert(0);
}
    break;
default:
    break;
}
return 1;
}
```

```
static int _snd_config_evaluate(snd_config_t *src,
    snd_config_t *root,
    snd_config_t **dst ATTRIBUTE_UNUSED,
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 5176) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "Invalid type for @func");
        return err;
    }
    assert(str);
    err = snd_config_search_definition(root, "func", str, &func_conf);
    if (err >= 0) {
        snd_config_iterator_t i, next;
        if (snd_config_get_type(func_conf) != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "Invalid type for func %s definition", str);
            err = -EINVAL;
            goto _err;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5177) - [public] snd_error

```
    snd_error(CORE, "Invalid type for @func");
    return err;
}
assert(str);
err = snd_config_search_definition(root, "func", str, &func_conf);
if (err >= 0) {
    snd_config_iterator_t i, next;
    if (snd_config_get_type(func_conf) != SND_CONFIG_TYPE_COMPOUND) {
        snd_error(CORE, "Invalid type for func %s definition", str);
        err = -EINVAL;
        goto _err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5180) - [public] assert

```
    assert(str);
    err = snd_config_search_definition(root, "func", str, &func_conf);
    if (err >= 0) {
        snd_config_iterator_t i, next;
        if (snd_config_get_type(func_conf) != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "Invalid type for func %s definition", str);
            err = -EINVAL;
            goto _err;
        }
        snd_config_for_each(i, next, func_conf) {
            snd_config_t *n = snd_config_iterator_entry(i);
            const char *id = n->id;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5182) - [public] if

```
    if (err >= 0) {
        snd_config_iterator_t i, next;
        if (snd_config_get_type(func_conf) != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(CORE, "Invalid type for func %s definition", str);
            err = -EINVAL;
            goto _err;
        }
        snd_config_for_each(i, next, func_conf) {
            snd_config_t *n = snd_config_iterator_entry(i);
            const char *id = n->id;
            if (strcmp(id, "comment") == 0)
                continue;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5184) - [public] if

```
    if (snd_config_get_type(func_conf) != SND_CONFIG_TYPE_COMPOUND) {
        snd_error(CORE, "Invalid type for func %s definition", str);
        err = -EINVAL;
        goto _err;
    }
    snd_config_for_each(i, next, func_conf) {
        snd_config_t *n = snd_config_iterator_entry(i);
        const char *id = n->id;
        if (strcmp(id, "comment") == 0)
            continue;
        if (strcmp(id, "lib") == 0) {
            err = snd_config_get_string(n, &lib);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 5185) - [public] snd_error

```
    snd_error(CORE, "Invalid type for func %s definition", str);
    err = -EINVAL;
    goto _err;
}
snd_config_for_each(i, next, func_conf) {
    snd_config_t *n = snd_config_iterator_entry(i);
    const char *id = n->id;
    if (strcmp(id, "comment") == 0)
        continue;
    if (strcmp(id, "lib") == 0) {
        err = snd_config_get_string(n, &lib);
        if (err < 0) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 5189) - [public] snd_config_for_each

```
    snd_config_for_each(i, next, func_conf) {
        snd_config_t *n = snd_config_iterator_entry(i);
        const char *id = n->id;
        if (strcmp(id, "comment") == 0)
            continue;
        if (strcmp(id, "lib") == 0) {
            err = snd_config_get_string(n, &lib);
            if (err < 0) {
                snd_error(CORE, "Invalid type for %s", id);
                goto _err;
            }
        }
        continue;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 5194) - [public] if

```
    if (strcmp(id, "lib") == 0) {
        err = snd_config_get_string(n, &lib);
        if (err < 0) {
            snd_error(CORE, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    if (strcmp(id, "func") == 0) {
        err = snd_config_get_string(n, &func_name);
        if (err < 0) {
            snd_error(CORE, "Invalid type for %s", id);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 5196) - [public] if

```
        if (err < 0) {
            snd_error(CORE, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    if (strcmp(id, "func") == 0) {
        err = snd_config_get_string(n, &func_name);
        if (err < 0) {
            snd_error(CORE, "Invalid type for %s", id);
            goto _err;
        }
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 5197) - [public] snd_error

```
        snd_error(CORE, "Invalid type for %s", id);
        goto _err;
    }
    continue;
}
if (strcmp(id, "func") == 0) {
    err = snd_config_get_string(n, &func_name);
    if (err < 0) {
        snd_error(CORE, "Invalid type for %s", id);
        goto _err;
    }
    continue;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 5202) - [public] if

```
    if (strcmp(id, "func") == 0) {
        err = snd_config_get_string(n, &func_name);
        if (err < 0) {
            snd_error(CORE, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    snd_error(CORE, "Unknown field %s", id);
}
if (!func_name) {
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 5204) - [public] if

```
        if (err < 0) {
            snd_error(CORE, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    snd_error(CORE, "Unknown field %s", id);
}
if (!func_name) {
    int len = 9 + strlen(str) + 1;
    buf = malloc(len);
    // ... (truncated, see source file for more)
}
```


dirtycode/alsa-lib/src/conf.c (line 5205) - [public] snd_error

```
#####snd_error(CORE, "Invalid type for %s", id);
#####goto _err;
#####}
#####continue;
#####}
#####snd_error(CORE, "Unknown field %s", id);
#####}
#####}
#####if (!func_name) {
#####int len = 9 + strlen(str) + 1;
#####buf = malloc(len);
#####if (! buf) {
#####    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5210) - [public] snd_error

```
#####snd_error(CORE, "Unknown field %s", id);
#####}
#####}
#####if (!func_name) {
#####int len = 9 + strlen(str) + 1;
#####buf = malloc(len);
#####if (! buf) {
#####    err = -ENOMEM;
#####    goto _err;
#####}
#####snprintf(buf, len, "snd_func_%s", str);
#####buf[len-1] = '\0';
#####    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5213) - [public] if

```
#####if (!func_name) {
#####int len = 9 + strlen(str) + 1;
#####buf = malloc(len);
#####if (! buf) {
#####    err = -ENOMEM;
#####    goto _err;
#####}
#####snprintf(buf, len, "snd_func_%s", str);
#####buf[len-1] = '\0';
#####func_name = buf;
#####}
#####h = INTERNAL(snd_dlopen)(lib, RTLD_NOW, errbuf, sizeof(errbuf));
#####    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5216) - [public] if

```
#####if (! buf) {
#####    err = -ENOMEM;
#####    goto _err;
#####}
#####snprintf(buf, len, "snd_func_%s", str);
#####buf[len-1] = '\0';
#####func_name = buf;
#####}
#####h = INTERNAL(snd_dlopen)(lib, RTLD_NOW, errbuf, sizeof(errbuf));
#####if (h)
#####    func = snd_dlsym(h, func_name, SND_DLSYM_VERSION(SND_CONFIG_DLSYM_VERSION_EVALUATE));
#####err = 0;
#####    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5220) - [public] snprintf

```
    ///snprintf(buf, len, "snd_func_%s", str);
    ///buf[len-1] = '\\0';
    ///func_name = buf;
    ///}
    ///h = INTERNAL(snd_dlopen)(lib, RTLD_NOW, errbuf, sizeof(errbuf));
    ///if (h)
    ///    func = snd_dlsym(h, func_name, SND_DLSYM_VERSION(SND_CONFIG_DLSYM_VERSION_EVALUATE));
    ///err = 0;
    ///if (!h) {
    ///    snd_error(CORE, "Cannot open shared library %s (%s)", lib, errbuf);
    ///    err = -ENOENT;
    ///    goto _errbuf;
    ///    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5228) - [public] if

```
    ///if (!h) {
    ///    snd_error(CORE, "Cannot open shared library %s (%s)", lib, errbuf);
    ///    err = -ENOENT;
    ///    goto _errbuf;
    ///} else if (!func) {
    ///    snd_error(CORE, "symbol %s is not defined inside %s", func_name, lib);
    ///    snd_dlclose(h);
    ///    err = -ENXIO;
    ///    goto _errbuf;
    ///}
    _err:
    ///if (func_conf)
    ///    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5229) - [public] snd_error

```
    ///snd_error(CORE, "Cannot open shared library %s (%s)", lib, errbuf);
    ///err = -ENOENT;
    ///goto _errbuf;
    ///} else if (!func) {
    ///    snd_error(CORE, "symbol %s is not defined inside %s", func_name, lib);
    ///    snd_dlclose(h);
    ///    err = -ENXIO;
    ///    goto _errbuf;
    ///}
    _err:
    ///if (func_conf)
    ///    snd_config_delete(func_conf);
    ///    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5233) - [public] snd_error

```
    ///snd_error(CORE, "symbol %s is not defined inside %s", func_name, lib);
    ///snd_dlclose(h);
    ///err = -ENXIO;
    ///goto _errbuf;
    ///}
    _err:
    ///if (func_conf)
    ///    snd_config_delete(func_conf);
    ///if (err >= 0) {
    ///    snd_config_t *eval;
    ///    err = func(&eval, root, src, private_data);
    ///    if (err < 0)
    ///        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5234) - [public] snd_dlclose

```
    snd_dlclose(h);
    err = -ENXIO;
    goto _errbuf;
}
_err:
if (func_conf)
    snd_config_delete(func_conf);
if (err >= 0) {
    snd_config_t *eval;
    err = func(&eval, root, src, private_data);
    if (err < 0)
        snd_error(CORE, "function %s returned error: %s", func_name, snd_strerror(err));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5240) - [public] snd_config_delete

```
    snd_config_delete(func_conf);
    if (err >= 0) {
        snd_config_t *eval;
        err = func(&eval, root, src, private_data);
        if (err < 0)
            snd_error(CORE, "function %s returned error: %s", func_name, snd_strerror(err));
        snd_dlclose(h);
        if (err >= 0 && eval)
            err = snd_config_substitute(src, eval);
    }
    _errbuf:
    free(buf);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5241) - [public] if

```
    if (err >= 0) {
        snd_config_t *eval;
        err = func(&eval, root, src, private_data);
        if (err < 0)
            snd_error(CORE, "function %s returned error: %s", func_name, snd_strerror(err));
        snd_dlclose(h);
        if (err >= 0 && eval)
            err = snd_config_substitute(src, eval);
    }
    _errbuf:
    free(buf);
    if (err < 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5245) - [public] snd_error

```
        snd_error(CORE, "function %s returned error: %s", func_name, snd_strerror(err));
        snd_dlclose(h);
        if (err >= 0 && eval)
            err = snd_config_substitute(src, eval);
    }
    _errbuf:
    free(buf);
    if (err < 0)
        return err;
    return 0;
}
return 1;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5246) - [public] snd_dlclose

```
    snd_dlclose(h);
    if (err >= 0 && eval)
        err = snd_config_substitute(src, eval);
}
    _errbuf:
    free(buf);
    if (err < 0)
        return err;
    return 0;
}
return 1;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5251) - [public] free

```
    free(buf);
    if (err < 0)
        return err;
    return 0;
}
return 1;
}

/**
 * \brief Evaluates a configuration node at runtime.
 * \param[in,out] config Handle to the source configuration node.
 * \param[in] root Handle to the root of the source configuration.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5275) - [public] snd_config_walk

```
return snd_config_walk(config, root, result, _snd_config_evaluate, NULL, private_data);
}
```

```
static int load_defaults(snd_config_t *subs, snd_config_t *defs)
{
    snd_config_iterator_t d, dnext;
    snd_config_for_each(d, dnext, defs) {
        snd_config_t *def = snd_config_iterator_entry(d);
        snd_config_iterator_t f, fnext;
        if (snd_config_get_type(def) != SND_CONFIG_TYPE_COMPOUND)
            continue;
        snd_config_for_each(f, fnext, def) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 5286) - [public] snd_config_for_each

```
    snd_config_for_each(f, fnext, def) {
        snd_config_t *fld = snd_config_iterator_entry(f);
        const char *id = fld->id;
        if (strcmp(id, "type") == 0)
            continue;
        if (strcmp(id, "default") == 0) {
            snd_config_t *deflt;
            int err;
            err = snd_config_copy(&deflt, fld);
            if (err < 0)
                return err;
            err = snd_config_set_id(deflt, def->id);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/conf.c (line 5291) - [public] if

```
    if (strcmp(id, "default") == 0) {
        snd_config_t *deflt;
        int err;
        err = snd_config_copy(&deflt, fld);
        if (err < 0)
            return err;
        err = snd_config_set_id(deflt, def->id);
        if (err < 0) {
            snd_config_delete(deflt);
            return err;
        }
        err = snd_config_add(subs, deflt);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5298) - [public] if

```
        if (err < 0) {
            snd_config_delete(deflt);
            return err;
        }
        err = snd_config_add(subs, deflt);
        if (err < 0) {
            snd_config_delete(deflt);
            return err;
        }
        continue;
    }
    snd_error(CORE, "Unknown field %s", id);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5299) - [public] snd_config_delete

```
        snd_config_delete(deflt);
        return err;
    }
    err = snd_config_add(subs, deflt);
    if (err < 0) {
        snd_config_delete(deflt);
        return err;
    }
    continue;
}
snd_error(CORE, "Unknown field %s", id);
return -EINVAL;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5303) - [public] if

```
        if (err < 0) {
            snd_config_delete(deflt);
            return err;
        }
        continue;
    }
    snd_error(CORE, "Unknown field %s", id);
    return -EINVAL;
}
return 0;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5304) - [public] snd_config_delete

```
#####snd_config_delete(deflt);
#####return err;
#####}
#####continue;
#####}
#####snd_error(CORE, "Unknown field %s", id);
#####return -EINVAL;
#####}
#####}
#####return 0;
#####}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 5309) - [public] snd_error

```
#####snd_error(CORE, "Unknown field %s", id);
#####return -EINVAL;
#####}
#####}
#####return 0;
#####}
```

static void skip_blank(const char **ptr)

```
{
  while (1) {
    switch (**ptr) {
      case ' ':
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5319) - [public] switch

```
#####switch (**ptr) {
#####case ' ':
#####case '\f':
#####case '\t':
#####case '\n':
#####case '\r':
#####break;
#####default:
#####return;
#####}
#####(*ptr)++;
#####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5386) - [public] switch

```
#####switch (**ptr) {
#####case '\f':
#####case '\t':
#####case '\n':
#####case '\r':
#####case ',':
#####case '=':
#####case '\0':
#####return 0;
#####default:
#####break;
#####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5413) - [public] switch

```
    switch (c) {
    case '\0':
        snd_error(CORE, "Unterminated string");
        return -EINVAL;
    case '\\':
        c = parse_char(ptr);
        if (c < 0) {
            if (alloc > bufsize)
                free(buf);
            return c;
        }
        break;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5415) - [public] snd_error

```
        snd_error(CORE, "Unterminated string");
        return -EINVAL;
    case '\\':
        c = parse_char(ptr);
        if (c < 0) {
            if (alloc > bufsize)
                free(buf);
            return c;
        }
        break;
    default:
        (*ptr)++;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5419) - [public] if

```
        if (c < 0) {
            if (alloc > bufsize)
                free(buf);
            return c;
        }
        break;
    default:
        (*ptr)++;
        if (c == delim) {
            *val = malloc(idx + 1);
            if (!*val)
                return -ENOMEM;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5421) - [public] free

```
            free(buf);
            return c;
        }
        break;
    default:
        (*ptr)++;
        if (c == delim) {
            *val = malloc(idx + 1);
            if (!*val)
                return -ENOMEM;
            memcpy(*val, buf, idx);
            (*val)[idx] = 0;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5427) - [public] if

```
    if (c == delim) {
        *val = malloc(idx + 1);
        if (!*val)
            return -ENOMEM;
        memcpy(*val, buf, idx);
        (*val)[idx] = 0;
        if (alloc > bufsize)
            free(buf);
        return 0;
    }
}

if (idx >= alloc) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5431) - [public] memcpy

```
    memcpy(*val, buf, idx);
    (*val)[idx] = 0;
    if (alloc > bufsize)
        free(buf);
    return 0;
}

if (idx >= alloc) {
    size_t old_alloc = alloc;
    alloc *= 2;
    if (old_alloc == bufsize) {
        buf = malloc(alloc);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5434) - [public] free

```
    free(buf);
    return 0;
}

if (idx >= alloc) {
    size_t old_alloc = alloc;
    alloc *= 2;
    if (old_alloc == bufsize) {
        buf = malloc(alloc);
        if (!buf)
            return -ENOMEM;
        memcpy(buf, _buf, old_alloc);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5438) - [public] if

```
    if (idx >= alloc) {
        size_t old_alloc = alloc;
        alloc *= 2;
        if (old_alloc == bufsize) {
            buf = malloc(alloc);
            if (!buf)
                return -ENOMEM;
            memcpy(buf, _buf, old_alloc);
        } else {
            char *buf2 = realloc(buf, alloc);
            if (!buf2) {
                free(buf);
                // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 5441) - [public] if

```
    if (old_alloc == bufsize) {
        buf = malloc(alloc);
        if (!buf)
            return -ENOMEM;
        memcpy(buf, _buf, old_alloc);
    } else {
        char *buf2 = realloc(buf, alloc);
        if (!buf2) {
            free(buf);
            return -ENOMEM;
        }
        buf = buf2;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5445) - [public] memcpy

```
        memcpy(buf, _buf, old_alloc);
    } else {
        char *buf2 = realloc(buf, alloc);
        if (!buf2) {
            free(buf);
            return -ENOMEM;
        }
        buf = buf2;
    }
    buf[idx++] = c;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5448) - [public] if

```
    if (!buf2) {
        free(buf);
        return -ENOMEM;
    }
    buf = buf2;
}
buf[idx++] = c;
}
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 5449) - [public] free

```
    free(buf);
    return -ENOMEM;
}
buf = buf2;
}
buf[idx++] = c;
}
}
```

/* Parse var=val or val */

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/conf.c (line 5524) - [public] while

```
while (1) {
    switch (str[--len]) {
        case ' ':
        case '\f':
        case '\t':
        case '\n':
        case '\r':
            continue;
        default:
            break;
    }
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5525) - [public] switch

```
switch (str[--len]) {
    case ' ':
    case '\f':
    case '\t':
    case '\n':
    case '\r':
        continue;
    default:
        break;
}
break;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5543) - [public] snd_input_close

```
snd_input_close(input);
if (err < 0)
    return err;
snd_config_for_each(i, next, subs) {
    snd_config_t *n = snd_config_iterator_entry(i);
    snd_config_t *d;
    const char *id = n->id;
    err = snd_config_search(defs, id, &d);
    if (err < 0) {
        snd_error(CORE, "Unknown parameter %s", id);
        return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5546) - [public] snd_config_for_each

```
snd_config_for_each(i, next, subs) {
    snd_config_t *n = snd_config_iterator_entry(i);
    snd_config_t *d;
    const char *id = n->id;
    err = snd_config_search(defs, id, &d);
    if (err < 0) {
        snd_error(CORE, "Unknown parameter %s", id);
        return err;
    }
}
return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5551) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "Unknown parameter %s", id);
        return err;
    }
    return 0;
}

while (1) {
    char buf[256];
    const char *var = buf;
    unsigned int varlen;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5552) - [public] snd_error

```
    snd_error(CORE, "Unknown parameter %s", id);
    return err;
}
return 0;
}

while (1) {
    char buf[256];
    const char *var = buf;
    unsigned int varlen;
    snd_config_t *def, *sub, *typ;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5572) - [public] if

```
    if (varlen > 0) {
        assert(varlen < sizeof(buf));
        memcpy(buf, str, varlen);
        buf[varlen] = 0;
    } else {
        sprintf(buf, "%d", arg);
    }
    err = snd_config_search_alias(defs, NULL, var, &def);
    if (err < 0) {
        snd_error(CORE, "Unknown parameter %s", var);
        goto _err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5573) - [public] assert

```
    assert(varlen < sizeof(buf));
    memcpy(buf, str, varlen);
    buf[varlen] = 0;
} else {
    sprintf(buf, "%d", arg);
}
err = snd_config_search_alias(defs, NULL, var, &def);
if (err < 0) {
    snd_error(CORE, "Unknown parameter %s", var);
    goto _err;
}
if (snd_config_get_type(def) != SND_CONFIG_TYPE_COMPOUND) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5574) - [public] memcpy

```
    memcpy(buf, str, varlen);
    buf[varlen] = 0;
} else {
    sprintf(buf, "%d", arg);
}
err = snd_config_search_alias(defs, NULL, var, &def);
if (err < 0) {
    snd_error(CORE, "Unknown parameter %s", var);
    goto _err;
}
if (snd_config_get_type(def) != SND_CONFIG_TYPE_COMPOUND) {
    snd_error(CORE, "Parameter %s definition is not correct", var);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5577) - [public] sprintf

```
    sprintf(buf, "%d", arg);
}
err = snd_config_search_alias(defs, NULL, var, &def);
if (err < 0) {
    snd_error(CORE, "Unknown parameter %s", var);
    goto _err;
}
if (snd_config_get_type(def) != SND_CONFIG_TYPE_COMPOUND) {
    snd_error(CORE, "Parameter %s definition is not correct", var);
    err = -EINVAL;
    goto _err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5580) - [public] if

```
if (err < 0) {
    snd_error(CORE, "Unknown parameter %s", var);
    goto _err;
}
if (snd_config_get_type(def) != SND_CONFIG_TYPE_COMPOUND) {
    snd_error(CORE, "Parameter %s definition is not correct", var);
    err = -EINVAL;
    goto _err;
}
var = def->id;
err = snd_config_search(subs, var, &sub);
if (err >= 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5581) - [public] snd_error

```
    snd_error(CORE, "Unknown parameter %s", var);
    goto _err;
}
if (snd_config_get_type(def) != SND_CONFIG_TYPE_COMPOUND) {
    snd_error(CORE, "Parameter %s definition is not correct", var);
    err = -EINVAL;
    goto _err;
}
var = def->id;
err = snd_config_search(subs, var, &sub);
if (err >= 0)
    snd_config_delete(sub);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5584) - [public] if

```
    if (snd_config_get_type(def) != SND_CONFIG_TYPE_COMPOUND) {
        snd_error(CORE, "Parameter %s definition is not correct", var);
        err = -EINVAL;
        goto _err;
    }
    var = def->id;
    err = snd_config_search(subs, var, &sub);
    if (err >= 0)
        snd_config_delete(sub);
    sub = NULL;
    err = snd_config_search(def, "type", &typ);
    if (err < 0) {
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 5585) - [public] snd_error

```
    snd_error(CORE, "Parameter %s definition is not correct", var);
    err = -EINVAL;
    goto _err;
}
var = def->id;
err = snd_config_search(subs, var, &sub);
if (err >= 0)
    snd_config_delete(sub);
sub = NULL;
err = snd_config_search(def, "type", &typ);
if (err < 0) {
    _invalid_type:
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/conf.c (line 5592) - [public] snd_config_delete

```
    snd_config_delete(sub);
    sub = NULL;
    err = snd_config_search(def, "type", &typ);
    if (err < 0) {
        _invalid_type:
        snd_error(CORE, "Parameter %s definition is missing a valid type info", var);
        goto _err;
    }
    err = snd_config_get_string(typ, &tmp);
    if (err < 0 || !tmp)
        goto _invalid_type;
    if (strcmp(tmp, "integer") == 0) {
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 5595) - [public] if

```
    if (err < 0) {
        _invalid_type:
        snd_error(CORE, "Parameter %s definition is missing a valid type info", var);
        goto _err;
    }
    err = snd_config_get_string(typ, &tmp);
    if (err < 0 || !tmp)
        goto _invalid_type;
    if (strcmp(tmp, "integer") == 0) {
        long v;
        err = snd_config_make(&sub, var, SND_CONFIG_TYPE_INTEGER);
        if (err < 0)
            // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/conf.c (line 5597) - [public] snd_error

```
    snd_error(CORE, "Parameter %s definition is missing a valid type info", var);
    goto _err;
}
err = snd_config_get_string(typ, &tmp);
if (err < 0 || !tmp)
    goto _invalid_type;
if (strcmp(tmp, "integer") == 0) {
    long v;
    err = snd_config_make(&sub, var, SND_CONFIG_TYPE_INTEGER);
    if (err < 0)
        goto _err;
    err = safe_strtol(val, &v);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5603) - [public] if

```
    if (strcmp(tmp, "integer") == 0) {
        long v;
        err = snd_config_make(&sub, var, SND_CONFIG_TYPE_INTEGER);
        if (err < 0)
            goto _err;
        err = safe_strtol(val, &v);
        if (err < 0) {
            snd_error(CORE, "Parameter %s must be an integer", var);
            goto _err;
        }
        err = snd_config_set_integer(sub, v);
        if (err < 0)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5609) - [public] if

```
        if (err < 0) {
            snd_error(CORE, "Parameter %s must be an integer", var);
            goto _err;
        }
        err = snd_config_set_integer(sub, v);
        if (err < 0)
            goto _err;
    } else if (strcmp(tmp, "integer64") == 0) {
        long long v;
        err = snd_config_make(&sub, var, SND_CONFIG_TYPE_INTEGER64);
        if (err < 0)
            goto _err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5610) - [public] snd_error

```
        snd_error(CORE, "Parameter %s must be an integer", var);
        goto _err;
    }
    err = snd_config_set_integer(sub, v);
    if (err < 0)
        goto _err;
    } else if (strcmp(tmp, "integer64") == 0) {
        long long v;
        err = snd_config_make(&sub, var, SND_CONFIG_TYPE_INTEGER64);
        if (err < 0)
            goto _err;
        err = safe_strtoll(val, &v);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5622) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "Parameter %s must be an integer", var);
        goto _err;
    }
    err = snd_config_set_integer64(sub, v);
    if (err < 0)
        goto _err;
    } else if (strcmp(tmp, "real") == 0) {
        double v;
        err = snd_config_make(&sub, var, SND_CONFIG_TYPE_REAL);
        if (err < 0)
            goto _err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5623) - [public] snd_error

```
        snd_error(CORE, "Parameter %s must be an integer", var);
        goto _err;
    }
    err = snd_config_set_integer64(sub, v);
    if (err < 0)
        goto _err;
    } else if (strcmp(tmp, "real") == 0) {
        double v;
        err = snd_config_make(&sub, var, SND_CONFIG_TYPE_REAL);
        if (err < 0)
            goto _err;
        err = safe_strtod(val, &v);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5635) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "Parameter %s must be a real", var);
        goto _err;
    }
    err = snd_config_set_real(sub, v);
    if (err < 0)
        goto _err;
    } else if (strcmp(tmp, "string") == 0) {
        err = snd_config_make(&sub, var, SND_CONFIG_TYPE_STRING);
        if (err < 0)
            goto _err;
        err = snd_config_set_string(sub, val);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5636) - [public] snd_error

```
        snd_error(CORE, "Parameter %s must be a real", var);
        goto _err;
    }
    err = snd_config_set_real(sub, v);
    if (err < 0)
        goto _err;
    } else if (strcmp(tmp, "string") == 0) {
        err = snd_config_make(&sub, var, SND_CONFIG_TYPE_STRING);
        if (err < 0)
            goto _err;
        err = snd_config_set_string(sub, val);
        if (err < 0)
            goto _err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5657) - [public] if

```
    if (err < 0) {
        _err:
        if (sub)
            snd_config_delete(sub);
        free(val);
        return err;
    }
    free(val);
    if (!*new)
        break;
    if (*new != ',')
        return -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5660) - [public] snd_config_delete

```
    snd_config_delete(sub);
    free(val);
    return err;
}
free(val);
if (!*new)
    break;
if (*new != ',')
    return -EINVAL;
str = new + 1;
arg++;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5661) - [public] free

```
    free(val);
    return err;
}
free(val);
if (!*new)
    break;
if (*new != ',')
    return -EINVAL;
str = new + 1;
arg++;
}
return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5664) - [public] free

```
    free(val);
    if (!*new)
        break;
    if (*new != ',')
        return -EINVAL;
    str = new + 1;
    arg++;
}
return 0;
}

/**
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/conf.c (line 5703) - [public] snd_error

```
    ■■snd_error(CORE, "Expand error (walk): %s", snd_strerror(err));
    ■■return err;
    ■}
    ■*result = res;
    ■return 1;
    }

    /**
     * \brief Expands a configuration node, applying arguments and functions.
     * \param[in] config Handle to the configuration node.
     * \param[in] root Handle to the root configuration node.
     * \param[in] args Arguments string, can be \c NULL.
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5734) - [public] if

```
    ■■if (args != NULL) {
        ■■■snd_error(CORE, "Unknown parameters %s", args);
        ■■■return -EINVAL;
        ■■}
    ■■err = snd_config_copy(&res, config);
    ■■if (err < 0)
        ■■■return err;
    ■} else {
        ■■err = snd_config_top(&subs);
        ■■if (err < 0)
            ■■■return err;
        ■■err = load_defaults(subs, defs);
        ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5735) - [public] snd_error

```
    ■■■snd_error(CORE, "Unknown parameters %s", args);
    ■■■return -EINVAL;
    ■■}
    ■■err = snd_config_copy(&res, config);
    ■■if (err < 0)
        ■■■return err;
    ■} else {
        ■■err = snd_config_top(&subs);
        ■■if (err < 0)
            ■■■return err;
        ■■err = load_defaults(subs, defs);
        ■■if (err < 0) {
            ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5746) - [public] if

```
    ■■if (err < 0) {
        ■■■snd_error(CORE, "Load defaults error: %s", snd_strerror(err));
        ■■■goto _end;
        ■■}
    ■■err = parse_args(subs, args, defs);
    ■■if (err < 0) {
        ■■■snd_error(CORE, "Parse arguments error: %s", snd_strerror(err));
        ■■■goto _end;
        ■■}
    ■■err = snd_config_evaluate(subs, root, private_data, NULL);
    ■■if (err < 0) {
        ■■■snd_error(CORE, "Args evaluate error: %s", snd_strerror(err));
        ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5747) - [public] snd_error

```
    ■■■snd_error(CORE, "Load defaults error: %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■■err = parse_args(subs, args, defs);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Parse arguments error: %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■■err = snd_config_evaluate(subs, root, private_data, NULL);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Args evaluate error: %s", snd_strerror(err));
    ■■■goto _end;
    ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5751) - [public] if

```
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Parse arguments error: %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■■err = snd_config_evaluate(subs, root, private_data, NULL);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Args evaluate error: %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■■err = snd_config_walk(config, root, &res, _snd_config_expand, _snd_config_expand_vars, subs);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Expand error (walk): %s", snd_strerror(err));
    ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5752) - [public] snd_error

```
    ■■■snd_error(CORE, "Parse arguments error: %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■■err = snd_config_evaluate(subs, root, private_data, NULL);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Args evaluate error: %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■■err = snd_config_walk(config, root, &res, _snd_config_expand, _snd_config_expand_vars, subs);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Expand error (walk): %s", snd_strerror(err));
    ■■■goto _end;
    ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5756) - [public] if

```
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Args evaluate error: %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■■err = snd_config_walk(config, root, &res, _snd_config_expand, _snd_config_expand_vars, subs);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Expand error (walk): %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■■err = snd_config_evaluate(res, root, private_data, NULL);
    ■■if (err < 0) {
    ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5757) - [public] snd_error

```
    ■■■snd_error(CORE, "Args evaluate error: %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■■err = snd_config_walk(config, root, &res, _snd_config_expand, _snd_config_expand_vars, subs);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Expand error (walk): %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■}
    ■err = snd_config_evaluate(res, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "Evaluate error: %s", snd_strerror(err));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5761) - [public] if

```
    ■■if (err < 0) {
    ■■■snd_error(CORE, "Expand error (walk): %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■}
    ■err = snd_config_evaluate(res, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "Evaluate error: %s", snd_strerror(err));
    ■■snd_config_delete(res);
    ■■goto _end;
    ■}
    ■*result = res;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5762) - [public] snd_error

```
    ■■■snd_error(CORE, "Expand error (walk): %s", snd_strerror(err));
    ■■■goto _end;
    ■■}
    ■}
    ■err = snd_config_evaluate(res, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "Evaluate error: %s", snd_strerror(err));
    ■■snd_config_delete(res);
    ■■goto _end;
    ■}
    ■*result = res;
    ■err = 1;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5768) - [public] snd_error

```
    ■■snd_error(CORE, "Evaluate error: %s", snd_strerror(err));
    ■■snd_config_delete(res);
    ■■goto _end;
    ■}
    ■*result = res;
    ■err = 1;
    _end:
    ■if (subs)
    ■■snd_config_delete(subs);
    ■return err;
}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5769) - [public] snd_config_delete

```
    ■■snd_config_delete(res);
    ■■goto _end;
    ■}
    ■*result = res;
    ■err = 1;
    _end:
    ■if (subs)
    ■■snd_config_delete(subs);
    ■return err;
    }

    /**
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5776) - [public] snd_config_delete

```
    ■■snd_config_delete(subs);
    ■return err;
    }

    /**
     * \brief Searches for a definition in a configuration tree, using
     *        aliases and expanding hooks and arguments.
     * \param[in] config Handle to the configuration (sub)tree to search.
     * \param[in] base Implicit key base, or \c NULL for none.
     * \param[in] name Key suffix, optionally with arguments.
     * \param[out] result The function puts the handle to the expanded found
     *                   node at the address specified by \a result.
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5820) - [public] memcpy

```
    ■■memcpy(key, name, args - name - 1);
    ■■key[args - name - 1] = '\0';
    ■} else {
    ■■key = (char *) name;
    ■}
    ■/*
    ■ * if key contains dot (.), the implicit base is ignored
    ■ * and the key starts from root given by the 'config' parameter
    ■ */
    ■snd_config_lock();
    ■err = snd_config_search_alias_hooks(config, strchr(key, '.') ? NULL : base, key, &conf);
    ■if (err < 0) {
    ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5832) - [public] snd_config_unlock

```
    ■■snd_config_unlock();
    ■■return err;
    ■}
    ■err = snd_config_expand(conf, config, args, NULL, result);
    ■snd_config_unlock();
    ■return err;
    }
```

```
#ifndef DOC_HIDDEN
void snd_config_set_hop(snd_config_t *conf, int hop)
{
    ■conf->hop = hop;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5849) - [public] if

```
    if (conf->hop >= SND_CONF_MAX_HOPS) {
        snd_error(CORE, "Too many definition levels (looped?)\n");
        return -EINVAL;
    }
    return conf->hop;
}
return 0;
}
#endif

#if 0
/* Not strictly needed, but useful to check for memory leaks */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5850) - [public] snd_error

```
    snd_error(CORE, "Too many definition levels (looped?)\n");
    return -EINVAL;
}
return conf->hop;
}
return 0;
}
#endif

#if 0
/* Not strictly needed, but useful to check for memory leaks */
void _snd_config_end(void) __attribute__((destructor));
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5861) - [public] _snd_config_end

```
#if 0
/* Not strictly needed, but useful to check for memory leaks */

void _snd_config_end(void) __attribute__((destructor));

static void _snd_config_end(void)
{
    int k;
    if (snd_config)
        snd_config_delete(snd_config);
    snd_config = 0;
    for (k = 0; k < files_info_count; ++k)
        free(files_info[k].name);
    free(files_info);
    files_info = NULL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5867) - [public] snd_config_delete

```
    snd_config_delete(snd_config);
    snd_config = 0;
    for (k = 0; k < files_info_count; ++k)
        free(files_info[k].name);
    free(files_info);
    files_info = NULL;
    files_info_count = 0;
}
#endif

#ifndef DOC_HIDDEN
size_t page_size(void)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/conf.c (line 5870) - [public] free

```
    free(files_info[k].name);
    free(files_info);
    files_info = NULL;
    files_info_count = 0;
}
#endif

#ifdef DOC_HIDDEN
size_t page_size(void)
{
    long s = sysconf(_SC_PAGE_SIZE);
    assert(s > 0);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/confeval.c (line 49) - [public] if

```
    if (*s == begin) {
        count++;
    } else if (*s == end) {
        count--;
        if (count == 0)
            return s + 1;
    }
    s++;
}
return NULL;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/confeval.c (line 116) - [public] if

```
    if (pos == END) {
        snd_error(CORE, "unexpected expression tail '%s'", s);
        return -EINVAL;
    }
    if (pos == OP) {
        switch (c) {
            case '+':
            case '-':
            case '*':
            case '/':
            case '%':
            case '|':
                // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/confeval.c (line 117) - [public] snd_error

```
    snd_error(CORE, "unexpected expression tail '%s'", s);
    return -EINVAL;
}
    if (pos == OP) {
        switch (c) {
            case '+':
            case '-':
            case '*':
            case '/':
            case '%':
            case '|':
            case '&': op = c; break;
                // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/confeval.c (line 120) - [public] if

```
    if (pos == OP) {
        switch (c) {
            case '+':
            case '-':
            case '*':
            case '/':
            case '%':
            case '|':
            case '&': op = c; break;
            default:
                snd_error(CORE, "unknown operation '%c'", c);
                return -EINVAL;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 121) - [public] switch

```
        switch (c) {
            case '+':
            case '-':
            case '*':
            case '/':
            case '%':
            case '|':
            case '&': op = c; break;
            default:
                snd_error(CORE, "unknown operation '%c'", c);
                return -EINVAL;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 130) - [public] snd_error

```
        snd_error(CORE, "unknown operation '%c'", c);
        return -EINVAL;
    }
    pos = RIGHT;
    s++;
    continue;
}
if (c == '(') {
    e = _find_end_of_expression(s + 1, '(', ')');
    off = 1;
    goto _expr;
} else if (c == '$') {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 137) - [public] if

```
    if (c == '(') {
        e = _find_end_of_expression(s + 1, '(', ')');
        off = 1;
        goto _expr;
    } else if (c == '$') {
        if (s[1] == '[') {
            e = _find_end_of_expression(s + 2, '[', ']');
            off = 2;
        }
        _expr:
        if (e == NULL)
            return -EINVAL;
        m = malloc(e - s - (off - 1));
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 142) - [public] if

```
    if (s[1] == '[') {
        e = _find_end_of_expression(s + 2, '[', ']');
        off = 2;
        _expr:
        if (e == NULL)
            return -EINVAL;
        m = malloc(e - s - (off - 1));
        if (m == NULL)
            return -ENOMEM;
        memcpy(m, s + off, e - s - off);
        m[e - s - (off + 1)] = '\0';
        err = _snd_eval_string(&tmp, m, fcn, private_data);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 151) - [public] memcpy

```
        memcpy(m, s + off, e - s - off);
        m[e - s - (off + 1)] = '\0';
        err = _snd_eval_string(&tmp, m, fcn, private_data);
        free(m);
        if (err < 0)
            return err;
        s = e;
        if (*s)
            s++;
    } else {
        e = s + 1;
        while (*e) {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 154) - [public] free

```
        free(m);
        if (err < 0)
            return err;
        s = e;
        if (*s)
            s++;
    } else {
        e = s + 1;
        while (*e) {
            if (!isalnum(*e) && *e != '_')
                break;
            e++;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 162) - [public] while

```
        while (*e) {
            if (!isalnum(*e) && *e != '_')
                break;
            e++;
        }
        m = malloc(e - s);
        if (m == NULL)
            return -ENOMEM;
        memcpy(m, s + 1, e - s - 1);
        m[e - s - 1] = '\0';
        err = fcn(&tmp, m, private_data);
        free(m);
        // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/confeval.c (line 170) - [public] memcpy

```

    memcpy(m, s + 1, e - s - 1);
    m[e - s - 1] = '\\0';
    err = fcn(&tmp, m, private_data);
    free(m);
    if (err < 0)
        return err;
    if (tmp == NULL) {
        err = snd_config_imake_integer(&tmp, NULL, 0);
        if (err < 0)
            return err;
    }
    s = e;
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/confeval.c (line 173) - [public] free

```

    free(m);
    if (err < 0)
        return err;
    if (tmp == NULL) {
        err = snd_config_imake_integer(&tmp, NULL, 0);
        if (err < 0)
            return err;
    }
    s = e;
}
err = _to_integer(op == LEFT ? &left : &right, tmp);
snd_config_delete(tmp);
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/confeval.c (line 176) - [public] if

```

    if (tmp == NULL) {
        err = snd_config_imake_integer(&tmp, NULL, 0);
        if (err < 0)
            return err;
    }
    s = e;
}
err = _to_integer(op == LEFT ? &left : &right, tmp);
snd_config_delete(tmp);
} else if (c == '-' || (c >= '0' && c <= '9')) {
    err = _parse_integer(op == LEFT ? &left : &right, &s);
} else {
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/confeval.c (line 184) - [public] snd_config_delete

```

    snd_config_delete(tmp);
} else if (c == '-' || (c >= '0' && c <= '9')) {
    err = _parse_integer(op == LEFT ? &left : &right, &s);
} else {
    return -EINVAL;
}
if (err < 0)
    return err;
pos = op == LEFT ? OP : END;
}
if (pos != OP && pos != END) {
    snd_error(CORE, "incomplete expression '%s'", save);
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/confeval.c (line 195) - [public] snd_error

```
    ■■snd_error(CORE, "incomplete expression '%s'", save);
    ■■return -EINVAL;
    ■}

    ■if (pos == END) {
    ■■switch (op) {
    ■■case '+': left = left + right; break;
    ■■case '-': left = left - right; break;
    ■■case '*': left = left * right; break;
    ■■case '/': left = left / right; break;
    ■■case '%': left = left % right; break;
    ■■case '|': left = left | right; break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 200) - [public] switch

```
    ■■switch (op) {
    ■■case '+': left = left + right; break;
    ■■case '-': left = left - right; break;
    ■■case '*': left = left * right; break;
    ■■case '/': left = left / right; break;
    ■■case '%': left = left % right; break;
    ■■case '|': left = left | right; break;
    ■■case '&': left = left & right; break;
    ■■default: return -EINVAL;
    ■■}
    ■}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/confeval.c (line 213) - [public] snd_config_imake_integer64

```
    ■■return snd_config_imake_integer64(dst, NULL, left);
    ■else
    ■■return snd_config_imake_integer(dst, NULL, left);
    }
    #endif /* DOC_HIDDEN */

    /**
     * \brief Evaluate an math expression in the string
     * \param[out] dst The function puts the handle to the new configuration
     *                node at the address specified by \a dst.
     * \param[in] s A string to evaluate
     * \param[in] fcn A function to get the variable contents
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 215) - [public] snd_config_imake_integer

```
    ■■return snd_config_imake_integer(dst, NULL, left);
    }
    #endif /* DOC_HIDDEN */

    /**
     * \brief Evaluate an math expression in the string
     * \param[out] dst The function puts the handle to the new configuration
     *                node at the address specified by \a dst.
     * \param[in] s A string to evaluate
     * \param[in] fcn A function to get the variable contents
     * \param[in] private_data A private value for the variable contents function
     * \return 0 if successful, otherwise a negative error code.
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confeval.c (line 239) - [public] snd_error

```
    ■■■snd_error(CORE, "wrong expression '%s'", s);
    ■} else {
    ■■err = fcn(dst, s + 1, private_data);
    ■}
    ■return err;
    }
```

dirtycode/alsa-lib/src/confmisc.c (line 128) - [public] if

```
    if (v < 0 || v > 1) {
        _invalid_value:
        snd_error(CORE, "Invalid value for %s", id);
        return -EINVAL;
    }
    return v;
}
err = snd_config_get_string(conf, &str);
if (err < 0) {
    snd_error(CORE, "Invalid type for %s", id);
    return -EINVAL;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 130) - [public] snd_error

```
    snd_error(CORE, "Invalid value for %s", id);
    return -EINVAL;
}
return v;
}
err = snd_config_get_string(conf, &str);
if (err < 0) {
    snd_error(CORE, "Invalid type for %s", id);
    return -EINVAL;
}
err = snd_config_get_bool_ascii(str);
if (err < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 137) - [public] snd_error

```
    snd_error(CORE, "Invalid type for %s", id);
    return -EINVAL;
}
err = snd_config_get_bool_ascii(str);
if (err < 0)
    goto _invalid_value;
return err;
}

/**
 * \brief Gets the card number from a configuration node.
 * \param conf Handle to the configuration node to be parsed.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 158) - [public] if

```
    if (snd_config_get_string(conf, &str)) {
        if (snd_config_get_id(conf, &id) >= 0)
            snd_error(CORE, "Invalid field %s", id);
            return -EINVAL;
    }
    err = snd_card_get_index(str);
    if (err < 0) {
        snd_error(CORE, "Cannot get card index for %s", str);
        return err;
    }
    v = err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 160) - [public] snd_error

```
    snd_error(CORE, "Invalid field %s", id);
    return -EINVAL;
}
err = snd_card_get_index(str);
if (err < 0) {
    snd_error(CORE, "Cannot get card index for %s", str);
    return err;
}
v = err;
}
if (v < 0 || v > INT_MAX)
    return -EINVAL;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 164) - [public] if

```
if (err < 0) {
    snd_error(CORE, "Cannot get card index for %s", str);
    return err;
}
v = err;
}
if (v < 0 || v > INT_MAX)
    return -EINVAL;
return v;
}

/**
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 165) - [public] snd_error

```
    snd_error(CORE, "Cannot get card index for %s", str);
    return err;
}
v = err;
}
if (v < 0 || v > INT_MAX)
    return -EINVAL;
return v;
}

/**
 * \brief Gets the control interface index from the given ASCII string.
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 185) - [public] if

```
if (safe_strtol(ascii, &v) >= 0) {
    if (v < 0 || v > SND_CTL_ELEM_IFACE_LAST)
        return -EINVAL;
    return v;
}
for (idx = 0; idx <= SND_CTL_ELEM_IFACE_LAST; idx++) {
    if (strcasecmp(snd_ctl_elem_iface_name(idx), ascii) == 0)
        return idx;
}
return -EINVAL;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 214) - [public] if

```
    if (v < 0 || v > SND_CTL_ELEM_IFACE_LAST) {
        _invalid_value:
        snd_error(CORE, "Invalid value for %s", id);
        return -EINVAL;
    }
    return v;
}
err = snd_config_get_string(conf, &str);
if (err < 0) {
    snd_error(CORE, "Invalid type for %s", id);
    return -EINVAL;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 216) - [public] snd_error

```
    snd_error(CORE, "Invalid value for %s", id);
    return -EINVAL;
}
return v;
}
err = snd_config_get_string(conf, &str);
if (err < 0) {
    snd_error(CORE, "Invalid type for %s", id);
    return -EINVAL;
}
err = snd_config_get_ctl_iface_ascii(str);
if (err < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 223) - [public] snd_error

```
    snd_error(CORE, "Invalid type for %s", id);
    return -EINVAL;
}
err = snd_config_get_ctl_iface_ascii(str);
if (err < 0)
    goto _invalid_value;
return err;
}

/*
 * Helper functions for the configuration file
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 266) - [public] snd_error

```
    snd_error(CORE, "field vars not found");
    goto __error;
}
err = snd_config_evaluate(n, root, private_data, NULL);
if (err < 0) {
    snd_error(CORE, "error evaluating vars");
    goto __error;
}
err = snd_config_search(src, "default", &d);
if (err < 0) {
    snd_error(CORE, "field default not found");
    goto __error;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 271) - [public] snd_error

```
    ■■snd_error(CORE, "error evaluating vars");
    ■■goto __error;
    ■}
    ■err = snd_config_search(src, "default", &d);
    ■if (err < 0) {
    ■■snd_error(CORE, "field default not found");
    ■■goto __error;
    ■}
    ■err = snd_config_evaluate(d, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "error evaluating default");
    ■■goto __error;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 276) - [public] snd_error

```
    ■■snd_error(CORE, "field default not found");
    ■■goto __error;
    ■}
    ■err = snd_config_evaluate(d, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "error evaluating default");
    ■■goto __error;
    ■}
    ■err = snd_config_get_ascii(d, &def);
    ■if (err < 0) {
    ■■snd_error(CORE, "error getting field default");
    ■■goto __error;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 281) - [public] snd_error

```
    ■■snd_error(CORE, "error evaluating default");
    ■■goto __error;
    ■}
    ■err = snd_config_get_ascii(d, &def);
    ■if (err < 0) {
    ■■snd_error(CORE, "error getting field default");
    ■■goto __error;
    ■}
    ■do {
    ■■hit = 0;
    ■■snd_config_for_each(i, next, n) {
    ■■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 286) - [public] snd_error

```
    ■■snd_error(CORE, "error getting field default");
    ■■goto __error;
    ■}
    ■do {
    ■■hit = 0;
    ■■snd_config_for_each(i, next, n) {
    ■■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■■const char *ptr;
    ■■■■long i;
    ■■■■if (snd_config_get_id(n, &id) < 0)
    ■■■■continue;
    ■■■■if (snd_config_get_type(n) != SND_CONFIG_TYPE_STRING) {
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 291) - [public] snd_config_for_each

```
    ■■■snd_config_for_each(i, next, n) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *ptr;
    ■■■long i;
    ■■■if (snd_config_get_id(n, &id) < 0)
    ■■■continue;
    ■■■if (snd_config_get_type(n) != SND_CONFIG_TYPE_STRING) {
    ■■■snd_error(CORE, "field %s is not a string", id);
    ■■■err = -EINVAL;
    ■■■goto __error;
    ■■■}
    ■■■err = safe_strtol(id, &i);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 297) - [public] if

```
    ■■■if (snd_config_get_type(n) != SND_CONFIG_TYPE_STRING) {
    ■■■snd_error(CORE, "field %s is not a string", id);
    ■■■err = -EINVAL;
    ■■■goto __error;
    ■■■}
    ■■■err = safe_strtol(id, &i);
    ■■■if (err < 0) {
    ■■■snd_error(CORE, "id of field %s is not an integer", id);
    ■■■err = -EINVAL;
    ■■■goto __error;
    ■■■}
    ■■■if (i == idx) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 298) - [public] snd_error

```
    ■■■snd_error(CORE, "field %s is not a string", id);
    ■■■err = -EINVAL;
    ■■■goto __error;
    ■■■}
    ■■■err = safe_strtol(id, &i);
    ■■■if (err < 0) {
    ■■■snd_error(CORE, "id of field %s is not an integer", id);
    ■■■err = -EINVAL;
    ■■■goto __error;
    ■■■}
    ■■■if (i == idx) {
    ■■■idx++;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 303) - [public] if

```
    ■■■if (err < 0) {
    ■■■snd_error(CORE, "id of field %s is not an integer", id);
    ■■■err = -EINVAL;
    ■■■goto __error;
    ■■■}
    ■■■if (i == idx) {
    ■■■idx++;
    ■■■err = snd_config_get_string(n, &ptr);
    ■■■if (err < 0) {
    ■■■snd_error(CORE, "invalid string for id %s", id);
    ■■■err = -EINVAL;
    ■■■goto __error;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 304) - [public] snd_error

```
#####snd_error(CORE, "id of field %s is not an integer", id);
#####err = -EINVAL;
#####goto __error;
#####}
#####if (i == idx) {
#####idx++;
#####err = snd_config_get_string(n, &ptr);
#####if (err < 0) {
#####snd_error(CORE, "invalid string for id %s", id);
#####err = -EINVAL;
#####goto __error;
#####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 308) - [public] if

```
#####if (i == idx) {
#####idx++;
#####err = snd_config_get_string(n, &ptr);
#####if (err < 0) {
#####snd_error(CORE, "invalid string for id %s", id);
#####err = -EINVAL;
#####goto __error;
#####}
#####res = getenv(ptr);
#####if (res != NULL && *res != '\0')
#####goto __ok;
#####hit = 1;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 311) - [public] if

```
#####if (err < 0) {
#####snd_error(CORE, "invalid string for id %s", id);
#####err = -EINVAL;
#####goto __error;
#####}
#####res = getenv(ptr);
#####if (res != NULL && *res != '\0')
#####goto __ok;
#####hit = 1;
#####}
#####} while (hit);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 312) - [public] snd_error

```
#####snd_error(CORE, "invalid string for id %s", id);
#####err = -EINVAL;
#####goto __error;
#####}
#####res = getenv(ptr);
#####if (res != NULL && *res != '\0')
#####goto __ok;
#####hit = 1;
#####}
#####} while (hit);
#####res = def;
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/confmisc.c (line 368) - [public] snd_config_delete

```
    ■■snd_config_delete(d);
    ■■return err;
    ■}
    ■err = safe_strtol(str, &v);
    ■if (err < 0) {
    ■■snd_config_delete(d);
    ■■return err;
    ■}
    ■snd_config_delete(d);
    ■err = snd_config_get_id(src, &id);
    ■if (err < 0)
    ■■return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 373) - [public] snd_config_delete

```
    ■■snd_config_delete(d);
    ■■return err;
    ■}
    ■snd_config_delete(d);
    ■err = snd_config_get_id(src, &id);
    ■if (err < 0)
    ■■return err;
    ■err = snd_config_imake_integer(dst, id, v);
    ■if (err < 0)
    ■■return err;
    ■return 0;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 417) - [public] snd_error

```
    ■■snd_error(CORE, "field strings not found");
    ■■goto __error;
    ■}
    ■err = snd_config_evaluate(n, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "error evaluating strings");
    ■■goto __error;
    ■}
    ■do {
    ■■hit = 0;
    ■■snd_config_for_each(i, next, n) {
    ■■■■snd_config_t *n = snd_config_iterator_entry(i);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 422) - [public] snd_error

```
    ■■snd_error(CORE, "error evaluating strings");
    ■■goto __error;
    ■}
    ■do {
    ■■hit = 0;
    ■■snd_config_for_each(i, next, n) {
    ■■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■■char *ptr;
    ■■■■const char *id;
    ■■■■long i;
    ■■■■if (snd_config_get_id(n, &id) < 0)
    ■■■■continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 427) - [public] snd_config_for_each

```
■■■snd_config_for_each(i, next, n) {
■■■snd_config_t *n = snd_config_iterator_entry(i);
■■■char *ptr;
■■■const char *id;
■■■long i;
■■■if (snd_config_get_id(n, &id) < 0)
■■■continue;
■■■err = safe_strtol(id, &i);
■■■if (err < 0) {
■■■snd_error(CORE, "id of field %s is not an integer", id);
■■■err = -EINVAL;
■■■goto __error;
■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 435) - [public] if

```
■■■if (err < 0) {
■■■snd_error(CORE, "id of field %s is not an integer", id);
■■■err = -EINVAL;
■■■goto __error;
■■■}
■■■if (i == idx) {
■■■idx++;
■■■err = snd_config_get_ascii(n, &ptr);
■■■if (err < 0) {
■■■snd_error(CORE, "invalid ascii string for id %s", id);
■■■err = -EINVAL;
■■■goto __error;
■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 436) - [public] snd_error

```
■■■snd_error(CORE, "id of field %s is not an integer", id);
■■■err = -EINVAL;
■■■goto __error;
■■■}
■■■if (i == idx) {
■■■idx++;
■■■err = snd_config_get_ascii(n, &ptr);
■■■if (err < 0) {
■■■snd_error(CORE, "invalid ascii string for id %s", id);
■■■err = -EINVAL;
■■■goto __error;
■■■}
■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 440) - [public] if

```
■■■if (i == idx) {
■■■idx++;
■■■err = snd_config_get_ascii(n, &ptr);
■■■if (err < 0) {
■■■snd_error(CORE, "invalid ascii string for id %s", id);
■■■err = -EINVAL;
■■■goto __error;
■■■}
■■■len1 = strlen(ptr);
■■■tmp = realloc(res, len + len1 + 1);
■■■if (tmp == NULL) {
■■■free(ptr);
■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 443) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "invalid ascii string for id %s", id);
        err = -EINVAL;
        goto __error;
    }
    len1 = strlen(ptr);
    tmp = realloc(res, len + len1 + 1);
    if (tmp == NULL) {
        free(ptr);
        err = -ENOMEM;
        goto __error;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 444) - [public] snd_error

```
    snd_error(CORE, "invalid ascii string for id %s", id);
    err = -EINVAL;
    goto __error;
}
len1 = strlen(ptr);
tmp = realloc(res, len + len1 + 1);
if (tmp == NULL) {
    free(ptr);
    err = -ENOMEM;
    goto __error;
}
memcpy(tmp + len, ptr, len1);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 450) - [public] if

```
    if (tmp == NULL) {
        free(ptr);
        err = -ENOMEM;
        goto __error;
    }
    memcpy(tmp + len, ptr, len1);
    free(ptr);
    len += len1;
    tmp[len] = '\0';
    res = tmp;
    hit = 1;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 451) - [public] free

```
    free(ptr);
    err = -ENOMEM;
    goto __error;
}
memcpy(tmp + len, ptr, len1);
free(ptr);
len += len1;
tmp[len] = '\0';
res = tmp;
hit = 1;
}
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 455) - [public] memcpy

```
    memcpy(tmp + len, ptr, len1);
    free(ptr);
    len += len1;
    tmp[len] = '\0';
    res = tmp;
    hit = 1;
}
} while (hit);
if (res == NULL) {
    snd_error(CORE, "empty string is not accepted");
    err = -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 456) - [public] free

```
    free(ptr);
    len += len1;
    tmp[len] = '\0';
    res = tmp;
    hit = 1;
}
} while (hit);
if (res == NULL) {
    snd_error(CORE, "empty string is not accepted");
    err = -EINVAL;
    goto __error;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 465) - [public] snd_error

```
    snd_error(CORE, "empty string is not accepted");
    err = -EINVAL;
    goto __error;
}
err = snd_config_get_id(src, &id);
if (err >= 0)
    err = snd_config_imake_string(dst, id, res);
__error:
    free(res);
    return err;
}
#ifdef DOC_HIDDEN
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 496) - [public] snd_error

```
    snd_error(CORE, "field integers not found");
    goto __error;
}
err = snd_config_evaluate(n, root, private_data, NULL);
if (err < 0) {
    snd_error(CORE, "error evaluating integers");
    goto __error;
}
do {
    hit = 0;
    snd_config_for_each(i, next, n) {
        snd_config_t *n = snd_config_iterator_entry(i);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 501) - [public] snd_error

```
    ■■snd_error(CORE, "error evaluating integers");
    ■■goto __error;
    ■}
    ■do {
    ■■hit = 0;
    ■■snd_config_for_each(i, next, n) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id;
    ■■■long i;
    ■■■if (snd_config_get_id(n, &id) < 0)
    ■■■■continue;
    ■■■err = safe_strtol(id, &i);
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 506) - [public] snd_config_for_each

```
    ■■snd_config_for_each(i, next, n) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id;
    ■■■long i;
    ■■■if (snd_config_get_id(n, &id) < 0)
    ■■■■continue;
    ■■■err = safe_strtol(id, &i);
    ■■■if (err < 0) {
    ■■■■snd_error(CORE, "id of field %s is not an integer", id);
    ■■■■err = -EINVAL;
    ■■■■goto __error;
    ■■■}
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 513) - [public] if

```
    ■■■if (err < 0) {
    ■■■■snd_error(CORE, "id of field %s is not an integer", id);
    ■■■■err = -EINVAL;
    ■■■■goto __error;
    ■■■}
    ■■■if (i == idx) {
    ■■■■idx++;
    ■■■■err = snd_config_get_integer(n, &val);
    ■■■■if (err < 0) {
    ■■■■■snd_error(CORE, "invalid integer for id %s", id);
    ■■■■■err = -EINVAL;
    ■■■■■goto __error;
    ■■■■}
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 514) - [public] snd_error

```
    ■■■■snd_error(CORE, "id of field %s is not an integer", id);
    ■■■■err = -EINVAL;
    ■■■■goto __error;
    ■■■}
    ■■■if (i == idx) {
    ■■■■idx++;
    ■■■■err = snd_config_get_integer(n, &val);
    ■■■■if (err < 0) {
    ■■■■■snd_error(CORE, "invalid integer for id %s", id);
    ■■■■■err = -EINVAL;
    ■■■■■goto __error;
    ■■■■}
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 518) - [public] if

```
    if (i == idx) {
        idx++;
        err = snd_config_get_integer(n, &val);
        if (err < 0) {
            snd_error(CORE, "invalid integer for id %s", id);
            err = -EINVAL;
            goto __error;
        }
        switch (op) {
            case 0: result += val; break;
            case 1: result *= val; break;
        }
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/confmisc.c (line 521) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "invalid integer for id %s", id);
        err = -EINVAL;
        goto __error;
    }
    switch (op) {
        case 0: result += val; break;
        case 1: result *= val; break;
    }
    hit = 1;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 522) - [public] snd_error

```
    snd_error(CORE, "invalid integer for id %s", id);
    err = -EINVAL;
    goto __error;
}
switch (op) {
    case 0: result += val; break;
    case 1: result *= val; break;
}
hit = 1;
}
} while (hit);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 526) - [public] switch

```
    switch (op) {
        case 0: result += val; break;
        case 1: result *= val; break;
    }
    hit = 1;
}
} while (hit);
/* Preserve pre-fix behavior for an empty integers list. */
if (op == 1 && idx == 0)
    result = 0;
err = snd_config_get_id(src, &id);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 566) - [public] snd_func_iops

```
    return snd_func_iops(dst, root, src, private_data, 0);
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(snd_func_iadd, SND_CONFIG_DLSYM_VERSION_EVALUATE);
#endif

/**
 * \brief Multiply the given integers.
 * \param dst The function puts the handle to the result configuration node
 *           (with type integer) at the address specified by \p dst.
 * \param root Handle to the root source node.
 * \param src Handle to the source node, with a definition for \c integers.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 595) - [public] snd_func_iops

```
    return snd_func_iops(dst, root, src, private_data, 1);
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(snd_func_imul, SND_CONFIG_DLSYM_VERSION_EVALUATE);
#endif

/**
 * \brief Returns the ALSA data directory.
 * \param dst The function puts the handle to the result configuration node
 *           (with type string) at the address specified by \p dst.
 * \param root Handle to the root source node.
 * \param src Handle to the source node.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 626) - [public] snd_config_imake_string

```
    return snd_config_imake_string(dst, id, snd_config_topdir());
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(snd_func_datadir, SND_CONFIG_DLSYM_VERSION_EVALUATE);
#endif

static int open_ctl(long card, snd_ctl_t **ctl)
{
    char name[16];
    snprintf(name, sizeof(name), "hw:%li", card);
    name[sizeof(name)-1] = '\0';
    return snd_ctl_open(ctl, name, 0);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 637) - [public] snd_ctl_open

```
    return snd_ctl_open(ctl, name, 0);
}

#if 0
static int string_from_integer(char **dst, long v)
{
    char str[32];
    char *res;
    sprintf(str, "%li", v);
    res = strdup(str);
    if (res == NULL)
        return -ENOMEM;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 660) - [public] snd_config_copy

```
    return snd_config_copy(dst, src);
    if (snd_config_get_type(*private_data) == SND_CONFIG_TYPE_COMPOUND) {
        err = snd_config_search(*private_data, id, private_data);
        if (err)
            goto notfound;
    }
    err = snd_config_test_id(*private_data, id);
    if (err) {
        notfound:
        snd_error(CORE, "field %s not found", id);
        return -EINVAL;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 669) - [public] snd_error

```
    snd_error(CORE, "field %s not found", id);
    return -EINVAL;
}
return 0;
}

/**
 * \brief Returns the string from \c private_data.
 * \param dst The function puts the handle to the result configuration node
 *           (with type string) at the address specified by \p dst.
 * \param root Handle to the root source node.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 704) - [public] snd_error

```
    snd_error(CORE, "field string is not a string");
    return err;
}
err = snd_config_get_id(src, &id);
if (err >= 0)
    err = snd_config_imake_string(dst, id, str);
return err;
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(snd_func_private_string, SND_CONFIG_DLSYM_VERSION_EVALUATE);
#endif

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 745) - [public] snd_error

```
    snd_error(CORE, "field integer is not a string");
    return err;
}
err = snd_config_get_id(src, &id);
if (err >= 0)
    err = snd_config_imake_integer(dst, id, val);
return err;
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(snd_func_private_integer, SND_CONFIG_DLSYM_VERSION_EVALUATE);
#endif

// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/confmisc.c (line 768) - [public] snd_error

```
    ■■snd_error(CORE, "could not open control for card %i", card);
    ■■goto __error;
    ■}
    ■err = snd_ctl_card_info(ctl, &info);
    ■if (err < 0) {
    ■■snd_error(CORE, "snd_ctl_card_info error: %s", snd_strerror(err));
    ■■goto __error;
    ■}
    ■res = strdup(snd_ctl_card_info_get_driver(&info));
    ■if (res == NULL)
    ■■err = -ENOMEM;
    ■else {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 773) - [public] snd_error

```
    ■■snd_error(CORE, "snd_ctl_card_info error: %s", snd_strerror(err));
    ■■goto __error;
    ■}
    ■res = strdup(snd_ctl_card_info_get_driver(&info));
    ■if (res == NULL)
    ■■err = -ENOMEM;
    ■else {
    ■■*driver = res;
    ■■err = 0;
    ■}
    __error:
    ■if (ctl)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 785) - [public] snd_ctl_close

```
    ■■snd_ctl_close(ctl);
    ■return err;
}
#endif

/**
 * \brief Returns the driver identification from \c private_data.
 * \param dst The function puts the handle to the result configuration node
 *            (with type string) at the address specified by \p dst.
 * \param root Handle to the root source node.
 * \param src Handle to the source node.
 * \param private_data Handle to the \c private_data node (type integer,
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 817) - [public] snd_error

```
    ■■snd_error(CORE, "field card not found");
    ■■return -EINVAL;
    ■}
    ■err = snd_config_get_integer(private_data, &card);
    ■if (err < 0) {
    ■■snd_error(CORE, "field card is not an integer");
    ■■return err;
    ■}
    ■if ((err = snd_determine_driver(card, &driver)) < 0)
    ■■return err;
    ■err = snd_config_get_id(src, &id);
    ■if (err >= 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 822) - [public] snd_error

```
    ■■snd_error(CORE, "field card is not an integer");
    ■■return err;
    ■}
    ■if ((err = snd_determine_driver(card, &driver)) < 0)
    ■■return err;
    ■err = snd_config_get_id(src, &id);
    ■if (err >= 0)
    ■■err = snd_config_imake_string(dst, id, driver);
    ■free(driver);
    ■return err;
    ■}
    #ifndef DOC_HIDDEN
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 846) - [public] snd_error

```
    ■■snd_error(CORE, "field card not found");
    ■■return err;
    ■}
    ■err = snd_config_evaluate(n, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "error evaluating card");
    ■■return err;
    ■}
    ■err = snd_config_get_ascii(n, &str);
    ■if (err < 0) {
    ■■snd_error(CORE, "field card is not an integer or a string");
    ■■return err;
    ■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 851) - [public] snd_error

```
    ■■snd_error(CORE, "error evaluating card");
    ■■return err;
    ■}
    ■err = snd_config_get_ascii(n, &str);
    ■if (err < 0) {
    ■■snd_error(CORE, "field card is not an integer or a string");
    ■■return err;
    ■}
    ■card = snd_card_get_index(str);
    ■if (card < 0)
    ■■snd_error(CORE, "cannot find card '%s'", str);
    ■free(str);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 856) - [public] snd_error

```
    ■■snd_error(CORE, "field card is not an integer or a string");
    ■■return err;
    ■}
    ■card = snd_card_get_index(str);
    ■if (card < 0)
    ■■snd_error(CORE, "cannot find card '%s'", str);
    ■free(str);
    ■return card;
    ■}

/**
 * \brief Returns the card number as integer.
 * // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/confmisc.c (line 982) - [public] snd_ctl_close

```
    ■■snd_ctl_close(ctl);
    ■■return err;
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(snd_func_card_id, SND_CONFIG_DLSYM_VERSION_EVALUATE);
    #endif

    /**
     * \brief Returns the name of a card.
     * \param dst The function puts the handle to the result configuration node
     *           (with type string) at the address specified by \p dst.
     * \param root Handle to the root source node.
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1019) - [public] snd_error

```
    ■■snd_error(CORE, "could not open control for card %i", card);
    ■■goto __error;
    ■}
    ■err = snd_ctl_card_info(ctl, &info);
    ■if (err < 0) {
    ■■snd_error(CORE, "snd_ctl_card_info error: %s", snd_strerror(err));
    ■■goto __error;
    ■}
    ■err = snd_config_get_id(src, &id);
    ■if (err >= 0)
    ■■err = snd_config_imake_safe_string(dst, id,
    ■■■■■■snd_ctl_card_info_get_name(&info));
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1024) - [public] snd_error

```
    ■■snd_error(CORE, "snd_ctl_card_info error: %s", snd_strerror(err));
    ■■goto __error;
    ■}
    ■err = snd_config_get_id(src, &id);
    ■if (err >= 0)
    ■■err = snd_config_imake_safe_string(dst, id,
    ■■■■■■snd_ctl_card_info_get_name(&info));
    ■__error:
    ■if (ctl)
    ■■snd_ctl_close(ctl);
    ■return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1030) - [public] snd_ctl_card_info_get_name

```
    ■■■■■■snd_ctl_card_info_get_name(&info));
    ■__error:
    ■if (ctl)
    ■■snd_ctl_close(ctl);
    ■return err;
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(snd_func_card_name, SND_CONFIG_DLSYM_VERSION_EVALUATE);
    #endif

    #ifdef DOXYGEN
    /* For consistency with the PCM Interface module, include documentation even
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1033) - [public] snd_ctl_close

```
    ■■snd_ctl_close(ctl);
    ■■return err;
    ■}
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(snd_func_card_name, SND_CONFIG_DLSYM_VERSION_EVALUATE);
    #endif

    #ifdef DOXYGEN
    /* For consistency with the PCM Interface module, include documentation even
     * when PCM module is not included in the build. */
    #ifndef BUILD_PCM
    #define BUILD_PCM
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1084) - [public] snd_error

```
    ■■snd_error(CORE, "field device not found");
    ■■goto __error;
    ■}
    ■err = snd_config_evaluate(n, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "error evaluating device");
    ■■goto __error;
    ■}
    ■err = snd_config_get_integer(n, &device);
    ■if (err < 0) {
    ■■snd_error(CORE, "field device is not an integer");
    ■■goto __error;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1089) - [public] snd_error

```
    ■■snd_error(CORE, "error evaluating device");
    ■■goto __error;
    ■}
    ■err = snd_config_get_integer(n, &device);
    ■if (err < 0) {
    ■■snd_error(CORE, "field device is not an integer");
    ■■goto __error;
    ■}
    ■if (snd_config_search(src, "subdevice", &n) >= 0) {
    ■■err = snd_config_evaluate(n, root, private_data, NULL);
    ■■if (err < 0) {
    ■■■■snd_error(CORE, "error evaluating subdevice");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1094) - [public] snd_error

```
    ■■snd_error(CORE, "field device is not an integer");
    ■■goto __error;
    ■}
    ■if (snd_config_search(src, "subdevice", &n) >= 0) {
    ■■err = snd_config_evaluate(n, root, private_data, NULL);
    ■■if (err < 0) {
    ■■■■snd_error(CORE, "error evaluating subdevice");
    ■■■■goto __error;
    ■■}
    ■err = snd_config_get_integer(n, &subdevice);
    ■if (err < 0) {
    ■■■■snd_error(CORE, "field subdevice is not an integer");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1099) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "error evaluating subdevice");
        goto __error;
    }
    err = snd_config_get_integer(n, &subdevice);
    if (err < 0) {
        snd_error(CORE, "field subdevice is not an integer");
        goto __error;
    }
}
err = open_ctl(card, &ctl);
if (err < 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1100) - [public] snd_error

```
    snd_error(CORE, "error evaluating subdevice");
    goto __error;
}
err = snd_config_get_integer(n, &subdevice);
if (err < 0) {
    snd_error(CORE, "field subdevice is not an integer");
    goto __error;
}
}
err = open_ctl(card, &ctl);
if (err < 0) {
    snd_error(CORE, "could not open control for card %li", card);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1104) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "field subdevice is not an integer");
        goto __error;
    }
}
err = open_ctl(card, &ctl);
if (err < 0) {
    snd_error(CORE, "could not open control for card %li", card);
    goto __error;
}
snd_pcm_info_set_device(&info, device);
snd_pcm_info_set_subdevice(&info, subdevice);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1105) - [public] snd_error

```
    snd_error(CORE, "field subdevice is not an integer");
    goto __error;
}
}
err = open_ctl(card, &ctl);
if (err < 0) {
    snd_error(CORE, "could not open control for card %li", card);
    goto __error;
}
snd_pcm_info_set_device(&info, device);
snd_pcm_info_set_subdevice(&info, subdevice);
err = snd_ctl_pcm_info(ctl, &info);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1111) - [public] snd_error

```
    ■■snd_error(CORE, "could not open control for card %li", card);
    ■■goto __error;
    ■}
    ■snd_pcm_info_set_device(&info, device);
    ■snd_pcm_info_set_subdevice(&info, subdevice);
    ■err = snd_ctl_pcm_info(ctl, &info);
    ■if (err < 0) {
    ■■snd_error(CORE, "snd_ctl_pcm_info error: %s", snd_strerror(err));
    ■■goto __error;
    ■}
    ■err = snd_config_get_id(src, &id);
    ■if (err >= 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1118) - [public] snd_error

```
    ■■snd_error(CORE, "snd_ctl_pcm_info error: %s", snd_strerror(err));
    ■■goto __error;
    ■}
    ■err = snd_config_get_id(src, &id);
    ■if (err >= 0)
    ■■err = snd_config_imake_string(dst, id,
    ■■■■■■snd_pcm_info_get_id(&info));
        __error:
    ■if (ctl)
    ■■snd_ctl_close(ctl);
    ■return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1124) - [public] snd_pcm_info_get_id

```
    ■■■■■■snd_pcm_info_get_id(&info));
        __error:
    ■if (ctl)
    ■■snd_ctl_close(ctl);
    ■return err;
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(snd_func_pcm_id, SND_CONFIG_DLSYM_VERSION_EVALUATE);
    #endif

    /**
     * \brief Returns the pcm card and device arguments (in form CARD=N,DEV=M)
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1127) - [public] snd_ctl_close

```
    ■■snd_ctl_close(ctl);
    ■return err;
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(snd_func_pcm_id, SND_CONFIG_DLSYM_VERSION_EVALUATE);
    #endif

    /**
     * \brief Returns the pcm card and device arguments (in form CARD=N,DEV=M)
     *         for pcm specified by class and index.
     * \param dst The function puts the handle to the result configuration node
     *         (with type string) at the address specified by \p dst.
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1167) - [public] snd_error

```
    ■■snd_error(CORE, "field class not found");
    ■■goto __out;
    ■}
    ■err = snd_config_evaluate(n, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "error evaluating class");
    ■■goto __out;
    ■}
    ■err = snd_config_get_integer(n, &class);
    ■if (err < 0) {
    ■■snd_error(CORE, "field class is not an integer");
    ■■goto __out;
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1172) - [public] snd_error

```
    ■■snd_error(CORE, "error evaluating class");
    ■■goto __out;
    ■}
    ■err = snd_config_get_integer(n, &class);
    ■if (err < 0) {
    ■■snd_error(CORE, "field class is not an integer");
    ■■goto __out;
    ■}
    ■err = snd_config_search(src, "index", &n);
    ■if (err < 0) {
    ■■snd_error(CORE, "field index not found");
    ■■goto __out;
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1177) - [public] snd_error

```
    ■■snd_error(CORE, "field class is not an integer");
    ■■goto __out;
    ■}
    ■err = snd_config_search(src, "index", &n);
    ■if (err < 0) {
    ■■snd_error(CORE, "field index not found");
    ■■goto __out;
    ■}
    ■err = snd_config_evaluate(n, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "error evaluating index");
    ■■goto __out;
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1182) - [public] snd_error

```
    ■■snd_error(CORE, "field index not found");
    ■■goto __out;
    ■}
    ■err = snd_config_evaluate(n, root, private_data, NULL);
    ■if (err < 0) {
    ■■snd_error(CORE, "error evaluating index");
    ■■goto __out;
    ■}
    ■err = snd_config_get_integer(n, &index);
    ■if (err < 0) {
    ■■snd_error(CORE, "field index is not an integer");
    ■■goto __out;
    ■    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/confmisc.c (line 1187) - [public] snd_error

```
    ■■snd_error(CORE, "error evaluating index");
    ■■goto __out;
    ■}
    ■err = snd_config_get_integer(n, &index);
    ■if (err < 0) {
    ■■snd_error(CORE, "field index is not an integer");
    ■■goto __out;
    ■}

    ■while(1) {
    ■■err = snd_card_next(&card);
    ■■if (err < 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1192) - [public] snd_error

```
    ■■snd_error(CORE, "field index is not an integer");
    ■■goto __out;
    ■}

    ■while(1) {
    ■■err = snd_card_next(&card);
    ■■if (err < 0) {
    ■■■■snd_error(CORE, "could not get next card");
    ■■■■goto __out;
    ■■}
    ■■if (card < 0)
    ■■■■break;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1198) - [public] if

```
    ■■if (err < 0) {
    ■■■■snd_error(CORE, "could not get next card");
    ■■■■goto __out;
    ■■}
    ■■if (card < 0)
    ■■■■break;
    ■■err = open_ctl(card, &ctl);
    ■■if (err < 0) {
    ■■■■snd_error(CORE, "could not open control for card %i", card);
    ■■■■goto __out;
    ■■}
    ■■dev = -1;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1199) - [public] snd_error

```
    ■■■■snd_error(CORE, "could not get next card");
    ■■■■goto __out;
    ■■}
    ■■if (card < 0)
    ■■■■break;
    ■■err = open_ctl(card, &ctl);
    ■■if (err < 0) {
    ■■■■snd_error(CORE, "could not open control for card %i", card);
    ■■■■goto __out;
    ■■}
    ■■dev = -1;
    ■■while(1) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1205) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "could not open control for card %i", card);
        goto __out;
    }
    dev = -1;
    while(1) {
        err = snd_ctl_pcm_next_device(ctl, &dev);
        if (err < 0) {
            snd_error(CORE, "could not get next pcm for card %i", card);
            goto __out;
        }
        if (dev < 0)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1206) - [public] snd_error

```
    snd_error(CORE, "could not open control for card %i", card);
    goto __out;
}
dev = -1;
while(1) {
    err = snd_ctl_pcm_next_device(ctl, &dev);
    if (err < 0) {
        snd_error(CORE, "could not get next pcm for card %i", card);
        goto __out;
    }
    if (dev < 0)
        break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1210) - [public] while

```
while(1) {
    err = snd_ctl_pcm_next_device(ctl, &dev);
    if (err < 0) {
        snd_error(CORE, "could not get next pcm for card %i", card);
        goto __out;
    }
    if (dev < 0)
        break;
    snd_pcm_info_set_device(&info, dev);
    err = snd_ctl_pcm_info(ctl, &info);
    if (err < 0)
        continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1212) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "could not get next pcm for card %i", card);
        goto __out;
    }
    if (dev < 0)
        break;
    snd_pcm_info_set_device(&info, dev);
    err = snd_ctl_pcm_info(ctl, &info);
    if (err < 0)
        continue;
    if (snd_pcm_info_get_class(&info) == (snd_pcm_class_t)class &&
        index == idx++)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1213) - [public] snd_error

```
    snd_error(CORE, "could not get next pcm for card %i", card);
    goto __out;
}
if (dev < 0)
    break;
snd_pcm_info_set_device(&info, dev);
err = snd_ctl_pcm_info(ctl, &info);
if (err < 0)
    continue;
if (snd_pcm_info_get_class(&info) == (snd_pcm_class_t)class &&
    index == idx++)
    goto __out;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1218) - [public] snd_pcm_info_set_device

```
    snd_pcm_info_set_device(&info, dev);
    err = snd_ctl_pcm_info(ctl, &info);
    if (err < 0)
        continue;
    if (snd_pcm_info_get_class(&info) == (snd_pcm_class_t)class &&
        index == idx++)
        goto __out;
}
snd_ctl_close(ctl);
ctl = NULL;
}
err = -ENODEV;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1226) - [public] snd_ctl_close

```
    snd_ctl_close(ctl);
    ctl = NULL;
}
err = -ENODEV;

__out:
if (ctl)
    snd_ctl_close(ctl);
if (err < 0)
    return err;
if ((err = snd_config_get_id(src, &id)) >= 0) {
    char name[32];
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1233) - [public] snd_ctl_close

```
    snd_ctl_close(ctl);
    if (err < 0)
        return err;
    if ((err = snd_config_get_id(src, &id)) >= 0) {
        char name[32];
        snprintf(name, sizeof(name), "CARD=%i,DEV=%i", card, dev);
        err = snd_config_imake_string(dst, id, name);
    }
    return err;
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(snd_func_pcm_args_by_class, SND_CONFIG_DLSYM_VERSION_EVALUATE);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1238) - [public] snprintf

```
■ ■snprintf(name, sizeof(name), "CARD=%i,DEV=%i", card, dev);
■ ■err = snd_config_imake_string(dst, id, name);
■ }
■ return err;
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(snd_func_pcm_args_by_class, SND_CONFIG_DLSYM_VERSION_EVALUATE);
#endif

/**
 * \brief Returns the PCM subdevice from \c private_data.
 * \param dst The function puts the handle to the result configuration node
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1274) - [public] snd_config_copy

```
■ ■return snd_config_copy(dst, src);
■ ■err = snd_config_test_id(private_data, "pcm_handle");
■ if (err) {
■ ■snd_error(CORE, "field pcm_handle not found");
■ ■return -EINVAL;
■ }
■ ■err = snd_config_get_pointer(private_data, &data);
■ pcm = (snd_pcm_t *)data;
■ if (err < 0) {
■ ■snd_error(CORE, "field pcm_handle is not a pointer");
■ ■return err;
■ }
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1277) - [public] snd_error

```
■ ■snd_error(CORE, "field pcm_handle not found");
■ ■return -EINVAL;
■ }
■ ■err = snd_config_get_pointer(private_data, &data);
■ pcm = (snd_pcm_t *)data;
■ if (err < 0) {
■ ■snd_error(CORE, "field pcm_handle is not a pointer");
■ ■return err;
■ }
■ ■err = snd_pcm_info(pcm, &info);
■ if (err < 0) {
■ ■snd_error(CORE, "snd_ctl_pcm_info error: %s", snd_strerror(err));
■ // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1283) - [public] snd_error

```
■ ■snd_error(CORE, "field pcm_handle is not a pointer");
■ ■return err;
■ }
■ ■err = snd_pcm_info(pcm, &info);
■ if (err < 0) {
■ ■snd_error(CORE, "snd_ctl_pcm_info error: %s", snd_strerror(err));
■ ■return err;
■ }
■ ■err = snd_config_get_id(src, &id);
■ if (err >= 0)
■ ■err = snd_config_imake_integer(dst, id,
■ ■■■■snd_pcm_info_get_subdevice(&info));
■ // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1288) - [public] snd_error

```
    ■■snd_error(CORE, "snd_ctl_pcm_info error: %s", snd_strerror(err));
    ■■return err;
    ■}
    ■err = snd_config_get_id(src, &id);
    ■if (err >= 0)
    ■■err = snd_config_imake_integer(dst, id,
    ■■■■snd_pcm_info_get_subdevice(&info));
    ■return err;
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(snd_func_private_pcm_subdevice, SND_CONFIG_DLSYM_VERSION_EVALUATE);
    #endif
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1294) - [public] snd_pcm_info_get_subdevice

```
    ■■■■snd_pcm_info_get_subdevice(&info));
    ■return err;
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(snd_func_private_pcm_subdevice, SND_CONFIG_DLSYM_VERSION_EVALUATE);
    #endif

    #endif /* BUILD_PCM */

    /**
     * \brief Copies the specified configuration node.
     * \param dst The function puts the handle to the result configuration node
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1334) - [public] if

```
    ■■if (err < 0) {
    ■■■snd_error(CORE, "error evaluating file");
    ■■■goto _end;
    ■■}
    ■err = snd_config_get_string(n, &file);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "file is not a string");
    ■■■goto _end;
    ■■}
    ■}
    ■err = snd_config_search(src, "name", &n);
    ■if (err >= 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1335) - [public] snd_error

```
    ■■■snd_error(CORE, "error evaluating file");
    ■■■goto _end;
    ■■}
    ■err = snd_config_get_string(n, &file);
    ■■if (err < 0) {
    ■■■snd_error(CORE, "file is not a string");
    ■■■goto _end;
    ■■}
    ■}
    ■err = snd_config_search(src, "name", &n);
    ■if (err >= 0) {
    ■■err = snd_config_evaluate(n, root, private_data, NULL);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1339) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "file is not a string");
        goto _end;
    }
    err = snd_config_search(src, "name", &n);
    if (err >= 0) {
        err = snd_config_evaluate(n, root, private_data, NULL);
        if (err < 0) {
            snd_error(CORE, "error evaluating name");
            goto _end;
        }
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/confmisc.c (line 1340) - [public] snd_error

```
    snd_error(CORE, "file is not a string");
    goto _end;
}
err = snd_config_search(src, "name", &n);
if (err >= 0) {
    err = snd_config_evaluate(n, root, private_data, NULL);
    if (err < 0) {
        snd_error(CORE, "error evaluating name");
        goto _end;
    }
    err = snd_config_get_string(n, &name);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/confmisc.c (line 1347) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "error evaluating name");
        goto _end;
    }
    err = snd_config_get_string(n, &name);
    if (err < 0) {
        snd_error(CORE, "name is not a string");
        goto _end;
    }
    if (!name) {
        err = -EINVAL;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/confmisc.c (line 1348) - [public] snd_error

```
    snd_error(CORE, "error evaluating name");
    goto _end;
}
err = snd_config_get_string(n, &name);
if (err < 0) {
    snd_error(CORE, "name is not a string");
    goto _end;
}
if (!name) {
    err = -EINVAL;
    snd_error(CORE, "name is not specified");
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/confmisc.c (line 1352) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "name is not a string");
        goto _end;
    }
    if (!name) {
        err = -EINVAL;
        snd_error(CORE, "name is not specified");
        goto _end;
    }
    if (file) {
        snd_input_t *input;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1353) - [public] snd_error

```
        snd_error(CORE, "name is not a string");
        goto _end;
    }
    if (!name) {
        err = -EINVAL;
        snd_error(CORE, "name is not specified");
        goto _end;
    }
    if (file) {
        snd_input_t *input;
        err = snd_input_stdio_open(&input, file, "r");
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1359) - [public] snd_error

```
        snd_error(CORE, "name is not specified");
        goto _end;
    }
    if (file) {
        snd_input_t *input;
        err = snd_input_stdio_open(&input, file, "r");
        if (err < 0) {
            snd_error(CORE, "Unable to open file %s: %s", file, snd_strerror(err));
            goto _end;
        }
        err = snd_config_load(root, input);
        snd_input_close(input);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1365) - [public] if

```
    if (err < 0) {
        snd_error(CORE, "Unable to open file %s: %s", file, snd_strerror(err));
        goto _end;
    }
    err = snd_config_load(root, input);
    snd_input_close(input);
    if (err < 0)
        goto _end;
    err = snd_config_search_definition(root, NULL, name, dst);
    if (err >= 0) {
        const char *id;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1366) - [public] snd_error

```
    snd_error(CORE, "Unable to open file %s: %s", file, snd_strerror(err));
    goto _end;
}
err = snd_config_load(root, input);
snd_input_close(input);
if (err < 0)
    goto _end;
}
err = snd_config_search_definition(root, NULL, name, dst);
if (err >= 0) {
    const char *id;
    err = snd_config_get_id(src, &id);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1370) - [public] snd_input_close

```
    snd_input_close(input);
    if (err < 0)
        goto _end;
}
err = snd_config_search_definition(root, NULL, name, dst);
if (err >= 0) {
    const char *id;
    err = snd_config_get_id(src, &id);
    if (err >= 0)
        err = snd_config_set_id(*dst, id);
} else {
    err = snd_config_search(src, "default", &n);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1383) - [public] snd_error

```
    snd_error(CORE, "Unable to find definition '%s'", name);
} else {
    const char *id;
    err = snd_config_evaluate(n, root, private_data, NULL);
    if (err < 0)
        return err;
    if ((err = snd_config_copy(dst, n)) >= 0) {
        if ((err = snd_config_get_id(src, &id)) < 0 ||
            (err = snd_config_set_id(*dst, id)) < 0)
            snd_config_delete(*dst);
    }
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/confmisc.c (line 1389) - [public] if

```
    if ((err = snd_config_copy(dst, n)) >= 0) {
        if ((err = snd_config_get_id(src, &id)) < 0 ||
            (err = snd_config_set_id(*dst, id)) < 0)
            snd_config_delete(*dst);
    }
}
_end:
return err;
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(snd_func_refer, SND_CONFIG_DLSYM_VERSION_EVALUATE);
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/confmisc.c (line 1392) - [public] snd_config_delete

```
■■■■■snd_config_delete(*dst);
■■■■}
■■■}
■}
_end:
■return err;
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(snd_func_refer, SND_CONFIG_DLSYM_VERSION_EVALUATE);
#endif

#ifdef DOC_HIDDEN
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 49) - [public] if

```
■■■if (ioctl(open_dev, SNDRV_CTL_IOCTL_CARD_INFO, &info) < 0) {
■■■int err = -errno;
■■■close(open_dev);
■■■return err;
■■■}
■■■close(open_dev);
■■■return info.card;
■} else {
■■■return -errno;
■}
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 51) - [public] close

```
■■■close(open_dev);
■■■return err;
■■■}
■■■close(open_dev);
■■■return info.card;
■} else {
■■■return -errno;
■}
}
```

```
static int snd_card_load1(int card)
{
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 54) - [public] close

```
■■■close(open_dev);
■■■return info.card;
■} else {
■■■return -errno;
■}
}
```

```
static int snd_card_load1(int card)
{
■int res;
■char control[sizeof(SND_FILE_CONTROL) + 10];

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 71) - [public] sprintf

```
    ■■sprintf(aload, SND_FILE_LOAD, card);
    ■■res = snd_card_load2(aload);
    ■}
    #endif
    ■return res;
}

/**
 * \brief Try to load the driver for a card.
 * \param card Card index.
 * \return 1 if driver is present, zero if driver is not present.
 */
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 117) - [public] if

```
    ■■if (snd_card_load(card)) {
    ■■*rcard = card;
    ■■return 0;
    ■}
    ■}
    ■*rcard = -1;
    ■return 0;
}

/**
 * \brief Convert a card string to the card index.
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 162) - [public] snd_card_load2

```
if (!string || *string == '\0')
return -EINVAL;
if ((isdigit(*string) && *(string + 1) == 0) ||
(isdigit(*string) && isdigit(*(string + 1)) && *(string + 2) == 0)) {
/* We got an index */
if (sscanf(string, "%i", &card) != 1)
return -EINVAL;
if (card < 0 || card >= SND_MAX_CARDS)
return -EINVAL;
err = snd_card_load1(card);
if (err >= 0)
return card;
return err;
}
if (string[0] == '/')
/* We got a device name */

    ■■return snd_card_load2(string);
    ■/* We got in ID */
    ■for (card = 0; card < SND_MAX_CARDS; card++) {
    #ifdef SUPPORT_ALOAD
    ■■if (! snd_card_load(card))
    ■■continue;
    #endif
    ■■if (snd_ctl_hw_open(&handle, NULL, card, 0) < 0)
    ■■continue;
    ■■if (snd_ctl_card_info(handle, &info) < 0) {
    ■■snd_ctl_close(handle);
    ■■continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 171) - [public] if

```
    if (snd_ctl_card_info(handle, &info) < 0) {
        snd_ctl_close(handle);
        continue;
    }
    snd_ctl_close(handle);
    if (!strcmp((const char *)info.id, string))
        return card;
    }
    return -ENODEV;
}

/**
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 172) - [public] snd_ctl_close

```
    snd_ctl_close(handle);
    continue;
}
snd_ctl_close(handle);
if (!strcmp((const char *)info.id, string))
    return card;
}
return -ENODEV;
}

/**
    * \brief Obtain the card name.
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 175) - [public] snd_ctl_close

```
    snd_ctl_close(handle);
    if (!strcmp((const char *)info.id, string))
        return card;
    }
    return -ENODEV;
}

/**
    * \brief Obtain the card name.
    *
    * \param card The index of the card.
    * \param name Result - card name corresponding to card index.
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 203) - [public] snd_ctl_close

```
    snd_ctl_close(handle);
    return err;
}
snd_ctl_close(handle);
*name = strdup((const char *)info.name);
if (*name == NULL)
    return -ENOMEM;
return 0;
}

/**
    * \brief Obtain the card long name.
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/cards.c (line 233) - [public] snd_ctl_close

```
    snd_ctl_close(handle);
    return err;
}
snd_ctl_close(handle);
*name = strdup((const char *)info.longname);
if (*name == NULL)
    return -ENOMEM;
return 0;
}
```

dirtycode/alsa-lib/src/control/control.c (line 241) - [public] snd_async_del_handler

```
    ■■snd_async_del_handler(h);
    ■}
    ■err = ctl->ops->close(ctl);
    ■free(ctl->name);
    ■snd_dlobj_cache_put(ctl->open_func);
    ■free(ctl);
    ■return err;
}

/**
 * \brief set nonblock mode
 * \param ctl CTL handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 578) - [public] snd_ctl_elem_id_set_numid

```
    ■■snd_ctl_elem_id_set_numid(&data.id, numid + i);

    ■■for (j = 0; j < member_count; j++)
    ■■■data.value.integer.value[j] = min;

    ■■err = ctl->ops->element_write(ctl, &data);
    ■■if (err < 0)
    ■■■return err;
    ■}

    ■return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 660) - [public] snd_ctl_elem_id_set_numid

```
    ■■snd_ctl_elem_id_set_numid(&data.id, numid + i);

    ■■for (j = 0; j < member_count; j++)
    ■■■data.value.integer64.value[j] = min;

    ■■err = ctl->ops->element_write(ctl, &data);
    ■■if (err < 0)
    ■■■return err;
    ■}

    ■return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 722) - [public] __snd_ctl_add_elem_set

```
    ■return __snd_ctl_add_elem_set(ctl, info, element_count, member_count);
}

/**
 * \brief Create and add some user-defined control elements of enumerated type.
 * \param ctl A handle of backend module for control interface.
 * \param info Common information for a new element set, with ID of the first new
 * ■ element.
 * \param element_count The number of elements added by this operation.
 * \param member_count The number of members which a element has to
 * ■ ■ represent its states.
 * \param items Range of possible values (0 ... \a items - 1).
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 798) - [public] strcpy

```
    strcpy(p, labels[i]);
    p += strlen(labels[i]) + 1;
}

err = __snd_ctl_add_elem_set(ctl, info, element_count, member_count);

free(buf);

return err;
}

/**
 * ... (truncated, see source file for more)
 */
```

dirtycode/alsa-lib/src/control/control.c (line 857) - [public] __snd_ctl_add_elem_set

```
return __snd_ctl_add_elem_set(ctl, info, element_count, member_count);
}

/**
 * \brief Create and add an user-defined control element of integer type.
 *
 * This is a wrapper function to snd_ctl_add_integer_elem_set() for a control
 * element. This doesn't fill the id data with full information, thus it's
 * recommended to use snd_ctl_add_integer_elem_set(), instead.
 */
int snd_ctl_elem_add_integer(snd_ctl_t *ctl, const snd_ctl_elem_id_t *id,
    unsigned int member_count,
    // ... (truncated, see source file for more)
)
```

dirtycode/alsa-lib/src/control/control.c (line 918) - [public] snd_ctl_add_boolean_elem_set

```
return snd_ctl_add_boolean_elem_set(ctl, &info, 1, member_count);
}

/**
 * \brief Create and add a user-defined control element of enumerated type.
 *
 * This is a wrapper function to snd_ctl_add_enumerated_elem_set() for a single
 * control element. This doesn't fill the id data with full information, thus
 * it's recommended to use snd_ctl_add_enumerated_elem_set(), instead.
 *
 * This function is added in version 1.0.25.
 */
// ... (truncated, see source file for more)
)
```

dirtycode/alsa-lib/src/control/control.c (line 1056) - [public] if

```
if (id->numid == 0) {
    err = -ENOENT;
    goto __err;
}

err = ctl->ops->element_tlv(ctl, op_flag, id->numid, tlv, tlv_size);
__err:
if (info)
    free(info);
return err;
}

// ... (truncated, see source file for more)
)
```

dirtycode/alsa-lib/src/control/control.c (line 1064) - [public] free

```
    free(info);
    return err;
}

/**
 * \brief Read structured data from an element set to given buffer.
 * \param ctl A handle of backend module for control interface.
 * \param id ID of an element.
 * \param tlv An array with members of unsigned int type.
 * \param tlv_size The length of the array.
 * \return 0 on success otherwise a negative error code
 *
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 1370) - [public] snd_error

```
    snd_error(CONTROL, "Invalid poll_fds %d", npfds);
    return -EIO;
}
pfd = alloca(sizeof(*pfd) * npfds);
err = snd_ctl_poll_descriptors(ctl, pfd, npfds);
if (err < 0)
    return err;
if (err != npfds) {
    snd_check(CONTROL, "invalid poll descriptors %d", err);
    return -EIO;
}
for (;;) {
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 1378) - [public] snd_check

```
    snd_check(CONTROL, "invalid poll descriptors %d", err);
    return -EIO;
}
for (;;) {
    err_poll = poll(pfd, npfds, timeout);
    if (err_poll < 0)
        return -errno;
    if (! err_poll)
        return 0;
    err = snd_ctl_poll_descriptors_revents(ctl, pfd, npfds, &revents);
    if (err < 0)
        return err;
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 1421) - [public] if

```
    if (err < 0) {
        snd_async_del_handler(h);
        return err;
    }
    *handler = h;
    return 0;
}

/**
 * \brief Return CTL handle related to an async handler
 * \param handler Async handler handle
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 1422) - [public] snd_async_del_handler

```
    snd_async_del_handler(h);
    return err;
}

*handler = h;
return 0;
}

/**
 * \brief Return CTL handle related to an async handler
 * \param handler Async handler handle
 * \return CTL handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1461) - [public] snd_error

```
    snd_error(CONTROL, "Invalid type for CTL %s definition", name);
} else
    snd_error(CONTROL, "Invalid type for CTL definition");
return -EINVAL;
}

err = snd_config_search(ctl_conf, "type", &conf);
if (err < 0) {
    snd_error(CONTROL, "type is not defined");
    return err;
}

err = snd_config_get_id(conf, &id);
if (err < 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1463) - [public] snd_error

```
    snd_error(CONTROL, "Invalid type for CTL definition");
    return -EINVAL;
}

err = snd_config_search(ctl_conf, "type", &conf);
if (err < 0) {
    snd_error(CONTROL, "type is not defined");
    return err;
}

err = snd_config_get_id(conf, &id);
if (err < 0) {
    snd_error(CONTROL, "unable to get id");
    return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1468) - [public] snd_error

```
    snd_error(CONTROL, "type is not defined");
    return err;
}

err = snd_config_get_id(conf, &id);
if (err < 0) {
    snd_error(CONTROL, "unable to get id");
    return err;
}

err = snd_config_get_string(conf, &str);
if (err < 0) {
    snd_error(CONTROL, "Invalid type for %s", id);
    return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1473) - [public] snd_error

```
    ■■snd_error(CONTROL, "unable to get id");
    ■■return err;
    ■}
    ■err = snd_config_get_string(conf, &str);
    ■if (err < 0) {
    ■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■return err;
    ■}
    ■err = snd_config_search_definition(ctl_root, "ctl_type", str, &type_conf);
    ■if (err >= 0) {
    ■■if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
    ■■■snd_error(CONTROL, "Invalid type for CTL type %s definition", str);
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1478) - [public] snd_error

```
    ■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■return err;
    ■}
    ■err = snd_config_search_definition(ctl_root, "ctl_type", str, &type_conf);
    ■if (err >= 0) {
    ■■if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
    ■■■snd_error(CONTROL, "Invalid type for CTL type %s definition", str);
    ■■■err = -EINVAL;
    ■■■goto _err;
    ■■}
    ■■snd_config_for_each(i, next, type_conf) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1483) - [public] if

```
    ■■if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
    ■■■snd_error(CONTROL, "Invalid type for CTL type %s definition", str);
    ■■■err = -EINVAL;
    ■■■goto _err;
    ■■}
    ■■snd_config_for_each(i, next, type_conf) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id;
    ■■■if (snd_config_get_id(n, &id) < 0)
    ■■■■continue;
    ■■■if (strcmp(id, "comment") == 0)
    ■■■■continue;
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1484) - [public] snd_error

```
    ■■■snd_error(CONTROL, "Invalid type for CTL type %s definition", str);
    ■■■err = -EINVAL;
    ■■■goto _err;
    ■■}
    ■■snd_config_for_each(i, next, type_conf) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id;
    ■■■if (snd_config_get_id(n, &id) < 0)
    ■■■■continue;
    ■■■if (strcmp(id, "comment") == 0)
    ■■■■continue;
    ■■■if (strcmp(id, "lib") == 0) {
    ■■■■// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/control.c (line 1488) - [public] snd_config_for_each

```
    ■■■snd_config_for_each(i, next, type_conf) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id;
    ■■■if (snd_config_get_id(n, &id) < 0)
    ■■■continue;
    ■■■if (strcmp(id, "comment") == 0)
    ■■■continue;
    ■■■if (strcmp(id, "lib") == 0) {
    ■■■err = snd_config_get_string(n, &lib);
    ■■■if (err < 0) {
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■■goto _err;
    ■■■// ... (truncated, see source file for more)
    ■■■}
```

dirtycode/alsa-lib/src/control/control.c (line 1495) - [public] if

```
    ■■■if (strcmp(id, "lib") == 0) {
    ■■■err = snd_config_get_string(n, &lib);
    ■■■if (err < 0) {
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■■goto _err;
    ■■■}
    ■■■continue;
    ■■■}
    ■■■if (strcmp(id, "open") == 0) {
    ■■■err = snd_config_get_string(n, &open_name);
    ■■■if (err < 0) {
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■// ... (truncated, see source file for more)
    ■■■}
```

dirtycode/alsa-lib/src/control/control.c (line 1497) - [public] if

```
    ■■■if (err < 0) {
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■■goto _err;
    ■■■}
    ■■■continue;
    ■■■}
    ■■■if (strcmp(id, "open") == 0) {
    ■■■err = snd_config_get_string(n, &open_name);
    ■■■if (err < 0) {
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■■goto _err;
    ■■■}
    ■■■// ... (truncated, see source file for more)
    ■■■}
```

dirtycode/alsa-lib/src/control/control.c (line 1498) - [public] snd_error

```
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■■goto _err;
    ■■■}
    ■■■continue;
    ■■■}
    ■■■if (strcmp(id, "open") == 0) {
    ■■■err = snd_config_get_string(n, &open_name);
    ■■■if (err < 0) {
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■■goto _err;
    ■■■}
    ■■■continue;
    ■■■// ... (truncated, see source file for more)
    ■■■}
```

dirtycode/alsa-lib/src/control/control.c (line 1503) - [public] if

```
    if (strcmp(id, "open") == 0) {
        err = snd_config_get_string(n, &open_name);
        if (err < 0) {
            snd_error(CONTROL, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    snd_error(CONTROL, "Unknown field %s", id);
    err = -EINVAL;
    goto _err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1505) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Invalid type for %s", id);
        goto _err;
    }
    continue;
}
snd_error(CONTROL, "Unknown field %s", id);
err = -EINVAL;
goto _err;
}
}
if (!open_name) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1506) - [public] snd_error

```
    snd_error(CONTROL, "Invalid type for %s", id);
    goto _err;
}
continue;
}
snd_error(CONTROL, "Unknown field %s", id);
err = -EINVAL;
goto _err;
}
}
if (!open_name) {
    buf = malloc(strlen(str) + 32);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1511) - [public] snd_error

```
    snd_error(CONTROL, "Unknown field %s", id);
    err = -EINVAL;
    goto _err;
}
}
if (!open_name) {
    buf = malloc(strlen(str) + 32);
    if (buf == NULL) {
        err = -ENOMEM;
        goto _err;
    }
    open_name = buf;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1518) - [public] if

```
    if (buf == NULL) {
        err = -ENOMEM;
        goto _err;
    }
    open_name = buf;
    sprintf(buf, "_snd_ctl_%s_open", str);
}
if (!lib) {
    const char *const *build_in = build_in_ctls;
    while (*build_in) {
        if (!strcmp(*build_in, str))
            break;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/control.c (line 1523) - [public] sprintf

```
    sprintf(buf, "_snd_ctl_%s_open", str);
}
if (!lib) {
    const char *const *build_in = build_in_ctls;
    while (*build_in) {
        if (!strcmp(*build_in, str))
            break;
        build_in++;
    }
    if (*build_in == NULL) {
        buf1 = malloc(strlen(str) + 32);
        if (buf1 == NULL) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/control/control.c (line 1527) - [public] while

```
    while (*build_in) {
        if (!strcmp(*build_in, str))
            break;
        build_in++;
    }
    if (*build_in == NULL) {
        buf1 = malloc(strlen(str) + 32);
        if (buf1 == NULL) {
            err = -ENOMEM;
            goto _err;
        }
        lib = buf1;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/control.c (line 1532) - [public] if

```
    if (*build_in == NULL) {
        buf1 = malloc(strlen(str) + 32);
        if (buf1 == NULL) {
            err = -ENOMEM;
            goto _err;
        }
        lib = buf1;
        sprintf(buf1, "libasound_module_ctl_%s.so", str);
    }
}
#ifdef PIC
snd_control_open_symbols();
// ... (truncated, see source file for more)
#endif
```

dirtycode/alsa-lib/src/control/control.c (line 1534) - [public] if

```
    if (buf1 == NULL) {
        err = -ENOMEM;
        goto _err;
    }
    lib = buf1;
    sprintf(buf1, "libasound_module_ctl_%s.so", str);
}
#endif
#ifdef PIC
snd_control_open_symbols();
#endif
open_func = snd_dlobj_cache_get(lib, open_name,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1539) - [public] sprintf

```
    sprintf(buf1, "libasound_module_ctl_%s.so", str);
}
#endif
#ifdef PIC
snd_control_open_symbols();
#endif
open_func = snd_dlobj_cache_get(lib, open_name,
    SND_DLSYM_VERSION(SND_CONTROL_DLSYM_VERSION), 1);
if (open_func) {
    err = open_func(ctlp, name, ctl_root, ctl_conf, mode);
    if (err >= 0) {
        (*ctlp)->open_func = open_func;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1546) - [public] SND_DLSYM_VERSION

```
    SND_DLSYM_VERSION(SND_CONTROL_DLSYM_VERSION), 1);
if (open_func) {
    err = open_func(ctlp, name, ctl_root, ctl_conf, mode);
    if (err >= 0) {
        (*ctlp)->open_func = open_func;
        err = 0;
    } else {
        snd_dlobj_cache_put(open_func);
    }
} else {
    err = -ENXIO;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1549) - [public] if

```
if (err >= 0) {
    (*ctlp)->open_func = open_func;
    err = 0;
} else {
    snd_dlobj_cache_put(open_func);
}
} else {
    err = -ENXIO;
}
_err:
if (type_conf)
    snd_config_delete(type_conf);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1553) - [public] snd_dlobj_cache_put

```
    ■■■snd_dlobj_cache_put(open_func);
    ■■}
    ■} else {
    ■■err = -ENXIO;
    ■}

    _err:
    ■if (type_conf)
    ■■snd_config_delete(type_conf);
    ■free(buf);
    ■free(buf1);
    ■return err;
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1560) - [public] snd_config_delete

```
    ■■snd_config_delete(type_conf);
    ■free(buf);
    ■free(buf1);
    ■return err;
    }

    static int snd_ctl_open_noupdate(snd_ctl_t **ctlp, snd_config_t *root,
    ■■■■ const char *name, int mode, int hop)
    {
    ■int err;
    ■snd_config_t *ctl_conf;
    ■const char *str;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1575) - [public] snd_error

```
    ■■snd_error(CONTROL, "Invalid CTL %s", name);
    ■■return err;
    ■}
    ■if (snd_config_get_string(ctl_conf, &str) >= 0)
    ■■err = snd_ctl_open_noupdate(ctlp, root, str, mode, hop + 1);
    ■else {
    ■■snd_config_set_hop(ctl_conf, hop);
    ■■err = snd_ctl_open_conf(ctlp, name, root, ctl_conf, mode);
    ■}
    ■snd_config_delete(ctl_conf);
    ■return err;
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1581) - [public] snd_config_set_hop

```
    ■■snd_config_set_hop(ctl_conf, hop);
    ■■err = snd_ctl_open_conf(ctlp, name, root, ctl_conf, mode);
    ■}
    ■snd_config_delete(ctl_conf);
    ■return err;
    }

    #ifndef DOC_HIDDEN
    int _snd_ctl_open_named_child(snd_ctl_t **pctl, const char *name,
    ■■■      snd_config_t *root, snd_config_t *conf,
    ■■■      int mode, snd_config_t *parent_conf)
    {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control.c (line 1599) - [public] snd_ctl_open_noupdate

```
    return snd_ctl_open_noupdate(pctl, root, str, mode, hop + 1);
    return snd_ctl_open_conf(pctl, name, root, conf, mode);
}
#endif

/**
 * \brief Opens a sound card.
 *
 * \param ctlp Returned CTL handle.
 * \param name A string identifying the card (See \ref control_cards_id).
 * \param mode Open mode (see #SND_CTL_NONBLOCK, #SND_CTL_ASYNC).
 *
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 1600) - [public] snd_ctl_open_conf

```
    return snd_ctl_open_conf(pctl, name, root, conf, mode);
}
#endif

/**
 * \brief Opens a sound card.
 *
 * \param ctlp Returned CTL handle.
 * \param name A string identifying the card (See \ref control_cards_id).
 * \param mode Open mode (see #SND_CTL_NONBLOCK, #SND_CTL_ASYNC).
 *
 * \return 0 on success otherwise a negative error code.
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 1645) - [public] snd_ctl_open_noupdate

```
    return snd_ctl_open_noupdate(ctlp, lconf, name, mode, 0);
}

/**
 * \brief Opens a fallback CTL
 * \param ctlp Returned CTL handle
 * \param root Configuration root
 * \param name ASCII identifier of the CTL handle used as fallback
 * \param orig_name The original ASCII name
 * \param mode Open mode (see #SND_CTL_NONBLOCK, #SND_CTL_ASYNC)
 * \return 0 on success otherwise a negative error code
 */
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 1664) - [public] free

```
    free((*ctlp)->name);
    (*ctlp)->name = orig_name ? strdup(orig_name) : NULL;
}
return err;
}

#ifndef DOC_HIDDEN
#define TYPE(v) [SND_CTL_ELEM_TYPE_##v] = #v
#define IFACE(v) [SND_CTL_ELEM_IFACE_##v] = #v
#define IFACE1(v, n) [SND_CTL_ELEM_IFACE_##v] = #n
#define EVENT(v) [SND_CTL_EVENT_##v] = #v

// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 1882) - [public] sizeof

```
    return sizeof(snd_ctl_elem_id_t);
}

/**
 * \brief allocate an invalid #snd_ctl_elem_id_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_ctl_elem_id_malloc(snd_ctl_elem_id_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_ctl_elem_id_t));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control.c (line 2131) - [public] sizeof

```
    return sizeof(snd_ctl_card_info_t);
}

/**
 * \brief Allocate an invalid #snd_ctl_card_info_t on the heap.
 *
 * Allocate space for a card info object on the heap. The allocated memory
 * must be freed using snd_ctl_card_info_free().
 *
 * See snd_ctl_card_info_t for details.
 *
 * \param ptr Pointer to a snd_ctl_card_info_t pointer. The address
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 2294) - [public] sizeof

```
    return sizeof(snd_ctl_event_t);
}

/**
 * \brief allocate an invalid #snd_ctl_event_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_ctl_event_malloc(snd_ctl_event_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_ctl_event_t));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control.c (line 2357) - [public] sizeof

```
    return sizeof(snd_ctl_elem_list_t);
}

/**
 * \brief allocate a #snd_ctl_elem_list_t using standard malloc.
 *
 * The memory can be released using snd_ctl_elem_list_free().
 *
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_ctl_elem_list_malloc(snd_ctl_elem_list_t **ptr)
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control.c (line 2569) - [public] sizeof

```
    return sizeof(snd_ctl_elem_info_t);
}

/**
 * \brief allocate an invalid #snd_ctl_elem_info_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_ctl_elem_info_malloc(snd_ctl_elem_info_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_ctl_elem_info_t));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control.c (line 3171) - [public] sizeof

```
    return sizeof(snd_ctl_elem_value_t);
}

/**
 * \brief Allocate an invalid #snd_ctl_elem_value_t on the heap.
 *
 * Allocate space for a value object on the heap. The allocated memory
 * must be freed using snd_ctl_elem_value_free().
 *
 * See snd_ctl_elem_value_t for details.
 *
 * \param ptr Pointer to a snd_ctl_elem_value_t pointer. The address
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control.c (line 3241) - [public] memcmp

```
    return memcmp(left, right, sizeof(*left));
}

/**
 * \brief Get the element identifier from the given element value.
 *
 * See snd_ctl_elem_value_t for more details.
 *
 * \param obj The element value.
 * \param ptr Pointer to an identifier object. The identifier is
 * stored there.
 */
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control_empty.c (line 86) - [public] if

```
    if (strcmp(id, "child") == 0) {
        child = n;
        continue;
    }
    snd_error(CONTROL, "Unknown field %s", id);
    return -EINVAL;
}
if (!child) {
    snd_error(CONTROL, "child is not defined");
    return -EINVAL;
}
return _snd_ctl_open_named_child(handlep, name, root, child, mode, conf);
// ... (truncated, see source file for more)

```


dirtycode/alsa-lib/src/control/control_empty.c (line 90) - [public] snd_error

```
    ■■snd_error(CONTROL, "Unknown field %s", id);
    ■■return -EINVAL;
    ■}
    ■if (!child) {
    ■■snd_error(CONTROL, "child is not defined");
    ■■return -EINVAL;
    ■}
    ■return _snd_ctl_open_named_child(handlep, name, root, child, mode, conf);
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(_snd_ctl_empty_open, SND_CONTROL_DLSYM_VERSION);
    #endif
```

dirtycode/alsa-lib/src/control/control_empty.c (line 94) - [public] snd_error

```
    ■■snd_error(CONTROL, "child is not defined");
    ■■return -EINVAL;
    ■}
    ■return _snd_ctl_open_named_child(handlep, name, root, child, mode, conf);
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(_snd_ctl_empty_open, SND_CONTROL_DLSYM_VERSION);
    #endif
```

dirtycode/alsa-lib/src/control/control_empty.c (line 97) - [public] _snd_ctl_open_named_child

```
    ■return _snd_ctl_open_named_child(handlep, name, root, child, mode, conf);
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(_snd_ctl_empty_open, SND_CONTROL_DLSYM_VERSION);
    #endif
```

dirtycode/alsa-lib/src/control/control_ext.c (line 106) - [public] snd_ctl_elem_id_clear

```
    ■■snd_ctl_elem_id_clear(ids);
    ■■ret = ext->callback->elem_list(ext, offset, ids);
    ■■if (ret < 0)
    ■■return ret;
    ■■ids->numid = offset + 1; /* fake number */
    ■■list->used++;
    ■■offset++;
    ■■ids++;
    ■}
    ■return 0;
    }
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/control_ext.c (line 176) - [public] sizeof

```
    ■■■■■■ sizeof(info->value.enumerated.name));
    ■■break;
    ■default:
    ■■ret = 0;
    ■■break;
    ■}
```

```
    err:
    ■if (ext->callback->free_key)
    ■■ext->callback->free_key(ext, key);

    ■return ret;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_ext.c (line 244) - [public] sizeof

```
        sizeof(control->value.bytes.data));
        break;
        case SND_CTL_ELEM_TYPE_IEC958:
            if (! ext->callback->read_iec958)
                goto err;
            ret = ext->callback->read_iec958(ext, key, (snd_aes_iec958_t *)&control->value.iec958);
            break;
        default:
            break;
    }

    err:
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_ext.c (line 297) - [public] sizeof

```
        sizeof(control->value.bytes.data));
        break;
        case SND_CTL_ELEM_TYPE_IEC958:
            if (! ext->callback->write_iec958)
                goto err;
            ret = ext->callback->write_iec958(ext, key, (snd_aes_iec958_t *)&control->value.iec958);
            break;
        default:
            break;
    }

    err:
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_ext.c (line 367) - [public] memcpy

```
        memcpy(tlv, ext->tlv.p, len);
        return 0;
    }
}

static int snd_ctl_ext_next_device(snd_ctl_t *handle ATTRIBUTE_UNUSED,
        int *device ATTRIBUTE_UNUSED)
{
    return -ENXIO;
}

static int snd_ctl_ext_prefer_subdevice(snd_ctl_t *handle ATTRIBUTE_UNUSED,
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_ext.c (line 419) - [public] memset

```
        memset(event, 0, sizeof(*event));
        return ext->callback->read_event(ext, &event->data.elem.id, &event->data.elem.mask);
    }

    return -EINVAL;
}

static int snd_ctl_ext_poll_descriptors_count(snd_ctl_t *handle)
{
    snd_ctl_ext_t *ext = handle->private_data;

    if (ext->callback->poll_descriptors_count)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_ext.c (line 559) - [public] if

```
    if (strcmp(id, "my_own_parameter") == 0) {
        ....
        continue;
    }
    snd_error(CONTROL, "Unknown field %s", id);
    return -EINVAL;
}
```

```
myctl = calloc(1, sizeof(*myctl));
if (myctl == NULL)
    return -ENOMEM;
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/control_ext.c (line 563) - [public] snd_error

```
snd_error(CONTROL, "Unknown field %s", id);
return -EINVAL;
}
```

```
myctl = calloc(1, sizeof(*myctl));
if (myctl == NULL)
    return -ENOMEM;
```

```
myctl->ext.version = SND_CTL_EXT_VERSION;
myctl->ext.card_idx = 0;
strcpy(myctl->ext.id, "Myctl");
strcpy(myctl->ext.name, "My Control");
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_ext.c (line 583) - [public] myctl_free

```
myctl_free(myctl);
return err;
}
```

```
*phandle = myctl->ext.handle;
return 0;
}
```

```
SND_CTL_PLUGIN_SYMBOL(myctl);
\endcode
```

Read the codes in alsa-plugins package for the real examples.

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/control_ext.c (line 715) - [public] snd_error

```
snd_error(CONTROL, "ctl_ext: Plugin version mismatch");
return -ENXIO;
}
```

```
err = snd_ctl_new(&ctl, SND_CTL_TYPE_EXT, name, mode);
if (err < 0)
    return err;
```

```
ext->handle = ctl;
```

```
ctl->ops = &snd_ctl_ext_ops;
ctl->private_data = ext;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_ext.c (line 741) - [public] snd_ctl_close

```
return snd_ctl_close(ext->handle);
}
```

dirtycode/alsa-lib/src/control/control_hw.c (line 74) - [public] snd_errornum

```
    ■■snd_errornum(CONTROL, "F_GETFL failed");
    ■■return -errno;
    ■}
    ■if (nonblock)
    ■■flags |= O_NONBLOCK;
    ■else
    ■■flags &= ~O_NONBLOCK;
    ■if (fcntl(fd, F_SETFL, flags) < 0) {
    ■■snd_errornum(CONTROL, "F_SETFL for O_NONBLOCK failed");
    ■■return -errno;
    ■}
    ■return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 82) - [public] snd_errornum

```
    ■■snd_errornum(CONTROL, "F_SETFL for O_NONBLOCK failed");
    ■■return -errno;
    ■}
    ■return 0;
    }

    static int snd_ctl_hw_async(snd_ctl_t *ctl, int sig, pid_t pid)
    {
    ■long flags;
    ■snd_ctl_hw_t *hw = ctl->private_data;
    ■int fd = hw->fd;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 95) - [public] snd_errornum

```
    ■■snd_errornum(CONTROL, "F_GETFL failed");
    ■■return -errno;
    ■}
    ■if (sig >= 0)
    ■■flags |= O_ASYNC;
    ■else
    ■■flags &= ~O_ASYNC;
    ■if (fcntl(fd, F_SETFL, flags) < 0) {
    ■■snd_errornum(CONTROL, "F_SETFL for O_ASYNC failed");
    ■■return -errno;
    ■}
    ■if (sig < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 103) - [public] snd_errornum

```
    ■■snd_errornum(CONTROL, "F_SETFL for O_ASYNC failed");
    ■■return -errno;
    ■}
    ■if (sig < 0)
    ■■return 0;
    ■if (fcntl(fd, F_SETSIG, (long)sig) < 0) {
    ■■snd_errornum(CONTROL, "F_SETSIG failed");
    ■■return -errno;
    ■}
    ■if (fcntl(fd, F_SETOWN, (long)pid) < 0) {
    ■■snd_errornum(CONTROL, "F_SETOWN failed");
    ■■return -errno;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 109) - [public] snd_errornum

```
    ■■snd_errornum(CONTROL, "F_SETSIG failed");
    ■■return -errno;
    ■}
    ■if (fcntl(fd, F_SETOWN, (long)pid) < 0) {
    ■■snd_errornum(CONTROL, "F_SETOWN failed");
    ■■return -errno;
    ■}
    ■return 0;
    }
```

```
static int snd_ctl_hw_subscribe_events(snd_ctl_t *handle, int subscribe)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 113) - [public] snd_errornum

```
    ■■snd_errornum(CONTROL, "F_SETOWN failed");
    ■■return -errno;
    ■}
    ■return 0;
    }
```

```
static int snd_ctl_hw_subscribe_events(snd_ctl_t *handle, int subscribe)
{
    ■snd_ctl_hw_t *hw = handle->private_data;
    ■if (ioctl(hw->fd, SNDRV_CTL_IOCTL_SUBSCRIBE_EVENTS, &subscribe) < 0) {
    ■■snd_errornum(CONTROL, "SNDRV_CTL_IOCTL_SUBSCRIBE_EVENTS failed");
    ■■return -errno;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 123) - [public] snd_errornum

```
    ■■snd_errornum(CONTROL, "SNDRV_CTL_IOCTL_SUBSCRIBE_EVENTS failed");
    ■■return -errno;
    ■}
    ■return 0;
    }
```

```
static int snd_ctl_hw_card_info(snd_ctl_t *handle, snd_ctl_card_info_t *info)
{
    ■snd_ctl_hw_t *hw = handle->private_data;
    ■if (ioctl(hw->fd, SNDRV_CTL_IOCTL_CARD_INFO, info) < 0) {
    ■■snd_errornum(CONTROL, "SNDRV_CTL_IOCTL_CARD_INFO failed");
    ■■return -errno;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 133) - [public] snd_errornum

```
    ■■snd_errornum(CONTROL, "SNDRV_CTL_IOCTL_CARD_INFO failed");
    ■■return -errno;
    ■}
    ■return 0;
    }
```

```
static int snd_ctl_hw_elem_list(snd_ctl_t *handle, snd_ctl_elem_list_t *list)
{
    ■snd_ctl_hw_t *hw = handle->private_data;
    ■if (ioctl(hw->fd, SNDRV_CTL_IOCTL_ELEM_LIST, list) < 0)
    ■■return -errno;
    ■return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 246) - [public] free

```
    free(xtlv);
    return -errno;
}
if (op_flag == 0) {
    unsigned int size;
    size = xtlv->tlv[SNDRV_CTL_TLVO_LEN] + 2 * sizeof(unsigned int);
    if (size > tlv_size) {
        free(xtlv);
        return -EFAULT;
    }
    memcpy(tlv, xtlv->tlv, size);
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 252) - [public] if

```
    if (size > tlv_size) {
        free(xtlv);
        return -EFAULT;
    }
    memcpy(tlv, xtlv->tlv, size);
}
free(xtlv);
return 0;
}

static int snd_ctl_hw_hwdep_next_device(snd_ctl_t *handle, int * device)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 253) - [public] free

```
    free(xtlv);
    return -EFAULT;
}
memcpy(tlv, xtlv->tlv, size);
}
free(xtlv);
return 0;
}

static int snd_ctl_hw_hwdep_next_device(snd_ctl_t *handle, int * device)
{
    snd_ctl_hw_t *hw = handle->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 256) - [public] memcpy

```
    memcpy(tlv, xtlv->tlv, size);
}
free(xtlv);
return 0;
}

static int snd_ctl_hw_hwdep_next_device(snd_ctl_t *handle, int * device)
{
    snd_ctl_hw_t *hw = handle->private_data;
    if (ioctl(hw->fd, SNDRV_CTL_IOCTL_HWDEP_NEXT_DEVICE, device) < 0)
        return -errno;
    return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 293) - [public] __snd_pcm_info_eld_fixup

```
    return __snd_pcm_info_eld_fixup(info);
    return 0;
}

static int snd_ctl_hw_pcm_prefer_subdevice(snd_ctl_t *handle, int subdev)
{
    snd_ctl_hw_t *hw = handle->private_data;
    if (ioctl(hw->fd, SNDRV_CTL_IOCTL_PCM_PREFER_SUBDEVICE, &subdev) < 0)
        return -errno;
    return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 379) - [public] sizeof

```
    sizeof(*event), res);

    return -EINVAL;
}
    return 1;
}

static const snd_ctl_ops_t snd_ctl_hw_ops = {
    .close = snd_ctl_hw_close,
    .nonblock = snd_ctl_hw_nonblock,
    .async = snd_ctl_hw_async,
    .subscribe_events = snd_ctl_hw_subscribe_events,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 441) - [public] snd_check

```
    snd_check(CONTROL, "Invalid card index %d", card);
    return -EINVAL;
}
    sprintf(filename, SNDRV_FILE_CONTROL, card);
    if (mode & SND_CTL_READONLY)
        fmode = O_RDONLY;
    else
        fmode = O_RDWR;
    if (mode & SND_CTL_NONBLOCK)
        fmode |= O_NONBLOCK;
    if (mode & SND_CTL_ASYNC)
        fmode |= O_ASYNC;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 455) - [public] snd_card_load

```
    snd_card_load(card);
    fd = snd_open_device(filename, fmode);
    if (fd < 0)
        return -errno;
}
    if (ioctl(fd, SNDRV_CTL_IOCTL_PVERSION, &ver) < 0) {
        err = -errno;
        close(fd);
        return err;
    }
    if (SNDRV_PROTOCOL_INCOMPATIBLE(ver, SNDRV_CTL_VERSION_MAX)) {
        close(fd);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 462) - [public] close

```
    close(fd);
    return err;
}
if (SNDRV_PROTOCOL_INCOMPATIBLE(ver, SNDRV_CTL_VERSION_MAX)) {
    close(fd);
    return -SND_ERROR_INCOMPATIBLE_VERSION;
}
hw = calloc(1, sizeof(snd_ctl_hw_t));
if (hw == NULL) {
    close(fd);
    return -ENOMEM;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 466) - [public] close

```
    close(fd);
    return -SND_ERROR_INCOMPATIBLE_VERSION;
}
hw = calloc(1, sizeof(snd_ctl_hw_t));
if (hw == NULL) {
    close(fd);
    return -ENOMEM;
}
hw->card = card;
hw->fd = fd;
hw->protocol = ver;

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 471) - [public] close

```
    close(fd);
    return -ENOMEM;
}
hw->card = card;
hw->fd = fd;
hw->protocol = ver;

err = snd_ctl_new(&ctl, SND_CTL_TYPE_HW, name, mode);
if (err < 0) {
    close(fd);
    free(hw);
    return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 480) - [public] close

```
    close(fd);
    free(hw);
    return err;
}
ctl->ops = &snd_ctl_hw_ops;
ctl->private_data = hw;
ctl->poll_fd = fd;
*handle = ctl;
return 0;
}

/*! \page control_plugins
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/control_hw.c (line 481) - [public] free

```
    free(hw);
    return err;
}
ctl->ops = &snd_ctl_hw_ops;
ctl->private_data = hw;
ctl->poll_fd = fd;
*handle = ctl;
return 0;
}
```

/*! \page control_plugins

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/control_hw.c (line 537) - [public] if

```
    if (strcmp(id, "card") == 0) {
        err = snd_config_get_card(n);
        if (err < 0)
            return err;
        card = err;
        continue;
    }
    return -EINVAL;
}
if (card < 0)
    return -EINVAL;
return snd_ctl_hw_open(handlep, name, card, mode);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_hw.c (line 548) - [public] snd_ctl_hw_open

```
return snd_ctl_hw_open(handlep, name, card, mode);
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(_snd_ctl_hw_open, SND_CONTROL_DLSYM_VERSION);
#endif
```

dirtycode/alsa-lib/src/control/control_remap.c (line 132) - [public] remap_load_list

```
static int remap_load_list(snd_ctl_remap_t *priv);

static snd_ctl_numid_t *remap_numid_temp(snd_ctl_remap_t *priv, unsigned int numid)
{
    priv->numid_temp.numid_child = numid;
    priv->numid_temp.numid_app = numid;
    return &priv->numid_temp;
}

static snd_ctl_numid_t *remap_find_numid_app(snd_ctl_remap_t *priv, unsigned int numid_app)
{
    snd_ctl_numid_t *numid;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 149) - [public] remap_numid_temp

```
return remap_numid_temp(priv, numid_app);
numid = priv->numid;
for (count = priv->numid_items; count > 0; count--, numid++)
    if (numid_app == numid->numid_app)
        return numid;
return NULL;
}

static snd_ctl_numid_t *remap_numid_new(snd_ctl_remap_t *priv, unsigned int numid_child,
    unsigned int numid_app)
{
    snd_ctl_numid_t *numid;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 168) - [public] memset

```
■memset(numid + priv->numid_alloc, 0, sizeof(*numid) * 16);
■priv->numid_alloc += 16;
■priv->numid = numid;
■}
■numid = &priv->numid[priv->numid_items++];
■numid->numid_child = numid_child;
■numid->numid_app = numid_app;
■debug("new numid: child %u app %u\n", numid->numid_child, numid->numid_app);
■return numid;
}
```

```
static snd_ctl_numid_t *remap_numid_child_new(snd_ctl_remap_t *priv, unsigned int numid_child)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 192) - [public] remap_numid_new

```
■return remap_numid_new(priv, numid_child, numid_app);
}
```

```
static snd_ctl_numid_t *remap_find_numid_child(snd_ctl_remap_t *priv, unsigned int numid_child)
{
■snd_ctl_numid_t *numid;
■size_t count;

■if (numid_child == 0)
■return NULL;
■if (!priv->numid_remap_active)
■return remap_numid_temp(priv, numid_child);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 203) - [public] remap_numid_temp

```
■return remap_numid_temp(priv, numid_child);
■numid = priv->numid;
■for (count = priv->numid_items; count > 0; count--, numid++)
■if (numid_child == numid->numid_child)
■■return numid;
■return remap_numid_child_new(priv, numid_child);
}
```

```
static void remap_forget_numid_child(snd_ctl_remap_t *priv, unsigned int numid_child)
{
■snd_ctl_numid_t *numid;
■size_t index;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 208) - [public] remap_numid_child_new

```
■return remap_numid_child_new(priv, numid_child);
}
```

```
static void remap_forget_numid_child(snd_ctl_remap_t *priv, unsigned int numid_child)
{
■snd_ctl_numid_t *numid;
■size_t index;

■if (!priv->numid_remap_active)
■return;
■numid = priv->numid;
■for (index = 0; index < priv->numid_items; index++) {
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 285) - [public] remap_find_map_numid

```
return remap_find_map_numid(priv, id->numid);
map = priv->map;
for (count = priv->map_items; count > 0; count--, map++)
    if (snd_ctl_elem_id_compare_set(id, &map->map_id) == 0)
        return map;
return NULL;
}
```

```
static int remap_id_to_child(snd_ctl_remap_t *priv, snd_ctl_elem_id_t *id, snd_ctl_remap_id_t **_rid)
{
    snd_ctl_remap_id_t *rid;
    snd_ctl_numid_t *numid;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 304) - [public] if

```
if (rid->id_app.numid == 0) {
    numid = remap_find_numid_app(priv, id->numid);
    if (numid) {
        rid->id_child.numid = numid->numid_child;
        rid->id_app.numid = numid->numid_app;
    }
}
*id = rid->id_child;
} else {
    numid = remap_find_numid_app(priv, id->numid);
    if (numid) {
        id->numid = numid->numid_child;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 306) - [public] if

```
if (numid) {
    rid->id_child.numid = numid->numid_child;
    rid->id_app.numid = numid->numid_app;
}
}
*id = rid->id_child;
} else {
    numid = remap_find_numid_app(priv, id->numid);
    if (numid) {
        id->numid = numid->numid_child;
    } else {
        if (reloaded || priv->list_complete)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 314) - [public] if

```
if (numid) {
    id->numid = numid->numid_child;
} else {
    if (reloaded || priv->list_complete)
        return -ENOENT;
    /* build whole numid mapping */
    err = remap_load_list(priv);
    if (err < 0)
        return err;
    reloaded = true;
    goto _retry;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 337) - [public] if

```
    if (err >= 0 && rid->id_app.numid == 0) {
        numid = remap_numid_child_new(priv, id->numid);
        if (numid == NULL)
            return -EIO;
        rid->id_child.numid = numid->numid_child;
        rid->id_app.numid = numid->numid_app;
    }
    *id = rid->id_app;
} else {
    if (err >= 0) {
        numid = remap_find_numid_child(priv, id->numid);
        if (numid == NULL)
            // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 346) - [public] if

```
    if (err >= 0) {
        numid = remap_find_numid_child(priv, id->numid);
        if (numid == NULL)
            return -EIO;
        id->numid = numid->numid_app;
    }
}
debug_id(id, "%s rid %p\n", __func__, rid);
return err;
}
```

```
static snd_ctl_sync_t *remap_find_sync_numid(snd_ctl_remap_t *priv, unsigned int numid)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 378) - [public] remap_find_sync_numid

```
    return remap_find_sync_numid(priv, id->numid);
    sync = priv->sync;
    for (count = priv->sync_items; count > 0; count--, sync++)
        for (index2 = 0; index2 < sync->control_items; index2++)
            if (snd_ctl_elem_id_compare_set(id, &sync->control_ids[index2]) == 0)
                return sync;
    return NULL;
}
```

```
static void remap_update_sync_id(snd_ctl_remap_t *priv, snd_ctl_elem_id_t *id)
{
    size_t count, index2;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 397) - [public] if

```
    if (snd_ctl_elem_id_compare_set(id, &sync->control_ids[index2]) == 0) {
        sync->control_ids[index2].numid = id->numid;
        break;
    }
}
```

```
static snd_ctl_sync_t *remap_find_sync_switch_numid(snd_ctl_remap_t *priv, unsigned int numid)
{
    size_t count;
    snd_ctl_sync_t *sync;

    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 424) - [public]

remap_find_sync_switch_numid

```
    return remap_find_sync_switch_numid(priv, id->numid);
    sync = priv->sync;
    for (count = priv->sync_items; count > 0; count--, sync++) {
        if (sync->switch_id.numid == 0)
            continue;
        if (snd_ctl_elem_id_compare_set(id, &sync->switch_id) == 0)
            return sync;
    }
    return NULL;
}
```

```
static void remap_free(snd_ctl_remap_t *priv)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 443) - [public] free

```
    free(sync->control_ids);
}
for (idx1 = 0; idx1 < priv->map_items; idx1++) {
    map = &priv->map[idx1];
    for (idx2 = 0; idx2 < map->controls_items; idx2++)
        free(map->controls[idx2].channel_map);
    free(map->controls);
}
free(priv->event_queue);
free(priv->sync);
free(priv->map);
free(priv->remap);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 448) - [public] free

```
    free(map->controls[idx2].channel_map);
    free(map->controls);
}
free(priv->event_queue);
free(priv->sync);
free(priv->map);
free(priv->remap);
free(priv->numid);
free(priv);
}
```

```
static int snd_ctl_remap_close(snd_ctl_t *ctl)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 449) - [public] free

```
    free(map->controls);
}
free(priv->event_queue);
free(priv->sync);
free(priv->map);
free(priv->remap);
free(priv->numid);
free(priv);
}
```

```
static int snd_ctl_remap_close(snd_ctl_t *ctl)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 470) - [public] snd_ctl_nonblock

```
    return snd_ctl_nonblock(priv->child, nonblock);
}

static int snd_ctl_remap_async(snd_ctl_t *ctl, int sig, pid_t pid)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    return snd_ctl_async(priv->child, sig, pid);
}

static int snd_ctl_remap_subscribe_events(snd_ctl_t *ctl, int subscribe)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 476) - [public] snd_ctl_async

```
    return snd_ctl_async(priv->child, sig, pid);
}

static int snd_ctl_remap_subscribe_events(snd_ctl_t *ctl, int subscribe)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    return snd_ctl_subscribe_events(priv->child, subscribe);
}

static int snd_ctl_remap_card_info(snd_ctl_t *ctl, snd_ctl_card_info_t *info)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 482) - [public] snd_ctl_subscribe_events

```
    return snd_ctl_subscribe_events(priv->child, subscribe);
}

static int snd_ctl_remap_card_info(snd_ctl_t *ctl, snd_ctl_card_info_t *info)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    return snd_ctl_card_info(priv->child, info);
}

static int _snd_ctl_remap_elem_list(snd_ctl_remap_t *priv, snd_ctl_elem_list_t *list)
{
    snd_ctl_elem_id_t *id;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 488) - [public] snd_ctl_card_info

```
    return snd_ctl_card_info(priv->child, info);
}

static int _snd_ctl_remap_elem_list(snd_ctl_remap_t *priv, snd_ctl_elem_list_t *list)
{
    snd_ctl_elem_id_t *id;
    snd_ctl_remap_id_t *rid;
    snd_ctl_numid_t *numid;
    unsigned int index;
    size_t index2;
    int err;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 506) - [public] if

```
    if (rid) {
        assert(id->numid > 0);
        rid->id_child.numid = id->numid;
    }
    numid = remap_find_numid_child(priv, id->numid);
    if (numid == NULL)
        return -EIO;
    id->numid = numid->numid_app;
    if (rid) {
        rid->id_app.numid = id->numid;
        *id = rid->id_app;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 507) - [public] assert

```
    assert(id->numid > 0);
    rid->id_child.numid = id->numid;
}
numid = remap_find_numid_child(priv, id->numid);
if (numid == NULL)
    return -EIO;
id->numid = numid->numid_app;
if (rid) {
    rid->id_app.numid = id->numid;
    *id = rid->id_app;
}
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 514) - [public] if

```
    if (rid) {
        rid->id_app.numid = id->numid;
        *id = rid->id_app;
    }
}
if (list->offset >= list->count + priv->map_items + priv->sync_switch_items)
    return 0;
index2 = 0;
if (list->offset > list->count)
    index2 = list->offset - list->count;
for ( ; index < list->space && index2 < priv->map_items; index2++, index++) {
    snd_ctl_map_t *map = &priv->map[index2];
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 550) - [public] _snd_ctl_remap_elem_list

```
    return _snd_ctl_remap_elem_list(priv, list);
}

static int remap_load_list(snd_ctl_remap_t *remap)
{
    snd_ctl_elem_list_t list;
    int err = 0;

    memset(&list, 0, sizeof(list));
    do {
        err = _snd_ctl_remap_elem_list(remap, &list);
        if (err < 0)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 609) - [public] snd_ctl_elem_info_clear

```
■snd_ctl_elem_info_clear(&info3);
■info3.id = map->controls[item].id_child;
■debug_id(&info3.id, "%s controls[%zd]\n", __func__, item);
■err = snd_ctl_elem_info(priv->child, &info3);
■if (err < 0)
■return err;
■if (info2.type != info3.type)
■return -EIO;
■if (ACCESS_BITS(info2.access) != ACCESS_BITS(info3.access))
■return -EIO;
■if (info2.type == SNDRV_CTL_ELEM_TYPE_BOOLEAN ||
■    info2.type == SNDRV_CTL_ELEM_TYPE_INTEGER) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 611) - [public] debug_id

```
■debug_id(&info3.id, "%s controls[%zd]\n", __func__, item);
■err = snd_ctl_elem_info(priv->child, &info3);
■if (err < 0)
■return err;
■if (info2.type != info3.type)
■return -EIO;
■if (ACCESS_BITS(info2.access) != ACCESS_BITS(info3.access))
■return -EIO;
■if (info2.type == SNDRV_CTL_ELEM_TYPE_BOOLEAN ||
■    info2.type == SNDRV_CTL_ELEM_TYPE_INTEGER) {
■    if (memcmp(&info2.value.integer, &info3.value.integer, sizeof(info2.value.integer)))
■return -EIO;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 681) - [public] remap_update_sync_id

```
■remap_update_sync_id(priv, &info->id);
■return remap_id_to_app(priv, &info->id, rid, err);
}

static int remap_map_elem_read(snd_ctl_remap_t *priv, snd_ctl_elem_value_t *control)
{
■snd_ctl_map_t *map;
■struct snd_ctl_map_ctl *mctl;
■snd_ctl_elem_value_t control2;
■size_t item, index, src_index;
■int err;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 682) - [public] remap_id_to_app

```
■return remap_id_to_app(priv, &info->id, rid, err);
}

static int remap_map_elem_read(snd_ctl_remap_t *priv, snd_ctl_elem_value_t *control)
{
■snd_ctl_map_t *map;
■struct snd_ctl_map_ctl *mctl;
■snd_ctl_elem_value_t control2;
■size_t item, index, src_index;
■int err;

■map = remap_find_map_id(priv, &control->id);
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/control_remap.c (line 701) - [public] snd_ctl_elem_value_clear

```
    ■■snd_ctl_elem_value_clear(&control2);
    ■■control2.id = mctl->id_child;
    ■■debug_id(&control2.id, "%s controls[%zd]\n", __func__, item);
    ■■err = snd_ctl_elem_read(priv->child, &control2);
    ■■if (err < 0)
    ■■■return err;
    ■■if (map->type == SNDRV_CTL_ELEM_TYPE_BOOLEAN ||
    ■■    map->type == SNDRV_CTL_ELEM_TYPE_INTEGER) {
    ■■■for (index = 0; index < mctl->channel_map_items; index++) {
    ■■■■long base_idx = mctl->src_channels * index;
    ■■■■long src = mctl->channel_map[base_idx];
    ■■■■if ((unsigned long)src < ARRAY_SIZE(control->value.integer.value))
    ■■■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 703) - [public] debug_id

```
    ■■debug_id(&control2.id, "%s controls[%zd]\n", __func__, item);
    ■■err = snd_ctl_elem_read(priv->child, &control2);
    ■■if (err < 0)
    ■■■return err;
    ■■if (map->type == SNDRV_CTL_ELEM_TYPE_BOOLEAN ||
    ■■    map->type == SNDRV_CTL_ELEM_TYPE_INTEGER) {
    ■■■for (index = 0; index < mctl->channel_map_items; index++) {
    ■■■■long base_idx = mctl->src_channels * index;
    ■■■■long src = mctl->channel_map[base_idx];
    ■■■■if ((unsigned long)src < ARRAY_SIZE(control->value.integer.value))
    ■■■■    control->value.integer.value[index] = control2.value.integer.value[src];
    ■■■■for (src_index = 1; src_index < mctl->src_channels; src_index++) {
    ■■■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 776) - [public] remap_id_to_app

```
    ■return remap_id_to_app(priv, &control->id, rid, err);
}

static int remap_map_elem_write(snd_ctl_remap_t *priv, snd_ctl_elem_value_t *control)
{
    ■snd_ctl_map_t *map;
    ■struct snd_ctl_map_ctl *mctl;
    ■snd_ctl_elem_value_t control2;
    ■size_t item, index, src_index;
    ■int err, changes;

    ■map = remap_find_map_id(priv, &control->id);
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 794) - [public] snd_ctl_elem_value_clear

```
    ■■snd_ctl_elem_value_clear(&control2);
    ■■control2.id = mctl->id_child;
    ■■debug_id(&control2.id, "%s controls[%zd]\n", __func__, item);
    ■■err = snd_ctl_elem_read(priv->child, &control2);
    ■■if (err < 0)
    ■■■return err;
    ■■changes = 0;
    ■■if (map->type == SNDRV_CTL_ELEM_TYPE_BOOLEAN ||
    ■■    map->type == SNDRV_CTL_ELEM_TYPE_INTEGER) {
    ■■■for (index = 0; index < mctl->channel_map_items; index++) {
    ■■■■long base_idx = mctl->src_channels * index;
    ■■■■long dst = mctl->channel_map[base_idx];
    ■■■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 796) - [public] debug_id

```
    debug_id(&control2.id, "%s controls[%zd]\n", __func__, item);
    err = snd_ctl_elem_read(priv->child, &control2);
    if (err < 0)
        return err;
    changes = 0;
    if (map->type == SNDRV_CTL_ELEM_TYPE_BOOLEAN ||
        map->type == SNDRV_CTL_ELEM_TYPE_INTEGER) {
        for (index = 0; index < mctl->channel_map_items; index++) {
            long base_idx = mctl->src_channels * index;
            long dst = mctl->channel_map[base_idx];
            if ((unsigned long)dst < ARRAY_SIZE(control->value.integer.value)) {
                changes |= control2.value.integer.value[dst] != control->value.integer.value[index];
                // ... (truncated, see source file for more)
            }
        }
    }
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 806) - [public] if

```
    if ((unsigned long)dst < ARRAY_SIZE(control->value.integer.value)) {
        changes |= control2.value.integer.value[dst] != control->value.integer.value[index];
        control2.value.integer.value[dst] = control->value.integer.value[index];
    }
    for (src_index = 1; src_index < mctl->src_channels; src_index++) {
        dst = mctl->channel_map[base_idx + src_index];
        if ((unsigned long)dst < ARRAY_SIZE(control->value.integer.value)) {
            changes |= control2.value.integer.value[dst] != control->value.integer.value[index];
            control2.value.integer.value[dst] = control->value.integer.value[index];
        }
    }
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 812) - [public] if

```
    if ((unsigned long)dst < ARRAY_SIZE(control->value.integer.value)) {
        changes |= control2.value.integer.value[dst] != control->value.integer.value[index];
        control2.value.integer.value[dst] = control->value.integer.value[index];
    }
}
} else if (map->type == SNDRV_CTL_ELEM_TYPE_INTEGER64) {
    for (index = 0; index < mctl->channel_map_items; index++) {
        long base_idx = mctl->src_channels * index;
        long dst = mctl->channel_map[base_idx];
        if ((unsigned long)dst < ARRAY_SIZE(control->value.integer64.value)) {
            changes |= control2.value.integer64.value[dst] != control->value.integer64.value[index];
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 822) - [public] if

```
    if ((unsigned long)dst < ARRAY_SIZE(control->value.integer64.value)) {
        changes |= control2.value.integer64.value[dst] != control->value.integer64.value[index];
        control2.value.integer64.value[dst] = control->value.integer64.value[index];
    }
    for (src_index = 1; src_index < mctl->src_channels; src_index++) {
        dst = mctl->channel_map[base_idx + src_index];
        if ((unsigned long)dst < ARRAY_SIZE(control->value.integer64.value)) {
            changes |= control2.value.integer64.value[dst] != control->value.integer64.value[index];
            control2.value.integer64.value[dst] = control->value.integer64.value[index];
        }
    }
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 828) - [public] if

```
        if ((unsigned long)dst < ARRAY_SIZE(control->value.integer64.value)) {
            changes |= control2.value.integer64.value[dst] != control->value.integer64.value[index];
            control2.value.integer64.value[dst] = control->value.integer64.value[index];
        }
    }
} else if (map->type == SNDRV_CTL_ELEM_TYPE_BYTES) {
    /* does it make sense to merge bytes? */
    if (mctl->src_channels > 1)
        return -EINVAL;
    for (index = 0; index < mctl->channel_map_items; index++) {
        long dst = mctl->channel_map[index];
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 840) - [public] if

```
        if ((unsigned long)dst < ARRAY_SIZE(control->value.bytes.data)) {
            changes |= control2.value.bytes.data[dst] != control->value.bytes.data[index];
            control2.value.bytes.data[dst] = control->value.bytes.data[index];
        }
    }
}
debug_id(&control2.id, "%s changes %d\n", __func__, changes);
if (changes > 0) {
    err = snd_ctl_elem_write(priv->child, &control2);
    if (err < 0)
        return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 846) - [public] debug_id

```
debug_id(&control2.id, "%s changes %d\n", __func__, changes);
if (changes > 0) {
    err = snd_ctl_elem_write(priv->child, &control2);
    if (err < 0)
        return err;
}
return 0;
}
```

```
static int remap_sync_elem_write(snd_ctl_remap_t *priv, snd_ctl_elem_value_t *control)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 847) - [public] if

```
if (changes > 0) {
    err = snd_ctl_elem_write(priv->child, &control2);
    if (err < 0)
        return err;
}
return 0;
}
```

```
static int remap_sync_elem_write(snd_ctl_remap_t *priv, snd_ctl_elem_value_t *control)
{
    snd_ctl_sync_t *sync;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 878) - [public] debug_id

```
■debug_id(&control2.id, "%s sync[%zd]\n", __func__, item);
■/* TODO: it's a blind write - no checks if the values are in range for all controls */
■err = snd_ctl_elem_write(priv->child, &control2);
■if (err < 0)
■return err;
■}
■return remap_id_to_app(priv, &control->id, NULL, 0);
}
```

```
static int snd_ctl_remap_elem_write(snd_ctl_t *ctl, snd_ctl_elem_value_t *control)
{
■snd_ctl_remap_t *priv = ctl->private_data;
  // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 884) - [public] remap_id_to_app

```
■return remap_id_to_app(priv, &control->id, NULL, 0);
}
```

```
static int snd_ctl_remap_elem_write(snd_ctl_t *ctl, snd_ctl_elem_value_t *control)
{
■snd_ctl_remap_t *priv = ctl->private_data;
■snd_ctl_remap_id_t *rid;
■int err;

■debug_id(&control->id, "%s\n", __func__);
■err = remap_map_elem_write(priv, control);
■if (err != -EREMAPNOTFOUND)
  // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 904) - [public] remap_id_to_app

```
■return remap_id_to_app(priv, &control->id, rid, err);
}
```

```
static int snd_ctl_remap_elem_lock(snd_ctl_t *ctl, snd_ctl_elem_id_t *id)
{
■snd_ctl_remap_t *priv = ctl->private_data;
■snd_ctl_remap_id_t *rid;
■int err;

■debug_id(id, "%s\n", __func__);
■err = remap_id_to_child(priv, id, &rid);
■if (err < 0)
  // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 918) - [public] remap_id_to_app

```
■return remap_id_to_app(priv, id, rid, err);
}
```

```
static int snd_ctl_remap_elem_unlock(snd_ctl_t *ctl, snd_ctl_elem_id_t *id)
{
■snd_ctl_remap_t *priv = ctl->private_data;
■snd_ctl_remap_id_t *rid;
■int err;

■debug_id(id, "%s\n", __func__);
■err = remap_id_to_child(priv, id, &rid);
■if (err < 0)
  // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 932) - [public] remap_id_to_app

```
    return remap_id_to_app(priv, id, rid, err);
}

static int remap_get_map_numid(snd_ctl_remap_t *priv, struct snd_ctl_map_ctl *mctl)
{
    struct snd_ctl_elem_info_t info;
    struct snd_ctl_numid_t *numid;
    int err;

    if (mctl->id_child.numid > 0)
        return 0;
    debug_id(&mctl->id_child, "%s get numid\n", __func__);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 985) - [public] if

```
    if (err < 0) {
        free(tlv2);
        return err;
    }
    memset(tlv2, 0, tlv_size);
    err = priv->child->ops->element_tlv(priv->child, op_flag, mctl->id_child.numid, tlv2, tlv_size);
    if (err < 0) {
        free(tlv2);
        return err;
    }
    if (memcmp(tlv, tlv2, tlv_size) != 0) {
        free(tlv2);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 986) - [public] free

```
    free(tlv2);
    return err;
}
memset(tlv2, 0, tlv_size);
err = priv->child->ops->element_tlv(priv->child, op_flag, mctl->id_child.numid, tlv2, tlv_size);
if (err < 0) {
    free(tlv2);
    return err;
}
if (memcmp(tlv, tlv2, tlv_size) != 0) {
    free(tlv2);
    return -EIO;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 989) - [public] memset

```
    memset(tlv2, 0, tlv_size);
    err = priv->child->ops->element_tlv(priv->child, op_flag, mctl->id_child.numid, tlv2, tlv_size);
    if (err < 0) {
        free(tlv2);
        return err;
    }
    if (memcmp(tlv, tlv2, tlv_size) != 0) {
        free(tlv2);
        return -EIO;
    }
    free(tlv2);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 991) - [public] if

```
    if (err < 0) {
        free(tlv2);
        return err;
    }
    if (memcmp(tlv, tlv2, tlv_size) != 0) {
        free(tlv2);
        return -EIO;
    }
    free(tlv2);
    return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 992) - [public] free

```
    free(tlv2);
    return err;
}
if (memcmp(tlv, tlv2, tlv_size) != 0) {
    free(tlv2);
    return -EIO;
}
free(tlv2);
return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 995) - [public] if

```
    if (memcmp(tlv, tlv2, tlv_size) != 0) {
        free(tlv2);
        return -EIO;
    }
    free(tlv2);
    return 0;
}

static int snd_ctl_remap_elem_tlv(snd_ctl_t *ctl, int op_flag,
    unsigned int numid,
    unsigned int *tlv, unsigned int tlv_size)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 996) - [public] free

```
    free(tlv2);
    return -EIO;
}
free(tlv2);
return 0;
}

static int snd_ctl_remap_elem_tlv(snd_ctl_t *ctl, int op_flag,
    unsigned int numid,
    unsigned int *tlv, unsigned int tlv_size)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1025) - [public]

snd_ctl_hwdep_next_device

```
    return snd_ctl_hwdep_next_device(priv->child, device);
}

static int snd_ctl_remap_hwdep_info(snd_ctl_t *ctl, snd_hwdep_info_t * info)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    return snd_ctl_hwdep_info(priv->child, info);
}

static int snd_ctl_remap_pcm_next_device(snd_ctl_t *ctl, int * device)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1031) - [public] snd_ctl_hwdep_info

```
    return snd_ctl_hwdep_info(priv->child, info);
}

static int snd_ctl_remap_pcm_next_device(snd_ctl_t *ctl, int * device)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    return snd_ctl_pcm_next_device(priv->child, device);
}

static int snd_ctl_remap_pcm_info(snd_ctl_t *ctl, snd_pcm_info_t * info)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1037) - [public] snd_ctl_pcm_next_device

```
    return snd_ctl_pcm_next_device(priv->child, device);
}

static int snd_ctl_remap_pcm_info(snd_ctl_t *ctl, snd_pcm_info_t * info)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    return snd_ctl_pcm_info(priv->child, info);
}

static int snd_ctl_remap_pcm_prefer_subdevice(snd_ctl_t *ctl, int subdev)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1043) - [public] snd_ctl_pcm_info

```
    return snd_ctl_pcm_info(priv->child, info);
}

static int snd_ctl_remap_pcm_prefer_subdevice(snd_ctl_t *ctl, int subdev)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    return snd_ctl_pcm_prefer_subdevice(priv->child, subdev);
}

static int snd_ctl_remap_rawmidi_next_device(snd_ctl_t *ctl, int * device)
{
    snd_ctl_remap_t *priv = ctl->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1049) - [public]

snd_ctl_pcm_prefer_subdevice

```
    return snd_ctl_pcm_prefer_subdevice(priv->child, subdev);
}
```

```
static int snd_ctl_remap_rawmidi_next_device(snd_ctl_t *ctl, int * device)
{
    return snd_ctl_remap_rawmidi_next_device(priv->child, device);
}
```

```
static int snd_ctl_remap_rawmidi_info(snd_ctl_t *ctl, snd_rawmidi_info_t * info)
{
    return snd_ctl_remap_rawmidi_info(priv->child, info);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1055) - [public]

snd_ctl_rawmidi_next_device

```
    return snd_ctl_rawmidi_next_device(priv->child, device);
}
```

```
static int snd_ctl_remap_rawmidi_info(snd_ctl_t *ctl, snd_rawmidi_info_t * info)
{
    return snd_ctl_remap_rawmidi_info(priv->child, info);
}
```

```
static int snd_ctl_remap_rawmidi_prefer_subdevice(snd_ctl_t *ctl, int subdev)
{
    return snd_ctl_remap_rawmidi_prefer_subdevice(priv->child, subdev);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1061) - [public] snd_ctl_rawmidi_info

```
    return snd_ctl_rawmidi_info(priv->child, info);
}
```

```
static int snd_ctl_remap_rawmidi_prefer_subdevice(snd_ctl_t *ctl, int subdev)
{
    return snd_ctl_remap_rawmidi_prefer_subdevice(priv->child, subdev);
}
```

```
static int snd_ctl_remap_set_power_state(snd_ctl_t *ctl, unsigned int state)
{
    return snd_ctl_remap_set_power_state(priv->child, state);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1067) - [public]

snd_ctl_rawmidi_prefer_subdevice

```
    return snd_ctl_rawmidi_prefer_subdevice(priv->child, subdev);
}
```

```
static int snd_ctl_remap_set_power_state(snd_ctl_t *ctl, unsigned int state)
{
    return snd_ctl_remap_set_power_state(priv->child, state);
}
```

```
static int snd_ctl_remap_get_power_state(snd_ctl_t *ctl, unsigned int *state)
{
    return snd_ctl_remap_get_power_state(priv->child, state);
    // ... (truncated, see source file for more)
}
```



```

dirtycode/alsa-lib/src/control/control_remap.c (line 1073) - [public] snd_ctl_set_power_state
    ■return snd_ctl_set_power_state(priv->child, state);
    }

    static int snd_ctl_remap_get_power_state(snd_ctl_t *ctl, unsigned int *state)
    {
        ■snd_ctl_remap_t *priv = ctl->private_data;
        ■return snd_ctl_get_power_state(priv->child, state);
    }

    static void _next_ptr(size_t *ptr, size_t count)
    {
        ■*ptr = (*ptr + 1) % count;
        // ... (truncated, see source file for more)
    }

dirtycode/alsa-lib/src/control/control_remap.c (line 1079) - [public] snd_ctl_get_power_state
    ■return snd_ctl_get_power_state(priv->child, state);
    }

    static void _next_ptr(size_t *ptr, size_t count)
    {
        ■*ptr = (*ptr + 1) % count;
    }

    static void event_add(snd_ctl_remap_t *priv, snd_ctl_elem_id_t *id,
        ■■ unsigned int numid_app, unsigned int event_mask)
    {
        ■snd_ctl_map_event_t *map_event;
        // ... (truncated, see source file for more)
    }

dirtycode/alsa-lib/src/control/control_remap.c (line 1097) - [public] if
    ■■if (priv->event_queue[head].numid_app == numid_app) {
        ■■■found = 1;
        ■■■priv->event_queue[head].event_mask |= event_mask;
        ■■■break;
        ■■}
    ■debug_id(id, "%s marking for read (already %d)\n", __func__, found);
    ■if (found)
        ■■return;
    ■map_event = &priv->event_queue[priv->event_queue_tail];
    ■map_event->id = id;
    ■map_event->numid_app = numid_app;
    ■map_event->event_mask = event_mask;
    // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/control_remap.c (line 1128) - [public] if
    ■■■if (mctl->id_child.numid == 0) {
        ■■■■if (snd_ctl_elem_id_compare_set(id, &mctl->id_child))
            ■■■■continue;
        ■■■■mctl->id_child.numid = id->numid;
        ■■■}
    ■■if (id->numid != mctl->id_child.numid)
        ■■■■continue;
    ■■debug_id(&map->map_id, "%s found (all)\n", __func__);
    ■■changed = 1;
    ■■}
    ■■if (changed)
        ■■■event_add(priv, &map->map_id, map->map_id.numid, event_mask);
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/control_remap.c (line 1135) - [public] debug_id

```
    debug_id(&map->map_id, "%s found (all)\n", __func__);
    changed = 1;
}
if (changed)
    event_add(priv, &map->map_id, map->map_id.numid, event_mask);
}

static void remap_event_for_all_sync_controls(snd_ctl_remap_t *priv,
    snd_ctl_elem_id_t *id,
    unsigned int event_mask)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1139) - [public] event_add

```
    event_add(priv, &map->map_id, map->map_id.numid, event_mask);
}

static void remap_event_for_all_sync_controls(snd_ctl_remap_t *priv,
    snd_ctl_elem_id_t *id,
    unsigned int event_mask)
{
    size_t count, index;
    snd_ctl_sync_t *sync;
    snd_ctl_numid_t *numid;
    snd_ctl_elem_id_t *sid;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1160) - [public] if

```
    if (sid->numid == 0) {
        if (snd_ctl_elem_id_compare_set(id, sid))
            continue;
        sid->numid = id->numid;
    }
    if (id->numid != sid->numid)
        continue;
    debug_id(sid, "%s found (all)\n", __func__);
    changed = 1;
    break;
}
if (!changed)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1167) - [public] debug_id

```
    debug_id(sid, "%s found (all)\n", __func__);
    changed = 1;
    break;
}
if (!changed)
    continue;
for (index = 0; index < sync->control_items; index++) {
    sid = &sync->control_ids[index];
    /* skip double updates */
    if (sid->numid == id->numid)
        continue;
    numid = remap_find_numid_child(priv, sid->numid);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1180) - [public] event_add

```
■■■■event_add(priv, sid, numid->numid_app, event_mask);
■■}
■}
}

static int snd_ctl_remap_read(snd_ctl_t *ctl, snd_ctl_event_t *event)
{
    ■■■■snd_ctl_remap_t *priv = ctl->private_data;
    ■■■■snd_ctl_remap_id_t *rid;
    ■■■■snd_ctl_numid_t *numid;
    ■■■■snd_ctl_map_event_t *map_event;
    ■■■■unsigned int numid_child;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1196) - [public] _next_ptr

```
■■_next_ptr(&priv->event_queue_head, priv->event_items);
■■memset(event, 0, sizeof(*event));
■■event->type = SNDRV_CTL_EVENT_ELEM;
■■event->data.elem.mask = map_event->event_mask;
■■event->data.elem.id = *map_event->id;
■■event->data.elem.id.numid = map_event->numid_app;
■■debug_id(&event->data.elem.id, "%s queue read\n", __func__);
■■return 1;
■}
■err = snd_ctl_read(priv->child, event);
■if (err < 0 || event->type != SNDRV_CTL_EVENT_ELEM)
■■return err;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1197) - [public] memset

```
■■memset(event, 0, sizeof(*event));
■■event->type = SNDRV_CTL_EVENT_ELEM;
■■event->data.elem.mask = map_event->event_mask;
■■event->data.elem.id = *map_event->id;
■■event->data.elem.id.numid = map_event->numid_app;
■■debug_id(&event->data.elem.id, "%s queue read\n", __func__);
■■return 1;
■}
■err = snd_ctl_read(priv->child, event);
■if (err < 0 || event->type != SNDRV_CTL_EVENT_ELEM)
■■return err;
■if (event->data.elem.mask == SNDRV_CTL_EVENT_MASK_REMOVE ||
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1202) - [public] debug_id

```
■■debug_id(&event->data.elem.id, "%s queue read\n", __func__);
■■return 1;
■}
■err = snd_ctl_read(priv->child, event);
■if (err < 0 || event->type != SNDRV_CTL_EVENT_ELEM)
■■return err;
■if (event->data.elem.mask == SNDRV_CTL_EVENT_MASK_REMOVE ||
    (event->data.elem.mask & (SNDRV_CTL_EVENT_MASK_VALUE | SNDRV_CTL_EVENT_MASK_INFO |
    ■■■■SNDRV_CTL_EVENT_MASK_ADD | SNDRV_CTL_EVENT_MASK_TLV)) != 0) {
■■debug_id(&event->data.elem.id, "%s event mask 0x%x\n", __func__, event->data.elem.mask);
■■numid_child = event->data.elem.id.numid;
■■remap_event_for_all_map_controls(priv, &event->data.elem.id, event->data.elem.mask);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1211) - [public] debug_id

```
    debug_id(&event->data.elem.id, "%s event mask 0x%x\n", __func__, event->data.elem.mask);
    numid_child = event->data.elem.id.numid;
    remap_event_for_all_map_controls(priv, &event->data.elem.id, event->data.elem.mask);
    remap_event_for_all_sync_controls(priv, &event->data.elem.id, event->data.elem.mask);
    rid = remap_find_id_child(priv, &event->data.elem.id);
    if (rid) {
        if (rid->id_child.numid == 0) {
            numid = remap_find_numid_child(priv, numid_child);
            if (numid == NULL)
                return -EIO;
            rid->id_child.numid = numid->numid_child;
            rid->id_app.numid = numid->numid_app;
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1213) - [public]

remap_event_for_all_map_controls

```
    remap_event_for_all_map_controls(priv, &event->data.elem.id, event->data.elem.mask);
    remap_event_for_all_sync_controls(priv, &event->data.elem.id, event->data.elem.mask);
    rid = remap_find_id_child(priv, &event->data.elem.id);
    if (rid) {
        if (rid->id_child.numid == 0) {
            numid = remap_find_numid_child(priv, numid_child);
            if (numid == NULL)
                return -EIO;
            rid->id_child.numid = numid->numid_child;
            rid->id_app.numid = numid->numid_app;
        }
        event->data.elem.id = rid->id_app;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1214) - [public]

remap_event_for_all_sync_controls

```
    remap_event_for_all_sync_controls(priv, &event->data.elem.id, event->data.elem.mask);
    rid = remap_find_id_child(priv, &event->data.elem.id);
    if (rid) {
        if (rid->id_child.numid == 0) {
            numid = remap_find_numid_child(priv, numid_child);
            if (numid == NULL)
                return -EIO;
            rid->id_child.numid = numid->numid_child;
            rid->id_app.numid = numid->numid_app;
        }
        event->data.elem.id = rid->id_app;
    } else {
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1216) - [public] if

```
    if (rid) {
        if (rid->id_child.numid == 0) {
            numid = remap_find_numid_child(priv, numid_child);
            if (numid == NULL)
                return -EIO;
            rid->id_child.numid = numid->numid_child;
            rid->id_app.numid = numid->numid_app;
        }
        event->data.elem.id = rid->id_app;
    } else {
        numid = remap_find_numid_child(priv, numid_child);
        if (numid == NULL)
            // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1217) - [public] if

```
    if (rid->id_child.numid == 0) {
        numid = remap_find_numid_child(priv, numid_child);
        if (numid == NULL)
            return -EIO;
        rid->id_child.numid = numid->numid_child;
        rid->id_app.numid = numid->numid_app;
    }
    event->data.elem.id = rid->id_app;
} else {
    numid = remap_find_numid_child(priv, numid_child);
    if (numid == NULL)
        return -EIO;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1232) - [public] remap_forget_numid_child

```
    remap_forget_numid_child(priv, numid_child);
}
return err;
}
```

```
static const snd_ctl_ops_t snd_ctl_remap_ops = {
    .close = snd_ctl_remap_close,
    .nonblock = snd_ctl_remap_nonblock,
    .async = snd_ctl_remap_async,
    .subscribe_events = snd_ctl_remap_subscribe_events,
    .card_info = snd_ctl_remap_card_info,
    .element_list = snd_ctl_remap_elem_list,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1274) - [public] memset

```
    memset(rid + priv->remap_alloc, 0, sizeof(*rid) * 16);
    priv->remap_alloc += 16;
    priv->remap = rid;
}
rid = &priv->remap[priv->remap_items++];
rid->id_child = *child;
rid->id_app = *app;
debug_id(&rid->id_child, "%s remap child\n", __func__);
debug_id(&rid->id_app, "%s remap app\n", __func__);
return 0;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/control_remap.c (line 1299) - [public] if

```
    if (snd_config_get_string(n, &str) < 0) {
        snd_error(CONTROL, "expected string with the target control id!");
        return -EINVAL;
    }
    snd_ctl_elem_id_clear(&app);
    err = snd_ctl_ascii_elem_id_parse(&app, str);
    if (err < 0) {
        snd_error(CONTROL, "unable to parse target id '%s'", str);
        return -EINVAL;
    }
    if (remap_find_id_app(priv, &app)) {
        snd_error(CONTROL, "duplicate target id '%s'", id);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1300) - [public] snd_error

```
    snd_error(CONTROL, "expected string with the target control id!");
    return -EINVAL;
}
snd_ctl_elem_id_clear(&app);
err = snd_ctl_ascii_elem_id_parse(&app, str);
if (err < 0) {
    snd_error(CONTROL, "unable to parse target id '%s'", str);
    return -EINVAL;
}
if (remap_find_id_app(priv, &app)) {
    snd_error(CONTROL, "duplicate target id '%s'", id);
    return -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1303) - [public] snd_ctl_elem_id_clear

```
    snd_ctl_elem_id_clear(&app);
    err = snd_ctl_ascii_elem_id_parse(&app, str);
    if (err < 0) {
        snd_error(CONTROL, "unable to parse target id '%s'", str);
        return -EINVAL;
    }
    if (remap_find_id_app(priv, &app)) {
        snd_error(CONTROL, "duplicate target id '%s'", id);
        return -EINVAL;
    }
    snd_ctl_elem_id_clear(&child);
    err = snd_ctl_ascii_elem_id_parse(&child, id);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1305) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "unable to parse target id '%s'", str);
        return -EINVAL;
    }
    if (remap_find_id_app(priv, &app)) {
        snd_error(CONTROL, "duplicate target id '%s'", id);
        return -EINVAL;
    }
    snd_ctl_elem_id_clear(&child);
    err = snd_ctl_ascii_elem_id_parse(&child, id);
    if (err < 0) {
        snd_error(CONTROL, "unable to parse source id '%s'", id);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1306) - [public] snd_error

```
    snd_error(CONTROL, "unable to parse target id '%s'", str);
    return -EINVAL;
}
if (remap_find_id_app(priv, &app)) {
    snd_error(CONTROL, "duplicate target id '%s'", id);
    return -EINVAL;
}
snd_ctl_elem_id_clear(&child);
err = snd_ctl_ascii_elem_id_parse(&child, id);
if (err < 0) {
    snd_error(CONTROL, "unable to parse source id '%s'", id);
    return -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1309) - [public] if

```
    if (remap_find_id_app(priv, &app)) {
        snd_error(CONTROL, "duplicate target id '%s'!", id);
        return -EINVAL;
    }
    snd_ctl_elem_id_clear(&child);
    err = snd_ctl_ascii_elem_id_parse(&child, id);
    if (err < 0) {
        snd_error(CONTROL, "unable to parse source id '%s'!", id);
        return -EINVAL;
    }
    if (remap_find_id_child(priv, &app)) {
        snd_error(CONTROL, "duplicate source id '%s'!", id);
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1310) - [public] snd_error

```
    snd_error(CONTROL, "duplicate target id '%s'!", id);
    return -EINVAL;
}
snd_ctl_elem_id_clear(&child);
err = snd_ctl_ascii_elem_id_parse(&child, id);
if (err < 0) {
    snd_error(CONTROL, "unable to parse source id '%s'!", id);
    return -EINVAL;
}
if (remap_find_id_child(priv, &app)) {
    snd_error(CONTROL, "duplicate source id '%s'!", id);
    return -EINVAL;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1313) - [public] snd_ctl_elem_id_clear

```
    snd_ctl_elem_id_clear(&child);
    err = snd_ctl_ascii_elem_id_parse(&child, id);
    if (err < 0) {
        snd_error(CONTROL, "unable to parse source id '%s'!", id);
        return -EINVAL;
    }
    if (remap_find_id_child(priv, &app)) {
        snd_error(CONTROL, "duplicate source id '%s'!", id);
        return -EINVAL;
    }
    err = add_to_remap(priv, &child, &app);
    if (err < 0)
        // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1315) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "unable to parse source id '%s'!", id);
        return -EINVAL;
    }
    if (remap_find_id_child(priv, &app)) {
        snd_error(CONTROL, "duplicate source id '%s'!", id);
        return -EINVAL;
    }
    err = add_to_remap(priv, &child, &app);
    if (err < 0)
        return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1316) - [public] snd_error

```
    ■■■snd_error(CONTROL, "unable to parse source id '%s'!", id);
    ■■■return -EINVAL;
    ■■}
    ■■if (remap_find_id_child(priv, &app)) {
    ■■■snd_error(CONTROL, "duplicate source id '%s'!", id);
    ■■■return -EINVAL;
    ■■}
    ■■err = add_to_remap(priv, &child, &app);
    ■■if (err < 0)
    ■■■return err;
    ■}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/control_remap.c (line 1319) - [public] if

```
    ■■if (remap_find_id_child(priv, &app)) {
    ■■■snd_error(CONTROL, "duplicate source id '%s'!", id);
    ■■■return -EINVAL;
    ■■}
    ■■err = add_to_remap(priv, &child, &app);
    ■■if (err < 0)
    ■■■return err;
    ■}
```

```
    ■return 0;
    }
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/control_remap.c (line 1320) - [public] snd_error

```
    ■■■snd_error(CONTROL, "duplicate source id '%s'!", id);
    ■■■return -EINVAL;
    ■■}
    ■■err = add_to_remap(priv, &child, &app);
    ■■if (err < 0)
    ■■■return err;
    ■}
```

```
    ■return 0;
    }
```

```
static int new_map(snd_ctl_remap_t *priv, snd_ctl_map_t **_map, snd_ctl_elem_id_t *id)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1340) - [public] memset

```
    ■■memset(map + priv->map_alloc, 0, sizeof(*map) * 16);
    ■■priv->map_alloc += 16;
    ■■priv->map = map;
    ■■}
    ■map = &priv->map[priv->map_items++];
    ■map->map_id = *id;
    ■numid = remap_numid_new(priv, 0, ++priv->numid_app_last);
    ■if (numid == NULL)
    ■■return -ENOMEM;
    ■map->map_id.numid = numid->numid_app;
    ■debug_id(&map->map_id, "%s created\n", __func__);
    ■*_map = map;
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/control_remap.c (line 1363) - [public] memset

```
    memset(mctl + map->controls_alloc, 0, sizeof(*mctl) * 4);
    map->controls_alloc += 4;
    map->controls = mctl;
}
mctl = &map->controls[map->controls_items++];
mctl->id_child = *id;
*_mctl = mctl;
return 0;
}
```

```
static int add_chn_to_map(struct snd_ctl_map_ctl *mctl, long idx, long src_idx, long val)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1379) - [public] snd_error

```
    snd_error(CONTROL, "Wrong channel mapping (extra source channel?)");
    return -EINVAL;
}
if (mctl->channel_map_alloc <= (size_t)idx) {
    map = realloc(mctl->channel_map, mctl->src_channels * (idx + 4) * sizeof(*map));
    if (map == NULL)
        return -ENOMEM;
    mctl->channel_map = map;
    off = mctl->channel_map_alloc;
    mctl->channel_map_alloc = idx + 4;
    for ( ; off < mctl->src_channels * mctl->channel_map_alloc; off++)
        map[off] = -1;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1407) - [public] if

```
    if (safe_strtol(dst_id, &idx) || snd_config_get_integer(n, &chn)) {
        snd_error(CONTROL, "Wrong channel mapping (%ld -> %ld)", idx, chn);
        return -EINVAL;
    }
    err = add_chn_to_map(mctl, idx, src_idx, chn);
    if (err < 0)
        return err;
    src_idx++;
}
return 0;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/control_remap.c (line 1408) - [public] snd_error

```
    snd_error(CONTROL, "Wrong channel mapping (%ld -> %ld)", idx, chn);
    return -EINVAL;
}
err = add_chn_to_map(mctl, idx, src_idx, chn);
if (err < 0)
    return err;
src_idx++;
}
return 0;
}
```

```
static int parse_map_vindex(struct snd_ctl_map_ctl *mctl, snd_config_t *conf)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1437) - [public] if

```
    if (snd_config_is_array(n) > 0) {
        err = add_chn_to_map_array(mctl, id, n);
    } else {
        if (safe_strtol(id, &idx) || snd_config_get_integer(n, &chn)) {
            snd_error(CONTROL, "Wrong channel mapping (%ld -> %ld)", idx, chn);
            return -EINVAL;
        }
        err = add_chn_to_map(mctl, idx, 0, chn);
    }
    if (err < 0)
        return err;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1440) - [public] if

```
    if (safe_strtol(id, &idx) || snd_config_get_integer(n, &chn)) {
        snd_error(CONTROL, "Wrong channel mapping (%ld -> %ld)", idx, chn);
        return -EINVAL;
    }
    err = add_chn_to_map(mctl, idx, 0, chn);
}
if (err < 0)
    return err;
}

return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1441) - [public] snd_error

```
    snd_error(CONTROL, "Wrong channel mapping (%ld -> %ld)", idx, chn);
    return -EINVAL;
}
err = add_chn_to_map(mctl, idx, 0, chn);
}
if (err < 0)
    return err;
}

return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1464) - [public] if

```
    if (strcmp(id, "vindex") == 0) {
        err = parse_map_vindex(mctl, n);
        if (err < 0)
            return err;
    }
}
return 0;
}

static int parse_map1(snd_ctl_map_t *map, snd_config_t *conf)
{
    snd_config_iterator_t i, next;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1485) - [public] snd_ctl_elem_id_clear

```
    ■■snd_ctl_elem_id_clear(&cid);
    ■■err = snd_ctl_ascii_elem_id_parse(&cid, id);
    ■■if (err < 0) {
    ■■■snd_error(CONTROL, "unable to parse control id '%s'!", id);
    ■■■return -EINVAL;
    ■■}
    ■■err = add_ctl_to_map(map, &mctl, &cid);
    ■■if (err < 0)
    ■■■return err;
    ■■err = parse_map_config(mctl, n);
    ■■if (err < 0)
    ■■■return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1487) - [public] if

```
    ■■if (err < 0) {
    ■■■snd_error(CONTROL, "unable to parse control id '%s'!", id);
    ■■■return -EINVAL;
    ■■}
    ■■err = add_ctl_to_map(map, &mctl, &cid);
    ■■if (err < 0)
    ■■■return err;
    ■■err = parse_map_config(mctl, n);
    ■■if (err < 0)
    ■■■return err;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1488) - [public] snd_error

```
    ■■■snd_error(CONTROL, "unable to parse control id '%s'!", id);
    ■■■return -EINVAL;
    ■■}
    ■■err = add_ctl_to_map(map, &mctl, &cid);
    ■■if (err < 0)
    ■■■return err;
    ■■err = parse_map_config(mctl, n);
    ■■if (err < 0)
    ■■■return err;
    ■}

    ■return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1516) - [public] snd_ctl_elem_id_clear

```
    ■■snd_ctl_elem_id_clear(&eid);
    ■■err = snd_ctl_ascii_elem_id_parse(&eid, id);
    ■■if (err < 0) {
    ■■■snd_error(CONTROL, "unable to parse id '%s'!", id);
    ■■■return -EINVAL;
    ■■}
    ■■err = new_map(priv, &map, &eid);
    ■■if (err < 0)
    ■■■return 0;
    ■■err = parse_map1(map, n);
    ■■if (err < 0)
    ■■■return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1518) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "unable to parse id '%s'", id);
        return -EINVAL;
    }
    err = new_map(priv, &map, &eid);
    if (err < 0)
        return 0;
    err = parse_map1(map, n);
    if (err < 0)
        return err;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1519) - [public] snd_error

```
    snd_error(CONTROL, "unable to parse id '%s'", id);
    return -EINVAL;
}

err = new_map(priv, &map, &eid);
if (err < 0)
    return 0;
err = parse_map1(map, n);
if (err < 0)
    return err;
}

return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1541) - [public] memset

```
    memset(sync + priv->sync_alloc, 0, sizeof(*sync) * 16);
    priv->sync_alloc += 16;
    priv->sync = sync;
}
return &priv->sync[priv->sync_items++];
}

static int parse_sync1(snd_ctl_remap_t *priv, unsigned int count, snd_config_t *conf)
{
    snd_config_iterator_t i, next;
    snd_ctl_elem_id_t *eid;
    snd_ctl_sync_t *sync;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1565) - [public] if

```
    if (snd_config_get_string(n, &str) < 0) {
        snd_error(CONTROL, "strings are expected in sync array");
        return -EINVAL;
    }
    eid = &sync->control_ids[index];
    snd_ctl_elem_id_clear(eid);
    err = snd_ctl_ascii_elem_id_parse(eid, str);
    if (err < 0) {
        snd_error(CONTROL, "unable to parse control id '%s'", str);
        return -EINVAL;
    }
    sync->control_items++;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1566) - [public] snd_error

```
    ■■■snd_error(CONTROL, "strings are expected in sync array");
    ■■■return -EINVAL;
    ■■}
    ■■eid = &sync->control_ids[index];
    ■■snd_ctl_elem_id_clear(eid);
    ■■err = snd_ctl_ascii_elem_id_parse(eid, str);
    ■■if (err < 0) {
    ■■■snd_error(CONTROL, "unable to parse control id '%s'!", str);
    ■■■return -EINVAL;
    ■■}
    ■■sync->control_items++;
    ■■index++;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1570) - [public] snd_ctl_elem_id_clear

```
    ■■snd_ctl_elem_id_clear(eid);
    ■■err = snd_ctl_ascii_elem_id_parse(eid, str);
    ■■if (err < 0) {
    ■■■snd_error(CONTROL, "unable to parse control id '%s'!", str);
    ■■■return -EINVAL;
    ■■}
    ■■sync->control_items++;
    ■■index++;
    ■■}

    ■return 0;
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1572) - [public] if

```
    ■■if (err < 0) {
    ■■■snd_error(CONTROL, "unable to parse control id '%s'!", str);
    ■■■return -EINVAL;
    ■■}
    ■■sync->control_items++;
    ■■index++;
    ■■}

    ■return 0;
    }

    static int parse_sync_compound(snd_ctl_remap_t *priv, snd_config_t *conf)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1573) - [public] snd_error

```
    ■■■snd_error(CONTROL, "unable to parse control id '%s'!", str);
    ■■■return -EINVAL;
    ■■}
    ■■sync->control_items++;
    ■■index++;
    ■■}

    ■return 0;
    }

    static int parse_sync_compound(snd_ctl_remap_t *priv, snd_config_t *conf)
    {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1598) - [public] if

```
    if (strcmp(id, "switch") == 0) {
        if (snd_config_get_string(n, &str) < 0) {
            snd_error(CONTROL, "String is expected for switch");
            return -EINVAL;
        }
        err = snd_ctl_ascii_elem_id_parse(&eid, str);
        if (err < 0) {
            snd_error(CONTROL, "unable to parse id '%s'", str);
            return -EINVAL;
        }
        eid_found = true;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1599) - [public] if

```
    if (snd_config_get_string(n, &str) < 0) {
        snd_error(CONTROL, "String is expected for switch");
        return -EINVAL;
    }
    err = snd_ctl_ascii_elem_id_parse(&eid, str);
    if (err < 0) {
        snd_error(CONTROL, "unable to parse id '%s'", str);
        return -EINVAL;
    }
    eid_found = true;
}
if (strcmp(id, "controls") == 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1600) - [public] snd_error

```
    snd_error(CONTROL, "String is expected for switch");
    return -EINVAL;
}
err = snd_ctl_ascii_elem_id_parse(&eid, str);
if (err < 0) {
    snd_error(CONTROL, "unable to parse id '%s'", str);
    return -EINVAL;
}
eid_found = true;
}
if (strcmp(id, "controls") == 0) {
    count = snd_config_is_array(n);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1604) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "unable to parse id '%s'", str);
        return -EINVAL;
    }
    eid_found = true;
}
if (strcmp(id, "controls") == 0) {
    count = snd_config_is_array(n);
    if (count <= 0) {
        snd_error(CONTROL, "Array is expected for sync!");
        return -EINVAL;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1605) - [public] snd_error

```
    snd_error(CONTROL, "unable to parse id '%s'!", str);
    return -EINVAL;
}
eid_found = true;
}
if (strcmp(id, "controls") == 0) {
    count = snd_config_is_array(n);
    if (count <= 0) {
        snd_error(CONTROL, "Array is expected for sync!");
        return -EINVAL;
    }
    err = parse_sync1(priv, count, n);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1610) - [public] if

```
    if (strcmp(id, "controls") == 0) {
        count = snd_config_is_array(n);
        if (count <= 0) {
            snd_error(CONTROL, "Array is expected for sync!");
            return -EINVAL;
        }
        err = parse_sync1(priv, count, n);
        if (err < 0)
            return err;
        controls_found = true;
    }
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1612) - [public] if

```
    if (count <= 0) {
        snd_error(CONTROL, "Array is expected for sync!");
        return -EINVAL;
    }
    err = parse_sync1(priv, count, n);
    if (err < 0)
        return err;
    controls_found = true;
}

if (!controls_found || !eid_found)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1613) - [public] snd_error

```
    snd_error(CONTROL, "Array is expected for sync!");
    return -EINVAL;
}
err = parse_sync1(priv, count, n);
if (err < 0)
    return err;
controls_found = true;
}

if (!controls_found || !eid_found)
    return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1644) - [public] if

```
    if (snd_config_get_type(n) == SND_CONFIG_TYPE_COMPOUND) {
        err = parse_sync_compound(priv, n);
    } else {
        count = snd_config_is_array(n);
        if (count <= 0) {
            snd_error(CONTROL, "Array is expected for sync!");
            return -EINVAL;
        }
        err = parse_sync1(priv, count, n);
    }
    if (err < 0)
        return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1648) - [public] if

```
    if (count <= 0) {
        snd_error(CONTROL, "Array is expected for sync!");
        return -EINVAL;
    }
    err = parse_sync1(priv, count, n);
    if (err < 0)
        return err;
    }

    return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1649) - [public] snd_error

```
    snd_error(CONTROL, "Array is expected for sync!");
    return -EINVAL;
}
err = parse_sync1(priv, count, n);
if (err < 0)
    return err;
}

return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1711) - [public] remap_free

```
    remap_free(priv);
    _noop:
    free(child->name);
    child->name = name ? strdup(name) : NULL;
    if (name && !child->name)
        return -ENOMEM;
    *handlep = child;
    return 0;
}

priv->event_items = priv->map_items;
for (index = 0; index < priv->sync_items; index++)
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/control_remap.c (line 1713) - [public] free

```
    free(child->name);
    child->name = name ? strdup(name) : NULL;
    if (name && !child->name)
        return -ENOMEM;
    *handlep = child;
    return 0;
}

priv->event_items = priv->map_items;
for (index = 0; index < priv->sync_items; index++)
    priv->event_items += priv->sync[index].control_items;
priv->event_queue = calloc(priv->event_items, sizeof(priv->event_queue[0]));
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1860) - [public] if

```
    if (strcmp(id, "remap") == 0) {
        remap = n;
        continue;
    }
    if (strcmp(id, "map") == 0) {
        map = n;
        continue;
    }
    if (strcmp(id, "sync") == 0) {
        sync = n;
        continue;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1864) - [public] if

```
    if (strcmp(id, "map") == 0) {
        map = n;
        continue;
    }
    if (strcmp(id, "sync") == 0) {
        sync = n;
        continue;
    }
    if (strcmp(id, "child") == 0) {
        child = n;
        continue;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1868) - [public] if

```
    if (strcmp(id, "sync") == 0) {
        sync = n;
        continue;
    }
    if (strcmp(id, "child") == 0) {
        child = n;
        continue;
    }
    snd_error(CONTROL, "Unknown field %s", id);
    return -EINVAL;
}
if (!child) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1872) - [public] if

```
    if (strcmp(id, "child") == 0) {
        child = n;
        continue;
    }
    snd_error(CONTROL, "Unknown field %s", id);
    return -EINVAL;
}
if (!child) {
    snd_error(CONTROL, "child is not defined");
    return -EINVAL;
}
err = _snd_ctl_open_child(&cctl, root, child, mode, conf);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1876) - [public] snd_error

```
    snd_error(CONTROL, "Unknown field %s", id);
    return -EINVAL;
}
if (!child) {
    snd_error(CONTROL, "child is not defined");
    return -EINVAL;
}
err = _snd_ctl_open_child(&cctl, root, child, mode, conf);
if (err < 0)
    return err;
err = snd_ctl_remap_open(handlep, name, remap, map, sync, cctl, mode);
if (err < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1880) - [public] snd_error

```
    snd_error(CONTROL, "child is not defined");
    return -EINVAL;
}
err = _snd_ctl_open_child(&cctl, root, child, mode, conf);
if (err < 0)
    return err;
err = snd_ctl_remap_open(handlep, name, remap, map, sync, cctl, mode);
if (err < 0)
    snd_ctl_close(cctl);
    return err;
}
#ifdef DOC_HIDDEN
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_remap.c (line 1888) - [public] snd_ctl_close

```
    snd_ctl_close(cctl);
    return err;
}
#ifdef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(_snd_ctl_remap_open, SND_CONTROL_DLSYM_VERSION);
#endif
```

dirtycode/alsa-lib/src/control/control_shm.c (line 63) - [public] snd_error

```
    snd_error(CONTROL, "Server has not done the cmd");
    return -EBADFD;
}
return ctrl->result;
}
```

```
static int snd_ctl_shm_action_fd(snd_ctl_t *ctl, int *fd)
{
    snd_ctl_shm_t *shm = ctl->private_data;
    int err;
    char buf[1] = {0};
    volatile snd_ctl_shm_ctrl_t *ctrl = shm->ctrl;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 82) - [public] snd_error

```
■snd_error(CONTROL, "Server has not done the cmd");
■return -EBADFD;
■}
■return ctrl->result;
}
```

```
static int snd_ctl_shm_close(snd_ctl_t *ctl)
{
■snd_ctl_shm_t *shm = ctl->private_data;
■volatile snd_ctl_shm_ctrl_t *ctrl = shm->ctrl;
■int result;
■ctrl->cmd = SND_CTL_IOCTL_CLOSE;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 115) - [public] snd_ctl_shm_action

```
■return snd_ctl_shm_action(ctl);
}
```

```
static int snd_ctl_shm_poll_descriptor(snd_ctl_t *ctl)
{
■snd_ctl_shm_t *shm = ctl->private_data;
■volatile snd_ctl_shm_ctrl_t *ctrl = shm->ctrl;
■int fd, err;
■ctrl->cmd = SND_CTL_IOCTL_POLL_DESCRIPTOR;
■err = snd_ctl_shm_action_fd(ctl, &fd);
■if (err < 0)
■■return err;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 136) - [public] snd_ctl_shm_action

```
■return snd_ctl_shm_action(ctl);
}
```

```
static int snd_ctl_shm_card_info(snd_ctl_t *ctl, snd_ctl_card_info_t *info)
{
■snd_ctl_shm_t *shm = ctl->private_data;
■volatile snd_ctl_shm_ctrl_t *ctrl = shm->ctrl;
■int err;
//■ctrl->u.card_info = *info;
■ctrl->cmd = SNDRV_CTL_IOCTL_CARD_INFO;
■err = snd_ctl_shm_action(ctl);
■if (err < 0)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 307) - [public] snd_ctl_shm_action

```
■return snd_ctl_shm_action(ctl);
}
```

```
static int snd_ctl_shm_rawmidi_next_device(snd_ctl_t *ctl, int * device)
{
■snd_ctl_shm_t *shm = ctl->private_data;
■volatile snd_ctl_shm_ctrl_t *ctrl = shm->ctrl;
■int err;
■ctrl->u.device = *device;
■ctrl->cmd = SNDRV_CTL_IOCTL_RAWMIDI_NEXT_DEVICE;
■err = snd_ctl_shm_action(ctl);
■if (err < 0)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 344) - [public] snd_ctl_shm_action

```
    return snd_ctl_shm_action(ctl);
}

static int snd_ctl_shm_set_power_state(snd_ctl_t *ctl, unsigned int state)
{
    struct snd_ctl_shm_t *shm = ctl->private_data;
    struct volatile snd_ctl_shm_ctl_t *ctrl = shm->ctrl;
    ctrl->u.power_state = state;
    ctrl->cmd = SNDRV_CTL_IOCTL_POWER;
    return snd_ctl_shm_action(ctl);
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 353) - [public] snd_ctl_shm_action

```
    return snd_ctl_shm_action(ctl);
}

static int snd_ctl_shm_get_power_state(snd_ctl_t *ctl, unsigned int *state)
{
    struct snd_ctl_shm_t *shm = ctl->private_data;
    struct volatile snd_ctl_shm_ctl_t *ctrl = shm->ctrl;
    int err;
    ctrl->cmd = SNDRV_CTL_IOCTL_POWER_STATE;
    err = snd_ctl_shm_action(ctl);
    if (err < 0)
        return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 428) - [public] close

```
    close(sock);
    return -errno;
}

return sock;
}
```

```
int snd_ctl_shm_open(snd_ctl_t **handlep, const char *name, const char *sockname, const char *sname, int
{
    struct snd_ctl_t *ctl;
    struct snd_ctl_shm_t *shm = NULL;
    struct snd_client_open_request_t *req;
    struct snd_client_open_answer_t ans;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 451) - [public] snd_error

```
    snd_error(CONTROL, "server for socket %s is not running", sockname);
    goto _err;
}

sock = result;

reqlen = sizeof(*req) + snamelen;
req = alloca(reqlen);
memcpy(req->name, sname, snamelen);
req->dev_type = SND_DEV_TYPE_CONTROL;
req->transport_type = SND_TRANSPORT_TYPE_SHM;
req->stream = 0;
req->mode = mode;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 466) - [public] snd_error

```
    ■■snd_error(CONTROL, "write error");
    ■■result = -errno;
    ■■goto _err;
    ■}
    ■if ((size_t) err != reqlen) {
    ■■snd_error(CONTROL, "write size error");
    ■■result = -EINVAL;
    ■■goto _err;
    ■}
    ■err = read(sock, &ans, sizeof(ans));
    ■if (err < 0) {
    ■■snd_error(CONTROL, "read error");
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 471) - [public] snd_error

```
    ■■snd_error(CONTROL, "write size error");
    ■■result = -EINVAL;
    ■■goto _err;
    ■}
    ■err = read(sock, &ans, sizeof(ans));
    ■if (err < 0) {
    ■■snd_error(CONTROL, "read error");
    ■■result = -errno;
    ■■goto _err;
    ■}
    ■if (err != sizeof(ans)) {
    ■■snd_error(CONTROL, "read size error");
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 477) - [public] snd_error

```
    ■■snd_error(CONTROL, "read error");
    ■■result = -errno;
    ■■goto _err;
    ■}
    ■if (err != sizeof(ans)) {
    ■■snd_error(CONTROL, "read size error");
    ■■result = -EINVAL;
    ■■goto _err;
    ■}
    ■if (ans.result < INT_MIN || ans.result > INT_MAX) {
    ■■snd_error(CONTROL, "invalid data error");
    ■■result = -EINVAL;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 482) - [public] snd_error

```
    ■■snd_error(CONTROL, "read size error");
    ■■result = -EINVAL;
    ■■goto _err;
    ■}
    ■if (ans.result < INT_MIN || ans.result > INT_MAX) {
    ■■snd_error(CONTROL, "invalid data error");
    ■■result = -EINVAL;
    ■■goto _err;
    ■}
    ■result = ans.result;
    ■if (result < 0)
    ■■goto _err;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 487) - [public] snd_error

```
    ■■snd_error(CONTROL, "invalid data error");
    ■■result = -EINVAL;
    ■■goto _err;
    ■}
    ■result = ans.result;
    ■if (result < 0)
    ■■goto _err;

    ■ctrl = shmat(ans.cookie, 0, 0);
    ■if (!ctrl) {
    ■■result = -errno;
    ■■goto _err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 519) - [public] snd_ctl_close

```
    ■■snd_ctl_close(ctl);
    ■■return err;
    ■}
    ■ctl->poll_fd = err;
    ■*handlep = ctl;
    ■return 0;

    _err:
    ■if (sock >= 0)
    ■■close(sock);
    ■if (ctrl)
    ■■shmdt(ctrl);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 528) - [public] close

```
    ■■close(sock);
    ■if (ctrl)
    ■■shmdt(ctrl);
    ■free(shm);
    ■return result;
}
```

```
int _snd_ctl_shm_open(snd_ctl_t **handlep, char *name, snd_config_t *root, snd_config_t *conf, int mode)
{
    ■snd_config_iterator_t i, next;
    ■const char *server = NULL;
    ■const char *ctl_name = NULL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 530) - [public] shmdt

```
    ■■shmdt(ctrl);
    ■free(shm);
    ■return result;
}
```

```
int _snd_ctl_shm_open(snd_ctl_t **handlep, char *name, snd_config_t *root, snd_config_t *conf, int mode)
{
    ■snd_config_iterator_t i, next;
    ■const char *server = NULL;
    ■const char *ctl_name = NULL;
    ■snd_config_t *sconfig;
    ■const char *sockname = NULL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 552) - [public] if

```
    if (strcmp(id, "server") == 0) {
        err = snd_config_get_string(n, &server);
        if (err < 0) {
            snd_error(CONTROL, "Invalid type for %s", id);
            return -EINVAL;
        }
        continue;
    }
    if (strcmp(id, "ctl") == 0) {
        err = snd_config_get_string(n, &ctl_name);
        if (err < 0) {
            snd_error(CONTROL, "Invalid type for %s", id);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/control/control_shm.c (line 554) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Invalid type for %s", id);
        return -EINVAL;
    }
    continue;
}
if (strcmp(id, "ctl") == 0) {
    err = snd_config_get_string(n, &ctl_name);
    if (err < 0) {
        snd_error(CONTROL, "Invalid type for %s", id);
        return -EINVAL;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_shm.c (line 555) - [public] snd_error

```
    snd_error(CONTROL, "Invalid type for %s", id);
    return -EINVAL;
}
continue;
}
if (strcmp(id, "ctl") == 0) {
    err = snd_config_get_string(n, &ctl_name);
    if (err < 0) {
        snd_error(CONTROL, "Invalid type for %s", id);
        return -EINVAL;
    }
    continue;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_shm.c (line 560) - [public] if

```
    if (strcmp(id, "ctl") == 0) {
        err = snd_config_get_string(n, &ctl_name);
        if (err < 0) {
            snd_error(CONTROL, "Invalid type for %s", id);
            return -EINVAL;
        }
        continue;
    }
    snd_error(CONTROL, "Unknown field %s", id);
    return -EINVAL;
}
if (!ctl_name) {
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/control_shm.c (line 562) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Invalid type for %s", id);
        return -EINVAL;
    }
    continue;
}
snd_error(CONTROL, "Unknown field %s", id);
return -EINVAL;
}
if (!ctl_name) {
    snd_error(CONTROL, "ctl is not defined");
    return -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 563) - [public] snd_error

```
    snd_error(CONTROL, "Invalid type for %s", id);
    return -EINVAL;
}
continue;
}
snd_error(CONTROL, "Unknown field %s", id);
return -EINVAL;
}
if (!ctl_name) {
    snd_error(CONTROL, "ctl is not defined");
    return -EINVAL;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 568) - [public] snd_error

```
    snd_error(CONTROL, "Unknown field %s", id);
    return -EINVAL;
}
if (!ctl_name) {
    snd_error(CONTROL, "ctl is not defined");
    return -EINVAL;
}
if (!server) {
    snd_error(CONTROL, "server is not defined");
    return -EINVAL;
}
err = snd_config_search_definition(root, "server", server, &sconfig);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 572) - [public] snd_error

```
    snd_error(CONTROL, "ctl is not defined");
    return -EINVAL;
}
if (!server) {
    snd_error(CONTROL, "server is not defined");
    return -EINVAL;
}
err = snd_config_search_definition(root, "server", server, &sconfig);
if (err < 0) {
    snd_error(CONTROL, "Unknown server %s", server);
    return -EINVAL;
}
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/control_shm.c (line 576) - [public] snd_error

```
    ■■snd_error(CONTROL, "server is not defined");
    ■■return -EINVAL;
    ■}
    ■err = snd_config_search_definition(root, "server", server, &sconfig);
    ■if (err < 0) {
    ■■snd_error(CONTROL, "Unknown server %s", server);
    ■■return -EINVAL;
    ■}
    ■if (snd_config_get_type(sconfig) != SND_CONFIG_TYPE_COMPOUND) {
    ■■snd_error(CONTROL, "Invalid type for server %s definition", server);
    ■■err = -EINVAL;
    ■■goto _err;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 581) - [public] snd_error

```
    ■■snd_error(CONTROL, "Unknown server %s", server);
    ■■return -EINVAL;
    ■}
    ■if (snd_config_get_type(sconfig) != SND_CONFIG_TYPE_COMPOUND) {
    ■■snd_error(CONTROL, "Invalid type for server %s definition", server);
    ■■err = -EINVAL;
    ■■goto _err;
    ■}
    ■snd_config_for_each(i, next, sconfig) {
    ■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■const char *id;
    ■■if (snd_config_get_id(n, &id) < 0)
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 585) - [public] snd_error

```
    ■■snd_error(CONTROL, "Invalid type for server %s definition", server);
    ■■err = -EINVAL;
    ■■goto _err;
    ■}
    ■snd_config_for_each(i, next, sconfig) {
    ■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■const char *id;
    ■■if (snd_config_get_id(n, &id) < 0)
    ■■■continue;
    ■■if (_snd_conf_generic_id(id))
    ■■■continue;
    ■■if (strcmp(id, "host") == 0)
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 598) - [public] if

```
    ■■if (strcmp(id, "socket") == 0) {
    ■■■err = snd_config_get_string(n, &sockname);
    ■■■if (err < 0) {
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■■goto _err;
    ■■■}
    ■■■continue;
    ■■}
    ■■if (strcmp(id, "port") == 0) {
    ■■■err = snd_config_get_integer(n, &port);
    ■■■if (err < 0) {
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 600) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Invalid type for %s", id);
        goto _err;
    }
    continue;
}
if (strcmp(id, "port") == 0) {
    err = snd_config_get_integer(n, &port);
    if (err < 0) {
        snd_error(CONTROL, "Invalid type for %s", id);
        goto _err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 601) - [public] snd_error

```
    snd_error(CONTROL, "Invalid type for %s", id);
    goto _err;
}
continue;
}
if (strcmp(id, "port") == 0) {
    err = snd_config_get_integer(n, &port);
    if (err < 0) {
        snd_error(CONTROL, "Invalid type for %s", id);
        goto _err;
    }
    continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 606) - [public] if

```
    if (strcmp(id, "port") == 0) {
        err = snd_config_get_integer(n, &port);
        if (err < 0) {
            snd_error(CONTROL, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    snd_error(CONTROL, "Unknown field %s", id);
    err = -EINVAL;
    goto _err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 608) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Invalid type for %s", id);
        goto _err;
    }
    continue;
}
snd_error(CONTROL, "Unknown field %s", id);
err = -EINVAL;
goto _err;
}

if (!sockname) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 609) - [public] snd_error

```
    ■■■■snd_error(CONTROL, "Invalid type for %s", id);
    ■■■■goto _err;
    ■■■}
    ■■■continue;
    ■■■}
    ■■snd_error(CONTROL, "Unknown field %s", id);
    ■■err = -EINVAL;
    ■■goto _err;
    ■■}

    ■if (!sockname) {
    ■■snd_error(CONTROL, "socket is not defined");
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 614) - [public] snd_error

```
    ■■snd_error(CONTROL, "Unknown field %s", id);
    ■■err = -EINVAL;
    ■■goto _err;
    ■■}

    ■if (!sockname) {
    ■■snd_error(CONTROL, "socket is not defined");
    ■■goto _err;
    ■■}
    ■err = snd_ctl_shm_open(handlep, name, sockname, ctl_name, mode);
    ■_err:
    ■snd_config_delete(sconfig);
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/control_shm.c (line 620) - [public] snd_error

```
    ■■snd_error(CONTROL, "socket is not defined");
    ■■goto _err;
    ■■}
    ■err = snd_ctl_shm_open(handlep, name, sockname, ctl_name, mode);
    ■_err:
    ■snd_config_delete(sconfig);
    ■return err;
    ■}
    SND_DLSYM_BUILD_VERSION(_snd_ctl_shm_open, SND_CONTROL_DLSYM_VERSION);
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 127) - [public] snd_ctl_elem_id_get_name

```
    ■■■■numid, iface, snd_ctl_elem_id_get_name(id));
    ■} else {
    ■■snprintf(buf, sizeof(buf), "iface=%s,name='%s'",
    ■■■■iface, snd_ctl_elem_id_get_name(id));
    ■■}
    ■buf[sizeof(buf)-1] = '\0';
    ■index = snd_ctl_elem_id_get_index(id);
    ■device = snd_ctl_elem_id_get_device(id);
    ■subdevice = snd_ctl_elem_id_get_subdevice(id);
    ■if (index) {
    ■■snprintf(buf1, sizeof(buf1), "index=%u", index);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 130) - [public] snd_ctl_elem_id_get_name

```
    ■■■■iface, snd_ctl_elem_id_get_name(id));
    ■■}
    ■buf[sizeof(buf)-1] = '\0';
    ■index = snd_ctl_elem_id_get_index(id);
    ■device = snd_ctl_elem_id_get_device(id);
    ■subdevice = snd_ctl_elem_id_get_subdevice(id);
    ■if (index) {
    ■■snprintf(buf1, sizeof(buf1), "index=%u", index);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■■■■strcat(buf, buf1);
    ■■}
    ■if (device) {
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 137) - [public] snprintf

```
    ■■snprintf(buf1, sizeof(buf1), "index=%u", index);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■■■strcat(buf, buf1);
    ■}
    ■if (device) {
    ■■snprintf(buf1, sizeof(buf1), "device=%u", device);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■■■strcat(buf, buf1);
    ■}
    ■if (subdevice) {
    ■■snprintf(buf1, sizeof(buf1), "subdevice=%u", subdevice);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 139) - [public] strcat

```
    ■■■strcat(buf, buf1);
    ■}
    ■if (device) {
    ■■snprintf(buf1, sizeof(buf1), "device=%u", device);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■■■strcat(buf, buf1);
    ■}
    ■if (subdevice) {
    ■■snprintf(buf1, sizeof(buf1), "subdevice=%u", subdevice);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■■■strcat(buf, buf1);
    ■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 142) - [public] snprintf

```
    ■■snprintf(buf1, sizeof(buf1), "device=%u", device);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■■■strcat(buf, buf1);
    ■}
    ■if (subdevice) {
    ■■snprintf(buf1, sizeof(buf1), "subdevice=%u", subdevice);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■■■strcat(buf, buf1);
    ■}
    ■return strdup(buf);
}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 144) - [public] strcat

```
    ■■■strcat(buf, buf1);
    ■}
    ■if (subdevice) {
    ■■snprintf(buf1, sizeof(buf1), "subdevice=%u", subdevice);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■■■strcat(buf, buf1);
    ■}
    ■return strdup(buf);
}

#ifdef DOC_HIDDEN
/* used by UCM parser, too */
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 147) - [public] snprintf

```
    ■■snprintf(buf1, sizeof(buf1), ",subdevice=%u", subdevice);
    ■■if (strlen(buf) + strlen(buf1) < sizeof(buf))
    ■■■strcat(buf, buf1);
    ■}
    ■return strdup(buf);
}

#ifdef DOC_HIDDEN
/* used by UCM parser, too */
int __snd_ctl_ascii_elem_id_parse(snd_ctl_elem_id_t *dst, const char *str,
    ■■■■ const char **ret_ptr)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 149) - [public] strcat

```
    ■■■strcat(buf, buf1);
    ■}
    ■return strdup(buf);
}

#ifdef DOC_HIDDEN
/* used by UCM parser, too */
int __snd_ctl_ascii_elem_id_parse(snd_ctl_elem_id_t *dst, const char *str,
    ■■■■ const char **ret_ptr)
{
    ■char buf[64];
    ■int c, size;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 151) - [public] strdup

```
    ■return strdup(buf);
}

#ifdef DOC_HIDDEN
/* used by UCM parser, too */
int __snd_ctl_ascii_elem_id_parse(snd_ctl_elem_id_t *dst, const char *str,
    ■■■■ const char **ret_ptr)
{
    ■char buf[64];
    ■int c, size;
    ■int err = -EINVAL;
    ■long l;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 171) - [public] if

```
    ■■if (!strncasecmp(str, "numid=", 6)) {
    ■■■str += 6;
    ■■■ptr = buf;
    ■■■size = 0;
    ■■■while (*str && *str != ',') {
    ■■■■if (size < (int)sizeof(buf) - 1) {
    ■■■■■*ptr++ = *str;
    ■■■■■size++;
    ■■■■}
    ■■■■str++;
    ■■■}
    ■■■*ptr = '\0';
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 175) - [public] while

```
    while (*str && *str != ',') {
        if (size < (int)sizeof(buf) - 1) {
            *ptr++ = *str;
            size++;
        }
        str++;
    }
    *ptr = '\0';
    if (safe_strtol(buf, &l) < 0)
        l = -1;
    if (l <= 0 || l >= INT32_MAX) {
        snd_error(CONTROL, "Invalid numid %ld (%s)", l, buf);
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 176) - [public] if

```
    if (size < (int)sizeof(buf) - 1) {
        *ptr++ = *str;
        size++;
    }
    str++;
}
*ptr = '\0';
if (safe_strtol(buf, &l) < 0)
    l = -1;
if (l <= 0 || l >= INT32_MAX) {
    snd_error(CONTROL, "Invalid numid %ld (%s)", l, buf);
    goto out;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 185) - [public] if

```
    if (l <= 0 || l >= INT32_MAX) {
        snd_error(CONTROL, "Invalid numid %ld (%s)", l, buf);
        goto out;
    }
    snd_ctl_elem_id_set_numid(dst, (int)l);
    while (isdigit(*str))
        str++;
    if (!strncasecmp(str, "iface=", 6)) {
        str += 6;
        if (!strncasecmp(str, "card", 4)) {
            snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_CARD);
            str += 4;
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 186) - [public] snd_error

```
    snd_error(CONTROL, "Invalid numid %ld (%s)", l, buf);
    goto out;
}
snd_ctl_elem_id_set_numid(dst, (int)l);
while (isdigit(*str))
    str++;
    if (!strncasecmp(str, "iface=", 6)) {
        str += 6;
        if (!strncasecmp(str, "card", 4)) {
            snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_CARD);
            str += 4;
        }
        if (!strncasecmp(str, "mixer", 5)) {
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 189) - [public] snd_ctl_elem_id_set_numid

```
    while (isdigit(*str))
        str++;
    } else if (!strncasecmp(str, "iface=", 6)) {
        str += 6;
        if (!strncasecmp(str, "card", 4)) {
            snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_CARD);
            str += 4;
        } else if (!strncasecmp(str, "mixer", 5)) {
            snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_MIXER);
            str += 5;
        } else if (!strncasecmp(str, "pcm", 3)) {
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 194) - [public] if

```
    if (!strncasecmp(str, "card", 4)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_CARD);
        str += 4;
    } else if (!strncasecmp(str, "mixer", 5)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_MIXER);
        str += 5;
    } else if (!strncasecmp(str, "pcm", 3)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_PCM);
        str += 3;
    } else if (!strncasecmp(str, "rawmidi", 7)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_RAWMIDI);
        str += 7;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 195) - [public] snd_ctl_elem_id_set_interface

```
    snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_CARD);
    str += 4;
    } else if (!strncasecmp(str, "mixer", 5)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_MIXER);
        str += 5;
    } else if (!strncasecmp(str, "pcm", 3)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_PCM);
        str += 3;
    } else if (!strncasecmp(str, "rawmidi", 7)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_RAWMIDI);
        str += 7;
    } else if (!strncasecmp(str, "timer", 5)) {
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 198) - [public] snd_ctl_elem_id_set_interface

```
    snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_MIXER);
    str += 5;
    } else if (!strncasecmp(str, "pcm", 3)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_PCM);
        str += 3;
    } else if (!strncasecmp(str, "rawmidi", 7)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_RAWMIDI);
        str += 7;
    } else if (!strncasecmp(str, "timer", 5)) {
        snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_TIMER);
        str += 5;
    } else if (!strncasecmp(str, "sequencer", 9)) {
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 201) - [public] snd_ctl_elem_id_set_interface

```
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_PCM);
#####str += 3;
####} else if (!strcasecmp(str, "rawmidi", 7)) {
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_RAWMIDI);
#####str += 7;
####} else if (!strcasecmp(str, "timer", 5)) {
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_TIMER);
#####str += 5;
####} else if (!strcasecmp(str, "sequencer", 9)) {
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_SEQUENCER);
#####str += 9;
####} else {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 204) - [public] snd_ctl_elem_id_set_interface

```
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_RAWMIDI);
#####str += 7;
####} else if (!strcasecmp(str, "timer", 5)) {
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_TIMER);
#####str += 5;
####} else if (!strcasecmp(str, "sequencer", 9)) {
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_SEQUENCER);
#####str += 9;
####} else {
#####goto out;
####}
####} else if (!strcasecmp(str, "name=", 5)) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 207) - [public] snd_ctl_elem_id_set_interface

```
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_TIMER);
#####str += 5;
####} else if (!strcasecmp(str, "sequencer", 9)) {
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_SEQUENCER);
#####str += 9;
####} else {
#####goto out;
####}
####} else if (!strcasecmp(str, "name=", 5)) {
#####str += 5;
#####ptr = buf;
#####size = 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 210) - [public] snd_ctl_elem_id_set_interface

```
#####snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_SEQUENCER);
#####str += 9;
####} else {
#####goto out;
####}
####} else if (!strcasecmp(str, "name=", 5)) {
#####str += 5;
#####ptr = buf;
#####size = 0;
#####if (*str == '\\' || *str == '\"') {
#####c = *str++;
#####while (*str && *str != c) {
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/ctlparse.c (line 219) - [public] if

```
    if (*str == '\\' || *str == '\"') {
        c = *str++;
        while (*str && *str != c) {
            if (size < (int)sizeof(buf) - 1) {
                *ptr++ = *str;
                size++;
            }
            str++;
        }
        if (*str == c)
            str++;
    } else {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 221) - [public] while

```
    while (*str && *str != c) {
        if (size < (int)sizeof(buf) - 1) {
            *ptr++ = *str;
            size++;
        }
        str++;
    }
    if (*str == c)
        str++;
} else {
    while (*str && *str != ',') {
        if (size < (int)sizeof(buf) - 1) {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 222) - [public] if

```
    if (size < (int)sizeof(buf) - 1) {
        *ptr++ = *str;
        size++;
    }
    str++;
}
if (*str == c)
    str++;
} else {
    while (*str && *str != ',') {
        if (size < (int)sizeof(buf) - 1) {
            *ptr++ = *str;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 231) - [public] while

```
    while (*str && *str != ',') {
        if (size < (int)sizeof(buf) - 1) {
            *ptr++ = *str;
            size++;
        }
        str++;
    }
    *ptr = '\0';
    snd_ctl_elem_id_set_name(dst, buf);
} else if (!strncasecmp(str, "index=", 6)) {
    str += 6;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 232) - [public] if

```
        if (size < (int)sizeof(buf) - 1) {
            *ptr++ = *str;
            size++;
        }
        str++;
    }
    *ptr = '\0';
    snd_ctl_elem_id_set_name(dst, buf);
} else if (!strncasecmp(str, "index=", 6)) {
    str += 6;
    snd_ctl_elem_id_set_index(dst, atoi(str));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 240) - [public] snd_ctl_elem_id_set_name

```
*ptr = '\0';

    snd_ctl_elem_id_set_name(dst, buf);
} else if (!strncasecmp(str, "index=", 6)) {
    str += 6;
    snd_ctl_elem_id_set_index(dst, atoi(str));
    while (isdigit(*str))
        str++;
} else if (!strncasecmp(str, "device=", 7)) {
    str += 7;
    snd_ctl_elem_id_set_device(dst, atoi(str));
    while (isdigit(*str))
        str++;
} else if (!strncasecmp(str, "subdevice=", 10)) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 243) - [public] snd_ctl_elem_id_set_index

```
    snd_ctl_elem_id_set_index(dst, atoi(str));
    while (isdigit(*str))
        str++;
} else if (!strncasecmp(str, "device=", 7)) {
    str += 7;
    snd_ctl_elem_id_set_device(dst, atoi(str));
    while (isdigit(*str))
        str++;
} else if (!strncasecmp(str, "subdevice=", 10)) {
    str += 10;
    snd_ctl_elem_id_set_subdevice(dst, atoi(str));
    while (isdigit(*str))
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 248) - [public] snd_ctl_elem_id_set_device

```
    snd_ctl_elem_id_set_device(dst, atoi(str));
    while (isdigit(*str))
        str++;
} else if (!strncasecmp(str, "subdevice=", 10)) {
    str += 10;
    snd_ctl_elem_id_set_subdevice(dst, atoi(str));
    while (isdigit(*str))
        str++;
}
if (*str == ',') {
    str++;
} else {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 253) - [public] snd_ctl_elem_id_set_subdevice

```
    snd_ctl_elem_id_set_subdevice(dst, atoi(str));
    while (isdigit(*str))
        str++;
}
if (*str == ',') {
    str++;
} else {
    /* when ret_ptr is given, allow to terminate gracefully
    * at the next space letter
    */
    if (ret_ptr && isspace(*str))
        break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 257) - [public] if

```
if (*str == ',') {
    str++;
} else {
    /* when ret_ptr is given, allow to terminate gracefully
    * at the next space letter
    */
    if (ret_ptr && isspace(*str))
        break;
    if (*str)
        goto out;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 286) - [public] __snd_ctl_ascii_elem_id_parse

```
return __snd_ctl_ascii_elem_id_parse(dst, str, NULL);
}
```

```
static int get_ctl_enum_item_index(snd_ctl_t *handle,
    snd_ctl_elem_info_t *info,
    const char **ptrp)
{
    char *ptr = (char *)*ptrp;
    int items, i, len;
    const char *name;
    char end;

```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/ctlparse.c (line 309) - [public] snd_ctl_elem_info_set_item

```
snd_ctl_elem_info_set_item(info, i);
if (snd_ctl_elem_info(handle, info) < 0)
    return -1;
name = snd_ctl_elem_info_get_item_name(info);
len = strlen(name);
if (strncmp(name, ptr, len))
    continue;
if (end == '\\0' && (ptr[len] == '\\0' || ptr[len] == ',' || ptr[len] == '\\n')) {
    *ptrp = ptr + len;
    return i;
}
if (end != '\\0' && ptr[len] == end) {
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/control/ctlparse.c (line 316) - [public] if
    if (end == '\0' && (ptr[len] == '\0' || ptr[len] == ',' || ptr[len] == '\n')) {
        *ptrp = ptr + len;
        return i;
    }
    if (end != '\0' && ptr[len] == end) {
        *ptrp = ptr + len + 1;
        return i;
    }
    return -1;
}

// ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/control/ctlparse.c (line 320) - [public] if
    if (end != '\0' && ptr[len] == end) {
        *ptrp = ptr + len + 1;
        return i;
    }
    return -1;
}

static unsigned int get_ctl_type_max_elements(snd_ctl_elem_type_t type)
{
    struct snd_ctl_elem_value value;

    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/control/ctlparse.c (line 335) - [public] ARRAY_SIZE
    return ARRAY_SIZE(value.value.integer.value);
case SND_CTL_ELEM_TYPE_INTEGER64:
    return ARRAY_SIZE(value.value.integer64.value);
case SND_CTL_ELEM_TYPE_ENUMERATED:
    return ARRAY_SIZE(value.value.enumerated.item);
case SND_CTL_ELEM_TYPE_BYTES:
    return ARRAY_SIZE(value.value.bytes.data);
default:
    return 0;
}

// ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/control/ctlparse.c (line 337) - [public] ARRAY_SIZE
    return ARRAY_SIZE(value.value.integer64.value);
case SND_CTL_ELEM_TYPE_ENUMERATED:
    return ARRAY_SIZE(value.value.enumerated.item);
case SND_CTL_ELEM_TYPE_BYTES:
    return ARRAY_SIZE(value.value.bytes.data);
default:
    return 0;
}

/**
 * \brief parse ASCII string as CTL element value
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 339) - [public] ARRAY_SIZE

```
    return ARRAY_SIZE(value.value.enumerated.item);
case SND_CTL_ELEM_TYPE_BYTES:
    return ARRAY_SIZE(value.value.bytes.data);
default:
    return 0;
}

/**
 * \brief parse ASCII string as CTL element value
 * \param handle CTL handle
 * \param dst destination CTL element value
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 341) - [public] ARRAY_SIZE

```
    return ARRAY_SIZE(value.value.bytes.data);
default:
    return 0;
}

/**
 * \brief parse ASCII string as CTL element value
 * \param handle CTL handle
 * \param dst destination CTL element value
 * \param info CTL element info structure
 * \param value source ASCII string
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 381) - [public] switch

```
switch (type) {
case SND_CTL_ELEM_TYPE_BOOLEAN:
    tmp = 0;
    if (!strncasecmp(ptr, "on", 2) ||
        !strncasecmp(ptr, "up", 2)) {
        tmp = 1;
        ptr += 2;
    } else if (!strncasecmp(ptr, "yes", 3)) {
        tmp = 1;
        ptr += 3;
    } else if (!strncasecmp(ptr, "toggle", 6)) {
        tmp = snd_ctl_elem_value_get_boolean(dst, idx);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 403) - [public] snd_ctl_elem_value_set_boolean

```
    snd_ctl_elem_value_set_boolean(dst, idx, tmp);
    break;
case SND_CTL_ELEM_TYPE_INTEGER:
    tmp = get_integer(&ptr,
        snd_ctl_elem_info_get_min(info),
        snd_ctl_elem_info_get_max(info));
    snd_ctl_elem_value_set_integer(dst, idx, tmp);
    break;
case SND_CTL_ELEM_TYPE_INTEGER64:
    tmp64 = get_integer64(&ptr,
        snd_ctl_elem_info_get_min64(info),
        snd_ctl_elem_info_get_max64(info));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 408) - [public] snd_ctl_elem_info_get_max

```
        snd_ctl_elem_info_get_max(info));
    snd_ctl_elem_value_set_integer(dst, idx, tmp);
    break;
case SND_CTL_ELEM_TYPE_INTEGER64:
    tmp64 = get_integer64(&ptr,
        snd_ctl_elem_info_get_min64(info),
        snd_ctl_elem_info_get_max64(info));
    snd_ctl_elem_value_set_integer64(dst, idx, tmp64);
    break;
case SND_CTL_ELEM_TYPE_ENUMERATED:
    tmp = get_ctl_enum_item_index(handle, info, &ptr);
    if (tmp < 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 409) - [public] snd_ctl_elem_value_set_integer

```
    snd_ctl_elem_value_set_integer(dst, idx, tmp);
    break;
case SND_CTL_ELEM_TYPE_INTEGER64:
    tmp64 = get_integer64(&ptr,
        snd_ctl_elem_info_get_min64(info),
        snd_ctl_elem_info_get_max64(info));
    snd_ctl_elem_value_set_integer64(dst, idx, tmp64);
    break;
case SND_CTL_ELEM_TYPE_ENUMERATED:
    tmp = get_ctl_enum_item_index(handle, info, &ptr);
    if (tmp < 0)
        tmp = get_integer(&ptr, 0,
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 414) - [public] snd_ctl_elem_info_get_max64

```
        snd_ctl_elem_info_get_max64(info));
    snd_ctl_elem_value_set_integer64(dst, idx, tmp64);
    break;
case SND_CTL_ELEM_TYPE_ENUMERATED:
    tmp = get_ctl_enum_item_index(handle, info, &ptr);
    if (tmp < 0)
        tmp = get_integer(&ptr, 0,
            snd_ctl_elem_info_get_items(info) - 1);
    snd_ctl_elem_value_set_enumerated(dst, idx, tmp);
    break;
case SND_CTL_ELEM_TYPE_BYTES:
    tmp = get_integer(&ptr, 0, 255);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 415) - [public]
snd_ctl_elem_value_set_integer64

```
    snd_ctl_elem_value_set_integer64(dst, idx, tmp64);
    break;
case SND_CTL_ELEM_TYPE_ENUMERATED:
    tmp = get_ctl_enum_item_index(handle, info, &ptr);
    if (tmp < 0)
        tmp = get_integer(&ptr, 0,
            snd_ctl_elem_info_get_items(info) - 1);
    snd_ctl_elem_value_set_enumerated(dst, idx, tmp);
    break;
case SND_CTL_ELEM_TYPE_BYTES:
    tmp = get_integer(&ptr, 0, 255);
    snd_ctl_elem_value_set_byte(dst, idx, tmp);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 421) - [public] snd_ctl_elem_info_get_items

```
    snd_ctl_elem_info_get_items(info) - 1);
    snd_ctl_elem_value_set_enumerated(dst, idx, tmp);
    break;
case SND_CTL_ELEM_TYPE_BYTES:
    tmp = get_integer(&ptr, 0, 255);
    snd_ctl_elem_value_set_byte(dst, idx, tmp);
    break;
default:
    break;
}
skip:
if (!strchr(value, ','))
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 422) - [public]
snd_ctl_elem_value_set_enumerated

```
    snd_ctl_elem_value_set_enumerated(dst, idx, tmp);
    break;
case SND_CTL_ELEM_TYPE_BYTES:
    tmp = get_integer(&ptr, 0, 255);
    snd_ctl_elem_value_set_byte(dst, idx, tmp);
    break;
default:
    break;
}
skip:
if (!strchr(value, ','))
    ptr = value;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/ctlparse.c (line 426) - [public] snd_ctl_elem_value_set_byte

```
    snd_ctl_elem_value_set_byte(dst, idx, tmp);
    break;
default:
    break;
}
skip:
if (!strchr(value, ','))
    ptr = value;
else if (*ptr == ',')
    ptr++;
}
return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/eld.c (line 56) - [public] snd_checknum

```
    snd_checknum(CONTROL, "Cannot open the associated CTL");
    return ret;
}

__fill_eld_ctl_id(&cinfo.id, info->device, info->subdevice);
value.id = cinfo.id;
ret = snd_ctl_elem_info(ctl, &cinfo);
if (ret >= 0 && cinfo.type == SND_CTL_ELEM_TYPE_BYTES)
    ret = snd_ctl_elem_read(ctl, &value);
snd_ctl_close(ctl);
if (ret == -ENOENT || cinfo.type != SND_CTL_ELEM_TYPE_BYTES || cinfo.count == 0)
    return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/eld.c (line 69) - [public] snd_checknum

```
    ■■snd_checknum(CONTROL, "Cannot read ELD");
    ■■return ret;
    ■}
    ■/* decode connected HDMI device name */
    ■eld = value.value.bytes.data;
    ■if (cinfo.count < 20 || cinfo.count > 256)
    ■■return -EIO;
    ■l = eld[4] & 0x1f;
    ■if (l == 0)
    ■■/* no monitor name detected */
    ■■goto __present;
    ■if (l > 16 || 20 + l > cinfo.count) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/eld.c (line 81) - [public] snd_error

```
    ■■snd_error(CONTROL, "ELD decode failed, using old HDMI output names");
    ■■return 0;
    ■}
    ■s = alloca(l + 1);
    ■/* sanitize */
    ■valid = 0;
    ■spc = 0;
    ■while (l > 0) {
        ■■l--;
        ■■c = eld[20 + l];
        ■■if (c <= ' ' || c >= 0x7f) {
            ■■■s[l] = ' ';
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/eld.c (line 91) - [public] if

```
    ■■if (c <= ' ' || c >= 0x7f) {
        ■■■s[l] = ' ';
    ■■} else {
        ■■■valid += !!isalnum(c);
        ■■■s[l] = c;
        ■■■if (spc == 0)
            ■■■■spc = l + 1;
    ■■}
    ■}
    ■if (valid > 3) {
        ■■s[spc] = '\0';
        ■■snd_strncpy((char *)info->name, s, sizeof(info->name));
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/eld.c (line 102) - [public] snd_strncpy

```
    ■■snd_strncpy((char *)info->name, s, sizeof(info->name));
    ■} else {
        __present:
        ■■strncat((char *)info->name, " *", sizeof(info->name) - 1);
        ■■((char *)info->name)[sizeof(info->name)-1] = '\0';
    ■}
    ■return 0;
    ■}
```

dirtycode/alsa-lib/src/control/eld.c (line 105) - [public] strncat

```
    ■■strncat((char *)info->name, " *", sizeof(info->name) - 1);
    ■■((char *)info->name)[sizeof(info->name)-1] = '\0';
    ■}
    ■return 0;
    ■}
```


dirtycode/alsa-lib/src/control/hcontrol.c (line 79) - [public] snd_ctl_close

```
■snd_ctl_close(ctl);
■return err;
}

/**
 * \brief Opens an HCTL
 * \param hctlp Returned HCTL handle
 * \param ctl underlying CTL handle
 * \return 0 on success otherwise a negative error code
 */
int snd_hctl_open_ctl(snd_hctl_t **hctlp, snd_ctl_t *ctl)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 133) - [public] snd_ctl_name

```
■return snd_ctl_name(hctl->ctl);
}

/**
 * \brief set nonblock mode
 * \param hctl HCTL handle
 * \param nonblock 0 = block, 1 = nonblock mode
 * \return 0 on success otherwise a negative error code
 */
int snd_hctl_nonblock(snd_hctl_t *hctl, int nonblock)
{
    ■assert(hctl);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 145) - [public] snd_ctl_nonblock

```
■return snd_ctl_nonblock(hctl->ctl, nonblock);
}

/**
 * \brief set async mode
 * \param hctl HCTL handle
 * \param sig Signal to raise: < 0 disable, 0 default (SIGIO)
 * \param pid Process ID to signal: 0 current
 * \return 0 on success otherwise a negative error code
 *
 * A signal is raised when a change happens.
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 160) - [public] snd_ctl_async

```
■return snd_ctl_async(hctl->ctl, sig, pid);
}

/**
 * \brief get count of poll descriptors for HCTL handle
 * \param hctl HCTL handle
 * \return count of poll descriptors
 */
int snd_hctl_poll_descriptors_count(snd_hctl_t *hctl)
{
    ■assert(hctl);
    ■return snd_ctl_poll_descriptors_count(hctl->ctl);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 171) - [public] snd_ctl_poll_descriptors_count

```
■return snd_ctl_poll_descriptors_count(hctl->ctl);
}

/**
 * \brief get poll descriptors
 * \param hctl HCTL handle
 * \param pfd array of poll descriptors
 * \param space space in the poll descriptor array
 * \return count of filled descriptors
 */
int snd_hctl_poll_descriptors(snd_hctl_t *hctl, struct pollfd *pfd, unsigned int space)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 184) - [public] snd_ctl_poll_descriptors

```
■return snd_ctl_poll_descriptors(hctl->ctl, pfd, space);
}

/**
 * \brief get returned events from poll descriptors
 * \param hctl HCTL handle
 * \param pfd array of poll descriptors
 * \param nfds count of poll descriptors
 * \param revents returned events
 * \return zero if success, otherwise a negative error code
 */
int snd_hctl_poll_descriptors_revents(snd_hctl_t *hctl, struct pollfd *pfd, unsigned int nfds, unsigned
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 198) - [public]

snd_ctl_poll_descriptors_revents

```
■return snd_ctl_poll_descriptors_revents(hctl->ctl, pfd, nfds, revents);
}

static int snd_hctl_throw_event(snd_hctl_t *hctl, unsigned int mask,
    ■■ snd_hctl_elem_t *elem)
{
    ■if (hctl->callback)
    ■■return hctl->callback(hctl, mask, elem);
    ■return 0;
}

static int snd_hctl_elem_throw_event(snd_hctl_elem_t *elem,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 222) - [public] if

```
■■if (!strcmp(*name, *names, strlen(*names))) {
    ■■■*name += strlen(*names);
    ■■■if (**name == ' ')
    ■■■■(*name)++;
    ■■■return res+1;
    ■■}
    ■}
    ■return NOT_FOUND;
}

static int get_compare_weight(const snd_ctl_elem_id_t *id)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 355) - [public] if

```
    if (!h) {
        hctl->alloc -= 32;
        return -ENOMEM;
    }
    hctl->pelems = h;
}
if (hctl->count == 0) {
    list_add_tail(&elem->list, &hctl->elems);
    hctl->pelems[0] = elem;
} else {
    idx = _snd_hctl_find_elem(hctl, &elem->id, &dir);
    assert(dir != 0);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 362) - [public] list_add_tail

```
    list_add_tail(&elem->list, &hctl->elems);
    hctl->pelems[0] = elem;
} else {
    idx = _snd_hctl_find_elem(hctl, &elem->id, &dir);
    assert(dir != 0);
    if (dir > 0) {
        list_add(&elem->list, &hctl->pelems[idx]->list);
        idx++;
    } else {
        list_add_tail(&elem->list, &hctl->pelems[idx]->list);
    }
    memmove(hctl->pelems + idx + 1,
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 366) - [public] assert

```
    assert(dir != 0);
    if (dir > 0) {
        list_add(&elem->list, &hctl->pelems[idx]->list);
        idx++;
    } else {
        list_add_tail(&elem->list, &hctl->pelems[idx]->list);
    }
    memmove(hctl->pelems + idx + 1,
            hctl->pelems + idx,
            (hctl->count - idx) * sizeof(snd_hctl_elem_t *));
    hctl->pelems[idx] = elem;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 367) - [public] if

```
    if (dir > 0) {
        list_add(&elem->list, &hctl->pelems[idx]->list);
        idx++;
    } else {
        list_add_tail(&elem->list, &hctl->pelems[idx]->list);
    }
    memmove(hctl->pelems + idx + 1,
            hctl->pelems + idx,
            (hctl->count - idx) * sizeof(snd_hctl_elem_t *));
    hctl->pelems[idx] = elem;
}
hctl->count++;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 368) - [public] list_add

```
    list_add(&elem->list, &hctl->pelems[idx]->list);
    idx++;
} else {
    list_add_tail(&elem->list, &hctl->pelems[idx]->list);
}
memmove(hctl->pelems + idx + 1,
        hctl->pelems + idx,
        (hctl->count - idx) * sizeof(snd_hctl_elem_t *));
hctl->pelems[idx] = elem;
}
hctl->count++;
return snd_hctl_throw_event(hctl, SNDRV_CTL_EVENT_MASK_ADD, elem);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 371) - [public] list_add_tail

```
list_add_tail(&elem->list, &hctl->pelems[idx]->list);
}
memmove(hctl->pelems + idx + 1,
        hctl->pelems + idx,
        (hctl->count - idx) * sizeof(snd_hctl_elem_t *));
hctl->pelems[idx] = elem;
}
hctl->count++;
return snd_hctl_throw_event(hctl, SNDRV_CTL_EVENT_MASK_ADD, elem);
}
```

```
static void snd_hctl_elem_remove(snd_hctl_t *hctl, unsigned int idx)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 379) - [public] snd_hctl_throw_event

```
return snd_hctl_throw_event(hctl, SNDRV_CTL_EVENT_MASK_ADD, elem);
}
```

```
static void snd_hctl_elem_remove(snd_hctl_t *hctl, unsigned int idx)
{
    snd_hctl_elem_t *elem = hctl->pelems[idx];
    unsigned int m;
    snd_hctl_elem_throw_event(elem, SNDRV_CTL_EVENT_MASK_REMOVE);
    list_del(&elem->list);
    free(elem);
    hctl->count--;
    m = hctl->count - idx;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 394) - [public] sizeof

```
    m * sizeof(snd_hctl_elem_t *));
}

/**
 * \brief free HCTL loaded elements
 * \param hctl HCTL handle
 * \return 0 on success otherwise a negative error code
 */
int snd_hctl_free(snd_hctl_t *hctl)
{
    while (hctl->count > 0)
        snd_hctl_elem_remove(hctl, hctl->count - 1);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 405) - [public] snd_hctl_elem_remove

```
■■snd_hctl_elem_remove(hctl, hctl->count - 1);
■free(hctl->pelems);
■hctl->pelems = 0;
■hctl->alloc = 0;
■INIT_LIST_HEAD(&hctl->elems);
■return 0;
}

static snd_hctl_t *compare_hctl;
static int hctl_compare(const void *a, const void *b) {
■return compare_hctl->compare(*(const snd_hctl_elem_t * const *) a,
■■■ *(const snd_hctl_elem_t * const *) b);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 414) - [internal] hctl_compare

```
static int hctl_compare(const void *a, const void *b) {
■return compare_hctl->compare(*(const snd_hctl_elem_t * const *) a,
■■■ *(const snd_hctl_elem_t * const *) b);
}
```

```
static void snd_hctl_sort(snd_hctl_t *hctl)
{
■unsigned int k;
#ifdef HAVE_LIBPTHREAD
■static pthread_mutex_t sync_lock = PTHREAD_MUTEX_INITIALIZER;
#endif

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 439) - [public] list_add_tail

```
■■list_add_tail(&hctl->pelems[k]->list, &hctl->elems);
}

/**
 * \brief Change HCTL compare function and reorder elements
 * \param hctl HCTL handle
 * \param compare Element compare function
 * \return 0 on success otherwise a negative error code
 */
int snd_hctl_set_compare(snd_hctl_t *hctl, snd_hctl_compare_t compare)
{
■assert(hctl);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 503) - [public] list_entry

```
■return list_entry(hctl->elems.next, snd_hctl_elem_t, list);
}

/**
 * \brief get last element for an HCTL
 * \param hctl HCTL handle
 * \return pointer to last element
 */
snd_hctl_elem_t *snd_hctl_last_elem(snd_hctl_t *hctl)
{
■assert(hctl);
■if (list_empty(&hctl->elems))
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 516) - [public] list_entry

```
    return list_entry(hctl->elems.prev, snd_hctl_elem_t, list);
}

/**
 * \brief get next HCTL element
 * \param elem HCTL element
 * \return pointer to next element
 */
snd_hctl_elem_t *snd_hctl_elem_next(snd_hctl_elem_t *elem)
{
    assert(elem);
    if (elem->list.next == &elem->hctl->elems)
        // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 529) - [public] list_entry

```
    return list_entry(elem->list.next, snd_hctl_elem_t, list);
}

/**
 * \brief get previous HCTL element
 * \param elem HCTL element
 * \return pointer to previous element
 */
snd_hctl_elem_t *snd_hctl_elem_prev(snd_hctl_elem_t *elem)
{
    assert(elem);
    if (elem->list.prev == &elem->hctl->elems)
        // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 542) - [public] list_entry

```
    return list_entry(elem->list.prev, snd_hctl_elem_t, list);
}

/**
 * \brief Search an HCTL element
 * \param hctl HCTL handle
 * \param id Element identifier
 * \return pointer to found HCTL element or NULL if it does not exists
 */
snd_hctl_elem_t *snd_hctl_find_elem(snd_hctl_t *hctl, const snd_ctl_elem_id_t *id)
{
    int dir;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 587) - [public] free

```
    free(hctl->pelems);
    hctl->pelems = malloc(hctl->alloc * sizeof(*hctl->pelems));
    if (!hctl->pelems) {
        err = -ENOMEM;
        goto _end;
    }
    for (idx = 0; idx < list.count; idx++) {
        snd_hctl_elem_t *elem;
        elem = calloc(1, sizeof(snd_hctl_elem_t));
        if (elem == NULL) {
            snd_hctl_free(hctl);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 589) - [public] if

```
    if (!hctl->pelems) {
        err = -ENOMEM;
        goto _end;
    }
    for (idx = 0; idx < list.count; idx++) {
        snd_hctl_elem_t *elem;
        elem = calloc(1, sizeof(snd_hctl_elem_t));
        if (elem == NULL) {
            snd_hctl_free(hctl);
            err = -ENOMEM;
            goto _end;
        }
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 597) - [public] if

```
    if (elem == NULL) {
        snd_hctl_free(hctl);
        err = -ENOMEM;
        goto _end;
    }
    elem->id = list.pids[idx];
    elem->hctl = hctl;
    elem->compare_weight = get_compare_weight(&elem->id);
    hctl->pelems[idx] = elem;
    list_add_tail(&elem->list, &hctl->elems);
    hctl->count++;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 598) - [public] snd_hctl_free

```
    snd_hctl_free(hctl);
    err = -ENOMEM;
    goto _end;
}
elem->id = list.pids[idx];
elem->hctl = hctl;
elem->compare_weight = get_compare_weight(&elem->id);
hctl->pelems[idx] = elem;
list_add_tail(&elem->list, &hctl->elems);
hctl->count++;
}
if (!hctl->compare)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 606) - [public] list_add_tail

```
    list_add_tail(&elem->list, &hctl->elems);
    hctl->count++;
}
if (!hctl->compare)
    hctl->compare = snd_hctl_compare_default;
snd_hctl_sort(hctl);
for (idx = 0; idx < hctl->count; idx++) {
    int res = snd_hctl_throw_event(hctl, SNDRV_CTL_EVENT_MASK_ADD,
        hctl->pelems[idx]);
    if (res < 0)
        return res;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 683) - [public] snd_error

```
    ■■snd_error(CONTROL, "Invalid poll_fds %d", npfds);
    ■■return -EIO;
    ■}
    ■pfd = alloca(sizeof(*pfd) * npfds);
    ■revents = alloca(sizeof(*revents) * npfds);
    ■err = snd_hctl_poll_descriptors(hctl, pfd, npfds);
    ■if (err < 0)
    ■■return err;
    ■if (err != npfds) {
    ■■snd_check(CONTROL, "invalid poll descriptors %d", err);
    ■■return -EIO;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 692) - [public] snd_check

```
    ■■snd_check(CONTROL, "invalid poll descriptors %d", err);
    ■■return -EIO;
    ■}
    ■do {
    ■■pollio = 0;
    ■■err_poll = poll(pfd, npfds, timeout);
    ■■if (err_poll < 0) {
    ■■■if (errno == EINTR && !CTLINABORT(hctl->ctl) && !(hctl->ctl->mode & SND_CTL_EINTR))
    ■■■■continue;
    ■■■return -errno;
    ■■}
    ■■if (! err_poll)
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 698) - [public] if

```
    ■■if (err_poll < 0) {
    ■■■if (errno == EINTR && !CTLINABORT(hctl->ctl) && !(hctl->ctl->mode & SND_CTL_EINTR))
    ■■■■continue;
    ■■■return -errno;
    ■■}
    ■■if (! err_poll)
    ■■■break;
    ■■err = snd_hctl_poll_descriptors_revents(hctl, pfd, npfds, revents);
    ■■if (err < 0)
    ■■■return err;
    ■■for (i = 0; i < npfds; i++) {
    ■■■if (revents[i] & (POLLERR | POLLNVAL))
    ■■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 747) - [public] snd_hctl_elem_remove

```
    ■■snd_hctl_elem_remove(hctl, (unsigned int) res);
    ■■return 0;
    ■}
    ■if (event->data.elem.mask & SNDRV_CTL_EVENT_MASK_ADD) {
    ■■elem = calloc(1, sizeof(snd_hctl_elem_t));
    ■■if (elem == NULL)
    ■■■return -ENOMEM;
    ■■elem->id = event->data.elem.id;
    ■■elem->hctl = hctl;
    ■■res = snd_hctl_elem_add(hctl, elem);
    ■■if (res < 0)
    ■■■return res;
    ■■■// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/hcontrol.c (line 811) - [public] snd_ctl_elem_info

```
    return snd_ctl_elem_info(elem->hctl->ctl, info);
}

/**
 * \brief Get value for an HCTL element
 * \param elem HCTL element
 * \param value HCTL element value
 * \return 0 otherwise a negative error code on failure
 */
int snd_hctl_elem_read(snd_hctl_elem_t *elem, snd_ctl_elem_value_t * value)
{
    assert(elem);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 826) - [public] snd_ctl_elem_read

```
    return snd_ctl_elem_read(elem->hctl->ctl, value);
}
```

```
/**
 * \brief Set value for an HCTL element
 * \param elem HCTL element
 * \param value HCTL element value
 * \retval 0 on success
 * \retval >1 on success when value was changed
 * \retval <0 a negative error code on failure
 */
int snd_hctl_elem_write(snd_hctl_elem_t *elem, snd_ctl_elem_value_t * value)
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 843) - [public] snd_ctl_elem_write

```
    return snd_ctl_elem_write(elem->hctl->ctl, value);
}
```

```
/**
 * \brief Get TLV value for an HCTL element
 * \param elem HCTL element
 * \param tlv TLV array for value
 * \param tlv_size size of TLV array in bytes
 * \return 0 otherwise a negative error code on failure
 */
int snd_hctl_elem_tlv_read(snd_hctl_elem_t *elem, unsigned int *tlv, unsigned int tlv_size)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 858) - [public] snd_ctl_elem_tlv_read

```
    return snd_ctl_elem_tlv_read(elem->hctl->ctl, &elem->id, tlv, tlv_size);
}
```

```
/**
 * \brief Set TLV value for an HCTL element
 * \param elem HCTL element
 * \param tlv TLV array for value
 * \retval 0 on success
 * \retval >1 on success when value was changed
 * \retval <0 a negative error code on failure
 */
int snd_hctl_elem_tlv_write(snd_hctl_elem_t *elem, const unsigned int *tlv)
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 874) - [public] snd_ctl_elem_tlv_write

```
    return snd_ctl_elem_tlv_write(elem->hctl->ctl, &elem->id, tlv);
}

/**
 * \brief Set TLV value for an HCTL element
 * \param elem HCTL element
 * \param tlv TLV array for value
 * \retval 0 on success
 * \retval >1 on success when value was changed
 * \retval <0 a negative error code on failure
 */
int snd_hctl_elem_tlv_command(snd_hctl_elem_t *elem, const unsigned int *tlv)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 890) - [public] snd_ctl_elem_tlv_command

```
    return snd_ctl_elem_tlv_command(elem->hctl->ctl, &elem->id, tlv);
}

/**
 * \brief Get HCTL handle for an HCTL element
 * \param elem HCTL element
 * \return HCTL handle
 */
snd_hctl_t *snd_hctl_elem_get_hctl(snd_hctl_elem_t *elem)
{
    assert(elem);
    return elem->hctl;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 61) - [public] memset

```
    memset(n + list->allocated, 0, 10 * sizeof(*n));
    list->allocated += 10;
    list->list = n;
}
if (name == NULL) {
    x = NULL;
} else {
    x = malloc(4 + strlen(name) + (description != NULL ? (4 + strlen(description) + 1) : 0) + 1);
    if (x == NULL)
        return -ENOMEM;
    memcpy(x, "NAME", 4);
    strcpy(x + 4, name);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 71) - [public] memcpy

```
    memcpy(x, "NAME", 4);
    strcpy(x + 4, name);
    if (description != NULL) {
        strcat(x, "|DESC");
        strcat(x, description);
    }
}
list->list[list->count++] = x;
return 0;
}

/**
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 72) - [public] strcpy

```
    strcpy(x + 4, name);
    if (description != NULL) {
        strcat(x, "|DESC");
        strcat(x, description);
    }
}

list->list[list->count++] = x;
return 0;
}

/**
 * Add a namehint from string given in a user configuration file
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 73) - [public] if

```
    if (description != NULL) {
        strcat(x, "|DESC");
        strcat(x, description);
    }
}

list->list[list->count++] = x;
return 0;
}

/**
 * Add a namehint from string given in a user configuration file
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 74) - [public] strcat

```
    strcat(x, "|DESC");
    strcat(x, description);
}
}

list->list[list->count++] = x;
return 0;
}

/**
 * Add a namehint from string given in a user configuration file
 */
static int hint_list_add_custom(struct hint_list *list,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 75) - [public] strcat

```
    strcat(x, description);
}
}

list->list[list->count++] = x;
return 0;
}

/**
 * Add a namehint from string given in a user configuration file
 */
static int hint_list_add_custom(struct hint_list *list,
    const char *entry)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 96) - [public] hint_list_add

```
    return hint_list_add(list, entry, NULL);

    name = strndup(entry, sep - entry);
    if (name == NULL)
        return -ENOMEM;

    err = hint_list_add(list, name, sep + 1);
    free(name);
    return err;
}

static void zero_handler(int prio ATTRIBUTE_UNUSED,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 129) - [public] snd_hwdep_info_set_device

```
    snd_hwdep_info_set_device(&info, device);
    if (snd_ctl_hwdep_info(list->ctl, &info) < 0)
        return 0;
    *res = strdup(snd_hwdep_info_get_name(&info));
    return 0;
}
#endif
#ifdef BUILD_PCM
case SND_CTL_ELEM_IFACE_PCM:
{
    snd_pcm_info_t info = {0};
    snd_pcm_info_set_device(&info, device);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 140) - [public] snd_pcm_info_set_device

```
    snd_pcm_info_set_device(&info, device);
    snd_pcm_info_set_stream(&info, stream ? SND_PCM_STREAM_CAPTURE : SND_PCM_STREAM_PLAYBACK);
    if (snd_ctl_pcm_info(list->ctl, &info) < 0)
        return 0;
    switch (snd_pcm_info_get_class(&info)) {
    case SND_PCM_CLASS_MODEM:
    case SND_PCM_CLASS_DIGITIZER:
        return -ENODEV;
    default:
        break;
    }
    *res = strdup(snd_pcm_info_get_name(&info));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 141) - [public] snd_pcm_info_set_stream

```
    snd_pcm_info_set_stream(&info, stream ? SND_PCM_STREAM_CAPTURE : SND_PCM_STREAM_PLAYBACK);
    if (snd_ctl_pcm_info(list->ctl, &info) < 0)
        return 0;
    switch (snd_pcm_info_get_class(&info)) {
    case SND_PCM_CLASS_MODEM:
    case SND_PCM_CLASS_DIGITIZER:
        return -ENODEV;
    default:
        break;
    }
    *res = strdup(snd_pcm_info_get_name(&info));
    return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 144) - [public] switch

```
    switch (snd_pcm_info_get_class(&info)) {
        case SND_PCM_CLASS_MODEM:
        case SND_PCM_CLASS_DIGITIZER:
            return -ENODEV;
        default:
            break;
    }
    *res = strdup(snd_pcm_info_get_name(&info));
    return 0;
}
#endif
#ifdef BUILD_RAWMIDI
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 159) - [public] snd_rawmidi_info_set_device

```
    snd_rawmidi_info_set_device(&info, device);
    snd_rawmidi_info_set_stream(&info, stream ? SND_RAWMIDI_STREAM_INPUT : SND_RAWMIDI_STREAM_OUTPUT);
    if (snd_ctl_rawmidi_info(list->ctl, &info) < 0)
        return 0;
    *res = strdup(snd_rawmidi_info_get_name(&info));
    return 0;
}
#endif
default:
    return 0;
}
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 160) - [public] snd_rawmidi_info_set_stream

```
    snd_rawmidi_info_set_stream(&info, stream ? SND_RAWMIDI_STREAM_INPUT : SND_RAWMIDI_STREAM_OUTPUT);
    if (snd_ctl_rawmidi_info(list->ctl, &info) < 0)
        return 0;
    *res = strdup(snd_rawmidi_info_get_name(&info));
    return 0;
}
#endif
default:
    return 0;
}
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 183) - [public] free

```
    free(str1);
    return NULL;
}
if (str1 != NULL || str2 != NULL) {
    if (str1 != NULL && str2 != NULL) {
        if (strcmp(str1, str2) == 0) {
            res = malloc(strlen(list->cardname) + strlen(str2) + 3);
            if (res != NULL) {
                strcpy(res, list->cardname);
                strcat(res, ", ");
                strcat(res, str2);
            }
        }
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 187) - [public] if

```
    if (str1 != NULL && str2 != NULL) {
        if (strcmp(str1, str2) == 0) {
            res = malloc(strlen(list->cardname) + strlen(str2) + 3);
            if (res != NULL) {
                strcpy(res, list->cardname);
                strcat(res, ", ");
                strcat(res, str2);
            }
        } else {
            res = malloc(strlen(list->cardname) + strlen(str2) + strlen(str1) + 6);
            if (res != NULL) {
                strcpy(res, list->cardname);
                // ... (truncated, see source file for more)
            }
        }
    }
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 188) - [public] if

```
    if (strcmp(str1, str2) == 0) {
        res = malloc(strlen(list->cardname) + strlen(str2) + 3);
        if (res != NULL) {
            strcpy(res, list->cardname);
            strcat(res, ", ");
            strcat(res, str2);
        }
    } else {
        res = malloc(strlen(list->cardname) + strlen(str2) + strlen(str1) + 6);
        if (res != NULL) {
            strcpy(res, list->cardname);
            strcat(res, ", ");
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 190) - [public] if

```
    if (res != NULL) {
        strcpy(res, list->cardname);
        strcat(res, ", ");
        strcat(res, str2);
    }
} else {
    res = malloc(strlen(list->cardname) + strlen(str2) + strlen(str1) + 6);
    if (res != NULL) {
        strcpy(res, list->cardname);
        strcat(res, ", ");
        strcat(res, str2);
        strcat(res, " / ");
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 191) - [public] strcpy

```
    strcpy(res, list->cardname);
    strcat(res, ", ");
    strcat(res, str2);
}
} else {
    res = malloc(strlen(list->cardname) + strlen(str2) + strlen(str1) + 6);
    if (res != NULL) {
        strcpy(res, list->cardname);
        strcat(res, ", ");
        strcat(res, str2);
        strcat(res, " / ");
        strcat(res, str1);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 192) - [public] strcat

```
        ■■■■■strcat(res, ", ");
        ■■■■■strcat(res, str2);
        ■■■■}
        ■■■■} else {
        ■■■■res = malloc(strlen(list->cardname) + strlen(str2) + strlen(str1) + 6);
        ■■■■if (res != NULL) {
        ■■■■■strcpy(res, list->cardname);
        ■■■■■strcat(res, ", ");
        ■■■■■strcat(res, str2);
        ■■■■■strcat(res, " / ");
        ■■■■■strcat(res, str1);
        ■■■■}
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 193) - [public] strcat

```
        ■■■■■strcat(res, str2);
        ■■■■}
        ■■■■} else {
        ■■■■res = malloc(strlen(list->cardname) + strlen(str2) + strlen(str1) + 6);
        ■■■■if (res != NULL) {
        ■■■■■strcpy(res, list->cardname);
        ■■■■■strcat(res, ", ");
        ■■■■■strcat(res, str2);
        ■■■■■strcat(res, " / ");
        ■■■■■strcat(res, str1);
        ■■■■}
        ■■■■}
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 197) - [public] if

```
        ■■■■if (res != NULL) {
        ■■■■■strcpy(res, list->cardname);
        ■■■■■strcat(res, ", ");
        ■■■■■strcat(res, str2);
        ■■■■■strcat(res, " / ");
        ■■■■■strcat(res, str1);
        ■■■■}
        ■■■■}
        ■■■■free(str2);
        ■■■■free(str1);
        ■■■■return res;
        ■■■} else {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 198) - [public] strcpy

```
        ■■■■■strcpy(res, list->cardname);
        ■■■■■strcat(res, ", ");
        ■■■■■strcat(res, str2);
        ■■■■■strcat(res, " / ");
        ■■■■■strcat(res, str1);
        ■■■■}
        ■■■■}
        ■■■■free(str2);
        ■■■■free(str1);
        ■■■■return res;
        ■■■} else {
        ■■■■if (str1 != NULL) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 199) - [public] strcat

```
        ■■■■■strcat(res, ", ");
        ■■■■■strcat(res, str2);
        ■■■■■strcat(res, " / ");
        ■■■■■strcat(res, str1);
        ■■■■■}
        ■■■■■}
        ■■■free(str2);
        ■■■free(str1);
        ■■■return res;
    ■■■} else {
        ■■■if (str1 != NULL) {
            ■■■■■str2 = "Input";
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 200) - [public] strcat

```
        ■■■■■strcat(res, str2);
        ■■■■■strcat(res, " / ");
        ■■■■■strcat(res, str1);
        ■■■■■}
        ■■■■■}
        ■■■free(str2);
        ■■■free(str1);
        ■■■return res;
    ■■■} else {
        ■■■if (str1 != NULL) {
            ■■■■■str2 = "Input";
        ■■■■■} else {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 201) - [public] strcat

```
        ■■■■■strcat(res, " / ");
        ■■■■■strcat(res, str1);
        ■■■■■}
        ■■■■■}
        ■■■free(str2);
        ■■■free(str1);
        ■■■return res;
    ■■■} else {
        ■■■if (str1 != NULL) {
            ■■■■■str2 = "Input";
        ■■■■■} else {
            ■■■■■str1 = str2;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 202) - [public] strcat

```
        ■■■■■strcat(res, str1);
        ■■■■■}
        ■■■■■}
        ■■■free(str2);
        ■■■free(str1);
        ■■■return res;
    ■■■} else {
        ■■■if (str1 != NULL) {
            ■■■■■str2 = "Input";
        ■■■■■} else {
            ■■■■■str1 = str2;
            ■■■■■str2 = "Output";
            // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/namehint.c (line 205) - [public] free

```
    free(str2);
    free(str1);
    return res;
} else {
    if (str1 != NULL) {
        str2 = "Input";
    } else {
        str1 = str2;
        str2 = "Output";
    }
    res = malloc(strlen(list->cardname) + strlen(str1) + 19);
    if (res == NULL) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 206) - [public] free

```
    free(str1);
    return res;
} else {
    if (str1 != NULL) {
        str2 = "Input";
    } else {
        str1 = str2;
        str2 = "Output";
    }
    res = malloc(strlen(list->cardname) + strlen(str1) + 19);
    if (res == NULL) {
        free(str1);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 209) - [public] if

```
    if (str1 != NULL) {
        str2 = "Input";
    } else {
        str1 = str2;
        str2 = "Output";
    }
    res = malloc(strlen(list->cardname) + strlen(str1) + 19);
    if (res == NULL) {
        free(str1);
        return NULL;
    }
    strcpy(res, list->cardname);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 216) - [public] if

```
    if (res == NULL) {
        free(str1);
        return NULL;
    }
    strcpy(res, list->cardname);
    strcat(res, ", ");
    strcat(res, str1);
    strcat(res, "|IOID");
    strcat(res, str2);
    free(str1);
    return res;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 217) - [public] free

```
    free(str1);
    return NULL;
}
strcpy(res, list->cardname);
strcat(res, ", ");
strcat(res, str1);
strcat(res, "|I0ID");
strcat(res, str2);
free(str1);
return res;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 220) - [public] strcpy

```
strcpy(res, list->cardname);
strcat(res, ", ");
strcat(res, str1);
strcat(res, "|I0ID");
strcat(res, str2);
free(str1);
return res;
}
/* if the specified device doesn't exist, skip this entry */
if (list->device >= 0 || list->device_input >= 0 || list->device_output >= 0)
return NULL;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 221) - [public] strcat

```
strcat(res, ", ");
strcat(res, str1);
strcat(res, "|I0ID");
strcat(res, str2);
free(str1);
return res;
}
/* if the specified device doesn't exist, skip this entry */
if (list->device >= 0 || list->device_input >= 0 || list->device_output >= 0)
return NULL;
return strdup(list->cardname);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 222) - [public] strcat

```
strcat(res, str1);
strcat(res, "|I0ID");
strcat(res, str2);
free(str1);
return res;
}
/* if the specified device doesn't exist, skip this entry */
if (list->device >= 0 || list->device_input >= 0 || list->device_output >= 0)
return NULL;
return strdup(list->cardname);
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 223) - [public] strcat

```
    ■■■strcat(res, "|I0ID");
    ■■■strcat(res, str2);
    ■■■free(str1);
    ■■■return res;
    ■■}
    ■}
    ■/* if the specified device doesn't exist, skip this entry */
    ■if (list->device >= 0 || list->device_input >= 0 || list->device_output >= 0)
    ■■return NULL;
    ■return strdup(list->cardname);
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 224) - [public] strcat

```
    ■■■strcat(res, str2);
    ■■■free(str1);
    ■■■return res;
    ■■}
    ■}
    ■/* if the specified device doesn't exist, skip this entry */
    ■if (list->device >= 0 || list->device_input >= 0 || list->device_output >= 0)
    ■■return NULL;
    ■return strdup(list->cardname);
    }

    #ifndef DOC_HIDDEN
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 225) - [public] free

```
    ■■■free(str1);
    ■■■return res;
    ■■}
    ■}
    ■/* if the specified device doesn't exist, skip this entry */
    ■if (list->device >= 0 || list->device_input >= 0 || list->device_output >= 0)
    ■■return NULL;
    ■return strdup(list->cardname);
    }

    #ifndef DOC_HIDDEN
    #define BUF_SIZE 128
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 232) - [public] strdup

```
    ■return strdup(list->cardname);
    }

    #ifndef DOC_HIDDEN
    #define BUF_SIZE 128
    #endif

    static int try_config(snd_config_t *config,
    ■■ struct hint_list *list,
    ■■ const char *base,
    ■■ const char *name)
    {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 266) - [public] sprintf

```
    ■■sprintf(buf, "%s:CARD=%s,DEV=%i", name, snd_ctl_card_info_get_id(list->info), list->device);
    ■■else if (list->card >= 0)
    ■■sprintf(buf, "%s:CARD=%s", name, snd_ctl_card_info_get_id(list->info));
    ■■else
    ■■strcpy(buf, name);
    ■■eh = snd_lib_log_set_local(&zero_handler);
    ■■err = snd_config_search_definition(config, base, buf, &res);
    ■■snd_lib_log_set_local(eh);
    ■■if (err < 0)
    ■■goto __skip_add;
    ■■cleanup_res = 1;
    ■■err = -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 268) - [public] sprintf

```
    ■■sprintf(buf, "%s:CARD=%s", name, snd_ctl_card_info_get_id(list->info));
    ■■else
    ■■strcpy(buf, name);
    ■■eh = snd_lib_log_set_local(&zero_handler);
    ■■err = snd_config_search_definition(config, base, buf, &res);
    ■■snd_lib_log_set_local(eh);
    ■■if (err < 0)
    ■■goto __skip_add;
    ■■cleanup_res = 1;
    ■■err = -EINVAL;
    ■■if (snd_config_get_type(res) != SND_CONFIG_TYPE_COMPOUND)
    ■■goto __cleanup;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 270) - [public] strcpy

```
    ■■strcpy(buf, name);
    ■■eh = snd_lib_log_set_local(&zero_handler);
    ■■err = snd_config_search_definition(config, base, buf, &res);
    ■■snd_lib_log_set_local(eh);
    ■■if (err < 0)
    ■■goto __skip_add;
    ■■cleanup_res = 1;
    ■■err = -EINVAL;
    ■■if (snd_config_get_type(res) != SND_CONFIG_TYPE_COMPOUND)
    ■■goto __cleanup;
    ■■if (snd_config_search(res, "type", NULL) < 0)
    ■■goto __cleanup;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 286) - [public] fprintf

```
    ■■■fprintf(stderr, "***** PCM '%s':\n", buf);
    ■■■snd_output_stdio_attach(&out, stderr, 0);
    ■■■snd_config_save(res, out);
    ■■■snd_output_close(out);
    ■■■fprintf(stderr, "\n");
    ■■}
    #endif

    ■■cfg1 = res;
    ■■level = 0;
    __hint:
    ■■level++;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 287) - [public] snd_output_stdio_attach

```
■■■snd_output_stdio_attach(&out, stderr, 0);
■■■snd_config_save(res, out);
■■■snd_output_close(out);
■■■fprintf(stderr, "\n");
■■■}
#endif

■cfg1 = res;
■level = 0;
    __hint:
■level++;
■if (snd_config_search(cfg1, "type", &cfg) >= 0 &&
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 288) - [public] snd_config_save

```
■■■snd_config_save(res, out);
■■■snd_output_close(out);
■■■fprintf(stderr, "\n");
■■■}
#endif

■cfg1 = res;
■level = 0;
    __hint:
■level++;
■if (snd_config_search(cfg1, "type", &cfg) >= 0 &&
    snd_config_get_string(cfg, &str) >= 0 &&
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 289) - [public] snd_output_close

```
■■■snd_output_close(out);
■■■fprintf(stderr, "\n");
■■■}
#endif

■cfg1 = res;
■level = 0;
    __hint:
■level++;
■if (snd_config_search(cfg1, "type", &cfg) >= 0 &&
    snd_config_get_string(cfg, &str) >= 0 &&
    strcmp(str, "hw") == 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 290) - [public] fprintf

```
■■■fprintf(stderr, "\n");
■■■}
#endif

■cfg1 = res;
■level = 0;
    __hint:
■level++;
■if (snd_config_search(cfg1, "type", &cfg) >= 0 &&
    snd_config_get_string(cfg, &str) >= 0 &&
    strcmp(str, "hw") == 0) {
■■■if (snd_config_search(cfg1, "device", &cfg) >= 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 300) - [public] strcmp

```
■   strcmp(str, "hw") == 0) {
■   if (snd_config_search(cfg1, "device", &cfg) >= 0) {
■       if (snd_config_get_integer(cfg, &dev) < 0) {
■           snd_error(CONTROL, "(%s) device must be an integer", buf);
■           err = -EINVAL;
■           goto __cleanup;
■       }
■   }
■   }

■   if (snd_config_search(cfg1, "hint", &cfg) >= 0) {
■       if (snd_config_get_type(cfg) != SND_CONFIG_TYPE_COMPOUND) {
■           // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 301) - [public] if

```
■   if (snd_config_search(cfg1, "device", &cfg) >= 0) {
■       if (snd_config_get_integer(cfg, &dev) < 0) {
■           snd_error(CONTROL, "(%s) device must be an integer", buf);
■           err = -EINVAL;
■           goto __cleanup;
■       }
■   }
■   }

■   if (snd_config_search(cfg1, "hint", &cfg) >= 0) {
■       if (snd_config_get_type(cfg) != SND_CONFIG_TYPE_COMPOUND) {
■           snd_error(CONTROL, "hint (%s) must be a compound", buf);
■           // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 302) - [public] if

```
■       if (snd_config_get_integer(cfg, &dev) < 0) {
■           snd_error(CONTROL, "(%s) device must be an integer", buf);
■           err = -EINVAL;
■           goto __cleanup;
■       }
■   }
■   }

■   if (snd_config_search(cfg1, "hint", &cfg) >= 0) {
■       if (snd_config_get_type(cfg) != SND_CONFIG_TYPE_COMPOUND) {
■           snd_error(CONTROL, "hint (%s) must be a compound", buf);
■           err = -EINVAL;
■           // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 303) - [public] snd_error

```
■       snd_error(CONTROL, "(%s) device must be an integer", buf);
■       err = -EINVAL;
■       goto __cleanup;
■   }
■   }
■   }

■   if (snd_config_search(cfg1, "hint", &cfg) >= 0) {
■       if (snd_config_get_type(cfg) != SND_CONFIG_TYPE_COMPOUND) {
■           snd_error(CONTROL, "hint (%s) must be a compound", buf);
■           err = -EINVAL;
■           goto __cleanup;
■           // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 311) - [public] if

```
    if (snd_config_get_type(cfg) != SND_CONFIG_TYPE_COMPOUND) {
        snd_error(CONTROL, "hint (%s) must be a compound", buf);
        err = -EINVAL;
        goto __cleanup;
    }
    if (list->card < 0 &&
        snd_config_search(cfg, "omit_noargs", &n) >= 0 &&
        snd_config_get_bool(n) > 0)
        goto __skip_add;
    if (level == 1 &&
        snd_config_search(cfg, "show", &n) >= 0 &&
        snd_config_get_bool(n) <= 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 312) - [public] snd_error

```
    snd_error(CONTROL, "hint (%s) must be a compound", buf);
    err = -EINVAL;
    goto __cleanup;
}
if (list->card < 0 &&
    snd_config_search(cfg, "omit_noargs", &n) >= 0 &&
    snd_config_get_bool(n) > 0)
    goto __skip_add;
if (level == 1 &&
    snd_config_search(cfg, "show", &n) >= 0 &&
    snd_config_get_bool(n) <= 0)
    goto __skip_add;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 326) - [public] snd_config_get_string

```
    snd_config_get_string(n, &str) >= 0) {
    buf1 = strdup(str);
    if (buf1 == NULL) {
        err = -ENOMEM;
        goto __cleanup;
    }
    if (snd_config_search(cfg, "device", &n) >= 0) {
        if (snd_config_get_integer(n, &dev) < 0) {
            snd_error(CONTROL, "(%s) device must be an integer", buf);
            err = -EINVAL;
            goto __cleanup;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 328) - [public] if

```
    if (buf1 == NULL) {
        err = -ENOMEM;
        goto __cleanup;
    }
    if (snd_config_search(cfg, "device", &n) >= 0) {
        if (snd_config_get_integer(n, &dev) < 0) {
            snd_error(CONTROL, "(%s) device must be an integer", buf);
            err = -EINVAL;
            goto __cleanup;
        }
        list->device_input = dev;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 333) - [public] if

```
    if (snd_config_search(cfg, "device", &n) >= 0) {
        if (snd_config_get_integer(n, &dev) < 0) {
            snd_error(CONTROL, "(%s) device must be an integer", buf);
            err = -EINVAL;
            goto __cleanup;
        }
        list->device_input = dev;
        list->device_output = dev;
    }
    if (snd_config_search(cfg, "device_input", &n) >= 0) {
        if (snd_config_get_integer(n, &list->device_input) < 0) {
            snd_error(CONTROL, "(%s) device_input must be an integer", buf);
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/control/namehint.c (line 334) - [public] if

```
    if (snd_config_get_integer(n, &dev) < 0) {
        snd_error(CONTROL, "(%s) device must be an integer", buf);
        err = -EINVAL;
        goto __cleanup;
    }
    list->device_input = dev;
    list->device_output = dev;
}
if (snd_config_search(cfg, "device_input", &n) >= 0) {
    if (snd_config_get_integer(n, &list->device_input) < 0) {
        snd_error(CONTROL, "(%s) device_input must be an integer", buf);
        err = -EINVAL;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 335) - [public] snd_error

```
    snd_error(CONTROL, "(%s) device must be an integer", buf);
    err = -EINVAL;
    goto __cleanup;
}
list->device_input = dev;
list->device_output = dev;
}
if (snd_config_search(cfg, "device_input", &n) >= 0) {
    if (snd_config_get_integer(n, &list->device_input) < 0) {
        snd_error(CONTROL, "(%s) device_input must be an integer", buf);
        err = -EINVAL;
        goto __cleanup;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 342) - [public] if

```
    if (snd_config_search(cfg, "device_input", &n) >= 0) {
        if (snd_config_get_integer(n, &list->device_input) < 0) {
            snd_error(CONTROL, "(%s) device_input must be an integer", buf);
            err = -EINVAL;
            goto __cleanup;
        }
        /* skip the counterpart if only a single direction is defined */
        if (list->device_output < 0)
            list->device_output = DEV_SKIP;
    }
    if (snd_config_search(cfg, "device_output", &n) >= 0) {
        if (snd_config_get_integer(n, &list->device_output) < 0) {
            // ... (truncated, see source file for more)
        }
    }
```



```

dirtycode/alsa-lib/src/control/namehint.c (line 343) - [public] if
    if (snd_config_get_integer(n, &list->device_input) < 0) {
        snd_error(CONTROL, "(%s) device_input must be an integer", buf);
        err = -EINVAL;
        goto __cleanup;
    }
    /* skip the counterpart if only a single direction is defined */
    if (list->device_output < 0)
        list->device_output = DEV_SKIP;
    if (snd_config_search(cfg, "device_output", &n) >= 0) {
        if (snd_config_get_integer(n, &list->device_output) < 0) {
            snd_error(CONTROL, "(%s) device_output must be an integer", buf);
            // ... (truncated, see source file for more)
        }
    }

dirtycode/alsa-lib/src/control/namehint.c (line 344) - [public] snd_error
    snd_error(CONTROL, "(%s) device_input must be an integer", buf);
    err = -EINVAL;
    goto __cleanup;
    /* skip the counterpart if only a single direction is defined */
    if (list->device_output < 0)
        list->device_output = DEV_SKIP;
    if (snd_config_search(cfg, "device_output", &n) >= 0) {
        if (snd_config_get_integer(n, &list->device_output) < 0) {
            snd_error(CONTROL, "(%s) device_output must be an integer", buf);
            err = -EINVAL;
            // ... (truncated, see source file for more)
        }
    }

dirtycode/alsa-lib/src/control/namehint.c (line 352) - [public] if
    if (snd_config_search(cfg, "device_output", &n) >= 0) {
        if (snd_config_get_integer(n, &list->device_output) < 0) {
            snd_error(CONTROL, "(%s) device_output must be an integer", buf);
            err = -EINVAL;
            goto __cleanup;
        }
        /* skip the counterpart if only a single direction is defined */
        if (list->device_input < 0)
            list->device_input = DEV_SKIP;
    }
    else if (level == 1 && !list->show_all)
        goto __skip_add;
    // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/namehint.c (line 353) - [public] if
    if (snd_config_get_integer(n, &list->device_output) < 0) {
        snd_error(CONTROL, "(%s) device_output must be an integer", buf);
        err = -EINVAL;
        goto __cleanup;
    }
    /* skip the counterpart if only a single direction is defined */
    if (list->device_input < 0)
        list->device_input = DEV_SKIP;
    }
    else if (level == 1 && !list->show_all)
        goto __skip_add;
    if (snd_config_search(cfg1, "slave", &cfg) >= 0 &&
        // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/namehint.c (line 354) - [public] snd_error

```
#####snd_error(CONTROL, "(%s) device_output must be an integer", buf);
#####err = -EINVAL;
#####goto __cleanup;
#####}
#####/* skip the counterpart if only a single direction is defined */
#####if (list->device_input < 0)
#####list->device_input = DEV_SKIP;
#####}
#####} else if (level == 1 && !list->show_all)
#####goto __skip_add;
#####if (snd_config_search(cfg1, "slave", &cfg) >= 0 &&
#####    snd_config_search(cfg, base, &cfg1) >= 0)
#####    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 380) - [public] snd_config_for_each

```
#####snd_config_for_each(i, next, cfg) {
#####/* skip the argument list */
#####if (snd_config_get_id(snd_config_iterator_entry(i), &str) < 0)
#####continue;
#####while (*str && *str >= '0' && *str <= '9') str++;
#####if (*str == '\0')
#####continue;
#####/* the argument definition must have the default */
#####if (snd_config_search(snd_config_iterator_entry(i),
#####    "default", NULL) < 0) {
#####err = -EINVAL;
#####goto __cleanup;
#####    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 401) - [public] if

```
#####if (str != NULL) {
#####level = (buf1 == NULL ? 0 : strlen(buf1)) + 1 + strlen(str);
#####buf2 = realloc((char *)str, level + 1);
#####if (buf2 != NULL) {
#####if (buf1 != NULL) {
#####str = strchr(buf2, '|');
#####if (str != NULL)
#####memmove(buf2 + (level - strlen(str)), str, strlen(str));
#####else
#####str = buf2 + strlen(buf2);
#####*(char *)str++ = '\n';
#####memcpy((char *)str, buf1, strlen(buf1));
#####    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 404) - [public] if

```
#####if (buf2 != NULL) {
#####if (buf1 != NULL) {
#####str = strchr(buf2, '|');
#####if (str != NULL)
#####memmove(buf2 + (level - strlen(str)), str, strlen(str));
#####else
#####str = buf2 + strlen(buf2);
#####*(char *)str++ = '\n';
#####memcpy((char *)str, buf1, strlen(buf1));
#####buf2[level] = '\0';
#####free(buf1);
#####}
#####    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 405) - [public] if

```

    if (buf1 != NULL) {
        str = strchr(buf2, '|');
        if (str != NULL)
            memmove(buf2 + (level - strlen(str)), str, strlen(str));
        else
            str = buf2 + strlen(buf2);
        *(char *)str++ = '\n';
        memcpy((char *)str, buf1, strlen(buf1));
        buf2[level] = '\0';
        free(buf1);
    }
    buf1 = buf2;
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/namehint.c (line 408) - [public] memmove

```

    memmove(buf2 + (level - strlen(str)), str, strlen(str));
    else
        str = buf2 + strlen(buf2);
        *(char *)str++ = '\n';
        memcpy((char *)str, buf1, strlen(buf1));
        buf2[level] = '\0';
        free(buf1);
    }
    buf1 = buf2;
} else {
    free((char *)str);
}
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/namehint.c (line 412) - [public] memcpy

```

    *(char *)str++ = '\n';

    memcpy((char *)str, buf1, strlen(buf1));
    buf2[level] = '\0';
    free(buf1);
}
buf1 = buf2;
} else {
    free((char *)str);
}
} else if (list->device >= 0)
    goto __skip_add;
err = hint_list_add(list, buf, buf1);
}
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/namehint.c (line 414) - [public] free

```

    free(buf1);
}
buf1 = buf2;
} else {
    free((char *)str);
}
} else if (list->device >= 0)
    goto __skip_add;
err = hint_list_add(list, buf, buf1);
}
__skip_add:
if (res && cleanup_res)
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/namehint.c (line 418) - [public] free

```
    free((char *)str);
}
} else if (list->device >= 0)
    goto __skip_add;
err = hint_list_add(list, buf, buf1);
}

__skip_add:
if (res && cleanup_res)
    snd_config_delete(res);
if (buf1)
    free(buf1);
free(buf);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 426) - [public] snd_config_delete

```
snd_config_delete(res);
if (buf1)
    free(buf1);
free(buf);
return err;
}

#ifdef DOC_HIDDEN
#define IFACE(v, fcn) [SND_CTL_ELEM_IFACE_##v] = (next_devices_t)fcn

typedef int (*next_devices_t)(snd_ctl_t *, int *);

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 428) - [public] free

```
free(buf1);
free(buf);
return err;
}

#ifdef DOC_HIDDEN
#define IFACE(v, fcn) [SND_CTL_ELEM_IFACE_##v] = (next_devices_t)fcn

typedef int (*next_devices_t)(snd_ctl_t *, int *);

static const next_devices_t next_devices[] = {
    IFACE(CARD, NULL),
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 436) - [public] int

```
typedef int (*next_devices_t)(snd_ctl_t *, int *);

static const next_devices_t next_devices[] = {
    IFACE(CARD, NULL),
    IFACE(HWDEP, snd_ctl_hwdep_next_device),
    IFACE(MIXER, NULL),
    IFACE(PCM, snd_ctl_pcm_next_device),
    IFACE(RAWMIDI, snd_ctl_rawmidi_next_device),
    IFACE(TIMER, NULL),
    IFACE(SEQUENCER, NULL)
};
#endif
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 475) - [public] if

```
    if (next_devices[list->iface] != NULL) {
        list->card = card;
        device = max_device = -1;
        err = next_devices[list->iface](list->ctl, &device);
        if (device < 0)
            err = -EINVAL;
        else
            max_device = device;
        while (err >= 0 && device >= 0) {
            err = next_devices[list->iface](list->ctl, &device);
            if (err >= 0 && device > max_device)
                max_device = device;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 483) - [public] while

```
    while (err >= 0 && device >= 0) {
        err = next_devices[list->iface](list->ctl, &device);
        if (err >= 0 && device > max_device)
            max_device = device;
    }
    ok = 0;
    for (device = 0; err >= 0 && device <= max_device; device++) {
        list->device = device;
        err = try_config(rw_config, list, list->siface, str);
        if (err < 0)
            break;
        ok++;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 503) - [public] if

```
    if (err < 0) {
        list->card = card;
        list->device = -1;
        err = try_config(rw_config, list, list->siface, str);
    }
    if (err == -ENOMEM)
        goto __error;
    err = 0;
__error:
    snd_ctl_close(list->ctl);
    return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 634) - [public] add_software_devices

```
    add_software_devices(local_config, local_config_rw, &list);
    err = snd_card_next(&card);
    if (err < 0)
        goto __error;
    while (card >= 0) {
        err = get_card_name(&list, card);
        if (err < 0)
            goto __error;
        err = add_card(local_config, local_config_rw, &list, card);
        if (err < 0)
            goto __error;
        err = snd_card_next(&card);
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 638) - [public] while

```
    while (card >= 0) {
        err = get_card_name(&list, card);
        if (err < 0)
            goto __error;
        err = add_card(local_config, local_config_rw, &list, card);
        if (err < 0)
            goto __error;
        err = snd_card_next(&card);
        if (err < 0)
            goto __error;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 653) - [public] snd_config_for_each

```
    snd_config_for_each(i, next, conf) {
        if (snd_config_get_string(snd_config_iterator_entry(i),
                                &str) < 0)
            continue;
        err = hint_list_add_custom(&list, str);
        if (err < 0)
            goto __error;
    }
    err = 0;
    __error:
    /* add an empty entry if nothing has been added yet; the caller
       // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 670) - [public] snd_device_name_free_hint

```
    snd_device_name_free_hint((void **)list.list);
    else
        *hints = (void **)list.list;
    free(list.cardname);
    snd_config_delete(local_config_rw);
    snd_config_delete(local_config);
    if (local_config_update)
        snd_config_update_free(local_config_update);
    return err;
}

/**
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 677) - [public] snd_config_update_free

```
    snd_config_update_free(local_config_update);
    return err;
}

/**
 * \brief Free a list of device name hints.
 * \param hints List to free
 * \result zero if success, otherwise a negative error code
 */
int snd_device_name_free_hint(void **hints)
{
    char **h;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 694) - [public] free

```
    free(*h);
    h++;
}
free(hints);
return 0;
}

/**
 * \brief Extract a value from a hint
 * \param hint A pointer to hint
 * \param id Hint value to extract ("NAME", "DESC", or "IOID", see below)
 * \result an allocated ASCII string if success, otherwise NULL
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 724) - [public] if

```
    if (memcmp(id, hint1, 4) != 0) {
        if (delim == NULL)
            return NULL;
        hint1 = delim + 1;
        continue;
    }
    if (delim == NULL)
        return strdup(hint1 + 4);
    size = delim - hint1 - 4;
    res = malloc(size + 1);
    if (res != NULL) {
        memcpy(res, hint1 + 4, size);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/namehint.c (line 731) - [public] strdup

```
        return strdup(hint1 + 4);
    size = delim - hint1 - 4;
    res = malloc(size + 1);
    if (res != NULL) {
        memcpy(res, hint1 + 4, size);
        res[size] = '\0';
    }
    return res;
}
return NULL;
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 734) - [public] if

```
    if (res != NULL) {
        memcpy(res, hint1 + 4, size);
        res[size] = '\0';
    }
    return res;
}
return NULL;
}
```

dirtycode/alsa-lib/src/control/namehint.c (line 735) - [public] memcpy

```
        memcpy(res, hint1 + 4, size);
        res[size] = '\0';
    }
    return res;
}
return NULL;
}
```

dirtycode/alsa-lib/src/control/setup.c (line 64) - [public] snd_ctl_elem_id_free

```
■snd_ctl_elem_id_free(elem->id);
■snd_ctl_elem_info_free(elem->info);
■snd_ctl_elem_value_free(elem->val);
■snd_ctl_elem_value_free(elem->mask);
■snd_ctl_elem_value_free(elem->old);
■list_del(&elem->list);
■free(elem);
■}
■if ((h->mode & SND_SCTL_NOFREE) == 0)
■err = snd_ctl_close(h->ctl);
■free(h);
■return err;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 65) - [public] snd_ctl_elem_info_free

```
■snd_ctl_elem_info_free(elem->info);
■snd_ctl_elem_value_free(elem->val);
■snd_ctl_elem_value_free(elem->mask);
■snd_ctl_elem_value_free(elem->old);
■list_del(&elem->list);
■free(elem);
■}
■if ((h->mode & SND_SCTL_NOFREE) == 0)
■err = snd_ctl_close(h->ctl);
■free(h);
■return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 66) - [public] snd_ctl_elem_value_free

```
■snd_ctl_elem_value_free(elem->val);
■snd_ctl_elem_value_free(elem->mask);
■snd_ctl_elem_value_free(elem->old);
■list_del(&elem->list);
■free(elem);
■}
■if ((h->mode & SND_SCTL_NOFREE) == 0)
■err = snd_ctl_close(h->ctl);
■free(h);
■return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 67) - [public] snd_ctl_elem_value_free

```
■snd_ctl_elem_value_free(elem->mask);
■snd_ctl_elem_value_free(elem->old);
■list_del(&elem->list);
■free(elem);
■}
■if ((h->mode & SND_SCTL_NOFREE) == 0)
■err = snd_ctl_close(h->ctl);
■free(h);
■return err;
}
/**
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/setup.c (line 68) - [public] snd_ctl_elem_value_free

```
    snd_ctl_elem_value_free(elem->old);
    list_del(&elem->list);
    free(elem);
}
if ((h->mode & SND_SCTL_NOFREE) == 0)
    err = snd_ctl_close(h->ctl);
free(h);
return err;
}

/**
 * \brief Install given values to control elements
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 69) - [public] list_del

```
    list_del(&elem->list);
    free(elem);
}
if ((h->mode & SND_SCTL_NOFREE) == 0)
    err = snd_ctl_close(h->ctl);
free(h);
return err;
}

/**
 * \brief Install given values to control elements
 * \param h Setup control handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 70) - [public] free

```
    free(elem);
}
if ((h->mode & SND_SCTL_NOFREE) == 0)
    err = snd_ctl_close(h->ctl);
free(h);
return err;
}

/**
 * \brief Install given values to control elements
 * \param h Setup control handle
 * \result zero if success, otherwise a negative error code
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 93) - [public] if

```
    if (elem->lock) {
        err = snd_ctl_elem_lock(h->ctl, elem->id);
        if (err < 0) {
            snd_error(CONTROL, "Cannot lock ctl elem");
            return err;
        }
    }
    err = snd_ctl_elem_read(h->ctl, elem->old);
    if (err < 0) {
        snd_error(CONTROL, "Cannot read ctl elem");
        return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 95) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Cannot lock ctl elem");
        return err;
    }
    err = snd_ctl_elem_read(h->ctl, elem->old);
    if (err < 0) {
        snd_error(CONTROL, "Cannot read ctl elem");
        return err;
    }
    count = snd_ctl_elem_info_get_count(elem->info);
    type = snd_ctl_elem_info_get_type(elem->info);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 96) - [public] snd_error

```
    snd_error(CONTROL, "Cannot lock ctl elem");
    return err;
}
err = snd_ctl_elem_read(h->ctl, elem->old);
if (err < 0) {
    snd_error(CONTROL, "Cannot read ctl elem");
    return err;
}
count = snd_ctl_elem_info_get_count(elem->info);
type = snd_ctl_elem_info_get_type(elem->info);
switch (type) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 101) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Cannot read ctl elem");
        return err;
    }
    count = snd_ctl_elem_info_get_count(elem->info);
    type = snd_ctl_elem_info_get_type(elem->info);
    switch (type) {
        case SND_CTL_ELEM_TYPE_BOOLEAN:
            for (k = 0; k < count; ++k) {
                int old, val, mask;
                old = snd_ctl_elem_value_get_boolean(elem->old, k);
                mask = snd_ctl_elem_value_get_boolean(elem->mask, k);
                // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 102) - [public] snd_error

```
    snd_error(CONTROL, "Cannot read ctl elem");
    return err;
}
count = snd_ctl_elem_info_get_count(elem->info);
type = snd_ctl_elem_info_get_type(elem->info);
switch (type) {
    case SND_CTL_ELEM_TYPE_BOOLEAN:
        for (k = 0; k < count; ++k) {
            int old, val, mask;
            old = snd_ctl_elem_value_get_boolean(elem->old, k);
            mask = snd_ctl_elem_value_get_boolean(elem->mask, k);
            old &= ~mask;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 107) - [public] switch

```
    switch (type) {
    case SND_CTL_ELEM_TYPE_BOOLEAN:
        for (k = 0; k < count; ++k) {
            int old, val, mask;
            old = snd_ctl_elem_value_get_boolean(elem->old, k);
            mask = snd_ctl_elem_value_get_boolean(elem->mask, k);
            old &= ~mask;
            if (old) {
                val = snd_ctl_elem_value_get_boolean(elem->val, k);
                val |= old;
                snd_ctl_elem_value_set_boolean(elem->val, k, val);
            }
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/control/setup.c (line 114) - [public] if

```
        if (old) {
            val = snd_ctl_elem_value_get_boolean(elem->val, k);
            val |= old;
            snd_ctl_elem_value_set_boolean(elem->val, k, val);
        }
    }
    break;
    case SND_CTL_ELEM_TYPE_INTEGER:
        for (k = 0; k < count; ++k) {
            long old, val, mask;
            old = snd_ctl_elem_value_get_integer(elem->old, k);
            mask = snd_ctl_elem_value_get_integer(elem->mask, k);
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/control/setup.c (line 117) - [public] snd_ctl_elem_value_set_boolean

```
        snd_ctl_elem_value_set_boolean(elem->val, k, val);
    }
}
break;
case SND_CTL_ELEM_TYPE_INTEGER:
    for (k = 0; k < count; ++k) {
        long old, val, mask;
        old = snd_ctl_elem_value_get_integer(elem->old, k);
        mask = snd_ctl_elem_value_get_integer(elem->mask, k);
        old &= ~mask;
        if (old) {
            val = snd_ctl_elem_value_get_integer(elem->val, k);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/control/setup.c (line 127) - [public] if

```
        if (old) {
            val = snd_ctl_elem_value_get_integer(elem->val, k);
            val |= old;
            snd_ctl_elem_value_set_integer(elem->val, k, val);
        }
    }
    break;
    case SND_CTL_ELEM_TYPE_ENUMERATED:
        for (k = 0; k < count; ++k) {
            unsigned int old, val, mask;
            old = snd_ctl_elem_value_get_enumerated(elem->old, k);
            mask = snd_ctl_elem_value_get_enumerated(elem->mask, k);
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/control/setup.c (line 130) - [public] snd_ctl_elem_value_set_integer

```

    snd_ctl_elem_value_set_integer(elem->val, k, val);
}
}
break;
case SND_CTL_ELEM_TYPE_ENUMERATED:
    for (k = 0; k < count; ++k) {
        unsigned int old, val, mask;
        old = snd_ctl_elem_value_get_enumerated(elem->old, k);
        mask = snd_ctl_elem_value_get_enumerated(elem->mask, k);
        old &= ~mask;
        if (old) {
            val = snd_ctl_elem_value_get_enumerated(elem->val, k);
            // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/setup.c (line 140) - [public] if

```

    if (old) {
        val = snd_ctl_elem_value_get_enumerated(elem->val, k);
        val |= old;
        snd_ctl_elem_value_set_enumerated(elem->val, k, val);
    }
}
break;
case SND_CTL_ELEM_TYPE_IEC958:
    count = sizeof(snd_aes_iec958_t);
    /* Fall through */
case SND_CTL_ELEM_TYPE_BYTES:
    for (k = 0; k < count; ++k) {
        // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/setup.c (line 143) - [public] snd_ctl_elem_value_set_enumerated

```

    snd_ctl_elem_value_set_enumerated(elem->val, k, val);
}
}
break;
case SND_CTL_ELEM_TYPE_IEC958:
    count = sizeof(snd_aes_iec958_t);
    /* Fall through */
case SND_CTL_ELEM_TYPE_BYTES:
    for (k = 0; k < count; ++k) {
        unsigned char old, val, mask;
        old = snd_ctl_elem_value_get_byte(elem->old, k);
        mask = snd_ctl_elem_value_get_byte(elem->mask, k);
        // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/setup.c (line 156) - [public] if

```

    if (old) {
        val = snd_ctl_elem_value_get_byte(elem->val, k);
        val |= old;
        snd_ctl_elem_value_set_byte(elem->val, k, val);
    }
}
break;
default:
    assert(0);
    break;
}
err = snd_ctl_elem_write(h->ctl, elem->val);
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/control/setup.c (line 159) - [public] snd_ctl_elem_value_set_byte

```
    snd_ctl_elem_value_set_byte(elem->val, k, val);
}
}
break;
default:
    assert(0);
    break;
}
err = snd_ctl_elem_write(h->ctl, elem->val);
if (err < 0) {
    snd_error(CONTROL, "Cannot write ctl elem");
    return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 164) - [public] assert

```
    assert(0);
    break;
}
err = snd_ctl_elem_write(h->ctl, elem->val);
if (err < 0) {
    snd_error(CONTROL, "Cannot write ctl elem");
    return err;
}
}
return 0;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/control/setup.c (line 168) - [public] if

```
if (err < 0) {
    snd_error(CONTROL, "Cannot write ctl elem");
    return err;
}
}
return 0;
}
```

```
/**
 * \brief Remove (restore) previous values from control elements
 * \param h Setup control handle
 * \result zero if success, otherwise a negative error code
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 169) - [public] snd_error

```
    snd_error(CONTROL, "Cannot write ctl elem");
    return err;
}
}
return 0;
}
```

```
/**
 * \brief Remove (restore) previous values from control elements
 * \param h Setup control handle
 * \result zero if success, otherwise a negative error code
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 188) - [public] if

```
    if (elem->lock) {
        err = snd_ctl_elem_unlock(h->ctl, elem->id);
        if (err < 0) {
            snd_error(CONTROL, "Cannot unlock ctl elem");
            return err;
        }
    }
    /* Only restore the old value if it differs from the requested
     * value, because if it has changed restoring the old value
     * overrides the change. Take for example, a voice modem with
     * a .conf that sets preserve off-hook. Start playback (on-hook
     * to off-hook), start record (off-hook to off-hook), stop
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 190) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Cannot unlock ctl elem");
        return err;
    }
    /* Only restore the old value if it differs from the requested
     * value, because if it has changed restoring the old value
     * overrides the change. Take for example, a voice modem with
     * a .conf that sets preserve off-hook. Start playback (on-hook
     * to off-hook), start record (off-hook to off-hook), stop
     * playback (off-hook to restore on-hook), stop record (on-hook
     * to restore off-hook), Clearly you don't want to leave the
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 191) - [public] snd_error

```
    snd_error(CONTROL, "Cannot unlock ctl elem");
    return err;
}
/* Only restore the old value if it differs from the requested
 * value, because if it has changed restoring the old value
 * overrides the change. Take for example, a voice modem with
 * a .conf that sets preserve off-hook. Start playback (on-hook
 * to off-hook), start record (off-hook to off-hook), stop
 * playback (off-hook to restore on-hook), stop record (on-hook
 * to restore off-hook), Clearly you don't want to leave the
 * modem "on the phone" now that there isn't any playback or
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 205) - [public] if

```
/* Only restore the old value if it differs from the requested
 * value, because if it has changed restoring the old value
 * overrides the change. Take for example, a voice modem with
 * a .conf that sets preserve off-hook. Start playback (on-hook
 * to off-hook), start record (off-hook to off-hook), stop
 * playback (off-hook to restore on-hook), stop record (on-hook
 * to restore off-hook), Clearly you don't want to leave the
 * modem "on the phone" now that there isn't any playback or
 * recording active.
*/
```

```
    if (elem->preserve && snd_ctl_elem_value_compare(elem->val, elem->old)) {
        err = snd_ctl_elem_write(h->ctl, elem->old);
        if (err < 0) {
            snd_error(CONTROL, "Cannot restore ctl elem");
            return err;
        }
    }
    return 0;
}
```

```
static int snd_config_get_ctl_elem_enumerated(snd_config_t *n, snd_ctl_t *ctl,
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 207) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Cannot restore ctl elem");
        return err;
    }
}

return 0;
}

static int snd_config_get_ctl_elem_enumerated(snd_config_t *n, snd_ctl_t *ctl,
        snd_ctl_elem_info_t *info)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 208) - [public] snd_error

```
    snd_error(CONTROL, "Cannot restore ctl elem");
    return err;
}

}

return 0;
}

static int snd_config_get_ctl_elem_enumerated(snd_config_t *n, snd_ctl_t *ctl,
        snd_ctl_elem_info_t *info)
{
    const char *str;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 237) - [public] snd_ctl_elem_info_set_item

```
    snd_ctl_elem_info_set_item(info, idx);
    err = snd_ctl_elem_info(ctl, info);
    if (err < 0) {
        snd_error(CONTROL, "Cannot obtain info for CTL elem");
        return err;
    }
    if (strcmp(str, snd_ctl_elem_info_get_item_name(info)) == 0)
        return idx;
    }
    return -1;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 239) - [public] if

```
    if (err < 0) {
        snd_error(CONTROL, "Cannot obtain info for CTL elem");
        return err;
    }
    if (strcmp(str, snd_ctl_elem_info_get_item_name(info)) == 0)
        return idx;
    }
    return -1;
}

static int snd_config_get_ctl_elem_value(snd_config_t *conf,
        snd_ctl_t *ctl,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 240) - [public] snd_error

```
    snd_error(CONTROL, "Cannot obtain info for CTL elem");
    return err;
}
if (strcmp(str, snd_ctl_elem_info_get_item_name(info)) == 0)
    return idx;
}
return -1;
}
```

```
static int snd_config_get_ctl_elem_value(snd_config_t *conf,
    snd_ctl_t *ctl,
    snd_ctl_elem_value_t *val,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 266) - [public] switch

```
switch (type) {
case SND_CTL_ELEM_TYPE_BOOLEAN:
    v = snd_config_get_bool(conf);
    if (v >= 0) {
        snd_ctl_elem_value_set_boolean(val, 0, v);
        if (mask)
            snd_ctl_elem_value_set_boolean(mask, 0, 1);
        return 0;
    }
    break;
case SND_CTL_ELEM_TYPE_INTEGER:
    err = snd_config_get_integer(conf, &v);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 269) - [public] if

```
if (v >= 0) {
    snd_ctl_elem_value_set_boolean(val, 0, v);
    if (mask)
        snd_ctl_elem_value_set_boolean(mask, 0, 1);
    return 0;
}
break;
case SND_CTL_ELEM_TYPE_INTEGER:
    err = snd_config_get_integer(conf, &v);
    if (err == 0) {
        snd_ctl_elem_value_set_integer(val, 0, v);
        if (mask)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 270) - [public] snd_ctl_elem_value_set_boolean

```
snd_ctl_elem_value_set_boolean(val, 0, v);
if (mask)
    snd_ctl_elem_value_set_boolean(mask, 0, 1);
return 0;
}
break;
case SND_CTL_ELEM_TYPE_INTEGER:
    err = snd_config_get_integer(conf, &v);
    if (err == 0) {
        snd_ctl_elem_value_set_integer(val, 0, v);
        if (mask)
            snd_ctl_elem_value_set_integer(mask, 0, ~0L);
        // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/setup.c (line 272) - [public] snd_ctl_elem_value_set_boolean

```
#####snd_ctl_elem_value_set_boolean(mask, 0, 1);
#####return 0;
####}
####break;
####case SND_CTL_ELEM_TYPE_INTEGER:
####err = snd_config_get_integer(conf, &v);
####if (err == 0) {
#####snd_ctl_elem_value_set_integer(val, 0, v);
#####if (mask)
#####snd_ctl_elem_value_set_integer(mask, 0, ~0L);
#####return 0;
####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 278) - [public] if

```
####if (err == 0) {
#####snd_ctl_elem_value_set_integer(val, 0, v);
#####if (mask)
#####snd_ctl_elem_value_set_integer(mask, 0, ~0L);
#####return 0;
####}
####break;
####case SND_CTL_ELEM_TYPE_ENUMERATED:
####v = snd_config_get_ctl_elem_enumerated(conf, ctl, info);
####if (v >= 0) {
#####snd_ctl_elem_value_set_enumerated(val, 0, v);
#####if (mask)
#####snd_ctl_elem_value_set_enumerated(mask, 0, ~0L);
#####return 0;
####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 279) - [public] snd_ctl_elem_value_set_integer

```
#####snd_ctl_elem_value_set_integer(val, 0, v);
#####if (mask)
#####snd_ctl_elem_value_set_integer(mask, 0, ~0L);
#####return 0;
####}
####break;
####case SND_CTL_ELEM_TYPE_ENUMERATED:
####v = snd_config_get_ctl_elem_enumerated(conf, ctl, info);
####if (v >= 0) {
#####snd_ctl_elem_value_set_enumerated(val, 0, v);
#####if (mask)
#####snd_ctl_elem_value_set_enumerated(mask, 0, ~0L);
#####return 0;
####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 281) - [public] snd_ctl_elem_value_set_integer

```
#####snd_ctl_elem_value_set_integer(mask, 0, ~0L);
#####return 0;
####}
####break;
####case SND_CTL_ELEM_TYPE_ENUMERATED:
####v = snd_config_get_ctl_elem_enumerated(conf, ctl, info);
####if (v >= 0) {
#####snd_ctl_elem_value_set_enumerated(val, 0, v);
#####if (mask)
#####snd_ctl_elem_value_set_enumerated(mask, 0, ~0L);
#####return 0;
####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 287) - [public] if

```
    if (v >= 0) {
        snd_ctl_elem_value_set_enumerated(val, 0, v);
        if (mask)
            snd_ctl_elem_value_set_enumerated(mask, 0, ~0);
        return 0;
    }
    break;
case SND_CTL_ELEM_TYPE_BYTES:
case SND_CTL_ELEM_TYPE_IEC958:
    break;
default:
    snd_error(CONTROL, "Unknown control type: %d", type);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 288) - [public] snd_ctl_elem_value_set_enumerated

```
    snd_ctl_elem_value_set_enumerated(val, 0, v);
    if (mask)
        snd_ctl_elem_value_set_enumerated(mask, 0, ~0);
    return 0;
}
break;
case SND_CTL_ELEM_TYPE_BYTES:
case SND_CTL_ELEM_TYPE_IEC958:
    break;
default:
    snd_error(CONTROL, "Unknown control type: %d", type);
    return -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 290) - [public] snd_ctl_elem_value_set_enumerated

```
    snd_ctl_elem_value_set_enumerated(mask, 0, ~0);
    return 0;
}
break;
case SND_CTL_ELEM_TYPE_BYTES:
case SND_CTL_ELEM_TYPE_IEC958:
    break;
default:
    snd_error(CONTROL, "Unknown control type: %d", type);
    return -EINVAL;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 298) - [public] snd_error

```
    snd_error(CONTROL, "Unknown control type: %d", type);
    return -EINVAL;
}
}
switch (type) {
case SND_CTL_ELEM_TYPE_IEC958:
    count = sizeof(snd_aes_iec958_t);
    /* Fall through */
case SND_CTL_ELEM_TYPE_BYTES:
    {
        const char *buf;
        err = snd_config_get_string(conf, &buf);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 310) - [public] if

```
    if (err >= 0) {
        int c1 = 0;
        unsigned int len = strlen(buf);
        unsigned int idx = 0;
        if (len % 2 != 0 || len > count * 2) {
            _bad_content:
            snd_error(CONTROL, "bad value content");
            return -EINVAL;
        }
        while (*buf) {
            int c = *buf++;
            if (c >= '0' && c <= '9')
                // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 314) - [public] if

```
        if (len % 2 != 0 || len > count * 2) {
            _bad_content:
            snd_error(CONTROL, "bad value content");
            return -EINVAL;
        }
        while (*buf) {
            int c = *buf++;
            if (c >= '0' && c <= '9')
                c -= '0';
            else if (c >= 'a' && c <= 'f')
                c = c - 'a' + 10;
            else if (c >= 'A' && c <= 'F')
                // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 316) - [public] snd_error

```
            snd_error(CONTROL, "bad value content");
            return -EINVAL;
        }
        while (*buf) {
            int c = *buf++;
            if (c >= '0' && c <= '9')
                c -= '0';
            else if (c >= 'a' && c <= 'f')
                c = c - 'a' + 10;
            else if (c >= 'A' && c <= 'F')
                c = c - 'A' + 10;
            else {
                // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 319) - [public] while

```
        while (*buf) {
            int c = *buf++;
            if (c >= '0' && c <= '9')
                c -= '0';
            else if (c >= 'a' && c <= 'f')
                c = c - 'a' + 10;
            else if (c >= 'A' && c <= 'F')
                c = c - 'A' + 10;
            else {
                goto _bad_content;
            }
            if (idx % 2 == 1) {
                // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 330) - [public] if

```
    if (idx % 2 == 1) {
        snd_ctl_elem_value_set_byte(val, idx / 2, c1 << 4 | c);
        if (mask)
            snd_ctl_elem_value_set_byte(mask, idx / 2, 0xff);
        } else
        {
            c1 = c;
            idx++;
        }
    return 0;
}
}
default:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 331) - [public] snd_ctl_elem_value_set_byte

```
    snd_ctl_elem_value_set_byte(val, idx / 2, c1 << 4 | c);
    if (mask)
        snd_ctl_elem_value_set_byte(mask, idx / 2, 0xff);
    } else
    {
        c1 = c;
        idx++;
    }
    return 0;
}
}
default:
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 333) - [public] snd_ctl_elem_value_set_byte

```
    snd_ctl_elem_value_set_byte(mask, idx / 2, 0xff);
    } else
    {
        c1 = c;
        idx++;
    }
    return 0;
}
}
default:
    break;
}
if (snd_config_get_type(conf) != SND_CONFIG_TYPE_COMPOUND) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 345) - [public] snd_error

```
    snd_error(CONTROL, "bad value type");
    return -EINVAL;
}

snd_config_for_each(i, next, conf) {
    snd_config_t *n = snd_config_iterator_entry(i);
    const char *id;
    if (snd_config_get_id(n, &id) < 0)
        continue;
    err = safe_strtol(id, &idx);
    if (err < 0 || idx < 0 || (unsigned int) idx >= count) {
        snd_error(CONTROL, "bad value index");
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 355) - [public] if

```
    if (err < 0 || idx < 0 || (unsigned int) idx >= count) {
        snd_error(CONTROL, "bad value index");
        return -EINVAL;
    }
    switch (type) {
        case SND_CTL_ELEM_TYPE_BOOLEAN:
            v = snd_config_get_bool(n);
            if (v < 0)
                goto _bad_content;
            snd_ctl_elem_value_set_boolean(val, idx, v);
            if (mask)
                snd_ctl_elem_value_set_boolean(mask, idx, 1);
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 356) - [public] snd_error

```
        snd_error(CONTROL, "bad value index");
        return -EINVAL;
    }
    switch (type) {
        case SND_CTL_ELEM_TYPE_BOOLEAN:
            v = snd_config_get_bool(n);
            if (v < 0)
                goto _bad_content;
            snd_ctl_elem_value_set_boolean(val, idx, v);
            if (mask)
                snd_ctl_elem_value_set_boolean(mask, idx, 1);
            break;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 359) - [public] switch

```
    switch (type) {
        case SND_CTL_ELEM_TYPE_BOOLEAN:
            v = snd_config_get_bool(n);
            if (v < 0)
                goto _bad_content;
            snd_ctl_elem_value_set_boolean(val, idx, v);
            if (mask)
                snd_ctl_elem_value_set_boolean(mask, idx, 1);
            break;
        case SND_CTL_ELEM_TYPE_INTEGER:
            err = snd_config_get_integer(n, &v);
            if (err < 0)
                // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 364) - [public] snd_ctl_elem_value_set_boolean

```
        snd_ctl_elem_value_set_boolean(val, idx, v);
        if (mask)
            snd_ctl_elem_value_set_boolean(mask, idx, 1);
        break;
        case SND_CTL_ELEM_TYPE_INTEGER:
            err = snd_config_get_integer(n, &v);
            if (err < 0)
                goto _bad_content;
            snd_ctl_elem_value_set_integer(val, idx, v);
            if (mask)
                snd_ctl_elem_value_set_integer(mask, idx, ~0L);
            break;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 366) - [public] snd_ctl_elem_value_set_boolean

```
    snd_ctl_elem_value_set_boolean(mask, idx, 1);
    break;
case SND_CTL_ELEM_TYPE_INTEGER:
    err = snd_config_get_integer(n, &v);
    if (err < 0)
        goto _bad_content;
    snd_ctl_elem_value_set_integer(val, idx, v);
    if (mask)
        snd_ctl_elem_value_set_integer(mask, idx, ~0L);
    break;
case SND_CTL_ELEM_TYPE_ENUMERATED:
    v = snd_config_get_ctl_elem_enumerated(n, ctl, info);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 372) - [public] snd_ctl_elem_value_set_integer

```
    snd_ctl_elem_value_set_integer(val, idx, v);
    if (mask)
        snd_ctl_elem_value_set_integer(mask, idx, ~0L);
    break;
case SND_CTL_ELEM_TYPE_ENUMERATED:
    v = snd_config_get_ctl_elem_enumerated(n, ctl, info);
    if (v < 0)
        goto _bad_content;
    snd_ctl_elem_value_set_enumerated(val, idx, v);
    if (mask)
        snd_ctl_elem_value_set_enumerated(mask, idx, ~0);
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 374) - [public] snd_ctl_elem_value_set_integer

```
    snd_ctl_elem_value_set_integer(mask, idx, ~0L);
    break;
case SND_CTL_ELEM_TYPE_ENUMERATED:
    v = snd_config_get_ctl_elem_enumerated(n, ctl, info);
    if (v < 0)
        goto _bad_content;
    snd_ctl_elem_value_set_enumerated(val, idx, v);
    if (mask)
        snd_ctl_elem_value_set_enumerated(mask, idx, ~0);
    break;
case SND_CTL_ELEM_TYPE_BYTES:
case SND_CTL_ELEM_TYPE_IEC958:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 380) - [public] snd_ctl_elem_value_set_enumerated

```
    snd_ctl_elem_value_set_enumerated(val, idx, v);
    if (mask)
        snd_ctl_elem_value_set_enumerated(mask, idx, ~0);
    break;
case SND_CTL_ELEM_TYPE_BYTES:
case SND_CTL_ELEM_TYPE_IEC958:
    err = snd_config_get_integer(n, &v);
    if (err < 0 || v < 0 || v > 255)
        goto _bad_content;
    snd_ctl_elem_value_set_byte(val, idx, v);
    if (mask)
        snd_ctl_elem_value_set_byte(mask, idx, 0xff);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 382) - [public] snd_ctl_elem_value_set_enumerated

```
    snd_ctl_elem_value_set_enumerated(mask, idx, ~0);
    break;
case SND_CTL_ELEM_TYPE_BYTES:
case SND_CTL_ELEM_TYPE_IEC958:
    err = snd_config_get_integer(n, &v);
    if (err < 0 || v < 0 || v > 255)
        goto _bad_content;
    snd_ctl_elem_value_set_byte(val, idx, v);
    if (mask)
        snd_ctl_elem_value_set_byte(mask, idx, 0xff);
    break;
default:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 389) - [public] snd_ctl_elem_value_set_byte

```
    snd_ctl_elem_value_set_byte(val, idx, v);
    if (mask)
        snd_ctl_elem_value_set_byte(mask, idx, 0xff);
    break;
default:
    break;
}
}
return 0;
}
```

```
static int add_elem(snd_sctl_t *h, snd_config_t *_conf, snd_config_t *private_data, int *quit)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 391) - [public] snd_ctl_elem_value_set_byte

```
    snd_ctl_elem_value_set_byte(mask, idx, 0xff);
    break;
default:
    break;
}
}
return 0;
}
```

```
static int add_elem(snd_sctl_t *h, snd_config_t *_conf, snd_config_t *private_data, int *quit)
{
    snd_config_t *conf;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 426) - [public] if

```
    if (strcmp(id, "iface") == 0 || strcmp(id, "interface") == 0) {
        const char *ptr;
        if ((err = snd_config_get_string(n, &ptr)) < 0) {
            snd_error(CONTROL, "field %s is not a string", id);
            goto _err;
        }
        if ((err = snd_config_get_ctl_iface_ascii(ptr)) < 0) {
            snd_error(CONTROL, "Invalid value for '%s'", id);
            goto _err;
        }
        iface = err;
        continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 428) - [public] if

```
    if ((err = snd_config_get_string(n, &ptr)) < 0) {
        snd_error(CONTROL, "field %s is not a string", id);
        goto _err;
    }
    if ((err = snd_config_get_ctl_iface_ascii(ptr)) < 0) {
        snd_error(CONTROL, "Invalid value for '%s'", id);
        goto _err;
    }
    iface = err;
    continue;
}
if (strcmp(id, "name") == 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 429) - [public] snd_error

```
    snd_error(CONTROL, "field %s is not a string", id);
    goto _err;
}
if ((err = snd_config_get_ctl_iface_ascii(ptr)) < 0) {
    snd_error(CONTROL, "Invalid value for '%s'", id);
    goto _err;
}
iface = err;
continue;
}
if (strcmp(id, "name") == 0) {
    if ((err = snd_config_get_string(n, &name)) < 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 432) - [public] if

```
    if ((err = snd_config_get_ctl_iface_ascii(ptr)) < 0) {
        snd_error(CONTROL, "Invalid value for '%s'", id);
        goto _err;
    }
    iface = err;
    continue;
}
if (strcmp(id, "name") == 0) {
    if ((err = snd_config_get_string(n, &name)) < 0) {
        snd_error(CONTROL, "field %s is not a string", id);
        goto _err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 433) - [public] snd_error

```
    snd_error(CONTROL, "Invalid value for '%s'", id);
    goto _err;
}
iface = err;
continue;
}
if (strcmp(id, "name") == 0) {
    if ((err = snd_config_get_string(n, &name)) < 0) {
        snd_error(CONTROL, "field %s is not a string", id);
        goto _err;
    }
    continue;
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/setup.c (line 439) - [public] if

```
    if (strcmp(id, "name") == 0) {
        if ((err = snd_config_get_string(n, &name)) < 0) {
            snd_error(CONTROL, "field %s is not a string", id);
            goto _err;
        }
        continue;
    }
    if (strcmp(id, "index") == 0) {
        if ((err = snd_config_get_integer(n, &index)) < 0) {
            snd_error(CONTROL, "field %s is not an integer", id);
            goto _err;
        }
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/setup.c (line 440) - [public] if

```
    if ((err = snd_config_get_string(n, &name)) < 0) {
        snd_error(CONTROL, "field %s is not a string", id);
        goto _err;
    }
    continue;
}
if (strcmp(id, "index") == 0) {
    if ((err = snd_config_get_integer(n, &index)) < 0) {
        snd_error(CONTROL, "field %s is not an integer", id);
        goto _err;
    }
    continue;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/control/setup.c (line 441) - [public] snd_error

```
    snd_error(CONTROL, "field %s is not a string", id);
    goto _err;
}
continue;
}
if (strcmp(id, "index") == 0) {
    if ((err = snd_config_get_integer(n, &index)) < 0) {
        snd_error(CONTROL, "field %s is not an integer", id);
        goto _err;
    }
    continue;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 446) - [public] if

```
    if (strcmp(id, "index") == 0) {
        if ((err = snd_config_get_integer(n, &index)) < 0) {
            snd_error(CONTROL, "field %s is not an integer", id);
            goto _err;
        }
        continue;
    }
    if (strcmp(id, "device") == 0) {
        if ((err = snd_config_get_integer(n, &device)) < 0) {
            snd_error(CONTROL, "field %s is not an integer", id);
            goto _err;
        }
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/control/setup.c (line 447) - [public] if

```
    if ((err = snd_config_get_integer(n, &index)) < 0) {
        snd_error(CONTROL, "field %s is not an integer", id);
        goto _err;
    }
    continue;
}
if (strcmp(id, "device") == 0) {
    if ((err = snd_config_get_integer(n, &device)) < 0) {
        snd_error(CONTROL, "field %s is not an integer", id);
        goto _err;
    }
    continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 448) - [public] snd_error

```
    snd_error(CONTROL, "field %s is not an integer", id);
    goto _err;
}
continue;
}
if (strcmp(id, "device") == 0) {
    if ((err = snd_config_get_integer(n, &device)) < 0) {
        snd_error(CONTROL, "field %s is not an integer", id);
        goto _err;
    }
    continue;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 453) - [public] if

```
    if (strcmp(id, "device") == 0) {
        if ((err = snd_config_get_integer(n, &device)) < 0) {
            snd_error(CONTROL, "field %s is not an integer", id);
            goto _err;
        }
        continue;
    }
    if (strcmp(id, "subdevice") == 0) {
        if ((err = snd_config_get_integer(n, &subdevice)) < 0) {
            snd_error(CONTROL, "field %s is not an integer", id);
            goto _err;
        }
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 454) - [public] if

```
    if ((err = snd_config_get_integer(n, &device)) < 0) {
        snd_error(CONTROL, "field %s is not an integer", id);
        goto _err;
    }
    continue;
}
if (strcmp(id, "subdevice") == 0) {
    if ((err = snd_config_get_integer(n, &subdevice)) < 0) {
        snd_error(CONTROL, "field %s is not an integer", id);
        goto _err;
    }
}
continue;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 455) - [public] snd_error

```
#####snd_error(CONTROL, "field %s is not an integer", id);
#####goto _err;
####}
###continue;
##}
##if (strcmp(id, "subdevice") == 0) {
###if ((err = snd_config_get_integer(n, &subdevice)) < 0) {
#####snd_error(CONTROL, "field %s is not an integer", id);
#####goto _err;
####}
###continue;
##}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 460) - [public] if

```
##if (strcmp(id, "subdevice") == 0) {
###if ((err = snd_config_get_integer(n, &subdevice)) < 0) {
#####snd_error(CONTROL, "field %s is not an integer", id);
#####goto _err;
####}
###continue;
##}
##if (strcmp(id, "lock") == 0) {
###err = snd_config_get_bool(n);
###if (err < 0)
#####goto _err;
###lock = err;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 461) - [public] if

```
#####if ((err = snd_config_get_integer(n, &subdevice)) < 0) {
#####snd_error(CONTROL, "field %s is not an integer", id);
#####goto _err;
####}
###continue;
##}
##if (strcmp(id, "lock") == 0) {
###err = snd_config_get_bool(n);
###if (err < 0)
#####goto _err;
###lock = err;
###continue;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 462) - [public] snd_error

```
#####snd_error(CONTROL, "field %s is not an integer", id);
#####goto _err;
####}
###continue;
##}
##if (strcmp(id, "lock") == 0) {
###err = snd_config_get_bool(n);
###if (err < 0)
#####goto _err;
###lock = err;
###continue;
##}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 467) - [public] if

```
    if (strcmp(id, "lock") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            goto _err;
        lock = err;
        continue;
    }
    if (strcmp(id, "preserve") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            goto _err;
        preserve = err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 474) - [public] if

```
    if (strcmp(id, "preserve") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            goto _err;
        preserve = err;
        continue;
    }
    if (strcmp(id, "value") == 0) {
        value = n;
        continue;
    }
    if (strcmp(id, "mask") == 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 481) - [public] if

```
    if (strcmp(id, "value") == 0) {
        value = n;
        continue;
    }
    if (strcmp(id, "mask") == 0) {
        mask = n;
        continue;
    }
    if (strcmp(id, "optional") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            goto _err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 485) - [public] if

```
    if (strcmp(id, "mask") == 0) {
        mask = n;
        continue;
    }
    if (strcmp(id, "optional") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            goto _err;
        optional = err;
        continue;
    }
    if (strcmp(id, "skip_rest") == 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 489) - [public] if

```
    if (strcmp(id, "optional") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            goto _err;
        optional = err;
        continue;
    }
    if (strcmp(id, "skip_rest") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            goto _err;
        skip_rest = err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 496) - [public] if

```
    if (strcmp(id, "skip_rest") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            goto _err;
        skip_rest = err;
        continue;
    }
    snd_error(CONTROL, "Unknown field %s", id);
    return -EINVAL;
}
if (name == NULL) {
    snd_error(CONTROL, "Missing control name");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 503) - [public] snd_error

```
    snd_error(CONTROL, "Unknown field %s", id);
    return -EINVAL;
}
if (name == NULL) {
    snd_error(CONTROL, "Missing control name");
    err = -EINVAL;
    goto _err;
}
if (value == NULL) {
    snd_error(CONTROL, "Missing control value");
    err = -EINVAL;
    goto _err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 507) - [public] snd_error

```
    snd_error(CONTROL, "Missing control name");
    err = -EINVAL;
    goto _err;
}
if (value == NULL) {
    snd_error(CONTROL, "Missing control value");
    err = -EINVAL;
    goto _err;
}
if (device < 0)
    device = 0;
if (subdevice < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 512) - [public] snd_error

```
    ■■snd_error(CONTROL, "Missing control value");
    ■■err = -EINVAL;
    ■■goto _err;
    ■}
    ■if (device < 0)
    ■■device = 0;
    ■if (subdevice < 0)
    ■■subdevice = 0;
    ■elem = calloc(1, sizeof(*elem));
    ■if (!elem)
    ■■return -ENOMEM;
    ■err = snd_ctl_elem_id_malloc(&elem->id);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 549) - [public] snd_error

```
    ■■■snd_error(CONTROL, "Cannot obtain info for CTL elem (%s,'%s',%li,%li,%li): %s", snd_ctl_elem_iface_name,
    ■■goto _err;
    ■} else {
    ■■if (skip_rest)
    ■■■*quit = 1;
    ■}
    ■snd_ctl_elem_value_set_id(elem->val, elem->id);
    ■snd_ctl_elem_value_set_id(elem->old, elem->id);
    ■if (mask) {
    ■■err = snd_config_get_ctl_elem_value(value, h->ctl, elem->val, NULL, elem->info);
    ■■if (err < 0)
    ■■■goto _err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 578) - [public] snd_ctl_elem_id_free

```
    ■■■snd_ctl_elem_id_free(elem->id);
    ■■if (elem->info)
    ■■■snd_ctl_elem_info_free(elem->info);
    ■■if (elem->val)
    ■■■snd_ctl_elem_value_free(elem->val);
    ■■if (elem->mask)
    ■■■snd_ctl_elem_value_free(elem->mask);
    ■■if (elem->old)
    ■■■snd_ctl_elem_value_free(elem->old);
    ■■free(elem);
    ■■if (err != -ENOMEM && optional)
    ■■■err = 0; /* ignore the error */
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 580) - [public] snd_ctl_elem_info_free

```
    ■■■snd_ctl_elem_info_free(elem->info);
    ■■if (elem->val)
    ■■■snd_ctl_elem_value_free(elem->val);
    ■■if (elem->mask)
    ■■■snd_ctl_elem_value_free(elem->mask);
    ■■if (elem->old)
    ■■■snd_ctl_elem_value_free(elem->old);
    ■■free(elem);
    ■■if (err != -ENOMEM && optional)
    ■■■err = 0; /* ignore the error */
    ■}
    ■snd_config_delete(conf);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 582) - [public] snd_ctl_elem_value_free

```
    snd_ctl_elem_value_free(elem->val);
    if (elem->mask)
        snd_ctl_elem_value_free(elem->mask);
    if (elem->old)
        snd_ctl_elem_value_free(elem->old);
    free(elem);
    if (err != -ENOMEM && optional)
        err = 0; /* ignore the error */
}
snd_config_delete(conf);
return err;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 584) - [public] snd_ctl_elem_value_free

```
    snd_ctl_elem_value_free(elem->mask);
    if (elem->old)
        snd_ctl_elem_value_free(elem->old);
    free(elem);
    if (err != -ENOMEM && optional)
        err = 0; /* ignore the error */
}
snd_config_delete(conf);
return err;
}

/**
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 586) - [public] snd_ctl_elem_value_free

```
    snd_ctl_elem_value_free(elem->old);
    free(elem);
    if (err != -ENOMEM && optional)
        err = 0; /* ignore the error */
}
snd_config_delete(conf);
return err;
}

/**
 * \brief Build setup control handle
 * \param sctl Result - setup control handle
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 587) - [public] free

```
    free(elem);
    if (err != -ENOMEM && optional)
        err = 0; /* ignore the error */
}
snd_config_delete(conf);
return err;
}

/**
 * \brief Build setup control handle
 * \param sctl Result - setup control handle
 * \param handle Master control handle
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 620) - [public] snd_ctl_close

```
    snd_ctl_close(handle);
    return -ENOMEM;
}
h->mode = mode;
h->ctl = handle;
INIT_LIST_HEAD(&h->elems);
snd_config_for_each(i, next, conf) {
    snd_config_t *n = snd_config_iterator_entry(i);
    err = add_elem(h, n, private_data, &quit);
    if (err < 0) {
        free_elems(h);
        return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 629) - [public] if

```
    if (err < 0) {
        free_elems(h);
        return err;
    }
    if (quit)
        break;
}
*sctl = h;
return 0;
}

/**
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 630) - [public] free_elems

```
    free_elems(h);
    return err;
}
if (quit)
    break;
}
*sctl = h;
return 0;
}

/**
    * \brief Free setup control handle
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/setup.c (line 648) - [public] free_elems

```
    return free_elems(sctl);
}
```

dirtycode/alsa-lib/src/control/tlv.c (line 72) - [public] snd_error

```
    snd_error(CONTROL, "TLV stream too short");
    return -EINVAL;
}

*db_tlv = NULL;
type = tlv[SNDRV_CTL_TLVO_TYPE];
size = tlv[SNDRV_CTL_TLVO_LEN];
tlv_size -= 2 * sizeof(int);
if (size > tlv_size) {
    snd_error(CONTROL, "TLV size error");
    return -EINVAL;
}
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/control/tlv.c (line 81) - [public] snd_error

```
    ■■snd_error(CONTROL, "TLV size error");
    ■■return -EINVAL;
    ■}
    ■switch (type) {
    ■case SND_CTL_TLV_CONTAINER:
    ■■size = int_index(size) * sizeof(int);
    ■■tlv += 2;
    ■■while (size > 0) {
    ■■■unsigned int len;
    ■■■err = snd_tlv_parse_db_info(tlv, size, db_tlv);
    ■■■if (err < 0 && err != -ENOENT)
    ■■■■return err; /* error */
    ■■■// ... (truncated, see source file for more)
    ■}
```

dirtycode/alsa-lib/src/control/tlv.c (line 88) - [public] while

```
    ■■while (size > 0) {
    ■■■unsigned int len;
    ■■■err = snd_tlv_parse_db_info(tlv, size, db_tlv);
    ■■■if (err < 0 && err != -ENOENT)
    ■■■■return err; /* error */
    ■■■if (err > 0)
    ■■■■return err; /* found */
    ■■■len = int_index(tlv[SNDCTL_TLV0_LEN]) + 2;
    ■■■size -= len * sizeof(int);
    ■■■tlv += len;
    ■■}
    ■■break;
    ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 112) - [public] if

```
    ■■if (size < minsize) {
    ■■■snd_error(CONTROL, "Invalid dB_scale TLV size");
    ■■■return -EINVAL;
    ■■}
    ■■if (size > MAX_TLV_RANGE_SIZE) {
    ■■■snd_error(CONTROL, "Too big dB_scale TLV size: %d", size);
    ■■■return -EINVAL;
    ■■}
    ■■*db_tlv = tlv;
    ■■return size + sizeof(int) * 2;
    ■}
    ■default:
    ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 113) - [public] snd_error

```
    ■■■snd_error(CONTROL, "Invalid dB_scale TLV size");
    ■■■return -EINVAL;
    ■■}
    ■■if (size > MAX_TLV_RANGE_SIZE) {
    ■■■snd_error(CONTROL, "Too big dB_scale TLV size: %d", size);
    ■■■return -EINVAL;
    ■■}
    ■■*db_tlv = tlv;
    ■■return size + sizeof(int) * 2;
    ■}
    ■default:
    ■■break;
    ■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 116) - [public] if

```
    if (size > MAX_TLV_RANGE_SIZE) {
        snd_error(CONTROL, "Too big dB_scale TLV size: %d", size);
        return -EINVAL;
    }
    *db_tlvp = tlv;
    return size + sizeof(int) * 2;
}
default:
    break;
}
return -ENOENT;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 117) - [public] snd_error

```
    snd_error(CONTROL, "Too big dB_scale TLV size: %d", size);
    return -EINVAL;
}
*db_tlvp = tlv;
return size + sizeof(int) * 2;
}
default:
    break;
}
return -ENOENT;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 150) - [public] while

```
    while (pos + 4 <= len) {
        long rmin, rmax;
        long submin, submax;
        submin = (int)tlv[pos];
        submax = (int)tlv[pos + 1];
        if (rangemax < submax)
            submax = rangemax;
        err = snd_tlv_get_dB_range(tlv + pos + 2,
            submin, submax,
            &rmin, &rmax);
        if (err < 0)
            return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 162) - [public] if

```
    if (pos > 2) {
        if (rmin < *min)
            *min = rmin;
        if (rmax > *max)
            *max = rmax;
    } else {
        *min = rmin;
        *max = rmax;
    }
    if (rangemax == submax)
        return 0;
    pos += int_index(tlv[pos + 3]) + 4;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 222) - [public] while

```
    while (pos + 4 <= len) {
        rangemin = (int)tlv[pos];
        rangemax = (int)tlv[pos + 1];
        if (volume >= rangemin && volume <= rangemax)
            return snd_tlv_convert_to_dB(tlv + pos + 2,
                rangemin, rangemax,
                volume, db_gain);
        pos += int_index(tlv[pos + 3]) + 4;
    }
    return -EINVAL;
}

case SND_CTL_TLVT_DB_SCALE: {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 249) - [public] if

```
    if (volume <= rangemin || rangemax <= rangemin) {
        if (type == SND_CTL_TLVT_DB_MINMAX_MUTE)
            *db_gain = SND_CTL_TLV_DB_GAIN_MUTE;
        else
            *db_gain = mindb;
    } else if (volume >= rangemax)
        *db_gain = maxdb;
    else
        *db_gain = (maxdb - mindb) * (volume - rangemin) /
            (rangemax - rangemin) + mindb;
    return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 316) - [public] while

```
    while (pos + 4 <= len) {
        long submin, submax;
        submin = (int)tlv[pos];
        submax = (int)tlv[pos + 1];
        if (rangemax < submax)
            submax = rangemax;
        if (!snd_tlv_get_dB_range(tlv + pos + 2,
            submin, submax,
            &dbmin, &dbmax) &&
            db_gain >= dbmin && db_gain <= dbmax)
            return snd_tlv_convert_from_dB(tlv + pos + 2,
                submin, submax,
                // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 329) - [public] if

```
        else if (db_gain < dbmin) {
            *value = xdir > 0 || pos == 2 ? submin : prev_submax;
            return 0;
        }
        prev_submax = submax;
        if (rangemax == submax)
            break;
        pos += int_index(tlv[pos + 3]) + 4;
    }
    *value = prev_submax;
    return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/control/tlv.c (line 403) - [public] pow

```
    ■■■pow(10.0, (double)min / 2000.0);
    ■■■vmax = !max ? 1.0 : pow(10.0, (double)max / 2000.0);
    ■■■v = pow(10.0, (double)db_gain / 2000.0);
    ■■■v = (v - vmin) * (rangemax - rangemin) / (vmax - vmin);
    ■■■if (xdir > 0)
    ■■■v = ceil(v);
    ■■■else if (xdir == 0)
    ■■■v = lrint(v);
    ■■■*value = (long)v + rangemin;
    ■■}
    ■■return 0;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 62) - [internal] snd_dlp_path_lock

```
static inline void snd_dlp_path_lock(void) {}
static inline void snd_dlp_path_unlock(void) {}
#endif

static void snd_dlinfo_origin(char *path, size_t path_len)
{
#ifdef DL_ORIGIN_AVAILABLE
    ■struct link_map *links;
    ■Dl_info info;
    ■char origin[PATH_MAX];
    ■if (dladdr1(&snd_dlp_path, &info, (void*)&links, RTLD_DL_LINKMAP) == 0)
    ■■return;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 63) - [internal] snd_dlp_path_unlock

```
static inline void snd_dlp_path_unlock(void) {}
#endif

static void snd_dlinfo_origin(char *path, size_t path_len)
{
#ifdef DL_ORIGIN_AVAILABLE
    ■struct link_map *links;
    ■Dl_info info;
    ■char origin[PATH_MAX];
    ■if (dladdr1(&snd_dlp_path, &info, (void*)&links, RTLD_DL_LINKMAP) == 0)
    ■■return;
    ■if (dlinfo(links, RTLD_DI_ORIGIN, origin))
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 95) - [public] if

```
    ■■if (env) {
    ■■■snd_plugin_dir = strdup(env);
    ■■} else {
    ■■■snd_dlinfo_origin(path, path_len);
    ■■}
    ■■snd_plugin_dir_set = 1;
    ■}
    ■■snprintf(path, path_len, "%s/%s",
    ■■ snd_plugin_dir ? snd_plugin_dir : ALSA_PLUGIN_DIR, name);
    ■■snd_dlp_path_unlock();
    ■■return 0;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 98) - [public] snd_dlinfo_origin

```
    snd_dlinfo_origin(path, path_len);
}
snd_plugin_dir_set = 1;
}
snprintf(path, path_len, "%s/%s",
snd_plugin_dir ? snd_plugin_dir : ALSA_PLUGIN_DIR, name);
snd_dlpath_unlock();
return 0;
}

/**
 * \brief Opens a dynamic library - ALSA wrapper for \c dlopen.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 132) - [public] if

```
if (self == NULL) {
    Dl_info dlinfo;
    if (dladdr(snd_dlopen, &dlinfo) > 0)
        self = dlinfo.dli_fname;
}
name = self;
}
#endif
#endif
#ifdef HAVE_LIBDL
/*
 * Handle the plugin dir not being on the default dlopen search
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 161) - [public] snprintf

```
snprintf(errbuf, errbuflen, "%s", dlerror());
#endif
return NULL;
}

#ifdef DOXYGEN
EXPORT_SYMBOL void *INTERNAL(snd_dlopen_old)(const char *name, int mode)
{
    return INTERNAL(snd_dlopen)(name, mode, NULL, 0);
}
#endif

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 169) - [public] INTERNAL

```
return INTERNAL(snd_dlopen)(name, mode, NULL, 0);
}
#endif

#ifdef DOC_HIDDEN
use_symbol_version(__snd_dlopen_old, snd_dlopen, ALSA_0.9);
use_default_symbol_version(__snd_dlopen, snd_dlopen, ALSA_1.1.6);
#endif /* DOC_HIDDEN */

/**
 * \brief Closes a dynamic library - ALSA wrapper for \c dlclose.
 * \param handle Library handle, similar to \c dlclose.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 193) - [public] dlclose

```
    return dlclose(handle);
#else
    return 0;
#endif
}

/**
 * \brief Verifies a dynamically loaded symbol.
 * \param handle Library handle, similar to \c dlsym.
 * \param name Symbol name.
 * \param version Version of the symbol.
 * \return Zero is successful, otherwise a negative error code.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 226) - [public] snd_error

```
    snd_error(CORE, "unable to verify version for symbol %s", name);
    return res;
#else
    return 0;
#endif
}

/**
 * \brief Resolves a symbol from a dynamic library - ALSA wrapper for \c dlsym.
 * \param handle Library handle, similar to \c dlsym.
 * \param name Symbol name.
 * \param version Version of the symbol.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 255) - [public] while

```
    while (link) {
        if (!strcmp(name, link->dlsym_name))
            return (void *)link->dlsym_ptr;
        link = link->next;
    }
    return NULL;
}

#endif
#ifdef HAVE_LIBDL
#ifdef VERSIONED_SYMBOLS
    if (version) {
        err = snd_dlsym_verify(handle, name, version);
        // ... (truncated, see source file for more)
    }
}
#endif
#endif
```

dirtycode/alsa-lib/src/dlmisc.c (line 271) - [public] dlsym

```
    return dlsym(handle, name);
#else
    return NULL;
#endif
}

/*
 * dlobj cache
 */

#ifndef DOC_HIDDEN
struct dlobj_cache {
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/dlmisc.c (line 304) - [internal] snd_dlobj_lock

```
static inline void snd_dlobj_lock(void) {}
static inline void snd_dlobj_unlock(void) {}
#endif

static LIST_HEAD(pcm_dlobj_list);

static struct dlobj_cache *
snd_dlobj_cache_get0(const char *lib, const char *name,
    const char *version, int verbose)
{
    struct list_head *p;
    struct dlobj_cache *c;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 305) - [internal] snd_dlobj_unlock

```
static inline void snd_dlobj_unlock(void) {}
#endif

static LIST_HEAD(pcm_dlobj_list);

static struct dlobj_cache *
snd_dlobj_cache_get0(const char *lib, const char *name,
    const char *version, int verbose)
{
    struct list_head *p;
    struct dlobj_cache *c;
    void *func, *dlobj;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 308) - [public] LIST_HEAD

```
static LIST_HEAD(pcm_dlobj_list);

static struct dlobj_cache *
snd_dlobj_cache_get0(const char *lib, const char *name,
    const char *version, int verbose)
{
    struct list_head *p;
    struct dlobj_cache *c;
    void *func, *dlobj;
    char errbuf[256];

    list_for_each(p, &pcm_dlobj_list) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 327) - [public] if

```
    if (strcmp(c->name, name) == 0) {
        c->refcnt++;
        return c;
    }

    errbuf[0] = '\0';
    dlobj = INTERNAL(snd_dlopen)(lib, RTLD_NOW,
        verbose ? errbuf : 0,
        verbose ? sizeof(errbuf) : 0);
    if (dlobj == NULL) {
        if (verbose)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 362) - [public] free

```
    free((void *)c->name);
    free((void *)c->lib);
    free(c);
    __err:
    snd_dlclose(dlobj);
    return NULL;
}
c->dlobj = dlobj;
c->func = func;
list_add_tail(&c->list, &pcm_dlobj_list);
return c;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 363) - [public] free

```
    free((void *)c->lib);
    free(c);
    __err:
    snd_dlclose(dlobj);
    return NULL;
}
c->dlobj = dlobj;
c->func = func;
list_add_tail(&c->list, &pcm_dlobj_list);
return c;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 364) - [public] free

```
    free(c);
    __err:
    snd_dlclose(dlobj);
    return NULL;
}
c->dlobj = dlobj;
c->func = func;
list_add_tail(&c->list, &pcm_dlobj_list);
return c;
}

void *snd_dlobj_cache_get(const char *lib, const char *name,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 366) - [public] snd_dlclose

```
    snd_dlclose(dlobj);
    return NULL;
}
c->dlobj = dlobj;
c->func = func;
list_add_tail(&c->list, &pcm_dlobj_list);
return c;
}

void *snd_dlobj_cache_get(const char *lib, const char *name,
    const char *version, int verbose)
{
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/dlmisc.c (line 418) - [public] if

```
    if (c->func == func) {
        refcnt = c->refcnt;
        if (c->refcnt > 0)
            c->refcnt--;
        snd_dlobj_unlock();
        return refcnt == 1 ? 0 : -EINVAL;
    }
    snd_dlobj_unlock();
    return -ENOENT;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/dlmisc.c (line 422) - [public] snd_dlobj_unlock

```
    snd_dlobj_unlock();
    return refcnt == 1 ? 0 : -EINVAL;
}
snd_dlobj_unlock();
return -ENOENT;
}
```

void snd_dlobj_cache_cleanup(void)

```
{
    struct list_head *p, *npos;
    struct dlobj_cache *c;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/dlmisc.c (line 440) - [public] list_del

```
    list_del(p);
    snd_dlclose(c->dlobj);
    free((void *)c->name); /* shut up gcc warning */
    free((void *)c->lib); /* shut up gcc warning */
    free(c);
}
snd_dlobj_unlock();
snd_dlpath_lock();
snd_plugin_dir_set = 0;
free(snd_plugin_dir);
snd_plugin_dir = NULL;
snd_dlpath_unlock();
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 441) - [public] snd_dlclose

```
    snd_dlclose(c->dlobj);
    free((void *)c->name); /* shut up gcc warning */
    free((void *)c->lib); /* shut up gcc warning */
    free(c);
}
snd_dlobj_unlock();
snd_dlpath_lock();
snd_plugin_dir_set = 0;
free(snd_plugin_dir);
snd_plugin_dir = NULL;
snd_dlpath_unlock();
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/dlmisc.c (line 442) - [public] free

```
    ■■free((void *)c->name); /* shut up gcc warning */
    ■■free((void *)c->lib); /* shut up gcc warning */
    ■■free(c);
    ■}
    ■snd_dlobj_unlock();
    ■snd_dlpath_lock();
    ■snd_plugin_dir_set = 0;
    ■free(snd_plugin_dir);
    ■snd_plugin_dir = NULL;
    ■snd_dlpath_unlock();
    ■}
    ■endif
```

dirtycode/alsa-lib/src/dlmisc.c (line 443) - [public] free

```
    snd_dlobj_lock();
    list_for_each_safe(p, npos, &pcm_dlobj_list) {
        c = list_entry(p, struct dlobj_cache, list);
        if (c->refcnt)
            continue;
        list_del(p);
        snd_dlclose(c->dlobj);
        free((void *)c->name); /* shut up gcc warning */

        ■■free((void *)c->lib); /* shut up gcc warning */
        ■■free(c);
        ■}
        ■snd_dlobj_unlock();
        ■snd_dlpath_lock();
        ■snd_plugin_dir_set = 0;
        ■free(snd_plugin_dir);
        ■snd_plugin_dir = NULL;
        ■snd_dlpath_unlock();
        ■}
        ■endif
```

dirtycode/alsa-lib/src/dlmisc.c (line 444) - [public] free

```
    snd_dlobj_lock();
    list_for_each_safe(p, npos, &pcm_dlobj_list) {
        c = list_entry(p, struct dlobj_cache, list);
        if (c->refcnt)
            continue;
        list_del(p);
        snd_dlclose(c->dlobj);
        free((void *)c->name); /* shut up gcc warning */
        free((void *)c->lib); /* shut up gcc warning */

        ■■free(c);
        ■}
        ■snd_dlobj_unlock();
        ■snd_dlpath_lock();
        ■snd_plugin_dir_set = 0;
        ■free(snd_plugin_dir);
        ■snd_plugin_dir = NULL;
        ■snd_dlpath_unlock();
        ■}
        ■endif
```

dirtycode/alsa-lib/src/error.c (line 37) - [public] snd_lib_error_default

```
static void snd_lib_error_default(const char *file, int line, const char *function, int errcode, const char *errstr)
{
    /**
     * Array of error codes in US ASCII.
     */
    static const char *snd_error_codes[] =
    {
        "Sound protocol is not compatible"
    };

    /**
     * \brief Returns the message for an error code.
     * // ... (truncated, see source file for more)
     */
}
```

dirtycode/alsa-lib/src/error.c (line 58) - [public] return

```
return (const char *) strerror(errnum);
errnum -= SND_ERROR_BEGIN;
if ((unsigned int) errnum >= sizeof(snd_error_codes) / sizeof(const char *))
    return "Unknown error";
return snd_error_codes[errnum];
}

#ifdef DOC_HIDDEN
#ifdef HAVE__THREAD
#define TLS_PFX__thread
#else
#define TLS_PFX/* NOP */
// ... (truncated, see source file for more)
#endif
#endif
```

dirtycode/alsa-lib/src/error.c (line 195) - [public] if

```
if (colon) {
    /** interface:level format */
    *colon = '\0';
    const char *interface_name = token;
    const char *level_str = colon + 1;
    int interface_num = -1;
    int level = -1;

    /** Try to find interface by name */
    for (i = 1; i <= SND_ILOG_LAST; i++) {
        if (snd_ilog_interface_names[i] &&
            strcmp(snd_ilog_interface_names[i], interface_name) == 0) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/error.c (line 206) - [public] strcmp

```
if (strcmp(snd_ilog_interface_names[i], interface_name) == 0) {
    interface_num = i;
    break;
}

/** If not found by name, try direct number */
if (interface_num < 0) {
    char *endptr;
    long val = strtol(interface_name, &endptr, 10);
    if (*endptr == '\0' && val >= 0 && val <= SND_ILOG_LAST)
        interface_num = val;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/error.c (line 213) - [public] if

/ If not found by name, try direct number */*

```
    if (interface_num < 0) {
        char *endptr;
        long val = strtol(interface_name, &endptr, 10);
        if (*endptr == '\0' && val >= 0 && val <= SND_ILOG_LAST)
            interface_num = val;
    }

    /* Parse level */
    for (i = 1; i <= SND_LOG_LAST; i++) {
        if (snd_log_prio_names[i] &&
            strcmp(snd_log_prio_names[i], level_str) == 0) {
            level = i;
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/error.c (line 223) - [public] strcmp

```
    if (strcmp(snd_log_prio_names[i], level_str) == 0) {
        level = i;
        break;
    }

    /* If not found by name, try direct number */
    if (level < 0) {
        char *endptr;
        long val = strtol(level_str, &endptr, 10);
        if (*endptr == '\0' && val >= 0 && val <= SND_LOG_LAST)
            level = val;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/error.c (line 230) - [public] if

/ If not found by name, try direct number */*

```
    if (level < 0) {
        char *endptr;
        long val = strtol(level_str, &endptr, 10);
        if (*endptr == '\0' && val >= 0 && val <= SND_LOG_LAST)
            level = val;
    }

    /* Store the interface-specific level */
    if (interface_num > 0 && level > 0)
        debug_config.interface_levels[interface_num] = level;
    } else {
        /* Global level only */
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/error.c (line 247) - [public] strcmp

```
    if (strcmp(snd_log_prio_names[i], token) == 0) {
        level = i;
        break;
    }

    /* If not found by name, try direct number */
    if (level < 0) {
        char *endptr;
        long val = strtol(token, &endptr, 10);
        if (*endptr == '\0' && val >= 0 && val <= SND_LOG_LAST)
            level = val;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/error.c (line 254) - [public] if

/ If not found by name, try direct number */*

```
    if (level < 0) {
        char *endptr;
        long val = strtol(token, &endptr, 10);
        if (*endptr == '\0' && val >= 0 && val <= SND_LOG_LAST)
            level = val;
    }

    if (level > 0)
        debug_config.global_level = level;

    token = strtok_r(NULL, ",", &saveptr);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/error.c (line 297) - [public] snd_lib_error_vdefault

```
static void snd_lib_error_vdefault(const char *file, int line, const char *function, int errcode, const char *fmt, ...)
{
    /**
     * \brief The default log handler function.
     * \param prio Priority value (SND_LOG_*).
     * \param interface Interface (SND_ILOG_*).
     * \param file The filename where the error was hit.
     * \param line The line number.
     * \param function The function name.
     * \param errcode The error code.
     * \param fmt The message (including the format characters).
     * \param ... Optional arguments.
     * // ... (truncated, see source file for more)
    */
}
```

dirtycode/alsa-lib/src/error.c (line 319) - [public] local_log

```
local_log(prio, interface, file, line, function, errcode, fmt, arg);
return;
}
if (local_error && prio == SND_LOG_ERROR) {
    local_error(file, line, function, errcode, fmt, arg);
    return;
}
if (snd_lib_error != snd_lib_error_default) {
    if (prio == SND_LOG_ERROR)
        snd_lib_error_vdefault(file, line, function, errcode, fmt, arg);
    /* ignore other priorities - restore old behaviour */
    return;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/error.c (line 323) - [public] local_error

```
local_error(file, line, function, errcode, fmt, arg);
return;
}
if (snd_lib_error != snd_lib_error_default) {
    if (prio == SND_LOG_ERROR)
        snd_lib_error_vdefault(file, line, function, errcode, fmt, arg);
    /* ignore other priorities - restore old behaviour */
    return;
}

if (!snd_lib_log_filter(prio, interface, NULL))
    return;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/error.c (line 328) - [public] snd_lib_error_vdefault

```
■■■snd_lib_error_vdefault(file, line, function, errcode, fmt, arg);
■■/* ignore other priorities - restore old behaviour */
■■return;
■■}

■■if (!snd_lib_log_filter(prio, interface, NULL))
■■return;

■■fprintf(stderr, "ALSA lib %s:%i:(%s) ", file, line, function);

■■text1 = snd_lib_log_priority(prio);
■■text2 = snd_lib_log_interface(interface);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/error.c (line 341) - [public] fprintf

```
■■fprintf(stderr, "[%s.%s] ", text1 ? text1 : "", text2 ? text2 : "");

■■vfprintf(stderr, fmt, arg);
■■if (errcode)
■■fprintf(stderr, ": %s", snd_strerror(errcode));
■■putc('\n', stderr);
}

/**
 * \brief Root log handler function.
 * \param prio Priority value (SND_LOG_*).
 * \param interface Interface (SND_ILOG_*).
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/error.c (line 345) - [public] fprintf

```
■■fprintf(stderr, ": %s", snd_strerror(errcode));
■■putc('\n', stderr);
}

/**
 * \brief Root log handler function.
 * \param prio Priority value (SND_LOG_*).
 * \param interface Interface (SND_ILOG_*).
 * \param file The filename where the error was hit.
 * \param line The line number.
 * \param function The function name.
 * \param errcode The error code.
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/error.c (line 393) - [public] assert

```
■■assert(0);
#endif
finish:
■■va_end(arg);
}

/**
 * \ingroup Error
 * Pointer to the error handler function.
 * For internal use only.
 */
snd_lib_log_handler_t snd_lib_vlog = snd_lib_vlog_default;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/error.c (line 527) - [public] memcpy

```
    memcpy(dst, src, len);
    dst[len] = '\0';
}
return ret;
}

/**
 * \brief Append a C-string into a sized buffer
 * \param dst Where to append the string to
 * \param src Where to copy the string from
 * \param size Size of destination buffer
 * \retval The total string length (no trimming)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/error.c (line 554) - [public] memcpy

```
    memcpy(dst, src, len);
    dst[len] = '\0';
}
return ret;
}
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 57) - [public] snd_error

```
    snd_error(HWDEP, "Invalid type for HWDEP %s definition", name);
} else
    snd_error(HWDEP, "Invalid type for HWDEP definition");
return -EINVAL;
}
err = snd_config_search(hwdep_conf, "type", &conf);
if (err < 0) {
    snd_error(HWDEP, "type is not defined");
    return err;
}
err = snd_config_get_id(conf, &id);
if (err < 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 59) - [public] snd_error

```
    snd_error(HWDEP, "Invalid type for HWDEP definition");
return -EINVAL;
}
err = snd_config_search(hwdep_conf, "type", &conf);
if (err < 0) {
    snd_error(HWDEP, "type is not defined");
    return err;
}
err = snd_config_get_id(conf, &id);
if (err < 0) {
    snd_error(HWDEP, "unable to get id");
    return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 64) - [public] snd_error

```
    snd_error(HWDEP, "type is not defined");
    return err;
}
err = snd_config_get_id(conf, &id);
if (err < 0) {
    snd_error(HWDEP, "unable to get id");
    return err;
}
err = snd_config_get_string(conf, &str);
if (err < 0) {
    snd_error(HWDEP, "Invalid type for %s", id);
    return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 69) - [public] snd_error

```
    ■■snd_error(HWDEP, "unable to get id");
    ■■return err;
    ■}
    ■err = snd_config_get_string(conf, &str);
    ■if (err < 0) {
    ■■snd_error(HWDEP, "Invalid type for %s", id);
    ■■return err;
    ■}
    ■err = snd_config_search_definition(hwdep_root, "hwdep_type", str, &type_conf);
    ■if (err >= 0) {
    ■■if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
    ■■■snd_error(HWDEP, "Invalid type for HWDEP type %s definition", str);
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 74) - [public] snd_error

```
    ■■snd_error(HWDEP, "Invalid type for %s", id);
    ■■return err;
    ■}
    ■err = snd_config_search_definition(hwdep_root, "hwdep_type", str, &type_conf);
    ■if (err >= 0) {
    ■■if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
    ■■■snd_error(HWDEP, "Invalid type for HWDEP type %s definition", str);
    ■■■err = -EINVAL;
    ■■■goto _err;
    ■■}
    ■■snd_config_for_each(i, next, type_conf) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 79) - [public] if

```
    ■■if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
    ■■■snd_error(HWDEP, "Invalid type for HWDEP type %s definition", str);
    ■■■err = -EINVAL;
    ■■■goto _err;
    ■■}
    ■■snd_config_for_each(i, next, type_conf) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id;
    ■■■if (snd_config_get_id(n, &id) < 0)
    ■■■■continue;
    ■■■if (strcmp(id, "comment") == 0)
    ■■■■continue;
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 80) - [public] snd_error

```
    ■■■snd_error(HWDEP, "Invalid type for HWDEP type %s definition", str);
    ■■■err = -EINVAL;
    ■■■goto _err;
    ■■}
    ■■snd_config_for_each(i, next, type_conf) {
    ■■■snd_config_t *n = snd_config_iterator_entry(i);
    ■■■const char *id;
    ■■■if (snd_config_get_id(n, &id) < 0)
    ■■■■continue;
    ■■■if (strcmp(id, "comment") == 0)
    ■■■■continue;
    ■■■if (strcmp(id, "lib") == 0) {
    ■■■// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/hwdep/hwdep.c (line 84) - [public] snd_config_for_each

```
    snd_config_for_each(i, next, type_conf) {
        snd_config_t *n = snd_config_iterator_entry(i);
        const char *id;
        if (snd_config_get_id(n, &id) < 0)
            continue;
        if (strcmp(id, "comment") == 0)
            continue;
        if (strcmp(id, "lib") == 0) {
            err = snd_config_get_string(n, &lib);
            if (err < 0) {
                snd_error(HWDEP, "Invalid type for %s", id);
                goto _err;
            }
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 91) - [public] if

```
    if (strcmp(id, "lib") == 0) {
        err = snd_config_get_string(n, &lib);
        if (err < 0) {
            snd_error(HWDEP, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    if (strcmp(id, "open") == 0) {
        err = snd_config_get_string(n, &open_name);
        if (err < 0) {
            snd_error(HWDEP, "Invalid type for %s", id);
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 93) - [public] if

```
    if (err < 0) {
        snd_error(HWDEP, "Invalid type for %s", id);
        goto _err;
    }
    continue;
}
if (strcmp(id, "open") == 0) {
    err = snd_config_get_string(n, &open_name);
    if (err < 0) {
        snd_error(HWDEP, "Invalid type for %s", id);
        goto _err;
    }
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 94) - [public] snd_error

```
    snd_error(HWDEP, "Invalid type for %s", id);
    goto _err;
}
continue;
}
if (strcmp(id, "open") == 0) {
    err = snd_config_get_string(n, &open_name);
    if (err < 0) {
        snd_error(HWDEP, "Invalid type for %s", id);
        goto _err;
    }
}
continue;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 99) - [public] if

```
    if (strcmp(id, "open") == 0) {
        err = snd_config_get_string(n, &open_name);
        if (err < 0) {
            snd_error(HWDEP, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    snd_error(HWDEP, "Unknown field %s", id);
    err = -EINVAL;
    goto _err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 101) - [public] if

```
    if (err < 0) {
        snd_error(HWDEP, "Invalid type for %s", id);
        goto _err;
    }
    continue;
}
snd_error(HWDEP, "Unknown field %s", id);
err = -EINVAL;
goto _err;
}
}
if (!open_name) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 102) - [public] snd_error

```
    snd_error(HWDEP, "Invalid type for %s", id);
    goto _err;
}
continue;
}
snd_error(HWDEP, "Unknown field %s", id);
err = -EINVAL;
goto _err;
}
}
if (!open_name) {
    open_name = buf;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 107) - [public] snd_error

```
    snd_error(HWDEP, "Unknown field %s", id);
    err = -EINVAL;
    goto _err;
}
}
if (!open_name) {
    open_name = buf;
    snprintf(buf, sizeof(buf), "_snd_hwdep_%s_open", str);
}
#ifdef PIC
snd_hwdep_open_symbols();
#endif
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 114) - [public] snprintf

```
    ■■snprintf(buf, sizeof(buf), "_snd_hwdep_%s_open", str);
    ■}
    #ifndef PIC
    ■■snd_hwdep_open_symbols();
    #endif
    ■h = INTERNAL(snd_dlopen)(lib, RTLD_NOW, errbuf, sizeof(errbuf));
    ■if (h)
    ■■open_func = snd_dlsym(h, open_name, SND_DLSYM_VERSION(SND_HWDEP_DLSYM_VERSION));
    ■err = 0;
    ■if (!h) {
    ■■snd_error(HWDEP, "Cannot open shared library %s (%s)", lib, errbuf);
    ■■err = -ENOENT;
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 124) - [public] snd_error

```
    ■■snd_error(HWDEP, "Cannot open shared library %s (%s)", lib, errbuf);
    ■■err = -ENOENT;
    ■} else if (!open_func) {
    ■■snd_error(HWDEP, "symbol %s is not defined inside %s", open_name, lib);
    ■■snd_dlclose(h);
    ■■err = -ENXIO;
    ■}

    _err:
    ■if (type_conf)
    ■■snd_config_delete(type_conf);
    ■if (err >= 0) {
    ■■err = open_func(hwdep, name, hwdep_root, hwdep_conf, mode);
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 127) - [public] snd_error

```
    ■■snd_error(HWDEP, "symbol %s is not defined inside %s", open_name, lib);
    ■■snd_dlclose(h);
    ■■err = -ENXIO;
    ■}

    _err:
    ■if (type_conf)
    ■■snd_config_delete(type_conf);
    ■if (err >= 0) {
    ■■err = open_func(hwdep, name, hwdep_root, hwdep_conf, mode);
    ■■if (err >= 0) {
    ■■■■(*hwdep)->dl_handle = h;
    ■■} else {
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 128) - [public] snd_dlclose

```
    ■■snd_dlclose(h);
    ■■err = -ENXIO;
    ■}

    _err:
    ■if (type_conf)
    ■■snd_config_delete(type_conf);
    ■if (err >= 0) {
    ■■err = open_func(hwdep, name, hwdep_root, hwdep_conf, mode);
    ■■if (err >= 0) {
    ■■■■(*hwdep)->dl_handle = h;
    ■■} else {
    ■■■snd_dlclose(h);
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 133) - [public] snd_config_delete

```
■ ■ snd_config_delete(type_conf);
■ ■ if (err >= 0) {
■ ■ err = open_func(hwdep, name, hwdep_root, hwdep_conf, mode);
■ ■ if (err >= 0) {
■ ■ ■ (*hwdep)->dl_handle = h;
■ ■ } else {
■ ■ ■ snd_dlclose(h);
■ ■ }
■ }
■ return err;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 136) - [public] if

```
■ ■ if (err >= 0) {
■ ■ ■ (*hwdep)->dl_handle = h;
■ ■ } else {
■ ■ ■ snd_dlclose(h);
■ ■ }
■ }
■ return err;
}
```

```
static int snd_hwdep_open_noupdate(snd_hwdep_t **hwdep, snd_config_t *root, const char *name, int mode)
{
■ int err;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 139) - [public] snd_dlclose

```
■ ■ ■ snd_dlclose(h);
■ ■ }
■ }
■ return err;
}
```

```
static int snd_hwdep_open_noupdate(snd_hwdep_t **hwdep, snd_config_t *root, const char *name, int mode)
{
■ int err;
■ snd_config_t *hwdep_conf;
■ err = snd_config_search_definition(root, "hwdep", name, &hwdep_conf);
■ if (err < 0) {
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 151) - [public] snd_error

```
■ ■ snd_error(HWDEP, "Unknown HwDep %s", name);
■ ■ return err;
■ }
■ err = snd_hwdep_open_conf(hwdep, name, root, hwdep_conf, mode);
■ snd_config_delete(hwdep_conf);
■ return err;
}
```

```
/**
 * \brief Opens a new connection to the HwDep interface.
 * \param hwdep Returned handle (NULL if not wanted)
 * \param name ASCII identifier of the HwDep handle
 // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 198) - [public] snd_hwdep_open_noupdate

```
    return snd_hwdep_open_noupdate(hwdep, lconf, name, mode);
}

/**
 * \brief close HwDep handle
 * \param hwdep HwDep handle
 * \return 0 on success otherwise a negative error code
 *
 * Closes the specified HwDep handle and frees all associated
 * resources.
 */
int snd_hwdep_close(snd_hwdep_t *hwdep)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 215) - [public] snd_dlclose

```
    snd_dlclose(hwdep->dl_handle);
    free(hwdep->name);
    free(hwdep);
    return err;
}

/**
 * \brief get identifier of HwDep handle
 * \param hwdep a Hwdep handle
 * \return ascii identifier of HwDep handle
 *
 * Returns the ASCII identifier of given HwDep handle. It's the same
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 271) - [public] switch

```
    switch (hwdep->mode & 0_ACCMODE) {
        case 0_WRONLY:
            pfd->events = POLLOUT|POLLERR|POLLNVAL;
            break;
        case 0_RDONLY:
            pfd->events = POLLIN|POLLERR|POLLNVAL;
            break;
        case 0_RDWR:
            pfd->events = POLLOUT|POLLIN|POLLERR|POLLNVAL;
            break;
        default:
            return -EIO;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 332) - [public] sizeof

```
    return sizeof(snd_hwdep_info_t);
}

/**
 * \brief allocate a new snd_hwdep_info_t structure
 * \param info returned pointer
 * \return 0 on success otherwise a negative error code if fails
 *
 * Allocates a new snd_hwdep_info_t structure using the standard
 * malloc C library function.
 */
int snd_hwdep_info_malloc(snd_hwdep_info_t **info)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 528) - [public] sizeof

```
    return sizeof(snd_hwdep_dsp_status_t);
}

/**
 * \brief allocate a new snd_hwdep_dsp_status_t structure
 * \param info returned pointer
 * \return 0 on success otherwise a negative error code if fails
 *
 * Allocates a new snd_hwdep_dsp_status_t structure using the standard
 * malloc C library function.
 */
int snd_hwdep_dsp_status_malloc(snd_hwdep_dsp_status_t **info)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 633) - [public] sizeof

```
    return sizeof(snd_hwdep_dsp_image_t);
}

/**
 * \brief allocate a new snd_hwdep_dsp_image_t structure
 * \param info returned pointer
 * \return 0 on success otherwise a negative error code if fails
 *
 * Allocates a new snd_hwdep_dsp_image_t structure using the standard
 * malloc C library function.
 */
int snd_hwdep_dsp_image_malloc(snd_hwdep_dsp_image_t **info)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 120) - [public] snd_card_load

```
    snd_card_load(card);
    fd = snd_open_device(filename, mode);
    if (fd < 0)
        return -errno;
}
if (ioctl(fd, SNDRV_HWDEP_IOCTL_PVERSION, &ver) < 0) {
    ret = -errno;
    close(fd);
    return ret;
}
if (SNDRV_PROTOCOL_INCOMPATIBLE(ver, SNDRV_HWDEP_VERSION_MAX)) {
    close(fd);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 127) - [public] close

```
    close(fd);
    return ret;
}
if (SNDRV_PROTOCOL_INCOMPATIBLE(ver, SNDRV_HWDEP_VERSION_MAX)) {
    close(fd);
    return -SND_ERROR_INCOMPATIBLE_VERSION;
}
hwdep = (snd_hwdep_t *) calloc(1, sizeof(snd_hwdep_t));
if (hwdep == NULL) {
    close(fd);
    return -ENOMEM;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 131) - [public] close

```
    close(fd);
    return -SND_ERROR_INCOMPATIBLE_VERSION;
}
hwdep = (snd_hwdep_t *) calloc(1, sizeof(snd_hwdep_t));
if (hwdep == NULL) {
    close(fd);
    return -ENOMEM;
}
hwdep->name = strdup(name);
hwdep->poll_fd = fd;
hwdep->mode = mode;
hwdep->type = SND_HWDEP_TYPE_HW;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 136) - [public] close

```
    close(fd);
    return -ENOMEM;
}
hwdep->name = strdup(name);
hwdep->poll_fd = fd;
hwdep->mode = mode;
hwdep->type = SND_HWDEP_TYPE_HW;
hwdep->ops = &snd_hwdep_hw_ops;
*handle = hwdep;
return 0;
}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 162) - [public] if

```
    if (strcmp(id, "card") == 0) {
        err = snd_config_get_card(n);
        if (err < 0)
            return err;
        card = err;
        continue;
    }
    if (strcmp(id, "device") == 0) {
        err = snd_config_get_integer(n, &device);
        if (err < 0)
            return err;
        continue;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 169) - [public] if

```
    if (strcmp(id, "device") == 0) {
        err = snd_config_get_integer(n, &device);
        if (err < 0)
            return err;
        continue;
    }
    snd_error(HWDEP, "Unexpected field %s", id);
    return -EINVAL;
}
if (card < 0)
    return -EINVAL;
return snd_hwdep_hw_open(hwdep, name, card, device, mode);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 175) - [public] snd_error

```
    snd_error(HWDEP, "Unexpected field %s", id);
    return -EINVAL;
}
if (card < 0)
    return -EINVAL;
return snd_hwdep_hw_open(hwdep, name, card, device, mode);
}
SND_DLSYM_BUILD_VERSION(_snd_hwdep_hw_open, SND_HWDEP_DLSYM_VERSION);
```

dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 180) - [public] snd_hwdep_hw_open

```
    return snd_hwdep_hw_open(hwdep, name, card, device, mode);
}
SND_DLSYM_BUILD_VERSION(_snd_hwdep_hw_open, SND_HWDEP_DLSYM_VERSION);
```

dirtycode/alsa-lib/src/input.c (line 130) - [public] fclose

```
    fclose(stdio->fp);
    free(stdio);
    return 0;
}

static int snd_input_stdio_scan(snd_input_t *input, const char *format, va_list args)
{
    snd_input_stdio_t *stdio = input->private_data;
    extern int vfscanf(FILE *, const char *, va_list);
    return vfscanf(stdio->fp, format, args);
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/input.c (line 138) - [public] vfscanf

```
    extern int vfscanf(FILE *, const char *, va_list);
    return vfscanf(stdio->fp, format, args);
}

static char *snd_input_stdio_gets(snd_input_t *input, char *str, size_t size)
{
    snd_input_stdio_t *stdio = input->private_data;
    return fgets(str, (int) size, stdio->fp);
}

static int snd_input_stdio_getc(snd_input_t *input)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/input.c (line 139) - [public] vfscanf

```
    return vfscanf(stdio->fp, format, args);
}

static char *snd_input_stdio_gets(snd_input_t *input, char *str, size_t size)
{
    snd_input_stdio_t *stdio = input->private_data;
    return fgets(str, (int) size, stdio->fp);
}

static int snd_input_stdio_getc(snd_input_t *input)
{
    snd_input_stdio_t *stdio = input->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/input.c (line 145) - [public] fgets

```
    return fgets(str, (int) size, stdio->fp);
}

static int snd_input_stdio_getc(snd_input_t *input)
{
    snd_input_stdio_t *stdio = input->private_data;
    return getc(stdio->fp);
}

static int snd_input_stdio_ungetc(snd_input_t *input, int c)
{
    snd_input_stdio_t *stdio = input->private_data;
    // ... (truncated, see source file for more)
```



```
dirtycode/alsa-lib/src/input.c (line 151) - [public] getc
    return getc(stdio->fp);
}

static int snd_input_stdio_ungetc(snd_input_t *input, int c)
{
    return ungetc(c, stdio->fp);
}

static const snd_input_ops_t snd_input_stdio_ops = {
    .close = snd_input_stdio_close,
    .scan = snd_input_stdio_scan,
    // ... (truncated, see source file for more)
}
```

```
dirtycode/alsa-lib/src/input.c (line 157) - [public] ungetc
    return ungetc(c, stdio->fp);
}

static const snd_input_ops_t snd_input_stdio_ops = {
    .close = snd_input_stdio_close,
    .scan = snd_input_stdio_scan,
    .gets = snd_input_stdio_gets,
    .getch = snd_input_stdio_getc,
    .ungetch = snd_input_stdio_ungetc,
};
#endif

// ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/input.c (line 189) - [public] free
    free(stdio);
    return -ENOMEM;
}

stdio->fp = fp;
stdio->close = _close;
input->type = SND_INPUT_STDIO;
input->ops = &snd_input_stdio_ops;
input->private_data = stdio;
*inputp = input;
return 0;
}

// ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/input.c (line 219) - [public] fclose
    fclose(fp);
    return err;
}

#ifndef DOC_HIDDEN

typedef struct _snd_input_buffer {
    unsigned char *buf;
    unsigned char *ptr;
    size_t size;
} snd_input_buffer_t;

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/input.c (line 242) - [public] vsscanf

```
■extern int vsscanf(const char *, const char *, va_list);
■/* FIXME: how can I obtain consumed chars count? */
■assert(0);
■return vsscanf((char *)buffer->ptr, format, args);
}
```

```
static char *snd_input_buffer_gets(snd_input_t *input, char *str, size_t size)
{
■snd_input_buffer_t *buffer = input->private_data;
■size_t bsize = buffer->size;
■while (--size > 0 && bsize > 0) {
■■unsigned char c = *buffer->ptr++;
■   // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/input.c (line 245) - [public] vsscanf

```
■return vsscanf((char *)buffer->ptr, format, args);
}
```

```
static char *snd_input_buffer_gets(snd_input_t *input, char *str, size_t size)
{
■snd_input_buffer_t *buffer = input->private_data;
■size_t bsize = buffer->size;
■while (--size > 0 && bsize > 0) {
■■unsigned char c = *buffer->ptr++;
■■bsize--;
■■*str++ = c;
■■if (c == '\n')
■   // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/input.c (line 316) - [public] free

```
■■free(buffer);
■■return -ENOMEM;
■}
■if (size < 0)
■■size = strlen(buf);
■buffer->buf = malloc((size_t)size + 1);
■if (!buffer->buf) {
■■free(input);
■■free(buffer);
■■return -ENOMEM;
■}
■memcpy(buffer->buf, buf, (size_t) size);
■   // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/input.c (line 323) - [public] free

```
■■free(input);
■■free(buffer);
■■return -ENOMEM;
■}
■memcpy(buffer->buf, buf, (size_t) size);
■buffer->buf[size] = 0;
■buffer->ptr = buffer->buf;
■buffer->size = size;
■input->type = SND_INPUT_BUFFER;
■input->ops = &snd_input_buffer_ops;
■input->private_data = buffer;
■*inputp = input;
■   // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/input.c (line 324) - [public] free

```
    free(buffer);
    return -ENOMEM;
}
memcpy(buffer->buf, buf, (size_t) size);
buffer->buf[size] = 0;
buffer->ptr = buffer->buf;
buffer->size = size;
input->type = SND_INPUT_BUFFER;
input->ops = &snd_input_buffer_ops;
input->private_data = buffer;
*inputp = input;
return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/bag.c (line 42) - [public] list_empty

```
return list_empty(bag);
}

int bag_add(bag_t *bag, void *ptr)
{
    bag_t *b = malloc(sizeof(*b));
    if (!b)
        return -ENOMEM;
    b->ptr = ptr;
    list_add_tail(&b->list, bag);
    return 0;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/bag.c (line 60) - [public] if

```
if (b->ptr == ptr) {
    list_del(&b->list);
    free(b);
    return 0;
}
return -ENOENT;
}

void bag_del_all(bag_t *bag)
{
    struct list_head *pos, *npos;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/bag.c (line 61) - [public] list_del

```
list_del(&b->list);
free(b);
return 0;
}
return -ENOENT;
}

void bag_del_all(bag_t *bag)
{
    struct list_head *pos, *npos;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/bag.c (line 62) - [public] free

```
    free(b);
    return 0;
}
}
return -ENOENT;
}
```

```
void bag_del_all(bag_t *bag)
{
    struct list_head *pos, *npos;
```

```
    list_for_each_safe(pos, npos, bag) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/bag.c (line 75) - [public] list_del

```
    list_del(&b->list);
    free(b);
}
}
```

dirtycode/alsa-lib/src/mixer/bag.c (line 76) - [public] free

```
    free(b);
}
}
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 104) - [public] bag_add

```
    return bag_add(&melem->helems, helem);
}
```

```
/**
 * \brief Detach an HCTL element from a mixer element
 * \param melem Mixer element
 * \param helem HCTL element
 * \return 0 on success otherwise a negative error code
 *
 * For use by mixer element class specific code.
 *
 * The implementation of mixer class typically calls it at #SND_CTL_EVENT_MASK_REMOVE event for
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 140) - [public] bag_empty

```
    return bag_empty(&melem->helems);
}
```

```
static int hctl_elem_event_handler(snd_hctl_elem_t *helem,
    unsigned int mask)
{
    bag_t *bag = snd_hctl_elem_get_callback_private(helem);
    if (mask == SND_CTL_EVENT_MASK_REMOVE) {
        int res = 0;
        int err;
        bag_iterator_t i, n;
        bag_for_each_safe(i, n, bag) {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 151) - [public] bag_for_each_safe

```
    bag_for_each_safe(i, n, bag) {
        snd_mixer_elem_t *melem = bag_iterator_entry(i);
        snd_mixer_class_t *class = melem->class;
        err = class->event(class, mask, helem, melem);
        if (err < 0)
            res = err;
    }
    // NOTE: Unsatisfied postcondition. Typically, some of registered implementation of
    // mixer class forget to detach mixer element from hcontrol element which has been
    // attached at ADD event.
    assert(bag_empty(bag));
    bag_free(bag);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 161) - [public] assert

*// NOTE: Unsatisfied postcondition. Typically, some of registered implementation of
// mixer class forget to detach mixer element from hcontrol element which has been
// attached at ADD event.*

```
    ■■assert(bag_empty(bag));
    ■■bag_free(bag);
    ■■return res;
    ■}
    ■if (mask & (SND_CTL_EVENT_MASK_VALUE | SND_CTL_EVENT_MASK_INFO)) {
    ■■int err = 0;
    ■■bag_iterator_t i, n;
    ■■bag_for_each_safe(i, n, bag) {
    ■■■snd_mixer_elem_t *melem = bag_iterator_entry(i);
    ■■■snd_mixer_class_t *class = melem->class;
    ■■■err = class->event(class, mask, helem, melem);
    ■■■if (err < 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 162) - [public] bag_free

```
    ■■bag_free(bag);
    ■■return res;
    ■}
    ■if (mask & (SND_CTL_EVENT_MASK_VALUE | SND_CTL_EVENT_MASK_INFO)) {
    ■■int err = 0;
    ■■bag_iterator_t i, n;
    ■■bag_for_each_safe(i, n, bag) {
    ■■■snd_mixer_elem_t *melem = bag_iterator_entry(i);
    ■■■snd_mixer_class_t *class = melem->class;
    ■■■err = class->event(class, mask, helem, melem);
    ■■■if (err < 0)
    ■■■■return err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 168) - [public] bag_for_each_safe

```
    ■■bag_for_each_safe(i, n, bag) {
    ■■■snd_mixer_elem_t *melem = bag_iterator_entry(i);
    ■■■snd_mixer_class_t *class = melem->class;
    ■■■err = class->event(class, mask, helem, melem);
    ■■■if (err < 0)
    ■■■■return err;
    ■■}
    ■}
    ■return 0;
    }
```

```
static int hctl_event_handler(snd_hctl_t *hctl, unsigned int mask,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 190) - [public] snd_hctl_elem_set_callback

```
    ■■snd_hctl_elem_set_callback(elem, hctl_elem_event_handler);
    ■■snd_hctl_elem_set_callback_private(elem, bag);
    ■■list_for_each(pos, &mixer->classes) {
    ■■■snd_mixer_class_t *c;
    ■■■c = list_entry(pos, snd_mixer_class_t, list);
    ■■■err = c->event(c, mask, elem, NULL);
    ■■■if (err < 0)
    ■■■■res = err;
    ■■}
    ■}
    ■return res;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 191) - [public] snd_hctl_elem_set_callback_private

```
■snd_hctl_elem_set_callback_private(elem, bag);
■list_for_each(pos, &mixer->classes) {
■    ■snd_mixer_class_t *c;
■    ■c = list_entry(pos, snd_mixer_class_t, list);
■    ■err = c->event(c, mask, elem, NULL);
■    ■if (err < 0)
■        ■res = err;
■}
■}
■return res;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/mixer/mixer.c (line 192) - [public] list_for_each

```
■list_for_each(pos, &mixer->classes) {
■    ■snd_mixer_class_t *c;
■    ■c = list_entry(pos, snd_mixer_class_t, list);
■    ■err = c->event(c, mask, elem, NULL);
■    ■if (err < 0)
■        ■res = err;
■}
■}
■return res;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/mixer/mixer.c (line 240) - [public] snd_hctl_close

```
■snd_hctl_close(hctl);
■return -ENOMEM;
■}
■err = snd_hctl_nonblock(hctl, 1);
■if (err < 0) {
■    ■snd_hctl_close(hctl);
■    ■free(slave);
■    ■return err;
■}
■snd_hctl_set_callback(hctl, hctl_event_handler);
■snd_hctl_set_callback_private(hctl, mixer);
■slave->hctl = hctl;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 245) - [public] snd_hctl_close

```
■snd_hctl_close(hctl);
■free(slave);
■return err;
■}
■snd_hctl_set_callback(hctl, hctl_event_handler);
■snd_hctl_set_callback_private(hctl, mixer);
■slave->hctl = hctl;
■list_add_tail(&slave->list, &mixer->slaves);
■return 0;
}
```

/**

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/mixer/mixer.c (line 246) - [public] free

```
    free(slave);
    return err;
}
snd_hctl_set_callback(hctl, hctl_event_handler);
snd_hctl_set_callback_private(hctl, mixer);
slave->hctl = hctl;
list_add_tail(&slave->list, &mixer->slaves);
return 0;
}

/**
 * \brief Detach a previously attached HCTL to an opened mixer freeing all related resources
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 268) - [public] if

```
    if (strcmp(name, snd_hctl_name(s->hctl)) == 0) {
        snd_hctl_close(s->hctl);
        list_del(pos);
        free(s);
        return 0;
    }
}
return -ENOENT;
}

/**
 * \brief Detach a previously attached HCTL to an opened mixer freeing all related resources
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 269) - [public] snd_hctl_close

```
    snd_hctl_close(s->hctl);
    list_del(pos);
    free(s);
    return 0;
}
}
return -ENOENT;
}

/**
 * \brief Detach a previously attached HCTL to an opened mixer freeing all related resources
 * \param mixer Mixer handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 270) - [public] list_del

```
    list_del(pos);
    free(s);
    return 0;
}
}
return -ENOENT;
}

/**
 * \brief Detach a previously attached HCTL to an opened mixer freeing all related resources
 * \param mixer Mixer handle
 * \param hctl HCTL previously attached
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 271) - [public] free

```
    free(s);
    return 0;
}
return -ENOENT;
}

/**
 * \brief Detach a previously attached HCTL to an opened mixer freeing all related resources
 * \param mixer Mixer handle
 * \param hctl HCTL previously attached
 * \return 0 on success otherwise a negative error code
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 292) - [public] if

```
    if (hctl == s->hctl) {
        list_del(pos);
        free(s);
        return 0;
    }
    return -ENOENT;
}

/**
 * \brief Obtain a HCTL pointer associated to given name
 * \param mixer Mixer handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 293) - [public] list_del

```
    list_del(pos);
    free(s);
    return 0;
}
return -ENOENT;
}

/**
 * \brief Obtain a HCTL pointer associated to given name
 * \param mixer Mixer handle
 * \param name HCTL previously attached
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 294) - [public] free

```
    free(s);
    return 0;
}
return -ENOENT;
}

/**
 * \brief Obtain a HCTL pointer associated to given name
 * \param mixer Mixer handle
 * \param name HCTL previously attached
 * \param hctl HCTL pointer
 * // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/mixer/mixer.c (line 314) - [public] if

```
    if (strcmp(name, snd_hctl_name(s->hctl)) == 0) {
        *hctl = s->hctl;
        return 0;
    }
    return -ENOENT;
}

static int snd_mixer_throw_event(snd_mixer_t *mixer, unsigned int mask,
    snd_mixer_elem_t *elem)
{
    mixer->events++;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 421) - [public] if

```
    if (!m) {
        mixer->alloc -= 32;
        return -ENOMEM;
    }
    mixer->pelems = m;
}
if (mixer->count == 0) {
    list_add_tail(&elem->list, &mixer->elems);
    mixer->pelems[0] = elem;
} else {
    idx = _snd_mixer_find_elem(mixer, elem, &dir);
    assert(dir != 0);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 428) - [public] list_add_tail

```
    list_add_tail(&elem->list, &mixer->elems);
    mixer->pelems[0] = elem;
} else {
    idx = _snd_mixer_find_elem(mixer, elem, &dir);
    assert(dir != 0);
    if (dir > 0) {
        list_add(&elem->list, &mixer->pelems[idx]->list);
        idx++;
    } else {
        list_add_tail(&elem->list, &mixer->pelems[idx]->list);
    }
    memmove(mixer->pelems + idx + 1,
        // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 432) - [public] assert

```
    assert(dir != 0);
    if (dir > 0) {
        list_add(&elem->list, &mixer->pelems[idx]->list);
        idx++;
    } else {
        list_add_tail(&elem->list, &mixer->pelems[idx]->list);
    }
    memmove(mixer->pelems + idx + 1,
        mixer->pelems + idx,
        (mixer->count - idx) * sizeof(snd_mixer_elem_t *));
    mixer->pelems[idx] = elem;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 433) - [public] if

```
    if (dir > 0) {
        list_add(&elem->list, &mixer->pelems[idx]->list);
        idx++;
    } else {
        list_add_tail(&elem->list, &mixer->pelems[idx]->list);
    }
    memmove(mixer->pelems + idx + 1,
            mixer->pelems + idx,
            (mixer->count - idx) * sizeof(snd_mixer_elem_t *));
    mixer->pelems[idx] = elem;
}
mixer->count++;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 434) - [public] list_add

```
list_add(&elem->list, &mixer->pelems[idx]->list);
idx++;
} else {
    list_add_tail(&elem->list, &mixer->pelems[idx]->list);
}
memmove(mixer->pelems + idx + 1,
        mixer->pelems + idx,
        (mixer->count - idx) * sizeof(snd_mixer_elem_t *));
mixer->pelems[idx] = elem;
}
mixer->count++;
return snd_mixer_throw_event(mixer, SND_CTL_EVENT_MASK_ADD, elem);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 437) - [public] list_add_tail

```
list_add_tail(&elem->list, &mixer->pelems[idx]->list);
}
memmove(mixer->pelems + idx + 1,
        mixer->pelems + idx,
        (mixer->count - idx) * sizeof(snd_mixer_elem_t *));
mixer->pelems[idx] = elem;
}
mixer->count++;
return snd_mixer_throw_event(mixer, SND_CTL_EVENT_MASK_ADD, elem);
}

/**
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 445) - [public] snd_mixer_throw_event

```
return snd_mixer_throw_event(mixer, SND_CTL_EVENT_MASK_ADD, elem);
}

/**
 * \brief Remove a mixer element
 * \param elem Mixer element
 * \return 0 on success otherwise a negative error code
 *
 * For use by mixer element class specific code.
 */
int snd_mixer_elem_remove(snd_mixer_elem_t *elem)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 468) - [public] snd_mixer_elem_detach

```
■■snd_mixer_elem_detach(elem, helem);
■}
■err = snd_mixer_elem_throw_event(elem, SND_CTL_EVENT_MASK_REMOVE);
■list_del(&elem->list);
■snd_mixer_elem_free(elem);
■mixer->count--;
■m = mixer->count - idx;
■if (m > 0)
■■memmove(mixer->pelems + idx,
■■■mixer->pelems + idx + 1,
■■■m * sizeof(snd_mixer_elem_t *));
■return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 478) - [public] sizeof

```
■■■m * sizeof(snd_mixer_elem_t *));
■return err;
}

/**
 * \brief Free a mixer element
 * \param elem Mixer element
 *
 * For use by mixer element class specific code.
 */
void snd_mixer_elem_free(snd_mixer_elem_t *elem)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 504) - [public] snd_mixer_elem_throw_event

```
■return snd_mixer_elem_throw_event(elem, SND_CTL_EVENT_MASK_INFO);
}

/**
 * \brief Mixer element values is changed
 * \param elem Mixer element
 * \return 0 on success otherwise a negative error code
 *
 * For use by mixer element class specific code.
 */
int snd_mixer_elem_value(snd_mixer_elem_t *elem)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 516) - [public] snd_mixer_elem_throw_event

```
■return snd_mixer_elem_throw_event(elem, SND_CTL_EVENT_MASK_VALUE);
}

/**
 * \brief Register mixer element class
 * \param class Mixer element class
 * \param mixer Mixer handle
 * \return 0 on success otherwise a negative error code
 *
 * For use by mixer element class specific code.
 */
int snd_mixer_class_register(snd_mixer_class_t *class, snd_mixer_t *mixer)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 540) - [public] while

```
    while (elem) {
        err = class->event(class, SND_CTL_EVENT_MASK_ADD, elem, NULL);
        if (err < 0)
            return err;
        elem = snd_hctl_elem_next(elem);
    }
}

return 0;
}

/**
 * \brief Unregister mixer element class and remove all its elements
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 565) - [public] snd_mixer_elem_remove

```
    snd_mixer_elem_remove(e);
}

if (class->private_free)
    class->private_free(class);
list_del(&class->list);
free(class);
return 0;
}

/**
 * \brief Load a mixer elements
 * \param mixer Mixer handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 603) - [public] snd_hctl_free

```
    snd_hctl_free(s->hctl);
}

}

/**
 * \brief Close a mixer and free all related resources
 * \param mixer Mixer handle
 * \return 0 on success otherwise a negative error code
 */
int snd_mixer_close(snd_mixer_t *mixer)
{
    int res = 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 619) - [public] snd_mixer_class_unregister

```
    snd_mixer_class_unregister(c);
}

assert(list_empty(&mixer->elems));
assert(mixer->count == 0);
free(mixer->pelems);
mixer->pelems = NULL;
while (!list_empty(&mixer->slaves)) {
    int err;
    snd_mixer_slave_t *s;
    s = list_entry(mixer->slaves.next, snd_mixer_slave_t, list);
    err = snd_hctl_close(s->hctl);
    if (err < 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 632) - [public] list_del

```
    list_del(&s->list);
    free(s);
}
free(mixer);
return res;
}

static int snd_mixer_compare_default(const snd_mixer_elem_t *c1,
    const snd_mixer_elem_t *c2)
{
    int d = c1->compare_weight - c2->compare_weight;
    if (d)
        // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 633) - [public] free

```
    free(s);
}
free(mixer);
return res;
}

static int snd_mixer_compare_default(const snd_mixer_elem_t *c1,
    const snd_mixer_elem_t *c2)
{
    int d = c1->compare_weight - c2->compare_weight;
    if (d)
        return d;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 667) - [public] list_add_tail

```
    list_add_tail(&mixer->pelems[k]->list, &mixer->elems);
    return 0;
}

/**
 * \brief Change mixer compare function and reorder elements
 * \param mixer Mixer handle
 * \param compare Element compare function
 * \return 0 on success otherwise a negative error code
 */
int snd_mixer_set_compare(snd_mixer_t *mixer, snd_mixer_compare_t compare)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 733) - [public] if

```
    if (space >= (unsigned int) n) {
        count += n;
        space -= n;
        pfd += n;
    } else
        space = 0;
}
return count;
}

/**
 * \brief get returned events from poll descriptors
 * // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 786) - [public] assert

```
    ■■assert(err == count);
    ■}
    ■err = poll(pfds, (unsigned int) count, timeout);
    ■if (err < 0)
    ■■return -errno;
    ■return 0;
    }

    /**
     * \brief get first element for a mixer
     * \param mixer Mixer handle
     * \return pointer to first element
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 804) - [public] list_entry

```
    ■return list_entry(mixer->elems.next, snd_mixer_elem_t, list);
    }

    /**
     * \brief get last element for a mixer
     * \param mixer Mixer handle
     * \return pointer to last element
     */
    snd_mixer_elem_t *snd_mixer_last_elem(snd_mixer_t *mixer)
    {
        ■assert(mixer);
        ■if (list_empty(&mixer->elems))
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 817) - [public] list_entry

```
    ■return list_entry(mixer->elems.prev, snd_mixer_elem_t, list);
    }

    /**
     * \brief get next mixer element
     * \param elem mixer element
     * \return pointer to next element
     */
    snd_mixer_elem_t *snd_mixer_elem_next(snd_mixer_elem_t *elem)
    {
        ■assert(elem);
        ■if (elem->list.next == &elem->class->mixer->elems)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 830) - [public] list_entry

```
    ■return list_entry(elem->list.next, snd_mixer_elem_t, list);
    }

    /**
     * \brief get previous mixer element
     * \param elem mixer element
     * \return pointer to previous element
     */
    snd_mixer_elem_t *snd_mixer_elem_prev(snd_mixer_elem_t *elem)
    {
        ■assert(elem);
        ■if (elem->list.prev == &elem->class->mixer->elems)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 843) - [public] list_entry

```
    return list_entry(elem->list.prev, snd_mixer_elem_t, list);
}

/**
 * \brief Handle pending mixer events invoking callbacks
 * \param mixer Mixer handle
 * \return Number of events that occurred on success, otherwise a negative error code on failure
 */
int snd_mixer_handle_events(snd_mixer_t *mixer)
{
    struct list_head *pos;
    assert(mixer);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/mixer.c (line 962) - [public] sizeof

```
    return sizeof(snd_mixer_class_t);
}

/**
 * \brief allocate an invalid #snd_mixer_class_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_mixer_class_malloc(snd_mixer_class_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_mixer_class_t));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 68) - [public] if

```
    if (options != NULL) {
        err = snd_mixer_attach(mixer, options->device);
        if (err < 0)
            return err;
    }
    return 0;
} else if (options->ver == 1) {
    if (options->abstract == SND_MIXER_SABSTRACT_BASIC)
        return snd_mixer_simple_basic_register(mixer, options, classp);
}
return -ENXIO;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 76) - [public] snd_mixer_simple_basic_register

```
    return snd_mixer_simple_basic_register(mixer, options, classp);
}
return -ENXIO;
}

#ifndef DOC_HIDDEN

#define CHECK_BASIC(xelem) \
{ \
    assert(xelem); \
    assert((xelem)->type == SND_MIXER_ELEM_SIMPLE); \
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 199) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_GVOLUME);
}

/**
 * \brief Return true if mixer simple element has only one switch control for both playback and capture
 * \param elem Mixer simple element handle
 * \return 0 separated control, 1 common control
 */
int snd_mixer_selem_has_common_switch(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_GSWITCH);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 210) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_GSWITCH);
}

/**
 * \brief Return name of mixer simple element channel
 * \param channel mixer simple element channel identifier
 * \return channel name
 */
const char *snd_mixer_selem_channel_name(snd_mixer_selem_channel_id_t channel)
{
    static const char *const array[SND_MIXER_SCHN_LAST + 1] = {
        [SND_MIXER_SCHN_FRONT_LEFT] = "Front Left",
        // ... (truncated, see source file for more)
    };
    return array[channel];
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 247) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_ACTIVE, 0);
}

/**
 * \brief Get info about channels of playback stream of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if not mono, 1 if mono
 */
int snd_mixer_selem_is_playback_mono(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_MONO, 0);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 258) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_MONO, 0);
}

/**
 * \brief Get info about channels of playback stream of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel Mixer simple element channel identifier
 * \return 0 if channel is not present, 1 if present
 */
int snd_mixer_selem_has_playback_channel(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel)
{
    CHECK_BASIC(elem);
    // ... (truncated, see source file for more)
}
```



```
dirtycode/alsa-lib/src/mixer/simple.c (line 270) - [public] sm_selem_ops
    return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_CHANNEL, (int)channel);
}

/**
 * \brief Get range for playback volume of a mixer simple element
 * \param elem Mixer simple element handle
 * \param min Pointer to returned minimum
 * \param max Pointer to returned maximum
 */
int snd_mixer_selem_get_playback_volume_range(snd_mixer_elem_t *elem,
    long *min, long *max)
{
    // ... (truncated, see source file for more)
}
```

```
dirtycode/alsa-lib/src/mixer/simple.c (line 284) - [public] sm_selem_ops
    return sm_selem_ops(elem)->get_range(elem, SM_PLAY, min, max);
}

/**
 * \brief Get range in dB for playback volume of a mixer simple element
 * \param elem Mixer simple element handle
 * \param min Pointer to returned minimum (dB * 100)
 * \param max Pointer to returned maximum (dB * 100)
 */
int snd_mixer_selem_get_playback_dB_range(snd_mixer_elem_t *elem,
    long *min, long *max)
{
    // ... (truncated, see source file for more)
}
```

```
dirtycode/alsa-lib/src/mixer/simple.c (line 298) - [public] sm_selem_ops
    return sm_selem_ops(elem)->get_dB_range(elem, SM_PLAY, min, max);
}

/**
 * \brief Set range for playback volume of a mixer simple element
 * \param elem Mixer simple element handle
 * \param min minimum volume value
 * \param max maximum volume value
 */
int snd_mixer_selem_set_playback_volume_range(snd_mixer_elem_t *elem,
    long min, long max)
{
    // ... (truncated, see source file for more)
}
```

```
dirtycode/alsa-lib/src/mixer/simple.c (line 313) - [public] sm_selem_ops
    return sm_selem_ops(elem)->set_range(elem, SM_PLAY, min, max);
}

/**
 * \brief Return info about playback volume control of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if no control is present, 1 if it's present
 */
int snd_mixer_selem_has_playback_volume(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_PVOLUME);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 324) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_PVOLUME);
}

/**
 * \brief Return info about playback volume control of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if control is separated per channel, 1 if control acts on all channels together
 */
int snd_mixer_selem_has_playback_volume_joined(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_PVOLUME_JOIN);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 335) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_PVOLUME_JOIN);
}

/**
 * \brief Return info about playback switch control existence of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if no control is present, 1 if it's present
 */
int snd_mixer_selem_has_playback_switch(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_PSWITCH);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 346) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_PSWITCH);
}

/**
 * \brief Return info about playback switch control of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if control is separated per channel, 1 if control acts on all channels together
 */
int snd_mixer_selem_has_playback_switch_joined(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_PSWITCH_JOIN);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 357) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_PSWITCH_JOIN);
}

/**
 * \brief Return corresponding dB value to an integer playback volume for a mixer simple element
 * \param elem Mixer simple element handle
 * \param value value to be converted to dB range
 * \param dBvalue pointer to returned dB value
 * \return 0 on success otherwise a negative error code
 */
int snd_mixer_selem_ask_playback_vol_dB(snd_mixer_elem_t *elem, long value, long *dBvalue)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 371) - [public] sm_selem_ops

```
■return sm_selem_ops(elem)->ask_vol_db(elem, SM_PLAY, value, dBvalue);
}

/**
 * \brief Return corresponding integer playback volume for given dB value for a mixer simple element
 * \param elem Mixer simple element handle
 * \param value value to be converted to dB range
 * \param dir rounding mode - rounds up if dir > 0, round to nearest if dir == 0,
 *       rounds down if dir < 0
 * \param dBvalue pointer to returned dB value
 * \return 0 on success otherwise a negative error code
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 387) - [public] sm_selem_ops

```
■return sm_selem_ops(elem)->ask_db_vol(elem, SM_PLAY, dBvalue, value, dir);
}
```

```
/**
 * \brief Return value of playback volume control of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param value pointer to returned value
 * \return 0 on success otherwise a negative error code
 */
```

```
int snd_mixer_selem_get_playback_volume(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel, long *value)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 401) - [public] sm_selem_ops

```
■return sm_selem_ops(elem)->get_volume(elem, SM_PLAY, channel, value);
}
```

```
/**
 * \brief Return value of playback volume in dB control of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param value pointer to returned value (dB * 100)
 * \return 0 on success otherwise a negative error code
 */
```

```
int snd_mixer_selem_get_playback_db(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel, long *value)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 421) - [public] sm_selem_ops

```
■return sm_selem_ops(elem)->get_db(elem, SM_PLAY, channel, value);
}
```

```
/**
 * \brief Return value of playback switch control of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param value pointer to returned value
 * \return 0 on success otherwise a negative error code
 */
```

```
int snd_mixer_selem_get_playback_switch(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel, int *value)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 435) - [public] sm_selem_ops

```
■return sm_selem_ops(elem)->get_switch(elem, SM_PLAY, channel, value);  
}
```

```
/**
```

```
 * \brief Set value of playback volume control of a mixer simple element  
 * \param elem Mixer simple element handle  
 * \param channel mixer simple element channel identifier  
 * \param value control value  
 * \return 0 on success otherwise a negative error code  
 */
```

```
int snd_mixer_selem_set_playback_volume(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel, long value)  
{  
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 449) - [public] sm_selem_ops

```
■return sm_selem_ops(elem)->set_volume(elem, SM_PLAY, channel, value);  
}
```

```
/**
```

```
 * \brief Set value in dB of playback volume control of a mixer simple element  
 * \param elem Mixer simple element handle  
 * \param channel mixer simple element channel identifier  
 * \param value control value in dB * 100  
 * \param dir rounding mode - rounds up if dir > 0, round to nearest if dir == 0,  
 *                          rounds down if dir < 0  
 * \return 0 on success otherwise a negative error code  
 */
```

```
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 465) - [public] sm_selem_ops

```
■return sm_selem_ops(elem)->set_dB(elem, SM_PLAY, channel, value, dir);  
}
```

```
/**
```

```
 * \brief Set value of playback volume control for all channels of a mixer simple element  
 * \param elem Mixer simple element handle  
 * \param value control value  
 * \return 0 on success otherwise a negative error code  
 */
```

```
int snd_mixer_selem_set_playback_volume_all(snd_mixer_elem_t *elem, long value)  
{  
    ■snd_mixer_selem_channel_id_t chn;  
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 527) - [public] sm_selem_ops

```
■return sm_selem_ops(elem)->set_switch(elem, SM_PLAY, channel, value);  
}
```

```
/**
```

```
 * \brief Set value of playback switch control for all channels of a mixer simple element  
 * \param elem Mixer simple element handle  
 * \param value control value  
 * \return 0 on success otherwise a negative error code  
 */
```

```
int snd_mixer_selem_set_playback_switch_all(snd_mixer_elem_t *elem, int value)  
{  
    ■snd_mixer_selem_channel_id_t chn;  
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 563) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->is(elem, SM_CAPT, SM_OPS_IS_MONO, 0);
}

/**
 * \brief Get info about channels of capture stream of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel Mixer simple element channel identifier
 * \return 0 if channel is not present, 1 if present
 */
int snd_mixer_selem_has_capture_channel(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel)
{
    CHECK_BASIC(elem);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 576) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->is(elem, SM_CAPT, SM_OPS_IS_CHANNEL, channel);
}

/**
 * \brief Get range for capture volume of a mixer simple element
 * \param elem Mixer simple element handle
 * \param min Pointer to returned minimum
 * \param max Pointer to returned maximum
 */
int snd_mixer_selem_get_capture_volume_range(snd_mixer_elem_t *elem,
    long *min, long *max)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 590) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->get_range(elem, SM_CAPT, min, max);
}

/**
 * \brief Get range in dB for capture volume of a mixer simple element
 * \param elem Mixer simple element handle
 * \param min Pointer to returned minimum (dB * 100)
 * \param max Pointer to returned maximum (dB * 100)
 */
int snd_mixer_selem_get_capture_dB_range(snd_mixer_elem_t *elem,
    long *min, long *max)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 604) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->get_dB_range(elem, SM_CAPT, min, max);
}

/**
 * \brief Set range for capture volume of a mixer simple element
 * \param elem Mixer simple element handle
 * \param min minimum volume value
 * \param max maximum volume value
 */
int snd_mixer_selem_set_capture_volume_range(snd_mixer_elem_t *elem,
    long min, long max)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 619) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->set_range(elem, SM_CAPT, min, max);
}

/**
 * \brief Return info about capture volume control of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if no control is present, 1 if it's present
 */
int snd_mixer_selem_has_capture_volume(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_CVOLUME);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 630) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_CVOLUME);
}

/**
 * \brief Return info about capture volume control of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if control is separated per channel, 1 if control acts on all channels together
 */
int snd_mixer_selem_has_capture_volume_joined(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_CVOLUME_JOIN);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 641) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_CVOLUME_JOIN);
}

/**
 * \brief Return info about capture switch control existence of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if no control is present, 1 if it's present
 */
int snd_mixer_selem_has_capture_switch(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_CSWITCH);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 652) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_CSWITCH);
}

/**
 * \brief Return info about capture switch control of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if control is separated per channel, 1 if control acts on all channels together
 */
int snd_mixer_selem_has_capture_switch_joined(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_CSWITCH_JOIN);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 663) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_CSWITCH_JOIN);
}

/**
 * \brief Return info about capture switch control of a mixer simple element
 * \param elem Mixer simple element handle
 * \return 0 if control is separated per element, 1 if control acts on other elements too (i.e. only one)
 */
int snd_mixer_selem_has_capture_switch_exclusive(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return COND_CAPS(elem, SM_CAP_CSWITCH_EXCL);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 674) - [public] COND_CAPS

```
    return COND_CAPS(elem, SM_CAP_CSWITCH_EXCL);
}

/**
 * \brief Return info about capture switch control of a mixer simple element
 * \param elem Mixer simple element handle
 * \return group for switch exclusivity (see #snd_mixer_selem_has_capture_switch_exclusive)
 */
int snd_mixer_selem_get_capture_group(snd_mixer_elem_t *elem)
{
    sm_selem_t *s;
    CHECK_BASIC(elem);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 703) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->ask_vol_db(elem, SM_CAPT, value, dBvalue);
}

/**
 * \brief Return corresponding integer capture volume for given dB value for a mixer simple element
 * \param elem Mixer simple element handle
 * \param dBvalue dB value to be converted to integer range
 * \param value pointer to returned integer value
 * \param dir rounding mode - rounds up if dir > 0, round to nearest if dir == 0,
 *       rounds down if dir < 0
 * \return 0 on success otherwise a negative error code
 */
// ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 719) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->ask_db_vol(elem, SM_CAPT, dBvalue, value, dir);
}

/**
 * \brief Return value of capture volume control of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param value pointer to returned value
 * \return 0 on success otherwise a negative error code
 */
int snd_mixer_selem_get_capture_volume(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel, long
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 733) - [public] sm_selem_ops

```
return sm_selem_ops(elem)->get_volume(elem, SM_CAPT, channel, value);
}
```

```
/**
```

```
 * \brief Return value of capture volume in dB control of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param value pointer to returned value (dB * 100)
 * \return 0 on success otherwise a negative error code
 */
```

```
int snd_mixer_selem_get_capture_db(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel, long *value)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 747) - [public] sm_selem_ops

```
return sm_selem_ops(elem)->get_db(elem, SM_CAPT, channel, value);
}
```

```
/**
```

```
 * \brief Return value of capture switch control of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param value pointer to returned value
 * \return 0 on success otherwise a negative error code
 */
```

```
int snd_mixer_selem_get_capture_switch(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel, int *value)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 761) - [public] sm_selem_ops

```
return sm_selem_ops(elem)->get_switch(elem, SM_CAPT, channel, value);
}
```

```
/**
```

```
 * \brief Set value of capture volume control of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param value control value
 * \return 0 on success otherwise a negative error code
 */
```

```
int snd_mixer_selem_set_capture_volume(snd_mixer_elem_t *elem, snd_mixer_selem_channel_id_t channel, long value)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple.c (line 775) - [public] sm_selem_ops

```
return sm_selem_ops(elem)->set_volume(elem, SM_CAPT, channel, value);
}
```

```
/**
```

```
 * \brief Set value in dB of capture volume control of a mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param value control value in dB * 100
 * \param dir rounding mode - rounds up if dir > 0, round to nearest if dir == 0,
 *                          rounds down if dir < 0
 * \return 0 on success otherwise a negative error code
 */
```

```
    // ... (truncated, see source file for more)
}
```



```

dirtycode/alsa-lib/src/mixer/simple.c (line 791) - [public] sm_selem_ops
    return sm_selem_ops(elem)->set_db(elem, SM_CAPT, channel, value, dir);
}

/**
 * \brief Set value of capture volume control for all channels of a mixer simple element
 * \param elem Mixer simple element handle
 * \param value control value
 * \return 0 on success otherwise a negative error code
 */
int snd_mixer_selem_set_capture_volume_all(snd_mixer_elem_t *elem, long value)
{
    return sm_selem_ops(elem)->set_db(elem, SM_CAPT, channel, value, dir);
    // ... (truncated, see source file for more)
}

dirtycode/alsa-lib/src/mixer/simple.c (line 853) - [public] sm_selem_ops
    return sm_selem_ops(elem)->set_switch(elem, SM_CAPT, channel, value);
}

/**
 * \brief Set value of capture switch control for all channels of a mixer simple element
 * \param elem Mixer simple element handle
 * \param value control value
 * \return 0 on success otherwise a negative error code
 */
int snd_mixer_selem_set_capture_switch_all(snd_mixer_elem_t *elem, int value)
{
    return sm_selem_ops(elem)->set_switch(elem, SM_CAPT, channel, value);
    // ... (truncated, see source file for more)
}

dirtycode/alsa-lib/src/mixer/simple.c (line 887) - [public] sm_selem_ops
    return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_ENUMERATED, 0);
}

/**
 * \brief Return true if mixer simple enumerated element belongs to the playback direction
 * \param elem Mixer simple element handle
 * \return 0 no playback direction, 1 playback direction
 */
int snd_mixer_selem_is_enum_playback(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_ENUMERATED, 1);
    // ... (truncated, see source file for more)
}

dirtycode/alsa-lib/src/mixer/simple.c (line 898) - [public] sm_selem_ops
    return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_ENUMERATED, 1);
}

/**
 * \brief Return true if mixer simple enumerated element belongs to the capture direction
 * \param elem Mixer simple element handle
 * \return 0 no capture direction, 1 capture direction
 */
int snd_mixer_selem_is_enum_capture(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    return sm_selem_ops(elem)->is(elem, SM_CAPT, SM_OPS_IS_ENUMERATED, 1);
    // ... (truncated, see source file for more)
}

```

```

dirtycode/alsa-lib/src/mixer/simple.c (line 909) - [public] sm_selem_ops
    return sm_selem_ops(elem)->is(elem, SM_CAPT, SM_OPS_IS_ENUMERATED, 1);
}

/**
 * \brief Return the number of enumerated items of the given mixer simple element
 * \param elem Mixer simple element handle
 * \return the number of enumerated items, otherwise a negative error code
 */
int snd_mixer_selem_get_enum_items(snd_mixer_elem_t *elem)
{
    CHECK_BASIC(elem);
    CHECK_ENUM(elem);
    // ... (truncated, see source file for more)
}

dirtycode/alsa-lib/src/mixer/simple.c (line 921) - [public] sm_selem_ops
    return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_ENUMCNT, 0);
}

/**
 * \brief get the enumerated item string for the given mixer simple element
 * \param elem Mixer simple element handle
 * \param item the index of the enumerated item to query
 * \param maxlen the maximal length to be stored
 * \param buf the buffer to store the name string
 * \return 0 if successful, otherwise a negative error code
 */
int snd_mixer_selem_get_enum_item_name(snd_mixer_elem_t *elem,
    // ... (truncated, see source file for more)
)

dirtycode/alsa-lib/src/mixer/simple.c (line 938) - [public] sm_selem_ops
    return sm_selem_ops(elem)->enum_item_name(elem, item, maxlen, buf);
}

/**
 * \brief get the current selected enumerated item for the given mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param itemp the pointer to store the index of the enumerated item
 * \return 0 if successful, otherwise a negative error code
 */
int snd_mixer_selem_get_enum_item(snd_mixer_elem_t *elem,
    snd_mixer_selem_channel_id_t channel,
    // ... (truncated, see source file for more)
)

dirtycode/alsa-lib/src/mixer/simple.c (line 954) - [public] sm_selem_ops
    return sm_selem_ops(elem)->get_enum_item(elem, channel, itemp);
}

/**
 * \brief set the current selected enumerated item for the given mixer simple element
 * \param elem Mixer simple element handle
 * \param channel mixer simple element channel identifier
 * \param item the enumerated item index
 * \return 0 if successful, otherwise a negative error code
 */
int snd_mixer_selem_set_enum_item(snd_mixer_elem_t *elem,
    snd_mixer_selem_channel_id_t channel,
    // ... (truncated, see source file for more)
)

```

dirtycode/alsa-lib/src/mixer/simple.c (line 970) - [public] sm_selem_ops

```
    return sm_selem_ops(elem)->set_enum_item(elem, channel, item);
}

/**
 * \brief get size of #snd_mixer_selem_id_t
 * \return size in bytes
 */
size_t snd_mixer_selem_id_sizeof()
{
    return sizeof(snd_mixer_selem_id_t);
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 979) - [public] sizeof

```
    return sizeof(snd_mixer_selem_id_t);
}

/**
 * \brief allocate an invalid #snd_mixer_selem_id_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_mixer_selem_id_malloc(snd_mixer_selem_id_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_mixer_selem_id_t));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 1079) - [public] while

```
    while (*str && *str != ',') {
        if (size < (int)sizeof(buf)) {
            *ptr++ = *str;
            size++;
        }
        str++;
    }
} else {
    c = *str++;
    while (*str && *str != c) {
        if (size < (int)sizeof(buf)) {
            *ptr++ = *str;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 1080) - [public] if

```
    if (size < (int)sizeof(buf)) {
        *ptr++ = *str;
        size++;
    }
    str++;
}
} else {
    c = *str++;
    while (*str && *str != c) {
        if (size < (int)sizeof(buf)) {
            *ptr++ = *str;
            size++;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 1088) - [public] while

```
while (*str && *str != c) {
    if (size < (int)sizeof(buf)) {
        *ptr++ = *str;
        size++;
    }
    str++;
}
if (*str == c)
    str++;
}
if (*str == '\\0') {
    *ptr = 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple.c (line 1089) - [public] if

```
if (size < (int)sizeof(buf)) {
    *ptr++ = *str;
    size++;
}
str++;
}
if (*str == c)
    str++;
}
if (*str == '\\0') {
    *ptr = 0;
    goto _set;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 56) - [public] int

```
typedef int (*snd_mixer_sbasic_init_t)(snd_mixer_class_t *class);
typedef int (*snd_mixer_sfbasic_init_t)(snd_mixer_class_t *class,
    snd_mixer_t *mixer,
    const char *device);

#endif /* !DOC_HIDDEN */

static int try_open(snd_mixer_class_t *class, const char *lib)
{
    class_priv_t *priv = snd_mixer_class_get_private(class);
    snd_mixer_event_t event_func;
    snd_mixer_sbasic_init_t init_func = NULL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 85) - [public] snd_error

```
snd_error(MIXER, "Unable to open library '%s' (%s)", xlib, errbuf);
free(xlib);
return -ENXIO;
}
priv->dlhandle = h;
event_func = snd_dlsym(h, "alsa_mixer_simple_event", NULL);
if (event_func == NULL) {
    snd_error(MIXER, "Symbol 'alsa_mixer_simple_event' was not found in '%s'", xlib);
    err = -ENXIO;
}
if (err == 0) {
    init_func = snd_dlsym(h, "alsa_mixer_simple_init", NULL);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 86) - [public] free

```
    ■■free(xlib);
    ■■return -ENXIO;
    ■}
    ■priv->dlhandle = h;
    ■event_func = snd_dlsym(h, "alsa_mixer_simple_event", NULL);
    ■if (event_func == NULL) {
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_event' was not found in '%s'", xlib);
    ■■err = -ENXIO;
    ■}
    ■if (err == 0) {
    ■■init_func = snd_dlsym(h, "alsa_mixer_simple_init", NULL);
    ■■if (init_func == NULL) {
    ■■■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 92) - [public] snd_error

```
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_event' was not found in '%s'", xlib);
    ■■err = -ENXIO;
    ■}
    ■if (err == 0) {
    ■■init_func = snd_dlsym(h, "alsa_mixer_simple_init", NULL);
    ■■if (init_func == NULL) {
    ■■■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_init' was not found in '%s'", xlib);
    ■■■■err = -ENXIO;
    ■■■}
    ■}
    ■free(xlib);
    ■err = err == 0 ? init_func(class) : err;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 97) - [public] if

```
    ■■if (init_func == NULL) {
    ■■■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_init' was not found in '%s'", xlib);
    ■■■■err = -ENXIO;
    ■■■}
    ■}
    ■free(xlib);
    ■err = err == 0 ? init_func(class) : err;
    ■if (err < 0)
    ■■return err;
    ■snd_mixer_class_set_event(class, event_func);
    ■return 1;
    }
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 98) - [public] snd_error

```
    ■■■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_init' was not found in '%s'", xlib);
    ■■■■err = -ENXIO;
    ■■■}
    ■}
    ■free(xlib);
    ■err = err == 0 ? init_func(class) : err;
    ■if (err < 0)
    ■■return err;
    ■snd_mixer_class_set_event(class, event_func);
    ■return 1;
    }
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 132) - [public] snd_error

```
    ■■snd_error(MIXER, "Unable to open library '%s'", xlib);
    ■■free(xlib);
    ■■return -ENXIO;
    ■}
    ■priv->dlhandle = h;
    ■event_func = snd_dlsym(h, "alsa_mixer_simple_event", NULL);
    ■if (event_func == NULL) {
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_event' was not found in '%s'", xlib);
    ■■err = -ENXIO;
    ■}
    ■if (err == 0) {
    ■■init_func = snd_dlsym(h, "alsa_mixer_simple_finit", NULL);
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 133) - [public] free

```
    ■■free(xlib);
    ■■return -ENXIO;
    ■}
    ■priv->dlhandle = h;
    ■event_func = snd_dlsym(h, "alsa_mixer_simple_event", NULL);
    ■if (event_func == NULL) {
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_event' was not found in '%s'", xlib);
    ■■err = -ENXIO;
    ■}
    ■if (err == 0) {
    ■■init_func = snd_dlsym(h, "alsa_mixer_simple_finit", NULL);
    ■■if (init_func == NULL) {
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 139) - [public] snd_error

```
    ■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_event' was not found in '%s'", xlib);
    ■■err = -ENXIO;
    ■}
    ■if (err == 0) {
    ■■init_func = snd_dlsym(h, "alsa_mixer_simple_finit", NULL);
    ■■if (init_func == NULL) {
    ■■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_finit' was not found in '%s'", xlib);
    ■■■err = -ENXIO;
    ■■}
    ■}
    ■free(xlib);
    ■err = err == 0 ? init_func(class, mixer, device) : err;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 144) - [public] if

```
    ■■if (init_func == NULL) {
    ■■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_finit' was not found in '%s'", xlib);
    ■■■err = -ENXIO;
    ■■}
    ■}
    ■free(xlib);
    ■err = err == 0 ? init_func(class, mixer, device) : err;
    ■if (err < 0)
    ■■return err;
    ■snd_mixer_class_set_event(class, event_func);
    ■return 1;
    }
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 145) - [public] snd_error

```
    ■■■snd_error(MIXER, "Symbol 'alsa_mixer_simple_finit' was not found in '%s'", xlib);
    ■■■err = -ENXIO;
    ■■}
    ■}
    ■free(xlib);
    ■err = err == 0 ? init_func(class, mixer, device) : err;
    ■if (err < 0)
    ■■return err;
    ■snd_mixer_class_set_event(class, event_func);
    ■return 1;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 163) - [public] try_open

```
    ■■return try_open(class, lib);
    ■components = snd_ctl_card_info_get_components(priv->info);
    ■while (*components != '\0') {
    ■■if (!strcmp(components, searchl, strlen(searchl)))
    ■■■return try_open(class, lib);
    ■■while (*components != ' ' && *components != '\0')
    ■■■components++;
    ■■while (*components == ' ' && *components != '\0')
    ■■■components++;
    ■}
    ■return 0;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 167) - [public] try_open

```
    ■■■return try_open(class, lib);
    ■■while (*components != ' ' && *components != '\0')
    ■■■components++;
    ■■while (*components == ' ' && *components != '\0')
    ■■■components++;
    ■}
    ■return 0;
    ■}
```

```
static int find_full(snd_mixer_class_t *class, snd_mixer_t *mixer,
    ■■    snd_config_t *top, const char *device)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 217) - [public] snd_config_for_each

```
    ■■snd_config_for_each(j, jnext, n) {
    ■■■snd_config_t *m = snd_config_iterator_entry(j);
    ■■■if (snd_config_get_id(m, &id) < 0)
    ■■■■continue;
    ■■■if (!strcmp(id, "searchl")) {
    ■■■■err = snd_config_get_string(m, (const char **)&searchl);
    ■■■■if (err < 0)
    ■■■■return err;
    ■■■■continue;
    ■■■}
    ■■■if (!strcmp(id, "lib")) {
    ■■■■err = snd_config_get_string(m, (const char **)&lib);
    ■■■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 221) - [public] if

```
    if (!strcmp(id, "searchl")) {
        err = snd_config_get_string(m, (const char **)&searchl);
        if (err < 0)
            return err;
        continue;
    }
    if (!strcmp(id, "lib")) {
        err = snd_config_get_string(m, (const char **)&lib);
        if (err < 0)
            return err;
        continue;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 227) - [public] if

```
    if (!strcmp(id, "lib")) {
        err = snd_config_get_string(m, (const char **)&lib);
        if (err < 0)
            return err;
        continue;
    }
    err = match(class, lib, searchl);
    if (err == 1)
        return 0;
    if (err < 0)
        return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 250) - [public] snd_dlclose

```
    snd_dlclose(priv->dlhandle);
    if (priv->info)
        snd_ctl_card_info_free(priv->info);
    if (priv->hctl) {
        if (priv->attach_flag)
            snd_mixer_detach_hctl(snd_mixer_class_get_mixer(class), priv->hctl);
        snd_hctl_close(priv->hctl);
    } else if (priv->ctl)
        snd_ctl_close(priv->ctl);
    free(priv->device);
    free(priv);
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 252) - [public] snd_ctl_card_info_free

```
    snd_ctl_card_info_free(priv->info);
    if (priv->hctl) {
        if (priv->attach_flag)
            snd_mixer_detach_hctl(snd_mixer_class_get_mixer(class), priv->hctl);
        snd_hctl_close(priv->hctl);
    } else if (priv->ctl)
        snd_ctl_close(priv->ctl);
    free(priv->device);
    free(priv);
}
/**
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/mixer/simple_abst.c (line 255) - [public] snd_mixer_detach_hctl

```
    ■■snd_mixer_detach_hctl(snd_mixer_class_get_mixer(class), priv->hctl);
    ■■snd_hctl_close(priv->hctl);
    ■} else if (priv->ctl)
    ■■snd_ctl_close(priv->ctl);
    ■free(priv->device);
    ■free(priv);
}

/**
 * \brief Register mixer simple element class - basic abstraction
 * \param mixer Mixer handle
 * \param options Options container
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 256) - [public] snd_hctl_close

```
    ■■snd_hctl_close(priv->hctl);
    ■} else if (priv->ctl)
    ■■snd_ctl_close(priv->ctl);
    ■free(priv->device);
    ■free(priv);
}

/**
 * \brief Register mixer simple element class - basic abstraction
 * \param mixer Mixer handle
 * \param options Options container
 * \param classp Pointer to returned mixer simple element class handle (or NULL
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 258) - [public] snd_ctl_close

```
    ■■snd_ctl_close(priv->ctl);
    ■free(priv->device);
    ■free(priv);
}

/**
 * \brief Register mixer simple element class - basic abstraction
 * \param mixer Mixer handle
 * \param options Options container
 * \param classp Pointer to returned mixer simple element class handle (or NULL
 * \return 0 on success otherwise a negative error code
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 284) - [public] free

```
    ■■free(priv);
    ■■return -EINVAL;
    ■}
    ■if (snd_mixer_class_malloc(&class)) {
    ■■free(priv);
    ■■return -ENOMEM;
    ■}
    ■priv->device = strdup(options->device);
    ■if (priv->device == NULL) {
    ■■free(priv);
    ■■snd_mixer_class_free(class);
    ■■return -ENOMEM;
    ■ // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 288) - [public] free

```
    free(priv);
    return -ENOMEM;
}
priv->device = strdup(options->device);
if (priv->device == NULL) {
    free(priv);
    snd_mixer_class_free(class);
    return -ENOMEM;
}
snd_mixer_class_set_compare(class, snd_mixer_selem_compare);
snd_mixer_class_set_private(class, priv);
snd_mixer_class_set_private_free(class, private_free);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 293) - [public] free

```
    free(priv);
    snd_mixer_class_free(class);
    return -ENOMEM;
}
snd_mixer_class_set_compare(class, snd_mixer_selem_compare);
snd_mixer_class_set_private(class, priv);
snd_mixer_class_set_private_free(class, private_free);
file = getenv("ALSA_MIXER_SIMPLE");
if (!file) {
    const char *topdir = snd_config_topdir();
    char *s = alloca(strlen(topdir) + strlen("smixer.conf") + 2);
    sprintf(s, "%s/smixer.conf", topdir);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 294) - [public] snd_mixer_class_free

```
    snd_mixer_class_free(class);
    return -ENOMEM;
}
snd_mixer_class_set_compare(class, snd_mixer_selem_compare);
snd_mixer_class_set_private(class, priv);
snd_mixer_class_set_private_free(class, private_free);
file = getenv("ALSA_MIXER_SIMPLE");
if (!file) {
    const char *topdir = snd_config_topdir();
    char *s = alloca(strlen(topdir) + strlen("smixer.conf") + 2);
    sprintf(s, "%s/smixer.conf", topdir);
    file = s;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 304) - [public] sprintf

```
    sprintf(s, "%s/smixer.conf", topdir);
    file = s;
}
err = snd_config_top(&top);
if (err < 0)
    goto __error;
err = snd_input_stdio_open(&input, file, "r");
if (err < 0) {
    snd_error(MIXER, "unable to open simple mixer configuration file '%s'", file);
    goto __error;
}
err = snd_config_load(top, input);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 312) - [public] snd_error

```
    ■■snd_error(MIXER, "unable to open simple mixer configuration file '%s'", file);
    ■■goto __error;
    ■}
    ■err = snd_config_load(top, input);
    ■snd_input_close(input);
    ■if (err < 0) {
    ■■snd_error(MIXER, "%s may be old or corrupted: consider to remove or fix it", file);
    ■■goto __error;
    ■}
    ■err = find_full(class, mixer, top, priv->device);
    ■if (err >= 0)
    ■■goto __full;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 318) - [public] snd_error

```
    ■■snd_error(MIXER, "%s may be old or corrupted: consider to remove or fix it", file);
    ■■goto __error;
    ■}
    ■err = find_full(class, mixer, top, priv->device);
    ■if (err >= 0)
    ■■goto __full;
    ■err = snd_ctl_open(&priv->ctl, priv->device, 0);
    ■if (err < 0) {
    ■■snd_error(MIXER, "unable to open control device '%s': %s", priv->device, snd_strerror(err));
    ■■goto __error;
    ■}
    ■err = snd_hctl_open_ctl(&priv->hctl, priv->ctl);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 326) - [public] snd_error

```
    ■■snd_error(MIXER, "unable to open control device '%s': %s", priv->device, snd_strerror(err));
    ■■goto __error;
    ■}
    ■err = snd_hctl_open_ctl(&priv->hctl, priv->ctl);
    ■if (err < 0)
    ■■goto __error;
    ■err = snd_ctl_card_info_malloc(&priv->info);
    ■if (err < 0)
    ■■goto __error;
    ■err = snd_ctl_card_info(priv->ctl, priv->info);
    ■if (err < 0)
    ■■goto __error;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 350) - [public] snd_config_delete

```
    ■■■snd_config_delete(top);
    ■■if (class)
    ■■■snd_mixer_class_free(class);
    ■■return err;
    ■}
    ■snd_config_delete(top);
    ■if (classp)
    ■■*classp = class;
    ■return 0;
    }

    /**
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 352) - [public] snd_mixer_class_free

```
    ■■■snd_mixer_class_free(class);
    ■■return err;
    ■}
    ■snd_config_delete(top);
    ■if (classp)
    ■■*classp = class;
    ■return 0;
    ■}

    /**
     * \brief Basic Mixer Abstraction - Get information about device
     * \param class Mixer class
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 122) - [public] if

```
    ■■if (!strcmp(*name, *names, strlen(*names))) {
    ■■*name += strlen(*names);
    ■■if (**name == ' ')
    ■■■(*name)++;
    ■■return res+1;
    ■■}
    ■}
    ■return MIXER_COMPARE_WEIGHT_NOT_FOUND;
    ■}
```

```
static int get_compare_weight(const char *name, unsigned int idx)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 248) - [public]

snd_ctl_elem_value_get_integer

```
    ■■■■snd_ctl_elem_value_get_integer(&ctl, idx1));
    ■}
    ■return 0;
    ■}
```

```
static int elem_read_switch(selem_none_t *s, int dir, selem_ctl_type_t type)
{
    ■snd_ctl_elem_value_t ctl = {0};
    ■unsigned int idx;
    ■int err;
    ■selem_ctl_t *c = &s->ctls[type];
    ■if ((err = snd_hctl_elem_read(c->elem, &ctl)) < 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 313) - [public]

snd_ctl_elem_value_get_enumerated

```
    ■■■■snd_ctl_elem_value_get_enumerated(&ctl, idx1);
    ■}
    ■return 0;
    ■}
```

```
static int selem_read(snd_mixer_elem_t *elem)
{
    ■selem_none_t *s;
    ■unsigned int idx;
    ■int err = 0;
    ■long pvol[32], cvol[32];
    ■unsigned int psw, csw;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 477) - [public] from_user

```
■■■■from_user(s, dir, c, s->str[dir].vol[idx]);
■if ((err = snd_hctl_elem_write(c->elem, &ctl)) < 0)
■■return err;
■return 0;
}

static int elem_write_switch(selem_none_t *s, int dir, selem_ctl_type_t type)
{
■snd_ctl_elem_value_t ctl = {0};
■unsigned int idx;
■int err;
■selem_ctl_t *c = &s->ctls[type];
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 508) - [public]

snd_ctl_elem_value_set_integer

```
■■snd_ctl_elem_value_set_integer(&ctl, idx, !!val);
■if ((err = snd_hctl_elem_write(c->elem, &ctl)) < 0)
■■return err;
■return 0;
}

static int elem_write_route(selem_none_t *s, int dir, selem_ctl_type_t type)
{
■snd_ctl_elem_value_t ctl = {0};
■unsigned int idx;
■int err;
■selem_ctl_t *c = &s->ctls[type];
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 523) - [public]

snd_ctl_elem_value_set_integer

```
■■snd_ctl_elem_value_set_integer(&ctl, idx, 0);
■for (idx = 0; idx < c->values; idx++)
■■snd_ctl_elem_value_set_integer(&ctl, idx * c->values + idx,
■■■■■■!!(s->str[dir].sw & (1 <= idx)));
■if ((err = snd_hctl_elem_write(c->elem, &ctl)) < 0)
■■return err;
■return 0;
}

static int elem_write_enum(selem_none_t *s)
{
■snd_ctl_elem_value_t ctl = {0};
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 568) - [public] elem_write_enum

```
■■return elem_write_enum(s);

■if (s->ctls[CTL_PLAYBACK_ENUM].elem)
■■return elem_write_enum(s);

■if (s->ctls[CTL_CAPTURE_ENUM].elem)
■■return elem_write_enum(s);

■if (s->ctls[CTL_SINGLE].elem) {
■■if (s->ctls[CTL_SINGLE].type == SND_CTL_ELEM_TYPE_INTEGER)
■■■err = elem_write_volume(s, SM_PLAY, CTL_SINGLE);
■■else
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 571) - [public] elem_write_enum

```
    return elem_write_enum(s);

    if (s->ctls[CTL_CAPTURE_ENUM].elem)
        return elem_write_enum(s);

    if (s->ctls[CTL_SINGLE].elem) {
        if (s->ctls[CTL_SINGLE].type == SND_CTL_ELEM_TYPE_INTEGER)
            err = elem_write_volume(s, SM_PLAY, CTL_SINGLE);
        else
            err = elem_write_switch(s, SM_PLAY, CTL_SINGLE);
        if (err < 0)
            return err;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 574) - [public] elem_write_enum

```
    return elem_write_enum(s);

    if (s->ctls[CTL_SINGLE].elem) {
        if (s->ctls[CTL_SINGLE].type == SND_CTL_ELEM_TYPE_INTEGER)
            err = elem_write_volume(s, SM_PLAY, CTL_SINGLE);
        else
            err = elem_write_switch(s, SM_PLAY, CTL_SINGLE);
        if (err < 0)
            return err;
    }
    if (s->ctls[CTL_GLOBAL_VOLUME].elem) {
        err = elem_write_volume(s, SM_PLAY, CTL_GLOBAL_VOLUME);
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 664) - [public] snd_mixer_selem_id_free

```
    snd_mixer_selem_id_free(simple->selem.id);
    /* free db range information */
    free(simple->str[0].db_info);
    free(simple->str[1].db_info);
    free(simple);
}

static int simple_update(snd_mixer_elem_t *melem)
{
    selem_none_t *simple;
    unsigned int caps, pchannels, cchannels;
    long pmin, pmax, cmin, cmax;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 690) - [public] if

```
    if (ctl->type == SND_CTL_ELEM_TYPE_INTEGER) {
        caps |= SM_CAP_GVOLUME;
        pmin = cmin = ctl->min;
        pmax = cmax = ctl->max;
    } else
        caps |= SM_CAP_GSWITCH;
    ctl = &simple->ctls[CTL_GLOBAL_SWITCH];
    if (ctl->elem) {
        if (pchannels < ctl->values)
            pchannels = ctl->values;
        if (cchannels < ctl->values)
            cchannels = ctl->values;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 918) - [public] strlen

```
*type = CTL_CAPTURE_VOLUME;

    return strlen("Capture");
}

if (!strcmp(name, "Capture Switch")) {
    *type = CTL_CAPTURE_SWITCH;
    return strlen("Capture");
}

for (p = suffixes; p->suffix; p++) {
    size_t slen = strlen(p->suffix);
    size_t l;
    if (nlen > slen) {
        l = nlen - slen;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 922) - [public] strlen

```
*type = CTL_CAPTURE_SWITCH;

    return strlen("Capture");
}

for (p = suffixes; p->suffix; p++) {
    size_t slen = strlen(p->suffix);
    size_t l;
    if (nlen > slen) {
        l = nlen - slen;
        if (strncmp(name + l, p->suffix, slen) == 0 &&
            (l < 1 || name[l-1] != '-')) { /* 3D Control - Switch */
            *type = p->type;
            return l;
        }
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 928) - [public] if

```
if (nlen > slen) {
    l = nlen - slen;
    if (strncmp(name + l, p->suffix, slen) == 0 &&
        (l < 1 || name[l-1] != '-')) { /* 3D Control - Switch */
        *type = p->type;
        return l;
    }
}

/* Special case - handle "Input Source" as a capture route.
 * Note that it's *NO* capture source. A capture source is split over
 * ... (truncated, see source file for more)
*/
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 947) - [public] strlen

```
*type = CTL_CAPTURE_ROUTE;

    return strlen(name);
}

if (strstr(name, "3D Control")) {
    if (strstr(name, "Depth")) {
        *type = CTL_PLAYBACK_VOLUME;
        return strlen(name);
    }
}

*type = CTL_SINGLE;
return strlen(name);
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 950) - [public] if

```
    if (strstr(name, "Depth")) {
        *type = CTL_PLAYBACK_VOLUME;
        return strlen(name);
    }

    *type = CTL_SINGLE;
    return strlen(name);
}
```

```
/*
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 952) - [public] strlen

```
*type = CTL_PLAYBACK_VOLUME;

    return strlen(name);
}

    *type = CTL_SINGLE;
    return strlen(name);
}
```

```
/*
    * Simple Mixer Operations
    */
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 957) - [public] strlen

```
*type = CTL_SINGLE;

    return strlen(name);
}
```

```
/*
    * Simple Mixer Operations
    */
```

```
static int _snd_mixer_selem_set_volume(snd_mixer_elem_t *elem, int dir, snd_mixer_selem_channel_id_t chan)
{
    selem_none_t *s = snd_mixer_elem_get_private(elem);
    if (s->selem.caps & SM_CAP_GVOLUME)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 995) - [public] if

```
    if (!(s->str[dir].sw & (1 << channel))) {
        s->str[dir].sw |= 1 << channel;
        return 1;
    } else {
        if (s->str[dir].sw & (1 << channel)) {
            s->str[dir].sw &= ~(1 << channel);
            return 1;
        }
        return 0;
    }

    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/mixer/simple_none.c (line 1000) - [public] if

```
    if (s->str[dir].sw & (1 << channel)) {
        s->str[dir].sw &= ~(1 << channel);
        return 1;
    }
    return 0;
}

static int is_ops(snd_mixer_elem_t *elem, int dir, int cmd, int val)
{
    selem_none_t *s = snd_mixer_elem_get_private(elem);

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1029) - [public] if

```
    if (val == 1) {
        if (dir == SM_PLAY && (s->selem.caps & SM_CAP_PENUM) && !(s->selem.caps & SM_CAP_CENUM) )
            return 1;
        if (dir == SM_CAPT && (s->selem.caps & SM_CAP_CENUM) && !(s->selem.caps & SM_CAP_PENUM) )
            return 1;
        return 0;
    }
    if (s->selem.caps & (SM_CAP_CENUM | SM_CAP_PENUM) )
        return 1;
    return 0;

    case SM_OPS_IS_ENUMCNT:
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1042) - [public] if

case SM_OPS_IS_ENUMCNT:

/* Both */

```
    if ( (s->selem.caps & (SM_CAP_CENUM | SM_CAP_PENUM)) == (SM_CAP_CENUM | SM_CAP_PENUM) ) {
        if (! s->ctls[CTL_GLOBAL_ENUM].elem)
            return -EINVAL;
        return s->ctls[CTL_GLOBAL_ENUM].max;
        /* Only Playback */
    } else if (s->selem.caps & SM_CAP_PENUM) {
        if (! s->ctls[CTL_PLAYBACK_ENUM].elem)
            return -EINVAL;
        return s->ctls[CTL_PLAYBACK_ENUM].max;
        /* Only Capture */
    } else if (s->selem.caps & SM_CAP_CENUM) {
        if (! s->ctls[CTL_CAPTURE_ENUM].elem)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1098) - [public] init_db_range

static int init_db_range(snd_hctl_elem_t *ctl, struct selem_str *rec);

```
static int convert_to_db(snd_hctl_elem_t *ctl, struct selem_str *rec,
    long volume, long *db_gain)
{
    if (init_db_range(ctl, rec) < 0)
        return -EINVAL;
    return snd_tlv_convert_to_db(rec->db_info, rec->min, rec->max,
        volume, db_gain);
}
```

/* initialize dB range information, reading TLV via hcontrol
// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1177) - [public] snd_tlv_get_dB_range

```
    return snd_tlv_get_dB_range(rec->db_info, rec->min, rec->max, min, max);
}

static int get_dB_range_ops(snd_mixer_elem_t *elem, int dir,
    long *min, long *max)
{
    selem_none_t *s = snd_mixer_elem_get_private(elem);
    selem_ctl_t *c;

    if (s->selem.caps & SM_CAP_GVOLUME)
        dir = SM_PLAY;
    c = get_selem_ctl(s, dir);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1191) - [public] get_dB_range

```
    return get_dB_range(c->elem, &s->str[dir], min, max);
}

static int convert_from_dB(snd_hctl_elem_t *ctl, struct selem_str *rec,
    long db_gain, long *value, int xdir)
{
    if (init_db_range(ctl, rec) < 0)
        return -EINVAL;

    return snd_tlv_convert_from_dB(rec->db_info, rec->min, rec->max,
        db_gain, value, xdir);
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1266) - [public] selem_write

```
    return selem_write(elem);
    return 0;
}

static int ask_dB_vol_ops(snd_mixer_elem_t *elem, int dir,
    long dbValue, long *value, int xdir)
{
    selem_none_t *s = snd_mixer_elem_get_private(elem);
    selem_ctl_t *c;

    if (s->selem.caps & SM_CAP_GVOLUME)
        dir = SM_PLAY;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1281) - [public] convert_from_dB

```
    return convert_from_dB(c->elem, &s->str[dir], dbValue, value, xdir);
}

static int set_dB_ops(snd_mixer_elem_t *elem, int dir,
    snd_mixer_selem_channel_id_t channel,
    long db_gain, int xdir)
{
    selem_none_t *s = snd_mixer_elem_get_private(elem);
    selem_ctl_t *c;
    long value;
    int err;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1301) - [public] set_volume_ops

```
    return set_volume_ops(elem, dir, channel, value);
}

static int set_switch_ops(snd_mixer_elem_t *elem, int dir,
    snd_mixer_selem_channel_id_t channel, int value)
{
    int changed;
    snd_mixer_selem_t *s = snd_mixer_elem_get_private(elem);
    if (s->selem.caps & SM_CAP_GSWITCH)
        dir = SM_PLAY;
    if (dir == SM_PLAY) {
        if (! (s->selem.caps & (SM_CAP_GSWITCH|SM_CAP_PSWITCH)))
            // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1322) - [public] selem_write

```
    return selem_write(elem);
    return 0;
}

static int enum_item_name_ops(snd_mixer_elem_t *elem,
    unsigned int item,
    size_t maxlen, char *buf)
{
    snd_mixer_selem_t *s = snd_mixer_elem_get_private(elem);
    snd_ctl_elem_info_t info = {0};
    snd_hctl_elem_t *helem;
    int type;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1408) - [public] snd_hctl_elem_write

```
    return snd_hctl_elem_write(helem, &ctl);
}

static struct sm_elem_ops simple_none_ops = {
    .is = is_ops,
    .get_range = get_range_ops,
    .get_dB_range = get_dB_range_ops,
    .set_range = set_range_ops,
    .ask_vol_dB = ask_vol_dB_ops,
    .ask_dB_vol = ask_dB_vol_ops,
    .get_volume = get_volume_ops,
    .get_dB = get_dB_ops,
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1461) - [public] if

```
    if (ctype == SND_CTL_ELEM_TYPE_ENUMERATED) {
        if (type == CTL_PLAYBACK_ROUTE)
            type = CTL_PLAYBACK_ENUM;
        else if (type == CTL_CAPTURE_ROUTE)
            type = CTL_CAPTURE_ENUM;
        else
            type = CTL_GLOBAL_ENUM;
        break;
    }
    if (ctype != SND_CTL_ELEM_TYPE_BOOLEAN)
        return 0;
#ifdef HAVE_SOFT_FLOAT
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1488) - [public] if

```
    if (ctype == SND_CTL_ELEM_TYPE_ENUMERATED) {
        if (type == CTL_PLAYBACK_SWITCH)
            type = CTL_PLAYBACK_ENUM;
        else if (type == CTL_CAPTURE_SWITCH)
            type = CTL_CAPTURE_ENUM;
        else
            type = CTL_GLOBAL_ENUM;
        break;
    }
    if (ctype != SND_CTL_ELEM_TYPE_BOOLEAN)
        return 0;
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1503) - [public] if

```
    if (ctype == SND_CTL_ELEM_TYPE_ENUMERATED) {
        if (type == CTL_PLAYBACK_VOLUME)
            type = CTL_PLAYBACK_ENUM;
        else if (type == CTL_CAPTURE_VOLUME)
            type = CTL_CAPTURE_ENUM;
        else
            type = CTL_GLOBAL_ENUM;
        break;
    }
    if (ctype != SND_CTL_ELEM_TYPE_INTEGER)
        return 0;
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1526) - [public] assert

```
    assert(0);
    break;
}
name1 = get_short_name(name);
if (snd_mixer_selem_id_malloc(&id))
    return -ENOMEM;
snd_mixer_selem_id_set_name(id, name1);
snd_mixer_selem_id_set_index(id, snd_hctl_elem_get_index(helem));
melem = snd_mixer_find_selem(snd_mixer_class_get_mixer(class), id);
if (!melem) {
    simple = calloc(1, sizeof(*simple));
    if (!simple) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1537) - [public] if

```
    if (!simple) {
        snd_mixer_selem_id_free(id);
        return -ENOMEM;
    }
    simple->selem.id = id;
    simple->selem.ops = &simple_none_ops;
    err = snd_mixer_elem_new(&melem, SND_MIXER_ELEM_SIMPLE,
        get_compare_weight(
            snd_mixer_selem_id_get_name(simple->selem.id),
            snd_mixer_selem_id_get_index(simple->selem.id)),
        simple, selem_free);
    if (err < 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1538) - [public] snd_mixer_selem_id_free

```
    snd_mixer_selem_id_free(id);
    return -ENOMEM;
}
simple->selem.id = id;
simple->selem.ops = &simple_none_ops;
err = snd_mixer_elem_new(&melem, SND_MIXER_ELEM_SIMPLE,
    get_compare_weight(
        snd_mixer_selem_id_get_name(simple->selem.id),
        snd_mixer_selem_id_get_index(simple->selem.id)),
    simple, selem_free);
if (err < 0) {
    snd_mixer_selem_id_free(id);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1548) - [public] if

```
if (err < 0) {
    snd_mixer_selem_id_free(id);
    free(simple);
    return err;
}
new = 1;
} else {
    simple = snd_mixer_elem_get_private(melem);
    snd_mixer_selem_id_free(id);
}
if (simple->ctls[type].elem) {
    snd_error(MIXER, "helem (%s, '%s', %u, %u, %u) appears twice or more",
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1549) - [public] snd_mixer_selem_id_free

```
    snd_mixer_selem_id_free(id);
    free(simple);
    return err;
}
new = 1;
} else {
    simple = snd_mixer_elem_get_private(melem);
    snd_mixer_selem_id_free(id);
}
if (simple->ctls[type].elem) {
    snd_error(MIXER, "helem (%s, '%s', %u, %u, %u) appears twice or more",
        snd_ctl_elem_iface_name(
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1550) - [public] free

```
    free(simple);
    return err;
}
new = 1;
} else {
    simple = snd_mixer_elem_get_private(melem);
    snd_mixer_selem_id_free(id);
}
if (simple->ctls[type].elem) {
    snd_error(MIXER, "helem (%s, '%s', %u, %u, %u) appears twice or more",
        snd_ctl_elem_iface_name(
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1556) - [public] snd_mixer_selem_id_free

```
    ■■snd_mixer_selem_id_free(id);
    ■}
    ■if (simple->ctls[type].elem) {
    ■■snd_error(MIXER, "helem (%s, '%s', %u, %u, %u) appears twice or more",
    ■■■■ snd_ctl_elem_iface_name(

    ■■■■ snd_hctl_elem_get_interface(helem)),

    ■■    snd_hctl_elem_get_name(helem),
    ■■    snd_hctl_elem_get_index(helem),
    ■■    snd_hctl_elem_get_device(helem),
    ■■    snd_hctl_elem_get_subdevice(helem));
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1567) - [public] snd_hctl_elem_get_subdevice

```
    ■■    snd_hctl_elem_get_subdevice(helem));
    ■■err = -EINVAL;
    ■■goto __error;
    ■}
    ■simple->ctls[type].elem = helem;
    ■simple->ctls[type].type = snd_ctl_elem_info_get_type(&info);
    ■simple->ctls[type].inactive = snd_ctl_elem_info_is_inactive(&info);
    ■simple->ctls[type].values = values;
    ■if ( (type == CTL_GLOBAL_ENUM) ||
    ■    (type == CTL_PLAYBACK_ENUM) ||
    ■    (type == CTL_CAPTURE_ENUM) ) {
    ■■simple->ctls[type].min = 0;
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1581) - [public] if

```
    ■■if (ctype == SND_CTL_ELEM_TYPE_INTEGER) {
    ■■■simple->ctls[type].min =
    ■■■■snd_ctl_elem_info_get_min(&info);
    ■■■simple->ctls[type].max =
    ■■■■snd_ctl_elem_info_get_max(&info);
    ■■}
    ■}
    ■switch (type) {
    ■case CTL_CAPTURE_SOURCE:
    ■■simple->capture_item = value;
    ■■break;
    ■default:
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1583) - [public] snd_ctl_elem_info_get_min

```
    ■■■■snd_ctl_elem_info_get_min(&info);
    ■■simple->ctls[type].max =
    ■■■■snd_ctl_elem_info_get_max(&info);
    ■■}
    ■}
    ■switch (type) {
    ■case CTL_CAPTURE_SOURCE:
    ■■simple->capture_item = value;
    ■■break;
    ■default:
    ■■break;
    ■}
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1585) - [public] snd_ctl_elem_info_get_max

```
■■■■■snd_ctl_elem_info_get_max(&info);
■■■}
■■■}
■■■switch (type) {
■■■case CTL_CAPTURE_SOURCE:
■■■simple->capture_item = value;
■■■break;
■■■default:
■■■break;
■■■}
■■■err = snd_mixer_elem_attach(melem, helem);
■■■if (err < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1618) - [public] snd_mixer_elem_free

```
■■■snd_mixer_elem_free(melem);
■■■return -EINVAL;
■■■}

static int simple_event_add(snd_mixer_class_t *class, snd_hctl_elem_t *helem)
{
■■■const char *name = snd_hctl_elem_get_name(helem);
■■■selem_ctl_type_t type;
■■■char ename[128];
■■■size_t len;

■■■if (snd_hctl_elem_get_interface(helem) != SND_CTL_ELEM_IFACE_MIXER)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1636) - [public] assert

```
■■■assert(err >= 0);
■■■if (snd_ctl_elem_info_get_type(&info) !=
■■■■■■SND_CTL_ELEM_TYPE_ENUMERATED)
■■■■■return 0;
■■■items = snd_ctl_elem_info_get_items(&info);
■■■for (k = 0; k < items; ++k) {
■■■const char *n;
■■■snd_ctl_elem_info_set_item(&info, k);
■■■err = snd_hctl_elem_info(helem, &info);
■■■if (err < 0)
■■■■■return err;
■■■n = snd_ctl_elem_info_get_item_name(&info);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1643) - [public] snd_ctl_elem_info_set_item

```
■■■snd_ctl_elem_info_set_item(&info, k);
■■■err = snd_hctl_elem_info(helem, &info);
■■■if (err < 0)
■■■■■return err;
■■■n = snd_ctl_elem_info_get_item_name(&info);
■■■err = simple_add1(class, n, helem, CTL_CAPTURE_SOURCE,
■■■■■k);
■■■if (err < 0)
■■■■■return err;
■■■}
■■■return 0;
■■■}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1662) - [public] simple_add1

```
■return simple_add1(class, ename, helem, type, 0);
}

static int simple_event_remove(snd_hctl_elem_t *helem,
■■■      snd_mixer_elem_t *melem)
{
■selem_none_t *simple = snd_mixer_elem_get_private(melem);
■int err;
■int k;
■for (k = 0; k <= CTL_LAST; k++) {
■■if (simple->ctls[k].elem == helem)
■■■break;
      // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1681) - [public] snd_mixer_elem_remove

```
■■return snd_mixer_elem_remove(melem);
■err = simple_update(melem);
■return snd_mixer_elem_info(melem);
}

static int simple_event(snd_mixer_class_t *class, unsigned int mask,
■■■snd_hctl_elem_t *helem, snd_mixer_elem_t *melem)
{
■int err;
■if (mask == SND_CTL_EVENT_MASK_REMOVE)
■■return simple_event_remove(helem, melem);
■if (mask & SND_CTL_EVENT_MASK_ADD) {
      // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1683) - [public] snd_mixer_elem_info

```
■return snd_mixer_elem_info(melem);
}

static int simple_event(snd_mixer_class_t *class, unsigned int mask,
■■■snd_hctl_elem_t *helem, snd_mixer_elem_t *melem)
{
■int err;
■if (mask == SND_CTL_EVENT_MASK_REMOVE)
■■return simple_event_remove(helem, melem);
■if (mask & SND_CTL_EVENT_MASK_ADD) {
■■err = simple_event_add(class, helem);
■■if (err < 0)
      // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1691) - [public] simple_event_remove

```
■■return simple_event_remove(helem, melem);
■if (mask & SND_CTL_EVENT_MASK_ADD) {
■■err = simple_event_add(class, helem);
■■if (err < 0)
■■■return err;
■}
■if (mask & SND_CTL_EVENT_MASK_INFO) {
■■err = simple_event_remove(helem, melem);
■■if (err < 0)
■■■return err;
■■err = simple_event_add(class, helem);
■■if (err < 0)
      // ... (truncated, see source file for more)
}
```


dirtycode/alsa-lib/src/mixer/simple_none.c (line 1710) - [public] if

```
    if (err) {
        err = snd_mixer_elem_value(melem);
        if (err < 0)
            return err;
    }
    return 0;
}

/**
 * \brief Register mixer simple element class - none abstraction
 * \param mixer Mixer handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1739) - [public] free

```
    free(class);
    return err;
}
if (classp)
    *classp = class;
return 0;
}
```

dirtycode/alsa-lib/src/output.c (line 139) - [public] fclose

```
    fclose(stdio->fp);
    free(stdio);
    return 0;
}

static int snd_output_stdio_print(snd_output_t *output, const char *format, va_list args)
{
    snd_output_stdio_t *stdio = output->private_data;
    return vfprintf(stdio->fp, format, args);
}

static int snd_output_stdio_puts(snd_output_t *output, const char *str)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/output.c (line 147) - [public] vfprintf

```
    return vfprintf(stdio->fp, format, args);
}

static int snd_output_stdio_puts(snd_output_t *output, const char *str)
{
    snd_output_stdio_t *stdio = output->private_data;
    return fputs(str, stdio->fp);
}

static int snd_output_stdio_putc(snd_output_t *output, int c)
{
    snd_output_stdio_t *stdio = output->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/output.c (line 153) - [public] fputs

```
    return fputs(str, stdio->fp);
}

static int snd_output_stdio_putc(snd_output_t *output, int c)
{
    snd_output_stdio_t *stdio = output->private_data;
    return putc(c, stdio->fp);
}

static int snd_output_stdio_flush(snd_output_t *output)
{
    snd_output_stdio_t *stdio = output->private_data;
    // ... (truncated, see source file for more)
```

```

dirtycode/alsa-lib/src/output.c (line 159) - [public] putc
    ■return putc(c, stdio->fp);
    }

    static int snd_output_stdio_flush(snd_output_t *output)
    {
        ■snd_output_stdio_t *stdio = output->private_data;
        ■return fflush(stdio->fp);
    }

    static const snd_output_ops_t snd_output_stdio_ops = {
        ■.close■■= snd_output_stdio_close,
        ■.print■■= snd_output_stdio_print,
        // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/output.c (line 165) - [public] fflush
    ■return fflush(stdio->fp);
    }

    static const snd_output_ops_t snd_output_stdio_ops = {
        ■.close■■= snd_output_stdio_close,
        ■.print■■= snd_output_stdio_print,
        ■.puts■■= snd_output_stdio_puts,
        ■.putc■■= snd_output_stdio_putc,
        ■.flush■■= snd_output_stdio_flush,
    };

    #endif
    // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/output.c (line 198) - [public] free
    ■■free(stdio);
    ■■return -ENOMEM;
    ■}
    ■stdio->fp = fp;
    ■stdio->close = _close;
    ■output->type = SND_OUTPUT_STDIO;
    ■output->ops = &snd_output_stdio_ops;
    ■output->private_data = stdio;
    ■*outputp = output;
    ■return 0;
    }

    // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/output.c (line 228) - [public] fclose
    ■■fclose(fp);
    ■return err;
    }

    #ifndef DOC_HIDDEN

    typedef struct _snd_output_buffer {
        ■unsigned char *buf;
        ■size_t alloc;
        ■size_t size;
    } snd_output_buffer_t;

    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/output.c (line 393) - [public] free

```
    free(buffer);
    return -ENOMEM;
}
buffer->buf = NULL;
buffer->alloc = 0;
buffer->size = 0;
output->type = SND_OUTPUT_BUFFER;
output->ops = &snd_output_buffer_ops;
output->private_data = buffer;
*outputp = output;
return 0;
}
```

dirtycode/alsa-lib/src/pcm/interval.c (line 93) - [public] assert

```
assert(n > 0);
*r = 0;
return UINT_MAX;
}
div64_32(&n, c, r);
if (n >= UINT_MAX) {
    *r = 0;
    return UINT_MAX;
}
return n;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/interval.c (line 119) - [public] if

```
if (i->openmin) {
    i->min++;
    i->openmin = 0;
}
}
if (snd_interval_checkempty(i)) {
    snd_interval_none(i);
    return -EINVAL;
}
return changed;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/interval.c (line 125) - [public] snd_interval_none

```
snd_interval_none(i);
return -EINVAL;
}
return changed;
}
```

```
int snd_interval_refine_max(snd_interval_t *i, unsigned int max, int openmax)
{
    int changed = 0;
    if (snd_interval_empty(i))
        return -ENOENT;
    if (i->max > max) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/interval.c (line 145) - [public] if

```
    if (i->openmax) {
        i->max--;
        i->openmax = 0;
    }
    if (snd_interval_checkempty(i)) {
        snd_interval_none(i);
        return -EINVAL;
    }
    return changed;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/interval.c (line 151) - [public] snd_interval_none

```
snd_interval_none(i);
return -EINVAL;
}
return changed;
}
```

/* r <- v */

```
int snd_interval_refine(snd_interval_t *i, const snd_interval_t *v)
{
    int changed = 0;
    if (snd_interval_empty(i))
        return -ENOENT;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/interval.c (line 184) - [public] if

```
    if (i->openmin) {
        i->min++;
        i->openmin = 0;
    }
    if (i->openmax) {
        i->max--;
        i->openmax = 0;
    }
    if (!i->openmin && !i->openmax && i->min == i->max)
        i->integer = 1;
    if (snd_interval_checkempty(i)) {
        snd_interval_none(i);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/interval.c (line 188) - [public] if

```
    if (i->openmax) {
        i->max--;
        i->openmax = 0;
    }
    if (!i->openmin && !i->openmax && i->min == i->max)
        i->integer = 1;
    if (snd_interval_checkempty(i)) {
        snd_interval_none(i);
        return -EINVAL;
    }
    return changed;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 195) - [public] snd_interval_none

```
■snd_interval_none(i);
■return -EINVAL;
■}
■return changed;
}

int snd_interval_refine_first(snd_interval_t *i)
{
■const unsigned int last_max = i->max;

■if (snd_interval_empty(i))
■return -ENOENT;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 240) - [public] snd_interval_refine

```
■return snd_interval_refine(i, &t);
}

void snd_interval_add(const snd_interval_t *a, const snd_interval_t *b, snd_interval_t *c)
{
■if (a->empty || b->empty) {
■snd_interval_none(c);
■return;
■}
■c->empty = 0;
■c->min = add(a->min, b->min);
■c->openmin = (a->openmin || b->openmin);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 246) - [public] snd_interval_none

```
■snd_interval_none(c);
■return;
■}
■c->empty = 0;
■c->min = add(a->min, b->min);
■c->openmin = (a->openmin || b->openmin);
■c->max = add(a->max, b->max);
■c->openmax = (a->openmax || b->openmax);
■c->integer = (a->integer && b->integer);
}

void snd_interval_sub(const snd_interval_t *a, const snd_interval_t *b, snd_interval_t *c)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 260) - [public] snd_interval_none

```
■snd_interval_none(c);
■return;
■}
■c->empty = 0;
■c->min = sub(a->min, b->max);
■c->openmin = (a->openmin || b->openmax);
■c->max = add(a->max, b->min);
■c->openmax = (a->openmax || b->openmin);
■c->integer = (a->integer && b->integer);
}

void snd_interval_mul(const snd_interval_t *a, const snd_interval_t *b, snd_interval_t *c)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 274) - [public] snd_interval_none

```
■ ■snd_interval_none(c);
■ ■return;
■ }
■ c->empty = 0;
■ c->min = mul(a->min, b->min);
■ c->openmin = (a->openmin || b->openmin);
■ c->max = mul(a->max, b->max);
■ c->openmax = (a->openmax || b->openmax);
■ c->integer = (a->integer && b->integer);
■ }
```

```
void snd_interval_div(const snd_interval_t *a, const snd_interval_t *b, snd_interval_t *c)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 289) - [public] snd_interval_none

```
■ ■snd_interval_none(c);
■ ■return;
■ }
■ c->empty = 0;
■ c->min = div32(a->min, b->max, &r);
■ c->openmin = (r || a->openmin || b->openmax);
■ if (b->min > 0) {
■ ■c->max = div32(a->max, b->min, &r);
■ ■if (r) {
■ ■■c->max++;
■ ■■c->openmax = 1;
■ ■} else
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 297) - [public] if

```
■ ■if (r) {
■ ■■c->max++;
■ ■■c->openmax = 1;
■ ■} else
■ ■■c->openmax = (a->openmax || b->openmin);
■ } else {
■ ■c->max = UINT_MAX;
■ ■c->openmax = 0;
■ }
■ c->integer = 0;
■ }
```

```
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 315) - [public] snd_interval_none

```
■ ■snd_interval_none(d);
■ ■return;
■ }
■ d->empty = 0;
■ d->min = muldiv32(a->min, b->min, c->max, &r);
■ d->openmin = (r || a->openmin || b->openmin || c->openmax);
■ d->max = muldiv32(a->max, b->max, c->min, &r);
■ if (r) {
■ ■d->max++;
■ ■d->openmax = 1;
■ } else
■ ■d->openmax = (a->openmax || b->openmax || c->openmin);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 336) - [public] snd_interval_none

```
■■snd_interval_none(c);
■■return;
■}
■c->empty = 0;
■c->min = muldiv32(a->min, b->min, k, &r);
■c->openmin = (r || a->openmin || b->openmin);
■c->max = muldiv32(a->max, b->max, k, &r);
■if (r) {
■■c->max++;
■■c->openmax = 1;
■} else
■■c->openmax = (a->openmax || b->openmax);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 357) - [public] snd_interval_none

```
■■snd_interval_none(c);
■■return;
■}
■c->empty = 0;
■c->min = muldiv32(a->min, k, b->max, &r);
■c->openmin = (r || a->openmin || b->openmax);
■if (b->min > 0) {
■■c->max = muldiv32(a->max, k, b->min, &r);
■■if (r) {
■■■■c->max++;
■■■■c->openmax = 1;
■■} else
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 365) - [public] if

```
■■if (r) {
■■■■c->max++;
■■■■c->openmax = 1;
■■} else
■■■■c->openmax = (a->openmax || b->openmin);
■} else {
■■c->max = UINT_MAX;
■■c->openmax = 0;
■}
■c->integer = 0;
}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 380) - [public] snd_output_printf

```
■■snd_output_printf(out, "NONE");
■else if (i->min == 0 && i->openmin == 0 &&
■■ i->max == UINT_MAX && i->openmax == 0)
■■snd_output_printf(out, "ALL");
■else if (snd_interval_single(i) && i->integer)
■■snd_output_printf(out, "%u", snd_interval_value(i));
■else
■■snd_output_printf(out, "%c%u %u%c",
■■■■i->openmin ? '(' : '[' ,
■■■■i->min, i->max,
■■■■i->openmax ? ')' : ']');
}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 383) - [public] snd_output_printf

```
    ■■snd_output_printf(out, "ALL");
    ■else if (snd_interval_single(i) && i->integer)
    ■■snd_output_printf(out, "%u", snd_interval_value(i));
    ■else
    ■■snd_output_printf(out, "%c%u %u%c",
    ■■■■i->openmin ? '(' : '[' ,
    ■■■■i->min, i->max,
    ■■■■i->openmax ? ')' : ']');
    }

    #if 0
    static void boundary_abs(int a, int adir, int *b, int *bdir)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 385) - [public] snd_output_printf

```
    ■■snd_output_printf(out, "%u", snd_interval_value(i));
    ■else
    ■■snd_output_printf(out, "%c%u %u%c",
    ■■■■i->openmin ? '(' : '[' ,
    ■■■■i->min, i->max,
    ■■■■i->openmax ? ')' : ']');
    }

    #if 0
    static void boundary_abs(int a, int adir, int *b, int *bdir)
    {
    ■if (a < 0 || (a == 0 && adir < 0)) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 413) - [public] assert

```
    ■■assert(*c > INT_MIN);
    ■■(*c)--;
    ■} else if (*cdir == 2) {
    ■■assert(*c < INT_MAX);
    ■■(*c)++;
    ■}
    }

    int boundary_lt(unsigned int a, int adir, unsigned int b, int bdir)
    {
    ■assert(a > 0 || adir >= 0);
    ■assert(b > 0 || bdir >= 0);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 416) - [public] assert

```
    ■■assert(*c < INT_MAX);
    ■■(*c)++;
    ■}
    }

    int boundary_lt(unsigned int a, int adir, unsigned int b, int bdir)
    {
    ■assert(a > 0 || adir >= 0);
    ■assert(b > 0 || bdir >= 0);
    ■if (adir < 0) {
    ■■a--;
    ■■adir = 1;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/interval.c (line 445) - [public] boundary_lt

```
    ■return boundary_lt(dmin, dmdir, dmax, dmaxdir);
    }
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 782) - [public] snd_async_del_handler

```
    snd_async_del_handler(h);
}
if (pcm->setup && !pcm->donot_close) {
    snd_pcm_drop(pcm);
    err = snd_pcm_hw_free(pcm);
    if (err < 0)
        res = err;
}
if (pcm->mmap_channels)
    snd_pcm_munmap(pcm);
if (pcm->ops->close)
    err = pcm->ops->close(pcm->op_arg);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 785) - [public] snd_pcm_drop

```
    snd_pcm_drop(pcm);
    err = snd_pcm_hw_free(pcm);
    if (err < 0)
        res = err;
}
if (pcm->mmap_channels)
    snd_pcm_munmap(pcm);
if (pcm->ops->close)
    err = pcm->ops->close(pcm->op_arg);
else
    err = -ENOSYS;
if (err < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 791) - [public] snd_pcm_munmap

```
    snd_pcm_munmap(pcm);
    if (pcm->ops->close)
        err = pcm->ops->close(pcm->op_arg);
    else
        err = -ENOSYS;
    if (err < 0)
        res = err;
    err = snd_pcm_free(pcm);
    if (err < 0)
        res = err;
    return res;
}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1006) - [public] snd_check

```
    snd_check(PCM, "PCM not set up");
    return -EIO;
}
if (! params->avail_min) {
    snd_check(PCM, "params->avail_min is 0");
    return -EINVAL;
}
#if 0
/* disable the check below - it looks too restrictive
 * (start_threshold is basically independent from avail_min)
 */
if (params->start_threshold <= pcm->buffer_size &&
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1010) - [public] snd_check

```
    ■■snd_check(PCM, "params->avail_min is 0");
    ■■return -EINVAL;
    ■}
    #if 0
    ■/* disable the check below - it looks too restrictive
    ■ * (start_threshold is basically independent from avail_min)
    ■ */
    ■if (params->start_threshold <= pcm->buffer_size &&
    ■     params->start_threshold > (pcm->buffer_size / params->avail_min) * params->avail_min) {
    ■■snd_check(PCM, "params->avail_min problem for start_threshold");
    ■■return -EINVAL;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1019) - [public] snd_check

```
    ■■snd_check(PCM, "params->avail_min problem for start_threshold");
    ■■return -EINVAL;
    ■}
    #endif
    ■__snd_pcm_lock(pcm->op_arg); /* forced lock due to pcm field change */
    ■if (pcm->ops->sw_params)
    ■■err = pcm->ops->sw_params(pcm->op_arg, params);
    ■else
    ■■err = -ENOSYS;
    ■if (err < 0) {
    ■■__snd_pcm_unlock(pcm->op_arg);
    ■■return err;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1029) - [public] __snd_pcm_unlock

```
    ■■__snd_pcm_unlock(pcm->op_arg);
    ■■return err;
    ■}
    ■pcm->tstamp_mode = params->tstamp_mode;
    ■pcm->tstamp_type = params->tstamp_type;
    ■pcm->period_step = params->period_step;
    ■pcm->avail_min = params->avail_min;
    ■pcm->period_event = sw_get_period_event(params);
    ■pcm->start_threshold = params->start_threshold;
    ■pcm->stop_threshold = params->stop_threshold;
    ■pcm->silence_threshold = params->silence_threshold;
    ■pcm->silence_size = params->silence_size;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1113) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■err = __snd_pcm_hwsync(pcm);
    ■snd_pcm_unlock(pcm->fast_op_arg);
    ■return err;
    ■}

    /**
    * \brief Obtain delay for a running PCM handle
    * \param pcm PCM handle
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1158) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■err = __snd_pcm_delay(pcm, delayp);
    ■snd_pcm_unlock(pcm->fast_op_arg);
    ■return err;
}

/**
 * \brief Resume from suspend, no samples are lost
 * \param pcm PCM handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1187) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■/* lock handled in the callback */
    ■if (pcm->fast_ops->resume)
    ■■err = pcm->fast_ops->resume(pcm->fast_op_arg);
    ■else
    ■■err = -ENOSYS;
    ■return err;
}

/**
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1216) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->htimestamp)
    ■■err = pcm->fast_ops->htimestamp(pcm->fast_op_arg, avail, tstamp);
    ■else
    ■■err = -ENOSYS;
    ■snd_pcm_unlock(pcm->fast_op_arg);
    ■return err;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1241) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■err = bad_pcm_state(pcm, ~P_STATE(DISCONNECTED), 0);
    ■if (err < 0)
    ■■return err;
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->prepare)
    ■■err = pcm->fast_ops->prepare(pcm->fast_op_arg);
    ■else
    ■■err = -ENOSYS;
    ■snd_pcm_unlock(pcm->fast_op_arg);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1271) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->reset)
    ■■err = pcm->fast_ops->reset(pcm->fast_op_arg);
    ■else
    ■■err = -ENOSYS;
    ■snd_pcm_unlock(pcm->fast_op_arg);
    ■return err;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm.c (line 1296) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE(PREPARED), 0);
    ■if (err < 0)
    ■■return err;
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■err = __snd_pcm_start(pcm);
    ■snd_pcm_unlock(pcm->fast_op_arg);
    ■return err;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm.c (line 1327) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE | P_STATE(SETUP) |
    ■■■ P_STATE(SUSPENDED), 0);
    ■if (err < 0)
    ■■return err;
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->drop)
    ■■err = pcm->fast_ops->drop(pcm->fast_op_arg);
    ■else
    ■■err = -ENOSYS;
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm.c (line 1331) - [public] P_STATE

```
    ■■■ P_STATE(SUSPENDED), 0);
    ■if (err < 0)
    ■■return err;
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->drop)
    ■■err = pcm->fast_ops->drop(pcm->fast_op_arg);
    ■else
    ■■err = -ENOSYS;
    ■snd_pcm_unlock(pcm->fast_op_arg);
    ■return err;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm.c (line 1364) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE | P_STATE_SETUP, P_STATE_SETUP);
    ■if (err < 0)
    ■■return err;
    ■if (err == 1)
    ■■return 0;
    ■/* lock handled in the callback */
    ■if (pcm->fast_ops->drain)
    ■■err = pcm->fast_ops->drain(pcm->fast_op_arg);
    ■else
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1398) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->pause)
    ■■err = pcm->fast_ops->pause(pcm->fast_op_arg, enable);
    ■else
    ■■err = -ENOSYS;
    ■snd_pcm_unlock(pcm->fast_op_arg);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1431) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->rewindable)
    ■■result = pcm->fast_ops->rewindable(pcm->fast_op_arg);
    ■else
    ■■result = -ENOSYS;
    ■snd_pcm_unlock(pcm->fast_op_arg);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1462) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■if (frames == 0)
    ■■return 0;
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->rewind)
    ■■result = pcm->fast_ops->rewind(pcm->fast_op_arg, frames);
    ■else
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1497) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->forwardable)
    ■■result = pcm->fast_ops->forwardable(pcm->fast_op_arg);
    ■else
    ■■result = -ENOSYS;
    ■snd_pcm_unlock(pcm->fast_op_arg);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1532) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■if (frames == 0)
    ■■return 0;
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■snd_pcm_lock(pcm->fast_op_arg);
    ■if (pcm->fast_ops->forward)
    ■■result = pcm->fast_ops->forward(pcm->fast_op_arg, frames);
    ■else
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1576) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■if (pcm->access != SND_PCM_ACCESS_RW_INTERLEAVED) {
    ■■snd_check(PCM, "invalid access type %s", snd_pcm_access_name(pcm->access));
    ■■return -EINVAL;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■return _snd_pcm_writei(pcm, buffer, size);
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1580) - [public] snd_check

```
    ■■snd_check(PCM, "invalid access type %s", snd_pcm_access_name(pcm->access));
    ■■return -EINVAL;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■return _snd_pcm_writei(pcm, buffer, size);
    }

    /**
     * \brief Write non interleaved frames to a PCM
     * \param pcm PCM handle
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1586) - [public] _snd_pcm_writei

```
    return _snd_pcm_writei(pcm, buffer, size);
}

/**
 * \brief Write non interleaved frames to a PCM
 * \param pcm PCM handle
 * \param bufs frames containing buffers (one for each channel)
 * \param size frames to be written
 * \return a positive number of frames actually written otherwise a
 * negative error code
 * \retval -EBADFD PCM is not in the right state (#SND_PCM_STATE_PREPARED or #SND_PCM_STATE_RUNNING)
 * \retval -EPIPE an underrun occurred
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1615) - [public] snd_check

```
    if (snd_check(pcm, "PCM not set up"))
        return -EIO;
}
if (pcm->access != SND_PCM_ACCESS_RW_NONINTERLEAVED) {
    if (snd_check(pcm, "invalid access type %s", snd_pcm_access_name(pcm->access)))
        return -EINVAL;
}
err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
if (err < 0)
    return err;
return _snd_pcm_writen(pcm, bufs, size);
}

// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1619) - [public] snd_check

```
    if (snd_check(pcm, "invalid access type %s", snd_pcm_access_name(pcm->access)))
        return -EINVAL;
}
err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
if (err < 0)
    return err;
return _snd_pcm_writen(pcm, bufs, size);
}

/**
 * \brief Read interleaved frames from a PCM
 * \param pcm PCM handle
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1625) - [public] _snd_pcm_writen

```
    return _snd_pcm_writen(pcm, bufs, size);
}

/**
 * \brief Read interleaved frames from a PCM
 * \param pcm PCM handle
 * \param buffer frames containing buffer
 * \param size frames to be read
 * \return a positive number of frames actually read otherwise a
 * negative error code
 * \retval -EBADFD PCM is not in the right state (#SND_PCM_STATE_PREPARED or #SND_PCM_STATE_RUNNING)
 * \retval -EPIPE an overrun occurred
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1654) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■if (pcm->access != SND_PCM_ACCESS_RW_INTERLEAVED) {
    ■■snd_check(PCM, "invalid access type %s", snd_pcm_access_name(pcm->access));
    ■■return -EINVAL;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■return _snd_pcm_readi(pcm, buffer, size);
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1658) - [public] snd_check

```
    ■■snd_check(PCM, "invalid access type %s", snd_pcm_access_name(pcm->access));
    ■■return -EINVAL;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■return _snd_pcm_readi(pcm, buffer, size);
}

/**
 * \brief Read non interleaved frames to a PCM
 * \param pcm PCM handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1664) - [public] _snd_pcm_readi

```
    ■return _snd_pcm_readi(pcm, buffer, size);
}

/**
 * \brief Read non interleaved frames to a PCM
 * \param pcm PCM handle
 * \param bufs frames containing buffers (one for each channel)
 * \param size frames to be read
 * \return a positive number of frames actually read otherwise a
 * negative error code
 * \retval -EBADFD PCM is not in the right state (#SND_PCM_STATE_PREPARED or #SND_PCM_STATE_RUNNING)
 * \retval -EPIPE an overrun occurred
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1693) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■if (pcm->access != SND_PCM_ACCESS_RW_NONINTERLEAVED) {
    ■■snd_check(PCM, "invalid access type %s", snd_pcm_access_name(pcm->access));
    ■■return -EINVAL;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■return _snd_pcm_readn(pcm, bufs, size);
}

// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 1697) - [public] snd_check

```
    ■■snd_check(PCM, "invalid access type %s", snd_pcm_access_name(pcm->access));
    ■■return -EINVAL;
    ■}
    ■err = bad_pcm_state(pcm, P_STATE_RUNNABLE, 0);
    ■if (err < 0)
    ■■return err;
    ■return _snd_pcm_readn(pcm, bufs, size);
    }

    /**
     * \brief Link two PCMs
     * \param pcm1 first PCM handle
     * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1703) - [public] _snd_pcm_readn

```
    ■■return _snd_pcm_readn(pcm, bufs, size);
    }

    /**
     * \brief Link two PCMs
     * \param pcm1 first PCM handle
     * \param pcm2 first PCM handle
     * \return 0 on success otherwise a negative error code
     *
     * The two PCMs will start/stop/prepare in sync.
     */
    int snd_pcm_link(snd_pcm_t *pcm1, snd_pcm_t *pcm2)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1777) - [public] snd_check

```
    ■■snd_check(PCM, "poll_fd < 0");
    ■■return -EIO;
    ■}
    ■if (space >= 1 && pfds) {
    ■■pfds->fd = pcm->poll_fd;
    ■■pfds->events = pcm->poll_events | POLLERR | POLLNVAL;
    ■} else {
    ■■return 0;
    ■}
    ■return 1;
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2182) - [public] strcasecmp

```
    ■■    strcasecmp(name, snd_pcm_format_names[format]) == 0) {
    ■■■return format;
    ■■}
    ■■if (snd_pcm_format_aliases[format] &&
    ■■    strcasecmp(name, snd_pcm_format_aliases[format]) == 0) {
    ■■■return format;
    ■■}
    ■}
    ■for (format = 0; format <= SND_PCM_FORMAT_LAST; format++) {
    ■■if (snd_pcm_format_descriptions[format] &&
    ■■    strcasecmp(name, snd_pcm_format_descriptions[format]) == 0) {
    ■■■return format;
    ■■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2186) - [public] strcasecmp

```
    if (strcasecmp(name, snd_pcm_format_aliases[format]) == 0) {
        return format;
    }
    for (format = 0; format <= SND_PCM_FORMAT_LAST; format++) {
        if (snd_pcm_format_descriptions[format] &&
            strcasecmp(name, snd_pcm_format_descriptions[format]) == 0) {
            return format;
        }
    }
    return SND_PCM_FORMAT_UNKNOWN;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2192) - [public] strcasecmp

```
    if (strcasecmp(name, snd_pcm_format_descriptions[format]) == 0) {
        return format;
    }
    return SND_PCM_FORMAT_UNKNOWN;
}

/**
 * \brief get name of PCM sample subformat
 * \param subformat PCM sample subformat
 * \return ascii name of PCM sample subformat
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2343) - [public] snd_check

```
    snd_check(pcm, "PCM not set up");
    return -EIO;
}

snd_output_printf(out, " stream      : %s\n", snd_pcm_stream_name(pcm->stream));
snd_output_printf(out, " access    : %s\n", snd_pcm_access_name(pcm->access));
snd_output_printf(out, " format     : %s\n", snd_pcm_format_name(pcm->format));
snd_output_printf(out, " subformat  : %s\n", snd_pcm_subformat_name(pcm->subformat));
snd_output_printf(out, " channels   : %u\n", pcm->channels);
snd_output_printf(out, " rate       : %u\n", pcm->rate);
snd_output_printf(out, " exact rate : %g (%u/%u)\n",
    (pcm->rate_den ? ((double) pcm->rate_num / pcm->rate_den) : 0.0),
    pcm->rate_num, pcm->rate_den);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2373) - [public] snd_check

```
    snd_check(pcm, "PCM not set up");
    return -EIO;
}

snd_output_printf(out, " tstamp_mode : %s\n", snd_pcm_tstamp_mode_name(pcm->tstamp_mode));
snd_output_printf(out, " tstamp_type : %s\n", snd_pcm_tstamp_type_name(pcm->tstamp_type));
snd_output_printf(out, " period_step : %d\n", pcm->period_step);
snd_output_printf(out, " avail_min   : %ld\n", pcm->avail_min);
snd_output_printf(out, " period_event : %i\n", pcm->period_event);
snd_output_printf(out, " start_threshold : %ld\n", pcm->start_threshold);
snd_output_printf(out, " stop_threshold  : %ld\n", pcm->stop_threshold);
snd_output_printf(out, " silence_threshold: %ld\n", pcm->silence_threshold);
snd_output_printf(out, " silence_size   : %ld\n", pcm->silence_size);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2452) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■return bytes * 8 / pcm->frame_bits;
}

/**
 * \brief Convert frames in bytes for a PCM
 * \param pcm PCM handle
 * \param frames quantity in frames
 * \return quantity expressed in bytes
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2468) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■return frames * pcm->frame_bits / 8;
}

/**
 * \brief Convert bytes in samples for a PCM
 * \param pcm PCM handle
 * \param bytes quantity in bytes
 * \return quantity expressed in samples
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2484) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■return bytes * 8 / pcm->sample_bits;
}

/**
 * \brief Convert samples in bytes for a PCM
 * \param pcm PCM handle
 * \param samples quantity in samples
 * \return quantity expressed in bytes
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2500) - [public] snd_check

```
    ■■snd_check(PCM, "PCM not set up");
    ■■return -EIO;
    ■}
    ■return samples * pcm->sample_bits / 8;
}

/**
 * \brief Add an async handler for a PCM
 * \param handler Returned handler handle
 * \param pcm PCM handle
 * \param callback Callback function
 * \param private_data Callback private data
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2532) - [public] if

```
    if (err < 0) {
        snd_async_del_handler(h);
        return err;
    }
    *handler = h;
    return 0;
}

/**
 * \brief Return PCM handle related to an async handler
 * \param handler Async handler handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2533) - [public] snd_async_del_handler

```
    snd_async_del_handler(h);
    return err;
}
*handler = h;
return 0;
}

/**
 * \brief Return PCM handle related to an async handler
 * \param handler Async handler handle
 * \return PCM handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2549) - [public] snd_check

```
    snd_check(PCM, "invalid handler type %d", handler->type);
    return NULL;
}
return handler->u.pcm;
}

static const char *const build_in_pcms[] = {
    "adpcm", "alaw", "copy", "dmix", "file", "hooks", "hw", "ladspa", "lfloat",
    "linear", "meter", "mulaw", "multi", "null", "empty", "plug", "rate", "route", "share",
    "shm", "dsnoop", "dshare", "asym", "iec958", "softvol", "mmap_emul",
    NULL
};
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2585) - [public] snd_error

```
    snd_error(PCM, "Invalid type for PCM %s%sdefinition (id: %s, value: %s)", name ? name : "", name ? " " : "");
    free(val);
    return -EINVAL;
}
err = snd_config_search(pcm_conf, "type", &conf);
if (err < 0) {
    snd_error(PCM, "type is not defined");
    return err;
}
err = snd_config_get_id(conf, &id);
if (err < 0) {
    snd_error(PCM, "unable to get id");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2586) - [public] free

```
    free(val);
    return -EINVAL;
}
err = snd_config_search(pcm_conf, "type", &conf);
if (err < 0) {
    snd_error(PCM, "type is not defined");
    return err;
}
err = snd_config_get_id(conf, &id);
if (err < 0) {
    snd_error(PCM, "unable to get id");
    return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2591) - [public] snd_error

```
    snd_error(PCM, "type is not defined");
    return err;
}
err = snd_config_get_id(conf, &id);
if (err < 0) {
    snd_error(PCM, "unable to get id");
    return err;
}
err = snd_config_get_string(conf, &str);
if (err < 0) {
    snd_error(PCM, "Invalid type for %s", id);
    return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2596) - [public] snd_error

```
    snd_error(PCM, "unable to get id");
    return err;
}
err = snd_config_get_string(conf, &str);
if (err < 0) {
    snd_error(PCM, "Invalid type for %s", id);
    return err;
}
err = snd_config_search_definition(pcm_root, "pcm_type", str, &type_conf);
if (err >= 0) {
    if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
        snd_error(PCM, "Invalid type for PCM type %s definition", str);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2601) - [public] snd_error

```
    snd_error(PCM, "Invalid type for %s", id);
    return err;
}
err = snd_config_search_definition(pcm_root, "pcm_type", str, &type_conf);
if (err >= 0) {
    if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
        snd_error(PCM, "Invalid type for PCM type %s definition", str);
        err = -EINVAL;
        goto _err;
    }
    snd_config_for_each(i, next, type_conf) {
        snd_config_t *n = snd_config_iterator_entry(i);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2606) - [public] if

```
    if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
        snd_error(PCM, "Invalid type for PCM type %s definition", str);
        err = -EINVAL;
        goto _err;
    }
    snd_config_for_each(i, next, type_conf) {
        snd_config_t *n = snd_config_iterator_entry(i);
        const char *id;
        if (snd_config_get_id(n, &id) < 0)
            continue;
        if (strcmp(id, "comment") == 0)
            continue;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2607) - [public] snd_error

```
    snd_error(PCM, "Invalid type for PCM type %s definition", str);
    err = -EINVAL;
    goto _err;
}
snd_config_for_each(i, next, type_conf) {
    snd_config_t *n = snd_config_iterator_entry(i);
    const char *id;
    if (snd_config_get_id(n, &id) < 0)
        continue;
    if (strcmp(id, "comment") == 0)
        continue;
    if (strcmp(id, "lib") == 0) {
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2611) - [public] snd_config_for_each

```
    snd_config_for_each(i, next, type_conf) {
        snd_config_t *n = snd_config_iterator_entry(i);
        const char *id;
        if (snd_config_get_id(n, &id) < 0)
            continue;
        if (strcmp(id, "comment") == 0)
            continue;
        if (strcmp(id, "lib") == 0) {
            err = snd_config_get_string(n, &lib);
            if (err < 0) {
                snd_error(PCM, "Invalid type for %s", id);
                goto _err;
            }
            // ... (truncated, see source file for more)
        }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2618) - [public] if

```
    if (strcmp(id, "lib") == 0) {
        err = snd_config_get_string(n, &lib);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    if (strcmp(id, "open") == 0) {
        err = snd_config_get_string(n, &open_name);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            // ... (truncated, see source file for more)
        }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2620) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        goto _err;
    }
    continue;
}
if (strcmp(id, "open") == 0) {
    err = snd_config_get_string(n, &open_name);
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        goto _err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2621) - [public] snd_error

```
    snd_error(PCM, "Invalid type for %s", id);
    goto _err;
}
continue;
}
if (strcmp(id, "open") == 0) {
    err = snd_config_get_string(n, &open_name);
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        goto _err;
    }
    continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2626) - [public] if

```
    if (strcmp(id, "open") == 0) {
        err = snd_config_get_string(n, &open_name);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            goto _err;
        }
        continue;
    }
    snd_error(PCM, "Unknown field %s", id);
    err = -EINVAL;
    goto _err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2628) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        goto _err;
    }
    continue;
}
snd_error(PCM, "Unknown field %s", id);
err = -EINVAL;
goto _err;
}
if (!open_name) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2629) - [public] snd_error

```
    snd_error(PCM, "Invalid type for %s", id);
    goto _err;
}
continue;
}
snd_error(PCM, "Unknown field %s", id);
err = -EINVAL;
goto _err;
}
}
if (!open_name) {
    buf = malloc(strlen(str) + 32);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2634) - [public] snd_error

```
    snd_error(PCM, "Unknown field %s", id);
    err = -EINVAL;
    goto _err;
}
}
if (!open_name) {
    buf = malloc(strlen(str) + 32);
    if (buf == NULL) {
        err = -ENOMEM;
        goto _err;
    }
    open_name = buf;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2641) - [public] if

```
    if (buf == NULL) {
        err = -ENOMEM;
        goto _err;
    }
    open_name = buf;
    sprintf(buf, "_snd_pcm_%s_open", str);
}
if (!lib) {
    const char *const *build_in = build_in_pcms;
    while (*build_in) {
        if (!strcmp(*build_in, str))
            break;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2646) - [public] sprintf

```
    sprintf(buf, "_snd_pcm_%s_open", str);
}
if (!lib) {
    const char *const *build_in = build_in_pcms;
    while (*build_in) {
        if (!strcmp(*build_in, str))
            break;
        build_in++;
    }
    if (*build_in == NULL) {
        buf1 = malloc(strlen(str) + 32);
        if (buf1 == NULL) {
            // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 2650) - [public] while

```
while (*build_in) {
    if (!strcmp(*build_in, str))
        break;
    build_in++;
}
if (*build_in == NULL) {
    buf1 = malloc(strlen(str) + 32);
    if (buf1 == NULL) {
        err = -ENOMEM;
        goto _err;
    }
    lib = buf1;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2655) - [public] if

```
if (*build_in == NULL) {
    buf1 = malloc(strlen(str) + 32);
    if (buf1 == NULL) {
        err = -ENOMEM;
        goto _err;
    }
    lib = buf1;
    sprintf(buf1, "libasound_module_pcm_%s.so", str);
}
#endif
snd_pcm_open_symbols();/* this call is for static linking only */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2657) - [public] if

```
if (buf1 == NULL) {
    err = -ENOMEM;
    goto _err;
}
lib = buf1;
sprintf(buf1, "libasound_module_pcm_%s.so", str);
}
#endif
snd_pcm_open_symbols();/* this call is for static linking only */
#endif
open_func = snd_dlobj_cache_get(lib, open_name,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2662) - [public] sprintf

```
sprintf(buf1, "libasound_module_pcm_%s.so", str);
}
}
#endif
snd_pcm_open_symbols();/* this call is for static linking only */
#endif
open_func = snd_dlobj_cache_get(lib, open_name,
    SND_DLSYM_VERSION(SND_PCM_DLSYM_VERSION), 1);
if (open_func) {
    err = open_func(pcm, name, pcm_root, pcm_conf, stream, mode);
    if (err >= 0) {
        if ((*pcm)->open_func) {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2669) - [public] SND_DLSYM_VERSION

```
    ■■■SND_DLSYM_VERSION(SND_PCM_DLSYM_VERSION), 1);
    ■if (open_func) {
    ■err = open_func(pcmp, name, pcm_root, pcm_conf, stream, mode);
    ■if (err >= 0) {
    ■■if ((*pcmp)->open_func) {
    ■■■/* only init plugin (like empty, asym) */
    ■■■snd_dlobj_cache_put(open_func);
    ■■} else {
    ■■■(*pcmp)->open_func = open_func;
    ■■}
    ■err = 0;
    ■} else {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2672) - [public] if

```
    ■if (err >= 0) {
    ■■if ((*pcmp)->open_func) {
    ■■■/* only init plugin (like empty, asym) */
    ■■■snd_dlobj_cache_put(open_func);
    ■■} else {
    ■■■(*pcmp)->open_func = open_func;
    ■■}
    ■err = 0;
    ■} else {
    ■■snd_dlobj_cache_put(open_func);
    ■}
    ■} else {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2673) - [public] if

```
    ■■if ((*pcmp)->open_func) {
    ■■■/* only init plugin (like empty, asym) */
    ■■■snd_dlobj_cache_put(open_func);
    ■■} else {
    ■■■(*pcmp)->open_func = open_func;
    ■■}
    ■err = 0;
    ■} else {
    ■■snd_dlobj_cache_put(open_func);
    ■}
    ■} else {
    ■err = -ENXIO;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2675) - [public] snd_dlobj_cache_put

```
static int snd_pcm_open_conf(snd_pcm_t **pcmp, const char *name,
snd_config_t *pcm_root, snd_config_t *pcm_conf,
snd_pcm_stream_t stream, int mode)
{
    const char *str;
    char *buf = NULL, *buf1 = NULL;
    int err;
    snd_config_t *conf, *type_conf = NULL, *tmp;
    snd_config_iterator_t i, next;
    const char *id;
    const char *lib = NULL, *open_name = NULL;
    int (*open_func)(snd_pcm_t **, const char *,
snd_config_t *, snd_config_t *,
snd_pcm_stream_t, int) = NULL;
#ifdef PIC
extern void *snd_pcm_open_symbols(void);
#endif
    if (snd_config_get_type(pcm_conf) != SND_CONFIG_TYPE_COMPOUND) {
        char *val;
        if (snd_config_get_id(pcm_conf, &id) < 0)
            id = NULL;
        if (snd_config_get_ascii(pcm_conf, &val) < 0)
            val = NULL;
        snd_error(PCM, "Invalid type for PCM %s%sdefinition (id: %s, value: %s)", name ? name : "", name ? " ": "", id, val);
        free(val);
        return -EINVAL;
    }
    err = snd_config_search(pcm_conf, "type", &conf);
    if (err < 0) {
        snd_error(PCM, "type is not defined");
        return err;
    }
    err = snd_config_get_id(conf, &id);
    if (err < 0) {
        snd_error(PCM, "unable to get id");
        return err;
    }
    err = snd_config_get_string(conf, &str);
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return err;
    }
    err = snd_config_search_definition(pcm_root, "pcm_type", str, &type_conf);
    if (err >= 0) {
        if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
            snd_error(PCM, "Invalid type for PCM type %s definition", str);
            err = -EINVAL;
            goto _err;
        }
        snd_config_for_each(i, next, type_conf) {
            snd_config_t *n = snd_config_iterator_entry(i);
            const char *id;
            if (snd_config_get_id(n, &id) < 0)
                continue;
            if (strcmp(id, "comment") == 0)
                continue;
            if (strcmp(id, "lib") == 0) {
                err = snd_config_get_string(n, &lib);
                if (err < 0) {
                    snd_error(PCM, "Invalid type for %s", id);
                }
            }
        }
    }
}
```

```

goto _err;
}
continue;
}
if (strcmp(id, "open") == 0) {
err = snd_config_get_string(n, &open_name);
if (err < 0) {
snd_error(PCM, "Invalid type for %s", id);
goto _err;
}
continue;
}
snd_error(PCM, "Unknown field %s", id);
err = -EINVAL;
goto _err;
}
}
if (!open_name) {
buf = malloc(strlen(str) + 32);
if (buf == NULL) {
err = -ENOMEM;
goto _err;
}
open_name = buf;
sprintf(buf, "_snd_pcm_%s_open", str);
}
if (!lib) {
const char *const *build_in = build_in_pcms;
while (*build_in) {
if (!strcmp(*build_in, str))
break;
build_in++;
}
if (*build_in == NULL) {
buf1 = malloc(strlen(str) + 32);
if (buf1 == NULL) {
err = -ENOMEM;
goto _err;
}
lib = buf1;
sprintf(buf1, "libasound_module_pcm_%s.so", str);
}
}
#ifdef PIC
snd_pcm_open_symbols(); /* this call is for static linking only */
#endif
open_func = snd_dlobj_cache_get(lib, open_name,
SND_DLSYM_VERSION(SND_PCM_DLSYM_VERSION), 1);
if (open_func) {
err = open_func(pcmp, name, pcm_root, pcm_conf, stream, mode);
if (err >= 0) {
if ((*pcmp)->open_func) {
/* only init plugin (like empty, asym) */
■■■■snd_dlobj_cache_put(open_func);
■■■} else {
■■■■(*pcmp)->open_func = open_func;
■■■}
■■■err = 0;
■■} else {
■■■snd_dlobj_cache_put(open_func);
■■}
}
}

```

```

    } else {
        err = -ENXIO;
    }
    if (err >= 0) {
        // ... (truncated, see source file for more)
    }

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2681) - [public] snd_dlobj_cache_put

```

    snd_dlobj_cache_put(open_func);
}
} else {
    err = -ENXIO;
}
if (err >= 0) {
    err = snd_config_search(pcm_root, "defaults.pcm.compat", &tmp);
    if (err >= 0) {
        long i;
        if (snd_config_get_integer(tmp, &i) >= 0) {
            if (i > 0)
                (*pcmp)->compat = 1;
            // ... (truncated, see source file for more)
        }
    }
}

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2688) - [public] if

```

    if (err >= 0) {
        long i;
        if (snd_config_get_integer(tmp, &i) >= 0) {
            if (i > 0)
                (*pcmp)->compat = 1;
        }
    } else {
        char *str = getenv("LIBASOUND_COMPAT");
        if (str && *str)
            (*pcmp)->compat = 1;
    }
    err = snd_config_search(pcm_root, "defaults.pcm.minperiodtime", &tmp);
    // ... (truncated, see source file for more)
}

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2690) - [public] if

```

    if (snd_config_get_integer(tmp, &i) >= 0) {
        if (i > 0)
            (*pcmp)->compat = 1;
    }
} else {
    char *str = getenv("LIBASOUND_COMPAT");
    if (str && *str)
        (*pcmp)->compat = 1;
}
err = snd_config_search(pcm_root, "defaults.pcm.minperiodtime", &tmp);
if (err >= 0)
    snd_config_get_integer(tmp, &(*pcmp)->minperiodtime);
// ... (truncated, see source file for more)
}

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2701) - [public] snd_config_get_integer

```

    snd_config_get_integer(tmp, &(*pcmp)->minperiodtime);
    err = 0;
}
_err:
if (type_conf)
    snd_config_delete(type_conf);
free(buf);
free(buf1);
return err;
}

```

```

static int snd_pcm_open_noupdate(snd_pcm_t **pcmp, snd_config_t *root,
    // ... (truncated, see source file for more)
)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2706) - [public] snd_config_delete

```
    ■■snd_config_delete(type_conf);
    ■free(buf);
    ■free(buf1);
    ■return err;
}

static int snd_pcm_open_noupdate(snd_pcm_t **pcmp, snd_config_t *root,
    ■■■■ const char *name, snd_pcm_stream_t stream,
    ■■■■ int mode, int hop)
{
    ■int err;
    ■snd_config_t *pcm_conf;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2722) - [public] snd_error

```
    ■■snd_error(PCM, "Unknown PCM %s", name);
    ■■return err;
    ■}
    ■if (snd_config_get_string(pcm_conf, &str) >= 0)
    ■■err = snd_pcm_open_noupdate(pcmp, root, str, stream, mode,
    ■■■■ hop + 1);
    ■else {
    ■■snd_config_set_hop(pcm_conf, hop);
    ■■err = snd_pcm_open_conf(pcmp, name, root, pcm_conf, stream, mode);
    ■}
    ■snd_config_delete(pcm_conf);
    ■return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2729) - [public] snd_config_set_hop

```
    ■■snd_config_set_hop(pcm_conf, hop);
    ■■err = snd_pcm_open_conf(pcmp, name, root, pcm_conf, stream, mode);
    ■}
    ■snd_config_delete(pcm_conf);
    ■return err;
}

/**
 * \brief Opens a PCM
 * \param pcmp Returned PCM handle
 * \param name ASCII identifier of the PCM handle
 * \param stream Wanted stream
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2779) - [public] snd_pcm_open_noupdate

```
    ■return snd_pcm_open_noupdate(pcmp, lconf, name, stream, mode, 0);
}

/**
 * \brief Opens a fallback PCM
 * \param pcmp Returned PCM handle
 * \param root Configuration root
 * \param name ASCII identifier of the PCM handle
 * \param orig_name The original ASCII name
 * \param stream Wanted stream
 * \param mode Open mode (see #SND_PCM_NONBLOCK, #SND_PCM_ASYNC)
 * \return 0 on success otherwise a negative error code
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2800) - [public] free

```
    free((*pcmp)->name);
    (*pcmp)->name = orig_name ? strdup(orig_name) : NULL;
}
return err;
}

#ifdef DOC_HIDDEN
int snd_pcm_new(snd_pcm_t **pcmp, snd_pcm_type_t type, const char *name,
snd_pcm_stream_t stream, int mode)
{
    snd_pcm_t *pcm;
#ifdef THREAD_SAFE_API
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2847) - [public] if

/ evaluate env var only once at the first open for consistency */*

```
    if (do_lock_enable == -1) {
        char *p = getenv("LIBASOUND_THREAD_SAFE");
        do_lock_enable = !p || *p != '0';
    }
    pcm->lock_enabled = do_lock_enable;
}
#endif
*pcmp = pcm;
return 0;
}

```

```
int snd_pcm_free(snd_pcm_t *pcm)
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2885) - [public] snd_pcm_open_conf

```
return snd_pcm_open_conf(pcm, name, root, conf, stream, mode);
}
#endif

/**
 * \brief Wait for a PCM to become ready
 * \param pcm PCM handle
 * \param timeout maximum time in milliseconds to wait,
 *        a -1 value means infinity (SND_PCM_WAIT_INFINITE),
 *        see also SND_PCM_WAIT_IO and SND_PCM_WAIT_DRAIN
 * \return a positive value on success otherwise a negative error code
 *        (-EPIPE for the xrun and -ESTRPIPE for the suspended status,
 *        // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2926) - [public] snd_pcm_wait_noccheck

```
return snd_pcm_wait_noccheck(pcm, timeout);
}

static int __snd_pcm_wait_io_timeout(snd_pcm_t *pcm)
{
    int timeout;

    /* period size is the time boundary */
    timeout = (pcm->period_size * 1000ULL) / pcm->rate;
    /* should not happen */
    if (timeout < 0)
        timeout = 0;
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2975) - [public] snd_error

```
    ■■snd_error(PCM, "Invalid poll_fds %d", npfds);
    ■■return -EIO;
    ■}
    ■pfd = alloca(sizeof(*pfd) * npfds);
    ■err = __snd_pcm_poll_descriptors(pcm, pfd, npfds);
    ■if (err < 0)
    ■■return err;
    ■if (err != npfds) {
    ■■snd_check(PCM, "invalid poll descriptors %d", err);
    ■■return -EIO;
    ■}
    ■if (timeout == SND_PCM_WAIT_IO)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2983) - [public] snd_check

```
    ■■snd_check(PCM, "invalid poll descriptors %d", err);
    ■■return -EIO;
    ■}
    ■if (timeout == SND_PCM_WAIT_IO)
    ■■timeout = __snd_pcm_wait_io_timeout(pcm);
    ■else if (timeout == SND_PCM_WAIT_DRAIN)
    ■■timeout = __snd_pcm_wait_drain_timeout(pcm);
    ■else if (timeout < -1)
    ■■snd_check(PCM, "invalid snd_pcm_wait timeout argument %d", timeout);
    ■do {
    ■■__snd_pcm_unlock(pcm->fast_op_arg);
    ■■err_poll = poll(pfd, npfds, timeout);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2991) - [public] snd_check

```
    ■■snd_check(PCM, "invalid snd_pcm_wait timeout argument %d", timeout);
    ■do {
    ■■__snd_pcm_unlock(pcm->fast_op_arg);
    ■■err_poll = poll(pfd, npfds, timeout);
    ■■__snd_pcm_lock(pcm->fast_op_arg);
    ■■if (err_poll < 0) {
    ■■■if (errno == EINTR && !PCMINABORT(pcm) && !(pcm->mode & SND_PCM_EINTR))
    ■■■■continue;
    ■■■return -errno;
    ■■}
    ■■if (! err_poll)
    ■■■break;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2993) - [public] __snd_pcm_unlock

```
    ■■__snd_pcm_unlock(pcm->fast_op_arg);
    ■■err_poll = poll(pfd, npfds, timeout);
    ■■__snd_pcm_lock(pcm->fast_op_arg);
    ■■if (err_poll < 0) {
    ■■■if (errno == EINTR && !PCMINABORT(pcm) && !(pcm->mode & SND_PCM_EINTR))
    ■■■■continue;
    ■■■return -errno;
    ■■}
    ■■if (! err_poll)
    ■■■break;
    ■■err = __snd_pcm_poll_revents(pcm, pfd, npfds, &revents);
    ■■if (err < 0)
        // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 2995) - [public] __snd_pcm_lock

```
    __snd_pcm_lock(pcm->fast_op_arg);
    if (err_poll < 0) {
        if (errno == EINTR && !PCMINABORT(pcm) && !(pcm->mode & SND_PCM_EINTR))
            continue;
        return -errno;
    }
    if (! err_poll)
        break;
    err = __snd_pcm_poll_revents(pcm, pfd, npfds, &revents);
    if (err < 0)
        return err;
    if (revents & (POLLERR | POLLNVAL)) {
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2996) - [public] if

```
    if (err_poll < 0) {
        if (errno == EINTR && !PCMINABORT(pcm) && !(pcm->mode & SND_PCM_EINTR))
            continue;
        return -errno;
    }
    if (! err_poll)
        break;
    err = __snd_pcm_poll_revents(pcm, pfd, npfds, &revents);
    if (err < 0)
        return err;
    if (revents & (POLLERR | POLLNVAL)) {
        /* check more precisely */
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3006) - [public] if

```
    if (revents & (POLLERR | POLLNVAL)) {
        /* check more precisely */
        err = pcm_state_to_error(__snd_pcm_state(pcm));
        return err < 0 ? err : -EIO;
    }
    while (!(revents & (POLLIN | POLLOUT)));
    #if 0 /* very useful code to test poll related problems */
    {
        snd_pcm_sframes_t avail_update;
        __snd_pcm_hwsync(pcm);
        avail_update = __snd_pcm_avail_update(pcm);
        if (avail_update < (snd_pcm_sframes_t)pcm->avail_min) {
            // ... (truncated, see source file for more)
        }
    }
#endif
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3015) - [public] __snd_pcm_hwsync

```
    __snd_pcm_hwsync(pcm);
    avail_update = __snd_pcm_avail_update(pcm);
    if (avail_update < (snd_pcm_sframes_t)pcm->avail_min) {
        printf("*** snd_pcm_wait() FATAL ERROR!!!\n");
        printf("avail_min = %li, avail_update = %li\n", pcm->avail_min, avail_update);
    }
    #endif
    return err_poll > 0 ? 1 : 0;
}
#endif

// ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 3017) - [public] if
    if (avail_update < (snd_pcm_sframes_t)pcm->avail_min) {
        printf("*** snd_pcm_wait() FATAL ERROR!!!\n");
        printf("avail_min = %li, avail_update = %li\n", pcm->avail_min, avail_update);
    }
}
#endif
return err_poll > 0 ? 1 : 0;
}
#endif

/**
 * \brief Return number of frames ready to be read (capture) / written (playback)
 * // ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 3018) - [public] printf
    printf("*** snd_pcm_wait() FATAL ERROR!!!\n");
    printf("avail_min = %li, avail_update = %li\n", pcm->avail_min, avail_update);
}
}
#endif
return err_poll > 0 ? 1 : 0;
}
#endif

/**
 * \brief Return number of frames ready to be read (capture) / written (playback)
 * \param pcm PCM handle
 * // ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 3019) - [public] printf
    printf("avail_min = %li, avail_update = %li\n", pcm->avail_min, avail_update);
}
}
#endif
return err_poll > 0 ? 1 : 0;
}
#endif

/**
 * \brief Return number of frames ready to be read (capture) / written (playback)
 * \param pcm PCM handle
 * \return a positive number of frames ready otherwise a negative
 * // ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 3081) - [public] snd_check
    snd_check(PCM, "PCM not set up");
    return -EIO;
}
snd_pcm_lock(pcm->fast_op_arg);
err = __snd_pcm_hwsync(pcm);
if (err < 0)
    result = err;
else
    result = __snd_pcm_avail_update(pcm);
snd_pcm_unlock(pcm->fast_op_arg);
return result;
}

// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3114) - [public] snd_check

```
    snd_check(PCM, "PCM not set up");
    return -EIO;
}
snd_pcm_lock(pcm->fast_op_arg);
err = __snd_pcm_hwsync(pcm);
if (err < 0)
    goto unlock;

/*
 * Delay value is relative to avail, so we have to
 * loop to avoid reporting stale delay data.
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3128) - [public] if

```
    if (sf < 0) {
        err = sf < INT_MIN ? -EOVERFLOW : (int)sf;
        goto unlock;
    }
    if (ok && sf == *availp)
        break;
    *availp = sf;
    err = __snd_pcm_delay(pcm, delayp);
    if (err < 0)
        goto unlock;
    ok = 1;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3190) - [public] while

```
    while (samples-- > 0) {
        if (dstbit) {
            *dst &= 0xf0;
            *dst |= s1;
        } else {
            *dst &= 0x0f;
            *dst |= s0;
        }
        dst += dst_step;
        dstbit += dstbit_step;
        if (dstbit == 8) {
            dst++;
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3191) - [public] if

```
    if (dstbit) {
        *dst &= 0xf0;
        *dst |= s1;
    } else {
        *dst &= 0x0f;
        *dst |= s0;
    }
    dst += dst_step;
    dstbit += dstbit_step;
    if (dstbit == 8) {
        dst++;
        dstbit = 0;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3200) - [public] if

```
    if (dstbit == 8) {
        dst++;
        dstbit = 0;
    }
    break;
}
case 8: {
    uint8_t sil = silence;
    while (samples-- > 0) {
        *dst = sil;
        dst += dst_step;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3209) - [public] while

```
while (samples-- > 0) {
    *dst = sil;
    dst += dst_step;
}
break;
}
case 16: {
    uint16_t sil = silence;
    while (samples-- > 0) {
        *(uint16_t*)dst = sil;
        dst += dst_step;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3217) - [public] while

```
while (samples-- > 0) {
    *(uint16_t*)dst = sil;
    dst += dst_step;
}
break;
}
case 24: {
    while (samples-- > 0) {
#ifdef SNDRV_LITTLE_ENDIAN
        *(dst + 0) = silence >> 0;
        *(dst + 1) = silence >> 8;
        *(dst + 2) = silence >> 16;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3224) - [public] while

```
while (samples-- > 0) {
#ifdef SNDRV_LITTLE_ENDIAN
    *(dst + 0) = silence >> 0;
    *(dst + 1) = silence >> 8;
    *(dst + 2) = silence >> 16;
#else
    *(dst + 2) = silence >> 0;
    *(dst + 1) = silence >> 8;
    *(dst + 0) = silence >> 16;
#endif
    dst += dst_step;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3240) - [public] while

```
    while (samples-- > 0) {
        *(uint32_t*)dst = sil;
        dst += dst_step;
    }
    break;
}
case 64: {
    while (samples-- > 0) {
        *(uint64_t*)dst = silence;
        dst += dst_step;
    }
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3247) - [public] while

```
    while (samples-- > 0) {
        *(uint64_t*)dst = silence;
        dst += dst_step;
    }
    break;
}
default:
    snd_check(PCM, "invalid format width %d", width);
    return -EINVAL;
}
return 0;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3254) - [public] snd_check

```
    snd_check(PCM, "invalid format width %d", width);
    return -EINVAL;
}
return 0;
}

/**
 * \brief Silence one or more areas
 * \param dst_areas areas specification (one for each channel)
 * \param dst_offset offset in frames inside area
 * \param channels channels count
 * \param frames frames to silence
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3280) - [public] while

```
    while (1) {
        channels1--;
        chns++;
        dst_areas++;
        if (channels1 == 0 ||
            dst_areas->addr != addr ||
            dst_areas->step != step ||
            dst_areas->first != dst_areas[-1].first + width)
            break;
    }
    if (chns > 1 && chns * width == step) {
        /* Collapse the areas */
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3290) - [public] if

```
    if (chns > 1 && chns * width == step) {
        /* Collapse the areas */
        snd_pcm_channel_area_t d;
        d.addr = begin->addr;
        d.first = begin->first;
        d.step = width;
        err = snd_pcm_area_silence(&d, dst_offset * chns, frames * chns, format);
        channels -= chns;
    } else {
        err = snd_pcm_area_silence(begin, dst_offset, frames, format);
        dst_areas = begin + 1;
        channels--;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3332) - [public] snd_pcm_area_silence

```
    return snd_pcm_area_silence(dst_area, dst_offset, samples, format);
    src = snd_pcm_channel_area_addr(src_area, src_offset);
    if (!dst_area->addr)
        return 0;
    dst = snd_pcm_channel_area_addr(dst_area, dst_offset);
    width = snd_pcm_format_physical_width(format);
    if (width < 0)
        return width;
    if (src_area->step == (unsigned int) width &&
        dst_area->step == (unsigned int) width) {
        size_t bytes = samples * width / 8;
        samples -= bytes * 8 / width;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3344) - [public] assert

```
    assert(src < dst || src >= dst + bytes);
    assert(dst < src || dst >= src + bytes);
    memcpy(dst, src, bytes);
    if (samples == 0)
        return 0;
}
src_step = src_area->step / 8;
dst_step = dst_area->step / 8;
switch (width) {
case 4: {
    int srcbit = src_area->first % 8;
    int srcbit_step = src_area->step % 8;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3345) - [public] assert

```
    assert(dst < src || dst >= src + bytes);
    memcpy(dst, src, bytes);
    if (samples == 0)
        return 0;
}
src_step = src_area->step / 8;
dst_step = dst_area->step / 8;
switch (width) {
case 4: {
    int srcbit = src_area->first % 8;
    int srcbit_step = src_area->step % 8;
    int dstbit = dst_area->first % 8;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3346) - [public] memcpy

```
    memcpy(dst, src, bytes);
    if (samples == 0)
        return 0;
}
src_step = src_area->step / 8;
dst_step = dst_area->step / 8;
switch (width) {
case 4: {
    int srcbit = src_area->first % 8;
    int srcbit_step = src_area->step % 8;
    int dstbit = dst_area->first % 8;
    int dstbit_step = dst_area->step % 8;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3358) - [public] while

```
    while (samples-- > 0) {
        unsigned char srcval;
        if (srcbit)
            srcval = *src & 0x0f;
        else
            srcval = *src & 0xf0;
        if (dstbit)
            *dst &= 0xf0;
        else
            *dst &= 0x0f;
        *dst |= srcval;
        src += src_step;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3371) - [public] if

```
        if (srcbit == 8) {
            src++;
            srcbit = 0;
        }
        dst += dst_step;
        dstbit += dstbit_step;
        if (dstbit == 8) {
            dst++;
            dstbit = 0;
        }
    }
    break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3377) - [public] if

```
        if (dstbit == 8) {
            dst++;
            dstbit = 0;
        }
    }
    break;
}
case 8: {
    while (samples-- > 0) {
        *dst = *src;
        src += src_step;
        dst += dst_step;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3385) - [public] while

```
    while (samples-- > 0) {
        *dst = *src;
        src += src_step;
        dst += dst_step;
    }
    break;
}
case 16: {
    while (samples-- > 0) {
        *(uint16_t*)dst = *(const uint16_t*)src;
        src += src_step;
        dst += dst_step;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3393) - [public] while

```
    while (samples-- > 0) {
        *(uint16_t*)dst = *(const uint16_t*)src;
        src += src_step;
        dst += dst_step;
    }
    break;
}
case 24:
    while (samples-- > 0) {
        *(dst + 0) = *(src + 0);
        *(dst + 1) = *(src + 1);
        *(dst + 2) = *(src + 2);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3401) - [public] while

```
    while (samples-- > 0) {
        *(dst + 0) = *(src + 0);
        *(dst + 1) = *(src + 1);
        *(dst + 2) = *(src + 2);
        src += src_step;
        dst += dst_step;
    }
    break;
case 32: {
    while (samples-- > 0) {
        *(uint32_t*)dst = *(const uint32_t*)src;
        src += src_step;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3410) - [public] while

```
    while (samples-- > 0) {
        *(uint32_t*)dst = *(const uint32_t*)src;
        src += src_step;
        dst += dst_step;
    }
    break;
}
case 64: {
    while (samples-- > 0) {
        *(uint64_t*)dst = *(const uint64_t*)src;
        src += src_step;
        dst += dst_step;
        // ... (truncated, see source file for more)
    }
}
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 3418) - [public] while

```
    while (samples-- > 0) {
        *(uint64_t*)dst = *(const uint64_t*)src;
        src += src_step;
        dst += dst_step;
    }
    break;
}
default:
    snd_check(PCM, "invalid format width %d", width);
    return -EINVAL;
}
return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3426) - [public] snd_check

```
    snd_check(PCM, "invalid format width %d", width);
    return -EINVAL;
}
return 0;
}

/**
 * \brief Copy one or more areas
 * \param dst_areas destination areas specification (one for each channel)
 * \param dst_offset offset in frames inside destination area
 * \param src_areas source areas specification (one for each channel)
 * \param src_offset offset in frames inside source area
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3451) - [public] snd_check

```
    snd_check(PCM, "invalid channels %d", channels);
    return -EINVAL;
}
if (! frames) {
    snd_check(PCM, "invalid frames %ld", frames);
    return -EINVAL;
}
while (channels > 0) {
    unsigned int step = src_areas->step;
    void *src_addr = src_areas->addr;
    const snd_pcm_channel_area_t *src_start = src_areas;
    void *dst_addr = dst_areas->addr;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3455) - [public] snd_check

```
    snd_check(PCM, "invalid frames %ld", frames);
    return -EINVAL;
}
while (channels > 0) {
    unsigned int step = src_areas->step;
    void *src_addr = src_areas->addr;
    const snd_pcm_channel_area_t *src_start = src_areas;
    void *dst_addr = dst_areas->addr;
    const snd_pcm_channel_area_t *dst_start = dst_areas;
    int channels1 = channels;
    unsigned int chns = 0;
    while (dst_areas->step == step) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3466) - [public] while

```
    while (dst_areas->step == step) {
        channels1--;
        chns++;
        src_areas++;
        dst_areas++;
        if (channels1 == 0 ||
            src_areas->step != step ||
            src_areas->addr != src_addr ||
            dst_areas->addr != dst_addr ||
            src_areas->first != src_areas[-1].first + width ||
            dst_areas->first != dst_areas[-1].first + width)
            break;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3479) - [public] if

```
    if (chns > 1 && chns * width == step) {
        if (src_offset != dst_offset ||
            src_start->addr != dst_start->addr ||
            src_start->first != dst_start->first) {
            /* Collapse the areas */
            snd_pcm_channel_area_t s, d;
            s.addr = src_start->addr;
            s.first = src_start->first;
            s.step = width;
            d.addr = dst_start->addr;
            d.first = dst_start->first;
            d.step = width;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3574) - [public] dump_one_param

```
    dump_one_param(params, k, out);
    for (k = SND_PCM_HW_PARAM_FIRST_INTERVAL; k <= SND_PCM_HW_PARAM_LAST_INTERVAL; k++)
        dump_one_param(params, k, out);
    return 0;
}

/**
 * \brief Check if hardware supports sample-resolution mmap for given configuration
 * \param params Configuration space
 * \retval 0 Hardware doesn't support sample-resolution mmap
 * \retval 1 Hardware supports sample-resolution mmap
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3576) - [public] dump_one_param

```
    dump_one_param(params, k, out);
    return 0;
}

/**
 * \brief Check if hardware supports sample-resolution mmap for given configuration
 * \param params Configuration space
 * \retval 0 Hardware doesn't support sample-resolution mmap
 * \retval 1 Hardware supports sample-resolution mmap
 *
 * This function should only be called when the configuration space
 * contains a single configuration. Call #snd_pcm_hw_params to choose
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3594) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !!(params->info & SNDRV_PCM_INFO_MMAP_VALID);
}

/**
 * \brief Check if hardware does double buffering for start/stop for given configuration
 * \param params Configuration space
 * \retval 0 Hardware doesn't do double buffering for start/stop
 * \retval 1 Hardware does double buffering for start/stop
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3614) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !!(params->info & SNDRV_PCM_INFO_DOUBLE);
}

/**
 * \brief Check if hardware does double buffering for data transfers for given configuration
 * \param params Configuration space
 * \retval 0 Hardware doesn't do double buffering for data transfers
 * \retval 1 Hardware does double buffering for data transfers
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3634) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !!(params->info & SNDRV_PCM_INFO_BATCH);
}

/**
 * \brief Check if hardware does block transfers for samples for given configuration
 * \param params Configuration space
 * \retval 0 Hardware doesn't block transfers
 * \retval 1 Hardware does block transfers
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3654) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !!(params->info & SNDRV_PCM_INFO_BLOCK_TRANSFER);
}

/**
 * \brief Check if timestamps are monotonic for given configuration
 * \param params Configuration space
 * \retval 0 Device doesn't do monotomic timestamps
 * \retval 1 Device does monotonic timestamps
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3674) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !(params->info & SND_PCM_INFO_MONOTONIC);
}

/**
 * \brief Check if hardware supports overrange detection
 * \param params Configuration space
 * \retval 0 Hardware doesn't support overrange detection
 * \retval 1 Hardware supports overrange detection
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3694) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !(params->info & SNDRV_PCM_INFO_OVERRANGE);
}

/**
 * \brief Check if hardware supports pause
 * \param params Configuration space
 * \retval 0 Hardware doesn't support pause
 * \retval 1 Hardware supports pause
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3714) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !(params->info & SNDRV_PCM_INFO_PAUSE);
}

/**
 * \brief Check if hardware supports resume
 * \param params Configuration space
 * \retval 0 Hardware doesn't support resume
 * \retval 1 Hardware supports resume
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3734) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !(params->info & SNDRV_PCM_INFO_RESUME);
}

/**
 * \brief Check if hardware does half-duplex only
 * \param params Configuration space
 * \retval 0 Hardware doesn't do half-duplex
 * \retval 1 Hardware does half-duplex
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3754) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !!(params->info & SNDRV_PCM_INFO_HALF_DUPLEX);
}

/**
 * \brief Check if hardware does joint-duplex (playback and capture are somewhat correlated)
 * \param params Configuration space
 * \retval 0 Hardware doesn't do joint-duplex
 * \retval 1 Hardware does joint-duplex
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3774) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !!(params->info & SNDRV_PCM_INFO_JOINT_DUPLEX);
}

/**
 * \brief Check if hardware supports synchronized start with sample resolution
 * \param params Configuration space
 * \retval 0 Hardware doesn't support synchronized start
 * \retval 1 Hardware supports synchronized start
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3794) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !!(params->info & SNDRV_PCM_INFO_SYNC_START);
}

/**
 * \brief Check if hardware can disable period wakeups
 * \param params Configuration space
 * \retval 0 Hardware cannot disable period wakeups
 * \retval 1 Hardware can disable period wakeups
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3810) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !!(params->info & SNDRV_PCM_INFO_NO_PERIOD_WAKEUP);
}

/**
 * \brief Check if hardware is capable of perfect drain
 * \param params Configuration space
 * \retval 0 Hardware doesn't do perfect drain
 * \retval 1 Hardware does perfect drain
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3833) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■return !!(params->info & SNDRV_PCM_INFO_PERFECT_DRAIN);
}

/**
 * \brief Check if hardware supports audio wallclock timestamps
 * \param params Configuration space
 * \retval 0 Hardware doesn't support audio wallclock timestamps
 * \retval 1 Hardware supports audio wallclock timestamps
 *
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3871) - [public] snd_check

```
    ■■snd_check(PCM, "invalid PCM info field");
    ■■return 0; /* FIXME: should be a negative error? */
    ■}
    ■switch (type) {
    ■case SNDRV_PCM_AUDIO_TSTAMP_TYPE_COMPAT:
    ■■return !!(params->info & SNDRV_PCM_INFO_HAS_WALL_CLOCK); /* deprecated */
    ■case SNDRV_PCM_AUDIO_TSTAMP_TYPE_DEFAULT:
    ■■return 1; /* always supported, based on hw_ptr */
    ■case SNDRV_PCM_AUDIO_TSTAMP_TYPE_LINK:
    ■■return !!(params->info & SNDRV_PCM_INFO_HAS_LINK_ETIME);
    ■case SNDRV_PCM_AUDIO_TSTAMP_TYPE_LINK_ABSOLUTE:
    ■■return !!(params->info & SNDRV_PCM_INFO_HAS_LINK_ABSOLUTE_ETIME);
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3908) - [public] snd_check

```
    ■■snd_check(PCM, "invalid rate_den value");
    ■■return -EINVAL;
    ■}
    ■*rate_num = params->rate_num;
    ■*rate_den = params->rate_den;
    ■return 0;
}

/**
 * \brief Get sample resolution info from a configuration space
 * \param params Configuration space
 * \return sample resolution (in bits) otherwise a negative error code if the info is not available
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3938) - [public] snd_check

```
    ■■snd_check(PCM, "invalid msbits value");
    ■■return -EINVAL;
    ■}
    ■return params->msbits;
}

/**
 * \brief Get hardware FIFO size info from a configuration space
 * \param params Configuration space
 * \return FIFO size in frames otherwise a negative error code if the info is not available
 *
 * This function should only be called when the configuration space
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3957) - [public] snd_check

```
■snd_check(PCM, "invalid PCM info field");
■return -EINVAL;
■}
■return params->fifo_size;
}

/**
 * \brief Get hardware synchronization ID from a PCM info container
 * \param params Configuration space
 * \return 16-byte synchronization ID (use #SND_PCM_HW_PARAMS_SYNC_SIZE)
 *
 * This synchronization ID determines the similar clocks for the
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3998) - [public] snd_pcm_hw_refine

```
■return snd_pcm_hw_refine(pcm, params);
}

/**
 * \brief get size of #snd_pcm_access_mask_t
 * \return size in bytes
 */
size_t snd_pcm_access_mask_sizeof()
{
■return sizeof(snd_pcm_access_mask_t);
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4007) - [public] sizeof

```
■return sizeof(snd_pcm_access_mask_t);
}

/**
 * \brief allocate an empty #snd_pcm_access_mask_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_pcm_access_mask_malloc(snd_pcm_access_mask_t **ptr)
{
■assert(ptr);
■*ptr = calloc(1, sizeof(snd_pcm_access_mask_t));
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4069) - [public] snd_mask_test

```
■return snd_mask_test((const snd_mask_t *) mask, (unsigned long) val);
}

/**
 * \brief test, if given a #snd_pcm_access_mask_t is empty
 * \param mask pointer to mask
 * \retval 0 not empty
 * \retval 1 empty
 */
int snd_pcm_access_mask_empty(const snd_pcm_access_mask_t *mask)
{
■return snd_mask_empty((const snd_mask_t *) mask);
// ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 4080) - [public] snd_mask_empty
    return snd_mask_empty((const snd_mask_t *) mask);
}

/**
 * \brief make an access type present in a #snd_pcm_access_mask_t
 * \param mask pointer to mask
 * \param val access type
 */
void snd_pcm_access_mask_set(snd_pcm_access_mask_t *mask, snd_pcm_access_t val)
{
    snd_mask_set((snd_mask_t *) mask, (unsigned long) val);
}

// ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 4109) - [public] sizeof
    return sizeof(snd_pcm_format_mask_t);
}

/**
 * \brief allocate an empty #snd_pcm_format_mask_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_pcm_format_mask_malloc(snd_pcm_format_mask_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_pcm_format_mask_t));
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 4171) - [public] snd_mask_test
    return snd_mask_test((const snd_mask_t *) mask, (unsigned long) val);
}

/**
 * \brief test, if given a #snd_pcm_format_mask_t is empty
 * \param mask pointer to mask
 * \retval 0 not empty
 * \retval 1 empty
 */
int snd_pcm_format_mask_empty(const snd_pcm_format_mask_t *mask)
{
    return snd_mask_empty((const snd_mask_t *) mask);
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 4182) - [public] snd_mask_empty
    return snd_mask_empty((const snd_mask_t *) mask);
}

/**
 * \brief make a format present in a #snd_pcm_format_mask_t
 * \param mask pointer to mask
 * \param val format
 */
void snd_pcm_format_mask_set(snd_pcm_format_mask_t *mask, snd_pcm_format_t val)
{
    snd_mask_set((snd_mask_t *) mask, (unsigned long) val);
}

// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 4212) - [public] sizeof

```
    return sizeof(snd_pcm_subformat_mask_t);
}

/**
 * \brief allocate an empty #snd_pcm_subformat_mask_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_pcm_subformat_mask_malloc(snd_pcm_subformat_mask_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_pcm_subformat_mask_t));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4274) - [public] snd_mask_test

```
    return snd_mask_test((const snd_mask_t *) mask, (unsigned long) val);
}

/**
 * \brief test, if given a #snd_pcm_subformat_mask_t is empty
 * \param mask pointer to mask
 * \retval 0 not empty
 * \retval 1 empty
 */
int snd_pcm_subformat_mask_empty(const snd_pcm_subformat_mask_t *mask)
{
    return snd_mask_empty((const snd_mask_t *) mask);
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4285) - [public] snd_mask_empty

```
    return snd_mask_empty((const snd_mask_t *) mask);
}

/**
 * \brief make a subformat present in a #snd_pcm_subformat_mask_t
 * \param mask pointer to mask
 * \param val subformat
 */
void snd_pcm_subformat_mask_set(snd_pcm_subformat_mask_t *mask, snd_pcm_subformat_t val)
{
    snd_mask_set((snd_mask_t *) mask, (unsigned long) val);
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4315) - [public] sizeof

```
    return sizeof(snd_pcm_hw_params_t);
}

/**
 * \brief allocate an invalid #snd_pcm_hw_params_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_pcm_hw_params_malloc(snd_pcm_hw_params_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_pcm_hw_params_t));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4381) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_ACCESS, access, 0);
}

/**
 * \brief Restrict a configuration space to contain only one access type
 * \param pcm PCM handle
 * \param params Configuration space
 * \param access access type
 * \return 0 otherwise a negative error code if configuration space would become empty
 */
int snd_pcm_hw_params_set_access(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_access_t access)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4393) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_ACCESS, access, 0);
}

/**
 * \brief Restrict a configuration space to contain only its first access type
 * \param pcm PCM handle
 * \param params Configuration space
 * \param access Returned first access type
 * \return 0 otherwise a negative error code
 */
#ifdef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_set_access_first)(snd_pcm_t *pcm, snd_pcm_hw_params_t *params)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4409) - [public] snd_pcm_hw_param_set_first

```
■return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_ACCESS, access, NULL);
}

/**
 * \brief Restrict a configuration space to contain only its last access type
 * \param pcm PCM handle
 * \param params Configuration space
 * \param access Returned last access type
 * \return 0 otherwise a negative error code
 */
#ifdef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_set_access_last)(snd_pcm_t *pcm, snd_pcm_hw_params_t *params)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4425) - [public] snd_pcm_hw_param_set_last

```
■return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_ACCESS, access, NULL);
}

/**
 * \brief Restrict a configuration space to contain only a set of access types
 * \param pcm PCM handle
 * \param params Configuration space
 * \param mask Access mask
 * \return 0 otherwise a negative error code
 */
int snd_pcm_hw_params_set_access_mask(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_access_mask_t
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4437) - [public] snd_pcm_hw_param_set_mask

```
■return snd_pcm_hw_param_set_mask(pcm, params, SND_TRY, SND_PCM_HW_PARAM_ACCESS, (snd_mask_t *) mask);
}

/**
 * \brief Get access mask from a configuration space
 * \param params Configuration space
 * \param mask Returned Access mask
 */
int snd_pcm_hw_params_get_access_mask(snd_pcm_hw_params_t *params, snd_pcm_access_mask_t *mask)
{
    ■if (params == NULL || mask == NULL)
        ■return -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4466) - [public] snd_pcm_hw_param_get

```
■return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_FORMAT, (unsigned int *)format, NULL);
}

/**
 * \brief Verify if a format is available inside a configuration space for a PCM
 * \param pcm PCM handle
 * \param params Configuration space
 * \param format format
 * \return 0 if available a negative error code otherwise
 */
int snd_pcm_hw_params_test_format(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_format_t format)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4478) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_FORMAT, format, 0);
}

/**
 * \brief Restrict a configuration space to contain only one format
 * \param pcm PCM handle
 * \param params Configuration space
 * \param format format
 * \return 0 otherwise a negative error code
 */
int snd_pcm_hw_params_set_format(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_format_t format)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4490) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_FORMAT, format, 0);
}

/**
 * \brief Restrict a configuration space to contain only its first format
 * \param pcm PCM handle
 * \param params Configuration space
 * \param format Returned first format
 * \return 0 otherwise a negative error code
 */
#ifdef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_set_format_first)(snd_pcm_t *pcm, snd_pcm_hw_params_t *param
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4506) - [public] snd_pcm_hw_param_set_first

```
■return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_FORMAT, (unsigned int *)format, NULL);
}

/**
 * \brief Restrict a configuration space to contain only its last format
 * \param pcm PCM handle
 * \param params Configuration space
 * \param format Returned last format
 * \return 0 otherwise a negative error code
 */
#ifdef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_set_format_last)(snd_pcm_t *pcm, snd_pcm_hw_params_t *params
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4522) - [public] snd_pcm_hw_param_set_last

```
■return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_FORMAT, (unsigned int *)format, NULL);
}

/**
 * \brief Restrict a configuration space to contain only a set of formats
 * \param pcm PCM handle
 * \param params Configuration space
 * \param mask Format mask
 * \return 0 otherwise a negative error code
 */
int snd_pcm_hw_params_set_format_mask(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_format_mask_t
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4534) - [public] snd_pcm_hw_param_set_mask

```
■return snd_pcm_hw_param_set_mask(pcm, params, SND_TRY, SND_PCM_HW_PARAM_FORMAT, (snd_mask_t *) mask);
}

/**
 * \brief Get format mask from a configuration space
 * \param params Configuration space
 * \param mask Returned Format mask
 */
void snd_pcm_hw_params_get_format_mask(snd_pcm_hw_params_t *params, snd_pcm_format_mask_t *mask)
{
    ■snd_pcm_format_mask_copy(mask, snd_pcm_hw_param_get_mask(params, SND_PCM_HW_PARAM_FORMAT));
}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4560) - [public] snd_pcm_hw_param_get

```
■return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_SUBFORMAT, (unsigned int *)subformat, NULL);
}

/**
 * \brief Verify if a subformat is available inside a configuration space for a PCM
 * \param pcm PCM handle
 * \param params Configuration space
 * \param subformat subformat value
 * \return 0 if available a negative error code otherwise
 */
int snd_pcm_hw_params_test_subformat(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_subformat_t sub
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4572) - [public] snd_pcm_hw_param_set

```
return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_SUBFORMAT, subformat, 0);
}
```

```
/**
```

```
* \brief Restrict a configuration space to contain only one subformat
* \param pcm PCM handle
* \param params Configuration space
* \param subformat subformat value
* \return 0 otherwise a negative error code if configuration space would become empty
*/
```

```
int snd_pcm_hw_params_set_subformat(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_subformat_t subf
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4584) - [public] snd_pcm_hw_param_set

```
return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_SUBFORMAT, subformat, 0);
}
```

```
/**
```

```
* \brief Restrict a configuration space to contain only its first subformat
* \param pcm PCM handle
* \param params Configuration space
* \param subformat Returned subformat
* \return 0 otherwise a negative error code
*/
```

```
#ifndef DOXYGEN
```

```
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_set_subformat_first)(snd_pcm_t *pcm, snd_pcm_hw_params_t *pa
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4600) - [public] snd_pcm_hw_param_set_first

```
return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_SUBFORMAT, (unsigned int *)subformat, NU
}
```

```
/**
```

```
* \brief Restrict a configuration space to contain only its last subformat
* \param pcm PCM handle
* \param params Configuration space
* \param subformat Returned subformat
* \return 0 otherwise a negative error code
*/
```

```
#ifndef DOXYGEN
```

```
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_set_subformat_last)(snd_pcm_t *pcm, snd_pcm_hw_params_t *pa
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4616) - [public] snd_pcm_hw_param_set_last

```
return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_SUBFORMAT, (unsigned int *)subformat, NU
}
```

```
/**
```

```
* \brief Restrict a configuration space to contain only a set of subformats
* \param pcm PCM handle
* \param params Configuration space
* \param mask Subformat mask
* \return 0 otherwise a negative error code
*/
```

```
int snd_pcm_hw_params_set_subformat_mask(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_subformat_m
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4628) - [public] snd_pcm_hw_param_set_mask

```
    return snd_pcm_hw_param_set_mask(pcm, params, SND_TRY, SND_PCM_HW_PARAM_SUBFORMAT, (snd_mask_t *) mask);
}

/**
 * \brief Get subformat mask from a configuration space
 * \param params Configuration space
 * \param mask Returned Subformat mask
 */
void snd_pcm_hw_params_get_subformat_mask(snd_pcm_hw_params_t *params, snd_pcm_subformat_mask_t *mask)
{
    snd_pcm_subformat_mask_copy(mask, snd_pcm_hw_param_get_mask(params, SND_PCM_HW_PARAM_SUBFORMAT));
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4654) - [public] snd_pcm_hw_param_get

```
    return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_CHANNELS, val, NULL);
}

/**
 * \brief Extract minimum channels count from a configuration space
 * \param params Configuration space
 * \param val minimum channels count
 * \return 0 otherwise a negative error code
 */
#ifdef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_get_channels_min)(const snd_pcm_hw_params_t *params, unsigned int *val)
#else
// ... (truncated, see source file for more)
#endif
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4669) - [public] snd_pcm_hw_param_get_min

```
    return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_CHANNELS, val, NULL);
}

/**
 * \brief Extract maximum channels count from a configuration space
 * \param params Configuration space
 * \param val maximum channels count
 * \return 0 otherwise a negative error code
 */
#ifdef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_get_channels_max)(const snd_pcm_hw_params_t *params, unsigned int *val)
#else
// ... (truncated, see source file for more)
#endif
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4684) - [public] snd_pcm_hw_param_get_max

```
    return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_CHANNELS, val, NULL);
}

/**
 * \brief Verify if a channels count is available inside a configuration space for a PCM
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val channels count
 * \return 0 if available a negative error code otherwise
 */
int snd_pcm_hw_params_test_channels(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int val)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4696) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_CHANNELS, val, 0);
}

/**
 * \brief Restrict a configuration space to contain only one channels count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val channels count
 * \return 0 otherwise a negative error code if configuration space would become empty
 */
int snd_pcm_hw_params_set_channels(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int val)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4708) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_CHANNELS, val, 0);
}

/**
 * \brief Restrict a configuration space with a minimum channels count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val minimum channels count (on return filled with actual minimum)
 * \return 0 otherwise a negative error code if configuration space would become empty
 */
int snd_pcm_hw_params_set_channels_min(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int *val)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4720) - [public] snd_pcm_hw_param_set_min

```
■return snd_pcm_hw_param_set_min(pcm, params, SND_TRY, SND_PCM_HW_PARAM_CHANNELS, val, NULL);
}

/**
 * \brief Restrict a configuration space with a maximum channels count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val maximum channels count (on return filled with actual maximum)
 * \return 0 otherwise a negative error code if configuration space would become empty
 */
int snd_pcm_hw_params_set_channels_max(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int *val)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4732) - [public] snd_pcm_hw_param_set_max

```
■return snd_pcm_hw_param_set_max(pcm, params, SND_TRY, SND_PCM_HW_PARAM_CHANNELS, val, NULL);
}

/**
 * \brief Restrict a configuration space to have channels counts in a given range
 * \param pcm PCM handle
 * \param params Configuration space
 * \param min minimum channels count (on return filled with actual minimum)
 * \param max maximum channels count (on return filled with actual maximum)
 * \return 0 otherwise a negative error code if configuration space would become empty
 */
int snd_pcm_hw_params_set_channels_minmax(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int *min,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4745) - [public] snd_pcm_hw_param_set_minmax

```
    return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY, SND_PCM_HW_PARAM_CHANNELS, min, NULL, max, NULL);
}

/**
 * \brief Restrict a configuration space to have channels count nearest to a target
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val target channels count, returned chosen channels count
 * \return 0 otherwise a negative error code if configuration space is empty
 */
#ifndef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_set_channels_near)(snd_pcm_t *pcm, snd_pcm_hw_params_t *params,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4761) - [public] snd_pcm_hw_param_set_near

```
    return snd_pcm_hw_param_set_near(pcm, params, SND_PCM_HW_PARAM_CHANNELS, val, NULL);
}

/**
 * \brief Restrict a configuration space to contain only its minimum channels count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val minimum channels count
 * \return 0 otherwise a negative error code
 */
#ifndef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_set_channels_first)(snd_pcm_t *pcm, snd_pcm_hw_params_t *params,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4777) - [public] snd_pcm_hw_param_set_first

```
    return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_CHANNELS, val, NULL);
}

/**
 * \brief Restrict a configuration space to contain only its maximum channels count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val maximum channels count
 * \return 0 otherwise a negative error code
 */
#ifndef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_set_channels_last)(snd_pcm_t *pcm, snd_pcm_hw_params_t *params,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4793) - [public] snd_pcm_hw_param_set_last

```
    return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_CHANNELS, val, NULL);
}

/**
 * \brief Extract rate from a configuration space
 * \param params Configuration space
 * \param val Returned approximate rate
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if the configuration space does not contain a single value
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 4812) - [public] snd_pcm_hw_param_get

```
■return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_RATE, val, dir);  
}
```

```
/**  
 * \brief Extract minimum rate from a configuration space  
 * \param params Configuration space  
 * \param val Returned approximate minimum rate  
 * \param dir Sub unit direction  
 * \return 0 otherwise a negative error code  
 *  
 * Exact value is <,> the returned one following dir (-1,0,1)  
 */  
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4830) - [public] snd_pcm_hw_param_get_min

```
■return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_RATE, val, dir);  
}
```

```
/**  
 * \brief Extract maximum rate from a configuration space  
 * \param params Configuration space  
 * \param val Returned approximate maximum rate  
 * \param dir Sub unit direction  
 * \return 0 otherwise a negative error code  
 *  
 * Exact value is <,> the returned one following dir (-1,0,1)  
 */  
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4848) - [public] snd_pcm_hw_param_get_max

```
■return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_RATE, val, dir);  
}
```

```
/**  
 * \brief Verify if a rate is available inside a configuration space for a PCM  
 * \param pcm PCM handle  
 * \param params Configuration space  
 * \param val approximate rate  
 * \param dir Sub unit direction  
 * \return 0 if available a negative error code otherwise  
 *  
 * Wanted exact value is <,> val following dir (-1,0,1)  
 // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4863) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_RATE, val, dir);  
}
```

```
/**  
 * \brief Restrict a configuration space to contain only one rate  
 * \param pcm PCM handle  
 * \param params Configuration space  
 * \param val approximate rate  
 * \param dir Sub unit direction  
 * \return 0 otherwise a negative error code if configuration space would become empty  
 *  
 * Wanted exact value is <,> val following dir (-1,0,1)  
 // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4878) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_RATE, val, dir);
}

/**
 * \brief Restrict a configuration space with a minimum rate
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate minimum rate (on return filled with actual minimum)
 * \param dir Sub unit direction (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted/actual exact minimum is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4893) - [public] snd_pcm_hw_param_set_min

```
■return snd_pcm_hw_param_set_min(pcm, params, SND_TRY, SND_PCM_HW_PARAM_RATE, val, dir);
}

/**
 * \brief Restrict a configuration space with a maximum rate
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate maximum rate (on return filled with actual maximum)
 * \param dir Sub unit direction (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted/actual exact maximum is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4908) - [public] snd_pcm_hw_param_set_max

```
■return snd_pcm_hw_param_set_max(pcm, params, SND_TRY, SND_PCM_HW_PARAM_RATE, val, dir);
}

/**
 * \brief Restrict a configuration space to have rates in a given range
 * \param pcm PCM handle
 * \param params Configuration space
 * \param min approximate minimum rate (on return filled with actual minimum)
 * \param mindir Sub unit direction for minimum (on return filled with actual direction)
 * \param max approximate maximum rate (on return filled with actual maximum)
 * \param maxdir Sub unit direction for maximum (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4925) - [public] snd_pcm_hw_param_set_minmax

```
■return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY, SND_PCM_HW_PARAM_RATE, min, mindir, max, maxdir);
}

/**
 * \brief Restrict a configuration space to have rate nearest to a target
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate target rate / returned approximate set rate
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if configuration space is empty
 *
 * target/chosen exact value is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4944) - [public] snd_pcm_hw_param_set_near
    return snd_pcm_hw_param_set_near(pcm, params, SND_PCM_HW_PARAM_RATE, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only its minimum rate
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val Returned minimum approximate rate
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)
*/

dirtycode/alsa-lib/src/pcm/pcm.c (line 4963) - [public] snd_pcm_hw_param_set_first
    return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_RATE, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only its maximum rate
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val Returned maximum approximate rate
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)
*/

dirtycode/alsa-lib/src/pcm/pcm.c (line 4982) - [public] snd_pcm_hw_param_set_last
    return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_RATE, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only real hardware rates
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val 0 = disable, 1 = enable (default) rate resampling
 * \return 0 otherwise a negative error code
 */
int snd_pcm_hw_params_set_rate_resample(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int val)
{
    // ... (truncated, see source file for more)
}

dirtycode/alsa-lib/src/pcm/pcm.c (line 5000) - [public] snd_pcm_hw_refine
    return snd_pcm_hw_refine(pcm, params);
}

/**
 * \brief Extract resample state from a configuration space
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val 0 = disable, 1 = enable rate resampling
 * \return 0 otherwise a negative error code
 */
int snd_pcm_hw_params_get_rate_resample(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int *val)
{
    // ... (truncated, see source file for more)
}

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5032) - [public] snd_pcm_hw_refine

```
    return snd_pcm_hw_refine(pcm, params);
}

/**
 * \brief Extract buffer accessibility from a configuration space
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val 0 = disable, 1 = enable exporting buffer
 * \return 0 otherwise a negative error code
 */
int snd_pcm_hw_params_get_export_buffer(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int *val)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5083) - [public] snd_pcm_hw_refine

```
    return snd_pcm_hw_refine(pcm, params);
}

/**
 * \brief Extract period wakeup flag from a configuration space
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val 0 = disabled, 1 = enabled period wakeups
 * \return Zero on success, otherwise a negative error code.
 */
int snd_pcm_hw_params_get_period_wakeup(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int *val)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5120) - [public] snd_pcm_hw_refine

```
    return snd_pcm_hw_refine(pcm, params);
}

/**
 * \brief Extract drain with the filling of silence samples from a configuration space
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val 0 = disabled, 1 = enabled
 * \return 0 otherwise a negative error code
 */
int snd_pcm_hw_params_get_drain_silence(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int *val)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5152) - [public] snd_pcm_hw_param_get

```
    return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}

/**
 * \brief Extract minimum period time from a configuration space
 * \param params Configuration space
 * \param val approximate minimum period duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Exact value is <,> the returned one following dir (-1,0,1)
 */
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5170) - [public] snd_pcm_hw_param_get_min

```
■return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}
```

```
/**
 * \brief Extract maximum period time from a configuration space
 * \param params Configuration space
 * \param val approximate maximum period duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Exact value is <,> the returned one following dir (-1,0,1)
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5188) - [public] snd_pcm_hw_param_get_max

```
■return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}
```

```
/**
 * \brief Verify if a period time is available inside a configuration space for a PCM
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate period duration in us
 * \param dir Sub unit direction
 * \return 0 if available a negative error code otherwise
 *
 * Wanted exact value is <,> val following dir (-1,0,1)
 // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5203) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}
```

```
/**
 * \brief Restrict a configuration space to contain only one period time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate period duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted exact value is <,> val following dir (-1,0,1)
 // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5218) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}
```

```
/**
 * \brief Restrict a configuration space with a minimum period time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate minimum period duration in us (on return filled with actual minimum)
 * \param dir Sub unit direction (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5234) - [public] snd_pcm_hw_param_set_min

```
■return snd_pcm_hw_param_set_min(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space with a maximum period time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate maximum period duration in us (on return filled with actual maximum)
 * \param dir Sub unit direction (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted/actual exact maximum is <,<=,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5249) - [public] snd_pcm_hw_param_set_max

```
■return snd_pcm_hw_param_set_max(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space to have period times in a given range
 * \param pcm PCM handle
 * \param params Configuration space
 * \param min approximate minimum period duration in us (on return filled with actual minimum)
 * \param mindir Sub unit direction for minimum (on return filled with actual direction)
 * \param max approximate maximum period duration in us (on return filled with actual maximum)
 * \param maxdir Sub unit direction for maximum (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5266) - [public] snd_pcm_hw_param_set_minmax

```
■return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIOD_TIME, min, mindir, max, maxdir);
}

/**
 * \brief Restrict a configuration space to have period time nearest to a target
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate target period duration in us / returned chosen approximate target period duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if configuration space is empty
 *
 * target/chosen exact value is <,<=,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5285) - [public] snd_pcm_hw_param_set_near

```
■return snd_pcm_hw_param_set_near(pcm, params, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only its minimum period time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val Returned approximate period duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Actual exact value is <,<=,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 5304) - [public] snd_pcm_hw_param_set_first
    return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only its maximum period time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val Returned maximum approximate period time
 * \param dir Sub unit direction
 * \return approximate period duration in us
 */
#ifdef DOXYGEN
    // ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 5321) - [public] snd_pcm_hw_param_set_last
    return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
}

/**
 * \brief Extract period size from a configuration space
 * \param params Configuration space
 * \param val Returned approximate period size in frames
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if the configuration space does not contain a single value
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 5403) - [public] snd_pcm_hw_param_set
    return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_PERIOD_SIZE, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only one period size
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate period size in frames
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted exact value is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 5418) - [public] snd_pcm_hw_param_set
    return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIOD_SIZE, val, dir);
}

/**
 * \brief Restrict a configuration space with a minimum period size
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate minimum period size in frames (on return filled with actual minimum)
 * \param dir Sub unit direction (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted/actual exact minimum is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5558) - [public] snd_pcm_hw_param_set_integer

```
■return snd_pcm_hw_param_set_integer(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIOD_SIZE);
}
```

```
/**
 * \brief Extract periods from a configuration space
 * \param params Configuration space
 * \param val approximate periods per buffer
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if the configuration space does not contain a single value
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5577) - [public] snd_pcm_hw_param_get

```
■return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_PERIODS, val, dir);
}
```

```
/**
 * \brief Extract minimum periods count from a configuration space
 * \param params Configuration space
 * \param val approximate minimum periods per buffer
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Exact value is <,> the returned one following dir (-1,0,1)
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5595) - [public] snd_pcm_hw_param_get_min

```
■return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_PERIODS, val, dir);
}
```

```
/**
 * \brief Extract maximum periods count from a configuration space
 * \param params Configuration space
 * \param val approximate maximum periods per buffer
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Exact value is <,> the returned one following dir (-1,0,1)
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5613) - [public] snd_pcm_hw_param_get_max

```
■return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_PERIODS, val, dir);
}
```

```
/**
 * \brief Verify if a periods count is available inside a configuration space for a PCM
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate periods per buffer
 * \param dir Sub unit direction
 * \return 0 if available a negative error code otherwise
 *
 * Wanted exact value is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 5628) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_PERIODS, val, dir);
}
```

```
/**
 * \brief Restrict a configuration space to contain only one periods count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate periods per buffer
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted exact value is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5643) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIODS, val, dir);
}
```

```
/**
 * \brief Restrict a configuration space with a minimum periods count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate minimum periods per buffer (on return filled with actual minimum)
 * \param dir Sub unit direction (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted/actual exact minimum is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5658) - [public] snd_pcm_hw_param_set_min

```
■return snd_pcm_hw_param_set_min(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIODS, val, dir);
}
```

```
/**
 * \brief Restrict a configuration space with a maximum periods count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate maximum periods per buffer (on return filled with actual maximum)
 * \param dir Sub unit direction (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted/actual exact maximum is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5673) - [public] snd_pcm_hw_param_set_max

```
■return snd_pcm_hw_param_set_max(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIODS, val, dir);
}
```

```
/**
 * \brief Restrict a configuration space to have periods counts in a given range
 * \param pcm PCM handle
 * \param params Configuration space
 * \param min approximate minimum periods per buffer (on return filled with actual minimum)
 * \param mindir Sub unit direction for minimum (on return filled with actual direction)
 * \param max approximate maximum periods per buffer (on return filled with actual maximum)
 * \param maxdir Sub unit direction for maximum (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5690) - [public] snd_pcm_hw_param_set_minmax

```
■return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIODS, min, mindir, max, maxdir);
}

/**
 * \brief Restrict a configuration space to have periods count nearest to a target
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate target periods per buffer / returned chosen approximate target periods per buffer
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if configuration space is empty
 *
 * target/chosen exact value is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5709) - [public] snd_pcm_hw_param_set_near

```
■return snd_pcm_hw_param_set_near(pcm, params, SND_PCM_HW_PARAM_PERIODS, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only its minimum periods count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val Returned approximate minimum periods per buffer
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5728) - [public] snd_pcm_hw_param_set_first

```
■return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_PERIODS, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only its maximum periods count
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val Returned approximate maximum periods per buffer
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5747) - [public] snd_pcm_hw_param_set_last

```
■return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_PERIODS, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only integer periods counts
 * \param pcm PCM handle
 * \param params Configuration space
 * \return 0 otherwise a negative error code if configuration space would become empty
 */
int snd_pcm_hw_params_set_periods_integer(snd_pcm_t *pcm, snd_pcm_hw_params_t *params)
{
    ■return snd_pcm_hw_param_set_integer(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIODS);
    // ... (truncated, see source file for more)
}

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 5758) - [public] snd_pcm_hw_param_set_integer
    return snd_pcm_hw_param_set_integer(pcm, params, SND_TRY, SND_PCM_HW_PARAM_PERIODS);
}
```

```
/**
 * \brief Extract buffer time from a configuration space
 * \param params Configuration space
 * \param val Returned buffer time in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if the configuration space does not contain a single value
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 5777) - [public] snd_pcm_hw_param_get
    return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}
```

```
/**
 * \brief Extract minimum buffer time from a configuration space
 * \param params Configuration space
 * \param val approximate minimum buffer duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Exact value is <,> the returned one following dir (-1,0,1)
 */
// ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 5795) - [public] snd_pcm_hw_param_get_min
    return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}
```

```
/**
 * \brief Extract maximum buffer time from a configuration space
 * \param params Configuration space
 * \param val approximate maximum buffer duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Exact value is <,> the returned one following dir (-1,0,1)
 */
// ... (truncated, see source file for more)

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 5813) - [public] snd_pcm_hw_param_get_max
    return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}
```

```
/**
 * \brief Verify if a buffer time is available inside a configuration space for a PCM
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate buffer duration in us
 * \param dir Sub unit direction
 * \return 0 if available a negative error code otherwise
 *
 * Wanted exact value is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5828) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only one buffer time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate buffer duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted exact value is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5843) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space with a minimum buffer time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate minimum buffer duration in us (on return filled with actual minimum)
 * \param dir Sub unit direction (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted/actual exact minimum is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5858) - [public] snd_pcm_hw_param_set_min

```
■return snd_pcm_hw_param_set_min(pcm, params, SND_TRY, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space with a maximum buffer time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate maximum buffer duration in us (on return filled with actual maximum)
 * \param dir Sub unit direction (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted/actual exact maximum is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5873) - [public] snd_pcm_hw_param_set_max

```
■return snd_pcm_hw_param_set_max(pcm, params, SND_TRY, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space to have buffer times in a given range
 * \param pcm PCM handle
 * \param params Configuration space
 * \param min approximate minimum buffer duration in us (on return filled with actual minimum)
 * \param mindir Sub unit direction for minimum (on return filled with actual direction)
 * \param max approximate maximum buffer duration in us (on return filled with actual maximum)
 * \param maxdir Sub unit direction for maximum (on return filled with actual direction)
 * \return 0 otherwise a negative error code if configuration space would become empty
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5890) - [public] snd_pcm_hw_param_set_minmax

```
■return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY, SND_PCM_HW_PARAM_BUFFER_TIME, min, mindir, max,
}

/**
 * \brief Restrict a configuration space to have buffer time nearest to a target
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate target buffer duration in us / returned chosen approximate target buffer duration
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code if configuration space is empty
 *
 * target/chosen exact value is <,> val following dir (-1,0,1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5909) - [public] snd_pcm_hw_param_set_near

```
■return snd_pcm_hw_param_set_near(pcm, params, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only its minimum buffer time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val Returned approximate minimum buffer duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5928) - [public] snd_pcm_hw_param_set_first

```
■return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}

/**
 * \brief Restrict a configuration space to contain only its maximum buffered time
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val Returned approximate maximum buffer duration in us
 * \param dir Sub unit direction
 * \return 0 otherwise a negative error code
 *
 * Actual exact value is <,> the approximate one following dir (-1, 0, 1)
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5947) - [public] snd_pcm_hw_param_set_last

```
■return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
}

/**
 * \brief Extract buffer size from a configuration space
 * \param params Configuration space
 * \param val Returned buffer size in frames
 * \return 0 otherwise a negative error code if the configuration space does not contain a single value
 */
#ifndef DOXYGEN
EXPORT_SYMBOL int INTERNAL(snd_pcm_hw_params_get_buffer_size)(const snd_pcm_hw_params_t *params, snd_pcm_t *pcm)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6021) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_BUFFER_SIZE, val, 0);
}

/**
 * \brief Restrict a configuration space to contain only one buffer size
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val buffer size in frames
 * \return 0 otherwise a negative error code if configuration space would become empty
 *
 * Wanted exact value is <,<=,> val following dir (-1,0,1)
 */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6035) - [public] snd_pcm_hw_param_set

```
■return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_BUFFER_SIZE, val, 0);
}

/**
 * \brief Restrict a configuration space with a minimum buffer size
 * \param pcm PCM handle
 * \param params Configuration space
 * \param val approximate minimum buffer size in frames (on return filled with actual minimum)
 * \return 0 otherwise a negative error code if configuration space would become empty
 */
int snd_pcm_hw_params_set_buffer_size_min(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_uframes_t
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6400) - [public] snd_check

```
■snd_check(PCM, "PCM not set up");
■return -EIO;
}
■__snd_pcm_lock(pcm); /* forced lock due to pcm field changes */
■snd_pcm_sw_params_current_no_lock(pcm, params);
■__snd_pcm_unlock(pcm);
■return 0;
}

/**
 * \brief Dump a software configuration
 * \param params Software configuration container
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6435) - [public] sizeof

```
■return sizeof(snd_pcm_sw_params_t);
}

/**
 * \brief allocate an invalid #snd_pcm_sw_params_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_pcm_sw_params_malloc(snd_pcm_sw_params_t **ptr)
{
    ■assert(ptr);
    ■*ptr = calloc(1, sizeof(snd_pcm_sw_params_t));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6503) - [public] snd_check

```
    ■■snd_check(PCM, "invalid start mode value %d", val);
    ■■return -EINVAL;
    ■}
    ■return 0;
}

#ifdef DOC_HIDDEN
link_warning(snd_pcm_sw_params_set_start_mode, "Warning: start_mode is deprecated, consider to use start_thr
#endif

/**
 * \brief (DEPRECATED) Get start mode from a software configuration container
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6551) - [public] snd_check

```
    ■■snd_check(PCM, "invalid xrun mode value %d", val);
    ■■return -EINVAL;
    ■}
    ■return 0;
}

#ifdef DOC_HIDDEN
link_warning(snd_pcm_sw_params_set_xrun_mode, "Warning: xrun_mode is deprecated, consider to use stop_thr
#endif

/**
 * \brief (DEPRECATED) Get xrun mode from a software configuration container
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6592) - [public] snd_check

```
    ■■snd_check(PCM, "invalid tstamp_mode value %d", val);
    ■■return -EINVAL;
    ■}
    ■params->tstamp_mode = val;
    ■return 0;
}

/**
 * \brief Get timestamp mode from a software configuration container
 * \param params Software configuration container
 * \param val Returned timestamp
 * \return 0 otherwise a negative error code
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6627) - [public] snd_check

```
    ■■snd_check(PCM, "invalid tstamp_type value %d", val);
    ■■return -EINVAL;
    ■}
    ■params->tstamp_type = val;
    ■return 0;
}

/**
 * \brief Get timestamp type from a software configuration container
 * \param params Software configuration container
 * \param val Returned timestamp type
 * \return 0 otherwise a negative error code
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6994) - [public] sizeof

```
    return sizeof(snd_pcm_status_t);
}

/**
 * \brief allocate an invalid #snd_pcm_status_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_pcm_status_malloc(snd_pcm_status_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_pcm_status_t));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7209) - [public] sizeof

```
    return sizeof(snd_pcm_info_t);
}

/**
 * \brief allocate an invalid #snd_pcm_info_t using standard malloc
 * \param ptr returned pointer
 * \return 0 on success otherwise negative error code
 */
int snd_pcm_info_malloc(snd_pcm_info_t **ptr)
{
    assert(ptr);
    *ptr = calloc(1, sizeof(snd_pcm_info_t));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7491) - [public] __snd_pcm_mmap_begin_generic

```
    return __snd_pcm_mmap_begin_generic(pcm, areas, offset, frames);
}
#endif

/**
 * \brief Application has completed the access to area requested with #snd_pcm_mmap_begin
 * \param pcm PCM handle
 * \param offset area offset in area steps (== frames)
 * \param frames area portion size in frames
 * \return count of transferred frames otherwise a negative error code
 *
 * You should pass this function the offset value that
 * // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7519) - [public] error

```
    error(avail);
    // at this point, we can transfer at least 'avail' frames

    // we want to process frames in chunks (period_size)
    if (avail < period_size)
        goto _skip;
    size = period_size;
    // it is possible that contiguous areas are smaller, thus we use a loop
    while (size > 0) {
        frames = size;

        err = snd_pcm_mmap_begin(pcm_handle, &areas, &offset, &frames);
        // ... (truncated, see source file for more)
    }
}
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 7527) - [public] while

// it is possible that contiguous areas are smaller, thus we use a loop

```
while (size > 0) {
    frames = size;

    err = snd_pcm_mmap_begin(pcm_handle, &areas, &offset, &frames);
    if (err < 0)
        error(err);
    // this function fills the areas from offset with count of frames
    generate_sine(areas, offset, frames, &phase);
    commitres = snd_pcm_mmap_commit(pcm_handle, offset, frames);
    if (commitres < 0 || commitres != frames)
        error(commitres >= 0 ? -EPIPE : commitres);

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7532) - [public] error

```
    error(err);
    // this function fills the areas from offset with count of frames
    generate_sine(areas, offset, frames, &phase);
    commitres = snd_pcm_mmap_commit(pcm_handle, offset, frames);
    if (commitres < 0 || commitres != frames)
        error(commitres >= 0 ? -EPIPE : commitres);

    size -= frames;
}
_skip:
\endcode
*
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7534) - [public] generate_sine

// this function fills the areas from offset with count of frames

```
generate_sine(areas, offset, frames, &phase);
commitres = snd_pcm_mmap_commit(pcm_handle, offset, frames);
if (commitres < 0 || commitres != frames)
    error(commitres >= 0 ? -EPIPE : commitres);

size -= frames;
}
_skip:
\endcode
*
* Look to the \link example_test_pcm Sine-wave generator \endlink example
* for more details about the generate_sine function.
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7537) - [public] error

```
    error(commitres >= 0 ? -EPIPE : commitres);

    size -= frames;
}
_skip:
\endcode
*
* Look to the \link example_test_pcm Sine-wave generator \endlink example
* for more details about the generate_sine function.
*
* The function is thread-safe when built with the proper option.
*/
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7580) - [public] snd_pcm_mmap_avail

```
    snd_pcm_mmap_avail(pcm));

return -EPIPE;
}
if (pcm->fast_ops->mmap_commit)
return pcm->fast_ops->mmap_commit(pcm->fast_op_arg, offset, frames);
else
return -ENOSYS;
}

int _snd_pcm_poll_descriptor(snd_pcm_t *pcm)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7645) - [public] switch

```
switch (state) {
case SND_PCM_STATE_PREPARED:
err = __snd_pcm_start(pcm);
if (err < 0)
goto _end;
break;
case SND_PCM_STATE_RUNNING:
err = __snd_pcm_hwsync(pcm);
if (err < 0)
goto _end;
break;
case SND_PCM_STATE_DRAINING:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7666) - [public] if

```
if (avail < 0) {
err = avail;
goto _end;
}
if (avail == 0) {
if (state == SND_PCM_STATE_DRAINING)
goto _end;
if (pcm->mode & SND_PCM_NONBLOCK) {
err = -EAGAIN;
goto _end;
}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7670) - [public] if

```
if (avail == 0) {
if (state == SND_PCM_STATE_DRAINING)
goto _end;
if (pcm->mode & SND_PCM_NONBLOCK) {
err = -EAGAIN;
goto _end;
}

err = __snd_pcm_wait_in_lock(pcm, SND_PCM_WAIT_IO);
if (err < 0)
break;
goto _again;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7673) - [public] if

```
    if (pcm->mode & SND_PCM_NONBLOCK) {
        err = -EAGAIN;
        goto _end;
    }

    err = __snd_pcm_wait_in_lock(pcm, SND_PCM_WAIT_IO);
    if (err < 0)
        break;
    goto _again;

}
frames = size;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7718) - [public] switch

```
switch (state) {
case SND_PCM_STATE_PREPARED:
case SND_PCM_STATE_PAUSED:
    break;
case SND_PCM_STATE_RUNNING:
    err = __snd_pcm_hwsync(pcm);
    if (err < 0)
        goto _end;
    break;
default:
    err = pcm_state_to_error(state);
    if (!err)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7734) - [public] if

```
if (avail < 0) {
    err = avail;
    goto _end;
}
if (state == SND_PCM_STATE_RUNNING &&
    size > (snd_pcm_uframes_t)avail) {
    if (snd_pcm_may_wait_for_avail_min(pcm, avail)) {
        if (pcm->mode & SND_PCM_NONBLOCK) {
            err = -EAGAIN;
            goto _end;
        }

        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7740) - [public] if

```
if (snd_pcm_may_wait_for_avail_min(pcm, avail)) {
    if (pcm->mode & SND_PCM_NONBLOCK) {
        err = -EAGAIN;
        goto _end;
    }

    err = snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_IO);
    if (err < 0)
        break;
    goto _again;
}
/* the snd_pcm_may_wait_for_avail_min may check against the
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7741) - [public] if

```
    if (pcm->mode & SND_PCM_NONBLOCK) {
        err = -EAGAIN;
        goto _end;
    }

    err = snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_IO);
    if (err < 0)
        break;
    goto _again;
}
/* the snd_pcm_may_wait_for_avail_min may check against the
 * updated hw.ptr (slaves), get the avail again here
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7755) - [public] if

```
    if (avail < 0) {
        err = avail;
        goto _end;
    }
}
frames = size;
if (frames > (snd_pcm_uframes_t) avail)
    frames = avail;
if (! frames)
    break;
err = func(pcm, areas, offset, frames);
if (err < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7769) - [public] if

```
    if (state == SND_PCM_STATE_PREPARED) {
        snd_pcm_sframes_t hw_avail = pcm->buffer_size - avail;
        hw_avail += frames;
        /* some plugins might automatically start the stream */
        state = __snd_pcm_state(pcm);
        if (state == SND_PCM_STATE_PREPARED &&
            hw_avail >= 0 &&
            (snd_pcm_uframes_t) hw_avail >= pcm->start_threshold) {
            err = __snd_pcm_start(pcm);
            if (err < 0)
                goto _end;
        }
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7838) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "Invalid slave definition");
        return -EINVAL;
    }
    to_free = 1;
}
if (snd_config_get_type(conf) != SND_CONFIG_TYPE_COMPOUND) {
    snd_error(PCM, "Invalid slave definition");
    err = -EINVAL;
    goto _err;
}
va_start(args, count);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7839) - [public] snd_error

```
    ■■■snd_error(PCM, "Invalid slave definition");
    ■■■return -EINVAL;
    ■■}
    ■■to_free = 1;
    ■■}
    ■■if (snd_config_get_type(conf) != SND_CONFIG_TYPE_COMPOUND) {
    ■■snd_error(PCM, "Invalid slave definition");
    ■■err = -EINVAL;
    ■■goto _err;
    ■■}
    ■■va_start(args, count);
    ■■for (k = 0; k < count; ++k) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7845) - [public] snd_error

```
    ■■snd_error(PCM, "Invalid slave definition");
    ■■err = -EINVAL;
    ■■goto _err;
    ■■}
    ■■va_start(args, count);
    ■■for (k = 0; k < count; ++k) {
    ■■fields[k].index = va_arg(args, int);
    ■■fields[k].flags = va_arg(args, int);
    ■■fields[k].ptr = va_arg(args, void *);
    ■■fields[k].present = 0;
    ■■}
    ■■va_end(args);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7864) - [public] if

```
    ■■if (strcmp(id, "pcm") == 0) {
    ■■if (pcm_conf != NULL)
    ■■■■snd_config_delete(pcm_conf);
    ■■if ((err = snd_config_copy(&pcm_conf, n)) < 0)
    ■■■■goto _err;
    ■■continue;
    ■■}
    ■■for (k = 0; k < count; ++k) {
    ■■unsigned int idx = fields[k].index;
    ■■long v;
    ■■assert(idx < SND_PCM_HW_PARAM_LAST_INTERVAL);
    ■■assert(names[idx]);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7866) - [public] snd_config_delete

```
    ■■■■snd_config_delete(pcm_conf);
    ■■if ((err = snd_config_copy(&pcm_conf, n)) < 0)
    ■■■■goto _err;
    ■■continue;
    ■■}
    ■■for (k = 0; k < count; ++k) {
    ■■unsigned int idx = fields[k].index;
    ■■long v;
    ■■assert(idx < SND_PCM_HW_PARAM_LAST_INTERVAL);
    ■■assert(names[idx]);
    ■■if (strcmp(id, names[idx]) != 0)
    ■■■■continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7874) - [public] assert

```
    assert(idx < SND_PCM_HW_PARAM_LAST_INTERVAL);
    assert(names[idx]);
    if (strcmp(id, names[idx]) != 0)
        continue;
    switch (idx) {
    case SND_PCM_HW_PARAM_FORMAT:
    {
        snd_pcm_format_t f;
        err = snd_config_get_string(n, &str);
        if (err < 0) {
            _invalid:
            snd_error(PCM, "invalid type for %s", id);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7875) - [public] assert

```
    assert(names[idx]);
    if (strcmp(id, names[idx]) != 0)
        continue;
    switch (idx) {
    case SND_PCM_HW_PARAM_FORMAT:
    {
        snd_pcm_format_t f;
        err = snd_config_get_string(n, &str);
        if (err < 0) {
            _invalid:
            snd_error(PCM, "invalid type for %s", id);
            goto _err;
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7878) - [public] switch

```
    switch (idx) {
    case SND_PCM_HW_PARAM_FORMAT:
    {
        snd_pcm_format_t f;
        err = snd_config_get_string(n, &str);
        if (err < 0) {
            _invalid:
            snd_error(PCM, "invalid type for %s", id);
            goto _err;
        }
        if ((fields[k].flags & SCONF_UNCHANGED) &&
            strcasecmp(str, "unchanged") == 0) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7883) - [public] if

```
    if (err < 0) {
        _invalid:
        snd_error(PCM, "invalid type for %s", id);
        goto _err;
    }
    if ((fields[k].flags & SCONF_UNCHANGED) &&
        strcasecmp(str, "unchanged") == 0) {
        *(snd_pcm_format_t*)fields[k].ptr = (snd_pcm_format_t) -2;
        break;
    }
    f = snd_pcm_format_value(str);
    if (f == SND_PCM_FORMAT_UNKNOWN) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7885) - [public] snd_error

```
        snd_error(PCM, "invalid type for %s", id);
        goto _err;
    }
    if ((fields[k].flags & SCONF_UNCHANGED) &&
        strcasecmp(str, "unchanged") == 0) {
        *(snd_pcm_format_t*)fields[k].ptr = (snd_pcm_format_t) -2;
        break;
    }
    f = snd_pcm_format_value(str);
    if (f == SND_PCM_FORMAT_UNKNOWN) {
        snd_error(PCM, "unknown format %s", str);
        err = -EINVAL;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7889) - [public] strcasecmp

```
        strcasecmp(str, "unchanged") == 0) {
        *(snd_pcm_format_t*)fields[k].ptr = (snd_pcm_format_t) -2;
        break;
    }
    f = snd_pcm_format_value(str);
    if (f == SND_PCM_FORMAT_UNKNOWN) {
        snd_error(PCM, "unknown format %s", str);
        err = -EINVAL;
        goto _err;
    }
    *(snd_pcm_format_t*)fields[k].ptr = f;
    break;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7894) - [public] if

```
    if (f == SND_PCM_FORMAT_UNKNOWN) {
        snd_error(PCM, "unknown format %s", str);
        err = -EINVAL;
        goto _err;
    }
    *(snd_pcm_format_t*)fields[k].ptr = f;
    break;
}
default:
    if ((fields[k].flags & SCONF_UNCHANGED)) {
        err = snd_config_get_string(n, &str);
        if (err >= 0 &&
            // ... (truncated, see source file for more)
        )
            continue;
    }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7895) - [public] snd_error

```
        snd_error(PCM, "unknown format %s", str);
        err = -EINVAL;
        goto _err;
    }
    *(snd_pcm_format_t*)fields[k].ptr = f;
    break;
}
default:
    if ((fields[k].flags & SCONF_UNCHANGED)) {
        err = snd_config_get_string(n, &str);
        if (err >= 0 &&
            strcasecmp(str, "unchanged") == 0) {
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7903) - [public] if

```
        if ((fields[k].flags & SCONF_UNCHANGED)) {
            err = snd_config_get_string(n, &str);
            if (err >= 0 &&
                strcmp(str, "unchanged") == 0) {
                *(int*)fields[k].ptr = -2;
                break;
            }
        }
        err = snd_config_get_integer(n, &v);
        if (err < 0)
            goto _invalid;
        *(int*)fields[k].ptr = v;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7906) - [public] strcmp

```
        strcmp(str, "unchanged") == 0) {
            *(int*)fields[k].ptr = -2;
            break;
        }
    }
    err = snd_config_get_integer(n, &v);
    if (err < 0)
        goto _invalid;
    *(int*)fields[k].ptr = v;
    break;
}
fields[k].present = 1;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7922) - [public] snd_error

```
    snd_error(PCM, "Unknown field %s", id);
    err = -EINVAL;
    goto _err;
}
if (!pcm_conf) {
    snd_error(PCM, "missing field pcm");
    err = -EINVAL;
    goto _err;
}
for (k = 0; k < count; ++k) {
    if ((fields[k].flags & SCONF_MANDATORY) && !fields[k].present) {
        snd_error(PCM, "missing field %s", names[fields[k].index]);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7927) - [public] snd_error

```
    snd_error(PCM, "missing field pcm");
    err = -EINVAL;
    goto _err;
}
for (k = 0; k < count; ++k) {
    if ((fields[k].flags & SCONF_MANDATORY) && !fields[k].present) {
        snd_error(PCM, "missing field %s", names[fields[k].index]);
        err = -EINVAL;
        goto _err;
    }
}
*_pcm_conf = pcm_conf;
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 7932) - [public] if

```
    if ((fields[k].flags & SCONF_MANDATORY) && !fields[k].present) {
        snd_error(PCM, "missing field %s", names[fields[k].index]);
        err = -EINVAL;
        goto _err;
    }
    *_pcm_conf = pcm_conf;
    pcm_conf = NULL;
    err = 0;
_err:
    if (pcm_conf)
        snd_config_delete(pcm_conf);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7933) - [public] snd_error

```
    snd_error(PCM, "missing field %s", names[fields[k].index]);
    err = -EINVAL;
    goto _err;
}
*_pcm_conf = pcm_conf;
pcm_conf = NULL;
err = 0;
_err:
    if (pcm_conf)
        snd_config_delete(pcm_conf);
    if (to_free)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7943) - [public] snd_config_delete

```
    snd_config_delete(pcm_conf);
    if (to_free)
        snd_config_delete(conf);
    return err;
}
```

```
static void snd_pcm_set_ptr(snd_pcm_t *pcm, snd_pcm_rbp_ptr_t *rbp_ptr,
    volatile snd_pcm_uframes_t *hw_ptr, int fd, off_t offset)
{
    rbp_ptr->master = NULL; /* I'm master */
    rbp_ptr->ptr = hw_ptr;
    rbp_ptr->fd = fd;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7945) - [public] snd_config_delete

```
    snd_config_delete(conf);
    return err;
}
```

```
static void snd_pcm_set_ptr(snd_pcm_t *pcm, snd_pcm_rbp_ptr_t *rbp_ptr,
    volatile snd_pcm_uframes_t *hw_ptr, int fd, off_t offset)
{
    rbp_ptr->master = NULL; /* I'm master */
    rbp_ptr->ptr = hw_ptr;
    rbp_ptr->fd = fd;
    rbp_ptr->offset = offset;
    if (rbp_ptr->changed)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7982) - [public] if

```
    if (a[idx] == NULL) {
        a[idx] = pcm;
        goto __found_free_place;
    }
    a = realloc(a, sizeof(snd_pcm_t *) * (slave_rbp_ptr->link_dst_count + 1));
    if (a == NULL) {
        pcm_rbp_ptr->ptr = NULL;
        pcm_rbp_ptr->fd = -1;
        pcm_rbp_ptr->offset = 0UL;
        return;
    }
    a[slave_rbp_ptr->link_dst_count++] = pcm;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8012) - [public] if

```
    if (a[idx] == pcm) {
        a[idx] = NULL;
        goto __found;
    }
    /* assert(0); */
    return;

__found:
    pcm_rbp_ptr->master = NULL;
    pcm_rbp_ptr->ptr = NULL;
    pcm_rbp_ptr->fd = -1;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8068) - [public] default_symbol_version

```
    default_symbol_version(__##name, name, new); \
    symbol_version(__old_##name, name, what);

#else

#define OBSOLETE1(name, what, new) \
    use_default_symbol_version(__##name, name, new);

#endif /* USE_VERSIONED_SYMBOLS */

#define __P_OLD_GET(pfx, name, val_type, ret_type) \
EXPORT_SYMBOL ret_type pfx##name(const snd_pcm_hw_params_t *params) \
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8069) - [public] symbol_version

```
    symbol_version(__old_##name, name, what);

#else

#define OBSOLETE1(name, what, new) \
    use_default_symbol_version(__##name, name, new);

#endif /* USE_VERSIONED_SYMBOLS */

#define __P_OLD_GET(pfx, name, val_type, ret_type) \
EXPORT_SYMBOL ret_type pfx##name(const snd_pcm_hw_params_t *params) \
{ \
    // ... (truncated, see source file for more)
```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8074) - [public] use_default_symbol_version
    use_default_symbol_version(__##name, name, new);

#endif /* USE_VERSIONED_SYMBOLS */

#define __P_OLD_GET(pfx, name, val_type, ret_type) \
EXPORT_SYMBOL ret_type pfx##name(const snd_pcm_hw_params_t *params) \
{ \
    val_type val; \
    if (INTERNAL(name)(params, &val) < 0) \
        return 0; \
    return (ret_type)val; \
}

    // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm.c (line 8337) - [public] free
    free(*p);
    free(maps);
}

/**
 * \brief Get the current channel map
 * \param pcm PCM instance
 * \return the current channel map, or NULL if error
 *
 * Note: the caller is requested to release the returned value via free()
 */
snd_pcm_chmap_t *snd_pcm_get_chmap(snd_pcm_t *pcm)
    // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm.c (line 8499) - [public] if
    if (i > 0) {
        len += snprintf(buf + len, maxlen - len, " ");
        if (len >= maxlen)
            return -ENOMEM;
    }
    if (map->pos[i] & SND_CHMAP_DRIVER_SPEC)
        len += snprintf(buf + len, maxlen - len, "%d", p);
    else {
        const char *name = chmap_names[p];
        if (name)
            len += snprintf(buf + len, maxlen - len,
                "%s", name);
        // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm.c (line 8517) - [public] if
    if (map->pos[i] & SND_CHMAP_PHASE_INVERSE) {
        len += snprintf(buf + len, maxlen - len, "[INV]");
        if (len >= maxlen)
            return -ENOMEM;
    }
    return len;
}

static int str_to_chmap(const char *str, int len)
{
    int val;
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8548) - [public] assert

```
    assert(chmap_names[val]);
    slen = strlen(chmap_names[val]);
    if (slen > len)
        continue;
    if (!strncasecmp(str, chmap_names[val], slen) &&
        !isalpha(str[slen])) {
        str += slen;
        break;
    }
}
if (val > SND_CHMAP_LAST)
    return -1;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8573) - [public] str_to_chmap

```
return str_to_chmap(str, strlen(str));
}

/**
 * \brief Convert from string to channel map
 * \param str The string to parse
 * \return The channel map
 *
 * Note: the caller is requested to release the returned value via free()
 */
snd_pcm_chmap_t *snd_pcm_chmap_parse_string(const char *str)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8604) - [public] if

```
if (*str == '[') {
    if (!strcmp(str, "[INV]", 5)) {
        val |= SND_CHMAP_PHASE_INVERSE;
        str += 5;
    }
}
tmp_map[ch] = val;
ch++;
for (; *str && !isalnum(*str); str++)
    ;
if (!*str)
    break;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8605) - [public] if

```
if (!strcmp(str, "[INV]", 5)) {
    val |= SND_CHMAP_PHASE_INVERSE;
    str += 5;
}
tmp_map[ch] = val;
ch++;
for (; *str && !isalnum(*str); str++)
    ;
if (!*str)
    break;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8649) - [public] free

```
    free(maps);
    return NULL;
}
return maps;
}

/* make a copy of chmap */
snd_pcm_chmap_t *_snd_pcm_copy_chmap(const snd_pcm_chmap_t *src)
{
    snd_pcm_chmap_t *map;

    map = malloc((src->channels + 1) * sizeof(int));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8683) - [public] if

```
    if (!maps[i]) {
        snd_pcm_free_chmaps(maps);
        return NULL;
    }
    memcpy(maps[i], src[i], (src[i]->map.channels + 2) * sizeof(int));
}
return maps;
}

/* select the channel map with the current PCM channels and make a copy */
snd_pcm_chmap_t *
_snd_pcm_choose_fixed_chmap(snd_pcm_t *pcm, snd_pcm_chmap_query_t * const *maps)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8684) - [public] snd_pcm_free_chmaps

```
    snd_pcm_free_chmaps(maps);
    return NULL;
}
    memcpy(maps[i], src[i], (src[i]->map.channels + 2) * sizeof(int));
}
return maps;
}

/* select the channel map with the current PCM channels and make a copy */
snd_pcm_chmap_t *
_snd_pcm_choose_fixed_chmap(snd_pcm_t *pcm, snd_pcm_chmap_query_t * const *maps)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8687) - [public] memcpy

```
    memcpy(maps[i], src[i], (src[i]->map.channels + 2) * sizeof(int));
}
return maps;
}

/* select the channel map with the current PCM channels and make a copy */
snd_pcm_chmap_t *
_snd_pcm_choose_fixed_chmap(snd_pcm_t *pcm, snd_pcm_chmap_query_t * const *maps)
{
    snd_pcm_chmap_query_t * const *p;

    for (p = maps; *p; p++) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8700) - [public] _snd_pcm_copy_chmap

```
    return _snd_pcm_copy_chmap(&(*p)->map);
}
return NULL;
}

/* make chmap_query array from the config tree;
 * conf must be a compound (array)
 */
snd_pcm_chmap_query_t **
_snd_pcm_parse_config_chmaps(snd_config_t *conf)
{
    snd_pcm_chmap_t *chmap;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8738) - [public] if

```
    if (_copy_to_fixed_query_map(maps + nums, chmap)) {
        free(chmap);
        goto error;
    }
    free(chmap);
    nums++;
}
return maps;

error:
snd_pcm_free_chmaps(maps);
return NULL;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8739) - [public] free

```
    free(chmap);
    goto error;
}
free(chmap);
nums++;
}
return maps;

error:
snd_pcm_free_chmaps(maps);
return NULL;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8742) - [public] free

```
    free(chmap);
    nums++;
}
return maps;

error:
snd_pcm_free_chmaps(maps);
return NULL;
}
#endif /* DOC_HIDDEN */

/*
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8787) - [public] snd_error

```
    snd_error(PCM, "%s occurred", s);
    err = snd_pcm_prepare(pcm);
    if (err < 0) {
        snd_error(PCM, "cannot recovery from %s, prepare failed: %s", s, snd_strerror(err));
        return err;
    }
    return 0;
}
if (err == -ESTRPIPE) {
    while ((err = snd_pcm_resume(pcm)) == -EAGAIN)
        /* wait until suspend flag is released */
        (void)poll(NULL, 0, 1000);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8789) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "cannot recovery from %s, prepare failed: %s", s, snd_strerror(err));
        return err;
    }
    return 0;
}
if (err == -ESTRPIPE) {
    while ((err = snd_pcm_resume(pcm)) == -EAGAIN)
        /* wait until suspend flag is released */
        (void)poll(NULL, 0, 1000);
    if (err < 0) {
        err = snd_pcm_prepare(pcm);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8790) - [public] snd_error

```
    snd_error(PCM, "cannot recovery from %s, prepare failed: %s", s, snd_strerror(err));
    return err;
}
return 0;
}
if (err == -ESTRPIPE) {
    while ((err = snd_pcm_resume(pcm)) == -EAGAIN)
        /* wait until suspend flag is released */
        (void)poll(NULL, 0, 1000);
    if (err < 0) {
        err = snd_pcm_prepare(pcm);
        if (err < 0) {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8799) - [public] if

```
    if (err < 0) {
        err = snd_pcm_prepare(pcm);
        if (err < 0) {
            snd_error(PCM, "cannot recovery from suspend, prepare failed: %s", snd_strerror(err));
            return err;
        }
    }
    return 0;
}
return err;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8801) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "cannot recovery from suspend, prepare failed: %s", snd_strerror(err));
        return err;
    }
    return 0;
}
return err;
}

/**
 * \brief Set the hardware and software parameters in a simple way
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8802) - [public] snd_error

```
    snd_error(PCM, "cannot recovery from suspend, prepare failed: %s", snd_strerror(err));
    return err;
}
return 0;
}
return err;
}

/**
 * \brief Set the hardware and software parameters in a simple way
 * \param pcm PCM handle
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8850) - [public] snd_strerror

```
    s, snd_strerror(err));

    return err;
}
/* set the selected read/write format */
err = snd_pcm_hw_params_set_access(pcm, &params, access);
if (err < 0) {
    snd_error(PCM, "Access type not available for %s: %s",
        s, snd_strerror(err));

    return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8858) - [public] snd_strerror

```
    s, snd_strerror(err));

    return err;
}
/* set the sample format */
err = snd_pcm_hw_params_set_format(pcm, &params, format);
if (err < 0) {
    snd_error(PCM, "Sample format not available for %s: %s",
        s, snd_strerror(err));

    return err;
}
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm.c (line 8866) - [public] snd_strerror

```
    s, snd_strerror(err));

    return err;
}
/* set the count of channels */
err = snd_pcm_hw_params_set_channels(pcm, &params, channels);
if (err < 0) {
    snd_error(PCM, "Channels count (%i) not available for %s: %s",
        channels, s, snd_strerror(err));

    return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8874) - [public] snd_strerror

```
    channels, s, snd_strerror(err));

    return err;
}
/* set the stream rate */
rrate = rate;
err = INTERNAL(snd_pcm_hw_params_set_rate_near)(pcm, &params, &rrate,
    0);
if (err < 0) {
    snd_error(PCM, "Rate %iHz not available for playback: %s",
        rate, snd_strerror(err));

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8884) - [public] snd_strerror

```
    rate, snd_strerror(err));

    return err;
}
if (rrate != rate) {
    snd_error(PCM, "Rate doesn't match (requested %iHz, get %iHz)",
        rate, rrate);

    return -EINVAL;
}
/* set the buffer time */
params_saved = params;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8905) - [public] if

```
if (err < 0) {
    snd_error(PCM, "Unable to set period time %i for %s: %s",
        period_time, s, snd_strerror(err));

    return err;
}
err = INTERNAL(snd_pcm_hw_params_get_period_size>(&params,
    &period_size, NULL);
if (err < 0) {
    snd_error(PCM, "Unable to get period size for %s: %s",
        s, snd_strerror(err));

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8907) - [public] snd_strerror

```
        period_time, s, snd_strerror(err));

    return err;
}
err = INTERNAL(snd_pcm_hw_params_get_period_size>(&params,
        &period_size, NULL));
if (err < 0) {
    snd_error(PCM, "Unable to get period size for %s: %s",
        s, snd_strerror(err));

    return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8913) - [public] if

```
if (err < 0) {
    snd_error(PCM, "Unable to get period size for %s: %s",
        s, snd_strerror(err));

    return err;
}
buffer_size = period_size * 4;
err = INTERNAL(snd_pcm_hw_params_set_buffer_size_near)(pcm,
        &params, &buffer_size);
if (err < 0) {
    snd_error(PCM, "Unable to set buffer size %lu %s: %s",
        buffer_size, s, snd_strerror(err));
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8915) - [public] snd_strerror

```
        s, snd_strerror(err));

    return err;
}
buffer_size = period_size * 4;
err = INTERNAL(snd_pcm_hw_params_set_buffer_size_near)(pcm,
        &params, &buffer_size);
if (err < 0) {
    snd_error(PCM, "Unable to set buffer size %lu %s: %s",
        buffer_size, s, snd_strerror(err));

    return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8922) - [public] if

```
if (err < 0) {
    snd_error(PCM, "Unable to set buffer size %lu %s: %s",
        buffer_size, s, snd_strerror(err));

    return err;
}
err = INTERNAL(snd_pcm_hw_params_get_buffer_size>(&params,
        &buffer_size);
if (err < 0) {
    snd_error(PCM, "Unable to get buffer size for %s: %s",
        s, snd_strerror(err));

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8924) - [public] snd_strerror

```
        &buffer_size, s, snd_strerror(err));

    return err;
}
err = INTERNAL(snd_pcm_hw_params_get_buffer_size)(&params,
        &buffer_size);
if (err < 0) {
    snd_error(PCM, "Unable to get buffer size for %s: %s",
        s, snd_strerror(err));

    return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8930) - [public] if

```
if (err < 0) {
    snd_error(PCM, "Unable to get buffer size for %s: %s",
        s, snd_strerror(err));

    return err;
} else {
    /* standard configuration buffer_time -> periods */
    err = INTERNAL(snd_pcm_hw_params_get_buffer_size)(&params,
        &buffer_size);
    if (err < 0) {
        snd_error(PCM, "Unable to get buffer size for %s: %s",
            // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8932) - [public] snd_strerror

```
        s, snd_strerror(err));

    return err;
} else {
    /* standard configuration buffer_time -> periods */
    err = INTERNAL(snd_pcm_hw_params_get_buffer_size)(&params,
        &buffer_size);
    if (err < 0) {
        snd_error(PCM, "Unable to get buffer size for %s: %s",
            s, snd_strerror(err));

        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8940) - [public] if

```
if (err < 0) {
    snd_error(PCM, "Unable to get buffer size for %s: %s",
        s, snd_strerror(err));

    return err;
}
err = INTERNAL(snd_pcm_hw_params_get_buffer_time)(&params,
        &latency, NULL);
if (err < 0) {
    snd_error(PCM, "Unable to get buffer time (latency) for %s: %s",
        s, snd_strerror(err));

    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8942) - [public] snd_strerror

```
        s, snd_strerror(err));

    return err;
}
err = INTERNAL(snd_pcm_hw_params_get_buffer_time>(&params,
        &latency, NULL);
if (err < 0) {
    snd_error(PCM, "Unable to get buffer time (latency) for %s: %s",
        s, snd_strerror(err));

    return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8948) - [public] if

```
if (err < 0) {
    snd_error(PCM, "Unable to get buffer time (latency) for %s: %s",
        s, snd_strerror(err));

    return err;
}
/* set the period time */
period_time = latency / 4;
err = INTERNAL(snd_pcm_hw_params_set_period_time_near)(pcm,
        &params, &period_time, NULL);
if (err < 0) {
    snd_error(PCM, "Unable to set period time %i for %s: %s",
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8950) - [public] snd_strerror

```
        s, snd_strerror(err));

    return err;
}
/* set the period time */
period_time = latency / 4;
err = INTERNAL(snd_pcm_hw_params_set_period_time_near)(pcm,
        &params, &period_time, NULL);
if (err < 0) {
    snd_error(PCM, "Unable to set period time %i for %s: %s",
        period_time, s, snd_strerror(err));

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8958) - [public] if

```
if (err < 0) {
    snd_error(PCM, "Unable to set period time %i for %s: %s",
        period_time, s, snd_strerror(err));

    return err;
}
err = INTERNAL(snd_pcm_hw_params_get_period_size>(&params,
        &period_size, NULL);
if (err < 0) {
    snd_error(PCM, "Unable to get period size for %s: %s",
        s, snd_strerror(err));

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8960) - [public] snd_strerror

```
        period_time, s, snd_strerror(err));

    return err;
}
err = INTERNAL(snd_pcm_hw_params_get_period_size)(&params,
            &period_size, NULL);
if (err < 0) {
    snd_error(PCM, "Unable to get period size for %s: %s",
        s, snd_strerror(err));

    return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8966) - [public] if

```
if (err < 0) {
    snd_error(PCM, "Unable to get period size for %s: %s",
        s, snd_strerror(err));

    return err;
}
/* write the parameters to device */
err = snd_pcm_hw_params(pcm, &params);
if (err < 0) {
    snd_error(PCM, "Unable to set hw params for %s: %s",
        s, snd_strerror(err));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8968) - [public] snd_strerror

```
        s, snd_strerror(err));

    return err;
}
/* write the parameters to device */
err = snd_pcm_hw_params(pcm, &params);
if (err < 0) {
    snd_error(PCM, "Unable to set hw params for %s: %s",
        s, snd_strerror(err));

    return err;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8977) - [public] snd_strerror

```
        s, snd_strerror(err));

    return err;
}
/* get the current swparams */
err = snd_pcm_sw_params_current(pcm, &swparams);
if (err < 0) {
    snd_error(PCM, "Unable to determine current swparams for %s: %s",
        s, snd_strerror(err));

    return err;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8986) - [public] snd_strerror

```
    s, snd_strerror(err));

    return err;
}
/*
 * start the transfer when the buffer is almost full:
 * (buffer_size / avail_min) * avail_min
 */
err = snd_pcm_sw_params_set_start_threshold(pcm, &swparams,
      (buffer_size / period_size) * period_size);
if (err < 0) {
    snd_error(PCM, "Unable to set start threshold mode for %s: %s",
        // ... (truncated, see source file for more)
    );
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8998) - [public] snd_strerror

```
    s, snd_strerror(err));

    return err;
}
/*
 * allow the transfer when at least period_size samples can be
 * processed
 */
err = snd_pcm_sw_params_set_avail_min(pcm, &swparams, period_size);
if (err < 0) {
    snd_error(PCM, "Unable to set avail min for %s: %s",
        s, snd_strerror(err));
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 9009) - [public] snd_strerror

```
    s, snd_strerror(err));

    return err;
}
/* write the parameters to the playback device */
err = snd_pcm_sw_params(pcm, &swparams);
if (err < 0) {
    snd_error(PCM, "Unable to set sw params for %s: %s",
        s, snd_strerror(err));
}

return err;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 9017) - [public] snd_strerror

```
    s, snd_strerror(err));

    return err;
}
return 0;
}

/**
 * \brief Get the transfer size parameters in a simple way
 * \param pcm PCM handle
 * \param buffer_size PCM ring buffer size in frames
 * \param period_size PCM period size in frames
 * // ... (truncated, see source file for more)
 */
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 139) - [public] if

```
    if (diff >= step) {
        adjust_idx |= i;
        diff -= step;
        pred_diff += step;
    }

    /* Update and clamp previous predicted value */
    state->pred_val += sign ? -pred_diff : pred_diff;

    if (state->pred_val > 32767) {
        state->pred_val = 32767;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 189) - [public] if

```
    if (code & i) {
        pred_diff += step;
    }
    state->pred_val += (sign) ? -pred_diff : pred_diff;

    /* Clamp output value */
    if (state->pred_val > 32767) {
        state->pred_val = 32767;
    } else if (state->pred_val < -32768) {
        state->pred_val = -32768;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 244) - [public] while

```
    while (frames1-- > 0) {
        int16_t sample;
        unsigned char v;
        if (srcbit)
            v = *src & 0x0f;
        else
            v = (*src >> 4) & 0x0f;
        sample = adpcm_decoder(v, states);
        goto *put;
#define PUT16_END after
#include "plugin_ops.h"
#undef PUT16_END
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 259) - [public] if

```
    if (srcbit == 8) {
        src++;
        srcbit = 0;
    }
    dst += dst_step;
}

void snd_pcm_adpcm_encode(const snd_pcm_channel_area_t *dst_areas,
    snd_pcm_uframes_t dst_offset,
    const snd_pcm_channel_area_t *src_areas,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 298) - [public] while

```
    while (frames1-- > 0) {
        int v;
        goto *get;
#define GET16_END after
#include "plugin_ops.h"
#undef GET16_END
    after:
        v = adpcm_encoder(sample, states);
        if (dstbit)
            *dst = (*dst & 0xf0) | v;
        else
            *dst = (*dst & 0x0f) | (v << 4);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 313) - [public] if

```
        if (dstbit == 8) {
            dst++;
            dstbit = 0;
        }
    }
}
```

#endif

```
static int snd_pcm_adpcm_hw_refine_cprepare(snd_pcm_t *pcm, snd_pcm_hw_params_t *params)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 425) - [public] if

```
    if (adpcm->sformat == SND_PCM_FORMAT_IMA_ADPCM) {
        adpcm->getput_idx = snd_pcm_linear_get_index(format, SND_PCM_FORMAT_S16);
        adpcm->func = snd_pcm_adpcm_encode;
    } else {
        adpcm->getput_idx = snd_pcm_linear_put_index(SND_PCM_FORMAT_S16, adpcm->sformat);
        adpcm->func = snd_pcm_adpcm_decode;
    }
} else {
    if (adpcm->sformat == SND_PCM_FORMAT_IMA_ADPCM) {
        adpcm->getput_idx = snd_pcm_linear_put_index(SND_PCM_FORMAT_S16, format);
        adpcm->func = snd_pcm_adpcm_decode;
    } else {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 433) - [public] if

```
    if (adpcm->sformat == SND_PCM_FORMAT_IMA_ADPCM) {
        adpcm->getput_idx = snd_pcm_linear_put_index(SND_PCM_FORMAT_S16, format);
        adpcm->func = snd_pcm_adpcm_decode;
    } else {
        adpcm->getput_idx = snd_pcm_linear_get_index(adpcm->sformat, SND_PCM_FORMAT_S16);
        adpcm->func = snd_pcm_adpcm_encode;
    }
}
assert(!adpcm->states);
adpcm->states = malloc(adpcm->plug.gen.slave->channels * sizeof(*adpcm->states));
if (adpcm->states == NULL)
    return -ENOMEM;
// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 453) - [public] snd_pcm_hw_free

```
■return snd_pcm_hw_free(adpcm->plug.gen.slave);
}

static int snd_pcm_adpcm_init(snd_pcm_t *pcm)
{
■snd_pcm_adpcm_t *adpcm = pcm->private_data;
■unsigned int k;
■for (k = 0; k < pcm->channels; ++k) {
■■adpcm->states[k].pred_val = 0;
■■adpcm->states[k].step_idx = 0;
■}
■return 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 511) - [public] snd_pcm_format_name

```
■■snd_pcm_format_name(adpcm->sformat));
■if (pcm->setup) {
■■snd_output_printf(out, "Its setup is:\n");
■■snd_pcm_dump_setup(pcm, out);
■}
■snd_output_printf(out, "Slave: ");
■snd_pcm_dump(adpcm->plug.gen.slave, out);
}

static const snd_pcm_ops_t snd_pcm_adpcm_ops = {
■.close = snd_pcm_generic_close,
■.info = snd_pcm_generic_info,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 513) - [public] snd_output_printf

```
■■snd_output_printf(out, "Its setup is:\n");
■■snd_pcm_dump_setup(pcm, out);
■}
■snd_output_printf(out, "Slave: ");
■snd_pcm_dump(adpcm->plug.gen.slave, out);
}

static const snd_pcm_ops_t snd_pcm_adpcm_ops = {
■.close = snd_pcm_generic_close,
■.info = snd_pcm_generic_info,
■.hw_refine = snd_pcm_adpcm_hw_refine,
■.hw_params = snd_pcm_adpcm_hw_params,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 514) - [public] snd_pcm_dump_setup

```
■■snd_pcm_dump_setup(pcm, out);
■}
■snd_output_printf(out, "Slave: ");
■snd_pcm_dump(adpcm->plug.gen.slave, out);
}

static const snd_pcm_ops_t snd_pcm_adpcm_ops = {
■.close = snd_pcm_generic_close,
■.info = snd_pcm_generic_info,
■.hw_refine = snd_pcm_adpcm_hw_refine,
■.hw_params = snd_pcm_adpcm_hw_params,
■.hw_free = snd_pcm_adpcm_hw_free,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 573) - [public] free

```
    free(adpcm);
    return err;
}
pcm->ops = &snd_pcm_adpcm_ops;
pcm->fast_ops = &snd_pcm_plugin_fast_ops;
pcm->private_data = adpcm;
pcm->poll_fd = slave->poll_fd;
pcm->poll_events = slave->poll_events;
pcm->tstamp_type = slave->tstamp_type;
snd_pcm_set_hw_ptr(pcm, &adpcm->plug.hw_ptr, -1, 0);
snd_pcm_set_appl_ptr(pcm, &adpcm->plug.appl_ptr, -1, 0);
*pcmp = pcm;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 649) - [public] if

```
    if (strcmp(id, "slave") == 0) {
        slave = n;
        continue;
    }
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (!slave) {
    snd_error(PCM, "slave is not defined");
    return -EINVAL;
}
err = snd_pcm_slave_conf(root, slave, &sconf, 1,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 653) - [public] snd_error

```
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (!slave) {
    snd_error(PCM, "slave is not defined");
    return -EINVAL;
}
err = snd_pcm_slave_conf(root, slave, &sconf, 1,
    SND_PCM_HW_PARAM_FORMAT, SCONF_MANDATORY, &sformat);
if (err < 0)
    return err;
if (snd_pcm_format_linear(sformat) != 1 &&
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 657) - [public] snd_error

```
    snd_error(PCM, "slave is not defined");
    return -EINVAL;
}
err = snd_pcm_slave_conf(root, slave, &sconf, 1,
    SND_PCM_HW_PARAM_FORMAT, SCONF_MANDATORY, &sformat);
if (err < 0)
    return err;
if (snd_pcm_format_linear(sformat) != 1 &&
    sformat != SND_PCM_FORMAT_IMA_ADPCM) {
    snd_config_delete(sconf);
    snd_error(PCM, "invalid slave format");
    return -EINVAL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 666) - [public] snd_config_delete

```
    ■■snd_config_delete(sconf);
    ■■snd_error(PCM, "invalid slave format");
    ■■return -EINVAL;
    ■}
    ■err = snd_pcm_open_slave(&spcm, root, sconf, stream, mode, conf);
    ■snd_config_delete(sconf);
    ■if (err < 0)
    ■■return err;
    ■err = snd_pcm_adpcm_open(pcmp, name, sformat, spcm, 1);
    ■if (err < 0)
    ■■snd_pcm_close(spcm);
    ■return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 667) - [public] snd_error

```
    ■■snd_error(PCM, "invalid slave format");
    ■■return -EINVAL;
    ■}
    ■err = snd_pcm_open_slave(&spcm, root, sconf, stream, mode, conf);
    ■snd_config_delete(sconf);
    ■if (err < 0)
    ■■return err;
    ■err = snd_pcm_adpcm_open(pcmp, name, sformat, spcm, 1);
    ■if (err < 0)
    ■■snd_pcm_close(spcm);
    ■return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 676) - [public] snd_pcm_close

```
    ■■snd_pcm_close(spcm);
    ■return err;
    }
    #ifndef DOC_HIDDEN
    SND_DLSYM_BUILD_VERSION(_snd_pcm_adpcm_open, SND_PCM_DLSYM_VERSION);
    #endif
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 168) - [public] while

```
    ■■while (frames1-- > 0) {
    ■■■int16_t sample = alaw_to_s16(*src);
    ■■■goto *put;
    #define PUT16_END after
    #include "plugin_ops.h"
    #undef PUT16_END
    ■■after:
    ■■■src += src_step;
    ■■■dst += dst_step;
    ■■}
    ■}
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 206) - [public] while

```
    ■■while (frames1-- > 0) {
    ■■■goto *get;
    #define GET16_END after
    #include "plugin_ops.h"
    #undef GET16_END
    ■■after:
    ■■■*dst = s16_to_alaw(sample);
    ■■■src += src_step;
    ■■■dst += dst_step;
    ■■}
    ■}
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 322) - [public] if

```
    if (alaw->sformat == SND_PCM_FORMAT_A_LAW) {
        alaw->getput_idx = snd_pcm_linear_get_index(format, SND_PCM_FORMAT_S16);
        alaw->func = snd_pcm_alaw_encode;
    } else {
        alaw->getput_idx = snd_pcm_linear_put_index(SND_PCM_FORMAT_S16, alaw->sformat);
        alaw->func = snd_pcm_alaw_decode;
    }
} else {
    if (alaw->sformat == SND_PCM_FORMAT_A_LAW) {
        alaw->getput_idx = snd_pcm_linear_put_index(SND_PCM_FORMAT_S16, format);
        alaw->func = snd_pcm_alaw_decode;
    } else {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 330) - [public] if

```
    if (alaw->sformat == SND_PCM_FORMAT_A_LAW) {
        alaw->getput_idx = snd_pcm_linear_put_index(SND_PCM_FORMAT_S16, format);
        alaw->func = snd_pcm_alaw_decode;
    } else {
        alaw->getput_idx = snd_pcm_linear_get_index(alaw->sformat, SND_PCM_FORMAT_S16);
        alaw->func = snd_pcm_alaw_encode;
    }
}
return 0;
}
```

```
static snd_pcm_uframes_t
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 385) - [public] snd_pcm_format_name

```
    snd_pcm_format_name(alaw->sformat));
    if (pcm->setup) {
        snd_output_printf(out, "Its setup is:\n");
        snd_pcm_dump_setup(pcm, out);
    }
    snd_output_printf(out, "Slave: ");
    snd_pcm_dump(alaw->plug.gen.slave, out);
}
```

```
static const snd_pcm_ops_t snd_pcm_alaw_ops = {
    .close = snd_pcm_generic_close,
    .info = snd_pcm_generic_info,
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 387) - [public] snd_output_printf

```
    snd_output_printf(out, "Its setup is:\n");
    snd_pcm_dump_setup(pcm, out);
}
    snd_output_printf(out, "Slave: ");
    snd_pcm_dump(alaw->plug.gen.slave, out);
}
```

```
static const snd_pcm_ops_t snd_pcm_alaw_ops = {
    .close = snd_pcm_generic_close,
    .info = snd_pcm_generic_info,
    .hw_refine = snd_pcm_alaw_hw_refine,
    .hw_params = snd_pcm_alaw_hw_params,
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 388) - [public] snd_pcm_dump_setup

```
    snd_pcm_dump_setup(pcm, out);
}
snd_output_printf(out, "Slave: ");
snd_pcm_dump(alaw->plug.gen.slave, out);
}

static const snd_pcm_ops_t snd_pcm_alaw_ops = {
    .close = snd_pcm_generic_close,
    .info = snd_pcm_generic_info,
    .hw_refine = snd_pcm_alaw_hw_refine,
    .hw_params = snd_pcm_alaw_hw_params,
    .hw_free = snd_pcm_generic_hw_free,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 448) - [public] free

```
    free(alaw);
    return err;
}
pcm->ops = &snd_pcm_alaw_ops;
pcm->fast_ops = &snd_pcm_plugin_fast_ops;
pcm->private_data = alaw;
pcm->poll_fd = slave->poll_fd;
pcm->poll_events = slave->poll_events;
pcm->tstamp_type = slave->tstamp_type;
snd_pcm_set_hw_ptr(pcm, &alaw->plug.hw_ptr, -1, 0);
snd_pcm_set_appl_ptr(pcm, &alaw->plug.appl_ptr, -1, 0);
*pcmp = pcm;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 524) - [public] if

```
    if (strcmp(id, "slave") == 0) {
        slave = n;
        continue;
    }
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (!slave) {
    snd_error(PCM, "slave is not defined");
    return -EINVAL;
}
err = snd_pcm_slave_conf(root, slave, &sconf, 1,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 528) - [public] snd_error

```
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (!slave) {
    snd_error(PCM, "slave is not defined");
    return -EINVAL;
}
err = snd_pcm_slave_conf(root, slave, &sconf, 1,
    SND_PCM_HW_PARAM_FORMAT, SCONF_MANDATORY, &sformat);
if (err < 0)
    return err;
if (snd_pcm_format_linear(sformat) != 1 &&
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 532) - [public] snd_error

```
    ■■snd_error(PCM, "slave is not defined");
    ■■return -EINVAL;
    ■}
    ■err = snd_pcm_slave_conf(root, slave, &sconf, 1,
    ■■■■ SND_PCM_HW_PARAM_FORMAT, SCONF_MANDATORY, &sformat);
    ■if (err < 0)
    ■■return err;
    ■if (snd_pcm_format_linear(sformat) != 1 &&
    ■    sformat != SND_PCM_FORMAT_A_LAW) {
    ■■snd_config_delete(sconf);
    ■■snd_error(PCM, "invalid slave format");
    ■■return -EINVAL;
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 541) - [public] snd_config_delete

```
    ■■snd_config_delete(sconf);
    ■■snd_error(PCM, "invalid slave format");
    ■■return -EINVAL;
    ■}
    ■err = snd_pcm_open_slave(&spcm, root, sconf, stream, mode, conf);
    ■snd_config_delete(sconf);
    ■if (err < 0)
    ■■return err;
    ■err = snd_pcm_alaw_open(pcmp, name, sformat, spcm, 1);
    ■if (err < 0)
    ■■snd_pcm_close(spcm);
    ■return err;
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 542) - [public] snd_error

```
    ■■snd_error(PCM, "invalid slave format");
    ■■return -EINVAL;
    ■}
    ■err = snd_pcm_open_slave(&spcm, root, sconf, stream, mode, conf);
    ■snd_config_delete(sconf);
    ■if (err < 0)
    ■■return err;
    ■err = snd_pcm_alaw_open(pcmp, name, sformat, spcm, 1);
    ■if (err < 0)
    ■■snd_pcm_close(spcm);
    ■return err;
    ■}
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 551) - [public] snd_pcm_close

```
    ■■snd_pcm_close(spcm);
    ■return err;
    ■}
    ■#ifndef DOC_HIDDEN
    ■SND_DLSYM_BUILD_VERSION(_snd_pcm_alaw_open, SND_PCM_DLSYM_VERSION);
    ■#endif
```

dirtycode/alsa-lib/src/pcm/pcm_asym.c (line 91) - [public] if

```
    ■■if (strcmp(id, "playback") == 0) {
    ■■■■if (stream == SND_PCM_STREAM_PLAYBACK)
    ■■■■slave = n;
    ■■■■continue;
    ■■}
    ■■if (strcmp(id, "capture") == 0) {
    ■■■■if (stream == SND_PCM_STREAM_CAPTURE)
    ■■■■slave = n;
    ■■■■continue;
    ■■}
    ■■snd_error(PCM, "Unknown field %s", id);
    ■■return -EINVAL;
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_asym.c (line 96) - [public] if

```
    if (strcmp(id, "capture") == 0) {
        if (stream == SND_PCM_STREAM_CAPTURE)
            slave = n;
        continue;
    }
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (! slave) {
    snd_error(PCM, "%s slave is not defined",
        stream == SND_PCM_STREAM_PLAYBACK ? "playback" : "capture");

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_asym.c (line 101) - [public] snd_error

```
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (! slave) {
    snd_error(PCM, "%s slave is not defined",
        stream == SND_PCM_STREAM_PLAYBACK ? "playback" : "capture");

    return -EINVAL;
}
err = snd_pcm_slave_conf(root, slave, &sconf, 0);
if (err < 0)
    return err;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 148) - [public] snd_output_printf

```
    snd_output_printf(out, "Its setup is:\n");
    snd_pcm_dump_setup(pcm, out);
}
snd_output_printf(out, "Slave: ");
snd_pcm_dump(copy->plug.gen.slave, out);
}

static const snd_pcm_ops_t snd_pcm_copy_ops = {
    .close = snd_pcm_generic_close,
    .info = snd_pcm_generic_info,
    .hw_refine = snd_pcm_copy_hw_refine,
    .hw_params = snd_pcm_copy_hw_params,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 149) - [public] snd_pcm_dump_setup

```
    snd_pcm_dump_setup(pcm, out);
}
snd_output_printf(out, "Slave: ");
snd_pcm_dump(copy->plug.gen.slave, out);
}

static const snd_pcm_ops_t snd_pcm_copy_ops = {
    .close = snd_pcm_generic_close,
    .info = snd_pcm_generic_info,
    .hw_refine = snd_pcm_copy_hw_refine,
    .hw_params = snd_pcm_copy_hw_params,
    .hw_free = snd_pcm_generic_hw_free,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 204) - [public] free

```
    free(copy);
    return err;
}
pcm->ops = &snd_pcm_copy_ops;
pcm->fast_ops = &snd_pcm_plugin_fast_ops;
pcm->private_data = copy;
pcm->poll_fd = slave->poll_fd;
pcm->poll_events = slave->poll_events;
pcm->tstamp_type = slave->tstamp_type;
snd_pcm_set_hw_ptr(pcm, &copy->plug.hw_ptr, -1, 0);
snd_pcm_set_appl_ptr(pcm, &copy->plug.appl_ptr, -1, 0);
*pcmp = pcm;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 277) - [public] if

```
    if (strcmp(id, "slave") == 0) {
        slave = n;
        continue;
    }
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (!slave) {
    snd_error(PCM, "slave is not defined");
    return -EINVAL;
}
err = snd_pcm_slave_conf(root, slave, &sconf, 0);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 281) - [public] snd_error

```
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (!slave) {
    snd_error(PCM, "slave is not defined");
    return -EINVAL;
}
err = snd_pcm_slave_conf(root, slave, &sconf, 0);
if (err < 0)
    return err;
err = snd_pcm_open_slave(&spcm, root, sconf, stream, mode, conf);
snd_config_delete(sconf);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 285) - [public] snd_error

```
    snd_error(PCM, "slave is not defined");
    return -EINVAL;
}
err = snd_pcm_slave_conf(root, slave, &sconf, 0);
if (err < 0)
    return err;
err = snd_pcm_open_slave(&spcm, root, sconf, stream, mode, conf);
snd_config_delete(sconf);
if (err < 0)
    return err;
err = snd_pcm_copy_open(pcmp, name, spcm, 1);
if (err < 0)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 297) - [public] snd_pcm_close

```
    snd_pcm_close(spcm);
    return err;
}
#ifdef DOC_HIDDEN
SND_DLSYM_BUILD_VERSION(_snd_pcm_copy_open, SND_PCM_DLSYM_VERSION);
#endif
```


dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 78) - [public] if

```
    if (semctl(dmix->semid, i, IPC_STAT, s) < 0) {
        int err = -errno;
        snd_pcm_direct_semaphore_discard(dmix);
        return err;
    }
    buf.sem_perm.gid = dmix->ipc_gid;
    s.buf = &buf;
    semctl(dmix->semid, i, IPC_SET, s);
}
return 0;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 80) - [public] snd_pcm_direct_semaphore_discard

```
snd_pcm_direct_semaphore_discard(dmix);
return err;
}
buf.sem_perm.gid = dmix->ipc_gid;
s.buf = &buf;
semctl(dmix->semid, i, IPC_SET, s);
}
return 0;
}
```

```
static unsigned int snd_pcm_direct_magic(snd_pcm_direct_t *dmix)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 85) - [public] semctl

```
semctl(dmix->semid, i, IPC_SET, s);
}
return 0;
}
```

```
static unsigned int snd_pcm_direct_magic(snd_pcm_direct_t *dmix)
{
    if (!dmix->direct_memory_access)
        return 0xa15ad300 + sizeof(snd_pcm_direct_share_t);
    else
        return 0xb15ad300 + sizeof(snd_pcm_direct_share_t);
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 131) - [public] snd_pcm_direct_shm_discard

```
snd_pcm_direct_shm_discard(dmix);
return err;
}
mlock(dmix->shmptr, sizeof(snd_pcm_direct_share_t));
if (shmctl(dmix->shmid, IPC_STAT, &buf) < 0) {
    err = -errno;
    snd_pcm_direct_shm_discard(dmix);
    return err;
}
if (first_instance) { /* we're the first user, clear the segment */
    memset(dmix->shmptr, 0, sizeof(snd_pcm_direct_share_t));
}
if (dmix->ipc_gid >= 0) {
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 137) - [public] snd_pcm_direct_shm_discard

```
    ■■snd_pcm_direct_shm_discard(dmix);
    ■■return err;
    ■}
    ■if (first_instance) { /* we're the first user, clear the segment */
    ■■memset(dmix->shmptr, 0, sizeof(snd_pcm_direct_share_t));
    ■■if (dmix->ipc_gid >= 0) {
    ■■■buf.shm_perm.gid = dmix->ipc_gid;
    ■■■shmctl(dmix->shmid, IPC_SET, &buf);
    ■■}
    ■■dmix->shmptr->magic = snd_pcm_direct_magic(dmix);
    ■■return 1;
    ■} else {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 141) - [public] memset

```
retryget:
dmix->shmid = shmget(dmix->ipc_key, sizeof(snd_pcm_direct_share_t),
dmix->ipc_perm);
if (dmix->shmid < 0 && errno == ENOENT) {
if ((dmix->shmid = shmget(dmix->ipc_key, sizeof(snd_pcm_direct_share_t),
IPC_CREAT | IPC_EXCL | dmix->ipc_perm)) != -1)
first_instance = 1;
else if (errno == EEXIST)
goto retryget;
}
err = -errno;
if (dmix->shmid < 0) {
if (errno == EINVAL)
if ((tmpid = shmget(dmix->ipc_key, 0, dmix->ipc_perm)) != -1)
if (!shmctl(tmpid, IPC_STAT, &buf))
if (!buf.shm_nattch)
/* no users so destroy the segment */
if (!shmctl(tmpid, IPC_RMID, NULL))
goto retryget;
return err;
}
dmix->shmptr = shmat(dmix->shmid, 0, 0);
if (dmix->shmptr == (void *) -1) {
err = -errno;
snd_pcm_direct_shm_discard(dmix);
return err;
}
mlock(dmix->shmptr, sizeof(snd_pcm_direct_share_t));
if (shmctl(dmix->shmid, IPC_STAT, &buf) < 0) {
err = -errno;
snd_pcm_direct_shm_discard(dmix);
return err;
}
if (first_instance) { /* we're the first user, clear the segment */
    ■■memset(dmix->shmptr, 0, sizeof(snd_pcm_direct_share_t));
    ■■if (dmix->ipc_gid >= 0) {
    ■■■buf.shm_perm.gid = dmix->ipc_gid;
    ■■■shmctl(dmix->shmid, IPC_SET, &buf);
    ■■}
    ■■dmix->shmptr->magic = snd_pcm_direct_magic(dmix);
    ■■return 1;
    ■} else {
    ■■if (dmix->shmptr->magic != snd_pcm_direct_magic(dmix)) {
    ■■■snd_pcm_direct_shm_discard(dmix);
    ■■■return -EINVAL;
    ■■}
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 142) - [public] if

```
    if (dmix->ipc_gid >= 0) {
        buf.shm_perm.gid = dmix->ipc_gid;
        shmctl(dmix->shmid, IPC_SET, &buf);
    }
    dmix->shmptr->magic = snd_pcm_direct_magic(dmix);
    return 1;
} else {
    if (dmix->shmptr->magic != snd_pcm_direct_magic(dmix)) {
        snd_pcm_direct_shm_discard(dmix);
        return -EINVAL;
    }
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 144) - [public] shmctl

```
    shmctl(dmix->shmid, IPC_SET, &buf);
}
dmix->shmptr->magic = snd_pcm_direct_magic(dmix);
return 1;
} else {
    if (dmix->shmptr->magic != snd_pcm_direct_magic(dmix)) {
        snd_pcm_direct_shm_discard(dmix);
        return -EINVAL;
    }
}
return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 149) - [public] if

```
    if (dmix->shmptr->magic != snd_pcm_direct_magic(dmix)) {
        snd_pcm_direct_shm_discard(dmix);
        return -EINVAL;
    }
}
return 0;
}

/* discard shared memory */
/*
 * Define snd_* functions to be used in server.
 * Since objects referred in a plugin can be released dynamically, a forked
 * // ... (truncated, see source file for more)
 */
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 150) - [public] snd_pcm_direct_shm_discard

```
    snd_pcm_direct_shm_discard(dmix);
    return -EINVAL;
}
return 0;
}

/* discard shared memory */
/*
 * Define snd_* functions to be used in server.
 * Since objects referred in a plugin can be released dynamically, a forked
 * server should have statically linked functions.
 * // ... (truncated, see source file for more)
 */
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 188) - [public] _snd_pcm_direct_shm_discard

```
    return _snd_pcm_direct_shm_discard(dmix);
}

/*
 * server side
 */

static int get_tmp_name(char *filename, size_t size)
{
    struct timeval tv;

    gettimeofday(&tv, NULL);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 215) - [public] snd_erronum

```
    snd_erronum(PCM, "socket failed");
    return result;
}

if (server)
    unlink(filename);
memset(addr, 0, size); /* make valgrind happy */
addr->sun_family = AF_LOCAL;
memcpy(addr->sun_path, filename, l);

if (server) {
    if (bind(sock, (struct sockaddr *) addr, size) < 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 220) - [public] unlink

```
    unlink(filename);
memset(addr, 0, size); /* make valgrind happy */
addr->sun_family = AF_LOCAL;
memcpy(addr->sun_path, filename, l);

if (server) {
    if (bind(sock, (struct sockaddr *) addr, size) < 0) {
        int result = -errno;
        snd_erronum(PCM, "bind failed: %s", filename);
        close(sock);
        return result;
    } else {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 226) - [public] if

```
    if (bind(sock, (struct sockaddr *) addr, size) < 0) {
        int result = -errno;
        snd_erronum(PCM, "bind failed: %s", filename);
        close(sock);
        return result;
    } else {
        if (chmod(filename, ipc_perm) < 0) {
            int result = -errno;
            snd_erronum(PCM, "chmod failed: %s", filename);
            close(sock);
            unlink(filename);
            return result;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 228) - [public] snd_errormap

```
    snd_errormap(PCM, "bind failed: %s", filename);
    close(sock);
    return result;
} else {
    if (chmod(filename, ipc_perm) < 0) {
        int result = -errno;
        snd_errormap(PCM, "chmod failed: %s", filename);
        close(sock);
        unlink(filename);
        return result;
    }
    if (chown(filename, -1, ipc_gid) < 0) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 229) - [public] close

```
    close(sock);
    return result;
} else {
    if (chmod(filename, ipc_perm) < 0) {
        int result = -errno;
        snd_errormap(PCM, "chmod failed: %s", filename);
        close(sock);
        unlink(filename);
        return result;
    }
    if (chown(filename, -1, ipc_gid) < 0) {
        #if 0 /* it's not fatal */
            // ... (truncated, see source file for more)
        #endif
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 232) - [public] if

```
    if (chmod(filename, ipc_perm) < 0) {
        int result = -errno;
        snd_errormap(PCM, "chmod failed: %s", filename);
        close(sock);
        unlink(filename);
        return result;
    }
    if (chown(filename, -1, ipc_gid) < 0) {
        #if 0 /* it's not fatal */
            int result = -errno;
            snd_errormap(PCM, "chown failed: %s", filename);
            close(sock);
            // ... (truncated, see source file for more)
        #endif
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 234) - [public] snd_errormap

```
    snd_errormap(PCM, "chmod failed: %s", filename);
    close(sock);
    unlink(filename);
    return result;
}
if (chown(filename, -1, ipc_gid) < 0) {
    #if 0 /* it's not fatal */
        int result = -errno;
        snd_errormap(PCM, "chown failed: %s", filename);
        close(sock);
        unlink(filename);
        return result;
        // ... (truncated, see source file for more)
    #endif
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 235) - [public] close

```
    close(sock);
    unlink(filename);
    return result;
}
if (chown(filename, -1, ipc_gid) < 0) {
    /* it's not fatal */
    int result = -errno;
    snd_errormsg(PCM, "chown failed: %s", filename);
    close(sock);
    unlink(filename);
    return result;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 236) - [public] unlink

```
    unlink(filename);
    return result;
}
if (chown(filename, -1, ipc_gid) < 0) {
    /* it's not fatal */
    int result = -errno;
    snd_errormsg(PCM, "chown failed: %s", filename);
    close(sock);
    unlink(filename);
    return result;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 239) - [public] if

```
if (chown(filename, -1, ipc_gid) < 0) {
    /* it's not fatal */
    int result = -errno;
    snd_errormsg(PCM, "chown failed: %s", filename);
    close(sock);
    unlink(filename);
    return result;
}
} else {
    if (connect(sock, (struct sockaddr *) addr, size) < 0) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 242) - [public] snd_errormsg

```
    snd_errormsg(PCM, "chown failed: %s", filename);
    close(sock);
    unlink(filename);
    return result;
}
} else {
    if (connect(sock, (struct sockaddr *) addr, size) < 0) {
        int result = -errno;
        snd_errormsg(PCM, "connect failed: %s", filename);
        close(sock);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 243) - [public] close

```
    close(sock);
    unlink(filename);
    return result;
#endif
}
} else {
    if (connect(sock, (struct sockaddr *) addr, size) < 0) {
        int result = -errno;
        snd_errornum(PCM, "connect failed: %s", filename);
        close(sock);
        return result;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 244) - [public] unlink

```
    unlink(filename);
    return result;
#endif
}
} else {
    if (connect(sock, (struct sockaddr *) addr, size) < 0) {
        int result = -errno;
        snd_errornum(PCM, "connect failed: %s", filename);
        close(sock);
        return result;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 250) - [public] if

```
    if (connect(sock, (struct sockaddr *) addr, size) < 0) {
        int result = -errno;
        snd_errornum(PCM, "connect failed: %s", filename);
        close(sock);
        return result;
    }
    return sock;
}

#if 0
#define SERVER_JOB_DEBUG
    // ... (truncated, see source file for more)
#endif
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 252) - [public] snd_errornum

```
    snd_errornum(PCM, "connect failed: %s", filename);
    close(sock);
    return result;
}
return sock;
}

#if 0
#define SERVER_JOB_DEBUG
#define server_printf(fmt, args...) printf(fmt, ##args)
#else
    // ... (truncated, see source file for more)
#endif
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 253) - [public] close

```
    close(sock);
    return result;
}
return sock;
}

#if 0
#define SERVER_JOB_DEBUG
#define server_printf(fmt, args...) printf(fmt, ##args)
#else
#undef SERVER_JOB_DEBUG
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 343) - [public] close

```
    close(i);
}

/* detach from parent */
setsid();

pfds[0].fd = dmix->server_fd;
pfds[0].events = POLLIN | POLLERR | POLLHUP;

server_printf("DIRECT SERVER STARTED\n");
while (1) {
    ret = poll(pfds, current + 1, 500);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 355) - [public] server_printf

```
server_printf("DIRECT SERVER: poll ret = %i, revents[0] = 0x%x, errno = %i\n", ret, pfds[0].revents, e
if (ret < 0) {
    if (errno == EINTR)
        continue;
    /* some error */
    break;
}
if (ret == 0 || (pfds[0].revents & (POLLERR | POLLHUP))) { /* timeout or error? */
    struct shmids buf;
    snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
    if (shmctl(dmix->shmids, IPC_STAT, &buf) < 0) {
        _snd_pcm_direct_shm_discard(dmix);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 356) - [public] if

```
if (ret < 0) {
    if (errno == EINTR)
        continue;
    /* some error */
    break;
}
if (ret == 0 || (pfds[0].revents & (POLLERR | POLLHUP))) { /* timeout or error? */
    struct shmids buf;
    snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
    if (shmctl(dmix->shmids, IPC_STAT, &buf) < 0) {
        _snd_pcm_direct_shm_discard(dmix);
        snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
        // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 362) - [public] if

```
    if (ret == 0 || (pfds[0].revents & (POLLERR | POLLHUP))) { /* timeout or error? */
        struct shm_id buf;
        snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
        if (shmctl(dmix->shmid, IPC_STAT, &buf) < 0) {
            _snd_pcm_direct_shm_discard(dmix);
            snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
            continue;
        }
        server_printf("DIRECT SERVER: nattch = %i\n", (int)buf.shm_nattch);
        if (buf.shm_nattch == 1) /* server is the last user, exit */
            break;
        snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 364) - [public] snd_pcm_direct_semaphore_down

```
    snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
    if (shmctl(dmix->shmid, IPC_STAT, &buf) < 0) {
        _snd_pcm_direct_shm_discard(dmix);
        snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
        continue;
    }
    server_printf("DIRECT SERVER: nattch = %i\n", (int)buf.shm_nattch);
    if (buf.shm_nattch == 1) /* server is the last user, exit */
        break;
    snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
    continue;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 365) - [public] if

```
    if (shmctl(dmix->shmid, IPC_STAT, &buf) < 0) {
        _snd_pcm_direct_shm_discard(dmix);
        snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
        continue;
    }
    server_printf("DIRECT SERVER: nattch = %i\n", (int)buf.shm_nattch);
    if (buf.shm_nattch == 1) /* server is the last user, exit */
        break;
    snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
    continue;
}
```

if (pfds[0].revents & POLLIN) {
 // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 366) - [public] _snd_pcm_direct_shm_discard

```
    _snd_pcm_direct_shm_discard(dmix);
    snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
    continue;
}
```

server_printf("DIRECT SERVER: nattch = %i\n", (int)buf.shm_nattch);
if (buf.shm_nattch == 1) /* server is the last user, exit */
 break;
snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
continue;
}

if (pfds[0].revents & POLLIN) {
 ret--;
 // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 367) - [public] snd_pcm_direct_semaphore_up

```
    snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
    continue;
}
server_printf("DIRECT SERVER: natch = %i\n", (int)buf.shm_natch);
if (buf.shm_natch == 1)/* server is the last user, exit */
    break;
snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
continue;
}
if (pfds[0].revents & POLLIN) {
    ret--;
    sck = accept(dmix->server_fd, 0, 0);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 370) - [public] server_printf

```
server_printf("DIRECT SERVER: natch = %i\n", (int)buf.shm_natch);
if (buf.shm_natch == 1)/* server is the last user, exit */
    break;
snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
continue;
}
if (pfds[0].revents & POLLIN) {
    ret--;
    sck = accept(dmix->server_fd, 0, 0);
    if (sck >= 0) {
        server_printf("DIRECT SERVER: new connection %i\n", sck);
        if (current == max) {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 373) - [public] snd_pcm_direct_semaphore_up

```
    snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
    continue;
}
if (pfds[0].revents & POLLIN) {
    ret--;
    sck = accept(dmix->server_fd, 0, 0);
    if (sck >= 0) {
        server_printf("DIRECT SERVER: new connection %i\n", sck);
        if (current == max) {
            close(sck);
        } else {
            unsigned char buf = 'A';
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 376) - [public] if

```
if (pfds[0].revents & POLLIN) {
    ret--;
    sck = accept(dmix->server_fd, 0, 0);
    if (sck >= 0) {
        server_printf("DIRECT SERVER: new connection %i\n", sck);
        if (current == max) {
            close(sck);
        } else {
            unsigned char buf = 'A';
            pfd[current+1].fd = sck;
            pfd[current+1].events = POLLIN | POLLERR | POLLHUP;
            snd_send_fd(sck, &buf, 1, dmix->hw_fd);
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 379) - [public] if

```
    if (sck >= 0) {
        server_printf("DIRECT SERVER: new connection %i\n", sck);
        if (current == max) {
            close(sck);
        } else {
            unsigned char buf = 'A';
            pfds[current+1].fd = sck;
            pfds[current+1].events = POLLIN | POLLERR | POLLHUP;
            _snd_send_fd(sck, &buf, 1, dmix->hw_fd);
            server_printf("DIRECT SERVER: fd sent ok\n");
            current++;
        }
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 380) - [public] server_printf

```
    server_printf("DIRECT SERVER: new connection %i\n", sck);
    if (current == max) {
        close(sck);
    } else {
        unsigned char buf = 'A';
        pfds[current+1].fd = sck;
        pfds[current+1].events = POLLIN | POLLERR | POLLHUP;
        _snd_send_fd(sck, &buf, 1, dmix->hw_fd);
        server_printf("DIRECT SERVER: fd sent ok\n");
        current++;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 381) - [public] if

```
    if (current == max) {
        close(sck);
    } else {
        unsigned char buf = 'A';
        pfds[current+1].fd = sck;
        pfds[current+1].events = POLLIN | POLLERR | POLLHUP;
        _snd_send_fd(sck, &buf, 1, dmix->hw_fd);
        server_printf("DIRECT SERVER: fd sent ok\n");
        current++;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 382) - [public] close

```
    close(sck);
} else {
    unsigned char buf = 'A';
    pfds[current+1].fd = sck;
    pfds[current+1].events = POLLIN | POLLERR | POLLHUP;
    _snd_send_fd(sck, &buf, 1, dmix->hw_fd);
    server_printf("DIRECT SERVER: fd sent ok\n");
    current++;
}
for (i = 0; i < current && ret > 0; i++) {
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 387) - [public] _snd_send_fd

```
#####_snd_send_fd(sck, &buf, 1, dmix->hw_fd);
#####server_printf("DIRECT SERVER: fd sent ok\n");
#####current++;
#####}
#####}
#####}
for (i = 0; i < current && ret > 0; i++) {
#####struct pollfd *pfd = &pfds[i+1];
#####unsigned char cmd;
#####server_printf("client %i revents = 0x%x\n", pfd->fd, pfd->revents);
#####if (pfd->revents & (POLLERR | POLLHUP)) {
#####ret--;
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 388) - [public] server_printf

```
#####server_printf("DIRECT SERVER: fd sent ok\n");
#####current++;
#####}
#####}
#####}
for (i = 0; i < current && ret > 0; i++) {
#####struct pollfd *pfd = &pfds[i+1];
#####unsigned char cmd;
#####server_printf("client %i revents = 0x%x\n", pfd->fd, pfd->revents);
#####if (pfd->revents & (POLLERR | POLLHUP)) {
#####ret--;
#####close(pfd->fd);
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 396) - [public] server_printf

```
#####server_printf("client %i revents = 0x%x\n", pfd->fd, pfd->revents);
#####if (pfd->revents & (POLLERR | POLLHUP)) {
#####ret--;
#####close(pfd->fd);
#####pfd->fd = -1;
#####continue;
#####}
#####if (!(pfd->revents & POLLIN))
#####continue;
#####ret--;
#####if (read(pfd->fd, &cmd, 1) == 1)
#####cmd = 0 /*process command */;
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 397) - [public] if

```
#####if (pfd->revents & (POLLERR | POLLHUP)) {
#####ret--;
#####close(pfd->fd);
#####pfd->fd = -1;
#####continue;
#####}
#####if (!(pfd->revents & POLLIN))
#####continue;
#####ret--;
#####if (read(pfd->fd, &cmd, 1) == 1)
#####cmd = 0 /*process command */;
#####}
#####// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 399) - [public] close

```

    close(pfd->fd);
    pfd->fd = -1;
    continue;
}
if (!(pfd->revents & POLLIN))
    continue;
ret--;
if (read(pfd->fd, &cmd, 1) == 1)
    cmd = 0 /*process command */;
}
for (i = 0; i < current; i++) {
    if (pfds[i+1].fd < 0) {
        // ... (truncated, see source file for more)
    }
}

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 410) - [public] if

```

    if (pfds[i+1].fd < 0) {
        if (i + 1 != max)
            memcpy(&pfds[i+1], &pfds[i+2], sizeof(struct pollfd) * (max - i - 1));
        current--;
    }
}
}
server_cleanup(dmixon);
server_printf("DIRECT SERVER EXIT\n");
#ifdef SERVER_JOB_DEBUG
close(0); close(1); close(2);
#endif
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 412) - [public] memcpy

```

    memcpy(&pfds[i+1], &pfds[i+2], sizeof(struct pollfd) * (max - i - 1));
    current--;
}
}
}
server_cleanup(dmixon);
server_printf("DIRECT SERVER EXIT\n");
#ifdef SERVER_JOB_DEBUG
close(0); close(1); close(2);
#endif
_exit(EXIT_SUCCESS);
}
// ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 442) - [public] close

```

close(dmixon->server_fd);
return ret;
}

ret = fork();
if (ret < 0) {
    close(dmixon->server_fd);
    return ret;
} else if (ret == 0) {
    ret = fork();
    if (ret == 0)
        server_job(dmixon);
    // ... (truncated, see source file for more)
}

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 448) - [public] close

```
    close(dmix->server_fd);
    return ret;
} else if (ret == 0) {
    ret = fork();
    if (ret == 0)
        server_job(dmix);
    _exit(EXIT_SUCCESS);
} else {
    waitpid(ret, NULL, 0);
}
dmix->server_pid = ret;
dmix->server = 1;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 453) - [public] server_job

```
server_job(dmix);
_exit(EXIT_SUCCESS);
} else {
    waitpid(ret, NULL, 0);
}
dmix->server_pid = ret;
dmix->server = 1;
return 0;
}
```

```
int snd_pcm_direct_server_discard(snd_pcm_direct_t *dmix)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 454) - [public] _exit

```
_exit(EXIT_SUCCESS);
} else {
    waitpid(ret, NULL, 0);
}
dmix->server_pid = ret;
dmix->server = 1;
return 0;
}
```

```
int snd_pcm_direct_server_discard(snd_pcm_direct_t *dmix)
{
    if (dmix->server) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 456) - [public] waitpid

```
waitpid(ret, NULL, 0);
}
dmix->server_pid = ret;
dmix->server = 1;
return 0;
}
```

```
int snd_pcm_direct_server_discard(snd_pcm_direct_t *dmix)
{
    if (dmix->server) {
        //kill(dmix->server_pid, SIGTERM);
        //waitpid(dmix->server_pid, NULL, 0);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 471) - [public] close

```
    close(dmix->server_fd);
    dmix->server_fd = -1;
}
dmix->server = 0;
return 0;
}
```

```
/*
 * client side
 */
```

```
int snd_pcm_direct_client_connect(snd_pcm_direct_t *dmix)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 494) - [public] close

```
    close(dmix->comm_fd);
    dmix->comm_fd = -1;
    return ret;
}
```

```
dmix->client = 1;
return 0;
}
```

```
int snd_pcm_direct_client_discard(snd_pcm_direct_t *dmix)
{
    if (dmix->client) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 506) - [public] close

```
    close(dmix->comm_fd);
    dmix->comm_fd = -1;
}
return 0;
}
```

```
/*
 * plugin helpers
 */
```

```
int snd_pcm_direct_nonblock(snd_pcm_t *pcm ATTRIBUTE_UNUSED, int nonblock ATTRIBUTE_UNUSED)
{
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 525) - [public] snd_timer_async

```
return snd_timer_async(dmix->timer, sig, pid);
}
```

```
/* empty the timer read queue */
```

```
int snd_pcm_direct_clear_timer_queue(snd_pcm_direct_t *dmix)
{
    int changed = 0;
    if (dmix->timer_need_poll) {
        while (poll(&dmix->timer_fd, 1, 0) > 0) {
            changed++;
            /* we don't need the value */
            if (dmix->tread) {
                // ... (truncated, see source file for more)
            }
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 533) - [public] while

```
    while (poll(&dmix->timer_fd, 1, 0) > 0) {
        changed++;
        /* we don't need the value */
        if (dmix->tread) {
            snd_timer_tread_t rbuf[4];
            snd_timer_read(dmix->timer, rbuf, sizeof(rbuf));
        } else {
            snd_timer_read_t rbuf;
            snd_timer_read(dmix->timer, &rbuf, sizeof(rbuf));
        }
    }
} else {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 536) - [public] if

```
/* empty the timer read queue */
int snd_pcm_direct_clear_timer_queue(snd_pcm_direct_t *dmix)
{
    int changed = 0;
    if (dmix->timer_need_poll) {
        while (poll(&dmix->timer_fd, 1, 0) > 0) {
            changed++;
            /* we don't need the value */
            if (dmix->tread) {
                snd_timer_tread_t rbuf[4];
                snd_timer_read(dmix->timer, rbuf, sizeof(rbuf));
            } else {
                snd_timer_read_t rbuf;
                snd_timer_read(dmix->timer, &rbuf, sizeof(rbuf));
            }
        }
    } else {
        if (dmix->tread) {
            snd_timer_tread_t rbuf[4];
            int len;
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 538) - [public] snd_timer_read

```
    snd_timer_read(dmix->timer, rbuf, sizeof(rbuf));
} else {
    snd_timer_read_t rbuf;
    snd_timer_read(dmix->timer, &rbuf, sizeof(rbuf));
}
} else {
    if (dmix->tread) {
        snd_timer_tread_t rbuf[4];
        int len;
        while ((len = snd_timer_read(dmix->timer, rbuf,
            sizeof(rbuf))) > 0
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 541) - [public] snd_timer_read

```
    snd_timer_read(dmix->timer, &rbuf, sizeof(rbuf));
}
} else {
    if (dmix->tread) {
        snd_timer_tread_t rbuf[4];
        int len;
        while ((len = snd_timer_read(dmix->timer, rbuf,
            sizeof(rbuf))) > 0
            && (++changed) &&
            len != sizeof(rbuf[0]))
        ;
        // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 545) - [public] if

```
    if (dmix->tread) {
        snd_timer_tread_t rbuf[4];
        int len;
        while ((len = snd_timer_read(dmix->timer, rbuf,
                                     sizeof(rbuf))) > 0
               && (++changed) &&
               len != sizeof(rbuf[0]))
            ;
    } else {
        snd_timer_read_t rbuf;
        while (snd_timer_read(dmix->timer, &rbuf, sizeof(rbuf)) > 0)
            changed++;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 588) - [public] snd_error

```
    snd_error(PCM, "SEMDOWN FAILED with err %d", semerr);
    return semerr;
}

state = snd_pcm_state(direct->spcm);
if (state != SND_PCM_STATE_XRUN && state != SND_PCM_STATE_SUSPENDED) {
    /* ignore... someone else already did recovery */
    semerr = snd_pcm_direct_semaphore_up(direct,
                                         DIRECT_IPC_SEM_CLIENT);
    if (semerr < 0) {
        snd_error(PCM, "SEMUP FAILED with err %d", semerr);
        return semerr;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 597) - [public] if

```
    if (semerr < 0) {
        snd_error(PCM, "SEMUP FAILED with err %d", semerr);
        return semerr;
    }
    return 0;
}

recoveries = direct->shmptr->s.recoveries;
recoveries = (recoveries + 1) & RECOVERIES_MASK;
if (state == SND_PCM_STATE_SUSPENDED)
    recoveries |= RECOVERIES_FLAG_SUSPENDED;
direct->shmptr->s.recoveries = recoveries;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 598) - [public] snd_error

```
    snd_error(PCM, "SEMUP FAILED with err %d", semerr);
    return semerr;
}
return 0;
}

recoveries = direct->shmptr->s.recoveries;
recoveries = (recoveries + 1) & RECOVERIES_MASK;
if (state == SND_PCM_STATE_SUSPENDED)
    recoveries |= RECOVERIES_FLAG_SUSPENDED;
direct->shmptr->s.recoveries = recoveries;

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 616) - [public] snd_pcm_resume

```
    snd_pcm_resume(direct->spcm);
    snd_pcm_drop(direct->spcm);
    snd_pcm_direct_timer_stop(direct);
    snd_pcm_direct_clear_timer_queue(direct);
}

ret = snd_pcm_prepare(direct->spcm);
if (ret < 0) {
    snd_error(PCM, "recover: unable to prepare slave");
    semerr = snd_pcm_direct_semaphore_up(direct,
        DIRECT_IPC_SEM_CLIENT);
    if (semerr < 0) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 617) - [public] snd_pcm_drop

```
    snd_pcm_drop(direct->spcm);
    snd_pcm_direct_timer_stop(direct);
    snd_pcm_direct_clear_timer_queue(direct);
}

ret = snd_pcm_prepare(direct->spcm);
if (ret < 0) {
    snd_error(PCM, "recover: unable to prepare slave");
    semerr = snd_pcm_direct_semaphore_up(direct,
        DIRECT_IPC_SEM_CLIENT);
    if (semerr < 0) {
        snd_error(PCM, "SEMUP FAILED with err %d", semerr);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 618) - [public] snd_pcm_direct_timer_stop

```
    snd_pcm_direct_timer_stop(direct);
    snd_pcm_direct_clear_timer_queue(direct);
}

ret = snd_pcm_prepare(direct->spcm);
if (ret < 0) {
    snd_error(PCM, "recover: unable to prepare slave");
    semerr = snd_pcm_direct_semaphore_up(direct,
        DIRECT_IPC_SEM_CLIENT);
    if (semerr < 0) {
        snd_error(PCM, "SEMUP FAILED with err %d", semerr);
        return semerr;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 619) - [public] snd_pcm_direct_clear_timer_queue

```
    snd_pcm_direct_clear_timer_queue(direct);
}

ret = snd_pcm_prepare(direct->spcm);
if (ret < 0) {
    snd_error(PCM, "recover: unable to prepare slave");
    semerr = snd_pcm_direct_semaphore_up(direct,
        DIRECT_IPC_SEM_CLIENT);
    if (semerr < 0) {
        snd_error(PCM, "SEMUP FAILED with err %d", semerr);
        return semerr;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 624) - [public] snd_error

```
    snd_error(PCM, "recover: unable to prepare slave");
    semerr = snd_pcm_direct_semaphore_up(direct,
    DIRECT_IPC_SEM_CLIENT);
    if (semerr < 0) {
        snd_error(PCM, "SEMUP FAILED with err %d", semerr);
        return semerr;
    }
    return ret;
}

if (direct->type == SND_PCM_TYPE_DSHARE) {
    const snd_pcm_channel_area_t *dst_areas;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 627) - [public] if

```
    if (semerr < 0) {
        snd_error(PCM, "SEMUP FAILED with err %d", semerr);
        return semerr;
    }
    return ret;
}

if (direct->type == SND_PCM_TYPE_DSHARE) {
    const snd_pcm_channel_area_t *dst_areas;
    dst_areas = snd_pcm_mmap_areas(direct->spcm);
    snd_pcm_areas_silence(dst_areas, 0, direct->spcm->channels,
        direct->spcm->buffer_size,
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 628) - [public] snd_error

```
    snd_error(PCM, "SEMUP FAILED with err %d", semerr);
    return semerr;
}
return ret;
}

if (direct->type == SND_PCM_TYPE_DSHARE) {
    const snd_pcm_channel_area_t *dst_areas;
    dst_areas = snd_pcm_mmap_areas(direct->spcm);
    snd_pcm_areas_silence(dst_areas, 0, direct->spcm->channels,
        direct->spcm->buffer_size,
        direct->spcm->format);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 644) - [public] snd_error

```
    snd_error(PCM, "recover: unable to start slave");
    semerr = snd_pcm_direct_semaphore_up(direct,
    DIRECT_IPC_SEM_CLIENT);
    if (semerr < 0) {
        snd_error(PCM, "SEMUP FAILED with err %d", semerr);
        return semerr;
    }
    return ret;
}
semerr = snd_pcm_direct_semaphore_up(direct,
    DIRECT_IPC_SEM_CLIENT);
if (semerr < 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 647) - [public] if

```
    if (semerr < 0) {
        snd_error(PCM, "SEMUP FAILED with err %d", semerr);
        return semerr;
    }
    return ret;
}
semerr = snd_pcm_direct_semaphore_up(direct,
    DIRECT_IPC_SEM_CLIENT);
if (semerr < 0) {
    snd_error(PCM, "SEMUP FAILED with err %d", semerr);
    return semerr;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 648) - [public] snd_error

```
    snd_error(PCM, "SEMUP FAILED with err %d", semerr);
    return semerr;
}
return ret;
}
semerr = snd_pcm_direct_semaphore_up(direct,
    DIRECT_IPC_SEM_CLIENT);
if (semerr < 0) {
    snd_error(PCM, "SEMUP FAILED with err %d", semerr);
    return semerr;
}
return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 656) - [public] snd_error

```
    snd_error(PCM, "SEMUP FAILED with err %d", semerr);
    return semerr;
}
return 0;
}

/*
 * enter xrun or suspended state, if slave xrun occurred or suspended
 * @return: 0 for no xrun/suspend or a negative error code for xrun/suspend
 */
int snd_pcm_direct_check_xrun(snd_pcm_direct_t *direct, snd_pcm_t *pcm)
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 694) - [public] gettimeofday

```
/* no matter how many xruns we missed -
 * so don't increment but just update to actual counter
 */
direct->recoveries = direct->shmptr->s.recoveries;
pcm->fast_ops->drop(pcm->fast_op_arg);
/* trigger_tstamp update is missing in drop callbacks */

    gettimeofday(&direct->trigger_tstamp, pcm->tstamp_type);
    /* no timer clear:
    * if slave already entered xrun again the event is lost.
    * snd_pcm_direct_clear_timer_queue(direct);
    */
    if (direct->recoveries & RECOVERIES_FLAG_SUSPENDED) {
        direct->state = SND_PCM_STATE_SUSPENDED;
        return -ESTRPIPE;
    } else {
        direct->state = SND_PCM_STATE_XRUN;
        return -EPIPE;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 697) - [public] snd_pcm_direct_clear_timer_queue

/ no timer clear:*

** if slave already entered xrun again the event is lost.*

```
    ■■ * snd_pcm_direct_clear_timer_queue(direct);
    ■■ */
    ■■if (direct->recoveries & RECOVERIES_FLAG_SUSPENDED) {
    ■■■direct->state = SND_PCM_STATE_SUSPENDED;
    ■■■return -ESTRPIPE;
    ■■} else {
    ■■■direct->state = SND_PCM_STATE_XRUN;
    ■■■return -EPIPE;
    ■■}
    ■}
    ■return 0;
    }
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 699) - [public] if

/ no timer clear:*

** if slave already entered xrun again the event is lost.*

** snd_pcm_direct_clear_timer_queue(direct);*

**/*

```
    ■■if (direct->recoveries & RECOVERIES_FLAG_SUSPENDED) {
    ■■■direct->state = SND_PCM_STATE_SUSPENDED;
    ■■■return -ESTRPIPE;
    ■■} else {
    ■■■direct->state = SND_PCM_STATE_XRUN;
    ■■■return -EPIPE;
    ■■}
    ■}
    ■return 0;
    }
```

*/**

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 730) - [public] snd_check

```
    ■■snd_check(PCM, "poll_fd < 0");
    ■■return -EIO;
    ■}
    ■if (space >= 1 && pfds) {
    ■■pfds->fd = pcm->poll_fd;
    ■■pfds->events = pcm->poll_events | POLLERR | POLLNVAL;
    ■} else {
    ■■return 0;
    ■}

    ■/* this will also evaluate slave state and enter xrun if necessary */
    ■/* using __snd_pcm_state() since this function is called inside lock */
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 763) - [public] __snd_pcm_avail_update

```
    ■■__snd_pcm_avail_update(pcm);
    ■■if (pcm->stream == SND_PCM_STREAM_PLAYBACK) {
    ■■■events |= POLLOUT;
    ■■■events &= ~POLLIN;
    ■■■avail = snd_pcm_mmap_playback_avail(pcm);
    ■■} else {
    ■■■avail = snd_pcm_mmap_capture_avail(pcm);
    ■■}
    ■■empty = avail < pcm->avail_min;
    ■}

    ■if (snd_pcm_direct_check_xrun(dmix, pcm) < 0 ||
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 764) - [public] if

```
    if (pcm->stream == SND_PCM_STREAM_PLAYBACK) {
        events |= POLLOUT;
        events &= ~POLLIN;
        avail = snd_pcm_mmap_playback_avail(pcm);
    } else {
        avail = snd_pcm_mmap_capture_avail(pcm);
    }
    empty = avail < pcm->avail_min;

    if (snd_pcm_direct_check_xrun(dmix, pcm) < 0 ||
        snd_pcm_state(dmix->spcm) == SND_PCM_STATE_SETUP) {
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 775) - [public] snd_pcm_state

```
    if (snd_pcm_state(dmix->spcm) == SND_PCM_STATE_SETUP) {
        events |= POLLERR;
    } else {
        if (empty) {
            /* here we have a race condition:
             * if period event arrived after the avail_update call
             * above we might clear this event with the following
             * clear_timer_queue.
             * There is no way to do this in atomic manner, so we
             * need to recheck avail_update if we successfully
             * cleared a poll event.
             */
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 778) - [public] if

```
    if (empty) {
        /* here we have a race condition:
         * if period event arrived after the avail_update call
         * above we might clear this event with the following
         * clear_timer_queue.
         * There is no way to do this in atomic manner, so we
         * need to recheck avail_update if we successfully
         * cleared a poll event.
         */
        if (snd_pcm_direct_clear_timer_queue(dmix))
            goto timer_changed;
        events &= ~(POLLOUT|POLLIN);
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 791) - [public] switch

```
/* here we have a race condition:
 * if period event arrived after the avail_update call
 * above we might clear this event with the following
 * clear_timer_queue.
 * There is no way to do this in atomic manner, so we
 * need to recheck avail_update if we successfully
 * cleared a poll event.
 */
if (snd_pcm_direct_clear_timer_queue(dmix))
goto timer_changed;
events &= ~(POLLOUT|POLLIN);
/* additional check */
```

```
switch (__snd_pcm_state(pcm)) {
case SND_PCM_STATE_XRUN:
case SND_PCM_STATE_SUSPENDED:
case SND_PCM_STATE_SETUP:
events |= POLLERR;
break;
default:
break;
}
}
}
*revents = events;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 811) - [public] snd_pcm_info

```
return snd_pcm_info(dmix->spcm, info);

memset(info, 0, sizeof(*info));
info->stream = pcm->stream;
info->card = -1;
/* FIXME: fill this with something more useful: we know the hardware name */
if (pcm->name) {
snd_strncpy((char *)info->id, pcm->name, sizeof(info->id));
snd_strncpy((char *)info->name, pcm->name, sizeof(info->name));
snd_strncpy((char *)info->subname, pcm->name, sizeof(info->subname));
}
info->subdevices_count = 1;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 818) - [public] snd_strncpy

```
snd_strncpy((char *)info->id, pcm->name, sizeof(info->id));
snd_strncpy((char *)info->name, pcm->name, sizeof(info->name));
snd_strncpy((char *)info->subname, pcm->name, sizeof(info->subname));
}
info->subdevices_count = 1;
return 0;
}

static inline snd_mask_t *hw_param_mask(snd_pcm_hw_params_t *params,
snd_pcm_hw_param_t var)
{
return &params->masks[var - SND_PCM_HW_PARAM_FIRST_MASK];
// ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 819) - [public] snd_strlcpy
    ■■snd_strlcpy((char *)info->name, pcm->name, sizeof(info->name));
    ■■snd_strlcpy((char *)info->subname, pcm->name, sizeof(info->subname));
    ■}
    ■info->subdevices_count = 1;
    ■return 0;
    }

    static inline snd_mask_t *hw_param_mask(snd_pcm_hw_params_t *params,
    ■■■■■snd_pcm_hw_param_t var)
    {
    ■return &params->masks[var - SND_PCM_HW_PARAM_FIRST_MASK];
    }

    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 820) - [public] snd_strlcpy
    ■■snd_strlcpy((char *)info->subname, pcm->name, sizeof(info->subname));
    ■}
    ■info->subdevices_count = 1;
    ■return 0;
    }

    static inline snd_mask_t *hw_param_mask(snd_pcm_hw_params_t *params,
    ■■■■■snd_pcm_hw_param_t var)
    {
    ■return &params->masks[var - SND_PCM_HW_PARAM_FIRST_MASK];
    }

    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 848) - [public] snd_error
    ■■snd_error(PCM, "dshare interval %i empty?", (int)var);
    ■■return -EINVAL;
    ■}
    ■if (snd_interval_refine(i, src))
    ■■params->cmask |= 1<<var;
    ■return 0;
    }

    static int hw_param_interval_refine_minmax(snd_pcm_hw_params_t *params,
    ■■■■■ snd_pcm_hw_param_t var,
    ■■■■■ unsigned int imin,
    ■■■■■ unsigned int imax)
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 866) - [public] hw_param_interval_refine_one
    ■return hw_param_interval_refine_one(params, var, &t);
    }

    /* this code is used 'as-is' from the alsa kernel code */
    static int snd_interval_step(struct snd_interval *i, unsigned int min,
    ■■■ unsigned int step)
    {
    ■unsigned int n;
    ■int changed = 0;
    ■n = (i->min - min) % step;
    ■if (n != 0 || i->openmin) {
    ■■i->min += step - n;
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 912) - [public] if

```
    if (snd_mask_empty(hw_param_mask(params, SND_PCM_HW_PARAM_ACCESS))) {
        snd_error(PCM, "dshare access mask empty?");
        return -EINVAL;
    }
    if (snd_mask_refine(hw_param_mask(params, SND_PCM_HW_PARAM_ACCESS), &access))
        params->cmask |= 1<<SND_PCM_HW_PARAM_ACCESS;
    }
    if (params->rmask & (1<<SND_PCM_HW_PARAM_FORMAT)) {
        if (snd_mask_empty(hw_param_mask(params, SND_PCM_HW_PARAM_FORMAT))) {
            snd_error(PCM, "dshare format mask empty?");
            return -EINVAL;
        }
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 913) - [public] snd_error

```
    snd_error(PCM, "dshare access mask empty?");
    return -EINVAL;
}
if (snd_mask_refine(hw_param_mask(params, SND_PCM_HW_PARAM_ACCESS), &access))
    params->cmask |= 1<<SND_PCM_HW_PARAM_ACCESS;
}
if (params->rmask & (1<<SND_PCM_HW_PARAM_FORMAT)) {
    if (snd_mask_empty(hw_param_mask(params, SND_PCM_HW_PARAM_FORMAT))) {
        snd_error(PCM, "dshare format mask empty?");
        return -EINVAL;
    }
    if (snd_mask_refine_set(hw_param_mask(params, SND_PCM_HW_PARAM_FORMAT),
        // ... (truncated, see source file for more)
    )
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 920) - [public] if

```
    if (snd_mask_empty(hw_param_mask(params, SND_PCM_HW_PARAM_FORMAT))) {
        snd_error(PCM, "dshare format mask empty?");
        return -EINVAL;
    }
    if (snd_mask_refine_set(hw_param_mask(params, SND_PCM_HW_PARAM_FORMAT),
        dshare->shmptr->hw.format))
        params->cmask |= 1<<SND_PCM_HW_PARAM_FORMAT;
    }
    //snd_mask_none(hw_param_mask(params, SND_PCM_HW_PARAM_SUBFORMAT));
    if (params->rmask & (1<<SND_PCM_HW_PARAM_CHANNELS)) {
        if (snd_interval_empty(hw_param_interval(params, SND_PCM_HW_PARAM_CHANNELS))) {
            snd_error(PCM, "dshare channels mask empty?");
            // ... (truncated, see source file for more)
        }
    }
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 921) - [public] snd_error

```
    snd_error(PCM, "dshare format mask empty?");
    return -EINVAL;
}
if (snd_mask_refine_set(hw_param_mask(params, SND_PCM_HW_PARAM_FORMAT),
    dshare->shmptr->hw.format))
    params->cmask |= 1<<SND_PCM_HW_PARAM_FORMAT;
}
//snd_mask_none(hw_param_mask(params, SND_PCM_HW_PARAM_SUBFORMAT));
if (params->rmask & (1<<SND_PCM_HW_PARAM_CHANNELS)) {
    if (snd_interval_empty(hw_param_interval(params, SND_PCM_HW_PARAM_CHANNELS))) {
        snd_error(PCM, "dshare channels mask empty?");
        return -EINVAL;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 930) - [public] if

```
    if (snd_interval_empty(hw_param_interval(params, SND_PCM_HW_PARAM_CHANNELS))) {
        snd_error(PCM, "dshare channels mask empty?");
        return -EINVAL;
    }
    err = snd_interval_refine_set(hw_param_interval(params, SND_PCM_HW_PARAM_CHANNELS), dshare->channels);
    if (err < 0)
        return err;
}
err = hw_param_interval_refine_one(params, SND_PCM_HW_PARAM_RATE,
    &dshare->shmptr->hw.rate);
if (err < 0)
    return err;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 931) - [public] snd_error

```
    snd_error(PCM, "dshare channels mask empty?");
    return -EINVAL;
}
err = snd_interval_refine_set(hw_param_interval(params, SND_PCM_HW_PARAM_CHANNELS), dshare->channels);
if (err < 0)
    return err;
}
err = hw_param_interval_refine_one(params, SND_PCM_HW_PARAM_RATE,
    &dshare->shmptr->hw.rate);
if (err < 0)
    return err;

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 979) - [public] if

```
    if (dshare->var_periodsize) {
        /* more tolerant settings... */
        if (dshare->shmptr->hw.buffer_size.max / 2 > period_size.max) {
            period_size.max = dshare->shmptr->hw.buffer_size.max / 2;
            period_size.openmax = dshare->shmptr->hw.buffer_size.openmax;
        }
        if (dshare->shmptr->hw.buffer_time.max / 2 > period_time.max) {
            period_time.max = dshare->shmptr->hw.buffer_time.max / 2;
            period_time.openmax = dshare->shmptr->hw.buffer_time.openmax;
        }
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 981) - [public] if

```
/* make sure buffer size does not exceed slave buffer size */
err = hw_param_interval_refine_minmax(params, SND_PCM_HW_PARAM_BUFFER_SIZE,
    2 * dshare->slave_period_size, dshare->slave_buffer_size);
if (err < 0)
    return err;
if (dshare->var_periodsize) {
    /* more tolerant settings... */
    if (dshare->shmptr->hw.buffer_size.max / 2 > period_size.max) {
        period_size.max = dshare->shmptr->hw.buffer_size.max / 2;
        period_size.openmax = dshare->shmptr->hw.buffer_size.openmax;
    }
    if (dshare->shmptr->hw.buffer_time.max / 2 > period_time.max) {
        period_time.max = dshare->shmptr->hw.buffer_time.max / 2;
        period_time.openmax = dshare->shmptr->hw.buffer_time.openmax;
    }
}

err = hw_param_interval_refine_one(params, SND_PCM_HW_PARAM_PERIOD_SIZE,
    &period_size);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 985) - [public] if

```
    if (dshare->shmptr->hw.buffer_time.max / 2 > period_time.max) {
        period_time.max = dshare->shmptr->hw.buffer_time.max / 2;
        period_time.openmax = dshare->shmptr->hw.buffer_time.openmax;
    }

    err = hw_param_interval_refine_one(params, SND_PCM_HW_PARAM_PERIOD_SIZE,
        &period_size);
    if (err < 0)
        return err;
    err = hw_param_interval_refine_one(params, SND_PCM_HW_PARAM_PERIOD_TIME,
        &period_time);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1060) - [public] snd_pcm_channel_info_shm

```
    return snd_pcm_channel_info_shm(pcm, info, -1);
}

int snd_pcm_direct_mmap(snd_pcm_t *pcm ATTRIBUTE_UNUSED)
{
    return 0;
}

int snd_pcm_direct_munmap(snd_pcm_t *pcm ATTRIBUTE_UNUSED)
{
    return 0;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1080) - [public] snd_pcm_query_chmaps

```
    return snd_pcm_query_chmaps(dmix->spcm);

    maps = calloc(2, sizeof(*maps));
    if (!maps)
        return NULL;
    maps[0] = calloc(dmix->channels + 2, sizeof(int *));
    if (!maps[0]) {
        free(maps);
        return NULL;
    }
    smaps = snd_pcm_query_chmaps(dmix->spcm);
    if (smaps == NULL) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1087) - [public] free

```
    free(maps);
    return NULL;
}

smaps = snd_pcm_query_chmaps(dmix->spcm);
if (smaps == NULL) {
    snd_pcm_free_chmaps(maps);
    return NULL;
}
maps[0]->type = SND_CHMAP_TYPE_FIXED;
maps[0]->map.channels = dmix->channels;
for (i = 0; i < dmix->channels; i++) {
    j = dmix->bindings[i];
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1092) - [public] snd_pcm_free_chmaps

```
    ■■snd_pcm_free_chmaps(maps);
    ■■return NULL;
    ■}
    ■maps[0]->type = SND_CHMAP_TYPE_FIXED;
    ■maps[0]->map.channels = dmix->channels;
    ■for (i = 0; i < dmix->channels; i++) {
    ■■j = dmix->bindings[i];
    ■■if (j == UINT_MAX || smaps[0]->map.channels < j)
    ■■■continue;
    ■■maps[0]->map.pos[i] = smaps[0]->map.pos[j];
    ■}
    ■return maps;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1109) - [public] snd_pcm_get_chmap

```
    ■return snd_pcm_get_chmap(dmix->spcm);
    }

    int snd_pcm_direct_set_chmap(snd_pcm_t *pcm, const snd_pcm_chmap_t *map)
    {
    ■snd_pcm_direct_t *dmix = pcm->private_data;
    ■return snd_pcm_set_chmap(dmix->spcm, map);
    }

    int snd_pcm_direct_prepare(snd_pcm_t *pcm)
    {
    ■snd_pcm_direct_t *dmix = pcm->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1115) - [public] snd_pcm_set_chmap

```
    ■return snd_pcm_set_chmap(dmix->spcm, map);
    }

    int snd_pcm_direct_prepare(snd_pcm_t *pcm)
    {
    ■snd_pcm_direct_t *dmix = pcm->private_data;
    ■int err;

    ■switch (snd_pcm_state(dmix->spcm)) {
    ■case SND_PCM_STATE_SETUP:
    ■case SND_PCM_STATE_XRUN:
    ■case SND_PCM_STATE_SUSPENDED:
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1130) - [public] snd_pcm_start

```
    ■■snd_pcm_start(dmix->spcm);
    ■■break;
    ■case SND_PCM_STATE_OPEN:
    ■case SND_PCM_STATE_DISCONNECTED:
    ■■return -EBADFD;
    ■default:
    ■■break;
    ■}
    ■snd_pcm_direct_check_interleave(dmix, pcm);
    ■dmix->state = SND_PCM_STATE_PREPARED;
    ■dmix->appl_ptr = dmix->last_appl_ptr = 0;
    ■dmix->hw_ptr = 0;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1142) - [public] snd_pcm_direct_set_timer_params

```
    return snd_pcm_direct_set_timer_params(dmix);
}

int snd_pcm_direct_resume(snd_pcm_t *pcm)
{
    snd_pcm_direct_t *dmix = pcm->private_data;
    int err;

    err = snd_pcm_direct_slave_recover(dmix);
    return err < 0 ? err : -ENOSYS;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1206) - [public] snd_error

```
    snd_error(PCM, "unable to find a valid configuration for slave");
    return -EINVAL;
}
ret = snd_pcm_hw_params_any(spcm, &hw_params);
if (ret < 0) {
    snd_error(PCM, "snd_pcm_hw_params_any failed");
    return ret;
}
ret = snd_pcm_hw_params_set_access(spcm, &hw_params,
    SND_PCM_ACCESS_MMAP_INTERLEAVED);
if (ret < 0) {
    ret = snd_pcm_hw_params_set_access(spcm, &hw_params,
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1211) - [public] snd_error

```
    snd_error(PCM, "snd_pcm_hw_params_any failed");
    return ret;
}
ret = snd_pcm_hw_params_set_access(spcm, &hw_params,
    SND_PCM_ACCESS_MMAP_INTERLEAVED);
if (ret < 0) {
    ret = snd_pcm_hw_params_set_access(spcm, &hw_params,
        SND_PCM_ACCESS_MMAP_NONINTERLEAVED);
    if (ret < 0) {
        snd_error(PCM, "slave plugin does not support mmap interleaved or mmap noninterleaved access");
        return ret;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1219) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "slave plugin does not support mmap interleaved or mmap noninterleaved access");
        return ret;
    }
    if (params->format == SND_PCM_FORMAT_UNKNOWN)
        ret = -EINVAL;
    else
        ret = snd_pcm_hw_params_set_format(spcm, &hw_params,
            params->format);
    if (ret < 0) {
        static const snd_pcm_format_t dmix_formats[] = {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1220) - [public] snd_error

```
    snd_error(PCM, "slave plugin does not support mmap interleaved or mmap noninterleaved access");
    return ret;
}
if (params->format == SND_PCM_FORMAT_UNKNOWN)
    ret = -EINVAL;
else
    ret = snd_pcm_hw_params_set_format(spcm, &hw_params,
        params->format);
if (ret < 0) {
    static const snd_pcm_format_t dmix_formats[] = {
        SND_PCM_FORMAT_S32,
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1251) - [public] if

```
if (ret < 0 && dmix->type != SND_PCM_TYPE_DMIX) {
    /* TODO: try to choose a good format */
    ret = INTERNAL(snd_pcm_hw_params_set_format_first)(spcm,
        &hw_params, &format);
}
if (ret < 0) {
    snd_error(PCM, "requested or auto-format is not available");
    return ret;
}
params->format = format;
}
ret = INTERNAL(snd_pcm_hw_params_set_channels_near)(spcm, &hw_params,
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1256) - [public] if

```
if (ret < 0) {
    snd_error(PCM, "requested or auto-format is not available");
    return ret;
}
params->format = format;
}
ret = INTERNAL(snd_pcm_hw_params_set_channels_near)(spcm, &hw_params,
    (unsigned int *)&params->channels);
if (ret < 0) {
    snd_error(PCM, "requested count of channels is not available");
    return ret;
}
// ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1257) - [public] snd_error

```
    snd_error(PCM, "requested or auto-format is not available");
    return ret;
}
params->format = format;
}
ret = INTERNAL(snd_pcm_hw_params_set_channels_near)(spcm, &hw_params,
    (unsigned int *)&params->channels);
if (ret < 0) {
    snd_error(PCM, "requested count of channels is not available");
    return ret;
}
ret = INTERNAL(snd_pcm_hw_params_set_rate_near)(spcm, &hw_params,
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1265) - [public] snd_error

```
    ■■snd_error(PCM, "requested count of channels is not available");
    ■■return ret;
    ■}
    ■ret = INTERNAL(snd_pcm_hw_params_set_rate_near)(spcm, &hw_params,
    ■■■■(unsigned int *)&params->rate, 0);
    ■if (ret < 0) {
    ■■snd_error(PCM, "requested rate is not available");
    ■■return ret;
    ■}

    ■buffer_is_not_initialized = 0;
    ■if (params->buffer_time > 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1271) - [public] snd_error

```
    ■■snd_error(PCM, "requested rate is not available");
    ■■return ret;
    ■}

    ■buffer_is_not_initialized = 0;
    ■if (params->buffer_time > 0) {
    ■■ret = INTERNAL(snd_pcm_hw_params_set_buffer_time_near)(spcm,
    ■■■&hw_params, (unsigned int *)&params->buffer_time, 0);
    ■■if (ret < 0) {
    ■■■snd_error(PCM, "unable to set buffer time");
    ■■■return ret;
    ■■}
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1279) - [public] if

```
    ■■if (ret < 0) {
    ■■■snd_error(PCM, "unable to set buffer time");
    ■■■return ret;
    ■■}
    ■} else if (params->buffer_size > 0) {
    ■■ret = INTERNAL(snd_pcm_hw_params_set_buffer_size_near)(spcm,
    ■■■&hw_params, (snd_pcm_uframes_t *)&params->buffer_size);
    ■■if (ret < 0) {
    ■■■snd_error(PCM, "unable to set buffer size");
    ■■■return ret;
    ■■}
    ■} else {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1280) - [public] snd_error

```
    ■■■snd_error(PCM, "unable to set buffer time");
    ■■■return ret;
    ■■}
    ■} else if (params->buffer_size > 0) {
    ■■ret = INTERNAL(snd_pcm_hw_params_set_buffer_size_near)(spcm,
    ■■■&hw_params, (snd_pcm_uframes_t *)&params->buffer_size);
    ■■if (ret < 0) {
    ■■■snd_error(PCM, "unable to set buffer size");
    ■■■return ret;
    ■■}
    ■} else {
    ■■buffer_is_not_initialized = 1;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1286) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "unable to set buffer size");
        return ret;
    }
    else {
        buffer_is_not_initialized = 1;
    }

    if (params->period_time > 0) {
        ret = INTERNAL(snd_pcm_hw_params_set_period_time_near)(spcm,
            &hw_params, (unsigned int *)&params->period_time, 0);
        if (ret < 0) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1287) - [public] snd_error

```
    snd_error(PCM, "unable to set buffer size");
    return ret;
}
else {
    buffer_is_not_initialized = 1;
}

if (params->period_time > 0) {
    ret = INTERNAL(snd_pcm_hw_params_set_period_time_near)(spcm,
        &hw_params, (unsigned int *)&params->period_time, 0);
    if (ret < 0) {
        snd_error(PCM, "unable to set period_time");
        // ... (truncated, see source file for more)
    }
}
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1297) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "unable to set period_time");
        return ret;
    }
    else if (params->period_size > 0) {
        ret = INTERNAL(snd_pcm_hw_params_set_period_size_near)(spcm,
            &hw_params, (snd_pcm_uframes_t *)&params->period_size,
            0);
        if (ret < 0) {
            snd_error(PCM, "unable to set period_size");
            return ret;
        }
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1298) - [public] snd_error

```
    snd_error(PCM, "unable to set period_time");
    return ret;
}
else if (params->period_size > 0) {
    ret = INTERNAL(snd_pcm_hw_params_set_period_size_near)(spcm,
        &hw_params, (snd_pcm_uframes_t *)&params->period_size,
        0);
    if (ret < 0) {
        snd_error(PCM, "unable to set period_size");
        return ret;
    }
}
// ... (truncated, see source file for more)
}
```


dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1305) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "unable to set period_size");
        return ret;
    }

    if (buffer_is_not_initialized && params->periods > 0) {
        unsigned int periods = params->periods;
        ret = INTERNAL(snd_pcm_hw_params_set_periods_near)(spcm,
            &hw_params, &params->periods, 0);
        if (ret < 0) {
            snd_error(PCM, "unable to set requested periods");
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1306) - [public] snd_error

```
    snd_error(PCM, "unable to set period_size");
    return ret;
}

if (buffer_is_not_initialized && params->periods > 0) {
    unsigned int periods = params->periods;
    ret = INTERNAL(snd_pcm_hw_params_set_periods_near)(spcm,
        &hw_params, &params->periods, 0);
    if (ret < 0) {
        snd_error(PCM, "unable to set requested periods");
        return ret;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1315) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "unable to set requested periods");
        return ret;
    }
    if (params->periods == 1) {
        params->periods = periods;
        if (params->period_time > 0) {
            params->period_time /= 2;
            goto __again;
        } else if (params->period_size > 0) {
            params->period_size /= 2;
            goto __again;
        }
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1316) - [public] snd_error

```
    snd_error(PCM, "unable to set requested periods");
    return ret;
}
if (params->periods == 1) {
    params->periods = periods;
    if (params->period_time > 0) {
        params->period_time /= 2;
        goto __again;
    } else if (params->period_size > 0) {
        params->period_size /= 2;
        goto __again;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1319) - [public] if

```
    if (params->periods == 1) {
        params->periods = periods;
        if (params->period_time > 0) {
            params->period_time /= 2;
            goto __again;
        } else if (params->period_size > 0) {
            params->period_size /= 2;
            goto __again;
        }
        snd_error(PCM, "unable to use stream with periods == 1");
        return ret;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1321) - [public] if

```
    if (params->period_time > 0) {
        params->period_time /= 2;
        goto __again;
    } else if (params->period_size > 0) {
        params->period_size /= 2;
        goto __again;
    }
    snd_error(PCM, "unable to use stream with periods == 1");
    return ret;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1328) - [public] snd_error

```
    snd_error(PCM, "unable to use stream with periods == 1");
    return ret;
}

ret = snd_pcm_hw_params(spcm, &hw_params);
if (ret < 0) {
    snd_error(PCM, "unable to install hw params");
    return ret;
}

/* store some hw_params values to shared info */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1335) - [public] snd_error

```
    snd_error(PCM, "unable to install hw params");
    return ret;
}

/* store some hw_params values to shared info */
dmix->shmptr->hw.format =
    snd_mask_value(hw_param_mask(&hw_params,
        SND_PCM_HW_PARAM_FORMAT));
dmix->shmptr->hw.rate =
    *hw_param_interval(&hw_params, SND_PCM_HW_PARAM_RATE);
dmix->shmptr->hw.buffer_size =
    *hw_param_interval(&hw_params, SND_PCM_HW_PARAM_BUFFER_SIZE);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1359) - [public] snd_error

```
    ■■snd_error(PCM, "unable to get current sw_params");
    ■■return ret;
    ■}

    ■ret = snd_pcm_sw_params_get_boundary(&sw_params, &boundary);
    ■if (ret < 0) {
    ■■snd_error(PCM, "unable to get boundary");
    ■■return ret;
    ■}
    ■ret = snd_pcm_sw_params_set_stop_threshold(spcm, &sw_params, boundary);
    ■if (ret < 0) {
    ■■snd_error(PCM, "unable to set stop threshold");
    ■    // ... (truncated, see source file for more)
    ■}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1365) - [public] snd_error

```
    ■■snd_error(PCM, "unable to get boundary");
    ■■return ret;
    ■}
    ■ret = snd_pcm_sw_params_set_stop_threshold(spcm, &sw_params, boundary);
    ■if (ret < 0) {
    ■■snd_error(PCM, "unable to set stop threshold");
    ■■return ret;
    ■}

    ■/* set timestamp mode to MMAP
    ■ * the slave timestamp is copied appropriately in dsnoop/dmix/dshare
    ■ * based on the tstamp_mode of each client
    ■    // ... (truncated, see source file for more)
    ■*/
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1370) - [public] snd_error

```
    ■■snd_error(PCM, "unable to set stop threshold");
    ■■return ret;
    ■}

    ■/* set timestamp mode to MMAP
    ■ * the slave timestamp is copied appropriately in dsnoop/dmix/dshare
    ■ * based on the tstamp_mode of each client
    ■ */
    ■ret = snd_pcm_sw_params_set_tstamp_mode(spcm, &sw_params,
    ■■■■■■■■SND_PCM_TSTAMP_ENABLE);
    ■if (ret < 0) {
    ■■snd_error(PCM, "unable to tstamp mode MMAP");
    ■    // ... (truncated, see source file for more)
    ■}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1381) - [public] snd_error

```
    ■■snd_error(PCM, "unable to tstamp mode MMAP");
    ■■return ret;
    ■}

    ■if (dmix->tstamp_type != -1) {
    ■■ret = snd_pcm_sw_params_set_tstamp_type(spcm, &sw_params,
    ■■■■■■■■dmix->tstamp_type);
    ■■if (ret < 0) {
    ■■■snd_error(PCM, "unable to set tstamp type");
    ■■■return ret;
    ■■}
    ■}

    ■    // ... (truncated, see source file for more)
    ■}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1388) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "unable to set tstamp type");
        return ret;
    }

    if (dmix->type != SND_PCM_TYPE_DMIX &&
        dmix->type != SND_PCM_TYPE_DSHARE)
        goto __skip_silencing;

    ret = snd_pcm_sw_params_set_silence_threshold(spcm, &sw_params, 0);
    if (ret < 0) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1389) - [public] snd_error

```
        snd_error(PCM, "unable to set tstamp type");
        return ret;
    }

    if (dmix->type != SND_PCM_TYPE_DMIX &&
        dmix->type != SND_PCM_TYPE_DSHARE)
        goto __skip_silencing;

    ret = snd_pcm_sw_params_set_silence_threshold(spcm, &sw_params, 0);
    if (ret < 0) {
        snd_error(PCM, "unable to set silence threshold");
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1400) - [public] snd_error

```
        snd_error(PCM, "unable to set silence threshold");
        return ret;
    }

    ret = snd_pcm_sw_params_set_silence_size(spcm, &sw_params, boundary);
    if (ret < 0) {
        snd_error(PCM, "unable to set silence threshold (please upgrade to 0.9.0rc8+ driver)");
        return ret;
    }
```

__skip_silencing:

```
    ret = snd_pcm_sw_params(spcm, &sw_params);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1405) - [public] snd_error

```
        snd_error(PCM, "unable to set silence threshold (please upgrade to 0.9.0rc8+ driver)");
        return ret;
    }
```

__skip_silencing:

```
    ret = snd_pcm_sw_params(spcm, &sw_params);
    if (ret < 0) {
        snd_error(PCM, "unable to install sw params (please upgrade to 0.9.0rc8+ driver)");
        return ret;
    }
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1413) - [public] snd_error

```
    ■■snd_error(PCM, "unable to install sw params (please upgrade to 0.9.0rc8+ driver)");
    ■■return ret;
    ■}

    ■if (dmix->type == SND_PCM_TYPE_DSHARE) {
    ■■const snd_pcm_channel_area_t *dst_areas;
    ■■dst_areas = snd_pcm_mmap_areas(spcm);
    ■■snd_pcm_areas_silence(dst_areas, 0, spcm->channels,
    ■■■■    spcm->buffer_size, spcm->format);
    ■}

    ■ret = snd_pcm_start(spcm);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1426) - [public] snd_error

```
    ■■snd_error(PCM, "unable to start PCM stream");
    ■■return ret;
    ■}

    ■if (snd_pcm_poll_descriptors_count(spcm) != 1) {
    ■■snd_error(PCM, "unable to use hardware pcm with fd more than one!!!");
    ■■return ret;
    ■}
    ■snd_pcm_poll_descriptors(spcm, &fd, 1);
    ■dmix->hw_fd = fd.fd;

    ■save_slave_setting(dmix, spcm);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1431) - [public] snd_error

```
    ■■snd_error(PCM, "unable to use hardware pcm with fd more than one!!!");
    ■■return ret;
    ■}
    ■snd_pcm_poll_descriptors(spcm, &fd, 1);
    ■dmix->hw_fd = fd.fd;

    ■save_slave_setting(dmix, spcm);

    ■/* Currently, we assume that each dmix client has the same
    ■ * hw_params setting.
    ■ * If the arbitrary hw_params is supported in future,
    ■ * boundary has to be taken from the slave config but
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1454) - [public] ioctl

```
    ■■ioctl(spcm->poll_fd, SNDRV_PCM_IOCTL_PVERSION, &ver);
    ■■if (ver < SNDRV_PROTOCOL_VERSION(2, 0, 8))
    ■■■■dmix->shmptr->use_server = 1;
    ■}

    ■return 0;
    }

    /*
    * the trick is used here; we cannot use effectively the hardware handle because
    * we cannot drive multiple accesses to appl_ptr; so we use slave timer of given
    * PCM hardware handle; it's not this easy and cheap?
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1479) - [public] snd_error

```
    ■■snd_error(PCM, "unable to info for slave pcm");
    ■■return ret;
    ■}
    ■sprintf(name, "hw:CLASS=%i,SCLASS=0,CARD=%i,DEV=%i,SUBDEV=%i",
    ■■(int)SND_TIMER_CLASS_PCM,
    ■■snd_pcm_info_get_card(&info),
    ■■snd_pcm_info_get_device(&info),
    ■■snd_pcm_info_get_subdevice(&info) * 2 + capture);
    ■ret = snd_timer_open(&dmix->timer, name,
    ■■SND_TIMER_OPEN_NONBLOCK | SND_TIMER_OPEN_TREAD);
    ■if (ret < 0) {
    ■■dmix->tread = 0;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1486) - [public] snd_pcm_info_get_subdevice

```
    ■■snd_pcm_info_get_subdevice(&info) * 2 + capture);
    ■ret = snd_timer_open(&dmix->timer, name,
    ■■SND_TIMER_OPEN_NONBLOCK | SND_TIMER_OPEN_TREAD);
    ■if (ret < 0) {
    ■■dmix->tread = 0;
    ■■ret = snd_timer_open(&dmix->timer, name,
    ■■■SND_TIMER_OPEN_NONBLOCK);
    ■■if (ret < 0) {
    ■■■snd_error(PCM, "unable to open timer '%s'", name);
    ■■■return ret;
    ■■}
    ■}
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1493) - [public] if

```
    ■■if (ret < 0) {
    ■■■snd_error(PCM, "unable to open timer '%s'", name);
    ■■■return ret;
    ■■}
    ■}

    ■if (snd_timer_poll_descriptors_count(dmix->timer) != 1) {
    ■■snd_error(PCM, "unable to use timer '%s' with more than one fd!", name);
    ■■return ret;
    ■}
    ■snd_timer_poll_descriptors(dmix->timer, &dmix->timer_fd, 1);
    ■dmix->poll_fd = dmix->timer_fd.fd;
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1494) - [public] snd_error

```
    ■■■snd_error(PCM, "unable to open timer '%s'", name);
    ■■■return ret;
    ■■}
    ■}

    ■if (snd_timer_poll_descriptors_count(dmix->timer) != 1) {
    ■■snd_error(PCM, "unable to use timer '%s' with more than one fd!", name);
    ■■return ret;
    ■}
    ■snd_timer_poll_descriptors(dmix->timer, &dmix->timer_fd, 1);
    ■dmix->poll_fd = dmix->timer_fd.fd;

    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1500) - [public] snd_error

```
    snd_error(PCM, "unable to use timer '%s' with more than one fd!", name);
    return ret;
}
snd_timer_poll_descriptors(dmix->timer, &dmix->timer_fd, 1);
dmix->poll_fd = dmix->timer_fd.fd;

dmix->timer_events = (1<<SND_TIMER_EVENT_MSUSPEND) |
    (1<<SND_TIMER_EVENT_MRESUME) |
    (1<<SND_TIMER_EVENT_MSTOP) |
    (1<<SND_TIMER_EVENT_STOP);

/*
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1516) - [public] ioctl

```
    ioctl(dmix->poll_fd, SNDRV_TIMER_IOCTL_PVERSION, &ver);
    /* In older versions, check via poll before read() is needed
    * because of the confliction between TIMER_START and
    * FIONBIO ioctls.
    */
    if (ver < SNDRV_PROTOCOL_VERSION(2, 0, 4))
        dmix->timer_need_poll = 1;
    /*
    * In older versions, timer uses pause events instead
    * suspend/resume events.
    */
    if (ver < SNDRV_PROTOCOL_VERSION(2, 0, 5)) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1527) - [public] if

```
/*
    * In older versions, timer uses pause events instead
    * suspend/resume events.
    */

    if (ver < SNDRV_PROTOCOL_VERSION(2, 0, 5)) {
        dmix->timer_events &= ~((1<<SND_TIMER_EVENT_MSUSPEND) |
            (1<<SND_TIMER_EVENT_MRESUME));
        dmix->timer_events |= (1<<SND_TIMER_EVENT_MPAUSE) |
            (1<<SND_TIMER_EVENT_MCONTINUE);
    }
    /* In older versions, use SND_TIMER_EVENT_START too.
    */
    if (ver < SNDRV_PROTOCOL_VERSION(2, 0, 6))
        dmix->timer_events |= 1<<SND_TIMER_EVENT_START;
}
return 0;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1601) - [public] snd_error

```
    snd_error(PCM, "unable to open hardware");
    return ret;
}

spcm = *spcmp;
spcm->donot_close = 1;
spcm->setup = 1;

copy_slave_setting(dmix, spcm);

/* Use the slave setting as SPCM, so far */
dmix->slave_buffer_size = spcm->buffer_size;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1619) - [public] snd_error

```
    ■■snd_error(PCM, "unable to mmap channels");
    ■■return ret;
    ■}
    ■return 0;
    }

    /*
     * open a slave PCM as secondary client (dup'ed fd)
     */
    int snd_pcm_direct_initialize_secondary_slave(snd_pcm_direct_t *dmix,
    ■■■■■■    snd_pcm_t *spcm,
    ■■■■■■    struct slave_params *params ATTRIBUTE_UNUSED)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1646) - [public] snd_error

```
    ■■snd_error(PCM, "unable to mmap channels");
    ■■return ret;
    ■}
    ■return 0;
    }

    int snd_pcm_direct_set_timer_params(snd_pcm_direct_t *dmix)
    {
    ■■snd_timer_params_t params = {0};
    ■■unsigned int filter;
    ■■int ret;

        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1660) - [public]
snd_timer_params_set_early_event

```
    ■■snd_timer_params_set_early_event(&params, 1);
    ■■snd_timer_params_set_ticks(&params, dmix->timer_ticks);
    ■■if (dmix->tread) {
    ■■filter = (1<<SND_TIMER_EVENT_TICK) |
    ■■■■ dmix->timer_events;
    ■■INTERNAL(snd_timer_params_set_filter>(&params, filter);
    ■}
    ■ret = snd_timer_params(dmix->timer, &params);
    ■■if (ret < 0) {
    ■■snd_error(PCM, "unable to set timer parameters");
    ■■return ret;
    ■}

        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1665) - [public] INTERNAL

```
    ■■INTERNAL(snd_timer_params_set_filter>(&params, filter);
    ■}
    ■ret = snd_timer_params(dmix->timer, &params);
    ■■if (ret < 0) {
    ■■snd_error(PCM, "unable to set timer parameters");
    ■■return ret;
    ■}
    ■return 0;
    }

    /*
     * ring buffer operation
     // ... (truncated, see source file for more)
```



```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1669) - [public] snd_error
    ■■snd_error(PCM, "unable to set timer parameters");
    ■■return ret;
    ■}
    ■return 0;
    ■}

    /*
     * ring buffer operation
     */
    int snd_pcm_direct_check_interleave(snd_pcm_direct_t *dmix, snd_pcm_t *pcm)
    {
        ■unsigned int chn, channels;
        // ... (truncated, see source file for more)
    }

```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1732) - [public] snd_error
    ■■snd_error(PCM, "invalid type for bindings");
    ■■return -EINVAL;
    ■}
    ■snd_config_for_each(i, next, cfg) {
        ■■snd_config_t *n = snd_config_iterator_entry(i);
        ■■const char *id;
        ■■long cchannel;
        ■■if (snd_config_get_id(n, &id) < 0)
            ■■■continue;
        ■■err = safe_strtol(id, &cchannel);
        ■■if (err < 0 || cchannel < 0) {
            ■■■snd_error(PCM, "invalid client channel in binding: %s", id);
            // ... (truncated, see source file for more)
        }
    }

```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1742) - [public] if
    ■■if (err < 0 || cchannel < 0) {
        ■■■snd_error(PCM, "invalid client channel in binding: %s", id);
        ■■■return -EINVAL;
        ■■}
    ■■if ((unsigned)cchannel >= count)
        ■■■count = cchannel + 1;
    ■}
    ■if (count == 0)
        ■■return 0;
    ■if (count > 1024) {
        ■■snd_error(PCM, "client channel out of range");
        ■■return -EINVAL;
        // ... (truncated, see source file for more)
    }

```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1743) - [public] snd_error
    ■■■snd_error(PCM, "invalid client channel in binding: %s", id);
    ■■■return -EINVAL;
    ■■}
    ■■if ((unsigned)cchannel >= count)
        ■■■count = cchannel + 1;
    ■}
    ■if (count == 0)
        ■■return 0;
    ■if (count > 1024) {
        ■■snd_error(PCM, "client channel out of range");
        ■■return -EINVAL;
        ■}
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1752) - [public] snd_error

```
    snd_error(PCM, "client channel out of range");
    return -EINVAL;
}
bindings = malloc(count * sizeof(unsigned int));
if (bindings == NULL)
    return -ENOMEM;
for (chn = 0; chn < count; chn++)
    bindings[chn] = UINT_MAX; /* don't route */
snd_config_for_each(i, next, cfg) {
    snd_config_t *n = snd_config_iterator_entry(i);
    const char *id;
    long cchannel, schannel;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1767) - [public] if

```
    if (err < 0) {
        free(bindings);
        return -EINVAL;
    }
    if (snd_config_get_integer(n, &schannel) < 0) {
        snd_error(PCM, "unable to get slave channel (should be integer type) in binding: %s", id);
        free(bindings);
        return -EINVAL;
    }
    if (schannel < 0 || schannel >= params->channels) {
        snd_error(PCM, "invalid slave channel number %ld in binding to %ld",
            schannel, cchannel);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1768) - [public] free

```
        free(bindings);
        return -EINVAL;
    }
    if (snd_config_get_integer(n, &schannel) < 0) {
        snd_error(PCM, "unable to get slave channel (should be integer type) in binding: %s", id);
        free(bindings);
        return -EINVAL;
    }
    if (schannel < 0 || schannel >= params->channels) {
        snd_error(PCM, "invalid slave channel number %ld in binding to %ld",
            schannel, cchannel);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1771) - [public] if

```
    if (snd_config_get_integer(n, &schannel) < 0) {
        snd_error(PCM, "unable to get slave channel (should be integer type) in binding: %s", id);
        free(bindings);
        return -EINVAL;
    }
    if (schannel < 0 || schannel >= params->channels) {
        snd_error(PCM, "invalid slave channel number %ld in binding to %ld",
            schannel, cchannel);
        free(bindings);
        return -EINVAL;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1772) - [public] snd_error

```
    snd_error(PCM, "unable to get slave channel (should be integer type) in binding: %s", id);
    free(bindings);
    return -EINVAL;
}
if (schannel < 0 || schannel >= params->channels) {
    snd_error(PCM, "invalid slave channel number %ld in binding to %ld",
        schannel, cchannel);

    free(bindings);
    return -EINVAL;
}
bindings[cchannel] = schannel;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1773) - [public] free

```
    free(bindings);
    return -EINVAL;
}
if (schannel < 0 || schannel >= params->channels) {
    snd_error(PCM, "invalid slave channel number %ld in binding to %ld",
        schannel, cchannel);

    free(bindings);
    return -EINVAL;
}
bindings[cchannel] = schannel;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1776) - [public] if

```
    if (schannel < 0 || schannel >= params->channels) {
        snd_error(PCM, "invalid slave channel number %ld in binding to %ld",
            schannel, cchannel);

        free(bindings);
        return -EINVAL;
    }
    bindings[cchannel] = schannel;
}
if (dmix->type == SND_PCM_TYPE_DSNOOP ||
    ! dmix->bindings)
    goto __skip_same_dst;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1780) - [public] free

```
    free(bindings);
    return -EINVAL;
}
bindings[cchannel] = schannel;
}
if (dmix->type == SND_PCM_TYPE_DSNOOP ||
    ! dmix->bindings)
    goto __skip_same_dst;
for (chn = 0; chn < count; chn++) {
    for (chn1 = 0; chn1 < count; chn1++) {
        if (chn == chn1)
            continue;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1792) - [public] if

```
    if (bindings[chn] == dmix->bindings[chn1]) {
        snd_error(PCM, "unable to route channels %d,%d to same destination %d", chn, chn1, bindings[chn]);
        free(bindings);
        return -EINVAL;
    }
    __skip_same_dst:
    dmix->bindings = bindings;
    dmix->channels = count;
    return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1793) - [public] snd_error

```
    snd_error(PCM, "unable to route channels %d,%d to same destination %d", chn, chn1, bindings[chn]);
    free(bindings);
    return -EINVAL;
}
__skip_same_dst:
dmix->bindings = bindings;
dmix->channels = count;
return 0;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1794) - [public] free

```
    free(bindings);
    return -EINVAL;
}
__skip_same_dst:
dmix->bindings = bindings;
dmix->channels = count;
return 0;
}

/*
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1821) - [public] if

```
    if (hop > SND_CONF_MAX_HOPS) {
        snd_error(PCM, "Too many definition levels (looped?)");
        return -EINVAL;
    }
    err = snd_config_search_definition(root, "pcm", str, &pcm_conf);
    if (err < 0) {
        snd_error(PCM, "Unknown slave PCM %s", str);
        return err;
    }
    err = _snd_pcm_direct_get_slave_ipc_offset(root, pcm_conf,
        direction,
        hop + 1);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1822) - [public] snd_error

```
    snd_error(PCM, "Too many definition levels (looped?)");
    return -EINVAL;
}
err = snd_config_search_definition(root, "pcm", str, &pcm_conf);
if (err < 0) {
    snd_error(PCM, "Unknown slave PCM %s", str);
    return err;
}
err = _snd_pcm_direct_get_slave_ipc_offset(root, pcm_conf,
    direction,
    hop + 1);
snd_config_delete(pcm_conf);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1826) - [public] if

```
if (err < 0) {
    snd_error(PCM, "Unknown slave PCM %s", str);
    return err;
}
err = _snd_pcm_direct_get_slave_ipc_offset(root, pcm_conf,
    direction,
    hop + 1);
snd_config_delete(pcm_conf);
return err;
}
```

```
#if 0/* for debug purposes */
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1827) - [public] snd_error

```
    snd_error(PCM, "Unknown slave PCM %s", str);
    return err;
}
err = _snd_pcm_direct_get_slave_ipc_offset(root, pcm_conf,
    direction,
    hop + 1);
snd_config_delete(pcm_conf);
return err;
}
```

```
#if 0/* for debug purposes */
{
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1833) - [public] snd_config_delete

```
    snd_config_delete(pcm_conf);
    return err;
}

#if 0/* for debug purposes */
{
    snd_output_t *out;
    snd_output_stdio_attach(&out, stderr, 0);
    snd_config_save(sconf, out);
    snd_output_close(out);
}
#endif
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1840) - [public] snd_output_stdio_attach

```
    snd_output_stdio_attach(&out, stderr, 0);
    snd_config_save(sconf, out);
    snd_output_close(out);
}
#endif

if (snd_config_search(sconf, "slave", &pcm_conf) >= 0) {
    if (snd_config_search(pcm_conf, "pcm", &pcm_conf) >= 0) {
        return _snd_pcm_direct_get_slave_ipc_offset(root,
            pcm_conf,
            direction,
            hop + 1);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1841) - [public] snd_config_save

```
    snd_config_save(sconf, out);
    snd_output_close(out);
}
#endif

if (snd_config_search(sconf, "slave", &pcm_conf) >= 0) {
    if (snd_config_search(pcm_conf, "pcm", &pcm_conf) >= 0) {
        return _snd_pcm_direct_get_slave_ipc_offset(root,
            pcm_conf,
            direction,
            hop + 1);
    } else {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1842) - [public] snd_output_close

```
    snd_output_close(out);
}
#endif

if (snd_config_search(sconf, "slave", &pcm_conf) >= 0) {
    if (snd_config_search(pcm_conf, "pcm", &pcm_conf) >= 0) {
        return _snd_pcm_direct_get_slave_ipc_offset(root,
            pcm_conf,
            direction,
            hop + 1);
    } else {
        if (snd_config_get_string(pcm_conf, &str) >= 0 &&
            // ... (truncated, see source file for more)
        )
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1847) - [public] if

```
    if (snd_config_search(pcm_conf, "pcm", &pcm_conf) >= 0) {
        return _snd_pcm_direct_get_slave_ipc_offset(root,
            pcm_conf,
            direction,
            hop + 1);
    } else {
        if (snd_config_get_string(pcm_conf, &str) >= 0 &&
            snd_config_search_definition(root, "pcm_slave",
                str, &pcm_conf) >= 0) {
            if (snd_config_search(pcm_conf, "pcm",
                &pcm_conf2) >= 0) {
                err =
                // ... (truncated, see source file for more)
            }
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1861) - [public] snd_config_delete

```

    snd_config_delete(pcm_conf);
    return err;
}

snd_config_delete(pcm_conf);
}
}

snd_config_for_each(i, next, sconf) {
    snd_config_t *n = snd_config_iterator_entry(i);
    const char *id, *str;
    if (snd_config_get_id(n, &id) < 0)
        // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1864) - [public] snd_config_delete

```

    snd_config_delete(pcm_conf);
}
}

snd_config_for_each(i, next, sconf) {
    snd_config_t *n = snd_config_iterator_entry(i);
    const char *id, *str;
    if (snd_config_get_id(n, &id) < 0)
        continue;
    if (strcmp(id, "type") == 0) {
        err = snd_config_get_string(n, &str);
        // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1874) - [public] if

```

    if (strcmp(id, "type") == 0) {
        err = snd_config_get_string(n, &str);
        if (err < 0) {
            snd_error(PCM, "Invalid value for PCM type definition");
            return -EINVAL;
        }
        if (strcmp(str, "hw")) {
            snd_error(PCM, "Invalid type '%s' for slave PCM", str);
            return -EINVAL;
        }
        continue;
    }
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1876) - [public] if

```

    if (err < 0) {
        snd_error(PCM, "Invalid value for PCM type definition");
        return -EINVAL;
    }
    if (strcmp(str, "hw")) {
        snd_error(PCM, "Invalid type '%s' for slave PCM", str);
        return -EINVAL;
    }
    continue;
}
if (strcmp(id, "card") == 0) {
    err = snd_config_get_card(n);
    // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1877) - [public] snd_error

```
    snd_error(PCM, "Invalid value for PCM type definition");
    return -EINVAL;
}
if (strcmp(str, "hw")) {
    snd_error(PCM, "Invalid type '%s' for slave PCM", str);
    return -EINVAL;
}
continue;
}
if (strcmp(id, "card") == 0) {
    err = snd_config_get_card(n);
    if (err < 0)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1880) - [public] if

```
    if (strcmp(str, "hw")) {
        snd_error(PCM, "Invalid type '%s' for slave PCM", str);
        return -EINVAL;
    }
    continue;
}
if (strcmp(id, "card") == 0) {
    err = snd_config_get_card(n);
    if (err < 0)
        return err;
    card = err;
    continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1881) - [public] snd_error

```
    snd_error(PCM, "Invalid type '%s' for slave PCM", str);
    return -EINVAL;
}
continue;
}
if (strcmp(id, "card") == 0) {
    err = snd_config_get_card(n);
    if (err < 0)
        return err;
    card = err;
    continue;
}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1886) - [public] if

```
    if (strcmp(id, "card") == 0) {
        err = snd_config_get_card(n);
        if (err < 0)
            return err;
        card = err;
        continue;
    }
    if (strcmp(id, "device") == 0) {
        err = snd_config_get_integer(n, &device);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            return err;
        }
        // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1893) - [public] if

```
    if (strcmp(id, "device") == 0) {
        err = snd_config_get_integer(n, &device);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            return err;
        }
        continue;
    }
    if (strcmp(id, "subdevice") == 0) {
        err = snd_config_get_integer(n, &subdevice);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1895) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return err;
    }
    continue;
}
if (strcmp(id, "subdevice") == 0) {
    err = snd_config_get_integer(n, &subdevice);
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return err;
    }
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1896) - [public] snd_error

```
    snd_error(PCM, "Invalid type for %s", id);
    return err;
}
continue;
}
if (strcmp(id, "subdevice") == 0) {
    err = snd_config_get_integer(n, &subdevice);
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return err;
    }
    continue;
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1901) - [public] if

```
    if (strcmp(id, "subdevice") == 0) {
        err = snd_config_get_integer(n, &subdevice);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            return err;
        }
        continue;
    }
    if (device < 0)
        device = 0;
    if (subdevice < 0)
        // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1903) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return err;
    }
    continue;
}
if (device < 0)
    device = 0;
if (subdevice < 0)
    subdevice = 0;
if (card < 0 || card >= (1 << (31 - 12 - 1)))
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1904) - [public] snd_error

```
    snd_error(PCM, "Invalid type for %s", id);
    return err;
}
continue;
}
if (device < 0)
    device = 0;
if (subdevice < 0)
    subdevice = 0;
if (card < 0 || card >= (1 << (31 - 12 - 1)))
    return -EOVERFLOW;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1929) - [public]

_snd_pcm_direct_get_slave_ipc_offset

```
return _snd_pcm_direct_get_slave_ipc_offset(root, sconf, direction, 0);
}

int snd_pcm_direct_parse_open_conf(snd_config_t *root, snd_config_t *conf,
    int stream, struct snd_pcm_direct_open_conf *rec)
{
    snd_config_iterator_t i, next;
    int ipc_key_add_uid = 0;
    snd_config_t *n;
    int err;

    rec->slave = NULL;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1971) - [public] if

```
if (strcmp(id, "ipc_key") == 0) {
    long key;
    err = snd_config_get_integer(n, &key);
    if (err < 0) {
        snd_error(PCM, "The field ipc_key must be an integer type");

        return err;
    }
    rec->ipc_key = key;
    continue;
}
if (strcmp(id, "ipc_perm") == 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1974) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "The field ipc_key must be an integer type");

        return err;
    }
    rec->ipc_key = key;
    continue;
}
if (strcmp(id, "ipc_perm") == 0) {
    long perm;
    err = snd_config_get_integer(n, &perm);
    if (err < 0) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1975) - [public] snd_error

```
    snd_error(PCM, "The field ipc_key must be an integer type");

    return err;
}
rec->ipc_key = key;
continue;
}
if (strcmp(id, "ipc_perm") == 0) {
    long perm;
    err = snd_config_get_integer(n, &perm);
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1982) - [public] if

```
    if (strcmp(id, "ipc_perm") == 0) {
        long perm;
        err = snd_config_get_integer(n, &perm);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            return err;
        }
        if ((perm & ~0777) != 0) {
            snd_error(PCM, "The field ipc_perm must be a valid file permission");
            return -EINVAL;
        }
        rec->ipc_perm = perm;
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1985) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return err;
    }
    if ((perm & ~0777) != 0) {
        snd_error(PCM, "The field ipc_perm must be a valid file permission");
        return -EINVAL;
    }
    rec->ipc_perm = perm;
    continue;
}
if (strcmp(id, "hw_ptr_alignment") == 0) {
    // ... (truncated, see source file for more)
}
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1986) - [public] snd_error
    snd_error(PCM, "Invalid type for %s", id);
    return err;
}
if ((perm & ~0777) != 0) {
    snd_error(PCM, "The field ipc_perm must be a valid file permission");
    return -EINVAL;
}
rec->ipc_perm = perm;
continue;
}
if (strcmp(id, "hw_ptr_alignment") == 0) {
    const char *str;
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1989) - [public] if
    if ((perm & ~0777) != 0) {
        snd_error(PCM, "The field ipc_perm must be a valid file permission");
        return -EINVAL;
    }
    rec->ipc_perm = perm;
    continue;
}
if (strcmp(id, "hw_ptr_alignment") == 0) {
    const char *str;
    err = snd_config_get_string(n, &str);
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1990) - [public] snd_error
    snd_error(PCM, "The field ipc_perm must be a valid file permission");
    return -EINVAL;
}
rec->ipc_perm = perm;
continue;
}
if (strcmp(id, "hw_ptr_alignment") == 0) {
    const char *str;
    err = snd_config_get_string(n, &str);
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return -EINVAL;
        // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1996) - [public] if
    if (strcmp(id, "hw_ptr_alignment") == 0) {
        const char *str;
        err = snd_config_get_string(n, &str);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            return -EINVAL;
        }
        if (strcmp(str, "no") == 0 || strcmp(str, "off") == 0)
            rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_NO;
        else if (strcmp(str, "roundup") == 0)
            rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_ROUNDUP;
        else if (strcmp(str, "rounddown") == 0)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1999) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return -EINVAL;
    }
    if (strcmp(str, "no") == 0 || strcmp(str, "off") == 0)
        rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_NO;
    else if (strcmp(str, "roundup") == 0)
        rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_ROUNDUP;
    else if (strcmp(str, "rounddown") == 0)
        rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_ROUNDDOWN;
    else if (strcmp(str, "auto") == 0)
        rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_AUTO;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2000) - [public] snd_error

```
    snd_error(PCM, "Invalid type for %s", id);
    return -EINVAL;
}
if (strcmp(str, "no") == 0 || strcmp(str, "off") == 0)
    rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_NO;
else if (strcmp(str, "roundup") == 0)
    rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_ROUNDUP;
else if (strcmp(str, "rounddown") == 0)
    rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_ROUNDDOWN;
else if (strcmp(str, "auto") == 0)
    rec->hw_ptr_alignment = SND_PCM_HW_PTR_ALIGNMENT_AUTO;
else {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2012) - [public] snd_error

```
    snd_error(PCM, "The field hw_ptr_alignment is invalid : %s", str);
    return -EINVAL;
}

continue;
}
if (strcmp(id, "tstamp_type") == 0) {
    const char *str;
    err = snd_config_get_string(n, &str);
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return -EINVAL;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2018) - [public] if

```
    if (strcmp(id, "tstamp_type") == 0) {
        const char *str;
        err = snd_config_get_string(n, &str);
        if (err < 0) {
            snd_error(PCM, "Invalid type for %s", id);
            return -EINVAL;
        }
        if (strcmp(str, "default") == 0)
            rec->tstamp_type = -1;
        else if (strcmp(str, "gettimeofday") == 0)
            rec->tstamp_type = SND_PCM_TSTAMP_TYPE_GETTIMEOFDAY;
        else if (strcmp(str, "monotonic") == 0)
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2021) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "Invalid type for %s", id);
        return -EINVAL;
    }
    if (strcmp(str, "default") == 0)
        rec->tstamp_type = -1;
    else if (strcmp(str, "gettimeofday") == 0)
        rec->tstamp_type = SND_PCM_TSTAMP_TYPE_GETTIMEOFDAY;
    else if (strcmp(str, "monotonic") == 0)
        rec->tstamp_type = SND_PCM_TSTAMP_TYPE_MONOTONIC;
    else if (strcmp(str, "monotonic_raw") == 0)
        rec->tstamp_type = SND_PCM_TSTAMP_TYPE_MONOTONIC_RAW;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2022) - [public] snd_error

```
    snd_error(PCM, "Invalid type for %s", id);
    return -EINVAL;
}
if (strcmp(str, "default") == 0)
    rec->tstamp_type = -1;
else if (strcmp(str, "gettimeofday") == 0)
    rec->tstamp_type = SND_PCM_TSTAMP_TYPE_GETTIMEOFDAY;
else if (strcmp(str, "monotonic") == 0)
    rec->tstamp_type = SND_PCM_TSTAMP_TYPE_MONOTONIC;
else if (strcmp(str, "monotonic_raw") == 0)
    rec->tstamp_type = SND_PCM_TSTAMP_TYPE_MONOTONIC_RAW;
else {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2034) - [public] snd_error

```
    snd_error(PCM, "The field tstamp_type is invalid : %s", str);
    return -EINVAL;
}
continue;
}
if (strcmp(id, "ipc_gid") == 0) {
    char *group;
    char *endp;
    err = snd_config_get_ascii(n, &group);
    if (err < 0) {
        snd_error(PCM, "The field ipc_gid must be a valid group");
        return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2039) - [public] if

```
    if (strcmp(id, "ipc_gid") == 0) {
        char *group;
        char *endp;
        err = snd_config_get_ascii(n, &group);
        if (err < 0) {
            snd_error(PCM, "The field ipc_gid must be a valid group");
            return err;
        }
        if (! *group) {
            rec->ipc_gid = -1;
            free(group);
            continue;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2043) - [public] if

```
    if (err < 0) {
        snd_error(PCM, "The field ipc_gid must be a valid group");
        return err;
    }
    if (! *group) {
        rec->ipc_gid = -1;
        free(group);
        continue;
    }
    if (isdigit(*group) == 0) {
        long clen = sysconf(_SC_GETGR_R_SIZE_MAX);
        size_t len = (clen == -1) ? 1024 : (size_t)clen;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2044) - [public] snd_error

```
        snd_error(PCM, "The field ipc_gid must be a valid group");
        return err;
    }
    if (! *group) {
        rec->ipc_gid = -1;
        free(group);
        continue;
    }
    if (isdigit(*group) == 0) {
        long clen = sysconf(_SC_GETGR_R_SIZE_MAX);
        size_t len = (clen == -1) ? 1024 : (size_t)clen;
        struct group grp, *pgrp;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2047) - [public] if

```
    if (! *group) {
        rec->ipc_gid = -1;
        free(group);
        continue;
    }
    if (isdigit(*group) == 0) {
        long clen = sysconf(_SC_GETGR_R_SIZE_MAX);
        size_t len = (clen == -1) ? 1024 : (size_t)clen;
        struct group grp, *pgrp;
        char *buffer = (char *)malloc(len);
        if (buffer == NULL)
            return -ENOMEM;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2049) - [public] free

```
        free(group);
        continue;
    }
    if (isdigit(*group) == 0) {
        long clen = sysconf(_SC_GETGR_R_SIZE_MAX);
        size_t len = (clen == -1) ? 1024 : (size_t)clen;
        struct group grp, *pgrp;
        char *buffer = (char *)malloc(len);
        if (buffer == NULL)
            return -ENOMEM;
        int st = getgrnam_r(group, &grp, buffer, len, &pgrp);
        if (st != 0 || !pgrp) {
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2052) - [public] if

```
    if (isdigit(*group) == 0) {
        long clen = sysconf(_SC_GETGR_R_SIZE_MAX);
        size_t len = (clen == -1) ? 1024 : (size_t)clen;
        struct group grp, *pgrp;
        char *buffer = (char *)malloc(len);
        if (buffer == NULL)
            return -ENOMEM;
        int st = getgrnam_r(group, &grp, buffer, len, &pgrp);
        if (st != 0 || !pgrp) {
            snd_error(PCM, "The field ipc_gid must be a valid group (create group %s)", group);
            free(buffer);
            return -EINVAL;
        }
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2060) - [public] if

```
        if (st != 0 || !pgrp) {
            snd_error(PCM, "The field ipc_gid must be a valid group (create group %s)", group);
            free(buffer);
            return -EINVAL;
        }
        rec->ipc_gid = pgrp->gr_gid;
        free(buffer);
    } else {
        rec->ipc_gid = strtol(group, &endp, 10);
    }
    free(group);
    continue;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2061) - [public] snd_error

```
        snd_error(PCM, "The field ipc_gid must be a valid group (create group %s)", group);
        free(buffer);
        return -EINVAL;
    }
    rec->ipc_gid = pgrp->gr_gid;
    free(buffer);
} else {
    rec->ipc_gid = strtol(group, &endp, 10);
}
free(group);
continue;
}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2062) - [public] free

```
        free(buffer);
        return -EINVAL;
    }
    rec->ipc_gid = pgrp->gr_gid;
    free(buffer);
} else {
    rec->ipc_gid = strtol(group, &endp, 10);
}
free(group);
continue;
}
if (strcmp(id, "ipc_key_add_uid") == 0) {
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2066) - [public] free

```
    free(buffer);
} else {
    rec->ipc_gid = strtol(group, &endp, 10);
}
free(group);
continue;
}
if (strcmp(id, "ipc_key_add_uid") == 0) {
    if ((err = snd_config_get_bool(n)) < 0) {
        snd_error(PCM, "The field ipc_key_add_uid must be a boolean type");
        return err;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2070) - [public] free

```
    free(group);
    continue;
}
if (strcmp(id, "ipc_key_add_uid") == 0) {
    if ((err = snd_config_get_bool(n)) < 0) {
        snd_error(PCM, "The field ipc_key_add_uid must be a boolean type");
        return err;
    }
    ipc_key_add_uid = err;
    continue;
}
if (strcmp(id, "slave") == 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2073) - [public] if

```
    if (strcmp(id, "ipc_key_add_uid") == 0) {
        if ((err = snd_config_get_bool(n)) < 0) {
            snd_error(PCM, "The field ipc_key_add_uid must be a boolean type");
            return err;
        }
        ipc_key_add_uid = err;
        continue;
    }
    if (strcmp(id, "slave") == 0) {
        rec->slave = n;
        continue;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2074) - [public] if

```
    if ((err = snd_config_get_bool(n)) < 0) {
        snd_error(PCM, "The field ipc_key_add_uid must be a boolean type");
        return err;
    }
    ipc_key_add_uid = err;
    continue;
}
if (strcmp(id, "slave") == 0) {
    rec->slave = n;
    continue;
}
if (strcmp(id, "bindings") == 0) {
    // ... (truncated, see source file for more)
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2075) - [public] snd_error
    snd_error(PCM, "The field ipc_key_add_uid must be a boolean type");
    return err;
}
ipc_key_add_uid = err;
continue;
}
if (strcmp(id, "slave") == 0) {
    rec->slave = n;
    continue;
}
if (strcmp(id, "bindings") == 0) {
    rec->bindings = n;
    // ... (truncated, see source file for more)
}
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2081) - [public] if
    if (strcmp(id, "slave") == 0) {
        rec->slave = n;
        continue;
    }
    if (strcmp(id, "bindings") == 0) {
        rec->bindings = n;
        continue;
    }
    if (strcmp(id, "slowptr") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            return err;
        // ... (truncated, see source file for more)
    }
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2085) - [public] if
    if (strcmp(id, "bindings") == 0) {
        rec->bindings = n;
        continue;
    }
    if (strcmp(id, "slowptr") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            return err;
        rec->slowptr = err;
        continue;
    }
    if (strcmp(id, "max_periods") == 0) {
        // ... (truncated, see source file for more)
    }
```

```
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2089) - [public] if
    if (strcmp(id, "slowptr") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            return err;
        rec->slowptr = err;
        continue;
    }
    if (strcmp(id, "max_periods") == 0) {
        long val;
        err = snd_config_get_integer(n, &val);
        if (err < 0)
            return err;
        // ... (truncated, see source file for more)
    }
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2096) - [public] if

```
    if (strcmp(id, "max_periods") == 0) {
        long val;
        err = snd_config_get_integer(n, &val);
        if (err < 0)
            return err;
        rec->max_periods = val;
        continue;
    }
    if (strcmp(id, "var_periodsize") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            return err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2104) - [public] if

```
    if (strcmp(id, "var_periodsize") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            return err;
        rec->var_periodsize = err;
        continue;
    }
    if (strcmp(id, "direct_memory_access") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            return err;
        rec->direct_memory_access = err;
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2111) - [public] if

```
    if (strcmp(id, "direct_memory_access") == 0) {
        err = snd_config_get_bool(n);
        if (err < 0)
            return err;
        rec->direct_memory_access = err;
        continue;
    }
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (!rec->slave) {
    snd_error(PCM, "slave is not defined");
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2118) - [public] snd_error

```
    snd_error(PCM, "Unknown field %s", id);
    return -EINVAL;
}
if (!rec->slave) {
    snd_error(PCM, "slave is not defined");
    return -EINVAL;
}
if (!rec->ipc_key) {
    snd_error(PCM, "Unique IPC key is not defined");
    return -EINVAL;
}
if (ipc_key_add_uid)
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2122) - [public] snd_error

```
    ■■snd_error(PCM, "slave is not defined");
    ■■return -EINVAL;
    ■}
    ■if (!rec->ipc_key) {
    ■■snd_error(PCM, "Unique IPC key is not defined");
    ■■return -EINVAL;
    ■}
    ■if (ipc_key_add_uid)
    ■■rec->ipc_key += getuid();
    ■err = snd_pcm_direct_get_slave_ipc_offset(root, conf, stream);
    ■if (err < 0)
    ■■return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2126) - [public] snd_error

```
    ■■snd_error(PCM, "Unique IPC key is not defined");
    ■■return -EINVAL;
    ■}
    ■if (ipc_key_add_uid)
    ■■rec->ipc_key += getuid();
    ■err = snd_pcm_direct_get_slave_ipc_offset(root, conf, stream);
    ■if (err < 0)
    ■■return err;
    ■rec->ipc_key += err;

    ■return 0;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2171) - [public] free

```
    ■■free(dmix);
    ■■return ret;
    ■}

    ■dmix->ipc_key = opts->ipc_key;
    ■dmix->ipc_perm = opts->ipc_perm;
    ■dmix->ipc_gid = opts->ipc_gid;
    ■dmix->tstamp_type = opts->tstamp_type;
    ■dmix->semid = -1;
    ■dmix->shmid = -1;
    ■dmix->shmptr = (void *) -1;
    ■dmix->type = type;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2190) - [public] if

```
    ■■if (ret < 0) {
    ■■■snd_error(PCM, "unable to create IPC semaphore");
    ■■■goto _err_nosem_free;
    ■■}
    ■■ret = snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
    ■■if (ret < 0) {
    ■■■snd_pcm_direct_semaphore_discard(dmix);
    ■■■if (--fail_sem_loop <= 0)
    ■■■■goto _err_nosem_free;
    ■■■continue;
    ■■}
    ■■break;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2191) - [public] snd_error

```
    ■■■snd_error(PCM, "unable to create IPC semaphore");
    ■■■goto _err_nosem_free;
    ■■}
    ■■ret = snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
    ■■if (ret < 0) {
    ■■■snd_pcm_direct_semaphore_discard(dmix);
    ■■■if (--fail_sem_loop <= 0)
    ■■■■goto _err_nosem_free;
    ■■■continue;
    ■■}
    ■■break;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2195) - [public] if

```
    ■■if (ret < 0) {
    ■■■snd_pcm_direct_semaphore_discard(dmix);
    ■■■if (--fail_sem_loop <= 0)
    ■■■■goto _err_nosem_free;
    ■■■continue;
    ■■}
    ■■break;
    ■}

    ■ret = snd_pcm_direct_shm_create_or_connect(dmix);
    ■if (ret < 0) {
    ■■snd_error(PCM, "unable to create IPC shm instance");
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2196) - [public]
snd_pcm_direct_semaphore_discard

```
    ■■■snd_pcm_direct_semaphore_discard(dmix);
    ■■■if (--fail_sem_loop <= 0)
    ■■■■goto _err_nosem_free;
    ■■■continue;
    ■■}
    ■■break;
    ■}

    ■ret = snd_pcm_direct_shm_create_or_connect(dmix);
    ■if (ret < 0) {
    ■■snd_error(PCM, "unable to create IPC shm instance");
    ■■snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2206) - [public] snd_error

```
    ■■snd_error(PCM, "unable to create IPC shm instance");
    ■■snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
    ■■goto _err_nosem_free;
    ■} else {
    ■■*_dmix = dmix;
    ■}

    ■return ret;
    _err_nosem_free:
    ■snd_pcm_free(*pcmp);
    ■*pcmp = NULL;
    _err_nosem:
    ■// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2207) - [public] snd_pcm_direct_semaphore_up

```
    ■■snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
    ■■goto _err_nosem_free;
    ■} else {
    ■■*_dmix = dmix;
    ■}

    ■return ret;
_err_nosem_free:
    ■snd_pcm_free(*pcmp);
    ■*pcmp = NULL;
_err_nosem:
    ■free(dmix->bindings);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 63) - [public] shm_sum_discard

```
/*
 *
 */

static int shm_sum_discard(snd_pcm_direct_t *dmix);

/*
 * sum ring buffer shared memory area
 */
static int shm_sum_create_or_connect(snd_pcm_direct_t *dmix)
{
    ■struct shmid_ds buf;
    ■int tmpid, err;
    ■size_t size;

    ■size = dmix->shmptr->s.channels *
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 76) - [public] sizeof

```
    ■ sizeof(signed int);
retryshm:
    ■dmix->u.dmix.shmid_sum = shmget(dmixmap->ipc_key + 1, size,
    ■■■■■IPC_CREAT | dmixmap->ipc_perm);
    ■err = -errno;
    ■if (dmixmap->u.dmix.shmid_sum < 0) {
    ■■if (errno == EINVAL)
    ■■if ((tmpid = shmget(dmixmap->ipc_key + 1, 0, dmixmap->ipc_perm)) != -1)
    ■■if (!shmctl(tmpid, IPC_STAT, &buf))
    ■■if (!buf.shm_nattch)
    ■■/* no users so destroy the segment */
    ■■if (!shmctl(tmpid, IPC_RMID, NULL))
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 93) - [public] shm_sum_discard

```
    ■■shm_sum_discard(dmixmap);
    ■■return err;
    ■}
    ■if (dmixmap->ipc_gid >= 0) {
    ■■buf.shm_perm.gid = dmixmap->ipc_gid;
    ■■shmctl(dmixmap->u.dmix.shmid_sum, IPC_SET, &buf);
    ■}
    ■dmixmap->u.dmix.sum_buffer = shmat(dmixmap->u.dmix.shmid_sum, 0, 0);
    ■if (dmixmap->u.dmix.sum_buffer == (void *) -1) {
    ■■err = -errno;
    ■■shm_sum_discard(dmixmap);
    ■■return err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 98) - [public] shmctl

```
    ■■shmctl(dmix->u.dmix.shmid_sum, IPC_SET, &buf);
    ■}
    ■dmix->u.dmix.sum_buffer = shmat(dmix->u.dmix.shmid_sum, 0, 0);
    ■if (dmix->u.dmix.sum_buffer == (void *) -1) {
    ■■err = -errno;
    ■■shm_sum_discard(dmix);
    ■■return err;
    ■}
    ■mlock(dmix->u.dmix.sum_buffer, size);
    ■return 0;
}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 103) - [public] shm_sum_discard

```
    ■■shm_sum_discard(dmix);
    ■■return err;
    ■}
    ■mlock(dmix->u.dmix.sum_buffer, size);
    ■return 0;
}

static int shm_sum_discard(snd_pcm_direct_t *dmix)
{
    ■struct shmids buf;
    ■int ret = 0;

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 204) - [public] sizeof

```
    ■■■    sizeof(signed int));
    ■■return;
    ■}
    ■for (chn = 0; chn < channels; chn++) {
    ■■dchn = dmix->bindings ? dmix->bindings[chn] : chn;
    ■■if (dchn >= dmix->shmptr->s.channels)
    ■■■continue;
    ■■src_step = src_areas[chn].step / 8;
    ■■dst_step = dst_areas[dchn].step / 8;
    ■■do_mix_areas(size,
    ■■■    ((unsigned char *)dst_areas[dchn].addr + dst_areas[dchn].first / 8) + dst_ofs * dst_step,
    ■■■    ((unsigned char *)src_areas[chn].addr + src_areas[chn].first / 8) + src_ofs * src_step,
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 272) - [public] sizeof

```
    ■■■    sizeof(signed int));
    ■■return;
    ■}
    ■for (chn = 0; chn < channels; chn++) {
    ■■dchn = dmix->bindings ? dmix->bindings[chn] : chn;
    ■■if (dchn >= dmix->shmptr->s.channels)
    ■■■continue;
    ■■src_step = src_areas[chn].step / 8;
    ■■dst_step = dst_areas[dchn].step / 8;
    ■■do_remix_areas(size,
    ■■■    ((unsigned char *)dst_areas[dchn].addr + dst_areas[dchn].first / 8) + dst_ofs * dst_step,
    ■■■    ((unsigned char *)src_areas[chn].addr + src_areas[chn].first / 8) + src_ofs * src_step,
    ■    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 299) - [public] snd_pcm_direct_semaphore_down

```
    snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
}

static void dmix_up_sem(snd_pcm_direct_t *dmix)
{
    if (dmix->u.dmix.use_sem)
        snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
}
#endif

/*
 * synchronize shm ring buffer with hardware
 * // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 305) - [public] snd_pcm_direct_semaphore_up

```
    snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
}
#endif

/*
 * synchronize shm ring buffer with hardware
 */
static void snd_pcm_dmix_sync_area(snd_pcm_t *pcm)
{
    snd_pcm_direct_t *dmix = pcm->private_data;
    snd_pcm_uframes_t slave_hw_ptr, slave_appl_ptr, slave_size;
    snd_pcm_uframes_t appl_ptr, size, transfer;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 374) - [public] mix_areas

```
    mix_areas(dmix, src_areas, dst_areas, appl_ptr, slave_appl_ptr, transfer);
    size -= transfer;
    if (! size)
        break;
    slave_appl_ptr += transfer;
    slave_appl_ptr %= dmix->slave_buffer_size;
    appl_ptr += transfer;
    appl_ptr %= pcm->buffer_size;
}
dmix_up_sem(dmix);
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 412) - [public] snd_timer_stop

```
    snd_timer_stop(dmix->timer);
    gettimeofday(&dmix->trigger_tstamp, pcm->tstamp_type);
    if (dmix->state == SND_PCM_STATE_RUNNING) {
        dmix->state = SND_PCM_STATE_XRUN;
        return -EPIPE;
    }
    dmix->state = SND_PCM_STATE_SETUP;
    /* clear queue to remove pending poll events */
    snd_pcm_direct_clear_timer_queue(dmix);
}
return 0;
}

// ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 413) - [public] gettimeofday

```
    gettimeofday(&dmixmap->trigger_tstamp, pcm->tstamp_type);
    if (dmixmap->state == SND_PCM_STATE_RUNNING) {
        dmixmap->state = SND_PCM_STATE_XRUN;
        return -EPIPE;
    }
    dmixmap->state = SND_PCM_STATE_SETUP;
    /* clear queue to remove pending poll events */
    snd_pcm_direct_clear_timer_queue(dmixmap);
}
return 0;
}
```

// ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 414) - [public] if

```
    if (dmixmap->state == SND_PCM_STATE_RUNNING) {
        dmixmap->state = SND_PCM_STATE_XRUN;
        return -EPIPE;
    }
    dmixmap->state = SND_PCM_STATE_SETUP;
    /* clear queue to remove pending poll events */
    snd_pcm_direct_clear_timer_queue(dmixmap);
}
return 0;
}
```

```
static int snd_pcm_dmixmap_sync_ptr(snd_pcm_t *pcm)
    // ... (truncated, see source file for more)
```

```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 420) - [public] snd_pcm_direct_clear_timer_queue
    old_slave_hw_ptr = dmix->slave_hw_ptr;
    dmix->slave_hw_ptr = slave_hw_ptr;
    diff = pcm_frame_diff(slave_hw_ptr, old_slave_hw_ptr, dmix->slave_boundary);
    if (diff == 0) /* fast path */
        return 0;
    if (dmix->state != SND_PCM_STATE_RUNNING &&
        dmix->state != SND_PCM_STATE_DRAINING)
        /* not really started yet - don't update hw_ptr */
        return 0;
    dmix->hw_ptr += diff;
    dmix->hw_ptr %= pcm->boundary;
    if (pcm->stop_threshold >= pcm->boundary) /* don't care */
        return 0;
    avail = snd_pcm_mmap_playback_avail(pcm);
    if (avail > dmix->avail_max)
        dmix->avail_max = avail;
    if (avail >= pcm->stop_threshold) {
        snd_timer_stop(dmix->timer);
        gettimeofday(&dmix->trigger_tstamp, pcm->tstamp_type);
        if (dmix->state == SND_PCM_STATE_RUNNING) {
            dmix->state = SND_PCM_STATE_XRUN;
            return -EPIPE;
        }
        dmix->state = SND_PCM_STATE_SETUP;
        /* clear queue to remove pending poll events */
        ■■snd_pcm_direct_clear_timer_queue(dmix);
        ■}
        ■return 0;
    }

    static int snd_pcm_dmix_sync_ptr(snd_pcm_t *pcm)
    {
        ■snd_pcm_direct_t *dmix = pcm->private_data;
        ■snd_pcm_uframes_t slave_hw_ptr;
        ■int err;

        ■if (dmix->slowptr)
            // ... (truncated, see source file for more)
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 432) - [public] snd_pcm_hwsync
        ■■snd_pcm_hwsync(dmix->spcm);
        ■slave_hw_ptr = *dmix->spcm->hw_ptr;
        ■err = snd_pcm_direct_check_xrun(dmix, pcm);
        ■if (err < 0)
            ■■return err;

        ■return snd_pcm_dmix_sync_ptr0(pcm, slave_hw_ptr);
    }

    /*
     * plugin implementation
     */
    // ... (truncated, see source file for more)

```

```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 438) - [public] snd_pcm_dmix_sync_ptr0
    ■return snd_pcm_dmix_sync_ptr0(pcm, slave_hw_ptr);
    }

    /*
     * plugin implementation
     */

    static snd_pcm_state_t snd_pcm_dmix_state(snd_pcm_t *pcm)
    {
        ■snd_pcm_direct_t *dmix = pcm->private_data;

        ■snd_pcm_direct_check_xrun(dmix, pcm);
        // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 465) - [public] snd_pcm_dmix_sync_ptr0
        ■■snd_pcm_dmix_sync_ptr0(pcm, status->hw_ptr);
        ■■status->delay = snd_pcm_mmap_playback_delay(pcm);
        ■■break;
        ■default:
        ■■break;
        ■}

        ■status->state = snd_pcm_dmix_state(pcm);
        ■status->hw_ptr = *pcm->hw_ptr; /* boundary may be different */
        ■status->appl_ptr = *pcm->appl_ptr; /* slave PCM doesn't set appl_ptr */
        ■status->trigger_tstamp = dmix->trigger_tstamp;
        ■status->avail = snd_pcm_mmap_playback_avail(pcm);
        // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 516) - [public] snd_pcm_dmix_sync_ptr
    switch(dmix->state) {
    case SNDRV_PCM_STATE_DRAINING:
    case SNDRV_PCM_STATE_RUNNING:
        /* sync slave PCM */

        ■■return snd_pcm_dmix_sync_ptr(pcm);
        ■case SNDRV_PCM_STATE_PREPARED:
        ■case SNDRV_PCM_STATE_SUSPENDED:
        ■case STATE_RUN_PENDING:
        ■■return 0;
        ■case SNDRV_PCM_STATE_XRUN:
        ■■return -EPIPE;
        ■case SNDRV_PCM_STATE_DISCONNECTED:
        ■■return -ENODEV;
        ■default:
        ■■return -EBADFD;
        ■}
        // ... (truncated, see source file for more)

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 568) - [public] snd_pcm_dmix_sync_area
        ■■snd_pcm_dmix_sync_area(pcm);
        ■}
        ■gettimestamp(&dmix->trigger_tstamp, pcm->tstamp_type);
        ■return 0;
    }

    static int snd_pcm_dmix_drop(snd_pcm_t *pcm)
    {
        ■snd_pcm_direct_t *dmix = pcm->private_data;
        ■if (dmix->state == SND_PCM_STATE_OPEN)
        ■■return -EBADFD;
        ■dmix->state = SND_PCM_STATE_SETUP;
        // ... (truncated, see source file for more)

```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 602) - [public] snd_pcm_dmix_start

```
    ■■■snd_pcm_dmix_start(pcm);
    ■■else {
    ■■■snd_pcm_dmix_drop(pcm);
    ■■return 0;
    ■■}
    ■}

    ■if (dmix->state == SND_PCM_STATE_XRUN) {
    ■■snd_pcm_dmix_drop(pcm);
    ■■return 0;
    ■}

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 604) - [public] snd_pcm_dmix_drop

```
    ■■■snd_pcm_dmix_drop(pcm);
    ■■return 0;
    ■■}
    ■}

    ■if (dmix->state == SND_PCM_STATE_XRUN) {
    ■■snd_pcm_dmix_drop(pcm);
    ■■return 0;
    ■}

    ■stop_threshold = pcm->stop_threshold;
    ■if (pcm->stop_threshold > pcm->buffer_size)
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 610) - [public] snd_pcm_dmix_drop

```
    ■■snd_pcm_dmix_drop(pcm);
    ■■return 0;
    ■}

    ■stop_threshold = pcm->stop_threshold;
    ■if (pcm->stop_threshold > pcm->buffer_size)
    ■■pcm->stop_threshold = pcm->buffer_size;
    ■dmix->state = SND_PCM_STATE_DRAINING;
    ■do {
    ■■err = snd_pcm_dmix_sync_ptr(pcm);
    ■■if (err < 0) {
    ■■■snd_pcm_dmix_drop(pcm);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 620) - [public] if

```
    ■■if (err < 0) {
    ■■■snd_pcm_dmix_drop(pcm);
    ■■goto done;
    ■■}
    ■■if (dmix->state == SND_PCM_STATE_DRAINING) {
    ■■■snd_pcm_dmix_sync_area(pcm);
    ■■■if ((pcm->mode & SND_PCM_NONBLOCK) == 0) {
    ■■■■snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_DRAIN);
    ■■■■snd_pcm_direct_clear_timer_queue(dmix); /* force poll to wait */
    ■■■■}

    ■■■switch (snd_pcm_state(dmix->spcm)) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 621) - [public] snd_pcm_dmix_drop

```
    snd_pcm_dmix_drop(pcm);
    goto done;
}

if (dmix->state == SND_PCM_STATE_DRAINING) {
    snd_pcm_dmix_sync_area(pcm);
    if ((pcm->mode & SND_PCM_NONBLOCK) == 0) {
        snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_DRAIN);
        snd_pcm_direct_clear_timer_queue(dmix); /* force poll to wait */
    }

    switch (snd_pcm_state(dmix->spcm)) {
    case SND_PCM_STATE_SUSPENDED:
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 624) - [public] if

```
    if (dmix->state == SND_PCM_STATE_DRAINING) {
        snd_pcm_dmix_sync_area(pcm);
        if ((pcm->mode & SND_PCM_NONBLOCK) == 0) {
            snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_DRAIN);
            snd_pcm_direct_clear_timer_queue(dmix); /* force poll to wait */
        }

        switch (snd_pcm_state(dmix->spcm)) {
        case SND_PCM_STATE_SUSPENDED:
            err = -ESTRPIPE;
            goto done;
        default:
            // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 625) - [public] snd_pcm_dmix_sync_area

```
    snd_pcm_dmix_sync_area(pcm);
    if ((pcm->mode & SND_PCM_NONBLOCK) == 0) {
        snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_DRAIN);
        snd_pcm_direct_clear_timer_queue(dmix); /* force poll to wait */
    }

    switch (snd_pcm_state(dmix->spcm)) {
    case SND_PCM_STATE_SUSPENDED:
        err = -ESTRPIPE;
        goto done;
    default:
        break;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 626) - [public] if

```
    if ((pcm->mode & SND_PCM_NONBLOCK) == 0) {
        snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_DRAIN);
        snd_pcm_direct_clear_timer_queue(dmix); /* force poll to wait */
    }

    switch (snd_pcm_state(dmix->spcm)) {
    case SND_PCM_STATE_SUSPENDED:
        err = -ESTRPIPE;
        goto done;
    default:
        break;
    }
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 627) - [public] snd_pcm_wait_nocheck

```
#####snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_DRAIN);
#####snd_pcm_direct_clear_timer_queue(dmix); /* force poll to wait */
#####}

#####switch (snd_pcm_state(dmix->spcm)) {
#####case SND_PCM_STATE_SUSPENDED:
#####err = -ESTRPIPE;
#####goto done;
#####default:
#####break;
#####}
#####}
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 628) - [public] snd_pcm_direct_clear_timer_queue

```
#####snd_pcm_direct_clear_timer_queue(dmix); /* force poll to wait */
#####}

#####switch (snd_pcm_state(dmix->spcm)) {
#####case SND_PCM_STATE_SUSPENDED:
#####err = -ESTRPIPE;
#####goto done;
#####default:
#####break;
#####}
#####}
#####if (pcm->mode & SND_PCM_NONBLOCK) {
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 631) - [public] switch

```
#####switch (snd_pcm_state(dmix->spcm)) {
#####case SND_PCM_STATE_SUSPENDED:
#####err = -ESTRPIPE;
#####goto done;
#####default:
#####break;
#####}
#####}
#####if (pcm->mode & SND_PCM_NONBLOCK) {
#####if (dmix->state == SND_PCM_STATE_DRAINING) {
#####err = -EAGAIN;
#####goto done;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 639) - [public] if

```
#####if (pcm->mode & SND_PCM_NONBLOCK) {
#####if (dmix->state == SND_PCM_STATE_DRAINING) {
#####err = -EAGAIN;
#####goto done;
#####}
#####}
#####while (dmix->state == SND_PCM_STATE_DRAINING);
done:
#####pcm->stop_threshold = stop_threshold;
#####return err;
}

// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 640) - [public] if

```
    if (dmix->state == SND_PCM_STATE_DRAINING) {
        err = -EAGAIN;
        goto done;
    }
} while (dmix->state == SND_PCM_STATE_DRAINING);
done:
pcm->stop_threshold = stop_threshold;
return err;
}
```

```
static int snd_pcm_dmix_drain(snd_pcm_t *pcm)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 668) - [public]

snd_pcm_mmap_playback_hw_rewindable

```
return snd_pcm_mmap_playback_hw_rewindable(pcm);
}
```

```
static snd_pcm_sframes_t snd_pcm_dmix_rewind(snd_pcm_t *pcm, snd_pcm_uframes_t frames)
{
    snd_pcm_direct_t *dmix = pcm->private_data;
    snd_pcm_uframes_t slave_appl_ptr, slave_size;
    snd_pcm_uframes_t appl_ptr, size, transfer, result, frames_to_remix;
    int err;
    const snd_pcm_channel_area_t *src_areas, *dst_areas;

    if (dmix->state == SND_PCM_STATE_RUNNING ||
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 733) - [public] remix_areas

```
remix_areas(dmix, src_areas, dst_areas, appl_ptr, slave_appl_ptr, transfer);
size -= transfer;
if (!size)
    break;
slave_appl_ptr += transfer;
slave_appl_ptr %= dmix->slave_buffer_size;
appl_ptr += transfer;
appl_ptr %= pcm->buffer_size;
}
dmix_up_sem(dmix);

snd_pcm_mmap_appl_backward(pcm, frames_to_remix);
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 755) - [public] snd_pcm_mmap_avail

```
return snd_pcm_mmap_avail(pcm);
}

static snd_pcm_sframes_t snd_pcm_dmix_forward(snd_pcm_t *pcm, snd_pcm_uframes_t frames)
{
    snd_pcm_sframes_t avail;

    avail = snd_pcm_dmix_forwardable(pcm);
    if (frames > (snd_pcm_uframes_t)avail)
        frames = avail;
    snd_pcm_mmap_appl_forward(pcm, frames);
    return frames;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 784) - [public] snd_timer_close

```
    ■■snd_timer_close(dmixmap->timer);
    ■■snd_pcm_direct_semaphore_down(dmixmap, DIRECT_IPC_SEM_CLIENT);
    ■■snd_pcm_close(dmixmap->spcm);
    ■if (dmixmap->server)
    ■■snd_pcm_direct_server_discard(dmixmap);
    ■if (dmixmap->client)
    ■■snd_pcm_direct_client_discard(dmixmap);
    ■shm_sum_discard(dmixmap);
    ■if (snd_pcm_direct_shm_discard(dmixmap)) {
    ■■if (snd_pcm_direct_semaphore_discard(dmixmap))
    ■■■■snd_pcm_direct_semaphore_final(dmixmap, DIRECT_IPC_SEM_CLIENT);
    ■} else
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 788) - [public] snd_pcm_direct_server_discard

```
    ■■snd_pcm_direct_server_discard(dmixmap);
    ■if (dmixmap->client)
    ■■snd_pcm_direct_client_discard(dmixmap);
    ■shm_sum_discard(dmixmap);
    ■if (snd_pcm_direct_shm_discard(dmixmap)) {
    ■■if (snd_pcm_direct_semaphore_discard(dmixmap))
    ■■■■snd_pcm_direct_semaphore_final(dmixmap, DIRECT_IPC_SEM_CLIENT);
    ■} else
    ■■snd_pcm_direct_semaphore_final(dmixmap, DIRECT_IPC_SEM_CLIENT);
    ■free(dmixmap->bindings);
    ■pcm->private_data = NULL;
    ■free(dmixmap);
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 790) - [public] snd_pcm_direct_client_discard

```
    ■■snd_pcm_direct_client_discard(dmixmap);
    ■shm_sum_discard(dmixmap);
    ■if (snd_pcm_direct_shm_discard(dmixmap)) {
    ■■if (snd_pcm_direct_semaphore_discard(dmixmap))
    ■■■■snd_pcm_direct_semaphore_final(dmixmap, DIRECT_IPC_SEM_CLIENT);
    ■} else
    ■■snd_pcm_direct_semaphore_final(dmixmap, DIRECT_IPC_SEM_CLIENT);
    ■free(dmixmap->bindings);
    ■pcm->private_data = NULL;
    ■free(dmixmap);
    ■return 0;
}
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 794) - [public] snd_pcm_direct_semaphore_final

```
    ■■■■snd_pcm_direct_semaphore_final(dmixmap, DIRECT_IPC_SEM_CLIENT);
    ■} else
    ■■snd_pcm_direct_semaphore_final(dmixmap, DIRECT_IPC_SEM_CLIENT);
    ■free(dmixmap->bindings);
    ■pcm->private_data = NULL;
    ■free(dmixmap);
    ■return 0;
}
```

```
static snd_pcm_sframes_t snd_pcm_dmixmap_mmap_commit(snd_pcm_t *pcm,
■■■■■■■ snd_pcm_uframes_t offset ATTRIBUTE_UNUSED,
■■■■■■■ snd_pcm_uframes_t size)
    // ... (truncated, see source file for more)
```


dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 796) - [public] snd_pcm_direct_semaphore_final

```
    ■■snd_pcm_direct_semaphore_final(dmix, DIRECT_IPC_SEM_CLIENT);
    ■free(dmix->bindings);
    ■pcm->private_data = NULL;
    ■free(dmix);
    ■return 0;
}

static snd_pcm_sframes_t snd_pcm_dmix_mmap_commit(snd_pcm_t *pcm,
    ■■■■■■ snd_pcm_uframes_t offset ATTRIBUTE_UNUSED,
    ■■■■■■ snd_pcm_uframes_t size)
{
    ■snd_pcm_direct_t *dmix = pcm->private_data;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 828) - [public] snd_pcm_dmix_sync_area

```
err = snd_pcm_direct_check_xrun(dmix, pcm);
if (err < 0)
return err;
if (!size)
return 0;
snd_pcm_mmap_appl_forward(pcm, size);
if (dmix->state == STATE_RUN_PENDING) {
if ((err = snd_pcm_dmix_start_timer(pcm, dmix)) < 0)
return err;
} else if (dmix->state == SND_PCM_STATE_RUNNING ||
dmix->state == SND_PCM_STATE_DRAINING) {
if ((err = snd_pcm_dmix_sync_ptr(pcm)) < 0)
return err;
}
if (dmix->state == SND_PCM_STATE_RUNNING ||
dmix->state == SND_PCM_STATE_DRAINING) {
/* ok, we commit the changes after the validation of area */
/* it's intended, although the result might be crappy */

    ■■snd_pcm_dmix_sync_area(pcm);
    ■■/* clear timer queue to avoid a bogus return from poll */
    ■■if (snd_pcm_mmap_playback_avail(pcm) < pcm->avail_min)
    ■■snd_pcm_direct_clear_timer_queue(dmix);
    ■}
    ■return size;
}

static snd_pcm_sframes_t snd_pcm_dmix_avail_update(snd_pcm_t *pcm)
{
    ■snd_pcm_direct_t *dmix = pcm->private_data;
    ■int err;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 831) - [public] snd_pcm_direct_clear_timer_queue

```
    ■■snd_pcm_direct_clear_timer_queue(dmix);
    ■}
    ■return size;
}

static snd_pcm_sframes_t snd_pcm_dmix_avail_update(snd_pcm_t *pcm)
{
    ■snd_pcm_direct_t *dmix = pcm->private_data;
    ■int err;

    ■if (dmix->state == SND_PCM_STATE_RUNNING ||
    ■    dmix->state == SND_PCM_STATE_DRAINING) {
        // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 849) - [public] snd_pcm_mmap_playback_avail

```
    return snd_pcm_mmap_playback_avail(pcm);
}

static int snd_pcm_dmix_htimestamp(snd_pcm_t *pcm,
    snd_pcm_uframes_t *avail,
    snd_htimestamp_t *tstamp)
{
    snd_pcm_direct_t *dmix = pcm->private_data;
    snd_pcm_uframes_t avail1;
    int ok = 0;

    while (1) {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 863) - [public] snd_pcm_dmix_sync_ptr

```
    snd_pcm_dmix_sync_ptr(pcm);
    avail1 = snd_pcm_mmap_playback_avail(pcm);
    if (ok && *avail == avail1)
        break;
    *avail = avail1;
    *tstamp = snd_pcm_hw_fast_tstamp(dmix->spcm);
    ok = 1;
}
return 0;
}
```

```
static int snd_pcm_dmix_poll_revents(snd_pcm_t *pcm, struct pollfd *pfd, unsigned int nfds, unsigned short revents)
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 878) - [public] snd_pcm_dmix_sync_area

```
    snd_pcm_dmix_sync_area(pcm);
    return snd_pcm_direct_poll_revents(pcm, pfd, nfds, revents);
}
```

```
static void snd_pcm_dmix_dump(snd_pcm_t *pcm, snd_output_t *out)
{
    snd_pcm_direct_t *dmix = pcm->private_data;

    snd_output_printf(out, "Direct Stream Mixing PCM\n");
    if (pcm->setup) {
        snd_output_printf(out, "Its setup is:\n");
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 879) - [public] snd_pcm_direct_poll_revents

```
    return snd_pcm_direct_poll_revents(pcm, pfd, nfds, revents);
}
```

```
static void snd_pcm_dmix_dump(snd_pcm_t *pcm, snd_output_t *out)
{
    snd_pcm_direct_t *dmix = pcm->private_data;

    snd_output_printf(out, "Direct Stream Mixing PCM\n");
    if (pcm->setup) {
        snd_output_printf(out, "Its setup is:\n");
        snd_pcm_dump_setup(pcm, out);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 889) - [public] snd_output_printf

```
■■snd_output_printf(out, "Its setup is:\n");
■■snd_pcm_dump_setup(pcm, out);
■}
■if (dmix->spcm)
■■snd_pcm_dump(dmix->spcm, out);
}

static const snd_pcm_ops_t snd_pcm_dmix_ops = {
■.close = snd_pcm_dmix_close,
■.info = snd_pcm_direct_info,
■.hw_refine = snd_pcm_direct_hw_refine,
■.hw_params = snd_pcm_direct_hw_params,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 890) - [public] snd_pcm_dump_setup

```
■■snd_pcm_dump_setup(pcm, out);
■}
■if (dmix->spcm)
■■snd_pcm_dump(dmix->spcm, out);
}

static const snd_pcm_ops_t snd_pcm_dmix_ops = {
■.close = snd_pcm_dmix_close,
■.info = snd_pcm_direct_info,
■.hw_refine = snd_pcm_direct_hw_refine,
■.hw_params = snd_pcm_direct_hw_params,
■.hw_free = snd_pcm_direct_hw_free,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 893) - [public] snd_pcm_dump

```
■■snd_pcm_dump(dmix->spcm, out);
}

static const snd_pcm_ops_t snd_pcm_dmix_ops = {
■.close = snd_pcm_dmix_close,
■.info = snd_pcm_direct_info,
■.hw_refine = snd_pcm_direct_hw_refine,
■.hw_params = snd_pcm_direct_hw_params,
■.hw_free = snd_pcm_direct_hw_free,
■.sw_params = snd_pcm_direct_sw_params,
■.channel_info = snd_pcm_direct_channel_info,
■.dump = snd_pcm_dmix_dump,
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 973) - [public] snd_error

```
■■snd_error(PCM, "The dmix plugin supports only playback stream");
■■return -EINVAL;
■}

■ret = _snd_pcm_direct_new(&pcm, &dmix, SND_PCM_TYPE_DMIX, name, opts, params, stream, mode);
■if (ret < 0)
■■return ret;
■first_instance = ret;

■pcm->ops = &snd_pcm_dmix_ops;
■pcm->fast_ops = &snd_pcm_dmix_fast_ops;
■pcm->private_data = dmix;
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 999) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "unable to open slave");
        goto _err;
    }

    if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
        snd_error(PCM, "dmix plugin can be only connected to hw plugin");
        ret = -EINVAL;
        goto _err;
    }

    ret = snd_pcm_direct_initialize_slave(dmix, spcm, params);
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1000) - [public] snd_error

```
    snd_error(PCM, "unable to open slave");
    goto _err;
}

if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
    snd_error(PCM, "dmix plugin can be only connected to hw plugin");
    ret = -EINVAL;
    goto _err;
}

ret = snd_pcm_direct_initialize_slave(dmix, spcm, params);
if (ret < 0) {
    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1004) - [public] if

```
    if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
        snd_error(PCM, "dmix plugin can be only connected to hw plugin");
        ret = -EINVAL;
        goto _err;
    }

    ret = snd_pcm_direct_initialize_slave(dmix, spcm, params);
    if (ret < 0) {
        snd_error(PCM, "unable to initialize slave");
        goto _err;
    }

    // ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1005) - [public] snd_error

```
    snd_error(PCM, "dmix plugin can be only connected to hw plugin");
    ret = -EINVAL;
    goto _err;
}

ret = snd_pcm_direct_initialize_slave(dmix, spcm, params);
if (ret < 0) {
    snd_error(PCM, "unable to initialize slave");
    goto _err;
}

dmix->spcm = spcm;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1011) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "unable to initialize slave");
        goto _err;
    }

    dmix->spcm = spcm;

    if (dmix->shmptr->use_server) {
        dmix->server_free = dmix_server_free;

        ret = snd_pcm_direct_server_create(dmix);
        if (ret < 0) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1012) - [public] snd_error

```
    snd_error(PCM, "unable to initialize slave");
    goto _err;
}

dmix->spcm = spcm;

if (dmix->shmptr->use_server) {
    dmix->server_free = dmix_server_free;

    ret = snd_pcm_direct_server_create(dmix);
    if (ret < 0) {
        snd_error(PCM, "unable to create server");
        // ... (truncated, see source file for more)
    }
}
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1018) - [public] if

```
    if (dmix->shmptr->use_server) {
        dmix->server_free = dmix_server_free;

        ret = snd_pcm_direct_server_create(dmix);
        if (ret < 0) {
            snd_error(PCM, "unable to create server");
            goto _err;
        }
    }

    dmix->shmptr->type = spcm->type;
} else {
    // ... (truncated, see source file for more)
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1022) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "unable to create server");
        goto _err;
    }
}

dmix->shmptr->type = spcm->type;
} else {
    if (dmix->shmptr->use_server) {
        /* up semaphore to avoid deadlock */
        snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
        ret = snd_pcm_direct_client_connect(dmix);
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 1023) - [public] snd_error

```
    snd_error(PCM, "unable to create server");
    goto _err;
}

dmixmap->shmaptr->type = spcm->type;
} else {
    if (dmixmap->shmaptr->use_server) {
        /* up semaphore to avoid deadlock */
        snd_pcm_direct_semaphore_up(dmixmap, DIRECT_IPC_SEM_CLIENT);
        ret = snd_pcm_direct_client_connect(dmixmap);
        if (ret < 0) {
            // ... (truncated, see source file for more)
        }
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 1030) - [public] if

```
    if (dmixmap->shmaptr->use_server) {
        /* up semaphore to avoid deadlock */
        snd_pcm_direct_semaphore_up(dmixmap, DIRECT_IPC_SEM_CLIENT);
        ret = snd_pcm_direct_client_connect(dmixmap);
        if (ret < 0) {
            snd_error(PCM, "unable to connect client");
            goto _err_nosem;
        }

        snd_pcm_direct_semaphore_down(dmixmap, DIRECT_IPC_SEM_CLIENT);
        ret = snd_pcm_direct_open_secondary_client(&spcm, dmixmap, "dmixmap_client");
        if (ret < 0)
            // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 1032) - [public] snd_pcm_direct_semaphore_up

```
dmixmap->shmaptr->type = spcm->type;
} else {
    if (dmixmap->shmaptr->use_server) {
        /* up semaphore to avoid deadlock */

        snd_pcm_direct_semaphore_up(dmixmap, DIRECT_IPC_SEM_CLIENT);
        ret = snd_pcm_direct_client_connect(dmixmap);
        if (ret < 0) {
            snd_error(PCM, "unable to connect client");
            goto _err_nosem;
        }

        snd_pcm_direct_semaphore_down(dmixmap, DIRECT_IPC_SEM_CLIENT);
        ret = snd_pcm_direct_open_secondary_client(&spcm, dmixmap, "dmixmap_client");
        if (ret < 0)
            goto _err;
    } else {
        // ... (truncated, see source file for more)
    }
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmixmap.c (line 1034) - [public] if

```
    if (ret < 0) {
        snd_error(PCM, "unable to connect client");
        goto _err_nosem;
    }

    snd_pcm_direct_semaphore_down(dmixmap, DIRECT_IPC_SEM_CLIENT);
    ret = snd_pcm_direct_open_secondary_client(&spcm, dmixmap, "dmixmap_client");
    if (ret < 0)
        goto _err;
} else {

    ret = snd_pcm_open_slave(&spcm, root, sconf, stream,
        // ... (truncated, see source file for more)
    );
}
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1035) - [public] snd_error

```
#####snd_error(PCM, "unable to connect client");
#####goto _err_nosem;
####}

####snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
####ret = snd_pcm_direct_open_secondary_client(&spcm, dmix, "dmix_client");
####if (ret < 0)
#####goto _err;
####} else {

####ret = snd_pcm_open_slave(&spcm, root, sconf, stream,
#####mode | SND_PCM_NONBLOCK |
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1039) - [public] snd_pcm_direct_semaphore_down

```
####snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
####ret = snd_pcm_direct_open_secondary_client(&spcm, dmix, "dmix_client");
####if (ret < 0)
#####goto _err;
####} else {

####ret = snd_pcm_open_slave(&spcm, root, sconf, stream,
#####mode | SND_PCM_NONBLOCK |
#####SND_PCM_APPEND,
#####NULL);
####if (ret < 0) {
#####/* all other streams have been closed;
// ... (truncated, see source file for more)
```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1049) - [public] if

```
####if (ret < 0) {
#####/* all other streams have been closed;
##### * retry as the first instance
##### */
#####if (ret == -EBADF) {
#####first_instance = 1;
#####goto retry;
#####}
#####snd_error(PCM, "unable to open slave");
#####goto _err;
#####}
####if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
// ... (truncated, see source file for more)
```

(List truncated to 4000 entries to keep the document size manageable.)